Toán cho AI

Lộ trình 35 bài

Ghi chú học tập bằng tiếng Việt

Tháng 11, 2026

Mục lục

	Toán cho AI: lộ trình 35 bài	3
00	Toán nền tảng và hướng dẫn đọc	7
01	The Matrix Calculus You Need For Deep Learning (2018)	31
02	Automatic Differentiation in Machine Learning: a Survey (2015)	38
03	A Tutorial on Principal Component Analysis (2014)	45
04	Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions (2009)	54
05	A Tutorial on Spectral Clustering (2007)	63
06	New Insights and Perspectives on the Natural Gradient Method (2014)	72
07	An Elementary Introduction to Information Geometry (2018)	80
08	Variational Inference: A Review for Statisticians (2016)	90
09	An Introduction to Variational Autoencoders (2019)	98
10	A Conceptual Introduction to Hamiltonian Monte Carlo (2017)	106
11	Stochastic Gradient Descent as Approximate Bayesian Inference (2017)	114
12	Computational Optimal Transport (2018)	122
13	Wasserstein GAN (2017)	133
14	Deep Variational Information Bottleneck (2016)	142
15	Online normalizer calculation for softmax (2018)	152
16	FP8 Formats for Deep Learning (2022)	158
17	Spectral Normalization for Generative Adversarial Networks (2018)	166
18	The Shattered Gradients Problem: If resnets are the answer, then what is the question? (2017)	175
19	Measuring the Intrinsic Dimension of Objective Landscapes (2018)	184
20	Benefits of Depth in Neural Networks (2016)	190
21	The Expressive Power of Neural Networks: A View from the Width (2017)	197
22	Deep Neural Networks as Gaussian Processes (2017)	203
23	Neural Tangent Kernel: Convergence and Generalization in Neural Networks (2018)	209
24	On Lazy Training in Differentiable Programming (2018)	216
25	The Principles of Deep Learning Theory (2021)	223
26	Exponential Expressivity in Deep Neural Networks through Transient Chaos (2016)	231
27	Deep Information Propagation (2016)	237
28	Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges (2021)	243
29	Understanding Diffusion Models: A Unified Perspective (2022)	253
30	Opening the Black Box of Deep Neural Networks via Information (2017)	260
31	Deep Learning: A Statistical Viewpoint (2021)	267
32	Benign Overfitting in Linear Regression (2019)	274
33	Gradient Descent Finds Global Minima of Deep Neural Networks (2018)	280
34	A Mean Field View of the Landscape of Two-Layer Neural Networks (2018)	287
35	A Tutorial on Thompson Sampling (2017)	294

Toán cho AI: lộ trình 35 bài

Thư mục này chứa ghi chú học tập bằng tiếng Việt về **toán nền tảng cho AI**: đại số tuyến tính và matrix calculus, autodiff, suy luận biến phân và MCMC, optimal transport, lý thuyết thông tin, số học dấu phẩy động, rồi lý thuyết học sâu (độ sâu/độ rộng, kernel, NTK, lan truyền tín hiệu, overparameterization). Lộ trình bắt đầu bằng phần ôn tập nền tảng, rồi đi từ công cụ tính toán đến các chủ đề lý thuyết chuyên sâu hơn.

Các bài là ghi chú diễn giải dựa trên tài liệu gốc; nguồn được dẫn trong từng bài.

> Đây là các ghi chú diễn giải, không phải bản dịch. Mục gợi ý áp dụng là bình luận và cần được phân biệt với kết luận của bài gốc. Các phép tính đánh dấu "tự tính" là ví dụ tính thêm.

Giai đoạn 0: Chuẩn bị

#	File	Nội dung
00	`00_preface_toan_va_huong_dan.md`	Toán và kiến thức nền không có trong paper, bảng thuật ngữ, cách đọc phản biện, lịch đọc, liên hệ giữa các chủ đề, 5 câu tự kiểm tra. **Đọc trước tiên.**

Giai đoạn 1: Công cụ tính toán và đại số

#	File	Bài	Ý chính
01	`01_2018_matrix_calculus.md`	The Matrix Calculus You Need For Deep Learning (2018)	Đạo hàm theo vector, Jacobian và quy tắc chuỗi; áp dụng vào backpropagation.
02	`02_2015_automatic_differentiation_survey.md`	Automatic Differentiation in Machine Learning: a Survey (2015)	Vi phân tự động, chế độ forward và reverse, cùng chi phí tính toán.
03	`03_2014_pca_tutorial.md`	A Tutorial on Principal Component Analysis (2014)	PCA qua ma trận hiệp phương sai và SVD.
04	`04_2009_randomized_matrix_decomposition.md`	Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions (2009)	Xấp xỉ phân tích ma trận hạng thấp bằng lấy mẫu ngẫu nhiên.
05	`05_2007_spectral_clustering_tutorial.md`	A Tutorial on Spectral Clustering (2007)	Phân cụm phổ qua đồ thị tương tự và ma trận Laplacian.

Giai đoạn 2: Hình học thông tin và suy luận biến phân

#	File	Bài	Ý chính
06	`06_2014_natural_gradient_insights.md`	New Insights and Perspectives on the Natural Gradient Method (2014)	Natural gradient, ma trận Fisher và Generalized Gauss–Newton.
07	`07_2018_information_geometry_intro.md`	An Elementary Introduction to Information Geometry (2018)	Hình học thông tin, metric Fisher, kết nối đối ngẫu và natural gradient.
08	`08_2016_variational_inference_review.md`	Variational Inference: A Review for Statisticians (2016)	Suy luận biến phân, ELBO, mean-field, CAVI và SVI.
09	`09_2019_vae_introduction.md`	An Introduction to Variational Autoencoders (2019)	VAE, ELBO, reparameterization, amortized inference và normalizing flow.

Giai đoạn 3: MCMC và SGD như suy luận Bayes

#	File	Bài	Ý chính
10	`10_2017_hamiltonian_monte_carlo.md`	A Conceptual Introduction to Hamiltonian Monte Carlo (2017)	Trực giác hình học và động lực Hamiltonian trong MCMC.
11	`11_2017_sgd_approximate_bayesian_inference.md`	Stochastic Gradient Descent as Approximate Bayesian Inference (2017)	SGD với learning rate hằng số như một xấp xỉ suy luận Bayes.

Giai đoạn 4: Optimal transport và thông tin

#	File	Bài	Ý chính
12	`12_2018_computational_optimal_transport.md`	Computational Optimal Transport (2018)	Lý thuyết và thuật toán vận tải tối ưu, từ Monge–Kantorovich đến Sinkhorn.
13	`13_2017_wasserstein_gan.md`	Wasserstein GAN (2017)	Khoảng cách Wasserstein và ứng dụng trong huấn luyện GAN.
14	`14_2016_deep_variational_information_bottleneck.md`	Deep Variational Information Bottleneck (2016)	Cận biến phân cho information bottleneck và học biểu diễn.

Giai đoạn 5: Số học và ổn định số, điều kiện hoá

#	File	Bài	Ý chính
15	`15_2018_online_softmax.md`	Online normalizer calculation for softmax (2018)	Tính hệ số chuẩn hóa softmax trực tuyến với tính ổn định số.
16	`16_2022_fp8_formats.md`	FP8 Formats for Deep Learning (2022)	Hai định dạng FP8 E4M3 và E5M2, cùng kết quả huấn luyện và suy luận.

#	File	Bài	Ý chính
17	`17_2018_spectral_normalization_gan.md`	Spectral Normalization for Generative Adversarial Networks (2018)	Spectral normalization để khống chế chuẩn phổ của discriminator.
18	`18_2017_shattered_gradients.md`	The Shattered Gradients Problem: If resnets are the answer, then what is the question? (2017)	Tương quan gradient theo độ sâu và vai trò của kết nối residual.

Giai đoạn 6: Biểu diễn, độ sâu, độ rộng, kernel và lan truyền tín hiệu

#	File	Bài	Ý chính
19	`19_2018_intrinsic_dimension.md`	Measuring the Intrinsic Dimension of Objective Landscapes (2018)	Đo chiều nội tại của bài toán tối ưu bằng không gian con ngẫu nhiên.
20	`20_2016_benefits_of_depth.md`	Benefits of Depth in Neural Networks (2016)	Các chặn về khả năng biểu diễn của mạng sâu so với mạng nông.
21	`21_2017_expressive_power_width.md`	The Expressive Power of Neural Networks: A View from the Width (2017)	Quan hệ giữa độ rộng mạng ReLU và khả năng xấp xỉ.
22	`22_2017_deep_nn_gaussian_processes.md`	Deep Neural Networks as Gaussian Processes (2017)	Giới hạn Gaussian process của mạng nơ-ron sâu vô hạn rộng.
23	`23_2018_ntk.md`	Neural Tangent Kernel: Convergence and Generalization in Neural Networks (2018)	Neural tangent kernel và động lực huấn luyện trong giới hạn rộng.
24	`24_2018_lazy_training.md`	On Lazy Training in Differentiable Programming (2018)	Lazy training và vai trò của thang tham số hóa.
25	`25_2021_principles_dl_theory.md`	The Principles of Deep Learning Theory (2021)	Lý thuyết hiệu dụng cho mạng sâu, độ rộng hữu hạn và khởi tạo.
26	`26_2016_exponential_expressivity_chaos.md`	Exponential Expressivity in Deep Neural Networks through Transient Chaos (2016)	Lan truyền tín hiệu và khả năng biểu diễn của mạng ngẫu nhiên sâu.
27	`27_2016_deep_information_propagation.md`	Deep Information Propagation (2016)	Thang độ sâu, lan truyền tín hiệu và khả năng huấn luyện.

Giai đoạn 7: Hình học, diffusion và thông tin trong mạng

#	File	Bài	Ý chính
28	`28_2021_geometric_deep_learning.md`	Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges (2021)	Đối xứng và khung hình học chung cho các kiến trúc học sâu.

#	File	Bài	Ý chính
29	`29_2022_understanding_diffusion_models.md`	Understanding Diffusion Models: A Unified Perspective (2022)	Diffusion từ góc nhìn VAE phân cấp và score-based model.
30	`30_2017_opening_black_box_information.md`	Opening the Black Box of Deep Neural Networks via Information (2017)	Information plane và các pha huấn luyện của mạng nhỏ.

Giai đoạn 8: Tổng quát hoá và overparameterization

#	File	Bài	Ý chính
31	`31_2021_deep_learning_statistical_viewpoint.md`	Deep Learning: A Statistical Viewpoint (2021)	Góc nhìn thống kê về quá tham số hóa và tổng quát hóa.
32	`32_2019_benign_overfitting_linear_regression.md`	Benign Overfitting in Linear Regression (2019)	Điều kiện cho benign overfitting trong hồi quy tuyến tính.
33	`33_2018_gradient_descent_global_minima_dnn.md`	Gradient Descent Finds Global Minima of Deep Neural Networks (2018)	Các kết quả hội tụ gradient descent cho mạng sâu quá tham số hóa.
34	`34_2018_mean_field_two_layer_networks.md`	A Mean Field View of the Landscape of Two-Layer Neural Networks (2018)	Động lực mean-field của SGD trong mạng nơ-ron hai lớp.

Giai đoạn 9: Bandit và quyết định tuần tự

#	File	Bài	Ý chính
35	`35_2017_tutorial_thompson_sampling.md`	A Tutorial on Thompson Sampling (2017)	Thompson sampling cho bandit và quyết định tuần tự.

Cách dùng

Đọc preface (00) trước. Các giai đoạn 1 và 5 dùng ngay trong thực hành; giai đoạn 6 và 8 là lý thuyết nặng, lần đầu có thể chỉ đọc mục 1, 2, 11 của từng bài.

Mỗi bài có 11 mục cố định: bối cảnh, ý tưởng chính, phương pháp chi tiết, thí nghiệm, phân tích, giới hạn, liên hệ, gợi ý áp dụng, thí nghiệm gợi ý, tóm tắt công thức, kết luận. Mục 6 trình bày các giới hạn và lưu ý khi diễn giải kết quả.

Liên quan

- `math content/` (35 bài): toán nền; `ml basics content/` (40 bài): ML cơ bản; `optimization content/` (51 bài): tối ưu, optimizer, hyperparameter.
- `llm content/`, `rl content/`, `distillation content/`, `pruning content/`, `interp content/`: các lộ trình chuyên đề xây trên ba thư mục nền này.

Bài 00

Toán nền tảng và hướng dẫn đọc

Tác giả: ghi chú tổng hợp từ kiến thức nền của người biên soạn (không phải ghi chú một paper cụ thể) **Venue / arXiv:** không áp dụng **Nguồn:** các bài 01-35 trong thư mục này; phần kiến thức nền chung (định nghĩa, định lý kinh điển trong đại số tuyến tính, giải tích, xác suất, lý thuyết thông tin, số học dấu phẩy động) được trình bày lại ở đây, không trích từ một paper cụ thể **Chủ đề:** toán nền cho AI từ con số 0: đại số tuyến tính (vector, ma trận, hạng, trị riêng, SVD, chuẩn, điều kiện số), giải tích nhiều biến (gradient, Jacobian, Hessian, chain rule, autodiff, backprop tay), xác suất (Gaussian đa biến, Bayes, MLE/MAP, bất đẳng thức tập trung, hình học chiều cao), lý thuyết thông tin (entropy, cross-entropy, KL, MI), số học máy tính (dấu phẩy động, log-sum-exp, ổn định số), và lý thuyết học sâu ở mức khái niệm (NTK, overparameterization) đủ để đọc các bài 19-34 của thư mục **Vị trí trong lộ trình:** Đọc phần này trước các bài trong sách để ôn lại kiến thức và thuật ngữ nền tảng.

> Phần này trình bày kiến thức nền chung, không phải bản dịch hay ghi chú của một bài báo cụ thể. Các tài liệu tham khảo gồm *Convex Optimization* (Boyd và Vandenberghe), *Mathematics for Machine Learning* (Deisenroth, Faisal và Ong), *Deep Learning* (Goodfellow, Bengio và Courville), *Probabilistic Machine Learning* (Murphy) và *Introduction to Linear Algebra* (Strang). Các ví dụ số đánh dấu "tự tính" có thể kiểm tra lại. Khi bài báo dùng ký hiệu hoặc quy ước khác, cần theo định nghĩa của bài đó.

1. Vì sao AI cần toán và bản đồ phân loại

1.1 Một câu tóm tắt

Một mạng nơ-ron là hàm số f_θ với các tham số θ, trong đó có các ma trận trọng số. Huấn luyện thường tìm tham số làm nhỏ hàm mất mát $\mathcal{L}(\theta)$ trên dữ liệu. Bốn câu hỏi sau giúp định hướng những công cụ toán được dùng trong sách:

1. Dữ liệu và tham số được **biểu diễn** bằng gì, và biến đổi tuyến tính giữa chúng có cấu trúc gì? (đại số tuyến tính, mục 2)
2. Làm sao **tính được hướng giảm** của $\mathcal{L}(\theta)$ một cách tự động và đúng, kể cả khi θ có hàng tỷ chiều? (giải tích nhiều biến và autodiff, mục 3)
3. Dữ liệu và nhiễu được mô tả bằng luật nào, và ta tin vào một ước lượng đến mức nào? (xác suất, mục 4)
4. "Khoảng cách" giữa hai phân phối (dự đoán và thực tế) được đo bằng gì, và tại sao loss của language model lại là một con số như vậy? (lý thuyết thông tin, mục 5)

Hai câu hỏi khác liên quan đến thực hành và lý thuyết: mọi phép tính trên chạy bằng số có độ chính xác hữu hạn trên GPU, vậy khi nào nó tràn số hoặc mất chính xác (số học máy tính, mục 6); và vì sao một mạng quá nhiều tham số so với dữ liệu vẫn học được mà không chỉ "học vẹt" (lý thuyết học sâu, mục 7).

1.2 Bản đồ: nhánh toán nào dùng ở đâu

Nhánh toán	Công cụ chính	Dùng để làm gì trong AI	Bài trong thư mục này
Đại số tuyến tính	vector, ma trận, hạng, trị riêng, SVD	biểu diễn dữ liệu/tham số; nén, giảm chiều, phân tích trọng số	01, 03, 04, 05, 17
Hình học thông tin	ma trận Fisher, natural gradient	tối ưu hoá "đúng hình học" của không gian phân phối	06, 07
Giải tích nhiều biến	gradient, Jacobian, Hessian, autodiff	tính gradient để chạy gradient descent	01, 02

Nhánh toán	Công cụ chính	Dùng để làm gì trong AI	Bài trong thư mục này
Suy luận biến phân và MCMC	ELBO, HMC, SGD như lấy mẫu Bayes	huấn luyện mô hình sinh, định lượng độ bất định	08, 09, 10, 11
Vận tải tối ưu	khoảng cách Wasserstein, Sinkhorn	đo khác biệt giữa hai phân phối khi KL không dùng được	12, 13
Lý thuyết thông tin	entropy, KL, information bottleneck	định nghĩa loss, nén biểu diễn, phân tích huấn luyện	14, 30
Số học máy tính	dấu phẩy động, log-sum-exp	giữ phép tính ổn định, chọn định dạng số khi huấn luyện	15, 16
Chuẩn hoá và Lipschitz	chuẩn ma trận, trị kỳ dị lớn nhất	ổn định huấn luyện GAN, kiểm soát độ nhạy	17, 18
Lý thuyết học sâu	NTK, mean-field, overparameterization	giải thích vì sao mạng rất lớn vẫn tối ưu và tổng quát hoá được	19-27, 31-34
Học sâu hình học, mô hình sinh, quyết định tuần tự	đối xứng/đa tạp, diffusion, bandit	các nhánh phụ, chi tiết ở mục 11	28, 09/13/29, 35

Thư mục `ml basics content/` và `optimization content/` dùng lại gần như toàn bộ các dòng trên, nhưng áp vào tình huống cụ thể hơn (một optimizer, một kiến trúc). Các thư mục chuyên đề (`llm content/`, `rl content/`, `distillation content/`, `pruning content/`, `interp content/`) dùng toán ở đây làm "ngôn ngữ chung": ví dụ KL divergence (mục 5) xuất hiện lại ở phạt KL trong RLHF, SVD và chuẩn Frobenius (mục 2) xuất hiện lại khi phân tích trọng số bị cắt tỉa.

1.3 Vì sao thứ tự math rồi mới ml basics rồi optimization

Thư mục này dạy **công cụ** (vector làm gì, gradient là gì, entropy đo gì) mà không gắn với một kiến trúc cụ thể. `ml basics content/` dùng công cụ đó để giải thích **hiện tượng** (double descent, generalization, adversarial example). `optimization content/` dùng công cụ đó để giải thích **thuật toán cập nhật tham số** (Adam, learning rate schedule). Đọc đúng thứ tự tránh việc phải dừng giữa một bài optimizer để tự hỏi "Hessian là gì".

---

2. Đại số tuyến tính

Đây là nhánh dùng nhiều nhất trong thư mục (01, 03, 04, 05, 17 trực tiếp; gần như mọi bài khác dùng ký hiệu ma trận). Mục này giới thiệu các khái niệm qua một ví dụ xuyên suốt.

2.1 Vector, chuẩn, tích vô hướng

Một vector $x \in \mathbb{R}^d$ là một cột d số. **Tích vô hướng** (dot product) $x^\top y = \sum_i x_i y_i$ đo "độ giống hướng". **Chuẩn Euclid** (độ dài) là $\|x\|_2 = \sqrt{x^\top x}$.

Ví dụ (tự tính): $x = (3, 4)$, $\|x\|_2 = \sqrt{9 + 16} = \sqrt{25} = 5$.

Góc giữa hai vector: $\cos\theta = \dfrac{x^\top y}{\|x\|_2 \|y\|_2}$. Khi $\cos\theta = 0$, hai vector **vuông góc** (orthogonal), tức "không mang thông tin chung" theo nghĩa tuyến tính.

2.2 Ma trận là một phép biến đổi tuyến tính

$W \in \mathbb{R}^{m \times n}$ biến một vector n chiều thành một vector m chiều: $y = Wx$. Nhân hai ma trận $W_2 W_1$ nghĩa là **ghép hai phép biến đổi**: áp W_1 trước, W_2 sau. Đây chính xác là cách một mạng nhiều lớp tuyến tính hoạt động (bỏ qua phi tuyến tạm thời): lớp 2 áp lên đầu ra của lớp 1.

2.3 Hạng (rank) và không gian con

Hạng của W là số chiều thật sự của ảnh (image) của phép biến đổi, tức số cột (hoặc hàng) độc lập tuyến tính. W cỡ $m \times n$ có hạng tối đa $\min(m, n)$.

Ví dụ rank-deficient (tự tính). Xét

$$C = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 1 & 0 & 1 \end{pmatrix}.$$

Hàng 2 bằng 2 lần hàng 1, nên hai hàng này phụ thuộc tuyến tính. Tính bằng `numpy.linalg.matrix_rank`: hạng của C là 2 (không phải 3). SVD của C cho ba giá trị kỳ dị khoảng 8.435, 0.918, và 5.3×10^{-16} (tự tính, giá trị thứ ba chỉ là nhiễu số do dấu phẩy động, về lý thuyết đúng là 0). Đây là cách thực dụng để phát hiện hạng thấp trên máy: đếm số giá trị kỳ dị "đáng kể" thay vì kỳ vọng đúng số 0 tuyệt đối.

2.4 Trị riêng, vector riêng, và định lý phổ (spectral theorem)

Với ma trận vuông A: $Av = \lambda v$ nghĩa là A chỉ **co giãn** v theo hệ số λ, không đổi hướng. λ là trị riêng (eigenvalue), v là vector riêng (eigenvector).

Ví dụ xuyên suốt (tự tính). Đặt

$$\Sigma = \begin{pmatrix} 4 & 2 \\ 2 & 4 \end{pmatrix}.$$

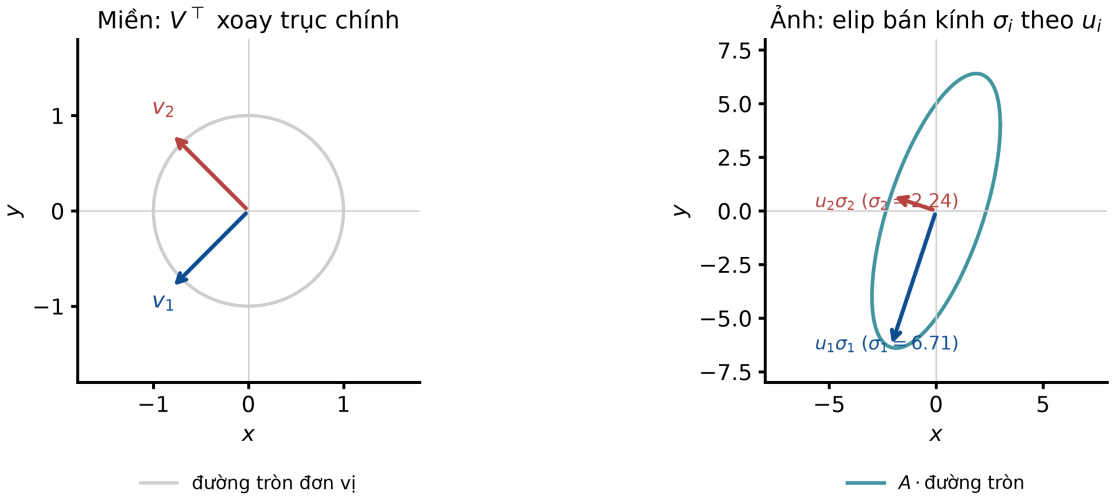
Σ đối xứng ($\Sigma = \Sigma^\top$), nên theo định lý phổ, nó có đủ $d = 2$ vector riêng **trực chuẩn** và trị riêng **thực**. Tính bằng `numpy.linalg.eigh`: trị riêng là 2 và 6; vector riêng tương ứng là $v_1 = (1, -1)/\sqrt{2}$ (ứng với $\lambda = 2$) và $v_2 = (1, 1)/\sqrt{2}$ (ứng với $\lambda = 6$). Kiểm tra tay: $\Sigma v_2 = \Sigma\,(1, 1)^\top/\sqrt{2} = (6, 6)^\top/\sqrt{2} = 6\,v_2$. Đúng.

Ý nghĩa hình học: Σ kéo giãn không gian theo hướng v_2 gấp 6 lần, theo hướng v_1 gấp 2 lần. Nếu Σ là ma trận hiệp phương sai (sẽ dùng lại ở mục 4), đây chính là "trục chính" của một đám mây điểm elip.

Lưu ý: ma trận **không đối xứng** nói chung có thể có trị riêng phức (ví dụ ma trận xoay 90 độ không có trục nào giữ nguyên hướng trong số thực). Phần lớn ma trận gặp trong AI (hiệp phương sai, Hessian, Fisher information, Gram matrix $X^\top X$) đối xứng hoặc đối xứng hoá được, nên trị riêng thực là trường hợp phổ biến nhất ta cần.

2.5 Phân tích giá trị kỳ dị (SVD)

Mọi ma trận $A \in \mathbb{R}^{m \times n}$ (vuông hay không, đối xứng hay không) đều phân tích được:



SVD hình học: đường tròn đơn vị qua V⊤ rồi co giãn σ_i thành elip theo u_i; ví dụ A=[[3,0],[4,5]] với σ≈6.71, 2.24. (minh hoạ, tự tính)

$$\boxed{A = U\Sigma_s V^\top}$$

với $U \in \mathbb{R}^{m\times m}$, $V \in \mathbb{R}^{n\times n}$ trực giao ($U^\top U = I$, $V^\top V = I$), và Σ_s là ma trận đường chéo không âm, các phần tử $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$ gọi là **giá trị kỳ dị** (singular value). Đọc theo chiều V, co giãn theo Σ_s, ghi theo chiều U.

Liên hệ eigen và SVD: nếu A đối xứng và bán xác định dương (positive semi-definite, mọi trị riêng ≥ 0) như Σ ở mục 2.4, SVD và phân tích trị riêng **trùng nhau**: $U = V$ = các vector riêng, giá trị kỳ dị = trị riêng. Kiểm tra (tự tính) trên Σ ở trên: `numpy.linalg.svd` cho giá trị kỳ dị $[6, 2]$, đúng bằng trị riêng (sắp xếp giảm dần) ở mục 2.4.

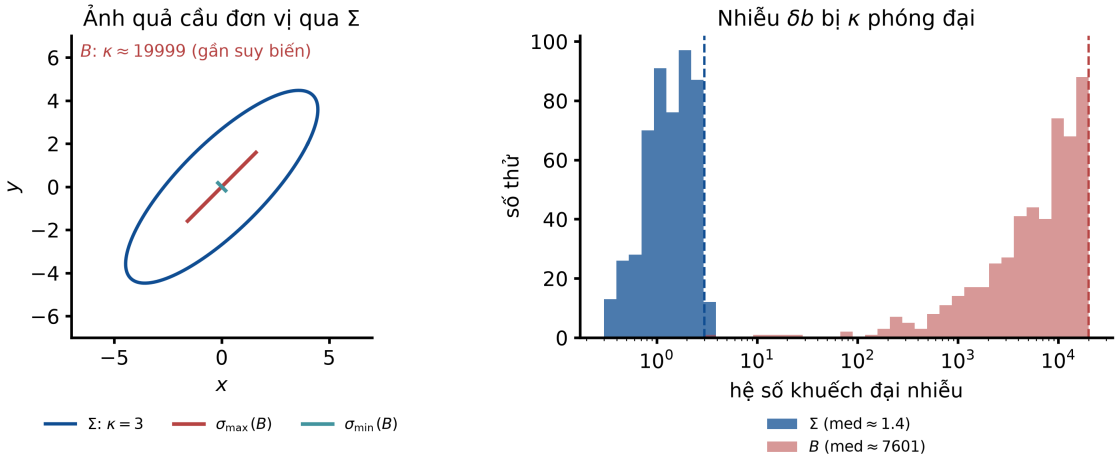
Ví dụ không đối xứng (tự tính). Với

$$A = \begin{pmatrix} 3 & 0 \\ 4 & 5 \end{pmatrix},$$

SVD cho giá trị kỳ dị khoảng 6.708 và 2.236 (không phải số nguyên đẹp như trị riêng của ma trận đối xứng, đây là điều bình thường). Hạng của A là 2 (cả hai giá trị kỳ dị đều khác 0, khớp với $\det A = 15 \neq 0$).

2.6 Chuẩn Frobenius, điều kiện số (condition number), xấp xỉ hạng thấp

Chuẩn Frobenius: $\|A\|_F = \sqrt{\sum_{ij} A_{ij}^2} = \sqrt{\sum_k \sigma_k^2}$ (tổng bình phương mọi phần tử, bằng tổng bình phương giá trị kỳ dị).



Điều kiện số κ=σ_max/σ_min: Σ ổn (κ=3) so với B gần suy biến (κ≈20000); nhiễu δb bị κ phóng đại. (minh hoạ, tự tính)

Điều kiện số: $\kappa(A) = \sigma_{\max}/\sigma_{\min}$. κ lớn nghĩa là A gần suy biến: nghịch đảo A^{-1} khuếch đại nhiễu rất mạnh theo hướng $\sigma_{\min}$.

Ví dụ (tự tính): $\kappa(\Sigma) = 6/2 = 3$ (ổn). Ngược lại, với

$$B = \begin{pmatrix} 1 & 0.9999 \\ 0.9999 & 1 \end{pmatrix},$$

trị riêng là khoảng 1.9999 và 0.0001, nên $\kappa(B) \approx 20000$ (tự tính): hai hàng gần như trùng nhau, nên nghịch đảo B^{-1} khuếch đại nhiễu lên khoảng 20000 lần. Đây chính là lý do thuật toán natural gradient (bài 06) hay least-squares với λ ridge (gặp lại ở mục 2.8) cần một hệ số chính quy hoá: cộng thêm λI để tránh nghịch đảo một ma trận gần suy biến.

Xấp xỉ hạng thấp tốt nhất (định lý Eckart-Young): với mọi $k < \operatorname{rank}(A)$, xấp xỉ hạng k gần A nhất theo chuẩn Frobenius là giữ lại k giá trị kỳ dị lớn nhất, bỏ phần còn lại:

$$\boxed{A_k = \sum_{i=1}^{k} \sigma_i u_i v_i^\top, \qquad \|A - A_k\|_F = \sqrt{\sum_{i>k} \sigma_i^2}}$$

Ví dụ (tự tính) trên Σ: xấp xỉ hạng 1 tốt nhất là $\Sigma_1 = 6\, v_2 v_2^\top = \begin{pmatrix} 3 & 3 \\ 3 & 3 \end{pmatrix}$, sai số Frobenius $\|\Sigma - \Sigma_1\|_F = \sigma_2 = 2$ (khớp với tính trực tiếp bằng numpy). Đây chính là nguyên lý của nén mô hình bằng phân tích hạng thấp (PCA ở bài 03, randomized SVD ở bài 04): giữ vài hướng có "năng lượng" (giá trị kỳ dị) lớn nhất, bỏ phần nhỏ.

2.7 Tích Kronecker

Tích Kronecker $A \otimes B$ ghép mỗi phần tử A_{ij} thành một khối $A_{ij}B$. Với A, B cỡ 2×2, kết quả cỡ 4×4.

Ví dụ (tự tính):

$$P = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad Q = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \qquad P \otimes Q = \begin{pmatrix} 0 & 1 & 0 & 2 \\ 1 & 0 & 2 & 0 \\ 0 & 3 & 0 & 4 \\ 3 & 0 & 4 & 0 \end{pmatrix}.$$

Tích Kronecker không xuất hiện trực tiếp trong 35 bài của thư mục này, nhưng xuất hiện trong `optimization content/` (Shampoo, bài opt 14; KFAC) để xây preconditioner xấp xỉ mà không cần lưu toàn bộ ma trận Hessian: nếu Hessian xấp xỉ bằng $A \otimes B$, nghịch đảo nó chỉ cần nghịch đảo riêng A và B (hai ma trận nhỏ) thay vì một ma trận khổng lồ, vì $(A \otimes B)^{-1} = A^{-1} \otimes B^{-1}$.

2.8 Bình phương tối thiểu và giả nghịch đảo

Bài toán: tìm W sao cho $WX \approx Y$ (ví dụ X là activation của một mạng, Y là mục tiêu cần khớp). Nghiệm bình phương tối thiểu có chính quy hoá ridge:

$$\boxed{W^\star = \arg\min_W \|WX - Y\|_F^2 + \lambda \|W\|_F^2 = YX^\top (XX^\top + \lambda I)^{-1}}$$

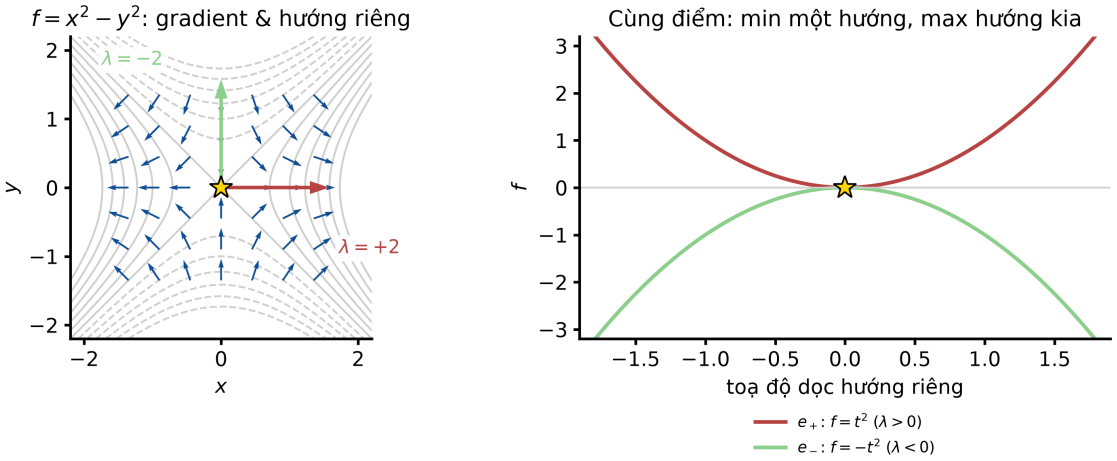
λI giữ vai trò giống mục 2.6: tránh nghịch đảo $XX^\top$ khi nó gần suy biến (ví dụ số mẫu ít hơn số chiều).

---

3. Giải tích nhiều biến

3.1 Gradient, Jacobian, Hessian

Cho hàm scalar (một số ra) $f : \mathbb{R}^d \to \mathbb{R}$. **Gradient** là vector đạo hàm riêng:



Điểm yên ngựa của f=x²−y²: gradient, hướng riêng Hessian với λ=+2 và λ=−2 (cực tiểu theo một hướng, cực đại theo hướng kia). (minh hoạ, tự tính)

$$\nabla f(x) = \left(\frac{\partial f}{\partial x_1}, \ldots, \frac{\partial f}{\partial x_d} \right)^\top.$$

∇f chỉ **hướng tăng nhanh nhất** của f; đi ngược hướng đó là gradient descent.

Cho hàm vector (nhiều số ra) $g : \mathbb{R}^n \to \mathbb{R}^m$, **Jacobian** $J \in \mathbb{R}^{m \times n}$ có $J_{ij} = \partial g_i / \partial x_j$: hàng i là gradient của đầu ra thứ i.

Hessian của f là ma trận đạo hàm bậc hai, $H_{ij} = \partial^2 f / \partial x_i \partial x_j$. Khi f khả vi liên tục hai lần, H **đối xứng** (đạo hàm chéo không phụ thuộc thứ tự lấy đạo hàm, định lý Schwarz).

Ví dụ (tự tính). $f(x, y) = x^2 y + \sin(y)$ tại điểm $(x, y) = (1, 0)$.

$$\nabla f = \big(2xy,\ x^2 + \cos y\big), \qquad \nabla f(1,0) = (0,\ 2).$$

$$H = \begin{pmatrix} 2y & 2x \\ 2x & -\sin y \end{pmatrix}, \qquad H(1,0) = \begin{pmatrix} 0 & 2 \\ 2 & 0 \end{pmatrix}.$$

Kiểm bằng sai phân hữu hạn (finite difference, bước $h = 10^{-4}$ đến 10^{-6}): gradient khớp $(0, 2.0)$, Hessian khớp $\begin{pmatrix} 0 & 2 \\ 2 & 0 \end{pmatrix}$ (sai số dưới 10^{-3}, đúng như lý thuyết sai phân hữu hạn dự đoán).

Trị riêng của $H(1,0)$ là $+2$ và -2 (tự tính, `numpy.linalg.eigvalsh`): một dương một âm, nghĩa là $(1, 0)$ là một **điểm yên ngựa** (saddle point): f tăng theo một hướng, giảm theo hướng khác. Phân biệt cực tiểu, cực đại, điểm yên bằng dấu trị riêng Hessian là công cụ trung tâm khi phân tích loss landscape (liên hệ `optimization content/` bài 26, 27).

3.2 Quy tắc chuỗi (chain rule): dạng scalar, dạng vector, dạng ma trận

Dạng scalar quen thuộc: $\dfrac{d}{dx} h(g(x)) = h'(g(x))\, g'(x)$.

Dạng vector (tổng quát hoá, đây là công cụ cốt lõi của bài 01): nếu $z = g(x)$ ($x \in \mathbb{R}^n$, $z \in \mathbb{R}^m$) và $y = h(z)$ ($y \in \mathbb{R}$), thì

$$\boxed{\nabla_x y = J_g^\top \nabla_z y}$$

với $J_g \in \mathbb{R}^{m \times n}$ là Jacobian của g. Đây chính là công thức mà backpropagation lặp lại ở mỗi lớp.

Ví dụ (tự tính). $g(x, y) = (x + y,\ xy)$, $h(u, v) = u^2 + v$, tại $(x, y) = (2, 3)$.

$$J_g = \begin{pmatrix} 1 & 1 \\ y & x \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 3 & 2 \end{pmatrix}, \qquad \nabla h = (2u,\ 1) = (2(x+y),\ 1) = (10,\ 1).$$

$$\nabla_{(x,y)}(h \circ g) = J_g^\top \nabla h = \begin{pmatrix} 1 & 3 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} 10 \\ 1 \end{pmatrix} = \begin{pmatrix} 13 \\ 12 \end{pmatrix}.$$

Kiểm bằng sai phân hữu hạn trực tiếp trên hàm hợp $f(x, y) = (x + y)^2 + xy$: ra đúng $(13, 12)$ (tự tính).

Dạng ma trận (dùng trong mọi lớp linear của mạng neural). Với $L = \|Wx - y\|_2^2$:

$$\boxed{\frac{\partial L}{\partial W} = 2(Wx - y)\, x^\top}$$

Ví dụ (tự tính): $W = \begin{pmatrix} 1 & 2 \\ 3 & -1 \end{pmatrix}$, $x = (1, 2)$, $y = (2, 1)$. $Wx = (5, 1)$, $Wx - y = (3, 0)$, $L = 9$.

$$\frac{\partial L}{\partial W} = 2(3,0)\,(1,2) = \begin{pmatrix} 6 & 12 \\ 0 & 0 \end{pmatrix}.$$

Sai phân hữu hạn trên từng phần tử của W cho đúng cùng ma trận này (tự tính, khớp đến 10^{-6}).

3.3 Automatic differentiation: forward mode và reverse mode

Autodiff (bài 02) tính đạo hàm **chính xác** (không phải xấp xỉ như sai phân hữu hạn) bằng cách phân rã hàm thành các phép toán cơ bản và áp chain rule lặp lại qua một "đồ thị tính toán" (computational graph).

Ví dụ. $f(x_1, x_2) = x_1 x_2 + \sin(x_1)$ tại $(x_1, x_2) = (2, 3)$. Đồ thị tính toán: $v_1 = x_1 x_2$, $v_2 = \sin(x_1)$, $f = v_1 + v_2$.

Forward mode: chọn một hướng "seed" cho đầu vào, lan truyền đạo hàm **cùng chiều** với giá trị, từ input ra output. Seed $(\dot{x}_1, \dot{x}_2) = (1, 0)$ (muốn $\partial f/\partial x_1$): $\dot{v}_1 = x_2\dot{x}_1 + x_1\dot{x}_2 = 3$, $\dot{v}_2 = \cos(x_1)\dot{x}_1 = \cos(2) \approx -0.416$, $\dot{f} = \dot{v}_1 + \dot{v}_2 \approx 2.584$. Seed $(0, 1)$ (muốn $\partial f/\partial x_2$): $\dot{v}_1 = x_1 = 2$, $\dot{v}_2 = 0$, $\dot{f} = 2$. **Cần một lượt chạy cho mỗi input.**

Reverse mode (chính là backpropagation): lan truyền đạo hàm **ngược chiều**, từ output về input, dùng "adjoint" $\bar{v} = \partial f/\partial v$. $\bar{f} = 1$ (gốc), $\bar{v}_1 = \bar{v}_2 = 1$ (vì $f = v_1 + v_2$), rồi $\bar{x}_1 = \bar{v}_1\, x_2 + \bar{v}_2 \cos(x_1) \approx 3 + (-0.416) = 2.584$, $\bar{x}_2 = \bar{v}_1\, x_1 = 2$. **Một lượt chạy duy nhất cho mọi input**, miễn là chỉ có một (hoặc ít) output.

Cả hai cho cùng kết quả $\partial f/\partial x_1 \approx 2.584$, $\partial f/\partial x_2 = 2$ (tự tính, khớp sai phân hữu hạn).

Vì sao mạng neural luôn dùng reverse mode: loss là **một số scalar**, còn số tham số có thể lên đến hàng tỷ. Forward mode cần một lượt chạy riêng cho **mỗi tham số** (không khả thi). Reverse mode cần **một** lượt backward duy nhất để lấy gradient theo **toàn bộ** tham số cùng lúc. Forward mode chỉ có lợi khi số input ít hơn hẳn số output (hiếm gặp trong deep learning, nhưng gặp trong một số phép tính Hessian-vector product).

3.4 Ví dụ tay: backpropagation qua mạng 2 lớp

Mạng: input $x \in \mathbb{R}^2$, lớp ẩn $z_1 = W_1 x + b_1 \in \mathbb{R}^2$, ReLU $a_1 = \max(0, z_1)$, lớp ra $\hat{y} = W_2 a_1 + b_2 \in \mathbb{R}$, loss $L = \frac{1}{2}(\hat{y} - t)^2$.

Số liệu (tự tính, đã kiểm bằng sai phân hữu hạn cho mọi tham số):

$$x = (1, 2), \quad W_1 = \begin{pmatrix} 0.5 & -0.3 \\ 0.2 & 0.8 \end{pmatrix}, \quad b_1 = (0.05, -0.1), \quad W_2 = (0.4, -0.6), \quad b_2 = 0.2, \quad t = 1.$$

Forward:

$$z_1 = W_1 x + b_1 = (0.5 \cdot 1 - 0.3 \cdot 2 + 0.05,\ 0.2 \cdot 1 + 0.8 \cdot 2 - 0.1) = (-0.05,\ 1.7).$$

$$a_1 = \max(0, z_1) = (0,\ 1.7) \qquad \text{(thành phần thứ nhất của } z_1 \text{ bị ReLU chặn vì âm).}$$

$$\hat{y} = W_2 a_1 + b_2 = 0.4 \cdot 0 + (-0.6) \cdot 1.7 + 0.2 = -0.82.$$

$$L = \tfrac{1}{2}(-0.82 - 1)^2 = \tfrac{1}{2}(1.82)^2 = 1.6562.$$

Backward (reverse-mode, từng bước):

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - t = -1.82.$$

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial \hat{y}}\, a_1^\top = -1.82\,(0,\ 1.7) = (0,\ -3.094), \qquad \frac{\partial L}{\partial b_2} = -1.82.$$

$$\frac{\partial L}{\partial a_1} = \frac{\partial L}{\partial \hat{y}}\, W_2 = -1.82\,(0.4, -0.6) = (-0.728,\ 1.092).$$

Qua ReLU: đạo hàm là 1 nơi $z_1 > 0$, 0 nơi $z_1 < 0$ (mask $= (0, 1)$ cho ví dụ này):

$$\frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial a_1} \odot \mathbb{1}[z_1 > 0] = (-0.728 \cdot 0,\ 1.092 \cdot 1) = (0,\ 1.092).$$

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial z_1}\, x^\top = \begin{pmatrix} 0 \\ 1.092 \end{pmatrix} (1,\ 2) = \begin{pmatrix} 0 & 0 \\ 1.092 & 2.184 \end{pmatrix}, \qquad \frac{\partial L}{\partial b_1} = (0,\ 1.092).$$

Mọi con số trên khớp chính xác với đạo hàm tính bằng sai phân hữu hạn trên từng phần tử của W_1, b_1, W_2, b_2 (tự tính, sai số dưới 10^{-6}).

Điểm đáng chú ý: ReLU chết (dead ReLU). Đơn vị ẩn thứ nhất có thành phần tương ứng của z_1 bằng $-0.05 < 0$, nên thành phần tương ứng của a_1 bằng 0, và toàn bộ dòng đầu của $\partial L/\partial W_1$ cùng thành phần đầu của $\partial L/\partial b_1$ đều **bằng 0**. Nếu đơn vị này giữ nguyên bị chặn qua nhiều bước cập nhật, dòng đầu của W_1 và thành phần đầu của b_1 sẽ không đổi, đơn vị đó không còn học được gì: đây chính là hiện tượng ”ReLU chết” sẽ nhắc lại ở mục 8. Hiện tượng này không phải lỗi tính toán, mà là hệ quả trực tiếp của đạo hàm ReLU bằng 0 phía âm.

---

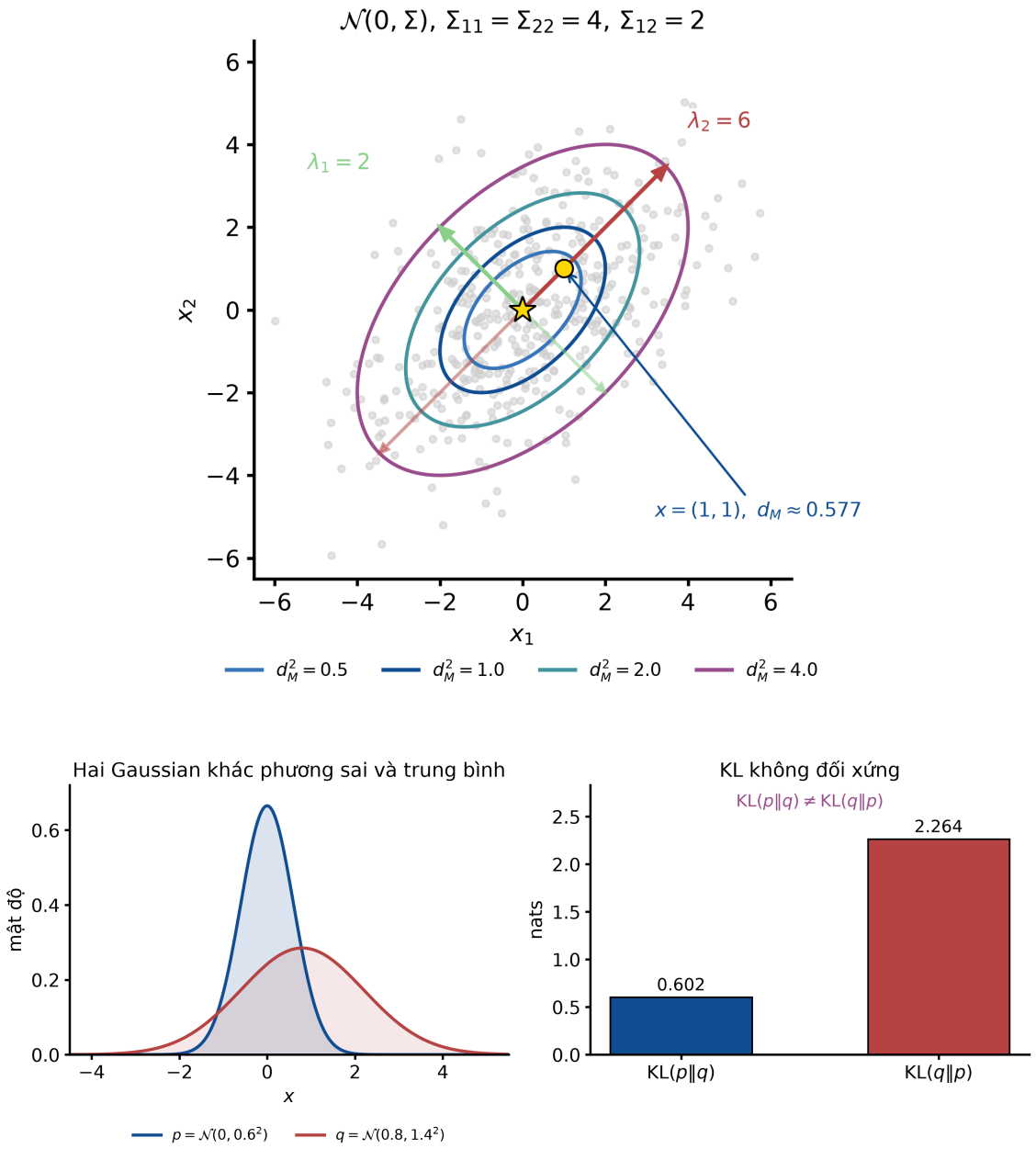
4. Xác suất

4.1 Biến ngẫu nhiên, kỳ vọng, phương sai

Một biến ngẫu nhiên X có phân phối xác suất mô tả khả năng nó nhận từng giá trị. **Kỳ vọng** $\mathbb{E}[X]$ là trung bình có trọng số xác suất; **phương sai** $\mathrm{Var}(X) = \mathbb{E}[(X - \mathbb{E}X)^2] = \mathbb{E}[X^2] - (\mathbb{E}X)^2$ đo độ phân tán. Với hai biến, **hiệp phương sai** $\mathrm{Cov}(X, Y) = \mathbb{E}[(X - \mathbb{E}X)(Y - \mathbb{E}Y)]$ đo mức hai biến thay đổi cùng nhau; ma trận hiệp phương sai của một vector ngẫu nhiên gom tất cả các hiệp phương sai cặp đôi lại, và luôn **đối xứng, bán xác định dương**, nên dùng được đúng bộ công cụ ở mục 2.4-2.6.

4.2 Gaussian đa biến

Vector ngẫu nhiên $X \in \mathbb{R}^d$ theo phân phối Gaussian đa biến $\mathcal{N}(\mu, \Sigma)$ có mật độ



Hai Gaussian khác trung bình/phương sai: KL(p‖q) ≠ KL(q‖p), minh hoạ KL không đối xứng. (minh hoạ, tự tính)

Gaussian đa biến N(0,Σ) với Σ_{11}=Σ_{22}=4, Σ_{12}=2: đường mức Mahalanobis, trục riêng λ=6 và λ=2, điểm x=(1,1). (minh hoạ, tự tính)

$$\boxed{p(x) = \frac{1}{(2\pi)^{d/2}\,|\Sigma|^{1/2}} \exp\Big(-\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)\Big)}$$

Đại lượng $(x-\mu)^\top \Sigma^{-1}(x-\mu)$ gọi là **khoảng cách Mahalanobis bình phương**: đo x cách μ bao nhiêu ”độ lệch chuẩn”, có tính đến hướng và độ co giãn của Σ (khác chuẩn Euclid thường, vốn coi mọi hướng như nhau).

Ví dụ (tự tính), dùng lại $\Sigma = \begin{pmatrix} 4 & 2 \\ 2 & 4 \end{pmatrix}$ **ở mục 2.4,** $\mu = (0, 0)$**, điểm** $x = (1, 1)$**.**

$$\Sigma^{-1} = \frac{1}{12}\begin{pmatrix} 4 & -2 \\ -2 & 4 \end{pmatrix} = \begin{pmatrix} 1/3 & -1/6 \\ -1/6 & 1/3 \end{pmatrix}.$$

$$\text{Mahalanobis}^2 = x^\top \Sigma^{-1} x = \tfrac{1}{3}, \qquad \text{Mahalanobis} = \sqrt{1/3} \approx 0.577.$$

$$|\Sigma| = 16 - 4 = 12, \qquad p(1,1) = \frac{1}{2\pi\sqrt{12}} \exp(-\tfrac{1}{6}) \approx 0.0389.$$

Vì $x = (1,1)$ nằm dọc hướng $v_2 = (1,1)/\sqrt{2}$ (trục có trị riêng **lớn** là 6, phương sai lớn), khoảng cách Mahalanobis của nó (0.577) nhỏ hơn khoảng cách Euclid ($\|x\|_2 = \sqrt{2} \approx 1.414$): theo hướng "dễ xảy ra" của Σ, cùng một khoảng cách Euclid trở nên "gần" hơn về xác suất.

4.3 Định lý Bayes

$$\boxed{P(A \mid B) = \frac{P(B \mid A)\, P(A)}{P(B)}}$$

Ví dụ kinh điển (tự tính). Bệnh hiếm, tỉ lệ mắc trong dân số $P(\text{bệnh}) = 1\%$. Test có độ nhạy (sensitivity) 99% ($P(\text{dương} \mid \text{bệnh}) = 0.99$) và tỉ lệ dương giả (false positive rate) 5% ($P(\text{dương} \mid \text{không bệnh}) = 0.05$). Hỏi: dương tính thì xác suất mắc bệnh thật là bao nhiêu?

$$P(\text{dương}) = 0.01 \cdot 0.99 + 0.99 \cdot 0.05 = 0.0099 + 0.0495 = 0.0594.$$

$$P(\text{bệnh} \mid \text{dương}) = \frac{0.0099}{0.0594} \approx 0.1667 \ (16.7\%).$$

Dù test có độ nhạy 99% (nghe rất chắc chắn), xác suất mắc bệnh thật khi dương tính chỉ khoảng 1/6, vì bệnh hiếm (prior nhỏ) và tỉ lệ dương giả không phải 0. Đây là ví dụ kinh điển về "base-rate fallacy": đánh giá một phân loại (reward model, bộ kiểm tra tự động) chỉ bằng độ nhạy, bỏ qua tỉ lệ cơ sở, dễ hiểu sai mức tin cậy thực tế của nó.

4.4 MLE và MAP

Maximum Likelihood Estimation (MLE): chọn tham số θ làm cực đại hoá xác suất quan sát được dữ liệu, $\hat{\theta}_{\text{MLE}} = \arg\max_\theta \prod_i p(x_i \mid \theta)$.

Ví dụ (tự tính). Tung một đồng xu lệch 10 lần, được 7 lần mặt "đúng" (head). Với $n = 10$, số lần đúng $k = 7$, log-likelihood là $\ell(p) = k \log p + (n-k)\log(1-p)$; đạo hàm bằng 0 cho $\hat{p}_{\text{MLE}} = k/n = 7/10 = 0.7$ (nghiệm quen thuộc: tần suất quan sát).

Maximum A Posteriori (MAP): thêm một **prior** $p(\theta)$ (niềm tin trước khi thấy dữ liệu), cực đại hoá hậu nghiệm $p(\theta \mid x) \propto p(x \mid \theta) p(\theta)$.

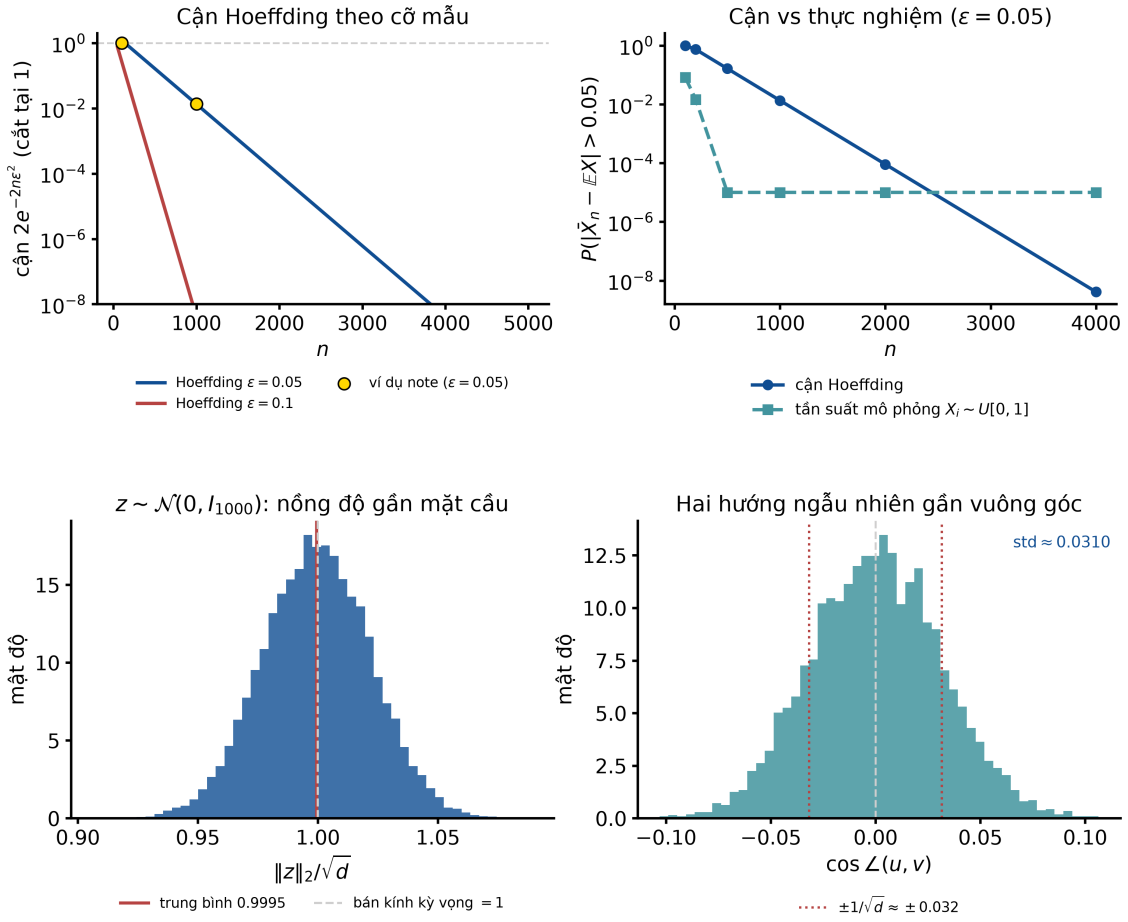
Ví dụ (tự tính). Prior $\text{Beta}(2,2)$ (niềm tin yếu rằng p gần 0.5) cho cùng dữ liệu 7/10. Hậu nghiệm của mô hình Bernoulli-Beta là $\text{Beta}(2+7,\ 2+3) = \text{Beta}(9,5)$. Mode (giá trị MAP) của $\text{Beta}(\alpha,\beta)$ là $(\alpha-1)/(\alpha+\beta-2)$:

$$\hat{p}_{\text{MAP}} = \frac{9-1}{9+5-2} = \frac{8}{12} \approx 0.667.$$

So với MLE 0.7, MAP bị "kéo" nhẹ về phía prior 0.5. Khi dữ liệu càng nhiều, ảnh hưởng của prior càng nhỏ và MAP tiến về MLE; đây là nguyên lý chung của suy luận biến phân và hậu nghiệm Bayes (bài 08, 09, 10).

4.5 Bất đẳng thức tập trung (concentration inequality)

Các bất đẳng thức này trả lời: trung bình của n mẫu lệch khỏi giá trị kỳ vọng thật bao xa, với xác suất bao nhiêu? **Bất đẳng thức Hoeffding** cho biến bị chặn trong $[0, 1]$:



Chiều cao d=1000: $\|z\|_2/\sqrt{d}$ tập trung quanh 1; cos góc hai hướng ngẫu nhiên ≈0±1/√d. (minh hoạ, tự tính)

Hoeffding: cận $2\exp(-2n\varepsilon^2)$; n=100,ε=0.05 vô nghĩa (>1), n=1000 hữu ích; so với tần suất mô phỏng. (minh hoạ, tự tính)

$$\boxed{P\left(|\bar{X}_n - \mathbb{E}X| > \epsilon\right) \leq 2\exp(-2n\epsilon^2)}$$

Ví dụ (tự tính). $\epsilon = 0.05$, $n = 100$: cận $= 2\exp(-2 \cdot 100 \cdot 0.0025) = 2\exp(-0.5) \approx 1.213$. Cận này **lớn hơn 1**, tức vô nghĩa (một xác suất không thể vượt 1): với $n = 100$ và sai số 0.05, Hoeffding không cho thông tin gì, cần n lớn hơn. Với $n = 1000$, cùng $\epsilon = 0.05$: cận $= 2\exp(-5) \approx 0.0135$, có ý nghĩa (xác suất lệch quá 5% dưới khoảng 1.35%). Với $\epsilon = 0.1$, $n = 1000$: cận $\approx 4.12 \times 10^{-9}$, rất chặt.

Bài học thực dụng: cận lý thuyết **chặt hay lỏng phụ thuộc mạnh vào** n **và** ϵ; luôn tính thử bằng số trước khi trích dẫn một bound như một bảo đảm thực tế (liên hệ mục 10, cách đọc paper lý thuyết phản biện).

4.6 Hình học không gian nhiều chiều

Trực giác hình học 2-3 chiều (hình cầu, góc vuông) **đánh lừa** khi d lớn (hàng trăm đến hàng nghìn chiều, đúng kích thước của một residual stream trong transformer).

Nồng độ của chuẩn (tự tính, mô phỏng). Với $z \sim \mathcal{N}(0, I_d)$, $d = 1000$: lấy 20000 mẫu, $\|z\|_2/\sqrt{d}$ có trung bình ≈ 0.9995 và độ lệch chuẩn chỉ ≈ 0.0224. Hầu hết khối lượng xác suất của một Gaussian chuẩn chiều

cao tập trung rất gần **mặt cầu** bán kính $\sqrt{d}$, không gần tâm (khác trực giác 2 chiều, nơi điểm "ngẫu nhiên" có thể ở bất kỳ đâu trong hình tròn).

Gần vuông góc thì có rất nhiều (tự tính, mô phỏng). Lấy một vector đơn vị cố định $u \in \mathbb{R}^{1000}$ và 5000 vector đơn vị ngẫu nhiên độc lập: cos góc với u có trung bình ≈ -0.0001 (coi như 0) và độ lệch chuẩn ≈ 0.0318, rất gần $1/\sqrt{d} = 1/\sqrt{1000} \approx 0.0316$ theo lý thuyết. Hai vector ngẫu nhiên trong không gian nhiều chiều **gần như luôn vuông góc**. Hệ quả (đã khai triển kỹ ở `interp content/00` mục 3.1): một không gian d chiều có thể chứa **nhiều hơn** d hướng "gần như độc lập" nếu chấp nhận một chút nhiễu khi chúng không hoàn toàn vuông góc nhau, đây là nền tảng hình học của superposition trong interpretability.

---

5. Lý thuyết thông tin

5.1 Entropy

Entropy $H(p) = -\sum_i p_i \log p_i$ đo độ "bất định" của một phân phối (đơn vị bit nếu dùng $\log_2$, nat nếu dùng ln). Entropy cực đại khi phân phối đều, bằng 0 khi phân phối tập trung hoàn toàn vào một giá trị.

Ví dụ (tự tính): đồng xu lệch $p = (0.3, 0.7)$: $H(p) = -0.3\log_2 0.3 - 0.7\log_2 0.7 \approx 0.881$ bit (so với đồng xu công bằng $p = (0.5, 0.5)$ có $H = 1$ bit, lớn nhất có thể cho 2 giá trị).

5.2 Cross-entropy và KL divergence

$$H(p, q) = -\sum_i p_i \log q_i, \qquad \boxed{\mathrm{KL}(p\|q) = \sum_i p_i \log \frac{p_i}{q_i} = H(p, q) - H(p) \ \geq 0}$$

$\mathrm{KL}(p\|q) = 0$ khi và chỉ khi $p = q$. KL **không đối xứng**: $\mathrm{KL}(p\|q) \neq \mathrm{KL}(q\|p)$ nói chung, nên không phải một "khoảng cách" thật theo nghĩa toán học (không thoả bất đẳng thức tam giác); đây là lý do bài 12, 13 (optimal transport, Wasserstein GAN) đề xuất một thước đo khác khi KL không phù hợp.

Ví dụ (tự tính): $p = (0.7, 0.2, 0.1)$, $q = (0.5, 0.3, 0.2)$:

$$H(p) \approx 1.157 \text{ bit}, \quad H(p, q) \approx 1.280 \text{ bit}, \quad \mathrm{KL}(p\|q) \approx 0.123 \text{ bit}.$$

5.3 Vì sao loss của language model là cross-entropy (và bằng NLL)

Khi huấn luyện, nhãn thật ở mỗi vị trí là **một token cụ thể**, tức phân phối p suy biến thành một "one-hot": $p_{\text{đúng}} = 1$, còn lại 0. Khi đó $H(p) = 0$ (không còn bất định gì để đo, nhãn đã biết chắc), và

$$H(p, q) = -\log q_{\text{đúng}} = \mathrm{KL}(p\|q).$$

Đây đúng là **negative log-likelihood (NLL)** của token đúng theo phân phối dự đoán q của model. Vậy "tối thiểu cross-entropy" và "tối thiểu KL tới phân phối thật" và "tối đa log-likelihood" là **cùng một bài toán** khi nhãn là one-hot. Ví dụ (tự tính): $q_{\text{đúng}} = 0.5$: NLL $= -\ln(0.5) \approx 0.693$ nat $= -\log_2(0.5) = 1$ bit. **Perplexity** (dùng để báo cáo chất lượng LM) chỉ là $\exp(\text{NLL trung bình theo nat})$, tức "số lựa chọn hiệu dụng trung bình mà model còn phân vân".

5.4 Mutual information (MI)

$$I(X; Y) = \mathrm{KL}\big(p(x, y) \,\|\, p(x)p(y)\big) = \sum_{x,y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)}$$

đo mức X và Y phụ thuộc nhau; $I = 0$ khi và chỉ khi độc lập.

Ví dụ (tự tính): bảng đồng thời $P(X,Y) = \begin{pmatrix} 0.4 & 0.1 \\ 0.1 & 0.4 \end{pmatrix}$ (cả hai biên $P(X) = P(Y) = (0.5, 0.5)$). $I(X;Y) \approx 0.278$ bit. Có thể kiểm chéo bằng công thức cho biến nhị phân đối xứng: $I = 1 - H_2(0.2)$ với H_2 là entropy nhị phân của tỉ lệ "lệch" $0.1/0.5 = 0.2$: $H_2(0.2) = -0.2\log_2 0.2 - 0.8\log_2 0.8 \approx 0.722$ bit, $1 - 0.722 = 0.278$ bit, khớp. Information bottleneck (bài 14, 30) dùng chính I này để đo một biểu diễn ẩn Z giữ được bao nhiêu thông tin về nhãn Y và bỏ đi bao nhiêu thông tin dư thừa từ input X.

---

6. Số học máy tính

6.1 Số dấu phẩy động: 3 phần, và đánh đổi exponent/mantissa

Một số dấu phẩy động chuẩn hoá được viết $(-1)^S \times 1.M \times 2^{E-\text{bias}}$: 1 bit dấu S, phần mũ E (quyết định **thang đo**, dynamic range), phần định trị M (quyết định **độ chính xác** trong một thang đo). Nhiều bit E hơn: biểu diễn được số rất lớn và rất nhỏ, nhưng mỗi "nấc" giữa hai số liền kề thưa hơn. Nhiều bit M hơn: chính xác hơn trong cùng một thang đo.

Định dạng	Dấu	Exponent	Mantissa	Max normal	Dùng khi nào
fp32	1	8	23	$\approx 3.4 \times 10^{38}$	mặc định, tích lũy số (optimizer state)
fp16 (IEEE half)	1	5	10	65504	suy luận/huấn luyện tiết kiệm bộ nhớ, dễ tràn số
bf16	1	8	7	$\approx 3.39 \times 10^{38}$	cùng dynamic range với fp32, ít chính xác hơn, khó tràn số
fp8 E4M3	1	4	3	448	trọng số, activation (bài 16)
fp8 E5M2	1	5	2	57344	gradient, cần dynamic range rộng (bài 16)

Hai dòng cuối lấy đúng định nghĩa và số liệu từ bài 16 trong thư mục này (paper định nghĩa chính xác E4M3: 1 dấu + 4 exponent + 3 mantissa, bias 7, max normal 448; E5M2: 1 dấu + 5 exponent + 2 mantissa, bias 15, max normal 57344). Khuyến nghị của paper 16: dùng E4M3 cho trọng số/activation (cần chính xác hơn, phân phối hẹp quanh 0), E5M2 cho gradient (cần dynamic range rộng hơn, nhiều giá trị rất nhỏ lẫn vài giá trị lớn).

6.2 Khoảng cách giữa các số biểu diễn được (ví dụ bf16, tự tính)

Với mantissa 7 bit (bf16), khoảng cách giữa hai số liền kề trong cùng một "octave" $[2^k, 2^{k+1})$ là 2^{k-7}: khoảng cách **nhân đôi** mỗi khi giá trị vượt một lũy thừa của 2. Mô phỏng làm tròn kiểu bf16 (tự tính, cắt mantissa về 7 bit, làm tròn đến số chẵn gần nhất):

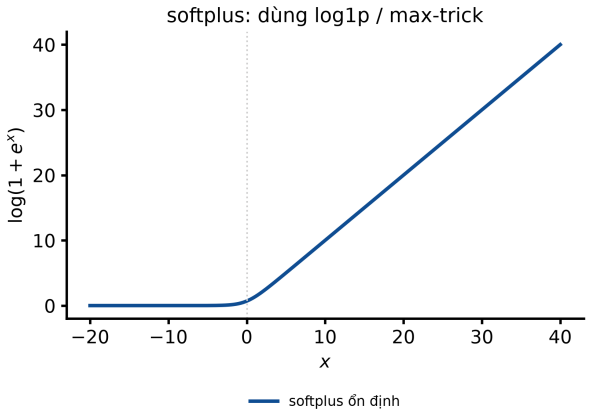
Giá trị gốc	Làm tròn về bf16	Octave
255	255 (đúng)	$[128, 256)$, bước 1
256	256 (đúng)	$[256, 512)$, bước 2
257	256	$[256, 512)$, bước 2
258	258 (đúng)	$[256, 512)$, bước 2
1000	1000 (đúng)	$[512, 1024)$, bước 4
1001	1000	$[512, 1024)$, bước 4

Quan sát: 257 không biểu diễn chính xác được trong bf16 (làm tròn về 256), vì bước nhảy ở octave $[256, 512)$ đã là 2; nhưng 255 (octave thấp hơn, bước nhảy 1) thì vẫn chính xác. Bài học: **sai số làm tròn của**

bf16/fp16/fp8 tăng theo độ lớn giá trị tuyệt đối, không phải hằng số, đây là lý do cần ”scaling factor” trước khi ép tensor về một định dạng hẹp (bài 16, mục 3.3 của chính bài đó).

6.3 log-sum-exp: vì sao softmax ”ngây thơ” tràn số

Softmax chuẩn: $p_i = e^{z_i} / \sum_j e^{z_j}$. Khi logit z lớn (ví dụ sau một vài lớp không chuẩn hoá tốt), e^{z_i} có thể tràn số.



$\mathrm{LSE}(c) = \log \sum_i e^{c_i}$, $c = (1000, 1001, 999)$

naive $\sum e^{c_i}$: **tràn (inf)**

max-trick: $m + \log \sum e^{c_i - m}$:

1001.407606 (ref 1001.407606)

cùng ý tưởng với online softmax (bài 15)

Log-sum-exp / softplus ổn định: naive exp(c) với c≈1000 tràn; max-trick cho đúng giá trị tham chiếu. (minh hoạ, tự tính)

Ví dụ (tự tính). Logits $z = (1000, 1001, 1002)$ tính trực tiếp bằng `numpy` ở độ chính xác float64: `exp(z)` cho ra `inf` ở cả ba phần tử (RuntimeWarning: overflow encountered in exp), tổng là `inf`, và `inf/inf = nan`. Softmax ”ngây thơ” cho ra `[nan, nan, nan]`, hoàn toàn vô dụng.

Thủ thuật log-sum-exp: trừ giá trị lớn nhất trước khi lấy exp, vì softmax không đổi khi cộng một hằng số vào mọi logit:

$$p_i = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}}, \qquad m = \max_j z_j, \qquad \log \sum_j e^{z_j} = m + \log \sum_j e^{z_j - m}$$

Với cùng $z = (1000, 1001, 1002)$, $m = 1002$: logit đã trừ là $(-2, -1, 0)$, hoàn toàn an toàn, softmax $\approx (0.0900,\ 0.2447,\ 0.6652)$ (tự tính, khớp ba chữ số thập phân với kỳ vọng lý thuyết $e^{-2} : e^{-1} : 1$ chuẩn hoá). $\log \sum_j e^{z_j} \approx 1002.408$ (tự tính): một số hợp lý, không tràn.

Đây chính là nội dung của bài 15 (online softmax): tính m và tổng theo kiểu ”chạy” (online), cập nhật dần khi quét qua vector, để không phải lưu toàn bộ vector logit hai lần; đây cũng là kỹ thuật lõi bên trong FlashAttention (`llm content/44, 45`) khi tính attention theo khối (tile) mà không vật chất hoá toàn bộ ma trận attention.

6.4 Catastrophic cancellation: công thức ”đúng về đại số” nhưng sai về số

Công thức $\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}X)^2$ đúng về đại số, nhưng khi $\mathbb{E}[X^2]$ và $(\mathbb{E}X)^2$ đều lớn và gần nhau, phép trừ hai số lớn gần bằng nhau ở độ chính xác hữu hạn gây **mất hết các chữ số có nghĩa**.

Ví dụ (tự tính). 10^5 mẫu Gaussian với trung bình 10^4, phương sai thật 1. Tính ở độ chính xác float64 làm chuẩn: phương sai ≈ 0.993. Ép dữ liệu và phép tính về float32 rồi dùng công thức một lượt $\mathbb{E}[X^2] - (\mathbb{E}X)^2$: trong một lần tính, kết quả ra đúng 0, sai khác 100% so với giá trị thật (kết quả cụ thể có thể lệch chút theo nền tảng/BLAS, nhưng hiện tượng mất chính xác gần như hoàn toàn là ổn định). Đổi sang công thức hai lượt (trừ trung bình trước, rồi bình phương, rồi lấy trung bình): Var ≈ 0.993, sai số tương đối chỉ khoảng 6×10^{-7} (tự tính), đúng bằng độ chính xác float32 kỳ vọng.

Bài học: khi viết code tính phương sai, chuẩn hoá (normalization layer), hay bất kỳ công thức ”trừ hai số lớn gần nhau” nào, nên dùng công thức hai lượt (hoặc thuật toán Welford), đặc biệt khi đang chạy ở fp16/bf16.

---

7. Lý thuyết học sâu ở mức khái niệm

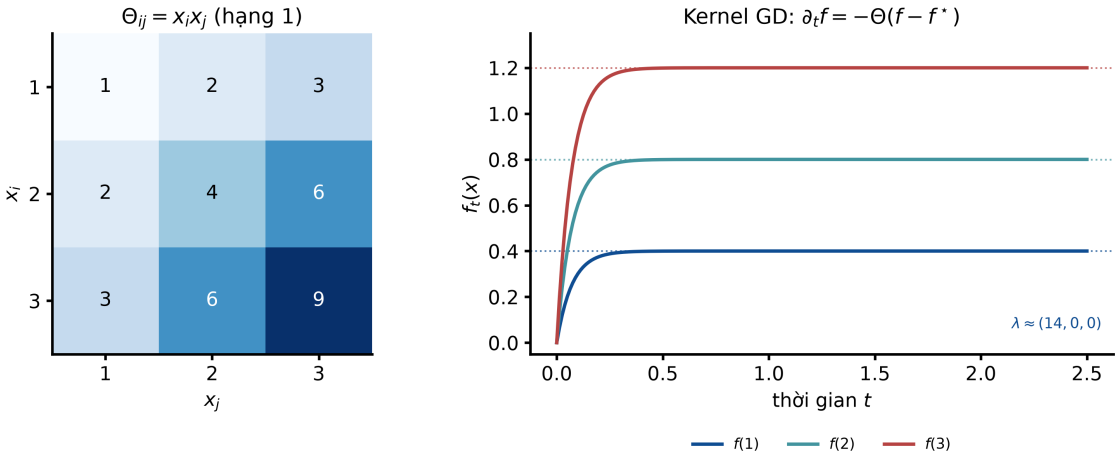
Mục này **không** chứng minh lại các định lý của bài 19-27, 31-34 (các bài đó tự làm việc đó, với giả thiết chặt chẽ hơn nhiều những gì nêu ở đây). Mục tiêu chỉ là đủ trực giác để đọc các bài đó **không bị chặn ngay ở trang đầu**.

7.1 Overparameterization: vì sao ”quá nhiều tham số” không tất yếu là xấu

Trực giác thống kê cổ điển (bias-variance trade-off, `ml basics content/` bài 02): mô hình nhiều tham số hơn dữ liệu sẽ ”khớp vẹt” (overfit), tổng quát hoá kém. Thực nghiệm hiện đại cho thấy mạng neural rất lớn, tham số nhiều hơn hẳn số điểm dữ liệu, **vẫn** tổng quát hoá tốt (`ml basics content/` bài 03, double descent). Bài 31 (khảo sát) tổng hợp hai mảng lý thuyết giải thích hiện tượng này: (a) overparameterization giúp **bài toán tối ưu dễ hơn**, gradient descent tới nghiệm khớp hoàn toàn dữ liệu dù landscape không lồi (bài 33 chứng minh cho vài kiến trúc cụ thể); (b) cách gradient descent **chọn nghiệm** trong số rất nhiều nghiệm khớp dữ liệu (implicit regularization) giúp tổng quát hoá tốt dù khớp hoàn hảo dữ liệu nhiễu (”benign overfitting”, bài 32 đặc trưng hoá chính xác điều kiện này cho hồi quy tuyến tính).

7.2 Neural Tangent Kernel (NTK): tuyến tính hoá quanh điểm khởi tạo

Ý tưởng (bài 23, 24): khi mạng **rất rộng**, huấn luyện bằng gradient descent gần giống huấn luyện phiên bản **tuyến tính hoá** của nó quanh tham số khởi tạo θ_0:



NTK đồ chơi $\Theta_i = x_i x$ (hạng 1) trên {1,2,3}; kernel GD hội tụ theo thành phần trong span của x. (minh hoạ, tự tính)

$$f(x;\theta) \approx f(x;\theta_0) + \nabla_\theta f(x;\theta_0)^\top (\theta - \theta_0).$$

Đây là một khai triển Taylor bậc 1 (liên hệ mục 3.1: đúng là tuyến tính hoá bằng gradient, giống ∇f ở đầu bài). Gradient descent trên vế phải tương đương **kernel gradient descent** với kernel

$$\boxed{\Theta(x, x') = \nabla_\theta f(x;\theta_0)^\top \nabla_\theta f(x';\theta_0)}$$

Bài 23 chứng minh: khi độ rộng tiến ra vô hạn, Θ gần như **không đổi** suốt quá trình huấn luyện (gọi là NTK), và động lực huấn luyện trở thành một bài toán **hồi quy kernel** tuyến tính, giải được bằng công cụ giải tích cổ điển, giải thích được tốc độ hội tụ.

Ví dụ tối giản (tự tính, minh hoạ, không phải NTK đầy đủ của một mạng thật). Model tuyến tính 1 tham số $f(x; w) = wx$: $\partial f/\partial w = x$, nên kernel là $\Theta(x, x') = x\,x'$. Với $x \in \{1, 2, 3\}$:

$$\Theta = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \\ 3 & 6 & 9 \end{pmatrix}$$

(tự tính, mỗi phần tử $\Theta_{ij} = x_i x_j$). Đây là ma trận hạng 1 (liên hệ mục 2.3): với model càng đơn giản, kernel càng thoái hoá; với mạng sâu thật, Θ có một công thức đóng dạng đệ quy qua các lớp (bài 23 suy ra cho ReLU) phức tạp hơn nhiều, không trình bày lại ở đây.

7.3 Lazy training và feature learning: hai chế độ khác nhau

Bài 24 chỉ ra: ”huấn luyện giống NTK” (gọi là **lazy training**, tham số hầu như không di chuyển xa khỏi θ_0, chỉ output thay đổi) là hệ quả của cách **scale** hàm số theo kích thước mô hình, không chỉ của việc có nhiều tham số. Ở chế độ ngược lại, **feature learning**, biểu diễn ẩn của mạng thực sự thay đổi trong lúc huấn luyện (học đặc trưng mới, không chỉ nội suy trong không gian đặc trưng cố định tại khởi tạo). Bài 24 cũng thực nghiệm cho thấy các CNN hiệu năng cao **không** nằm trong chế độ lazy. Bài học thực dụng: NTK là một công cụ phân tích rất đẹp về mặt toán nhưng mô tả đúng nhất một chế độ (lazy) mà các mô hình tốt nhất trong thực tế **không** hoạt động ở đó, nên cần đọc có phê phán (xem lại mục 10).

7.4 Lan truyền tín hiệu qua mạng sâu chưa huấn luyện (mean-field)

Câu hỏi của bài 26, 27: một mạng rất sâu, **ngẫu nhiên, chưa huấn luyện**, còn truyền được tín hiệu không? Bằng lý thuyết mean-field: với một số phân phối trọng số/bias ban đầu, tồn tại một **ranh giới order-to-chaos**; gần ranh giới (”critical”), tín hiệu và gradient truyền ổn định qua nhiều lớp; xa ranh giới, tín hiệu co về hằng số (order) hoặc khuếch đại hỗn loạn (chaos), cả hai triệt tiêu thông tin phân biệt giữa các input khi lớp sâu dần. Đây là nền tảng cho các công thức khởi tạo trọng số có kiểm soát phương sai (`optimization content/` bài 37, Kaiming init) và một phần động lực của skip-connection (bài 18: resnet làm chậm hẳn quá trình ”nhiễu hoá” gradient theo độ sâu).

7.5 Information bottleneck nhìn từ huấn luyện thật

Bài 30 quan sát thực nghiệm (không phải lý thuyết hình thức như 26/27): trong lúc huấn luyện một DNN, thông tin tương hỗ (mục 5.4) giữa biểu diễn ẩn và input/nhãn trải qua hai pha, drift (khớp dữ liệu nhanh) rồi diffusion (nén biểu diễn, bỏ thông tin dư thừa không liên quan nhãn). Bài 14 xây một cận dưới biến phân khả vi cho đúng mục tiêu information bottleneck này để huấn luyện trực tiếp bằng gradient descent (không chỉ quan sát), kết nối lại với suy luận biến phân (mục 4.4, bài 08, 09).

7.6 Khi nào đủ để đọc bài 19-34, khi nào cần đọc kỹ hơn

Nếu mục tiêu là **hiểu trực giác** (tại sao mô hình lớn vẫn học được, tại sao init quan trọng, NTK nghĩa là gì): mục 7.1-7.5 ở đây là đủ để đọc phần dẫn nhập và kết luận của các bài 19-34. Nếu mục tiêu là **dùng lại chứng minh hoặc công thức chính xác** (ví dụ suy ra công thức NTK cho một kiến trúc cụ thể, hay điều kiện chính xác của benign overfitting): phải đọc trực tiếp bài tương ứng, phần dẫn nhập ở đây không đủ.

8. Cạm bẫy thực hành

8.1 Quy ước hàng/cột và layout của gradient

Có hai quy ước phổ biến cho gradient của hàm scalar theo vector: ”denominator layout” (∇f là vector cột, cùng chiều x, dùng trong phần lớn tài liệu machine learning) và ”numerator layout” (gradient là vector hàng, phổ biến hơn trong một số sách giải tích). Bài 01 trong thư mục này chọn rõ một quy ước và nói thẳng lý do; khi so một công thức giữa hai paper khác nhau mà thấy ”bị chuyển vị” (transpose) so với công thức đã biết, nhiều khả năng chỉ là khác quy ước layout, không phải một bên sai.

8.2 ReLU chết (dead ReLU) và gradient biến mất

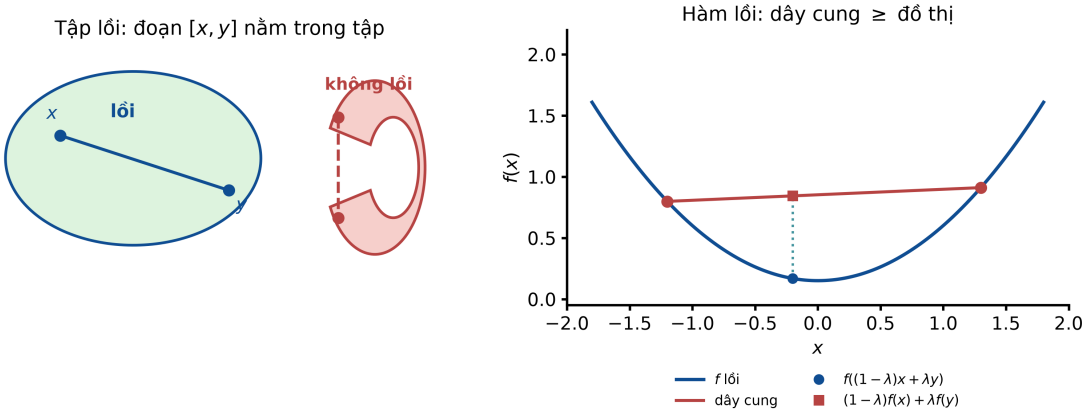
Mục 3.4 đã cho một ví dụ cụ thể: khi $z < 0$ ở một đơn vị ReLU, gradient qua đơn vị đó bằng 0 tuyệt đối, không phải ”rất nhỏ”. Nếu toàn bộ batch khiến một neuron luôn nhận $z < 0$, neuron đó không học được nữa cho tới khi (nếu có) một thay đổi ở lớp trước đẩy z dương trở lại. Khắc phục thường gặp: learning rate nhỏ hơn, khởi tạo cẩn thận (mục 7.4), hoặc đổi activation (Leaky ReLU, GELU) để giữ một gradient nhỏ khác 0 ở phía âm.

8.3 Nhầm công thức mẫu (sample) và tổng thể (population)

Phương sai mẫu ”không chệch” chia cho $n - 1$ (hiệu chỉnh Bessel), phương sai MLE chia cho n. Hai công thức cho hai số **khác nhau** (chênh nhau hệ số $n/(n-1)$, đáng kể khi n nhỏ). Nhiều thư viện (ví dụ `numpy.std` mặc định chia n, `pandas.std` mặc định chia $n-1$) chọn mặc định khác nhau: luôn kiểm tra tài liệu hàm trước khi so sánh số giữa hai công cụ.

8.4 Hướng bất đẳng thức Jensen

Với f **lồi** (convex): $\mathbb{E}[f(X)] \geq f(\mathbb{E}[X])$. Với f **lõm** (concave, ví dụ log): $\mathbb{E}[\log X] \leq \log \mathbb{E}[X]$. Dấu bất đẳng thức bị đảo khi nhầm lồi với lõm là lỗi rất thường gặp khi suy ra ELBO (evidence lower bound, bài 08, 09): ELBO là một **cận dưới** của log-likelihood chính vì log lõm.



Tập lồi (mọi đoạn thẳng giữa hai điểm nằm trong tập) và hàm lồi (dây cung nằm trên đồ thị, bất đẳng thức Jensen). (minh hoạ, tự tính)

8.5 Tràn số và mất số khi không dùng log-sum-exp

Lặp lại từ mục 6.3: bất kỳ chỗ nào có `softmax`, `logsumexp`, hay tính log-likelihood của một tích nhiều xác suất nhỏ (ví dụ log-likelihood của cả một câu dài), luôn làm việc trong không gian **log** và dùng log-sum-exp, không nhân trực tiếp nhiều số nhỏ hơn 1 rồi mới lấy log.

8.6 Ma trận gần suy biến làm nổ số (ill-conditioning)

Mục 2.6 cho ví dụ cụ thể: $\kappa \approx 20000$ khi hai hàng gần như trùng nhau. Bất kỳ phép nghịch đảo ma trận trong code (natural gradient, Gauss-Newton, least-squares không ridge) nên luôn cộng thêm một λI nhỏ, hoặc dùng giả nghịch đảo (pseudo-inverse) có cắt ngưỡng giá trị kỳ dị nhỏ, trước khi tin vào kết quả.

8.7 Lưu ý khi viết công thức markdown/pandoc cho các bài trong thư mục

Khi build các bài (01-35) hoặc file này thành PDF qua pandoc: (a) luôn đóng đủ cặp dấu mở và đóng công thức, một dấu lẻ làm pandoc đọc sai toàn bộ phần còn lại của file; (b) không để một chữ số đứng dính ngay sau phần đóng công thức mà không có khoảng trắng ở giữa, ví dụ viết "k = 5" (có khoảng trắng trước số 5) thay vì để số 5 dính liền ngay sau phần đóng công thức của k; (c) trong bảng markdown, không dùng dấu gạch đứng trần `|` bên trong công thức (nó bị hiểu là dấu phân cột), dùng `\mid` cho điều kiện xác suất, `\vert` hoặc `\Vert` cho giá trị tuyệt đối/chuẩn; (d) tránh viết địa chỉ dạng `tên@miền` ngoài khối code, một số bộ render Markdown tự biến nó thành liên kết; (e) không dùng `\par` trong tiêu đề.

8.8 So sánh không cùng điều kiện

Khi đọc kết quả thực nghiệm (`ml basics content/`, `optimization content/`), luôn hỏi: cùng số bước huấn luyện hay cùng số giờ GPU? Cùng kích thước batch? Cùng được tune hyperparameter công bằng không? Rất nhiều "phương pháp mới thắng baseline" chỉ vì baseline chưa được tune đủ kỹ (nguyên tắc này lặp lại ở hầu hết thư mục chuyên đề, ví dụ `rl content/00` mục 7).

---

9. Bảng thuật ngữ Anh-Việt

English	Tiếng Việt / ghi chú
Vector, matrix, tensor	vector, ma trận, tensor (mảng nhiều chiều)
Rank	hạng (số chiều độc lập tuyến tính)
Eigenvalue, eigenvector	trị riêng, vector riêng
Singular value, SVD	giá trị kỳ dị, phân tích giá trị kỳ dị
Condition number	điều kiện số (độ "khó nghịch đảo")
Low-rank approximation	xấp xỉ hạng thấp
Frobenius norm	chuẩn Frobenius
Orthogonal, orthonormal	trực giao (vuông góc), trực chuẩn (vuông góc và chuẩn 1)
Positive (semi-)definite	(bán) xác định dương
Gradient	gradient (vector đạo hàm riêng)
Jacobian	ma trận Jacobian (đạo hàm của hàm vector)
Hessian	ma trận Hessian (đạo hàm bậc hai)
Chain rule	quy tắc chuỗi
Automatic differentiation (AD)	vi phân tự động (autodiff)
Forward mode / reverse mode	chế độ thuận / chế độ ngược
Backpropagation	lan truyền ngược
Saddle point	điểm yên ngựa
Random variable	biến ngẫu nhiên
Expectation, variance, covariance	kỳ vọng, phương sai, hiệp phương sai
Multivariate Gaussian	Gaussian đa biến (phân phối chuẩn nhiều chiều)
Mahalanobis distance	khoảng cách Mahalanobis
Bayes' theorem	định lý Bayes
Prior, posterior, likelihood	niềm tin trước (tiên nghiệm), hậu nghiệm, hàm hợp lý

English	Tiếng Việt / ghi chú
Maximum likelihood estimation (MLE)	ước lượng hợp lý cực đại
Maximum a posteriori (MAP)	ước lượng hậu nghiệm cực đại
Estimator, bias, consistency	bộ ước lượng, độ chệch, tính nhất quán
Concentration inequality	bất đẳng thức tập trung
Entropy	entropy (độ bất định)
Cross-entropy	entropy chéo
Kullback-Leibler (KL) divergence	độ lệch KL
Mutual information (MI)	thông tin tương hỗ
Information bottleneck	nút cổ chai thông tin
Negative log-likelihood (NLL)	log-hợp lý âm
Perplexity	độ rối (của language model)
Floating point	số dấu phẩy động
Mantissa, exponent	định trị, phần mũ
Overflow, underflow	tràn số (lên), tràn số (xuống 0)
Numerical stability	ổn định số
log-sum-exp	log-tổng-exp (thủ thuật tính softmax an toàn)
Catastrophic cancellation	mất số do trừ hai số gần nhau
Overparameterization	quá tham số hoá
Neural Tangent Kernel (NTK)	nhân tiếp tuyến thần kinh
Lazy training / feature learning	huấn luyện ”lười” (gần tuyến tính hoá) / học đặc trưng
Mean-field theory	lý thuyết mean-field (trường trung bình)
Benign overfitting	khớp quá độ lành tính
Implicit regularization	chính quy hoá ngầm
Kronecker product	tích Kronecker
Natural gradient	gradient tự nhiên
Fisher information matrix	ma trận thông tin Fisher
Evidence lower bound (ELBO)	cận dưới chứng cớ
Optimal transport, Wasserstein distance	vận tải tối ưu, khoảng cách Wasserstein

10. Cách đọc paper toán, lý thuyết một cách phản biện

1. **Phân biệt ba loại câu:** điều paper *chứng minh* (có giả thiết rõ, có bước suy luận), điều paper *quan sát thực nghiệm* (đúng trên setup cụ thể, chưa chắc tổng quát), và điều *ta suy ra thêm*. Ghi chú trong thư mục này luôn tách rõ bằng ”(tự tính)” và các cụm ”paper nói” / ”suy luận của người ghi chú”.
2. **Đọc giả thiết trước khi đọc kết luận.** Một định lý ”mạng rất rộng thì NTK không đổi” chỉ đúng dưới giả thiết độ rộng tiến ra vô hạn và cách khởi tạo/scale cụ thể (mục 7.2-7.3); áp trực tiếp cho một mạng hữu hạn, scale khác, là ngoài phạm vi định lý.
3. **Luôn thử tính bằng số trước khi tin một bound.** Mục 4.5 cho thấy cùng một bất đẳng thức đúng về đại số có thể vô nghĩa (> 1) hoặc rất chặt, tuỳ n, ϵ. Cắm số cụ thể vào trước khi dùng một bound để ”chứng minh” điều gì trong thực hành.
4. **Phân biệt kết quả tồn tại (existence) và kết quả xây dựng được (constructive).** ”Tồn tại một hàm mà mạng sâu tính được nhưng mạng nông không tính được” (bài 20) không có nghĩa là mọi hàm thực tế đều như vậy.

5. **Chú ý thang đo của thực nghiệm minh hoạ.** Nhiều kết quả lý thuyết học sâu (mục 7) chứng minh trên mạng tuyến tính, mạng 2 lớp, hoặc giới hạn vô hạn chiều rộng; khoảng cách giữa đó và một transformer vài tỷ tham số có thể rất lớn, paper thường tự nêu rõ giới hạn này ở phần ”limitations” hoặc ”discussion”.
6. **Khi một paper là sách/khảo sát rất dài** (bài 12, 25, 28, 31), kiểm phạm vi đã đọc được ghi ở đầu bài ghi chú tương ứng (thường trong trường ”Vị trí trong lộ trình” hoặc mục 1) trước khi coi đó là tóm tắt toàn bộ cuốn sách.
7. **Theo dõi ký hiệu riêng của từng paper.** d có thể là số chiều ở một bài, là ”depth” (độ sâu) ở bài khác; σ có thể là độ lệch chuẩn hoặc một hàm activation. Không mang ký hiệu từ bài này sang bài khác mà không kiểm lại định nghĩa.

---

11. Lịch đọc gợi ý

Thư mục đã có `README.md` chia 35 bài thành 10 giai đoạn (giai đoạn 0 là preface này); lịch dưới đây dùng lại đúng nhóm đó, kèm ghi chú bài nào nặng và có thể lùi lại.

Giai đoạn 1 (bài 01-05): công cụ tính toán và đại số. `01_2018_matrix_calculus.md`, `02_2015_automatic_differentiation_survey.md`, `03_2014_pca_tutorial.md`, `04_2009_randomized_matrix_decomposition.md`, `05_2007_spectral_clustering_tutorial.md`. Lý do nhóm: đây là công cụ tính toán dùng ngay trong code, không cần nền lý thuyết nào trước. Đọc đầy đủ, không bài nào nên bỏ qua.

Giai đoạn 2 (bài 06-09): hình học thông tin và suy luận biến phân. `06_2014_natural_gradient_insights.md`, `07_2018_information_geometry_intro.md`, `08_2016_variational_inference_review.md`, `09_2019_vae_introduction.md`. Bài 07 (hình học thông tin, đa tạp dually flat) là bài **nặng** nhất nhóm này về mức độ hình thức; nếu chỉ cần hiểu natural gradient để dùng, đọc kỹ bài 06 và lướt bài 07.

Giai đoạn 3 (bài 10-11): MCMC và SGD như suy luận Bayes. `10_2017_hamiltonian_monte_carlo.md`, `11_2017_sgd_approximate_bayesian_inference.md`. Có thể lùi lại nếu không làm việc trực tiếp với lấy mẫu Bayes hay uncertainty quantification.

Giai đoạn 4 (bài 12-14): vận tải tối ưu và thông tin. `12_2018_computational_optimal_transport.md` (monograph khoảng 250 trang, **nặng nhất của cả thư mục** về độ dài, lần đầu chỉ nên đọc phần Monge/Kantorovich và Sinkhorn), `13_2017_wasserstein_gan.md`, `14_2016_deep_variational_information_bottleneck.md`.

Giai đoạn 5 (bài 15-18): số học và ổn định số. `15_2018_online_softmax.md`, `16_2022_fp8_formats.md`, `17_2018_spectral_normalization_gan.md`, `18_2017_shattered_gradients.md`. Thực dụng, dùng ngay khi viết code; không bài nào nên bỏ qua.

Giai đoạn 6 (bài 19-27): biểu diễn, độ sâu, độ rộng, kernel, lan truyền tín hiệu. `19_2018_intrinsic_dimension.md`, `20_2016_benefits_of_depth.md`, `21_2017_expressive_power_width.md`, `22_2017_deep_nn_gaussian_processes.md`, `23_2018_ntk.md`, `24_2018_lazy_training.md`, `25_2021_principles_dl_theory.md` (sách ~470 trang, chỉ đọc phần được chọn, xem ghi chú đầu bài), `26_2016_exponential_expressivity_chaos.md`, `27_2016_deep_information_propagation.md`. README đã ghi rõ: đây là **lý thuyết nặng nhất** của thư mục cùng với giai đoạn 8; lần đầu đọc, chỉ cần mục 1, 2, 11 (bối cảnh, ý tưởng chính, kết luận) của từng bài, dựa vào mục 7 ở trên để nắm trực giác, rồi quay lại đọc kỹ sau khi đã quen. Nếu phải chọn ưu tiên trong nhóm này: 23 (NTK) và 24 (lazy training) hữu ích nhất cho việc đọc các bài LLM theory hiện đại; 20, 21, 26, 27 có thể lùi lại nếu không làm việc trực tiếp với lý thuyết biểu diễn.

Giai đoạn 7 (bài 28-30): hình học, diffusion, thông tin trong mạng. `28_2021_geometric_deep_learning.md` (dài, chỉ đọc phạm vi đã chọn ở đầu bài), `29_2022_understanding_diffusion_models.md`,

`30_2017_opening_black_box_information.md`. Bài 28 hữu ích nếu làm việc với kiến trúc có cấu trúc đối xứng (graph, mesh); bài 29 cần nếu đụng tới mô hình diffusion.

Giai đoạn 8 (bài 31-34): tổng quát hoá và overparameterization. `31_2021_deep_learning_statistical_viewpoint.md` (khảo sát, đọc phần tổng quan trước), `32_2019_benign_overfitting_linear_regression.md`, `33_2018_gradient_descent_global_minima_dnn.md`, `34_2018_mean_field_two_layer_networks.md`. Cùng mức nặng như giai đoạn 6 (nhiều chứng minh hình thức); đọc bài 31 trước để có bản đồ tổng quan, 3 bài còn lại là ví dụ cụ thể hoá, có thể chỉ đọc kết luận lần đầu.

Giai đoạn 9 (bài 35): bandit và quyết định tuần tự. `35_2017_tutorial_thompson_sampling.md`. Độc lập với các giai đoạn trước, có thể đọc bất cứ lúc nào, hữu ích trước khi đọc `rl content/` (bandit là RL đơn giản nhất).

Đường tắt cho người chỉ cần đọc để hiểu các thư mục chuyên đề khác (không cần lý thuyết sâu): preface này (mục 1-6, 9) → 01, 02 (matrix calculus, autodiff) → 03 (PCA) → 15, 16 (ổn định số, fp8) → đọc mục 7 ở trên thay cho 19-27, 31-34 → quay lại đọc chi tiết bài cụ thể chỉ khi một bài ở thư mục chuyên đề trích dẫn trực tiếp.

---

12. Liên hệ giữa các chủ đề

Đại số tuyến tính cung cấp ngôn ngữ cho biểu diễn dữ liệu, phép chiếu và xấp xỉ hạng thấp. Giải tích và vi phân tự động giúp xây dựng các thuật toán tối ưu; xác suất và lý thuyết thông tin giúp mô tả bất định và so sánh phân phối. Số học máy tính nhắc ta kiểm tra độ ổn định của các công thức khi triển khai. Các bài về lý thuyết học sâu tiếp tục kết nối những công cụ này với khả năng biểu diễn và động lực học huấn luyện.

13. Năm câu tự kiểm tra

Câu 1. Ma trận $\Sigma = \begin{pmatrix} 4 & 2 \\ 2 & 4 \end{pmatrix}$ có trị riêng 2 và 6. Viết xấp xỉ hạng 1 tốt nhất của Σ theo chuẩn Frobenius, và tính sai số xấp xỉ.

Đáp án.

Theo định lý Eckart-Young (mục 2.6), xấp xỉ hạng 1 tốt nhất giữ lại giá trị kỳ dị (ở đây bằng trị riêng, vì Σ đối xứng bán xác định dương) lớn nhất, tức $\lambda = 6$ với vector riêng $v_2 = (1,1)/\sqrt{2}$: $\Sigma_1 = 6\, v_2 v_2^\top = \begin{pmatrix} 3 & 3 \\ 3 & 3 \end{pmatrix}$. Sai số Frobenius bằng giá trị kỳ dị bị bỏ: $\|\Sigma - \Sigma_1\|_F = 2$ (tự tính, khớp numpy).

Câu 2. Vì sao reverse-mode autodiff (backpropagation) hiệu quả hơn forward-mode cho việc huấn luyện mạng neural, trong khi cả hai đều tính đạo hàm chính xác?

Đáp án.

Loss của mạng neural là **một số scalar** (ít output), nhưng số tham số (input của phép tính gradient) có thể lên tới hàng tỷ. Forward-mode cần một lượt chạy riêng cho **mỗi** input (ở đây là mỗi tham số) để lấy đạo hàm theo input đó; với hàng tỷ tham số, điều này không khả thi. Reverse-mode cần đúng **một** lượt chạy xuôi và một lượt chạy ngược để lấy gradient theo **toàn bộ** tham số cùng lúc (mục 3.3), vì nó lan truyền đạo hàm từ phía có ít output (1) về phía có nhiều input.

Câu 3. Một reward model hay bộ phân loại có ”độ nhạy” (sensitivity) 99% khi phát hiện một hành vi hiếm (tỉ lệ cơ sở 1%), và tỉ lệ dương giả 5%. Nếu nó báo ”phát hiện”, xác suất hành vi đó thật sự xảy ra là bao nhiêu? Bài học rút ra cho việc đánh giá một bộ kiểm tra tự động là gì?

Đáp án.

Theo Bayes (mục 4.3), $P(\text{thật} \mid \text{báo phát hiện}) = \dfrac{0.01 \times 0.99}{0.01 \times 0.99 + 0.99 \times 0.05} = \dfrac{0.0099}{0.0594} \approx 0.167$ (khoảng 16,7%). Dù độ nhạy rất cao, xác suất đúng khi có báo động chỉ khoảng 1/6, vì hành vi hiếm (tỉ lệ cơ sở nhỏ) và vẫn có dương giả. Bài học: không thể chỉ nhìn độ nhạy của một bộ kiểm tra để tin vào kết quả nó báo, phải tính cả tỉ lệ cơ sở thật của thứ đang tìm.

Câu 4. Trong ví dụ backprop 2 lớp ở mục 3.4, vì sao $\partial L/\partial W_1$ có dòng đầu bằng 0 hoàn toàn? Nếu tình trạng này giữ nguyên qua nhiều bước cập nhật, hệ quả là gì?

Đáp án.

Vì đơn vị ẩn thứ nhất có thành phần tương ứng của z_1 bằng $-0.05 < 0$, nên thành phần tương ứng của $a_1 = \text{ReLU}(z_1)$ bằng 0 và đạo hàm của ReLU tại điểm âm bằng 0 tuyệt đối; gradient không thể ”chảy qua” một cổng có đạo hàm 0, nên mọi gradient theo dòng tương ứng của W_1 và thành phần tương ứng của b_1 đều bị nhân với 0. Nếu tình trạng đó giữ nguyên qua nhiều bước (ReLU chết, mục 8.2), đơn vị đó đóng băng vĩnh viễn: không còn đóng góp gì vào việc học, tương đương lãng phí một phần công suất của mạng.

Câu 5. Vì sao cross-entropy loss khi huấn luyện language model lại đúng bằng negative log-likelihood (NLL) của token đúng, và vì sao hai thứ tưởng như khác nhau (một là ”khoảng cách giữa phân phối”, một là ”log-hợp lý”) lại là cùng một số?

Đáp án.

Nhãn thật ở một vị trí là một token cụ thể, nên phân phối ”thật” p là one-hot: $p_{\text{đúng}} = 1$, còn lại 0, và $H(p) = 0$ (mục 5.1, không còn bất định vì đã biết chắc). Khi đó cross-entropy rút gọn: $H(p, q) = -\sum_i p_i \log q_i = -\log q_{\text{đúng}}$ (chỉ còn một số hạng khác 0), và vì $H(p) = 0$ nên $\text{KL}(p\|q) = H(p, q) - H(p) = H(p, q)$ cũng bằng đúng số đó. Mà $-\log q_{\text{đúng}}$ chính là định nghĩa của NLL. Ba cách nhìn (tối thiểu cross-entropy, tối thiểu KL tới nhãn thật, tối đa log-likelihood) trùng nhau chỉ vì nhãn là one-hot; với nhãn ”mềm” (soft label, như trong distillation, mục 12) $H(p) \neq 0$ và ba cách nhìn **không còn trùng nhau tuyệt đối**, dù vẫn liên hệ qua đúng công thức $H(p, q) = \text{KL}(p\|q) + H(p)$.

---

14. Bảng tra công thức

Khái niệm	Công thức	Mục
Chuẩn Euclid	$\|x\|_2 = \sqrt{x^\top x}$	2.1
Trị riêng	$Av = \lambda v$	2.4
SVD	$A = U\Sigma_s V^\top$	2.5
Điều kiện số	$\kappa(A) = \sigma_{\max}/\sigma_{\min}$	2.6
Xấp xỉ hạng thấp (Eckart-Young)	$A_k = \sum_{i \le k} \sigma_i u_i v_i^\top$	2.6
Least squares có ridge	$W^\star = YX^\top(XX^\top + \lambda I)^{-1}$	2.8
Gradient, Jacobian, Hessian	∇f, $J_{ij} = \partial g_i/\partial x_j$, $H_{ij} = \partial^2 f/\partial x_i \partial x_j$	3.1
Chain rule (vector)	$\nabla_x y = J_g^\top \nabla_z y$	3.2
$\partial/\partial W$ của $\|Wx - y\|^2$	$2(Wx - y)x^\top$	3.2
Gaussian đa biến	$p(x) = \dfrac{1}{(2\pi)^{d/2}\|\Sigma\|^{1/2}} \exp(-\frac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu))$	4.2
Định lý Bayes	$P(A \mid B) = P(B \mid A)P(A)/P(B)$	4.3
MAP của Beta-Bernoulli	$\hat{p} = (\alpha - 1)/(\alpha + \beta - 2)$	4.4
Hoeffding	$P(\|\bar{X}_n - \mathbb{E}X\| > \epsilon) \le 2\exp(-2n\epsilon^2)$	4.5
Entropy	$H(p) = -\sum_i p_i \log p_i$	5.1

Khái niệm	Công thức	Mục
Cross-entropy, KL	$H(p,q) = -\sum p_i \log q_i$, $\mathrm{KL}(p\|q) = H(p,q) - H(p)$	5.2
NLL = cross-entropy khi p one-hot	$-\log q_{\text{đúng}}$	5.3
Mutual information	$I(X;Y) = \mathrm{KL}(p(x,y)\|p(x)p(y))$	5.4
log-sum-exp ổn định	$\log \sum_j e^{z_j} = m + \log \sum_j e^{z_j - m}$, $m = \max_j z_j$	6.3
NTK	$\Theta(x,x') = \nabla_\theta f(x;\theta_0)^\top \nabla_\theta f(x';\theta_0)$	7.2

Tài liệu này không có đường link gốc vì không ghi chú một paper cụ thể. Phần công cụ nên đọc kèm: Boyd và Vandenberghe, "Convex Optimization"; Deisenroth, Faisal và Ong, "Mathematics for Machine Learning"; Goodfellow, Bengio và Courville, "Deep Learning"; Murphy, "Probabilistic Machine Learning"; Strang, "Introduction to Linear Algebra". Bước tiếp theo: mở `01_2018_matrix_calculus.md`*.*

Bài 01

The Matrix Calculus You Need For Deep Learning (2018)

Tác giả: Terence Parr, Jeremy Howard (University of San Francisco, chương trình MS in Data Science; đồng thời liên quan tới fast.ai).
Venue / arXiv: arXiv:1802.01528 (v3, 02/07/2018); paper không nêu venue hội nghị/tạp chí (đây là tài liệu tutorial tự xuất bản, có bản HTML song song tạo từ cùng nguồn markup; trang đầu còn ghi tay ngày "July 3, 2018", lệch 1 ngày so với stamp arXiv v3 thật là 02/07/2018, chi tiết nhỏ không ảnh hưởng nội dung).
Nguồn: https://arxiv.org/abs/1802.01528
Chủ đề: dựng lại từ đầu phần vi tích phân ma trận (đạo hàm theo vector, Jacobian, 3 dạng quy tắc chuỗi) cần để hiểu backpropagation, rồi áp dụng trực tiếp để tính gradient của một neuron ReLU và của hàm loss MSE.
Vị trí trong lộ trình: đây là bài ĐẦU TIÊN trong batch math 01-09 (không có bài trước trong batch này). Nó dựng nền (đạo hàm vector, Jacobian, chain rule) mà bài tiếp theo, math 02 (automatic differentiation survey), dùng trực tiếp để định nghĩa chính xác forward-mode/reverse-mode AD. Đọc trước math 02.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (bản arXiv); các phép tính đánh dấu "tự tính" là của người ghi chú. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc.

1. Bối cảnh

Để huấn luyện mạng nơ-ron bằng gradient descent, ta cần đạo hàm của hàm mất mát theo từng tham số, gồm trọng số w và bias b. Các thư viện như PyTorch tính đạo hàm tự động (xem bài 02), nhưng giải tích ma trận vẫn cần thiết để đọc các công trình học sâu và kiểm tra phép tính gradient tự viết. Công cụ này kết hợp đại số tuyến tính với giải tích nhiều biến.

Vấn đề paper nêu ra: tài liệu online về matrix calculus thường là danh sách công thức khô khan, ít giải thích, hướng tới người đọc đã quen ký hiệu toán chặt. Mục tiêu của bài là **dựng lại từ đầu** một số quy tắc đạo hàm ma trận quan trọng nhất cho deep learning, chỉ cần kiến thức calculus 1 (đạo hàm hàm một biến) làm nền.

Các khái niệm cần nắm trước khi đọc (paper giả định người đọc đã quen mạng neural ở mức thực hành):

- **Đạo hàm (derivative)** $\frac{d}{dx}f(x)$: tốc độ thay đổi của f theo x.
- **Đạo hàm riêng (partial derivative)** $\frac{\partial}{\partial x}f(x,y)$: đạo hàm theo x, coi các biến khác là hằng số.
- **Gradient** $\nabla f(x,y) = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]$: vector (hàng, theo quy ước paper) gom tất cả đạo hàm riêng của MỘT hàm trả về scalar.
- **Jacobian**: khi có NHIỀU hàm trả về scalar (gộp thành vector hàm $\mathbf{y} = \mathbf{f}(\mathbf{x})$), Jacobian là ma trận gồm các gradient xếp thành hàng.

Bài toán cụ thể paper giải quyết tới cuối: tính $\frac{\partial C}{\partial w}$ và $\frac{\partial C}{\partial b}$ cho một neuron đơn `activation(x) = max(0, w · x + b)` (fully-connected + ReLU) và loss MSE, đây chính là bước cần cho một lần cập nhật gradient descent.

2. Ý tưởng chính

Bài báo suy ra các quy tắc giải tích ma trận bằng cách viết biểu thức vector hoặc ma trận thành các phương trình vô hướng tương đương, lấy đạo hàm rồi viết lại dưới dạng gọn. Cách trình bày này phân biệt ba dạng quy tắc chuỗi thường dễ bị nhầm lẫn:

1. **Single-variable chain rule**: $\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx}$, chỉ đúng khi có DUY NHẤT một đường truyền dữ liệu từ x tới y.

2. **Single-variable total-derivative chain rule**: khi một biến trung gian phụ thuộc x qua nhiều đường, phải CỘNG đóng góp của tất cả các đường: $\frac{\partial f(x,u_1,\ldots,u_n)}{\partial x} = \frac{\partial f}{\partial x} + \sum_i \frac{\partial f}{\partial u_i}\frac{\partial u_i}{\partial x}$.
3. **Vector chain rule**: $\frac{\partial}{\partial \mathbf{x}}\mathbf{f}(\mathbf{g}(\mathbf{x})) = \frac{\partial \mathbf{f}}{\partial \mathbf{g}}\frac{\partial \mathbf{g}}{\partial \mathbf{x}}$ (tích hai Jacobian), và đây là dạng **tổng quát nhất**: hai dạng trên chỉ là trường hợp đặc biệt của nó. Thông điệp cốt lõi của paper: ”chỉ cần nhớ vector chain rule, còn lại tự suy ra”.

Áp dụng toàn bộ máy này cho một neuron `max(0, w·x + b)` và loss MSE cho ra công thức gradient cuối cùng (mục 3.6), đúng là công thức mọi người quen dùng khi lập trình gradient descent tay cho hồi quy tuyến tính/logistic.

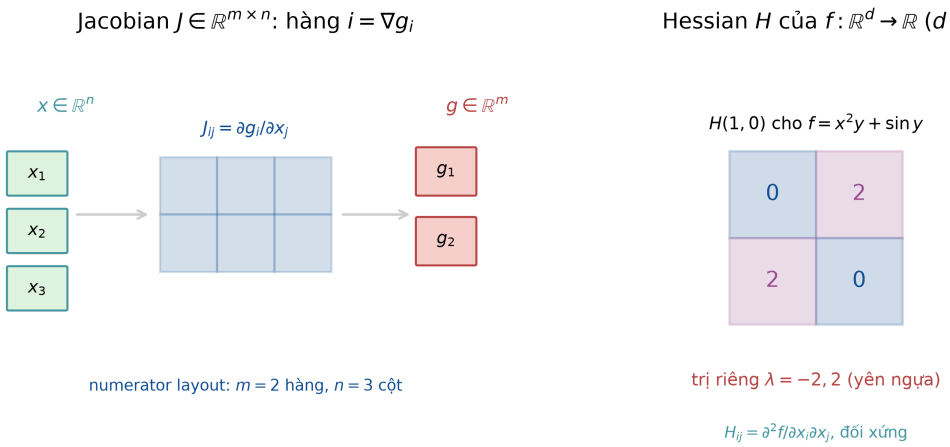
3. Phương pháp chi tiết

3.1 Ôn lại quy tắc đạo hàm scalar (Mục 2)

Bảng các quy tắc cơ bản (Power rule $\frac{d}{dx}x^n = nx^{n-1}$, Sum/Difference rule, Product rule $\frac{d}{dx}fg = f\frac{dg}{dx} + \frac{df}{dx}g$, Chain rule $\frac{d}{dx}f(g(x)) = \frac{df(u)}{du}\frac{du}{dx}$ với $u = g(x)$). Bài báo nhấn mạnh nên coi $\frac{d}{dx}$ là một **toán tử** (map một hàm số thành hàm đạo hàm của nó), có tính phân phối và cho phép đưa hằng số ra ngoài, giúp đơn giản hoá biểu thức phức tạp bằng cách chia nhỏ.

3.2 Đạo hàm riêng và gradient (Mục 3)

Với $f(x, y)$, mỗi biến có một đạo hàm riêng ($\frac{\partial}{\partial x}$ coi y là hằng số, và ngược lại). Gom các đạo hàm riêng của MỘT hàm số scalar lại thành vector hàng gọi là **gradient**: $\nabla f(x, y) = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right]$. Ví dụ paper: $f(x, y) = 3x^2y \Rightarrow \nabla f = [6xy, 3x^2]$.



Hình dạng Jacobian (numerator layout, hàng = gradient từng đầu ra) và Hessian đối xứng của hàm scalar; ví dụ H(1,0) có trị riêng ±2. (minh hoạ, tự tính)

3.3 Jacobian (Mục 4, 4.1)

Khi có nhiều hàm scalar cùng nhận các tham số đó, xếp các gradient thành HÀNG của một ma trận, ta được **Jacobian**. Với $\mathbf{y} = \mathbf{f}(\mathbf{x})$, $\mathbf{f}$ gồm m hàm $f_1, \ldots, f_m$, $\mathbf{x}$ có n phần tử, Jacobian là ma trận $m \times n$:

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \begin{bmatrix} \nabla f_1(\mathbf{x}) \\ \vdots \\ \nabla f_m(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} f_1(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_1(\mathbf{x}) \\ \vdots & & \vdots \\ \frac{\partial}{\partial x_1} f_m(\mathbf{x}) & \cdots & \frac{\partial}{\partial x_n} f_m(\mathbf{x}) \end{bmatrix}$$

Paper dùng **numerator layout** (hàng = hàm, cột = biến); nhiều tài liệu khác dùng denominator layout, chỉ là Jacobian transpose. Ví dụ minh hoạ paper đưa: Jacobian của hàm đồng nhất $\mathbf{f}(\mathbf{x}) = \mathbf{x}$ là ma trận đơn vị I (chứng minh bằng cách tính từng phần tử: $\frac{\partial}{\partial x_j} x_i = 0$ nếu $j \neq i$, $= 1$ nếu $j = i$).

3.4 Phép toán element-wise và "diagonal condition" (Mục 4.2, 4.3, 8.2, 8.3)

Với phép toán element-wise $\mathbf{y} = \mathbf{f}(\mathbf{w}) \odot \mathbf{g}(\mathbf{x})$ (ví dụ $\mathbf{w} + \mathbf{x}$, $\mathbf{w} - \mathbf{x}$, tích Hadamard $\mathbf{w} \otimes \mathbf{x}$, chia element-wise), nếu f_i chỉ phụ thuộc w_i và g_i chỉ phụ thuộc x_i ("element-wise diagonal condition"), Jacobian suy biến thành **ma trận đường chéo**:

$$\frac{\partial \mathbf{y}}{\partial \mathbf{w}} = \text{diag}\left(\ldots, \frac{\partial}{\partial w_i}(f_i(w_i)g_i(x_i)), \ldots\right)$$

Bảng kết quả cụ thể (paper, Mục 8.2): $\frac{\partial(\mathbf{w}+\mathbf{x})}{\partial \mathbf{w}} = I$; $\frac{\partial(\mathbf{w}-\mathbf{x})}{\partial \mathbf{w}} = I$, $\frac{\partial(\mathbf{w}-\mathbf{x})}{\partial \mathbf{x}} = -I$; $\frac{\partial(\mathbf{w}\otimes\mathbf{x})}{\partial \mathbf{w}} = \text{diag}(\mathbf{x})$, $\frac{\partial(\mathbf{w}\otimes\mathbf{x})}{\partial \mathbf{x}} = \text{diag}(\mathbf{w})$. Mở rộng tự nhiên là **scalar expansion** ($\mathbf{y} = \mathbf{x} + z$ hay $\mathbf{y} = \mathbf{x}z$ với z scalar): $\frac{\partial(\mathbf{x}+z)}{\partial \mathbf{x}} = I$, $\frac{\partial(\mathbf{x}+z)}{\partial z} = \mathbf{1}$ (vector cột toàn 1); $\frac{\partial(\mathbf{x}z)}{\partial \mathbf{x}} = Iz$, $\frac{\partial(\mathbf{x}z)}{\partial z} = \mathbf{x}$.

3.5 Vector sum reduction và đạo hàm dot product (Mục 4.4, 8.4)

Với $y = \text{sum}(\mathbf{f}(\mathbf{x})) = \sum_i f_i(\mathbf{x})$, gradient là $\nabla y = \left[\sum_i \frac{\partial f_i}{\partial x_1}, \ldots, \sum_i \frac{\partial f_i}{\partial x_n}\right]$. Trường hợp riêng quan trọng nhất cho deep learning: **đạo hàm của dot product** $y = \mathbf{w} \cdot \mathbf{x} = \text{sum}(\mathbf{w} \otimes \mathbf{x})$. Đặt biến trung gian $\mathbf{u} = \mathbf{w} \otimes \mathbf{x}$, dùng vector chain rule (xem 3.6):

$$\frac{\partial \mathbf{u}}{\partial \mathbf{w}} = \text{diag}(\mathbf{x}), \qquad \frac{\partial y}{\partial \mathbf{u}} = \mathbf{1}^T \qquad \Rightarrow \qquad \frac{\partial y}{\partial \mathbf{w}} = \mathbf{1}^T \text{diag}(\mathbf{x}) = \mathbf{x}^T, \quad \text{tương tự } \frac{\partial y}{\partial \mathbf{x}} = \mathbf{w}^T$$

Paper kiểm lại bằng cách tính trực tiếp từng phần tử scalar ($\frac{\partial}{\partial w_j} \sum_i w_i x_i = x_j$) và khớp với kết quả trên.

3.6 Ba loại chain rule (Mục 4.5)

(a) Single-variable chain rule. Với $y = f(g(x))$, đặt $u = g(x)$: $\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx}$. Chỉ áp dụng khi x ảnh hưởng tới y qua **đúng một** đường dữ liệu (data-flow path). Ví dụ paper: $y = \sin(x^2)$, đặt $u = x^2$: $\frac{dy}{dx} = \cos(u) \cdot 2x = 2x\cos(x^2)$.

(b) Single-variable total-derivative chain rule. Khi biểu thức có biến trung gian phụ thuộc x qua NHIỀU đường (ví dụ $y = x + x^2$, viết lại $u_1 = x^2$, $u_2(x, u_1) = x + u_1$), áp dụng trực tiếp single-variable chain rule sẽ SAI (paper minh hoạ: nếu "mù quáng" lấy $\frac{du_2}{du_1}\frac{du_1}{dx} = 1 \times 2x = 2x$ thì thiếu hẳn đường trực tiếp $x \to y$, kết quả đúng phải là $1 + 2x$). Công thức tổng quát (total derivative, "luật tổng các đạo hàm"):

$$\frac{\partial f(x, u_1, \ldots, u_n)}{\partial x} = \frac{\partial f}{\partial x} + \sum_{i=1}^{n} \frac{\partial f}{\partial u_i}\frac{\partial u_i}{\partial x}$$

Paper kiểm bằng số: với $x = 1$ (nên $y = 1 + 1^2 = 2$), tăng $\Delta x = 1$ ($\hat{y} = 2 + 2^2 = 6$), $\Delta y = 6 - 2 = 4$ khớp với công thức đúng $1 + 2x = 1 + 2 = 3$... paper thực ra dùng ví dụ số để chỉ ra đạo hàm **đúng** phải lớn hơn giá trị sai $2x = 2$ (ở $x = 1$), còn $\Delta y/\Delta x = 4$ là xấp xỉ sai phân hữu hạn (không bằng đúng đạo hàm giải tích $1 + 2x = 3$ tại điểm này, chỉ minh hoạ hướng đúng/sai, paper không khẳng định hai số này phải bằng nhau).

(c) Vector chain rule. Tổng quát hoá (b) sang vector: với $\mathbf{y} = \mathbf{f}(\mathbf{g}(\mathbf{x}))$,

$$\boxed{\frac{\partial}{\partial \mathbf{x}}\mathbf{f}(\mathbf{g}(\mathbf{x})) = \frac{\partial \mathbf{f}}{\partial \mathbf{g}}\frac{\partial \mathbf{g}}{\partial \mathbf{x}}}$$

là **tích hai Jacobian** (Jacobian $m \times k$ nhân Jacobian $k \times n$ ra Jacobian $m \times n$, với $m = |\mathbf{f}|, k = |\mathbf{g}|, n = |\mathbf{x}|$; thứ tự nhân quan trọng vì nhân ma trận không giao hoán). Paper chứng minh bằng ví dụ 2 hàm $f_1 = \ln(g_1)$, $f_2 = \sin(g_2)$ với $g_1 = x^2, g_2 = 3x$: tính tay từng phần tử theo quy tắc (b) rồi gom lại khớp đúng với công thức tích Jacobian. Khi các hàm thoả "diagonal condition" (mỗi f_i chỉ phụ thuộc g_i, mỗi g_i chỉ phụ thuộc x_i), vector chain rule suy biến thành diag $\left(\frac{\partial f_i}{\partial g_i}\frac{\partial g_i}{\partial x_i}\right)$, tức chain rule scalar áp từng phần tử.

Ba dạng chain rule này không phải ba công cụ tách biệt: (a) và (b) chỉ là trường hợp đặc biệt của (c) khi các đại lượng là scalar.

Ví dụ số "tự tính" minh hoạ (a): tại $x = 1$, công thức $\frac{dy}{dx} = 2x\cos(x^2)$ cho $\frac{dy}{dx}|_{x=1} = 2 \times 1 \times \cos(1) \approx 2 \times 0{,}5403 = 1{,}0806$ (tự tính; paper chỉ đưa công thức ký hiệu, không tính giá trị số cụ thể này).

3.7 Gradient của một neuron ReLU (Mục 5)

Với $z(\mathbf{w}, b, \mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b$ và activation$(\mathbf{x}) = \max(0, z)$:

$$\frac{\partial z}{\partial \mathbf{w}} = \mathbf{x}^T, \qquad \frac{\partial z}{\partial b} = 1$$

Đạo hàm của $\max(0, z)$ là hàm từng khúc: $\frac{\partial}{\partial z}\max(0, z) = 0$ nếu $z \le 0$, $= 1$ nếu $z > 0$. Dùng vector chain rule với biến trung gian z:

$$\frac{\partial\, \text{activation}}{\partial \mathbf{w}} = \begin{cases} \mathbf{0}^T & z \le 0 \\ \mathbf{x}^T & z > 0 \end{cases}, \qquad \frac{\partial\, \text{activation}}{\partial b} = \begin{cases} 0 & z \le 0 \\ 1 & z > 0 \end{cases}$$

3.8 Gradient của loss MSE theo $\mathbf{w}$ và b (Mục 6)

Với N mẫu, $C(\mathbf{w}, b, X, \mathbf{y}) = \frac{1}{N}\sum_{i=1}^{N}(y_i - \max(0, \mathbf{w}\cdot\mathbf{x}_i + b))^2$. Đặt $u =$ activation, $v = y - u$, $C = \frac{1}{N}\sum v^2$, áp dụng chain rule liên tiếp ($\frac{\partial C}{\partial v} = \frac{2}{N}\sum v$, rồi $\frac{\partial v}{\partial u} = -1$, rồi $\frac{\partial u}{\partial \mathbf{w}}$ như ở 3.7), đặt $e_i = \mathbf{w}\cdot\mathbf{x}_i + b - y_i$ (lỗi, dấu ngược với $v_i = y_i - u_i$):

$$\boxed{\frac{\partial C}{\partial \mathbf{w}} = \frac{2}{N}\sum_{i=1}^{N} e_i\,\mathbf{x}_i^T \;\;(\text{khi } z_i > 0), \qquad \frac{\partial C}{\partial b} = \frac{2}{N}\sum_{i=1}^{N} e_i \;\;(\text{khi } z_i > 0)}$$

(cả hai bằng $\mathbf{0}/0$ khi $z_i \le 0$ vì ReLU chặn gradient). Gradient là **trung bình có trọng số theo lỗi**: mẫu có lỗi lớn kéo gradient mạnh hơn theo hướng $\mathbf{x}_i$. Cập nhật gradient descent: $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta\frac{\partial C}{\partial \mathbf{w}}$, $b_{t+1} = b_t - \eta\frac{\partial C}{\partial b}$.

Ví dụ số "tự tính" minh hoạ công thức trên (paper không có ví dụ số kiểu này, đây là của người ghi chú để kiểm tra công thức): cho $N = 1$, $\mathbf{w} = [1, 2]$, $\mathbf{x} = [3, 4]$, $b = -5$, mục tiêu $y = 4$. Tính $z = \mathbf{w}\cdot\mathbf{x} + b = 1 \times 3 + 2 \times 4 - 5 = 3 + 8 - 5 = 6 > 0$, nên activation $= 6$ và $e = z - y = 6 - 4 = 2$. Áp công thức: $\frac{\partial C}{\partial \mathbf{w}} = 2 \times e \times \mathbf{x}^T = 2 \times 2 \times [3, 4] = [12, 16]$ (tự tính) và $\frac{\partial C}{\partial b} = 2 \times e = 4$ (tự tính). Với $\eta = 0{,}01$: $\mathbf{w}_{t+1} = [1, 2] - 0{,}01 \times [12, 16] = [0{,}88,\ 1{,}84]$, $b_{t+1} = -5 - 0{,}01 \times 4 = -5{,}04$ (tự tính), đúng hướng giảm z (và do đó giảm e^2) vì $e > 0$.

3.9 Kết hợp $\mathbf{w}$ và b

Paper gợi ý thủ thuật thực hành: ghép $\hat{\mathbf{w}} = [\mathbf{w}^T, b]^T$ và thêm 1 vào cuối $\mathbf{x}$: $\hat{\mathbf{x}} = [\mathbf{x}^T, 1]^T$, khi đó $\mathbf{w}\cdot\mathbf{x} + b = \hat{\mathbf{w}}\cdot\hat{\mathbf{x}}$, chỉ cần tính MỘT gradient thay vì hai (đây chính là "augmented weight vector", một mẹo thường thấy khi lập trình hồi quy tuyến tính).

4. Thí nghiệm và kết quả chính

Đây là paper tutorial về toán, **không có thí nghiệm/benchmark thực nghiệm** theo nghĩa huấn luyện mô hình hay so sánh số liệu trên dữ liệu thật. ”Kết quả chính” của bài là các công thức/định danh được dựng lại từ đầu và kiểm chứng chéo (bằng cách tính cả theo đường vector và theo đường quy về scalar, rồi so khớp):

Kết quả	Công thức
Jacobian hàm đồng nhất	$\frac{\partial \mathbf{x}}{\partial \mathbf{x}} = I$
Jacobian phép element-wise (diagonal condition)	$\text{diag}(\dots)$
Đạo hàm dot product	$\frac{\partial(\mathbf{w}\cdot\mathbf{x})}{\partial \mathbf{w}} = \mathbf{x}^T$, $\frac{\partial(\mathbf{w}\cdot\mathbf{x})}{\partial \mathbf{x}} = \mathbf{w}^T$
Vector chain rule	$\frac{\partial}{\partial \mathbf{x}}\mathbf{f}(\mathbf{g}(\mathbf{x})) = \frac{\partial \mathbf{f}}{\partial \mathbf{g}}\frac{\partial \mathbf{g}}{\partial \mathbf{x}}$
Gradient neuron ReLU + MSE	$\frac{\partial C}{\partial \mathbf{w}} = \frac{2}{N}\sum e_i \mathbf{x}_i^T$

Mỗi công thức đều được tác giả **kiểm hai lần**: một lần dựng bằng máy vector/matrix, một lần quy hoàn toàn về các phương trình scalar rồi lấy đạo hàm từng phần tử, và hai kết quả khớp nhau (ví dụ rõ nhất ở Mục 4.5.3 cho vector chain rule, và ở Mục 5 cho đạo hàm dot product).

5. Phân tích / ablation

- **Numerator layout vs denominator layout:** paper chọn numerator layout (hàng Jacobian = hàm, cột = biến); nhiều tài liệu/bài báo khác dùng denominator layout (transpose của numerator). Đây không phải một ”lựa chọn trung lập”; người đọc papers khác cần luôn kiểm layout trước khi so khớp công thức, nếu không shape sẽ không khớp.
- **Điều kiện áp dụng từng loại chain rule:** bài báo nhấn mạnh sai lầm phổ biến là áp dụng single-variable chain rule (a) khi thực ra cần total-derivative chain rule (b) vì có nhiều đường dữ liệu (ví dụ $y = x + x^2$). Việc luôn luôn dùng vector chain rule (c) tránh được lỗi này vì nó tự động ”cộng gộp” mọi đường qua phép nhân Jacobian.
- **Diagonal condition là chìa khoá tối ưu hoá tính toán:** phần lớn các phép toán trong neuron (element-wise +, -, $\otimes$) có Jacobian là ma trận đường chéo thưa, không cần lưu toàn bộ ma trận $n \times n$ dày; đây chính là lý do các framework autodiff không bao giờ vật lý hoá Jacobian đầy đủ mà làm việc với vector-Jacobian product (liên hệ bài sau, math 02).
- **Trường hợp ReLU bị ”tắt” ($z \leq 0$):** gradient của cả activation và loss triệt tiêu hoàn toàn. Đây chính là nguồn gốc hiện tượng ”dying ReLU” quen biết trong thực hành (không được paper này đặt tên, nhưng là hệ quả toán học trực tiếp của công thức đạo hàm từng khúc ở Mục 5).

6. Giới hạn

Do tác giả nêu:

- Phạm vi chỉ là đạo hàm theo **vector**, không phải theo **ma trận** đầy đủ (ví dụ đạo hàm của loss theo toàn bộ ma trận trọng số một layer, không phải theo một vector **w** của một neuron); tác giả nói rõ ở Mục 7 (Summary) đây là ”bước tiếp theo” để người đọc tự học thêm, không nằm trong bài.
- Automatic differentiation (forward/backward mode cho cả một đồ thị tính toán lớn) được coi là ”beyond the scope of this article, but we’re setting the stage for a future article” (Mục 4.5.1): tức paper chỉ dừng ở quy tắc đạo hàm toán học tay, không trình bày thuật toán AD thực thụ (đó là nội dung bài 02).
- Ví dụ xuyên suốt chỉ là MỘT neuron fully-connected + ReLU + MSE, chưa mở rộng cho nhiều layer, hàm activation khác, hay loss khác (ví dụ cross-entropy/softmax).

Người ghi chú nhận xét:

- Lối trình bày ”quy về scalar rồi suy ra công thức vector” rất dễ hiểu cho người mới nhưng **không phải chứng minh toán học chặt** theo chuẩn mực của một bài báo lý thuyết; mục tiêu là sư phạm (pedagogical), không phải rigor.
- Vì chỉ xử lý một neuron/một layer, công thức Mục 6 không trực tiếp cho thấy vì sao backpropagation qua nhiều layer lại hiệu quả hơn forward-mode AD (đó là lý do thực sự reverse-mode được ưa dùng trong deep learning, chỉ được nêu sơ lược ở cuối Mục 4.5.1 và sẽ được bài 02 làm rõ).
- Có một điểm không nhất quán nhỏ về ngày tháng đã nêu ở header (trang đầu ”July 3, 2018” so với stamp arXiv ”02/07/2018”), không ảnh hưởng nội dung toán.

7. Liên hệ với các bài khác

- **math 02** (Automatic Differentiation in Machine Learning: a Survey): dùng trực tiếp vector chain rule/Jacobian của bài này để định nghĩa chính xác forward-mode (tương ứng $\frac{du}{dx}\frac{dy}{du}$, nhân ”từ trái”, theo hướng $x \to y$) và reverse-mode (tương ứng $\frac{dy}{du}\frac{du}{dx}$, nhân ”từ phải”, theo hướng $y \to x$), paper 01 đã gợi mở chính hai hướng nhân này ở cuối Mục 4.5.1 nhưng không đi sâu.
- **math 23 (Neural Tangent Kernel), math 24 (Lazy Training):** cả hai phân tích mạng neural qua tuyến tính hoá bậc nhất (Jacobian của hàm mạng theo tham số tại điểm khởi tạo), chính là Jacobian $\frac{\partial \mathbf{f}}{\partial \mathbf{x}}$ tổng quát hoá từ công thức neuron đơn ở bài này.
- **pruning content/07_2019_importance_estimation_taylor.md:** chấm ”độ quan trọng” của một tham số/kênh khi cắt tỉa bằng khai triển Taylor bậc 1 (dùng gradient) hoặc bậc 2 (dùng tích gradient, xấp xỉ Hessian) của loss; đây chính là ứng dụng trực tiếp của máy đạo hàm dựng ở bài này cho một mục đích khác (ước lượng thay đổi loss khi xoá một tham số) thay vì cập nhật gradient descent.
- **interp content/02_2021_mathematical_framework.md** (A Mathematical Framework for Transformer Circuits): phân tích attention bằng cách phân rã các phép nhân ma trận tuyến tính thành tích các Jacobian/ma trận con, cùng tinh thần ”theo dõi shape và phép nhân ma trận cẩn thận” mà bài 01 rèn luyện.

8. Gợi ý áp dụng

Quy tắc chuỗi và quy ước chiều của gradient hữu ích khi kiểm tra một phép backward tự viết. Với hàm mất mát vô hướng và nhiều tham số, reverse-mode thường là lựa chọn phù hợp; bài 02 giải thích chi tiết chi phí của hai chế độ vi phân tự động.

9. Thí nghiệm gợi ý

1. **Tự viết backward cho một neuron ReLU + MSE (như Mục 5-6 của paper) bằng NumPy, so với `torch.autograd`:** tạo $\mathbf{w}, b, \mathbf{x}, y$ ngẫu nhiên nhỏ, tính `activation` và `loss` tay, tính $\frac{\partial C}{\partial \mathbf{w}}, \frac{\partial C}{\partial b}$ theo công thức 3.8, so với gradient PyTorch tính qua `loss.backward()`. Kỳ vọng: sai số tuyệt đối ở mức 10^{-6} hoặc nhỏ hơn (float32); nếu lệch nhiều, kiểm tra lại dấu hoặc điều kiện ReLU ($z \leq 0$).
2. **Kiểm tra numerator vs denominator layout bằng code:** tính Jacobian của một hàm vector nhỏ $\mathbf{f} : \mathbb{R}^3 \to \mathbb{R}^2$ bằng `torch.autograd.functional.jacobian`, so sánh shape trả về với quy ước numerator layout của bài; diễn giải tại sao cần transpose nếu dùng quy ước khác.
3. **Minh hoạ ”diagonal condition”:** lấy một phép toán element-wise (ví dụ tanh áp lên vector) qua `torch.autograd.functional.jacobian`, in toàn bộ ma trận Jacobian $n \times n$ và kiểm tra trực quan nó đúng là ma trận đường chéo (các phần tử ngoài đường chéo bằng 0 trong giới hạn sai số số học).
4. **So sánh chi phí reverse-mode vs forward-mode tay:** với một hàm tổng hợp nhiều lớp phi tuyến ($f_4(f_3(f_2(f_1(x))))$ như ví dụ Mục 4.5.1 của paper), đo thời gian tính gradient bằng `torch.autograd.functional.jvp` (forward-mode, kiểu hình minh hoạ ”forward differentiation” của bài) so với `vjp`/`backward` (reverse-mode) khi tăng số chiều input, giữ output là scalar; kỳ vọng reverse-mode nhanh hơn rõ rệt khi input lớn, khớp với trực giác bài 01 đã gợi mở.

5. **Khảo sát vùng "chết" của ReLU trên mô hình nhỏ thật:** lấy 1 layer Linear+ReLU nhỏ của một mô hình toy, chạy forward trên một batch dữ liệu, đo tỉ lệ neuron có $z \leq 0$ (gradient = 0 theo công thức 3.7) và quan sát tỉ lệ này thay đổi thế nào khi khởi tạo trọng số khác đi.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức
Gradient	$\nabla f(\mathbf{x}) = \left[\frac{\partial f}{\partial x_1}, \ldots, \frac{\partial f}{\partial x_n}\right]$
Jacobian (numerator layout)	$\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$, hàng i = $\nabla f_i(\mathbf{x})$
Element-wise diagonal condition	$\frac{\partial \mathbf{y}}{\partial \mathbf{w}} = \text{diag}\left(\frac{\partial}{\partial w_i}(f_i(w_i) \odot g_i(x_i))\right)$
Đạo hàm dot product	$\frac{\partial(\mathbf{w}\cdot\mathbf{x})}{\partial \mathbf{w}} = \mathbf{x}^T$, $\frac{\partial(\mathbf{w}\cdot\mathbf{x})}{\partial \mathbf{x}} = \mathbf{w}^T$
Single-variable chain rule	$\frac{dy}{dx} = \frac{dy}{du}\frac{du}{dx}$ (1 đường duy nhất)
Total-derivative chain rule	$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial x} + \sum_i \frac{\partial f}{\partial u_i}\frac{\partial u_i}{\partial x}$
Vector chain rule (tổng quát nhất)	$\frac{\partial}{\partial \mathbf{x}}\mathbf{f}(\mathbf{g}(\mathbf{x})) = \frac{\partial \mathbf{f}}{\partial \mathbf{g}}\frac{\partial \mathbf{g}}{\partial \mathbf{x}}$
Gradient neuron ReLU	$\frac{\partial \text{ activation}}{\partial \mathbf{w}} = \mathbf{x}^T$ nếu $z > 0$, $\mathbf{0}^T$ nếu $z \leq 0$
Gradient loss MSE (ReLU)	$\frac{\partial C}{\partial \mathbf{w}} = \frac{2}{N}\sum_i e_i \mathbf{x}_i^T$, $e_i = \mathbf{w}\cdot\mathbf{x}_i + b - y_i$
Cập nhật gradient descent	$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta\frac{\partial C}{\partial \mathbf{w}}$

11. Kết luận ngắn

Bài này dựng lại từ đầu, bằng cách quy mọi biểu thức vector/matrix về các phương trình scalar rồi gom lại, các công cụ đạo hàm cần cho deep learning: gradient, Jacobian (numerator layout), điều kiện để Jacobian suy biến thành ma trận đường chéo, và ba dạng chain rule mà tất cả đều chỉ là trường hợp đặc biệt của **vector chain rule** $\frac{\partial}{\partial \mathbf{x}}\mathbf{f}(\mathbf{g}(\mathbf{x})) = \frac{\partial \mathbf{f}}{\partial \mathbf{g}}\frac{\partial \mathbf{g}}{\partial \mathbf{x}}$. Áp dụng toàn bộ máy đó cho một neuron ReLU và loss MSE cho ra đúng công thức gradient descent quen thuộc. Đây là nền toán thuần tuý (không thí nghiệm, không benchmark), chuẩn bị trực tiếp cho bài sau (math 02) trình bày automatic differentiation như một thuật toán tổng quát hoá chính quy tắc chuỗi này cho một đồ thị tính toán lớn bất kỳ, thay vì tính tay cho từng kiến trúc.

Nguồn gốc: https://arxiv.org/abs/1802.01528

Bài 02

Automatic Differentiation in Machine Learning: a Survey (2015)

Tác giả: Atılım Güneş Baydin (University of Oxford), Barak A. Pearlmutter (National University of Ireland Maynooth), Alexey Andreyevich Radul (Massachusetts Institute of Technology), Jeffrey Mark Siskind (Purdue University).
Venue / arXiv: arXiv:1502.05767 (v4, 05/02/2018); paper không nêu venue hội nghị/tạp chí trong văn bản (bản arXiv này không có dòng tạp chí; đã kiểm kỹ toàn văn, không có self-citation venue).
Nguồn: https://arxiv.org/abs/1502.05767
Chủ đề: khảo sát automatic differentiation (AD) từ góc nhìn machine learning: phân biệt AD với đạo hàm tay/số/ký hiệu, hai chế độ forward và reverse (reverse mode chính là tổng quát hoá của backpropagation), ứng dụng trong optimization/neural network/computer vision/NLP/suy luận xác suất, và các cách cài đặt (elemental library, source transformation, operator overloading).
Vị trí trong lộ trình: đọc sau math 01 (Matrix Calculus), dùng trực tiếp vector chain rule/Jacobian của bài đó để định nghĩa chính xác forward-mode và reverse-mode AD; đọc trước math 03.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (bản arXiv); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc.

1. Bối cảnh

Có nhiều cách tính đạo hàm trong chương trình: suy công thức bằng tay rồi cài đặt, dùng sai phân hữu hạn hoặc dùng một hệ thống biến đổi ký hiệu như Mathematica hay Maple. Cách đầu dễ mắc lỗi; sai phân hữu hạn là phép xấp xỉ và cần $O(n)$ lần gọi hàm để tính gradient n chiều; đạo hàm ký hiệu có thể làm biểu thức phình to qua các lần áp dụng quy tắc tích. Bài khảo sát vi phân tự động (automatic differentiation, AD) như một cách tiếp cận khác trong học máy.

Lịch sử ngắn (Mục 3.3): ý tưởng AD có từ thập niên 1950; forward mode được Wengert (1964) phát hiện như một phương pháp tổng quát; reverse mode có nguồn gốc từ Pontryagin maximum principle (lý thuyết điều khiển, cuối 1950), được Linnainmaa (1970, 1976) công bố sớm, Werbos (1974) hình thức hoá, và Speelpenning (1980) đưa ra cài đặt **tự động hoàn toàn** đầu tiên. Trong machine learning, backpropagation (trường hợp đặc biệt của reverse-mode AD) bị ”phát hiện lại” nhiều lần độc lập (Parker 1985, Werbos, nhóm Parallel Distributed Processing) trước khi Rumelhart và đồng nghiệp (1986) làm nó nổi tiếng. Bài báo nhấn mạnh: hiểu toán học sau backprop là đủ nền để hiểu AD nói chung, vì backprop **chỉ là một trường hợp** của reverse-mode AD áp cho hàm mạng neural ghép với loss.

Một nguồn nhầm lẫn paper muốn giải quyết: nhiều người gọi bất cứ thứ gì không phải đạo hàm tay là ”automatic differentiation”, trong khi AD là một **kỹ thuật cụ thể**: tính giá trị số của đạo hàm (không phải biểu thức đạo hàm) bằng cách ”tích luỹ” (accumulate) giá trị trong lúc chạy code, khác hẳn cả numerical diff (xấp xỉ) và symbolic diff (biến đổi biểu thức).

2. Ý tưởng chính

AD phân tích phép tính thành các phép toán sơ cấp có đạo hàm đã biết, như cộng, nhân, hàm lượng giác, log và exp. Trong cách diễn giải này, quy tắc chuỗi được áp dụng qua các phép toán đã thực thi để lan truyền giá trị đạo hàm. Dãy phép toán và giá trị tương ứng được gọi là evaluation trace hay Wengert list.

Vì làm việc trên **evaluation trace cụ thể** (không phải biểu thức closed-form trừu tượng), AD có 2 lợi thế lớn: (a) **chính xác tới độ chính xác máy** (không phải xấp xỉ như numerical diff) chỉ với **chi phí hằng số** so với hàm gốc (không bị expression swell như symbolic diff); (b) **hoạt động với code có control flow** (if, loop, recursion) vì bất kể đường thực thi nào được chọn, kết quả cuối vẫn chỉ là một trace số cụ thể để chain rule áp lên, điều symbolic differentiation không làm được tốt (phải biết trước biểu thức đóng).

Có **hai chế độ** AD, khác nhau về **hướng** áp chain rule: **forward mode** (lan truyền đạo hàm theo cùng hướng dữ liệu, từ input ra output) và **reverse mode** (lan truyền ngược, từ output về input); reverse mode chính là backpropagation tổng quát hoá cho MỌI chương trình số, không chỉ cho mạng neural.

3. Phương pháp chi tiết

3.1 AD không phải numerical hay symbolic differentiation (Mục 2)

Numerical differentiation dùng xấp xỉ giới hạn:

$$\frac{\partial f(\mathbf{x})}{\partial x_i} \approx \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x})}{h} \tag{1}$$

với $\mathbf{e}_i$ là vector đơn vị thứ i, $h > 0$ nhỏ. Có hai loại lỗi đối nghịch nhau khi chọn h: **truncation error** (lỗi do h không thực sự là 0, giảm khi $h \to 0$) và **round-off error** (lỗi do giới hạn độ chính xác dấu phẩy động, tăng khi h quá nhỏ); Hình 3 của paper minh họa hai đường lỗi này cắt nhau. Dạng center difference giảm bớt lỗi bậc nhất:

$$\frac{\partial f(\mathbf{x})}{\partial x_i} = \frac{f(\mathbf{x} + h\mathbf{e}_i) - f(\mathbf{x} - h\mathbf{e}_i)}{2h} + O(h^2) \tag{2}$$

nhưng vẫn cần $O(n)$ lần gọi hàm cho gradient n chiều (Jacobian $m \times n$ cần $2mn$ lần gọi), không khả thi khi n lên tới hàng triệu/tỷ tham số như deep learning hiện đại.

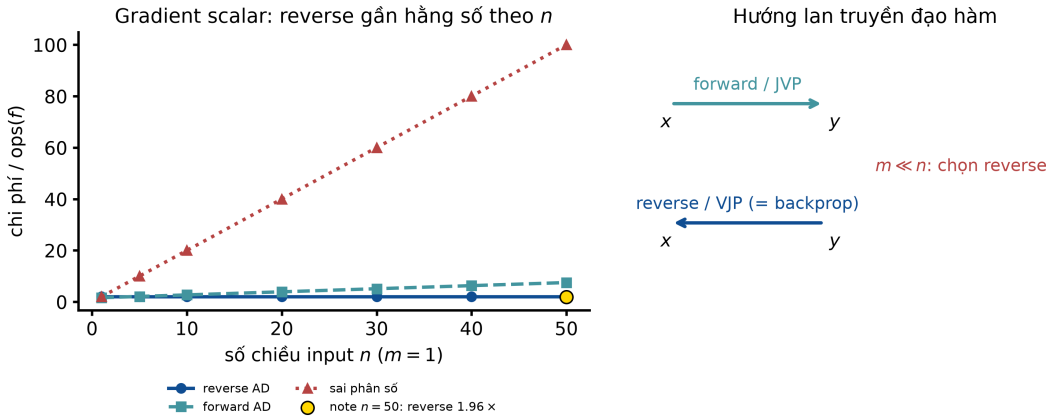
Symbolic differentiation áp quy tắc biến đổi biểu thức (ví dụ quy tắc tích, công thức 3) một cách máy móc. Vấn đề: nếu $h(x) = f(x)g(x)$ có phần chung giữa h và h' (là f, g), việc thế trực tiếp đạo hàm con vào công thức sẽ **nhân đôi lặp lại** các biểu thức con, dẫn tới "expression swell". Paper dùng ví dụ logistic map $l_{n+1} = 4l_n(1 - l_n)$ (Bảng 1): đạo hàm của l_4 theo x nếu không rút gọn có độ dài tăng vượt xa cả hàm gốc qua mỗi bước lặp.

3.2 Evaluation trace / Wengert list (Mục 3)

AD biểu diễn một hàm $f : \mathbb{R}^n \to \mathbb{R}^m$ bằng ba nhóm biến trung gian theo ký hiệu Griewank & Walther: $v_{i-n} = x_i$ ($i = 1, \ldots, n$, biến input), v_i ($i = 1, \ldots, l$, biến trung gian), $y_{m-i} = v_{l-i}$ (biến output). Ví dụ xuyên suốt bài (Hình 4, Bảng 2-3): $y = f(x_1, x_2) = \ln(x_1) + x_1 x_2 - \sin(x_2)$.

3.3 Forward mode (Mục 3.1)

Gắn mỗi biến trung gian v_i một "tangent" $\dot{v}_i = \frac{\partial v_i}{\partial x_1}$ (đạo hàm theo MỘT input cố định), rồi chạy chain rule trên từng phép toán sơ cấp để tạo "tangent trace" đi **song song, cùng hướng** với trace gốc (primal trace).



Chi phí gradient scalar: reverse AD gần hằng số theo n (note n=50 ≈1.96×f); forward/sai phân tăng theo n. (minh hoạ, tự tính)

Ví dụ paper (Bảng 2), tại $(x_1, x_2) = (2, 5)$, seed $\dot{x}_1 = 1, \dot{x}_2 = 0$ (tính $\partial y/\partial x_1$):

Primal	Giá trị	Tangent	Giá trị
$v_{-1} = x_1$	2	$\dot{v}_{-1} = \dot{x}_1$	1
$v_0 = x_2$	5	$\dot{v}_0 = \dot{x}_2$	0
$v_1 = \ln v_{-1}$	$\ln 2$	$\dot{v}_1 = \dot{v}_{-1}/v_{-1}$	$1/2$
$v_2 = v_{-1} \times v_0$	10	$\dot{v}_2 = \dot{v}_{-1}v_0 + \dot{v}_0 v_{-1}$	5
$v_3 = \sin v_0$	$\sin 5 \approx -0{,}959$	$\dot{v}_3 = \dot{v}_0 \cos v_0$	0
$v_4 = v_1 + v_2$	$0{,}693 + 10$	$\dot{v}_4 = \dot{v}_1 + \dot{v}_2$	$5{,}5$
$v_5 = v_4 - v_3$	$10{,}693 + 0{,}959 = 11{,}652$	$\dot{v}_5 = \dot{v}_4 - \dot{v}_3$	$5{,}5$
$y = v_5$	$11{,}652$	$\dot{y}$	$5{,}5$

Một lần "forward pass" chỉ cho **một cột** của Jacobian (đạo hàm theo MỘT input, với MỌI output). Forward mode thuận lợi khi $f : \mathbb{R} \to \mathbb{R}^m$ (một pass lấy hết); khi $f : \mathbb{R}^n \to \mathbb{R}$ (một output, nhiều input như loss trong ML) cần n **lần pass** để có đủ gradient $\nabla f = [\partial y/\partial x_1, \ldots, \partial y/\partial x_n]$, đây là điểm bất lợi chính của forward mode cho ML. Forward mode cũng cho phép tính **Jacobian-vector product** $J_f \mathbf{r}$ (công thức 4) chỉ bằng MỘT pass, bằng cách seed $\dot{\mathbf{x}} = \mathbf{r}$; trường hợp đặc biệt $f : \mathbb{R}^n \to \mathbb{R}$ cho đạo hàm theo hướng $\nabla f \cdot \mathbf{r}$.

Dual numbers (Mục 3.1.1): về mặt toán, forward mode tương đương tính hàm trên **số đôi** (dual number) $v + \dot{v}\epsilon$ với $\epsilon^2 = 0, \epsilon \neq 0$ (chuỗi Taylor cắt bậc 1). Quy tắc cộng/nhân trên số đôi tự động khớp quy tắc đạo hàm: $(v + \dot{v}\epsilon)(u + \dot{u}\epsilon) = (vu) + (v\dot{u} + \dot{v}u)\epsilon$ (đúng là quy tắc tích). Hệ số của ϵ sau khi áp hàm f trên $v + 1\epsilon$ chính là $f'(v)$ (công thức 5); chain rule hoạt động tự nhiên qua 2 lần áp công thức này.

3.4 Reverse mode (Mục 3.2)

Gắn mỗi biến trung gian một "adjoint" $\bar{v}_i = \frac{\partial y_j}{\partial v_i}$ (độ nhạy của MỘT output y_j theo mọi biến trung gian). Quy trình 2 giai đoạn: (1) chạy forward, lưu lại trace và giá trị các v_i (bookkeeping); (2) chạy backward từ output ($\bar{v}_{\text{output}} = \bar{y} = \partial y/\partial y = 1$), lan truyền ngược $\bar{v}_i$ theo total-derivative chain rule (xem math 01, mục 3.6b): mỗi biến nhận đóng góp từ TẤT CẢ biến "con" nó ảnh hưởng tới.

Cùng ví dụ, cùng điểm $(2, 5)$ (Bảng 3), MỘT lần reverse pass cho CẢ HAI đạo hàm cùng lúc:

$$\bar{x}_1 = \frac{\partial y}{\partial x_1} = 5{,}5, \qquad \bar{x}_2 = \frac{\partial y}{\partial x_2} = 1{,}716$$

Kiểm bằng tay (tự tính, đối chiếu công thức giải tích trực tiếp của $y = \ln(x_1) + x_1x_2 - \sin(x_2)$): $\frac{\partial y}{\partial x_1} = \frac{1}{x_1} + x_2 = \frac{1}{2} + 5 = 5{,}5$ (khớp $\bar{x}_1$); $\frac{\partial y}{\partial x_2} = x_1 - \cos(x_2) = 2 - \cos(5) \approx 2 - 0{,}2837 = 1{,}7163$ (khớp $\bar{x}_2 = 1{,}716$, sai số làm tròn). Hai cách tính (đạo hàm giải tích trực tiếp và reverse-mode AD theo Bảng 3 của paper) cho cùng kết quả, đúng như lý thuyết chain rule đảm bảo.

Chi phí (Mục 3.2): với $f : \mathbb{R}^n \to \mathbb{R}^m$, gọi $\text{ops}(f)$ là số phép toán để tính f một lần. Forward mode tính Jacobian $m \times n$ trong thời gian $n\,c\,\text{ops}(f)$; reverse mode trong thời gian $m\,c\,\text{ops}(f)$, với hằng số $c < 6$, thường $c \in [2, 3]$ (dẫn theo Griewank & Walther 2008, không phải tự chứng minh trong bài này). Khi $m \ll n$ (loss scalar, triệu tham số, đúng tình huống ML), **reverse mode thắng rõ**: chỉ MỘT pass cho cả gradient n chiều, đây chính là lý do backpropagation (= reverse-mode AD cho mạng neural ghép loss) là kỹ thuật chủ đạo trong deep learning, không phải forward mode.

Đánh đổi: reverse mode cần lưu lại toàn bộ trace trung gian (tape) trong bộ nhớ, tăng theo số phép toán của hàm, đây là bất lợi chính so với forward mode (gần như không cần bộ nhớ ngoài). Paper dẫn ví dụ cụ thể từ nghiên cứu khác (Gruslys và đồng nghiệp 2016, không phải thí nghiệm của chính tác giả bài này): kỹ thuật checkpointing cho backpropagation-through-time (BPTT) trên mạng hồi quy, tiết kiệm **tới 95% bộ nhớ, đổi lại tăng 33% thời gian tính toán** trong một trường hợp cụ thể họ báo cáo.

3.5 Hessian-vector product không cần lưu toàn bộ Hessian (Mục 4.1)

Kỹ thuật ”reverse-on-forward” (Pearlmutter 1994, được paper dẫn lại): với $f : \mathbb{R}^n \to \mathbb{R}$, điểm $\mathbf{x}$ và vector $\mathbf{v}$, trước tiên dùng **forward mode** để tính đạo hàm theo hướng $\nabla f \cdot \mathbf{v}$ (seed $\dot{\mathbf{x}} = \mathbf{v}$), sau đó áp **reverse mode** lên kết quả đó để được $\nabla^2 f \cdot \mathbf{v} = H_f\mathbf{v}$. Toàn bộ chỉ tốn $O(n)$ (không phải $O(n^2)$ như vật lý hoá Hessian đầy đủ rồi nhân vector), dù H_f là ma trận $n \times n$. Đây là cơ sở cho các phương pháp tối ưu bậc hai xấp xỉ (quasi-Newton, stochastic Newton) hoạt động được ở quy mô lớn.

3.6 Ứng dụng trong machine learning (Mục 4)

Paper liệt kê 5 nhóm ứng dụng, mỗi nhóm kèm trích dẫn công trình cụ thể (không phải thí nghiệm riêng của bài): (i) **gradient-based optimization** nói chung (SGD, Newton/quasi-Newton dùng Hessian-vector product ở 3.5); (ii) **neural network/deep learning/differentiable programming**, nhấn mạnh backprop chỉ là ”trường hợp đặc biệt của AD” và framework hiện đại (autograd, Chainer, PyTorch) cung cấp AD tổng quát ”define-by-run” (dynamic computational graph) thay cho kiểu ”define-and-run” (static graph, như Theano/TensorFlow thời đó); (iii) **computer vision** (inverse graphics/render vi phân, bundle adjustment, đăng ký ảnh y tế: dẫn một kết quả từ nghiên cứu khác báo cáo **tăng tốc 6 lần** so với numerical diff center-difference, không phải số của chính bài này); (iv) **NLP** (parsing, sentence tracking); (v) **suy luận xác suất** (HMC/NUTS trong Stan, variational inference, reparameterization trick cho biến liên tục, REINFORCE/REBAR/RELAX cho biến rời rạc).

3.7 Các cách cài đặt (Mục 5)

Ba nhóm chính: **(a) Elemental libraries** (người code tự gọi hàm thư viện AD cho từng phép sơ cấp, đơn giản nhất nhưng thủ công nhất); **(b) Source code transformation/compiler** (tiền xử lý code nguồn để sinh code đạo hàm, ví dụ ADIFOR, Tapenade, Tangent cho Python/Numpy); **(c) Operator overloading** (định nghĩa lại các phép toán cơ bản trên kiểu dữ liệu AD-aware, cách phổ biến nhất trong ML hiện đại: ADOL-C dùng ”tape” ghi lại phép toán để ”chạy lại ngược”, PyTorch/Chainer/autograd theo hướng này). Paper cảnh báo các chi phí/rủi ro thực hành: cấp phát bộ nhớ cho số đôi tốn kém nếu làm ẩu, method dispatch của operator overloading có thể chậm cả bậc độ lớn so với tính toán số thuần, và lớp lỗi **”perturbation confusion”**: khi hai lượt AD lồng nhau tác động cùng đoạn code, hai ϵ hình thức phải được giữ phân biệt, dễ lẫn nếu cài không cẩn thận.

Lưu ý về xấp xỉ số (Mục 5): AD lấy đúng đạo hàm của **chương trình đã viết**, không phải đạo hàm ”lý tưởng” của hàm toán mà chương trình xấp xỉ. Ví dụ paper nêu: nếu e^x được tính bằng một công thức xấp xỉ từng khúc (piecewise-rational), AD trên công thức đó cho đạo hàm của XẤP XỈ đó, có thể kém hơn chính xấp xỉ

gốc của $\frac{de^x}{dx} = e^x$ (paper ghi chú trong thực hành hiện đại điều này không còn là vấn đề vì e^x thường là hàm nguyên thuỷ cứng trong phần cứng).

4. Thí nghiệm và kết quả chính

Đây là paper **survey**, nhưng khác math 01/03, bài này CÓ MỘT thí nghiệm định lượng thật của chính tác giả (Mục 4.1, Bảng 4 và Hình 5): đo thời gian tính hàm năng lượng tự do Helmholtz (dựa trên phương trình trạng thái Peng-Robinson, có n biến độc lập) và gradient của nó bằng numerical diff (center difference), forward AD, và reverse AD, với n từ 1 đến 50, trên máy Intel Core i7-4785T 2.20GHz, 16GB RAM, dùng công cụ DiffSharp 0.5.7, lấy trung bình 1000 lần chạy. Thời gian hàm gốc tại $n = 1$ là $0{,}0023$ ms (mốc chuẩn hoá).

n	f gốc (so n=1)	∇f numerical (so n=1)	∇f forward AD (so n=1)	∇f reverse AD (so n=1)
1	1	1,08	1,34	1,52
8	5,12	35,55	13,69	11,12
15	14,51	176,79	51,54	31,37
22	29,11	499,43	132,33	67,27
29	52,58	1045,29	251,32	113,99
36	84,00	1986,70	469,84	174,62
43	127,33	3269,36	815,55	254,15
50	174,44	4995,96	1342,07	342,33

Paper còn cho cột ”so với f cùng n” (chi phí tương đối của gradient so với hàm gốc CÙNG kích thước): tại $n = 50$, numerical diff tốn $28{,}63\times$ chi phí của $f(n = 50)$, forward AD tốn $7{,}69\times$, còn **reverse AD chỉ tốn** $1{,}96\times$ và tỉ số này gần như không đổi khi n tăng (từ $1{,}52\times$ ở $n = 1$ xuống dao động quanh $2\times$), đúng khớp lý thuyết $m\,c\,\mathrm{ops}(f)$ với $m = 1$ (gradient scalar) không phụ thuộc n. Đây là bằng chứng số cụ thể cho tuyên bố ”reverse mode AD gần như không đắt hơn hàm gốc là bao, bất kể số chiều input”, trong khi numerical diff và cả forward mode đều tăng chi phí tuyến tính theo n.

Các số liệu khác trong Mục 4 (tăng tốc 6 lần của Grabner và đồng nghiệp, tiết kiệm 95% bộ nhớ của Gruslys và đồng nghiệp) là **trích dẫn từ công trình khác**, không phải thí nghiệm của chính 4 tác giả bài survey này; paper chỉ tổng hợp lại làm minh chứng cho luận điểm.

5. Phân tích / ablation

- **Forward vs reverse, phụ thuộc hình dạng n so với m:** bảng ở Mục 4 cho thấy rõ cả numerical, forward, và reverse đều tăng theo n về giá trị tuyệt đối (vì hàm Helmholtz có $m = n$ do gradient có n thành phần), nhưng tốc độ tăng khác hẳn: numerical tăng gần $O(n^2)$ hiệu dụng khi tính đủ gradient (vì mỗi thành phần cần 1-2 lần gọi f có chi phí tăng theo n), forward AD tăng gần tuyến tính theo n do cần n pass, còn reverse AD gần hằng số tương đối so với f cùng kích thước.
- **Đánh đổi bộ nhớ-thời gian của reverse mode:** là trục đánh đổi DUY NHẤT mà reverse mode phải trả so với forward mode (vốn gần như không cần nhớ thêm); checkpointing là kỹ thuật chính để kiểm soát đánh đổi này, đặc biệt quan trọng trên GPU bộ nhớ hạn chế.
- **AD không ”sửa” vấn đề số học:** paper chỉ rõ tổng nhiều số lớn gần triệt tiêu nhau (ví dụ $\nabla_w E = \sum_i \nabla_w E_i \to 0$ khi gần cực tiểu) vẫn gặp đúng rủi ro cộng số dấu phẩy động dù tính bằng AD; AD chỉ đảm bảo đúng đạo hàm của code đã viết, không đảm bảo code đó ổn định số học.
- **Khác biệt workload ML và workload AD truyền thống:** paper nhận xét ML hiện đại compute-bound (dồn vào kernel ma trận tối ưu cao), phù hợp operator-overloading trên phép toán mức cao; mô phỏng khoa học truyền thống (nguồn gốc lịch sử của AD) thường bandwidth-bound, nên hợp với source-code

transformation/compiler hơn. Đây là lý do operator overloading (PyTorch-style) thắng thế trong ML dù cộng đồng AD truyền thống từng ưu tiên source transformation.

6. Giới hạn

Do tác giả nêu:

- Các vấn đề số học sâu (subgradient cho hàm không khả vi, xấp xỉ gần 0 cho $|\cdot|, \|\cdot\|_2, \sqrt{\cdot}$) được nói rõ là ”ngoài phạm vi hiện tại” (Mục 5), chỉ trỏ tới tài liệu khác.
- Hằng số chi phí $c < 6$, $c \in [2, 3]$ của reverse/forward mode là dẫn lại từ Griewank & Walther (2008), không phải kết quả tự chứng minh của 4 tác giả.
- Nested/higher-order AD (cần cho hypergradient, tối ưu siêu tham số, HMC Riemannian) được xếp vào ”hướng nghiên cứu tương lai” (Mục 6 Kết luận), chưa trình bày chi tiết kỹ thuật trong bài.
- Bài khảo sát các framework tại thời điểm viết (Theano, TensorFlow tĩnh, autograd, Chainer, PyTorch bản đầu); không thể phản ánh các phát triển AD sau 2018 (ví dụ JAX, chế độ compile-and-trace hiện đại).

Người ghi chú nhận xét:

- Thí nghiệm định lượng DUY NHẤT của bài (Bảng 4/Hình 5) dùng một hàm năng lượng tự do kiểu khoa học tính toán (scientific computing), KHÔNG phải một mạng neural/deep learning workload thật; với một paper ”khảo sát AD cho machine learning”, việc thiếu một benchmark trực tiếp trên kiến trúc neural (ví dụ MLP/CNN nhỏ) để đo chi phí AD trong bối cảnh ML cụ thể là một khoảng trống đáng chú ý, dù không ảnh hưởng tới tính đúng đắn lý thuyết được trình bày.
- Các số liệu tăng tốc/giảm bộ nhớ dẫn từ công trình khác (6 lần, 95%) chỉ là MỘT trường hợp cụ thể được báo cáo trong bài gốc đó, không phải mức cải thiện trung bình hay đảm bảo chung; cần cẩn trọng khi tổng quát hoá.

7. Liên hệ với các bài khác

- **math 01** (The Matrix Calculus You Need For Deep Learning): cung cấp nền chain rule/Jacobian mà bài này dùng trực tiếp; forward mode AD tương ứng cách nhân Jacobian ”từ trong ra ngoài” theo hướng dữ liệu, reverse mode tương ứng cách nhân ”từ ngoài vào trong” mà math 01 đã gợi mở ở cuối Mục 4.5.1 của bài đó nhưng không khai triển thành thuật toán tổng quát.
- **math 03** (A Tutorial on Principal Component Analysis): bài tiếp theo trong batch, không phụ thuộc trực tiếp vào AD nhưng cùng mạch ”công cụ toán nền cho ML” của coordinator.
- **math 23 (Neural Tangent Kernel), math 24 (Lazy Training):** cả hai phân tích mạng neural qua Jacobian của hàm mạng theo tham số; trong thực hành, Jacobian đó luôn được TÍNH bằng chính kỹ thuật AD (forward-mode hoặc Jacobian-vector product) mô tả ở Mục 3.3-3.5 của bài này, dù hai bài đó tập trung vào hệ quả lý thuyết hơn là cách tính.
- **math 25** (The Principles of Deep Learning Theory): các phân tích lý thuyết sâu về mạng neural ở quy mô lớn đều mặc định có sẵn gradient/Hessian chính xác tới máy nhờ AD, đây là tiền đề thực hành mà bài 02 giải thích vì sao khả thi ở quy mô triệu/tỷ tham số.

8. Gợi ý áp dụng

Reverse-mode phù hợp khi hàm có nhiều đầu vào và ít đầu ra, chẳng hạn một loss vô hướng. Checkpointing đánh đổi thêm phép tính để giảm bộ nhớ lưu giá trị trung gian; tích Hessian–vector cho phép khảo sát độ cong mà không cần lập toàn bộ ma trận Hessian.

9. Thí nghiệm gợi ý

1. **Tái lập mini-bản của Bảng 4/Hình 5:** viết một hàm toy nhiều biến (ví dụ tổng vài hạng log/mũ của n biến, n từ 1 đến 50), đo thời gian numerical diff (center difference, NumPy) so với `torch.autograd.backward` (reverse) và `torch.func.jvp` (forward) trên CPU/GPU nhỏ; kỳ vọng tái hiện đúng xu hướng: reverse AD gần hằng số tương đối so với f, numerical/forward tăng theo n.
2. **Kiểm lại ví dụ số của paper bằng code:** cài $y = \ln(x_1) + x_1 x_2 - \sin(x_2)$ trong PyTorch với $x_1 = 2, x_2 = 5$ (`requires_grad=True`), gọi `.backward()`, so `x1.grad, x2.grad` với $\bar{x}_1 = 5{,}5, \bar{x}_2 \approx 1{,}716$ của Bảng 3; đo sai số tuyệt đối, kỳ vọng dưới 10^{-4}.
3. **Đo đánh đổi của gradient checkpointing** trên một MLP/Transformer-block nhỏ 15-20 lớp: so sánh peak memory (`torch.cuda.max_memory_allocated`) và thời gian một bước train khi bật/tắt `torch.utils.checkpoint`, đối chiếu tỉ lệ tiết kiệm bộ nhớ/tăng compute với con số paper dẫn (95%/33%) để xem mức đánh đổi có cùng cỡ không trên kiến trúc nhỏ của mình.
4. **Hessian-vector product toy:** trên một mạng nhỏ (vài nghìn tham số), tính $H\mathbf{v}$ bằng kỹ thuật reverse-on-forward (`torch.autograd.functional.vhp` hoặc double-backward tay) và so thời gian/bộ nhớ với vật lý hoá toàn bộ Hessian rồi nhân vector trực tiếp; kỳ vọng chênh lệch rõ khi số tham số tăng.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý
Numerical diff	$\frac{\partial f}{\partial x_i} \approx \frac{f(\mathbf{x}+h\mathbf{e}_i)-f(\mathbf{x})}{h}$, $O(n)$ lần gọi f
Center difference	$\frac{f(\mathbf{x}+h\mathbf{e}_i)-f(\mathbf{x}-h\mathbf{e}_i)}{2h} + O(h^2)$
Dual number	$v + \dot{v}\epsilon$, $\epsilon^2 = 0$; $f(v + \dot{v}\epsilon) = f(v) + f'(v)\dot{v}\epsilon$
Forward mode	tangent $\dot{v}_i = \partial v_i / \partial x_1$; 1 pass = 1 cột Jacobian; tốt khi $f : \mathbb{R} \to \mathbb{R}^m$
Reverse mode	adjoint $\bar{v}_i = \partial y_j / \partial v_i$; 1 pass = 1 hàng Jacobian (cả gradient nếu $m = 1$); tốt khi $f : \mathbb{R}^n \to \mathbb{R}$, n lớn
Chi phí	forward: $n\, c\, \text{ops}(f)$; reverse: $m\, c\, \text{ops}(f)$, $c < 6$, $c \in [2, 3]$
Jacobian-vector product (forward)	$J_f\mathbf{r}$, seed $\dot{\mathbf{x}} = \mathbf{r}$, 1 pass
Hessian-vector product	reverse-on-forward: $H_f\mathbf{v}$ trong $O(n)$, không cần vật lý hoá H_f
Backpropagation	trường hợp đặc biệt của reverse-mode AD, với $y =$ loss scalar

11. Kết luận ngắn

Bài khảo sát định nghĩa chính xác AD như một kỹ thuật khác hẳn đạo hàm tay/số/ký hiệu: tính đúng tới độ chính xác máy với chi phí chỉ tăng hằng số so với hàm gốc, nhờ lan truyền chain rule trên evaluation trace cụ thể của chương trình. Hai chế độ forward (tangent, theo hướng dữ liệu) và reverse (adjoint, ngược hướng dữ liệu, tổng quát hoá backpropagation) có chi phí đối nghịch theo tỉ lệ input/output, và thí nghiệm định lượng của chính bài (hàm Helmholtz, Bảng 4/Hình 5) xác nhận bằng số rằng reverse mode gần như không đắt hơn hàm gốc bất kể số chiều, đúng lý do nó là nền tảng của mọi pipeline huấn luyện deep learning hiện đại. Bài này chuyển hoá các quy tắc toán tay ở math 01 thành một thuật toán tổng quát cho bất kỳ chương trình số nào, đặt nền cho mọi framework autodiff (PyTorch và tương tự) mà các bài sau trong lộ trình sử dụng mà không cần suy lại từ đầu.

Nguồn gốc: https://arxiv.org/abs/1502.05767

Bài 03

A Tutorial on Principal Component Analysis (2014)

Tác giả: Jonathon Shlens (Google Research, Mountain View, CA)
Venue / arXiv: arXiv:1404.1100 (v1, 03/04/2014); paper không nêu venue hội nghị/tạp chí, đây là một bản tutorial tự xuất bản (trang đầu ghi ”Dated: April 7, 2014; Version 3.02”)
Nguồn: https://arxiv.org/abs/1404.1100
Chủ đề: Đại số tuyến tính ứng dụng, giảm chiều dữ liệu (PCA qua eigen-decomposition của ma trận hiệp phương sai và qua SVD)
Vị trí trong lộ trình: Bài số 3 trong nhóm 9 bài toán nền (math 01-09) giới thiệu công cụ toán nền tảng. Đọc sau math 01 (matrix calculus) và math 02 (automatic differentiation), vì các khái niệm eigenvector, eigenvalue, đạo hàm ma trận ở hai bài đó sẽ được dùng lại. Đây là bài mở đầu nhóm con ”phương pháp eigen/SVD”, đứng trước math 04 (randomized SVD) và math 05 (spectral clustering).

> Ghi chú diễn giải PCA bằng tiếng Việt, dựa trên bài báo và hai phụ lục về chứng minh và mã Matlab. Các ví dụ đánh dấu “tự tính” được bổ sung để minh họa công thức.

1. Bối cảnh

Principal Component Analysis (PCA) được dùng trong nhiều lĩnh vực, từ thần kinh học, đồ họa máy tính đến nén dữ liệu và học máy. Jonathon Shlens trình bày PCA từ trực giác đến công thức đại số tuyến tính, nhằm giải thích cách thức và lý do phương pháp hoạt động.

Đây là một bản tutorial tự xuất bản trên arXiv, không công bố ở hội nghị hay tạp chí nào. Một điểm khác thường so với các bài báo khoa học điển hình: bài không có danh sách tài liệu tham khảo (reference list) nào ở cuối. Toàn bộ nội dung được tác giả tự suy diễn từ các định lý đại số tuyến tính cơ bản (được chứng minh lại trong Phụ lục A), kèm hai đoạn mã Matlab minh họa ở Phụ lục B. Giả định duy nhất mà tác giả đặt ra là người đọc đã có kiến thức đại số tuyến tính cơ bản (vector, ma trận, tích vô hướng, giá trị riêng).

Bài báo gồm phần trình bày PCA và hai phụ lục: một phụ lục chứng minh các định lý đại số tuyến tính, phụ lục còn lại cung cấp mã Matlab.

Trong lộ trình học, bài này đóng vai trò nền tảng cho nhóm các phương pháp dựa trên eigen-decomposition/SVD: những kỹ thuật như randomized SVD (math 04) hay spectral clustering (math 05) đều xoay quanh việc tìm một tập vector riêng (eigenvector) ”tốt” của một ma trận nào đó, và PCA là ví dụ đơn giản nhất, dễ hình dung nhất của mô thức này khi ma trận đó là ma trận hiệp phương sai của dữ liệu.

2. Ý tưởng chính

Dữ liệu ban đầu được ghi trong một hệ tọa độ có thể do vị trí cảm biến hoặc camera quyết định và chưa phản ánh rõ cấu trúc của nó. PCA tìm một cơ sở trực chuẩn mới bằng phép quay hệ tọa độ, sao cho khi biểu diễn dữ liệu trong cơ sở này:

- Các chiều (biến) mới không còn tương quan với nhau (hiệp phương sai chéo bằng 0), nghĩa là loại bỏ được sự dư thừa (redundancy) giữa các biến đo.
- Các chiều được xếp hạng theo phương sai giảm dần, chiều có phương sai lớn nhất được coi là mang nhiều ”cấu trúc tín hiệu” nhất (theo giả định signal-to-noise ratio, SNR).

Để đi đến công thức cụ thể, paper đặt ra ba giả định tường minh (paper tự liệt kê ở mục ”Summary of Assumptions”):

1. **Tuyến tính (Linearity):** cơ sở mới chỉ được phép là một tổ hợp tuyến tính của cơ sở cũ, tức bài toán được quy về bài toán đổi cơ sở $Y = PX$ với P là ma trận biến đổi.
2. **Phương sai lớn = cấu trúc quan trọng:** các hướng có phương sai lớn nhất được giả định chứa thông tin tín hiệu quan trọng nhất, còn phương sai nhỏ được coi là nhiễu.

Với ba giả định này, PCA trở thành bài toán: tìm ma trận trực chuẩn P sao cho ma trận hiệp phương sai của dữ liệu đã biến đổi, $C_Y = \frac{1}{n}YY^T$, là một ma trận đường chéo (diagonal), và các giá trị trên đường chéo (chính là các phương sai) được xếp theo thứ tự giảm dần. Paper chứng minh nghiệm của bài toán này chính là: các hàng của P là các eigenvector của ma trận hiệp phương sai $C_X = \frac{1}{n}XX^T$ của dữ liệu gốc, xếp theo eigenvalue giảm dần. Đây gọi là các ”principal components”.

Một cách diễn đạt tương đương (và được paper trình bày là cách tổng quát hơn) là qua phân tích suy biến giá trị kỳ dị (Singular Value Decomposition, SVD): các principal component chính là các vector riêng bên phải (right singular vectors) của một ma trận liên quan trực tiếp đến X.

3. Phương pháp chi tiết

3.1. Ví dụ dẫn nhập: lò xo và ba camera

Paper mở đầu bằng một ví dụ vật lý: một quả bóng gắn vào lò xo lý tưởng dao động dọc theo một trục duy nhất (gọi là trục x). Vì người thực nghiệm không biết trước đâu là trục đúng, họ đặt ba camera A, B, C ở các góc tùy ý quanh hệ thống, mỗi camera ghi lại một hình chiếu 2D của vị trí quả bóng với tần số 120 Hz. Mỗi mẫu dữ liệu tại một thời điểm là một vector 6 chiều $\vec{X} = (x_A, y_A, x_B, y_B, x_C, y_C)^T$. Nếu ghi 10 phút, ta có $10 \times 60 \times 120 = 72000$ vector như vậy.

Dù bản chất vật lý chỉ có 1 chiều chuyển động thật (x), do cách đo đạc (ba camera ở các góc bất kỳ), dữ liệu thô ”trông” như có 6 chiều độc lập. Câu hỏi của PCA là: từ 6 chiều đo được (nhiễu, dư thừa, không phải cơ sở ”đúng”), làm sao suy ra được hướng chuyển động thật?

Paper gọi cơ sở đo đạc ban đầu (ví dụ (x_A, y_A) của một camera) là ”naive basis” (cơ sở ngây thơ), vì nó chỉ phản ánh cách đặt thiết bị đo, không phản ánh cấu trúc vật lý thật của hệ thống.

3.2. Đổi cơ sở (change of basis)

Gọi X là ma trận dữ liệu gốc kích thước $m \times n$ (m là số loại đo đạc, mỗi cột là một mẫu/thời điểm; trong ví dụ lò xo thì $m = 6$, $n = 72000$). PCA tìm một ma trận P (kích thước $m \times m$, trực chuẩn) để biến đổi:

$$PX = Y$$

Trong đó các hàng p_i của P là tập cơ sở mới, các cột x_i của X là các mẫu dữ liệu gốc, và các cột y_i của Y là dữ liệu đã biểu diễn lại. Paper chỉ ra: mỗi phần tử của y_i là tích vô hướng của x_i với một hàng p_j của P, tức y_i chính là hình chiếu của x_i lên các vector $p_1, \ldots, p_m$. Nói cách khác, các hàng của P đóng vai trò một cơ sở trực chuẩn mới để biểu diễn lại các cột của X.

Hai câu hỏi còn lại mà paper đặt ra: ”cách biểu diễn lại X tốt nhất là gì?” và ”chọn P như thế nào?”. Để trả lời, cần thêm tiêu chí: thế nào là ”biểu diễn tốt”.

3.3. Phương sai, nhiễu và dư thừa

Paper định nghĩa tỉ số tín hiệu trên nhiễu (SNR) là $\text{SNR} = \sigma^2_{\text{signal}}/\sigma^2_{\text{noise}}$: SNR cao nghĩa là đo đạc chính xác, SNR thấp nghĩa là dữ liệu nhiễu nhiều. Minh họa bằng Hình 2 của paper (dữ liệu mô phỏng của một camera): hướng có phương sai lớn nhất của đám mây điểm không trùng với trục đo đạc ban đầu (x_A, y_A), mà là một hướng xiên (đường fit tốt nhất qua đám mây). Giả định cốt lõi: hướng có phương sai lớn nhất chứa động lực học (dynamics) quan trọng nhất của hệ thống.

Bên cạnh nhiễu, còn có vấn đề dư thừa (redundancy): nhiều cảm biến có thể đo cùng một thông tin (ví dụ hai camera quá gần nhau, hoặc cùng một đại lượng đo bằng hai đơn vị khác nhau). Paper minh họa bằng Hình 3: nếu hai biến đo r_1, r_2 không tương quan (plot bên trái), không có gì để loại bỏ; nếu chúng tương quan cao (plot bên phải), chỉ cần giữ lại một biến là đủ, đây chính là ý tưởng cốt lõi của giảm chiều dữ liệu.

Để định lượng dư thừa, paper định nghĩa hiệp phương sai (covariance) giữa hai tập đo $A = \{a_1, \ldots, a_n\}$ và $B = \{b_1, \ldots, b_n\}$ (giả định trung bình bằng 0):

$$\sigma_A^2 = \frac{1}{n}\sum_i a_i^2, \quad \sigma_B^2 = \frac{1}{n}\sum_i b_i^2, \quad \sigma_{AB}^2 = \frac{1}{n}\sum_i a_i b_i = \frac{1}{n} a b^T$$

với a, b là các vector hàng. Paper nêu rõ hai tính chất: $\sigma_{AB} = 0$ khi và chỉ khi A, B không tương quan; $\sigma_{AB}^2 = \sigma_A^2$ khi $A = B$. Paper cũng có một chú thích (footnote 2) quan trọng: trong công thức chính của bài, hiệp phương sai dùng hệ số $1/n$ để đơn giản hóa việc suy diễn, nhưng trong thực hành (và lý thuyết ước lượng thống kê), người ta thường dùng $1/(n-1)$. Đây là điểm cần lưu ý: **công thức suy diễn chính trong mục III-V của paper dùng $1/n$, nhưng hai đoạn mã Matlab ở Phụ lục B lại dùng $1/(N-1)$** (`covariance = 1/(N-1) * data * data';` và `Y = data' / sqrt(N-1);`). Đây không phải một lỗi hay mâu thuẫn bị bỏ sót; paper đã tự nêu rõ sự khác biệt này ở footnote 2, nhưng người đọc cần nhận ra chủ động rằng công thức lý thuyết (dùng n) và mã thực hành (dùng $n-1$) không giống nhau tuyệt đối, chỉ xấp xỉ nhau khi n lớn.

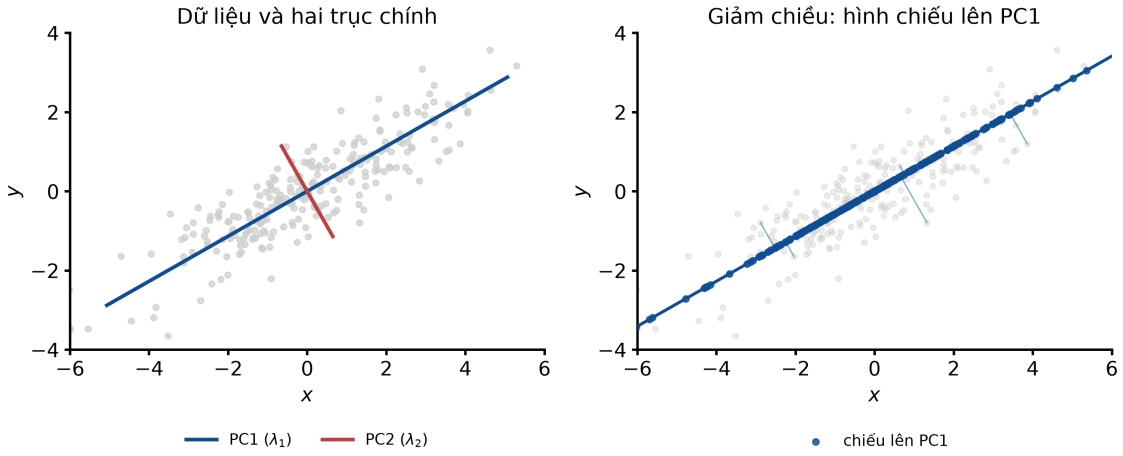
Tổng quát hóa cho m loại đo đạc, định nghĩa ma trận hiệp phương sai:

$$C_X \equiv \frac{1}{n} X X^T$$

Paper liệt kê các tính chất của C_X: là ma trận vuông đối xứng $m \times m$ (chứng minh ở Phụ lục A, Định lý 2); các phần tử trên đường chéo là phương sai của từng loại đo; các phần tử ngoài đường chéo là hiệp phương sai giữa các cặp đo đạc. Mục tiêu: tìm cách biến đổi để có một ma trận hiệp phương sai mới C_Y sao cho (1) mọi phần tử ngoài đường chéo bằng 0 (dữ liệu mới không còn tương quan), và (2) các phần tử trên đường chéo xếp theo thứ tự giảm dần (các chiều xếp hạng theo "độ quan trọng").

3.4. Lời giải 1: eigen-decomposition của ma trận hiệp phương sai

Paper chọn cách đơn giản nhất để chéo hóa C_Y: giả định P là ma trận trực chuẩn (orthonormal). Khi đó:



PCA hình học: hai trục chính là eigenvector của ma trận hiệp phương sai; giảm chiều bằng hình chiếu lên PC1. (minh hoạ, tự tính)

$$C_Y = \frac{1}{n} YY^T = \frac{1}{n}(PX)(PX)^T = P\left(\frac{1}{n}XX^T\right)P^T = PC_XP^T$$

Theo định lý đại số tuyến tính (Phụ lục A, Định lý 4): mọi ma trận đối xứng A có thể chéo hóa bằng ma trận các eigenvector của nó, $A = EDE^T$, với D là ma trận đường chéo chứa các eigenvalue, E là ma trận các eigenvector (đã chuẩn hóa) xếp theo cột. Chọn $P \equiv E^T$ (mỗi hàng của P là một eigenvector của C_X), và dùng tính chất ma trận trực chuẩn có nghịch đảo bằng chuyển vị ($P^{-1} = P^T$, Định lý 1), ta có:

$$C_Y = PC_XP^T = P(E^TDE)P^T = (PP^T)D(PP^T) = D$$

Vậy: **các thành phần chính (principal components) của X chính là các eigenvector của $C_X = \frac{1}{n}XX^T$**, và phần tử thứ i trên đường chéo của C_Y chính là phương sai của dữ liệu đã chiếu lên eigenvector thứ i. Thuật toán thực hành (paper nêu): (1) trừ đi giá trị trung bình của mỗi loại đo, (2) tính các eigenvector của C_X, xếp theo eigenvalue giảm dần.

3.5. Lời giải 2: qua SVD

Paper trình bày một cách tổng quát hơn qua phân tích SVD, và thừa nhận đây là phần "mathematically involved" nhất, có thể bỏ qua mà không mất tính liên tục của bài. Cho một ma trận X bất kỳ kích thước $n \times m$ (chú ý: paper đổi chiều quy ước $m \times n$ thành $n \times m$ chỉ trong mục này), X^TX là ma trận đối xứng $m \times m$ hạng r, có các eigenvector trực chuẩn $\hat{v}_1, \ldots, \hat{v}_r$ với eigenvalue $\lambda_1, \ldots, \lambda_r$. Định nghĩa singular value $\sigma_i \equiv \sqrt{\lambda_i}$ và vector $\hat{u}_i \equiv \frac{1}{\sigma_i} X\hat{v}_i$. Dạng vô hướng của SVD:

$$X\hat{v}_i = \sigma_i \hat{u}_i$$

Ghép tất cả thành dạng ma trận (xếp các $\hat{v}_i$ thành cột của V, các $\hat{u}_i$ thành cột của U, các σ_i lên đường chéo của Σ):

$$X = U\Sigma V^T$$

Đây chính là định nghĩa SVD: mọi ma trận X đều có thể phân tích thành (trực giao) $\times$ (đường chéo) $\times$ (trực giao), tương ứng với phép quay, phép co giãn, rồi một phép quay khác.

Lưu ý về ký hiệu: ở đoạn này paper định nghĩa một ma trận mới cũng được gọi là Y, nhưng với vai trò hoàn toàn khác so với $Y = PX$ ở mục III-IV. Cụ thể, để kết nối SVD với PCA, paper đặt $Y \equiv \frac{1}{\sqrt{n}} X^T$ (kích thước $n \times m$, mỗi cột có trung bình 0), rồi chỉ ra $Y^TY = \frac{1}{n}XX^T = C_X$ (chính là ma trận hiệp phương sai). Nếu lấy SVD của ma trận Y này, các cột của V (trong $Y = U\Sigma V^T$) chính là các eigenvector của C_X, tức chính là các principal component. Đây không phải một mâu thuẫn toán học, nhưng là một điểm **dễ gây nhầm lẫn khi đọc**: chữ "Y" được paper dùng lại cho hai đối tượng khác nhau ở hai mục khác nhau (nhận xét của người ghi chú, không phải do paper nói sai).

Thuật toán thực hành thứ hai (SVD): (1) trừ trung bình mỗi loại đo, (2) tính SVD của ma trận $\frac{1}{\sqrt{n}} X^T$, lấy V làm các thành phần chính. Phụ lục B cho hai đoạn mã Matlab tương ứng hai thuật toán này (`pca1` dùng eigen-decomposition của hiệp phương sai, `pca2` dùng SVD).

3.6. Ví dụ số tự tính: PCA tay cho 4 mẫu, 2 chiều

Để kiểm chứng toàn bộ công thức ở trên bằng số cụ thể (tự tính), xét một bộ dữ liệu giả định đã có trung bình 0, 2 loại đo A, B (tức $m = 2$), 4 mẫu ($n = 4$):

mẫu	A	B
1	2	2
2	-2	-2
3	1	-1
4	-1	1

Dễ thấy mẫu 1, 2 nằm đúng trên đường $B = A$, còn mẫu 3, 4 nằm đúng trên đường $B = -A$, tức đây là một phiên bản số của ”dữ liệu tương quan cao” ở Hình 3 của paper, nhưng với hai cụm tương quan ngược dấu nhau.

Bước 1, tính hiệp phương sai (dùng công thức $1/n$ như mục III-V của paper):

$$\sigma_A^2 = \frac{1}{4}(4+4+1+1) = 2.5, \quad \sigma_B^2 = \frac{1}{4}(4+4+1+1) = 2.5$$

$$\sigma_{AB}^2 = \frac{1}{4}\big(2\cdot 2 + (-2)\cdot(-2) + 1\cdot(-1) + (-1)\cdot 1\big) = \frac{1}{4}(4+4-1-1) = 1.5$$

Vậy $C_X = \begin{pmatrix} 2.5 & 1.5 \\ 1.5 & 2.5 \end{pmatrix}$.

Bước 2, tìm eigenvalue: phương trình đặc trưng $(2.5-\lambda)^2 - 1.5^2 = 0 \Rightarrow \lambda = 2.5 \pm 1.5$, cho $\lambda_1 = 4$, $\lambda_2 = 1$. Kiểm tra: $\lambda_1 + \lambda_2 = 5 = \sigma_A^2 + \sigma_B^2$ (tổng phương sai được bảo toàn qua phép quay, đúng như tính chất trace của ma trận đối xứng khi chéo hóa bằng ma trận trực chuẩn).

Bước 3, tìm eigenvector: với $\lambda_1 = 4$: $(2.5-4)x + 1.5y = 0 \Rightarrow -x + y = 0 \Rightarrow x = y$, chuẩn hóa được $p_1 = (1/\sqrt{2}, 1/\sqrt{2})$. Với $\lambda_2 = 1$: $(2.5-1)x + 1.5y = 0 \Rightarrow x + y = 0 \Rightarrow x = -y$, chuẩn hóa $p_2 = (1/\sqrt{2}, -1/\sqrt{2})$.

Bước 4, chiếu dữ liệu lên các thành phần chính và kiểm tra lại phương sai: chiếu mẫu 1, $(2,2)$, lên p_1: $(2+2)/\sqrt{2} = 2\sqrt{2} \approx 2.828$; lên p_2: $(2-2)/\sqrt{2} = 0$. Chiếu mẫu 3, $(1,-1)$, lên p_1: $(1-1)/\sqrt{2} = 0$; lên p_2: $(1+1)/\sqrt{2} = \sqrt{2} \approx 1.414$. Làm tương tự cho mẫu 2, 4, ta được tọa độ chiếu lên p_1 là $\{2\sqrt{2}, -2\sqrt{2}, 0, 0\}$ và lên p_2 là $\{0, 0, \sqrt{2}, -\sqrt{2}\}$.

Phương sai của tọa độ chiếu lên p_1: $\frac{1}{4}\big[(2\sqrt{2})^2 + (-2\sqrt{2})^2 + 0 + 0\big] = \frac{1}{4}(8+8) = 4$, đúng bằng λ_1. Phương sai của tọa độ chiếu lên p_2: $\frac{1}{4}\big[0 + 0 + (\sqrt{2})^2 + (-\sqrt{2})^2\big] = \frac{1}{4}(2+2) = 1$, đúng bằng λ_2. Vậy phép tính khớp hoàn toàn với công thức $C_Y = D$ ở mục 3.4: sau khi đổi sang cơ sở $P = \{p_1, p_2\}$, hiệp phương sai mới là $\begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix}$, đường chéo, đúng như mục tiêu của PCA. Về mặt hình học, p_1 chính là hướng mà mẫu 1, 2 nằm trên đó (đường $B = A$), còn p_2 là hướng mà mẫu 3, 4 nằm trên đó (đường $B = -A$): PCA đã ”tìm lại” chính xác hai trục ẩn mà dữ liệu được tạo ra từ đó, không cần biết trước quy luật sinh dữ liệu (toàn bộ ví dụ này là tự tính, không lấy từ paper).

4. Thí nghiệm và kết quả chính

Đây là một bài tutorial lý thuyết, **không có benchmark định lượng** theo nghĩa so sánh với baseline trên bộ dữ liệu thật. Paper không chạy PCA trên một dataset thực tế nào và không báo cáo số liệu (thời gian chạy, độ chính xác, v.v.). ”Kết quả chính” của paper là các kết quả suy diễn đại số tuyến tính và các ví dụ minh họa mang tính khái niệm:

- **Kết quả toán học trung tâm:** PCA tương đương với việc tìm eigenvector của ma trận hiệp phương sai $C_X = \frac{1}{n}XX^T$ (mục 3.4), hoặc tương đương tìm right singular vector của ma trận $\frac{1}{\sqrt{n}}X^T$ qua SVD (mục 3.5). Hai cách này cho cùng một tập thành phần chính.
- **Ví dụ minh họa lò xo:** paper khẳng định (không kèm số liệu, chỉ là lập luận định tính) rằng áp dụng PCA lên 6 chiều dữ liệu camera sẽ ”khôi phục” được một chiều chính mang hầu hết phương sai, tương ứng với hướng chuyển động $\hat{x}$ thật của lò xo, mặc dù dữ liệu thô có 6 chiều.
- **”Quick Summary of PCA” (Hình 5 của paper):** paper tổng kết quy trình thực hành thành 3 bước: (1) tổ chức dữ liệu thành ma trận $m \times n$ (m loại đo, n mẫu), (2) trừ trung bình của mỗi loại đo, (3) tính SVD hoặc eigenvector của ma trận hiệp phương sai.
- **Hai ví dụ phản biện (Hình 6):** (a) theo dõi một người trên vòng đu quay (ferris wheel): động lực học thực chất chỉ phụ thuộc một biến góc quay θ (phi tuyến với tọa độ Descartes), PCA không khôi phục được biến này; (b) một bộ dữ liệu có phân phối không-Gauss và các trục không trực giao, khiến hướng phương sai lớn nhất không phải là câu trả lời ”đúng” mà ta mong đợi. Cả hai ví dụ này cũng chỉ minh họa bằng hình vẽ, không có số liệu định lượng kèm theo.
- **Phụ lục B (mã nguồn):** hai hàm Matlab `pca1` (dùng eigen-decomposition của hiệp phương sai) và `pca2` (dùng SVD) được cung cấp để người đọc tự chạy, nhưng paper không so sánh tốc độ hay độ chính xác giữa hai cách cài đặt này trên dữ liệu cụ thể nào.

Nói cách khác, đóng góp chính của paper nằm ở phần suy diễn và trực giác, không phải ở phần thực nghiệm. Đây là điểm khác biệt rõ với math 02 (bài trước trong batch), nơi vẫn có một bảng thực nghiệm định lượng thật (benchmark thời gian chạy của forward/reverse-mode AD).

5. Phân tích / ablation

Vì không có thí nghiệm định lượng, phần ”ablation” ở đây được hiểu theo nghĩa phân tích các giả định và điều kiện áp dụng mà paper tự đặt ra:

So sánh hai lời giải (eigen-decomposition vs SVD): về mặt đại số, hai cách hoàn toàn tương đương: eigenvector của C_X cũng chính là các cột V trong SVD của $\frac{1}{\sqrt{n}}X^T$. Paper trình bày cách 1 (mục V) là ”straightforward” và cách 2 (mục VI) là tổng quát hơn nhưng phức tạp hơn về đại số. Paper không bàn đến sự khác biệt về độ ổn định số học (numerical stability) giữa việc tính trực tiếp eigenvector của XX^T so với tính SVD của X trực tiếp; đây là một điểm thường được nhắc tới trong thực hành tính toán số (tính XX^T rồi lấy eigen có thể khuếch đại lỗi số học do bình phương điều kiện số của X) nhưng **không được paper này đề cập**, nên nếu muốn dùng lập luận đó phải ghi là ”(ngoài paper, kiến thức chung)”.

Ba giả định của PCA và khi nào chúng có vấn đề (paper tự thảo luận ở mục VII):

Giả định	Nội dung	Khi nào có vấn đề (theo paper)
I. Tuyến tính	Cơ sở mới là tổ hợp tuyến tính của cơ sở cũ	Khi động lực học thật là phi tuyến (ví dụ Hình 6a, vòng đu quay với biến góc θ)
II. Phương sai lớn = cấu trúc quan trọng	Thành phần phương sai cao mang thông tin, phương sai thấp là nhiễu	Giả định này ”có thể sai” khi SNR thấp hoặc khi cấu trúc quan trọng lại nằm ở phương sai nhỏ; paper nêu đây là giả định ”mạnh và đôi khi không đúng”
III. Các thành phần chính trực giao	P là ma trận trực chuẩn	Khi dữ liệu thật phân bố theo các trục không trực giao (Hình 6b); đây là giả định kỹ thuật để bài toán có nghiệm đại số gọn, không phải yêu cầu vật lý

Về tính tối ưu của PCA: paper khẳng định ”it can be proved that under a common loss function, mean squared error (i.e. L2 norm), PCA provides the optimal reduced representation of the data” nhưng **không đưa**

ra chứng minh hay trích dẫn nguồn cho khẳng định này trong văn bản được đọc. Đây là một khẳng định được nêu nhưng không được chứng minh trong bài, cần ghi nhận rõ là ”paper nói” chứ ghi chú này không tự kiểm chứng được tính đúng đắn của phát biểu đó.

Về ”phi tham số” (non-parametric): bài báo nêu đây vừa là ưu điểm (không cần chọn tham số, kết quả duy nhất và không phụ thuộc người dùng) vừa là nhược điểm (PCA ”không quan tâm” đến cách dữ liệu được tạo ra, nên không thể tận dụng kiến thức tiên nghiệm, ví dụ không nhận ra cấu trúc tuần hoàn của ví dụ vòng đu quay).

6. Giới hạn

Các giới hạn mà **bài báo nêu** (mục VII ”Discussion”):

- PCA chỉ loại bỏ được các phụ thuộc bậc hai (second-order dependencies, tức tương quan tuyến tính đo bằng hiệp phương sai). Nếu cấu trúc quan trọng của dữ liệu nằm ở các phụ thuộc bậc cao hơn (higher-order dependencies), PCA sẽ không phát hiện ra. Paper có chú thích (footnote 7) nêu: phụ thuộc bậc hai là đủ để mô tả toàn bộ cấu trúc khi dữ liệu phân phối Gauss (trong trường hợp đó bậc một và bậc hai là thống kê đủ, sufficient statistics).
- Ví dụ thất bại cụ thể: vòng đu quay (Hình 6a, động lực học phi tuyến theo góc θ) và dữ liệu không-Gauss/trục không trực giao (Hình 6b).
- Paper gợi ý hai hướng khắc phục nhưng không đi sâu: kernel PCA (áp dụng một phi tuyến/kernel lên dữ liệu trước khi làm PCA, ví dụ đổi sang tọa độ cực cho ví dụ vòng đu quay) và Independent Component Analysis, ICA (yêu cầu các chiều giảm xuống độc lập thống kê, không chỉ không tương quan). Về ICA, paper nhận xét thẳng: ”suffers from the fact that solutions are (sometimes) difficult to compute”, nhưng không trình bày thuật toán ICA cụ thể nào.

Một số quan sát **của người ghi chú** (không phải do paper tự phê bình, nhưng là đặc điểm khách quan của văn bản):

- Paper hoàn toàn không có danh mục tài liệu tham khảo; mọi khẳng định lịch sử hoặc liên hệ với các phương pháp khác (ICA, kernel PCA) đều không có trích dẫn kèm theo, nên không thể lần ngược lại nguồn gốc các khẳng định đó từ chính văn bản này.
- Như đã nêu ở mục 3.5, paper dùng lại ký hiệu Y cho hai đối tượng khác nhau ở hai mục khác nhau (dữ liệu đã đổi cơ sở ở mục III-IV, và ma trận chuẩn hóa $\frac{1}{\sqrt{n}}X^T$ dùng để tính SVD ở mục VI.C). Đây là một điểm dễ gây nhầm lẫn khi đọc nhanh.
- Công thức hiệp phương sai trong thân bài dùng hệ số $1/n$, còn mã Matlab ở Phụ lục B dùng $1/(N-1)$; paper có nêu khác biệt này ở một footnote ngắn nhưng không nhấn mạnh, người đọc dễ bỏ sót nếu chỉ đọc phần công thức chính.
- Toàn bộ các ví dụ số của paper (ví dụ lò xo, Hình 2, 3, 6) chỉ mang tính minh họa định tính qua hình vẽ, không có dữ liệu số cụ thể nào được in kèm trong văn bản để người đọc tự kiểm tra lại bằng tay; đây là lý do ghi chú này phải tự xây dựng một ví dụ số riêng ở mục 3.6 để có một ví dụ ”tự tính” kiểm chứng công thức.

7. Liên hệ với các bài khác

- **math 04 (Finding Structure with Randomness, Halko-Martinsson-Tropp):** bài này mở rộng trực tiếp lời giải SVD của PCA (mục 3.5 ở trên) sang quy mô lớn, dùng random projection để tính một SVD xấp xỉ nhanh hơn nhiều so với tính SVD đầy đủ khi ma trận dữ liệu rất lớn. Nói cách khác, math 04 trả lời câu hỏi ”làm sao tính PCA/SVD hiệu quả khi m, n rất lớn”, trong khi bài hiện tại chỉ trình bày lời giải ”đúng nhưng tốn kém” (eigen-decomposition hoặc SVD đầy đủ).
- **math 05 (A Tutorial on Spectral Clustering, von Luxburg):** cũng dựa trên eigen-decomposition, nhưng của một ma trận khác, ma trận Laplacian của đồ thị tương tự (similarity graph) giữa các điểm dữ

liệu, thay vì ma trận hiệp phương sai. Sự tương phản thú vị: PCA tìm hướng phương sai lớn nhất (dữ liệu ”trải rộng” nhất) để giữ thông tin, còn spectral clustering tìm các eigenvector gắn với eigenvalue *nhỏ nhất* của Laplacian để tìm cấu trúc cụm (cluster). Cả hai đều minh họa mô thức chung ”phân tích dữ liệu bằng eigenvector của một ma trận mô tả cấu trúc dữ liệu”, nhưng chọn ma trận và tiêu chí khác nhau.

- **math 19 (Measuring the Intrinsic Dimension of Objective Landscapes, Li và cộng sự 2018):** đặt câu hỏi cùng họ với PCA, ”dữ liệu/bài toán thực chất cần bao nhiêu chiều để mô tả đầy đủ”, nhưng áp dụng cho không gian tham số của mạng neural (landscape mục tiêu huấn luyện) thay vì không gian đặc trưng của dữ liệu, và dùng phương pháp random subspace training thay vì eigen-decomposition của hiệp phương sai.
- **ml basics 40 (UMAP, McInnes và cộng sự):** một phương pháp giảm chiều phi tuyến hiện đại, trực tiếp giải quyết đúng loại thất bại mà paper này tự nêu ở Hình 6 (dữ liệu có cấu trúc phi tuyến hoặc phân phối không phù hợp với giả định tuyến tính/Gauss của PCA).

8. Gợi ý áp dụng

PCA có thể dùng để thăm dò số chiều cần thiết để giải thích phần lớn phương sai của dữ liệu hoặc activation. Kết quả phụ thuộc mẫu và chỉ phản ánh cấu trúc tuyến tính bậc hai; phương sai thấp không tự chứng minh rằng một đặc trưng không cần thiết cho tác vụ.

9. Thí nghiệm gợi ý

Các thí nghiệm nhỏ dưới đây đều có thể chạy trên GPU, dùng PyTorch (hàm `torch.pca_lowrank` hoặc tự viết qua `torch.linalg.eigh`/`torch.linalg.svd`):

1. **Đo phổ phương sai:** lấy activation từ một mạng nhỏ trên tập dữ liệu đại diện, tính PCA và vẽ phương sai tích lũy theo số thành phần.
2. **So sánh tầng đầu và tầng cuối:** lặp lại thí nghiệm 1 cho tầng gần đầu vào và tầng gần đầu ra, so sánh hai đường cong. Giả thuyết cần kiểm tra: các tầng sâu hơn có xu hướng ”nén” biểu diễn vào không gian chiều thấp hơn (đường cong dốc nhanh hơn) so với tầng đầu.
3. **Tái tạo lại hai thuật toán `pca1`/`pca2` của Phụ lục B bằng PyTorch/NumPy trên dữ liệu toy 2D/3D có nhiễu Gauss** (mô phỏng lại ví dụ lò xo bằng code, ví dụ sinh dữ liệu dọc theo một trục cộng nhiễu Gauss nhỏ ở các trục khác), xác nhận hai cách (eigen-decomposition của hiệp phương sai và SVD) cho đúng cùng kết quả (các eigenvalue và eigenvector phải khớp, trong giới hạn sai số số học), đồng thời đo thời gian chạy của hai cách khi tăng dần n (số mẫu) để cảm nhận sự khác biệt thực hành giữa hai cách cài đặt.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / định nghĩa
Đổi cơ sở	$Y = PX$, P trực chuẩn ($P^{-1} = P^T$)
Ma trận hiệp phương sai	$C_X = \frac{1}{n} XX^T$ (đường chéo: phương sai; ngoài đường chéo: hiệp phương sai)
Mục tiêu PCA	Tìm P sao cho $C_Y = PC_XP^T = D$ (ma trận đường chéo, các phần tử giảm dần)
Lời giải 1	Các hàng của P là eigenvector của C_X, xếp theo eigenvalue λ_i giảm dần; phương sai theo hướng p_i bằng λ_i
SVD tổng quát	$X = U\Sigma V^T$; với $X\hat{v}_i = \sigma_i \hat{u}_i$, $\sigma_i = \sqrt{\lambda_i}$, λ_i là eigenvalue của X^TX

Khái niệm	Công thức / định nghĩa
Lời giải 2 (qua SVD)	Đặt $Z \equiv \frac{1}{\sqrt{n}} X^T$; các cột V trong SVD của Z là các principal component của X
Quy trình thực hành (3 bước)	(1) Tổ chức dữ liệu thành ma trận $m \times n$; (2) trừ trung bình mỗi loại đo; (3) tính SVD hoặc eigenvector của hiệp phương sai
Ví dụ tự tính (mục 3.6)	$C_X = \begin{pmatrix} 2.5 & 1.5 \\ 1.5 & 2.5 \end{pmatrix}$, $\lambda_1 = 4$ ứng với $p_1 = (1/\sqrt{2}, 1/\sqrt{2})$, $\lambda_2 = 1$ ứng với $p_2 = (1/\sqrt{2}, -1/\sqrt{2})$, kiểm tra $\lambda_1 + \lambda_2 = 5 = \sigma_A^2 + \sigma_B^2$

11. Kết luận ngắn

Bài báo trình bày lại PCA từ gốc, cho thấy toàn bộ kỹ thuật thực chất chỉ là: chọn một phép quay trực chuẩn của hệ trục tọa độ sao cho dữ liệu, biểu diễn trong hệ trục mới, trở nên ”decorrelated” (hiệp phương sai chéo) và được xếp hạng theo phương sai, và lời giải của bài toán đó chính là eigenvector của ma trận hiệp phương sai, hoặc tương đương, các right singular vector trong SVD của dữ liệu đã chuẩn hóa. Đây là một bài tutorial thuần lý thuyết, không có benchmark thực nghiệm, nhưng là nền tảng quan trọng cho nhóm các phương pháp dựa trên eigen-decomposition sẽ gặp lại ở các bài tiếp theo của lộ trình (randomized SVD ở math 04, spectral clustering ở math 05). Giới hạn cốt lõi mà bài báo nêu: PCA chỉ xử lý được cấu trúc tuyến tính, phụ thuộc bậc hai; khi dữ liệu có cấu trúc phi tuyến hoặc phân phối không trực giao/không-Gauss, PCA có thể cho kết quả không phản ánh đúng cấu trúc thật của hệ thống.

Nguồn gốc: https://arxiv.org/abs/1404.1100

Bài 04

Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions (2009)

Tác giả: N. Halko (University of Colorado at Boulder), P. G. Martinsson (University of Colorado at Boulder), J. A. Tropp (California Institute of Technology)
Venue / arXiv: arXiv:0909.4061 (v2, 14/12/2010); paper không nêu venue hội nghị/tạp chí trong phần văn bản chính (chỉ có các trích dẫn tới tạp chí SIAM khác trong danh mục tham khảo, không phải venue của chính bài này)
Nguồn: https://arxiv.org/abs/0909.4061
Chủ đề: Đại số tuyến tính số ngẫu nhiên hoá: tính xấp xỉ ma trận hạng thấp (SVD, phân tích giá trị riêng, QR, interpolative decomposition) bằng lấy mẫu/chiếu ngẫu nhiên
Vị trí trong lộ trình: Bài số 4 trong nhóm 9 bài toán nền (math 01-09). Đọc sau math 03 (PCA/SVD cổ điển, FILE ĐÃ TỒN TẠI), vì bài này chính là cách tính nhanh, ngẫu nhiên hoá cho đúng bài toán SVD/eigen-decomposition đã trình bày ở math 03 khi ma trận quá lớn để tính SVD đầy đủ. Đứng trước math 05 (spectral clustering), nơi cũng cần tính eigenvector của ma trận lớn (Laplacian) một cách hiệu quả.

> Ghi chú này được viết lại bằng tiếng Việt dựa trên toàn văn bài báo gốc (chuyển PDF sang text, 4271 dòng, khoảng 74 trang), không phải bản dịch từng câu. Đây là bài DÀI nhất trong nhóm 9 bài, nên ghi chú tập trung vào phần cốt lõi: toàn bộ Part I (Giới thiệu, mục 1-3) và Part II (Thuật toán, mục 4-7, bao gồm cả mục 7 ”Numerical examples” với các thí nghiệm số thật), còn Part III (Lý thuyết, mục 8-11) chỉ lấy các định lý chính (phát biểu kết quả, không chép lại chi tiết từng bước chứng minh kỹ thuật dài của phần lý thuyết độ tập trung đo). Ngày tải bài báo: 09/11/2026.

1. Bối cảnh

Các phép phân tích ma trận như QR, SVD và phân tích trị riêng là công cụ chung cho nhiều bài toán khoa học và kỹ thuật. Bài báo dẫn nhận xét của G. W. Stewart về ảnh hưởng của cách tiếp cận này, rồi nêu bốn khó khăn khi áp dụng các thuật toán cổ điển cho dữ liệu lớn:

1. Ma trận trong các ứng dụng hiện đại (data mining) có kích thước ”khủng”.
2. Dữ liệu thường bị thiếu hoặc không chính xác, nên việc tính một phân tích chính xác tuyệt đối tới sai số máy là lãng phí tính toán.
3. Chi phí truyền dữ liệu (data transfer) ngày càng quan trọng hơn số phép toán (flop); các kỹ thuật cần ít lượt đọc dữ liệu (pass) hơn có thể nhanh hơn trong thực tế dù số flop tương đương hoặc nhiều hơn.
4. Kiến trúc phần cứng (ví dụ GPU) đòi hỏi thuật toán số phải thích nghi.

Mục tiêu của paper là chứng minh rằng các thuật toán ngẫu nhiên (randomized algorithms) giải quyết tốt các vấn đề này: nhanh hơn, ổn định hơn (robust hơn) so với thuật toán tất định cổ điển trong nhiều trường hợp, và cho phép đánh đổi độ chính xác với tốc độ. Bài báo nhấn mạnh ngay từ đầu (Remark 1.1): khác với phương pháp Monte Carlo cổ điển (độ chính xác thấp, không chắc chắn), các thuật toán được trình bày ở đây ”tương đối không nhạy với chất lượng của nguồn ngẫu nhiên” và cho kết quả chính xác cao, với xác suất thất bại có thể làm nhỏ tuỳ ý (ví dụ dưới 10^{-15}) mà chi phí tính toán tăng thêm không đáng kể.

Về cấu trúc: paper có 3 phần chính theo cách paper tự chia (”Outline of paper”, mục 1.7): Part I là Giới thiệu (mục 1-3, gồm tổng quan, lịch sử/công trình liên quan, và kiến thức nền đại số tuyến tính); Part II là Thuật toán (mục 4-7, trình bày chi tiết các thuật toán cho Stage A và Stage B, kèm mục 7 là các thí nghiệm số thật); Part III là Lý thuyết (mục 8-11, phân tích sai số chặt chẽ bằng cả đại số tuyến tính tất định và xác suất). Ba tác

giả thuộc University of Colorado at Boulder (Halko, Martinsson) và Caltech (Tropp), được tài trợ bởi NSF và ONR.

Các từ khóa paper tự liệt kê trong abstract cho thấy phạm vi rộng: dimension reduction, eigenvalue decomposition, interpolative decomposition, Johnson-Lindenstrauss lemma, matrix approximation, parallel algorithm, pass-efficient algorithm, principal component analysis, randomized algorithm, random matrix, rank-revealing QR factorization, singular value decomposition, streaming algorithm. Paper tự mô tả mình là một bài khảo sát (survey) kết hợp với mở rộng các kết quả có sẵn trong văn liệu ("This work surveys and extends recent research").

2. Ý tưởng chính

Ý tưởng trung tâm là chia bài toán tính một phân tích hạng thấp của ma trận A (kích thước $m \times n$) thành hai giai đoạn, gọi là Stage A và Stage B:

- **Stage A:** tính một ma trận Q có các cột trực chuẩn (orthonormal), sao cho $A \approx QQ^*A$. Nói cách khác, cột của Q tạo thành một cơ sở xấp xỉ cho "range" (không gian cột) của A, và ta muốn số cột của Q càng ít càng tốt mà vẫn giữ độ chính xác.
- **Stage B:** dùng Q đã có để tính một phân tích chuẩn (SVD, QR, eigen-decomposition) của A bằng các phương pháp tất định cổ điển, áp dụng lên ma trận đã "thu nhỏ" qua Q.

Điểm mới cốt lõi của paper nằm ở Stage A: thay vì dùng các thuật toán tất định (ví dụ rank-revealing QR hay Krylov subspace), paper dùng lấy mẫu ngẫu nhiên (random sampling) để tìm cơ sở này một cách rất hiệu quả. Trực giác (mục 1.3.2 của paper): nếu A có hạng chính xác k, và ta lấy k vector ngẫu nhiên $\omega^{(i)}$ rồi tính $y^{(i)} = A\omega^{(i)}$, thì vì tính ngẫu nhiên, tập $\{\omega^{(i)}\}$ "nhìn chung" (generic) độc lập tuyến tính và không rơi vào không gian null của A, nên tập $\{y^{(i)}\}$ cũng độc lập tuyến tính và trải khắp toàn bộ range của A. Chỉ cần trực chuẩn hóa các $y^{(i)}$ này là có được cơ sở cho range của A.

Khi $A = B + E$ với B là phần hạng-k "tín hiệu" cần giữ và E là nhiễu nhỏ, lấy đúng k mẫu có thể không đủ (vì E làm lệch hướng các vector mẫu ra khỏi range của B). Giải pháp: lấy thêm một số mẫu dư ra, gọi là tham số lấy dư (oversampling) p, tức lấy $k + p$ mẫu thay vì k. Paper chỉ ra xác suất thất bại giảm siêu mũ (superexponentially) theo p, nên trong thực hành chỉ cần $p = 5$ hoặc $p = 10$ đã cho kết quả rất tốt.

Về hiệu quả tính toán, paper tóm tắt ba kịch bản (mục 1.4), trong đó bài toán mô hình là tìm k thành phần SVD chủ đạo của ma trận $m \times n$:

Kịch bản	Thuật toán cổ điển	Thuật toán ngẫu nhiên
Ma trận dày, vừa bộ nhớ	$O(mnk)$ flop (rank-revealing QR)	$O(mn\log(k))$ flop (dùng ma trận ngẫu nhiên có cấu trúc)
Ma trận thưa/có cấu trúc (tích ma trận-vector nhanh)	Krylov subspace (Lanczos/Arnoldi), chi phí $\sim k\,T_{mult} + (m+n)k^2$	Cùng bậc chi phí $O(k\,T_{mult} + (m+n)k^2)$ nhưng ổn định hơn, dễ song song hóa
Ma trận quá lớn, lưu ngoài bộ nhớ nhanh (out-of-core/streaming)	cần $O(k)$ lượt đọc dữ liệu	chỉ cần một số hằng lượt đọc, có thể chỉ 1 lượt

Đây là ý tưởng mang tính "hạ tầng tính toán": không thay đổi đích đến (vẫn là SVD/QR/eigen-decomposition xấp xỉ), mà thay đổi cách tìm subspace quan trọng ở Stage A để tận dụng kiến trúc máy tính hiện đại (nhân ma trận song song, ít lượt đọc dữ liệu).

3. Phương pháp chi tiết

3.1. Hai bài toán hình thức hóa

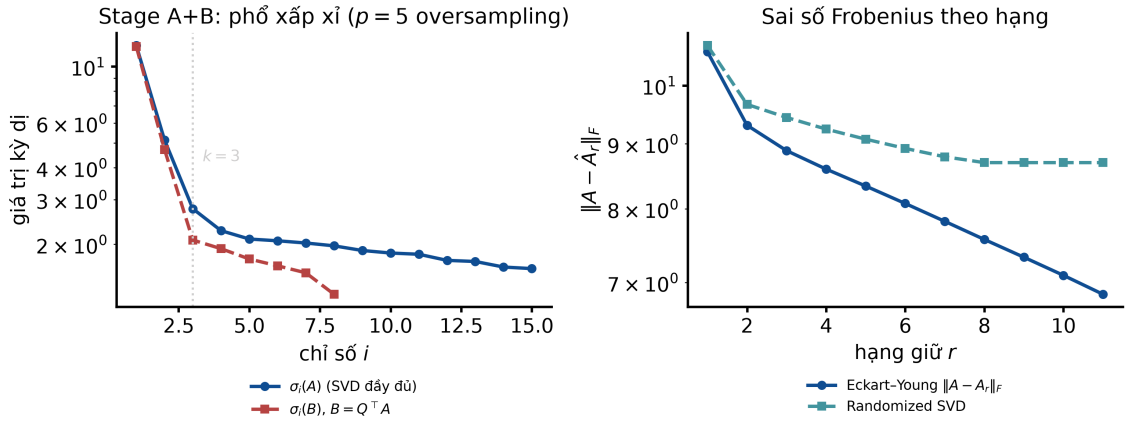
Paper hình thức hóa hai biến thể của bài toán xấp xỉ hạng thấp (mục 1.3.1):

- **Fixed-precision problem:** cho ma trận A và sai số cho phép ε, tìm Q với số cột $k = k(\varepsilon)$ ít nhất sao cho $\|A - QQ^*A\| \le \varepsilon$ (chuẩn toán tử ℓ_2). Theo lý thuyết SVD cổ điển, nghiệm tối ưu của bài toán này chính là σ_{j+1} (giá trị kỳ dị thứ $j+1$) khi Q chứa j vector kỳ dị trái chủ đạo.
- **Fixed-rank problem:** cho trước hạng đích k và tham số lấy dư p, tìm Q với $k+p$ cột trực chuẩn sao cho $\|A - QQ^*A\| \approx \min_{\text{rank}(X)\le k} \|A - X\|$.

Hai bài toán này có thể quy đổi qua lại: thuật toán cho fixed-rank có thể chỉnh để giải fixed-precision bằng cách tăng dần số mẫu và theo dõi sai số còn lại (mục 4.3-4.4).

3.2. Proto-Algorithm và Randomized Range Finder (Algorithm 4.1)

Phiên bản đơn giản nhất của thuật toán Stage A (paper gọi là ”Proto-Algorithm”, sau đó hình thức hóa thành Algorithm 4.1 ”Randomized Range Finder”):



Randomized SVD: phổ của B=Q⊤A xấp xỉ σᵢ(A) với oversampling p=5; sai số Frobenius gần Eckart–Young. (minh hoạ, tự tính)

1. Vẽ một ma trận kiểm thử (test matrix) ngẫu nhiên Gauss Ω kích thước $n \times \ell$ (với $\ell = k + p$).
2. Tính $Y = A\Omega$ (kích thước $m \times \ell$).
3. Trực chuẩn hóa các cột của Y (ví dụ qua phân tích QR, $Y = QR$) để được Q.

Số flop cần cho thuật toán này: $T_{basic} \sim \ell n T_{rand} + \ell\, T_{mult} + \ell^2 m$, trong đó T_{rand} là chi phí sinh một số ngẫu nhiên Gauss, T_{mult} là chi phí nhân A với một vector. Paper nhận xét thực nghiệm: hiệu năng của thuật toán phụ thuộc rất ít vào chất lượng bộ sinh số ngẫu nhiên; khâu trực chuẩn hóa (bước 3) nên dùng Gram-Schmidt có trực chuẩn hóa kép (double orthogonalization) hoặc phép phản xạ Householder/phép quay Givens vì cột của Y thường gần phụ thuộc tuyến tính (ill-conditioned).

3.3. Ước lượng sai số và thuật toán thích nghi (Algorithm 4.2)

Để giải bài toán fixed-precision (khi hạng số k chưa biết trước), paper phát triển một ước lượng sai số xác suất, rẻ về tính toán, dựa trên Lemma 4.1: với B là ma trận thực $m \times n$, r là số nguyên dương, $\alpha > 1$, và $\{\omega^{(i)}\}_{i=1}^{r}$ là các vector Gauss chuẩn độc lập, thì

$$\|B\| \le \alpha \sqrt{\frac{2}{\pi}} \max_{i=1,\dots,r} \|B\omega^{(i)}\|$$

trừ với xác suất α^{-r}. Áp dụng với $B = (I - QQ^*)A$ và $\alpha = 10$, ta có công cụ kiểm tra xem Q hiện tại đã đủ tốt chưa mà chỉ cần vài tích ma trận-vector, không cần tính lại sai số đầy đủ. Dựa trên đó, Algorithm 4.2

("Adaptive Randomized Range Finder") xây dựng Q tăng dần theo từng cột, dừng khi r vector mẫu liên tiếp có chuẩn đủ nhỏ, đạt sai số mong muốn với xác suất ít nhất $1 - \min\{m, n\} \cdot 10^{-r}$.

3.4. Lặp luỹ thừa (power iteration) khi phổ giá trị kỳ dị suy giảm chậm

Nhược điểm của Algorithm 4.1: nếu các giá trị kỳ dị của A suy giảm chậm (phổ "phẳng"), các vector kỳ dị tương ứng giá trị nhỏ vẫn "chen vào" mẫu ngẫu nhiên và làm giảm chất lượng cơ sở Q. Giải pháp (mục 4.5, Algorithm 4.3 "Randomized Power Iteration"): áp dụng thuật toán cho ma trận $B = (AA^*)^q A$ với q là một số nguyên nhỏ (thường $q = 1$ hoặc 2), thay vì áp dụng trực tiếp cho A. Ma trận B có cùng vector kỳ dị với A nhưng giá trị kỳ dị suy giảm nhanh hơn rất nhiều: $\sigma_j(B) = \sigma_j(A)^{2q+1}$. Heuristic của paper: nếu thuật toán cơ bản cho sai số trong khoảng C lần tối ưu, thì phiên bản lũy thừa cho sai số trong khoảng $C^{1/(2q+1)}$ lần tối ưu, tức khoảng cách tới tối ưu giảm theo cấp số mũ khi q tăng.

Paper cảnh báo (Remark 4.3): khi thực hiện trong số học dấu phẩy động, việc nhân lặp A, A^* nhiều lần sẽ làm mất hết thông tin về các mode kỳ dị nhỏ do lỗi làm tròn (nếu độ chính xác máy là μ, thông tin của các giá trị kỳ dị nhỏ hơn $\mu^{1/(2q+1)}\|A\|$ sẽ bị "xóa"). Khắc phục bằng cách trực chuẩn hóa lại sau mỗi lần nhân A hoặc A^*, cho ra Algorithm 4.4 ("Randomized Subspace Iteration"), được paper khuyến nghị dùng trong thực hành vì chi phí tương đương Algorithm 4.3 nhưng chính xác hơn nhiều trong số học thực tế.

3.5. Ma trận ngẫu nhiên có cấu trúc (SRFT) để tăng tốc

Nút nghẽn tính toán của Algorithm 4.1 là tích $A\Omega$. Nếu Ω là ma trận Gauss chuẩn, chi phí này là $O(mn\ell)$, không hơn gì rank-revealing QR cổ điển. Paper đề xuất dùng ma trận ngẫu nhiên có cấu trúc, ví dụ Subsampled Random Fourier Transform (SRFT):

$$\Omega = \sqrt{\frac{n}{\ell}}\, DFR$$

với D là ma trận đường chéo ngẫu nhiên (các phần tử trên vòng tròn đơn vị phức), F là biến đổi Fourier rời rạc unitary $n \times n$, R là ma trận chọn ngẫu nhiên ℓ trong n cột của ma trận đơn vị. Vì có thể tính $A\Omega$ qua FFT lấy mẫu con (subsampled FFT), chi phí giảm xuống $O(mn \log(\ell))$. Đây là Algorithm 4.5 ("Fast Randomized Range Finder").

3.6. Stage B: từ cơ sở Q tới phân tích chuẩn

Khi đã có Q với $\|A - QQ^*A\| \le \varepsilon$, paper trình bày nhiều cách để lấy ra SVD/eigen-decomposition cụ thể:

- **Direct SVD (Algorithm 5.1):** tính $B = Q^*A$ (ma trận nhỏ $\ell \times n$), lấy SVD của B là $B = \tilde{U}\Sigma V^*$, rồi $U = Q\tilde{U}$. Sai số không suy giảm so với sai số của Q: $\|A - U\Sigma V^*\| \le \varepsilon$.
- **Row extraction (Algorithm 5.2):** dùng interpolative decomposition của Q để trích ra k hàng của chính A mà không cần tính tích ma trận-ma trận nào, nhanh hơn nhưng kém chính xác hơn Algorithm 5.1.
- **Biến thể cho ma trận Hermitian/positive semidefinite (Algorithm 5.3-5.5, trong đó có phương pháp Nyström)** và **thuật toán một lượt đọc (single-pass, Algorithm 5.6)** cho trường hợp không thể quay lại đọc A lần hai (dữ liệu stream).

Tổng hợp Stage A (với power iteration) + Stage B (Direct SVD) cho ra "Prototype for Randomized SVD" mà paper trình bày ngay từ phần giới thiệu:

1. Sinh ma trận kiểm thử Gauss Ω kích thước $n \times 2k$.
2. $Y = (AA^*)^q A\Omega$ (nhân xen kẽ A và A^*).
3. Trực chuẩn hóa Y để được Q.
4. $B = Q^*A$.
5. SVD của B: $B = \tilde{U}\Sigma V^*$.
6. $U = Q\tilde{U}$.

Chi phí: $T_{randSVD} = (2q+2)k\,T_{mult} + O(k^2(m+n))$.

3.7. Ví dụ tự tính: minh hoạ cơ chế "lấy mẫu ngẫu nhiên tìm range" trên ma trận hạng 1

Để minh họa cụ thể trực giác ở mục 1.3.2 của paper (nếu A có hạng chính xác k, lấy k mẫu đủ để trải khắp range của A), xét ma trận hạng 1 đơn giản, tự chọn (không lấy từ paper), kích thước 3×2:

$$A = \begin{pmatrix} 1 & 1 \\ 2 & 2 \\ 2 & 2 \end{pmatrix}$$

Đây chính là $A = xy^T$ với $x = (1,2,2)^T$, $y = (1,1)^T$, nên $\text{rank}(A) = 1$ và $\text{range}(A) = \text{span}\{x\}$. Để tính tay dễ hiểu, thay cho vector Gauss thật, dùng hai vector kiểm thử cụ thể $\omega^{(1)} = (1,0)^T$ và $\omega^{(2)} = (0,1)^T$ (đóng vai trò minh họa cơ chế, không phải mẫu Gauss thật theo đúng Algorithm 4.1):

$$y^{(1)} = A\omega^{(1)} = (1,2,2)^T = x, \qquad y^{(2)} = A\omega^{(2)} = (1,2,2)^T = x$$

Cả hai mẫu đều cho đúng x (vì A hạng 1, mọi cột của A đều là một hằng số nhân với x). Trực chuẩn hóa $y^{(1)}$: $q = x/\|x\| = (1,2,2)^T/3 = (1/3,\ 2/3,\ 2/3)^T$. Đây chính xác là một cơ sở trực chuẩn cho range của A, đạt được chỉ với một mẫu (vì hạng đúng là 1, không cần lấy dư $p > 0$ với ví dụ "sạch", không nhiễu này). Điều này khớp chính xác với phát biểu của paper ở mục 1.3.2: "to produce an orthonormal basis for the range of A, we just need to orthonormalize the sample vectors" khi A có hạng chính xác k (tự tính, minh họa cho trường hợp lý tưởng không nhiễu; paper tự nói rõ trường hợp có nhiễu $A = B + E$ cần lấy dư thêm p mẫu, không được minh họa số cụ thể ở đây vì cần ma trận phức tạp hơn).

4. Thí nghiệm và kết quả chính

4.1. Kết quả lý thuyết chính

Paper phát biểu ngay từ phần giới thiệu hai định lý trung tâm (các bản đơn giản hóa của Theorem 10.6 và Corollary 10.10 ở Part III):

Theorem 1.1 (sai số trung bình của Proto-Algorithm với ma trận Gauss): với A là ma trận thực $m \times n$, hạng đích $k \geq 2$, tham số lấy dư $p \geq 2$, $k + p \leq \min\{m,n\}$, thực hiện proto-algorithm với ma trận kiểm thử Gauss chuẩn để được Q ($m \times (k+p)$ cột trực chuẩn), thì

$$\mathbb{E}\|A - QQ^*A\| \leq \left(1 + \frac{4\sqrt{k+p}}{p-1}\sqrt{\min\{m,n\}}\right)\sigma_{k+1}$$

trong đó σ_{k+1} là giá trị kỳ dị thứ $k+1$ của A (sai số nhỏ nhất có thể đạt được bằng bất kỳ cơ sở nào). Với $p = 5$, với xác suất ít nhất $1 - 6p^{-p}$, sai số thực tế nằm trong khoảng $\left(1 + 11\sqrt{k+p}\sqrt{\min\{m,n\}}\right)\sigma_{k+1}$, cho thấy chỉ cần lấy dư $p = 5$ là đủ dùng được trong thực hành.

Theorem 1.2 (Randomized SVD với power iteration): với ma trận A, số mũ q, hạng đích k ($2 \leq k \leq 0.5\min\{m,n\}$), chạy thuật toán Randomized SVD (phần 3.6 ở trên) để được phân tích hạng $2k$, $U\Sigma V^*$, thì

$$\mathbb{E}\|A - U\Sigma V^*\| \leq \left[1 + 4\sqrt{\frac{2\min\{m,n\}}{k-1}}\right]^{1/(2q+1)}\sigma_{k+1}$$

Đây là kết quả mà bài báo lưu ý "is new" (chưa có trong các công trình trước): hệ số nhân trong ngoặc bị đẩy về 1 theo cấp số mũ khi q tăng.

Phần lý thuyết chi tiết (Part III, mục 9) cung cấp một định lý tất định tổng quát hơn (Theorem 9.1), chỉ phụ thuộc vào tương tác giữa ma trận kiểm thử Ω và các vector kỳ dị phải của A:

$$\|(I - P_Y)A\|^2 \leq \|\Sigma_2\|^2 + \|\Sigma_2\Omega_2\Omega_1^\dagger\|^2$$

(với Σ_1, Σ_2 là khối giá trị kỳ dị lớn/nhỏ, Ω_1, Ω_2 là Ω biểu diễn trong hệ trục kỳ dị phải). Theorem 9.2 chỉ ra power scheme cải thiện sai số theo lũy thừa $1/(2q+1)$; Theorem 9.3 chỉ ra bước cắt (truncation) về đúng hạng k chỉ làm tăng sai số thêm tối đa σ_{k+1}.

4.2. Thí nghiệm số thật (mục 7 của paper, cuối Part II)

Khác với math 01/03 (lý thuyết thuần, không benchmark), paper này có mục 7 ”Numerical examples” với **nhiều thí nghiệm số thật**, chạy trên các ma trận cụ thể:

- **Toán tử tích phân Laplace (mục 7.1):** ma trận 200×200 rời rạc hóa một toán tử lớp đơn (single-layer potential) cho phương trình Laplace, phổ giá trị kỳ dị suy giảm nhanh. Chạy Algorithm 4.2 trong Matlab v6.5. Hình 7.2 cho thấy sai số thực tế e_ℓ rất gần sai số tối thiểu lý thuyết $\sigma_{\ell+1}$, và ước lượng sai số rẻ f_ℓ luôn là chặn trên của sai số thực, bảo thủ (pessimistic) khoảng 10 lần. Lặp lại 2000 lần (Hình 7.3) xác nhận hành vi này ổn định qua các lần thử.
- **Ma trận Laplace rời rạc trên lưới điện trở (mục 7.1 tiếp):** ma trận 1596×532 định nghĩa ngầm (chỉ truy cập qua tích ma trận-vector, mỗi lần cần giải một hệ thưa $\sim 140{,}000 \times 140{,}000$), cho kết quả định tính tương tự (Hình 7.4).
- **Ma trận Laplacian đồ thị từ ảnh (mục 7.2):** từ ảnh grayscale 95×95 pixel, xây ma trận trọng số 9025×9025 (dùng kernel Gauss trên các patch 5×5), rồi Laplacian chuẩn hóa. Dùng power scheme Algorithm 4.3 với $q = 0, 1, 2, 3$: Hình 7.5 cho thấy $q = 0$ (không lặp lũy thừa) cho kết quả xấp xỉ eigenvalue rất kém, nhưng $q = 1$ đã cải thiện rõ rệt, các eigenvalue lớn đã khá chính xác.
- **Eigenfaces (mục 7.3):** ma trận $98{,}304 \times 7254$ từ 7254 ảnh khuôn mặt (cơ sở dữ liệu FERET), cần 5.4 GB nếu lưu dạng số thực kép, không vừa bộ nhớ nhanh. Dùng power scheme pass-efficient, ước lượng sai số qua kỹ thuật Remark 4.1 (vì không thể tính sai số thật ở quy mô này). Hình 7.6: $q = 0$ cho kết quả rất kém, cải thiện nhanh khi q tăng, hầu như không cải thiện thêm sau 3-5 lượt đọc dữ liệu.
- **So sánh tốc độ với ma trận ngẫu nhiên có cấu trúc (mục 7.4, Bảng 7.1):** so sánh 4 phương pháp (`direct`: rank-revealing QR cổ điển, `gauss`: Algorithm 4.1 với Gauss, `srft`: Algorithm 4.1 với SRFT, `svd`: SVD đầy đủ LAPACK `dgesdd`) trên ma trận Gauss ngẫu nhiên $n \times n$, đo thời gian thật (giây) trên một máy ”office desktop” Pentium IV 3.2 GHz, 2 GB RAM. Một vài số liệu trích từ Bảng 7.1 (tự chọn trích, số liệu thật của paper): với $n = 4096$, $\ell = 320$: `direct` $= 5.00 \times 10^1$ giây, `gauss` $= 2.41 \times 10^1$ giây, `srft` $= 1.21 \times 10^1$ giây; SVD đầy đủ cho $n = 4096$ tốn 6.90×10^2 giây. Hình 7.7 (acceleration factor) cho thấy SRFT thường tăng tốc mạnh so với `direct`, Gauss tăng tốc vừa phải.

Paper tự nhấn mạnh: các ví dụ ở mục 7 chỉ ở quy mô vừa phải; ”Computational examples illustrating truly large data matrices have been reported elsewhere” (dẫn một công trình khác [112]), tức benchmark ở quy mô siêu lớn không nằm trong chính bài báo này.

5. Phân tích / ablation

Ảnh hưởng của số bước lặp lũy thừa q (ablation có sẵn trong paper): cả hai thí nghiệm ở mục 7.2 (Laplacian đồ thị) và 7.3 (Eigenfaces) đều là một dạng ablation theo $q = 0, 1, 2, 3$. Kết luận nhất quán: $q = 0$ (không lặp lũy thừa) luôn cho kết quả kém khi phổ giá trị kỳ dị suy giảm chậm; chỉ cần $q = 1$ hoặc $q = 2$ đã cải thiện mạnh; tăng q quá mức không mang lại lợi ích đáng kể thêm (Eigenfaces: ”hầu như không cải thiện thêm sau 3-5 lượt đọc”). Đây là bằng chứng thực nghiệm trực tiếp cho Theorem 9.2 (sai số giảm theo lũy thừa $1/(2q+1)$ khi q tăng).

Phân tích sâu hơn Bảng 7.1: SRFT không phải luôn thắng. Khi đọc kỹ toàn bộ Bảng 7.1, có một quan sát quan trọng mà phần văn bản chính của paper không nhấn mạnh rõ: ở $n = 2048$, $\ell = 1280$ (tức ℓ gần bằng n, khoảng 62%), `srft` mất 3.13×10^2 giây, CHẬM HƠN cả `direct` (7.97×10^1 giây)! Tương tự ở $n = 4096$, $\ell = 1280$: `srft` $= 4.03 \times 10^2$ giây, cũng chậm hơn `direct` (2.40×10^2 giây). Đây là **quan sát của người ghi chú dựa trên số liệu thật trong Bảng 7.1 của paper**, không phải khẳng định mà bài báo nêu rõ bằng lời: khi tỉ lệ ℓ/n đủ lớn (ở đây ℓ chiếm hơn một nửa n), phương pháp SRFT (được quảng cáo là nhanh hơn nhờ FFT lấy mẫu con) lại trở nên chậm hơn phương pháp tất định cổ điển. Điều này cho thấy tuyên bố tổng quát trong abstract ("randomized algorithms... beat its classical competitors in terms of accuracy, speed, and robustness") cần hiểu có điều kiện: lợi thế tốc độ của SRFT chủ yếu xảy ra khi $\ell \ll n$ (hạng đích nhỏ hơn nhiều so với kích thước ma trận), đúng với giả định "numerical rank thấp" mà toàn bộ phương pháp dựa vào, nhưng không phải lúc nào cũng đúng.

Ước lượng sai số luôn bảo thủ, không chỉ đúng trung bình: Hình 7.3 (2000 lần thử) cho thấy ước lượng f_ℓ luôn là chặn trên của sai số thật e_ℓ, lệch khoảng hệ số 10, đúng như thiết kế lý thuyết (hệ số an toàn $\alpha = 10$ trong Lemma 4.1). Đây là một đánh đổi thiết kế có chủ ý: người dùng "trả giá" bằng việc có thể lấy nhiều mẫu hơn mức tối thiểu cần thiết, để đổi lại một cam kết xác suất chắc chắn hơn.

So sánh phân phối ma trận kiểm thử (mục 7.4, Hình 7.8): trên cùng ma trận Laplace 200×200, so 4 phân phối cho Ω (Gauss, Ortho tức ma trận trực chuẩn ngẫu nhiên đều, SRFT, GSRFT) qua 100.000 lần thử. Kết quả: SRFT thực ra hơi tốt hơn Gauss trong ví dụ cụ thể này; SRFT và GSRFT cho sai số gần như giống nhau; Ortho chỉ tốt hơn rõ rệt khi ma trận kiểm thử gần vuông (không tốt hơn đáng kể trong 3/4 trường hợp còn lại). Đây là bằng chứng thực nghiệm cho thấy lựa chọn phân phối ngẫu nhiên cụ thể ít quan trọng hơn so với việc có lấy mẫu ngẫu nhiên hay không.

6. Giới hạn

Các giới hạn **bài báo nêu rõ**:

- Thuật toán cơ bản (Algorithm 4.1, không lặp lũy thừa) hoạt động kém khi phổ giá trị kỳ dị của A suy giảm chậm; bắt buộc phải dùng power iteration (Algorithm 4.3/4.4), làm tăng số lượt đọc dữ liệu lên $2q + 1$ lần.
- Power iteration thực hiện trong số học dấu phẩy động dễ mất thông tin về các mode kỳ dị nhỏ do lỗi làm tròn (Remark 4.3); phải trực chuẩn hóa lại giữa các bước (Algorithm 4.4) để khắc phục, với chi phí gần như không đổi nhưng cần thêm bước triển khai.
- Ma trận ngẫu nhiên có cấu trúc (SRFT) không thích nghi tốt với bài toán fixed-precision, vì mẫu thường phải tính hàng loạt (Remark 4.4); và paper tự nói rõ "it is not currently known whether such an iterative scheme can be accelerated to $O(mn \log(k))$ complexity using 'fast' random transforms such as the SRFT" (Remark 4.7, câu hỏi mở tại thời điểm viết bài).
- Krylov subspace method (đối thủ cổ điển cho ma trận thưa/có cấu trúc) "đôi khi" (sometimes) vẫn cho kết quả chính xác hơn phương pháp ngẫu nhiên với cùng ngân sách flop, mặc dù phương pháp ngẫu nhiên ổn định hơn và dễ song song hóa hơn (mục 1.4.2, bài báo nêu đây không phải phương pháp ngẫu nhiên luôn thắng tuyệt đối).
- Row extraction (Algorithm 5.2) nhanh hơn Direct SVD (Algorithm 5.1) nhưng kém chính xác hơn; paper khuyến nghị dùng Algorithm 5.1 trừ khi tốc độ quan trọng hơn (mục 5.2).
- Các thí nghiệm số trong chính bài báo chỉ ở quy mô vừa (lớn nhất là ma trận $98{,}304 \times 7254$, 5.4 GB); các khẳng định về "truly massive data sets" dựa một phần vào trích dẫn các công trình khác, không phải thí nghiệm trực tiếp trong bài này.

Quan sát **của người ghi chú** (không phải do paper tự phê bình bằng lời, nhưng là đặc điểm khách quan rút ra từ chính dữ liệu của paper, như đã nêu ở mục 5): trong Bảng 7.1, khi ℓ/n lớn (ví dụ $\ell = 1280$, $n = 2048$ hoặc 4096), phương pháp `srft` chậm hơn cả phương pháp tất định `direct`, một trường hợp mà tuyên bố tổng quát "randomized beats classical" của paper không áp dụng.

7. Liên hệ với các bài khác

- **math 03 (A Tutorial on Principal Component Analysis, Shlens, FILE ĐÃ TỒN TẠI):** bài này mở rộng trực tiếp lời giải SVD của PCA trình bày ở math 03. Khi ma trận dữ liệu X quá lớn để tính SVD đầy đủ, các thuật toán ở đây (Randomized Range Finder, Randomized SVD) cho một cách tính PCA xấp xỉ nhanh hơn nhiều, với sai số kiểm soát được theo Theorem 1.1/1.2.
- **math 05 (A Tutorial on Spectral Clustering, von Luxburg):** cần tính các eigenvector của ma trận Laplacian của đồ thị similarity. Thí nghiệm ở chính mục 7.2 của paper này (ma trận Laplacian đồ thị 9025×9025 từ ảnh) là một ví dụ trực tiếp của đúng loại bài toán mà math 05 cần giải, chỉ khác là ở đây Laplacian xây từ patch ảnh cho khử nhiễu, còn math 05 xây Laplacian từ similarity graph cho phân cụm.
- **opt 14 (Shampoo: Preconditioned Stochastic Tensor Optimization):** liên hệ gián tiếp, suy luận của người ghi chú (không phải nội dung của paper này hay của Shampoo nêu rõ): Shampoo cần tính nghịch đảo căn bậc hai của các ma trận preconditioner lớn cho mỗi lớp; về nguyên tắc, các kỹ thuật xấp xỉ hạng thấp ngẫu nhiên ở đây có thể dùng để giảm chi phí tính toán này, nhưng đây là một liên hệ khái niệm, không phải điều hai bài báo trực tiếp thảo luận.
- **math 19 (Measuring the Intrinsic Dimension of Objective Landscapes):** cùng mạch ”tìm hạng/chiều hiệu dụng thấp của một đối tượng lớn” như ở math 03, nhưng math 19 áp dụng cho không gian tham số mạng neural bằng một phương pháp hoàn toàn khác (random subspace training), không dùng SVD ngẫu nhiên.

8. Gợi ý áp dụng

Randomized SVD hữu ích khi cần xấp xỉ hạng thấp của ma trận lớn hoặc chỉ có thể thực hiện phép nhân với ma trận. Tốc độ thực tế phụ thuộc kích thước, phổ giá trị kỳ dị và số lượt lặp, nên cần so sánh với SVD trực tiếp trên bài toán cụ thể.

9. Thí nghiệm gợi ý

2. **Ablation theo số bước lặp lũy thừa q:** lặp lại thí nghiệm kiểu Hình 7.5/7.6 của paper nhưng trên một weight matrix thật của mô hình (thay cho ảnh/Eigenfaces), đo sai số xấp xỉ giá trị kỳ dị lớn nhất với $q = 0, 1, 2, 3$. Kỳ vọng: $q = 0$ cho kết quả rõ ràng kém hơn, cải thiện mạnh ngay ở $q = 1$, giống mẫu hình quan sát được ở hai thí nghiệm thật của paper.
3. **Ablation theo tham số lấy dư p:** cố định hạng đích k, thay đổi $p = 0, 5, 10, 20$, đo sai số xấp xỉ. Kỳ vọng: sai số giảm nhanh khi tăng p từ 0 lên 5-10, sau đó lợi ích giảm dần (diminishing returns), khớp khẳng định ”rarely any advantage to select $p > k$” của paper.
4. **Thử nghiệm ma trận ngẫu nhiên có cấu trúc ở quy mô mô hình nhỏ:** so sánh thời gian chạy giữa Gauss và một biến thể cấu trúc đơn giản (ví dụ random sign matrix kết hợp FFT/Hadamard gần giống SRFT) trên một ma trận kích thước thực tế của mô hình (ví dụ 2048×2048 hoặc hidden size tương tự), với ℓ (hạng đích + lấy dư) ở các tỉ lệ khác nhau so với n (ví dụ $\ell/n = 0.05, 0.3, 0.6$). Kỳ vọng kiểm chứng quan sát ở mục 5: SRFT nhanh khi ℓ/n nhỏ nhưng có thể chậm hơn phương pháp tất định khi ℓ/n lớn.
5. **Kiểm tra độ tin cậy của ước lượng sai số rẻ (Lemma 4.1 kiểu):** với một ma trận đủ nhỏ để tính được sai số thật $\|A - QQ^*A\|$ chính xác, so sánh với ước lượng xác suất rẻ (dùng vài vector dò ngẫu nhiên). Đo tỉ lệ giữa ước lượng và sai số thật qua nhiều lần thử. Kỳ vọng: ước lượng luôn là chặn trên, bảo thủ khoảng một hệ số không đổi, giống quan sát ở Hình 7.3 của paper (hệ số khoảng 10).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / mô tả
Khung 2 giai đoạn	Stage A: tìm Q trực chuẩn với $A \approx QQ^*A$. Stage B: tính phân tích chuẩn từ Q
Proto-Algorithm / Algorithm 4.1	Ω ngẫu nhiên $n \times \ell$; $Y = A\Omega$; $Q =$ cơ sở trực chuẩn của Y
Oversampling	$\ell = k + p$; $p = 5$ hoặc 10 thường đủ dùng
Power iteration	$B = (AA^*)^q A$; $\sigma_j(B) = \sigma_j(A)^{2q+1}$; cần trực chuẩn hóa giữa các bước (Algorithm 4.4) để tránh mất thông tin do làm tròn
SRFT	$\Omega = \sqrt{n/\ell}\, DFR$; cho phép tính $A\Omega$ trong $O(mn \log \ell)$ qua FFT lấy mẫu con
Theorem 1.1	$\mathbb{E}\|A - QQ^*A\| \le \left(1 + \frac{4\sqrt{k+p}}{p-1}\sqrt{\min\{m,n\}}\right)\sigma_{k+1}$
Theorem 1.2 (Randomized SVD)	$\mathbb{E}\|A - U\Sigma V^*\| \le \left[1 + 4\sqrt{2\min\{m,n\}/(k-1)}\right]^{1/(2q+1)} \sigma_{k+1}$
Chi phí dày đặc	Cổ điển $O(mnk)$ so với ngẫu nhiên có cấu trúc $O(mn \log k)$
Ví dụ tự tính (mục 3.7)	A hạng 1 (3×2); mẫu $A\omega$ cho đúng hướng $x = (1,2,2)^T$; trực chuẩn hóa còn $q = (1/3, 2/3, 2/3)^T$, khớp range(A) chỉ với 1 mẫu
Quan sát từ Bảng 7.1 (người ghi chú)	SRFT nhanh khi $\ell \ll n$ nhưng có thể chậm hơn `direct` khi ℓ/n lớn (ví dụ $\ell = 1280$, $n = 2048$ hoặc 4096)

11. Kết luận ngắn

Bài báo trình bày một khung hai giai đoạn để tính xấp xỉ hạng thấp của ma trận lớn: dùng lấy mẫu ngẫu nhiên (random projection) để tìm nhanh một subspace nắm bắt phần lớn ”hành động” của ma trận (Stage A), rồi dùng các phương pháp tất định cổ điển trên ma trận đã thu nhỏ để lấy ra SVD/QR/eigen-decomposition cụ thể (Stage B). Đóng góp chính không chỉ là ý tưởng (đã có từ trước trong một số công trình riêng lẻ) mà là tổng hợp, mở rộng, và đặc biệt là phân tích sai số chặt chẽ (cả tất định và xác suất) cùng một lượng thí nghiệm số thực tế đáng kể trên nhiều loại ma trận khác nhau (toán tử tích phân, Laplacian đồ thị, cơ sở dữ liệu khuôn mặt). Bài học thực hành quan trọng nhất: khi phổ giá trị kỳ dị suy giảm chậm, phải kết hợp power iteration; khi hạng đích k nhỏ hơn nhiều kích thước ma trận, ma trận ngẫu nhiên có cấu trúc (SRFT) mang lại tốc độ vượt trội, nhưng lợi thế này không phải tuyệt đối, như chính số liệu Bảng 7.1 của paper cho thấy ở các tỉ lệ ℓ/n lớn. Đây là bài quan trọng để hiểu cách các ý tưởng ở math 03 (PCA/SVD) có thể mở rộng sang quy mô lớn, và là bước đệm kỹ thuật cho các phương pháp eigen-based khác trong lộ trình (math 05).

Nguồn gốc: https://arxiv.org/abs/0909.4061

Bài 05

A Tutorial on Spectral Clustering (2007)

Tác giả: Ulrike von Luxburg (Max Planck Institute for Biological Cybernetics, Tübingen, Đức)
Venue / arXiv: arXiv:0711.0189 (v1, 01/11/2007); bài báo nêu rõ venue ngay trang đầu: ”This article appears in Statistics and Computing, 17(4), 2007”
Nguồn: https://arxiv.org/abs/0711.0189
Chủ đề: Đại số tuyến tính ứng dụng, phân cụm phổ (spectral clustering) dựa trên eigen-decomposition của ma trận Laplacian của đồ thị tương tự (similarity graph)
Vị trí trong lộ trình: Bài số 5 trong nhóm 9 bài toán nền (math 01-09). Đọc sau math 03 (PCA) và math 04 (randomized SVD), cùng thuộc nhóm con ”phương pháp dựa trên eigenvector/SVD” của batch, nhưng áp dụng eigen-decomposition cho một ma trận khác (Laplacian của đồ thị) thay vì ma trận hiệp phương sai dữ liệu. Đây là bài cuối của nhóm con này, mở đường sang nhóm ”hình học thông tin” ở math 06-07.

> Ghi chú diễn giải bài hướng dẫn phân cụm phổ bằng tiếng Việt. Các ví dụ đánh dấu “tự tính” minh họa công thức và không phải kết quả thực nghiệm của bài gốc.

1. Bối cảnh

Phân cụm chia một tập dữ liệu thành các nhóm sao cho những điểm trong cùng nhóm tương tự nhau theo một tiêu chí đã chọn. So với các phương pháp như k-means hay single linkage, phân cụm phổ (spectral clustering) có thể cho kết quả tốt và sử dụng được các công cụ đại số tuyến tính tiêu chuẩn. Bài hướng dẫn giải thích trực giác và cơ sở toán học của phương pháp này.

Khác với math 03 (PCA tutorial, không có danh mục tài liệu tham khảo), bài này có danh mục tham khảo đầy đủ với hơn 60 trích dẫn, kể cả phần ”Outlook and further reading” (mục 9) tổng kết lịch sử: spectral clustering bắt nguồn từ Donath và Hoffman (1973, dùng eigenvector của ma trận kề để phân hoạch đồ thị) và Fiedler (1973, phát hiện mối liên hệ giữa phân hoạch đồ thị và eigenvector thứ hai của Laplacian), rồi được phổ biến trong cộng đồng machine learning qua các công trình của Shi và Malik (2000), Ng, Jordan và Weiss (2002), Meila và Shi (2001).

Bài báo lưu ý đây là ”a self-contained introduction”, không yêu cầu kiến thức toán nào ngoài đại số tuyến tính cơ bản, nhưng cũng tự nhận không cố gắng tổng kết toàn bộ văn liệu (literature) về spectral clustering vì khối lượng quá lớn. Cấu trúc: mục 2-3 giới thiệu các đối tượng toán học (similarity graph, graph Laplacian); mục 4 trình bày các thuật toán; mục 5-7 giải thích tại sao thuật toán hoạt động theo ba góc nhìn khác nhau (graph cut, random walk, perturbation theory); mục 8 bàn các vấn đề thực hành; mục 9 tổng kết lịch sử và mở rộng.

Ghi chú này đọc toàn bộ văn bản (1866 dòng sau khi chuyển PDF, khoảng 32 trang kể cả tài liệu tham khảo), bao gồm cả các mệnh đề/chứng minh ngắn trong mục 3, 5, 7.

2. Ý tưởng chính

Ta biểu diễn dữ liệu bằng một đồ thị tương tự (similarity graph) $G = (V, E)$, trong đó mỗi điểm dữ liệu là một đỉnh, và cạnh giữa hai đỉnh được gán trọng số bằng độ tương tự giữa hai điểm đó. Bài toán phân cụm được phát biểu lại thành bài toán phân hoạch đồ thị: tìm một phân hoạch sao cho tổng trọng số các cạnh *giữa* các nhóm càng nhỏ càng tốt (điểm khác nhóm thì khác nhau), còn tổng trọng số cạnh *trong* mỗi nhóm càng lớn càng tốt (điểm cùng nhóm thì giống nhau).

Phát biểu bằng công thức, phân hoạch đồ thị tối ưu theo nghĩa ”mincut” (cắt nhỏ nhất) là bài toán dễ giải, nhưng trong thực hành nó thường cho nghiệm tầm thường (cắt tách một đỉnh lẻ ra khỏi phần còn lại). Để tránh điều này, người ta thêm điều kiện ”cân bằng” (các nhóm phải đủ lớn), dẫn đến hai tiêu chí phổ biến:

RatioCut (đo kích thước nhóm bằng số đỉnh) và Ncut, normalized cut (đo kích thước nhóm bằng tổng trọng số cạnh gắn với nhóm, gọi là vol(A)). Cả hai bài toán tối ưu có điều kiện cân bằng này đều là NP-hard.

Spectral clustering chính là một phép **thư giãn liên tục (continuous relaxation)** của hai bài toán NP-hard đó: thay vì tìm một vector chỉ báo rời rạc (chỉ nhận 2 giá trị, biểu thị "thuộc nhóm nào"), ta cho phép vector này nhận giá trị thực bất kỳ. Nghiệm của bài toán thư giãn chính là các eigenvector của một ma trận Laplacian của đồ thị (ứng với các eigenvalue nhỏ nhất). Quy trình thực hành tổng quát: (1) xây dựng đồ thị tương tự, (2) tính ma trận Laplacian, (3) lấy k eigenvector đầu tiên (ứng với k eigenvalue nhỏ nhất), (4) dùng các eigenvector này để "nhúng" mỗi điểm dữ liệu vào một không gian $\mathbb{R}^k$ mới (mỗi điểm trở thành một hàng của ma trận chứa các eigenvector làm cột), (5) chạy k-means trên không gian nhúng đó để lấy cụm cuối cùng.

3. Phương pháp chi tiết

3.1. Đồ thị tương tự (similarity graph)

Cho n điểm dữ liệu $x_1, \dots, x_n$ và độ tương tự $s_{ij} \geq 0$ giữa mỗi cặp. Ký hiệu: $W = (w_{ij})$ là ma trận kề có trọng số (weighted adjacency matrix), $d_i = \sum_j w_{ij}$ là bậc (degree) của đỉnh i, D là ma trận đường chéo chứa các d_i. Với một tập con đỉnh $A \subset V$, paper định nghĩa hai cách đo "kích thước": $|A|$ (số đỉnh) và $\text{vol}(A) = \sum_{i \in A} d_i$ (tổng bậc). $\text{cut}(A, \bar{A}) := \sum_{i \in A, j \in \bar{A}} w_{ij}$ là tổng trọng số các cạnh nối A với phần bù $\bar{A}$.

Paper nêu ba cách xây dựng đồ thị tương tự phổ biến:

- **ε-neighborhood graph:** nối hai điểm nếu khoảng cách giữa chúng nhỏ hơn ε; thường coi là đồ thị không trọng số vì mọi cạnh đã có khoảng cách tương đương.
- **k-nearest neighbor graph:** nối v_i với v_j nếu v_j thuộc k láng giềng gần nhất của v_i (hoặc ngược lại). Vì quan hệ "láng giềng gần nhất" không đối xứng, có hai cách làm đồ thị vô hướng: bỏ qua chiều (kNN graph thông thường) hoặc chỉ giữ cạnh khi cả hai chiều đều thỏa (mutual kNN graph).
- **Fully connected graph:** nối mọi cặp điểm có độ tương tự dương, trọng số bằng chính s_{ij}; thường dùng với hàm Gauss $s(x_i, x_j) = \exp(-\|x_i - x_j\|^2/(2\sigma^2))$, với σ đóng vai trò tương tự ε.

3.2. Ma trận Laplacian của đồ thị

Laplacian không chuẩn hóa (unnormalized):

$$L = D - W$$

Paper chứng minh (Proposition 1) các tính chất quan trọng:

1. Với mọi $f \in \mathbb{R}^n$: $f^\top L f = \frac{1}{2} \sum_{i,j=1}^n w_{ij}(f_i - f_j)^2$ (dạng toàn phương).
2. L đối xứng và bán xác định dương (positive semi-definite).
3. Eigenvalue nhỏ nhất của L là 0, với eigenvector tương ứng là vector hằng $\mathbf{1}$.
4. L có n eigenvalue thực không âm $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$.

Một tính chất quan trọng khác (Proposition 2): **số thành phần liên thông của đồ thị bằng đúng số bội (multiplicity) của eigenvalue** 0, và không gian riêng (eigenspace) ứng với eigenvalue 0 được trải (span) bởi các vector chỉ báo (indicator vector) $\mathbf{1}_{A_1}, \dots, \mathbf{1}_{A_k}$ của từng thành phần liên thông $A_1, \dots, A_k$.

Hai Laplacian chuẩn hóa (normalized):

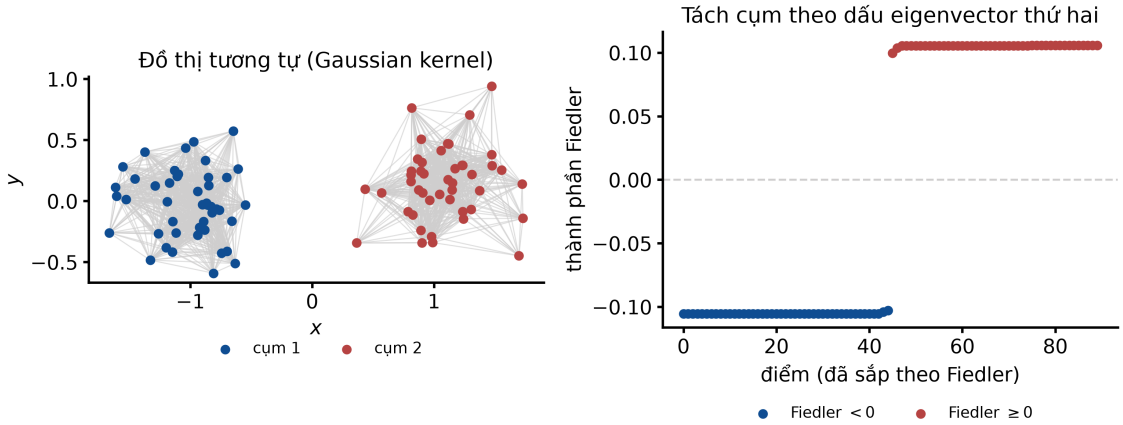
$$L_{sym} := D^{-1/2} L D^{-1/2} = I - D^{-1/2} W D^{-1/2}, \qquad L_{rw} := D^{-1} L = I - D^{-1} W$$

Paper nêu L_{rw} liên hệ chặt với một random walk trên đồ thị (ma trận chuyển $P = D^{-1}W$, $L_{rw} = I - P$). Các tính chất quan trọng (Proposition 3, 4): (a) λ là eigenvalue của L_{rw} với eigenvector u khi và chỉ khi λ là eigenvalue của L_{sym} với eigenvector $w = D^{1/2}u$; (b) 0 là eigenvalue của L_{rw} với eigenvector $\mathbf{1}$ (vector

hằng), còn 0 là eigenvalue của L_{sym} với eigenvector $D^{1/2}\mathbf{1}$ (không phải vector hằng); (c) số thành phần liên thông cũng bằng bội của eigenvalue 0 cho cả hai Laplacian chuẩn hóa, tương tự Proposition 2.

3.3. Ba thuật toán spectral clustering

Paper trình bày ba thuật toán, khác nhau ở loại Laplacian dùng:



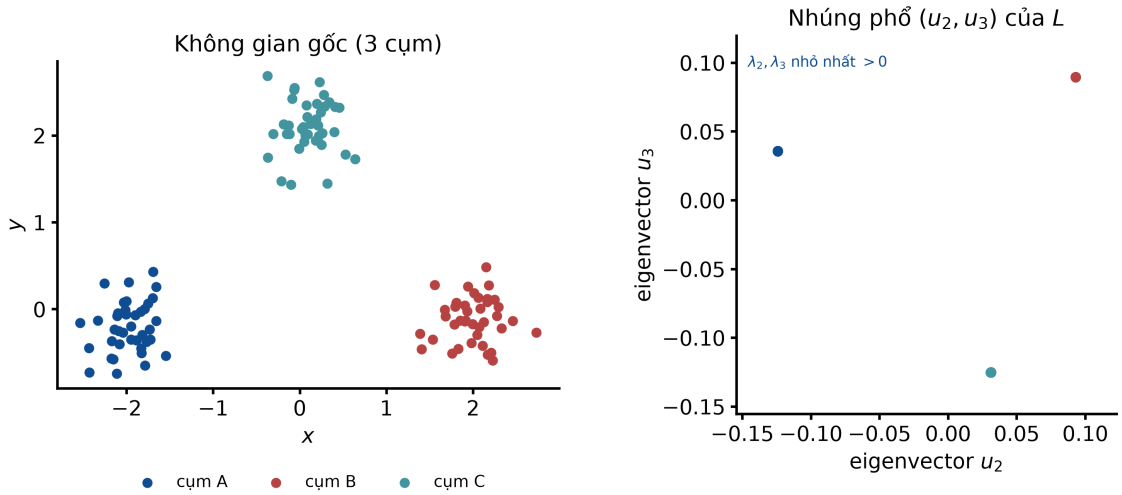
Phân cụm phổ: đồ thị tương tự Gaussian kernel và tách hai cụm theo dấu của vector Fiedler (eigenvector thứ hai của Laplacian). (minh hoạ, tự tính)

Thuật toán	Laplacian dùng	Bước đặc biệt
Unnormalized spectral clustering	L	không có
Normalized theo Shi và Malik (2000)	giải bài toán eigenvalue tổng quát $Lu = \lambda Du$ (tương đương eigenvector của L_{rw})	không có
Normalized theo Ng, Jordan và Weiss (2002)	L_{sym}	chuẩn hóa mỗi hàng của ma trận eigenvector về norm 1 trước khi chạy k-means

Cả ba đều theo khung chung: (1) xây đồ thị tương tự từ ma trận độ tương tự S, lấy W; (2) tính Laplacian tương ứng; (3) lấy k eigenvector đầu $u_1, \dots, u_k$, xếp thành các cột của ma trận $U \in \mathbb{R}^{n \times k}$ (riêng thuật toán Ng-Jordan-Weiss còn chuẩn hóa hàng của U thành ma trận T); (4) coi mỗi hàng của U (hoặc T) là một điểm $y_i \in \mathbb{R}^k$; (5) chạy k-means trên $\{y_i\}$ để được cụm cuối cùng.

3.4. Ví dụ số tự tính: Laplacian của đồ thị nhỏ

Ví dụ 1 (kiểm chứng Proposition 1 và 2, đồ thị 4 đỉnh, 2 thành phần liên thông): xét đồ thị gồm 2 cạnh tách biệt, $w_{12} = 1$ (nối đỉnh 1, 2) và $w_{34} = 1$ (nối đỉnh 3, 4), không có cạnh nào khác. Degree: $d_1 = d_2 = d_3 = d_4 = 1$, nên $D = I$. Ma trận Laplacian:



Nhúng phổ (u_2,u_3) của Laplacian tách rõ 3 cụm dù trong không gian gốc chúng tách theo hình học. (minh hoạ, tự tính)

$$L = \begin{pmatrix} 1 & -1 & 0 & 0 \\ -1 & 1 & 0 & 0 \\ 0 & 0 & 1 & -1 \\ 0 & 0 & -1 & 1 \end{pmatrix}$$

Đây là ma trận khối chéo gồm hai khối $\begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}$ giống nhau. Mỗi khối có eigenvalue 0 (eigenvector $(1,1)$) và eigenvalue 2 (eigenvector $(1,-1)$). Vậy L có 4 eigenvalue: $0, 0, 2, 2$, tức **eigenvalue** 0 **có bội 2, đúng bằng số thành phần liên thông (2)**, khớp với Proposition 2 (tự tính). Kiểm tra eigenvector của eigenvalue 0: với $f = (1,1,0,0)^\top$ (vector chỉ báo của thành phần $\{1,2\}$), $Lf = (1{\cdot}1{-}1{\cdot}1,\ -1{\cdot}1{+}1{\cdot}1,\ 0,\ 0)^\top = (0,0,0,0)^\top$, đúng là eigenvector của eigenvalue 0. Kiểm tra bằng công thức dạng toàn phương của Proposition 1: $f^\top Lf = \frac{1}{2}\left[w_{12}(f_1-f_2)^2 + w_{21}(f_2-f_1)^2 + w_{34}(f_3-f_4)^2 + w_{43}(f_4-f_3)^2\right] = \frac{1}{2}[0+0+0+0] = 0$, khớp với $\lambda\|f\|^2 = 0 \cdot 2 = 0$.

Với $f = (1,-1,0,0)^\top$ (hướng thứ hai trong khối đầu): $Lf = (1\cdot 1 - (-1)\cdot(-1)$ sai, tính lại: Lf hàng 1 $= 1\cdot 1 + (-1)\cdot(-1) = 1+1 = 2$, hàng $2 = -1\cdot 1 + 1\cdot(-1) = -1-1 = -2$, hàng 3, 4 bằng 0), vậy $Lf = (2,-2,0,0)^\top = 2f$, đúng eigenvalue 2. Kiểm tra dạng toàn phương: $f^\top Lf = \frac{1}{2}[1\cdot(1-(-1))^2 + 1\cdot((-1)-1)^2] = \frac{1}{2}[4+4] = 4$, khớp $\lambda\|f\|^2 = 2\cdot(1+1) = 4$ (tự tính).

Ví dụ 2 (kiểm chứng sự khác biệt giữa L, L_{sym}, L_{rw} trên đồ thị bậc không đều): xét đồ thị hình sao 3 đỉnh: đỉnh 1 nối với đỉnh 2 ($w_{12} = 1$) và đỉnh 1 nối với đỉnh 3 ($w_{13} = 1$), không có cạnh 2-3. Degree: $d_1 = 2, d_2 = 1, d_3 = 1$, $D = \text{diag}(2,1,1)$.

$$L = \begin{pmatrix} 2 & -1 & -1 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \end{pmatrix}$$

Kiểm tra eigenvector 0 của L là vector hằng $(1,1,1)^\top$: $L(1,1,1)^\top = (2-1-1,\ -1+1,\ -1+1)^\top = (0,0,0)^\top$, đúng (tự tính).

Tính $L_{sym} = D^{-1/2}LD^{-1/2}$ với $D^{-1/2} = \text{diag}(1/\sqrt{2}, 1, 1)$: phần tử $(1,1)$ là $2 \cdot \frac{1}{\sqrt{2}} \cdot \frac{1}{\sqrt{2}} = 1$; phần tử $(1,2) = (1,3)$ là $-1 \cdot \frac{1}{\sqrt{2}} \cdot 1 = -1/\sqrt{2}$; phần tử $(2,2) = (3,3) = 1$; phần tử $(2,3) = 0$. Vậy:

$$L_{sym} = \begin{pmatrix} 1 & -1/\sqrt{2} & -1/\sqrt{2} \\ -1/\sqrt{2} & 1 & 0 \\ -1/\sqrt{2} & 0 & 1 \end{pmatrix}$$

Theo Proposition 3/4, eigenvector của eigenvalue 0 phải là $D^{1/2}\mathbf{1} = (\sqrt{2}, 1, 1)^\top$ (không còn là vector hằng). Kiểm tra: hàng 1 của $L_{sym}(\sqrt{2}, 1, 1)^\top = 1 \cdot \sqrt{2} + (-1/\sqrt{2}) \cdot 1 + (-1/\sqrt{2}) \cdot 1 = \sqrt{2} - 2/\sqrt{2} = \sqrt{2} - \sqrt{2} = 0$; hàng 2: $-1/\sqrt{2} \cdot \sqrt{2} + 1 \cdot 1 + 0 = -1 + 1 = 0$; hàng 3 tương tự 0. Vậy $(\sqrt{2}, 1, 1)^\top$ đúng là eigenvector của eigenvalue 0 của L_{sym}, khớp chính xác với phát biểu của paper (tự tính).

Tính $L_{rw} = D^{-1}L = \text{diag}(1/2, 1, 1) \cdot L = \begin{pmatrix} 1 & -0.5 & -0.5 \\ -1 & 1 & 0 \\ -1 & 0 & 1 \end{pmatrix}$. Kiểm tra eigenvector 0 là vector hằng $(1,1,1)^\top$ (theo Proposition 3/4, phần đầu): hàng 1: $1 - 0.5 - 0.5 = 0$; hàng 2: $-1 + 1 + 0 = 0$; hàng 3: $-1 + 0 + 1 = 0$. Đúng. Vậy trong ví dụ bậc không đều này, L_{rw} vẫn giữ eigenvector hằng cho eigenvalue 0 giống L, còn L_{sym} thì không, đúng như sự phân biệt mà paper nêu giữa hai loại Laplacian chuẩn hóa (tự tính, toàn bộ ví dụ 2 là phép tính độc lập, không chép từ paper, chỉ dùng công thức paper cung cấp).

4. Thí nghiệm và kết quả chính

Đây là một bài tutorial, không có benchmark định lượng so với các thuật toán khác trên dữ liệu thật theo nghĩa bảng số liệu độ chính xác. "Kết quả chính" của paper là các minh họa trên **dữ liệu mô phỏng nhỏ (toy data)**, dùng để làm rõ trực giác, không phải so sánh hiệu năng:

- **Hình 1 (mục 4):** dữ liệu 1 chiều gồm 200 điểm lấy mẫu từ hỗn hợp 4 phân phối Gauss (mixture of four Gaussians). Paper tính eigenvalue/eigenvector của L và L_{rw} cho cả đồ thị 10-nearest-neighbor và đồ thị fully-connected. Kết quả minh họa: với đồ thị kNN (các cụm tách thành các thành phần liên thông riêng), 4 eigenvalue đầu đúng bằng 0 và các eigenvector tương ứng là vector chỉ báo cụm; với đồ thị fully-connected (chỉ 1 thành phần liên thông vì hàm Gauss luôn dương), eigenvalue 0 có bội 1 (eigenvector hằng), còn 3 eigenvector tiếp theo mang thông tin phân biệt 4 cụm qua dấu của các phần tử (threshold ở 0).
- **Hình 3 (mục 8.1):** so sánh ε-neighborhood graph, kNN graph và mutual kNN graph trên một bộ dữ liệu 2D có 3 cụm (hai "vành trăng" mật độ khác nhau và một cụm Gauss). Minh họa định tính: ε-graph khó chọn tham số khi dữ liệu có mật độ khác nhau theo vùng; kNN graph nối được các vùng mật độ khác nhau nhưng dễ tách rời thành nhiều thành phần; mutual kNN graph nằm "ở giữa", không trộn các mức mật độ khác nhau.
- **Hình 4 (mục 8.3):** minh họa "eigengap heuristic" (chọn số cụm k dựa vào khoảng cách lớn giữa eigenvalue thứ k và $k+1$) trên 3 bộ dữ liệu có phương sai cụm tăng dần (cụm càng "nhòe" thì khoảng cách eigenvalue càng khó phát hiện).
- **Hình 5 (mục 8.5):** minh họa hiện tượng unnormalized spectral clustering (dùng L) có thể hội tụ về eigenvector xấp xỉ hàm Dirac (gần như chỉ khác 0 tại một đỉnh duy nhất) khi eigenvalue tiến gần $\min_j d_j$ (bậc nhỏ nhất trong đồ thị), minh họa bằng cách tăng σ của kernel Gauss ($\sigma = 2$ và $\sigma = 5$) trên cùng bộ dữ liệu của Hình 1.

Ngoài các minh họa toy data này, paper còn trình bày một **kết quả lý thuyết định lượng thật** (không phải thí nghiệm số của chính paper, mà là kết quả được paper diễn giải lại từ Guattery và Miller, 1998): trên một lớp đồ thị gọi là "cockroach graph" (dạng cái thang với vài thanh ngang bị bỏ), nhát cắt RatioCut tối ưu cho $k = 2$ có giá trị $\text{RatioCut}(A, \bar{A}) = 2/k$ (cắt dọc thang), nhưng unnormalized spectral clustering luôn tìm ra

nhát cắt ngang với giá trị RatioCut$(B, \bar{B}) = 1$, tức **tệ hơn nghiệm tối ưu một hệ số** $k/2$ (paper nói rõ đây là "a factor in the order of n"). Đây là một bằng chứng định lượng cụ thể (dù không phải paper tự chạy thí nghiệm, mà dẫn lại kết quả đã công bố) cho thấy phép thư giãn phổ (spectral relaxation) không có đảm bảo xấp xỉ (approximation guarantee) nào so với bài toán tổ hợp gốc.

5. Phân tích / ablation

Paper giải thích vì sao spectral clustering hoạt động qua **ba góc nhìn độc lập**, và phần này tổng hợp lại như một dạng "ablation" về việc thuật toán nào phù hợp khi nào:

Góc nhìn 1, graph cut (mục 5): minh họa cho $k = 2$: đặt $f_i = \sqrt{|\bar{A}|/|A|}$ nếu $v_i \in A$ và $-\sqrt{|A|/|\bar{A}|}$ nếu $v_i \in \bar{A}$, paper chứng minh $f^\top L f = |V| \cdot \text{RatioCut}(A, \bar{A})$ và $f \perp \mathbf{1}$, $\|f\| = \sqrt{n}$. Thư giãn điều kiện rời rạc của f (cho f nhận giá trị thực tùy ý) biến bài toán tối thiểu RatioCut thành bài toán Rayleigh-Ritz chuẩn, nghiệm là eigenvector thứ hai của L. Tương tự, thư giãn Ncut (dùng $g = D^{1/2} f$) cho nghiệm là eigenvector thứ hai của L_{sym} (tương đương L_{rw}). Vậy: **thư giãn RatioCut dẫn tới unnormalized spectral clustering, thư giãn Ncut dẫn tới normalized spectral clustering**.

Góc nhìn 2, random walk (mục 6): $L_{rw} = I - P$ với $P = D^{-1}W$ là ma trận chuyển (transition matrix) của một random walk trên đồ thị; eigenvalue λ của L_{rw} tương ứng eigenvalue $1 - \lambda$ của P. Paper chứng minh (Proposition 5, dẫn từ Meila và Shi 2001) Ncut$(A, \bar{A}) = P(\bar{A} \mid A) + P(A \mid \bar{A})$ khi random walk khởi động từ phân phối dừng: tối thiểu Ncut tương đương tìm một nhát cắt sao cho random walk hiếm khi "nhảy" qua lại giữa hai nhóm. Paper cũng giới thiệu khoảng cách commute (commute distance, hay resistance distance) qua ma trận nghịch đảo tổng quát (pseudo-inverse) $L^\dagger$ của L, và tự cảnh báo rằng mối liên hệ giữa phép nhúng của spectral clustering và commute-time embedding chỉ là **"a rather loose relation"** (không hoàn toàn trùng khớp, hai phép nhúng chỉ gần giống nhau trong một số trường hợp đặc biệt), paper nói rõ "we have not yet seen a precise mathematical statement about this".

Góc nhìn 3, perturbation theory (mục 7): dùng định lý Davis-Kahan để chặn khoảng cách giữa không gian riêng của Laplacian "lý tưởng" (các cụm hoàn toàn tách biệt) và Laplacian "bị nhiễu" (các cụm có một ít kết nối chéo) theo chuẩn của phần nhiễu $\|H\|$ chia cho khoảng cách phổ (eigengap) δ. Kết luận: **eigengap lớn thì nhiễu nhỏ không làm lệch nhiều eigenvector**, tức thuật toán ổn định hơn. Mục này cũng nêu hai điều kiện kỹ thuật cần có để lập luận trên hợp lý: (a) thứ tự eigenvalue phải có ý nghĩa (đúng với các Laplacian, nhưng không đúng nếu dùng trực tiếp eigenvector của ma trận tương tự S hoặc W, nên paper khuyến cáo không dùng S/W trực tiếp để phân cụm); (b) các phần tử eigenvector trên mỗi thành phần phải "đủ xa 0", nếu không (như trường hợp L_{sym} khi đồ thị có đỉnh bậc rất thấp) bước chuẩn hóa hàng có thể khuếch đại nhiễu và gây phân loại sai cho các đỉnh bậc thấp (outlier).

So sánh tổng hợp ba thuật toán (mục 8.5): paper đưa ra khuyến nghị rõ ràng, dựa trên ba lập luận:

Lập luận	Kết luận của paper
Mục tiêu phân cụm (tối thiểu cắt giữa nhóm và tối đa tương tự trong nhóm)	Normalized (dùng Ncut) đạt cả 2 mục tiêu; Unnormalized (dùng RatioCut) chỉ đạt mục tiêu tối thiểu cắt, không trực tiếp tối đa tương tự nội nhóm
Tính nhất quán (consistency) khi $n \to \infty$	Normalized (cả L_{sym} và L_{rw}) được chứng minh hội tụ về một phân hoạch có ý nghĩa; Unnormalized có thể không hội tụ, hoặc hội tụ về nghiệm suy biến (cụm chỉ gồm 1 điểm), liên quan hiện tượng eigenvector xấp xỉ Dirac khi eigenvalue vượt quá $\min_j d_j$
L_{sym} so với L_{rw}	Cả hai đều có cơ sở lý thuyết tốt, nhưng eigenvector của L_{rw} là vector chỉ báo cụm "sạch" (không bị nhân thêm $D^{1/2}$ như L_{sym}); L_{sym} không có ưu thế tính toán nào hơn, nên paper khuyến nghị dùng L_{rw}

Kết luận cuối của paper ở mục này: nên dùng **normalized spectral clustering với** L_{rw} làm lựa chọn mặc định.

6. Giới hạn

Các giới hạn mà **bài báo nêu rõ**, khá nhiều và khá thẳng thắn:

- **Không có cơ sở lý thuyết vững cho việc chọn đồ thị tương tự:** paper viết thẳng ”to our knowledge there has been no systematic study which investigates the effects of the similarity graph and its parameters... None of the recommendations above is based on a firm theoretic ground”. Với mutual kNN graph, paper thậm chí thừa nhận ”we have to admit that we are a bit lost for rules of thumb” khi chọn tham số k.
- **Eigengap heuristic chỉ hiệu quả khi cụm tách biệt rõ:** khi các cụm ”nhòe” hoặc chồng lấp, khoảng cách eigenvalue không còn rõ ràng, heuristic trở nên mơ hồ, như minh họa ở Hình 4.
- **Không có đảm bảo xấp xỉ:** phép thư giãn phổ không đảm bảo nghiệm gần với nghiệm tối ưu của bài toán tổ hợp gốc (RatioCut/Ncut); ví dụ cockroach graph cho thấy sai khác có thể lớn tới hệ số bậc n (mục 4, dẫn từ Guattery và Miller 1998). Paper cũng nhắc tới kết quả tổng quát hơn: xấp xỉ balanced graph cut trong hệ số hằng là NP-hard (Bui và Jones, 1992).
- **Mối quan hệ giữa chọn k (số cụm) và chọn tham số đồ thị (ví dụ k của kNN graph) không được hiểu rõ:** paper nói ”to our knowledge nothing non-trivial is known on their interactions”.
- **Unnormalized spectral clustering có thể hội tụ về nghiệm suy biến** (cụm 1 điểm) khi $n \to \infty$, trừ khi các eigenvalue dùng để phân cụm nhỏ hơn hẳn $\min_j d_j$; đây là lý do chính paper khuyến cáo tránh dùng L trực tiếp.
- L_{sym} **có thể phân loại sai các đỉnh bậc thấp (outlier):** vì eigenvector lý tưởng của L_{sym} là $D^{1/2}\mathbf{1}_{A_i}$, các đỉnh có bậc rất thấp sẽ có giá trị eigenvector rất nhỏ; bước chuẩn hóa hàng trong thuật toán Ng-Jordan-Weiss có thể khuếch đại nhiễu ở các giá trị nhỏ này và gây phân loại nhầm.
- **Bước k-means cuối cùng không có gì ”đặc biệt bắt buộc”**: paper tự nói việc dùng k-means ở bước cuối là lựa chọn tùy ý (arbitrary), chỉ là lựa chọn phổ biến nhất; có thể thay bằng siêu phẳng (Lang, 2006) hoặc hậu xử lý khác (Bach và Jordan, 2004).

Quan sát **của người ghi chú**: paper này khác biệt rõ so với math 01-03 trong batch (những bài không có hoặc có rất ít chỗ tự phê bình định lượng), vì tác giả liên tục tự nêu các giới hạn thực hành kèm trích dẫn cụ thể, khiến phần ”giới hạn” ở đây có nhiều nội dung xác thực từ chính văn bản hơn so với các bài trước.

7. Liên hệ với các bài khác

- **math 03 (A Tutorial on Principal Component Analysis, Shlens):** cả hai phương pháp đều dựa trên eigen-decomposition, nhưng của hai ma trận khác nhau và theo hai mục tiêu khác nhau. PCA tìm eigenvector ứng với eigenvalue *lớn nhất* của ma trận hiệp phương sai (hướng phương sai cao nhất để giữ thông tin), còn spectral clustering tìm eigenvector ứng với eigenvalue *nhỏ nhất* của Laplacian (hướng mà hàm f biến thiên ít giữa các điểm tương tự, để tách cụm). Đáng chú ý, paper hiện tại tự trích dẫn trực tiếp mối liên hệ này ở mục 9: Bengio và cộng sự (2004) diễn giải ma trận $D^{-1/2}WD^{-1/2}$ như một kernel matrix (liên hệ kernel PCA), còn Saerens, Fouss, Yen và Dupont (2004) có bài báo tên ”The principal components analysis of a graph, and its relationships to spectral clustering” nêu đúng mối liên hệ giữa hai bài toán.
- **math 04 (Finding Structure with Randomness, Halko-Martinsson-Tropp):** paper hiện tại (mục 8.2) nói rằng để tính k eigenvector đầu của một Laplacian lớn nhưng thưa (sparse), người ta dùng power method hoặc phương pháp Krylov subspace như Lanczos; các kỹ thuật ngẫu nhiên hóa của math 04 (randomized range finder, randomized SVD) là một hướng hiện đại hơn để giải cùng bài toán con này (tìm vài eigenvector/singular vector đầu một cách hiệu quả ở quy mô lớn), dù bản thân paper 2007 này không đề cập tới randomized linear algebra (hợp lý vì ra đời 2 năm trước math 04).

- **math 06 (New Insights and Perspectives on the Natural Gradient Method, Martens):** bài tiếp theo trong batch, chuyển hẳn sang nhóm chủ đề "hình học thông tin / Fisher information", không chia sẻ công cụ toán trực tiếp với spectral clustering, nhưng cùng thuộc lộ trình "các phương pháp dùng eigen-decomposition của một ma trận mô tả cấu trúc bài toán" nếu nhìn rộng (Fisher information matrix cũng cần eigen-decomposition/ma trận đối xứng dương, dù không phải Laplacian đồ thị).

8. Gợi ý áp dụng

Spectral clustering giúp khám phá cấu trúc nhóm khi dữ liệu được biểu diễn bằng đồ thị tương tự. Kết quả phụ thuộc cách xây dựng đồ thị và các tham số như số láng giềng hoặc độ rộng kernel; cần kiểm tra độ ổn định của cụm khi thay đổi các lựa chọn đó.

9. Thí nghiệm gợi ý

Các thí nghiệm nhỏ dưới đây chạy được trên GPU (dùng PyTorch/NumPy/scikit-learn cho bước k-means và eigen-decomposition cuối):

1. **Tái tạo ví dụ toy của Hình 1:** sinh 200 điểm 1D từ hỗn hợp 4 phân phối Gauss, xây đồ thị 10-NN và đồ thị fully-connected (kernel Gauss $\sigma = 1$), tính eigenvalue/eigenvector của L và L_{rw}, kiểm tra xem 4 eigenvalue đầu có gần 0 không (đồ thị kNN) và xem 3 eigenvector tiếp theo của đồ thị fully-connected có phân biệt đúng 4 cụm qua dấu không, so với kỳ vọng mô tả ở mục 4.
2. **Spectral clustering trên attention map của một layer nhỏ:** lấy attention weight trung bình (qua vài batch) của một layer attention trong một mô hình nhỏ, đối xứng hóa thành W, tính L_{rw}, lấy k eigenvector đầu, chạy k-means để nhóm các token/vị trí. Đo: các nhóm tìm được có tương ứng với cấu trúc ngữ nghĩa hợp lý (ví dụ token trong cùng một từ, hoặc token hình ảnh cùng một vùng ảnh) không.
3. **So sánh eigengap heuristic trên dữ liệu tổng hợp với mức nhiễu khác nhau:** lặp lại thí nghiệm giống Hình 4 của paper (3 bộ Gauss mixture với phương sai tăng dần), vẽ phổ eigenvalue của L_{rw}, quan sát khoảng cách eigenvalue thứ k và $k+1$ thu hẹp dần khi cụm chồng lấp nhiều hơn, kiểm tra trực tiếp tuyên bố của paper rằng heuristic "kém hiệu quả khi cụm nhòe".
4. **Kiểm tra hiện tượng Dirac-eigenvector của unnormalized Laplacian:** trên cùng bộ dữ liệu Gauss mixture, tăng dần σ của kernel Gauss khi xây đồ thị fully-connected, tính eigenvector của L (không chuẩn hóa), đo độ "tập trung" của mỗi eigenvector (ví dụ bằng entropy hoặc tỉ lệ phần tử lớn nhất trên tổng), kiểm tra xem khi eigenvalue tiến gần $\min_j d_j$ thì eigenvector có thực sự trở nên gần giống hàm Dirac (chỉ lớn ở một tọa độ) như paper mô tả ở Hình 5 không.
5. **So sánh ba thuật toán (unnormalized, Shi-Malik, Ng-Jordan-Weiss) trên một đồ thị có vài đỉnh bậc rất thấp (outlier):** tạo một đồ thị tổng hợp có 2 cụm dày đặc và thêm vài điểm "lạc" (bậc rất thấp), chạy cả ba thuật toán, so sánh tỉ lệ các điểm lạc bị phân loại sai giữa Ng-Jordan-Weiss (dùng L_{sym}, dự kiến nhạy với outlier do bước chuẩn hóa hàng) và Shi-Malik/unnormalized, kiểm tra trực tiếp cảnh báo ở mục 7.2 của paper.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / định nghĩa
Ma trận kề, bậc	$W = (w_{ij})$, $d_i = \sum_j w_{ij}$, $D = \text{diag}(d_1, \dots, d_n)$
Laplacian không chuẩn hóa	$L = D - W$; $f^\top L f = \frac{1}{2}\sum_{ij} w_{ij}(f_i - f_j)^2$; eigenvalue nhỏ nhất 0, eigenvector $\mathbf{1}$
Số thành phần liên thông	bằng bội của eigenvalue 0 của L (và của L_{sym}, L_{rw}); eigenspace trải bởi các vector chỉ báo thành phần
Laplacian chuẩn hóa	$L_{sym} = D^{-1/2} L D^{-1/2}$ (eigenvector 0 là $D^{1/2}\mathbf{1}$); $L_{rw} = D^{-1}L = I - P$ với $P = D^{-1}W$ (eigenvector 0 là $\mathbf{1}$)

Khái niệm	Công thức / định nghĩa
RatioCut, Ncut	RatioCut $= \sum_i \text{cut}(A_i, \bar{A}_i)/\lvert A_i\rvert$; Ncut $= \sum_i \text{cut}(A_i, \bar{A}_i)/\text{vol}(A_i)$
Thư giãn phổ	thư giãn RatioCut $\to$ eigenvector của L (unnormalized); thư giãn Ncut $\to$ eigenvector của L_{sym}/L_{rw} (normalized)
Thuật toán chung	xây đồ thị, tính Laplacian, lấy k eigenvector đầu làm cột của U, mỗi hàng của U là điểm nhúng $y_i \in \mathbb{R}^k$, chạy k-means trên $\{y_i\}$
Eigengap heuristic	chọn k sao cho $\lambda_1, \ldots, \lambda_k$ rất nhỏ nhưng λ_{k+1} lớn hẳn
Ví dụ tự tính 1	đồ thị 2 cạnh tách biệt: L có eigenvalue $\{0, 0, 2, 2\}$, bội của 0 bằng 2 = số thành phần liên thông
Ví dụ tự tính 2	đồ thị sao 3 đỉnh, $d = (2, 1, 1)$: eigenvector 0 của L là $(1, 1, 1)$, của L_{sym} là $(\sqrt{2}, 1, 1)$, của L_{rw} là $(1, 1, 1)$

11. Kết luận ngắn

Bài báo giải thích spectral clustering từ ba góc nhìn độc lập (cắt đồ thị, random walk, lý thuyết nhiễu loạn), cho thấy bản chất thuật toán là một phép thư giãn liên tục của các bài toán phân hoạch đồ thị NP-hard (RatioCut, Ncut), với nghiệm là các eigenvector ứng với eigenvalue nhỏ nhất của ma trận Laplacian (không chuẩn hóa hoặc chuẩn hóa). Khuyến nghị thực hành của paper: dùng đồ thị k-nearest-neighbor, Laplacian chuẩn hóa L_{rw}, vì có cơ sở lý thuyết tốt hơn (nhất quán khi $n \to \infty$, không có hiện tượng eigenvector suy biến như Laplacian không chuẩn hóa). Giới hạn cốt lõi mà bài báo lưu ý: việc chọn đồ thị tương tự và các tham số liên quan thiếu cơ sở lý thuyết chắc chắn, và phép thư giãn phổ không có đảm bảo xấp xỉ nào so với bài toán tổ hợp gốc. Trong lộ trình học, bài này minh họa một mô thức dùng lại nhiều lần: phân tích cấu trúc dữ liệu/bài toán qua eigen-decomposition của một ma trận đối xứng mô tả quan hệ giữa các thực thể, cùng họ với PCA (math 03) nhưng áp dụng cho ma trận và mục tiêu hoàn toàn khác.

Nguồn gốc: https://arxiv.org/abs/0711.0189

Bài 06

New Insights and Perspectives on the Natural Gradient Method (2014)

Tác giả: James Martens (DeepMind, London)
Venue / arXiv: arXiv:1412.1193 (v11, 19/09/2020); công bố chính thức tại Journal of Machine Learning Research 21 (2020), trang 1-76 (Submitted 11/17, Revised 6/20, Published 8/20, Editor: Léon Bottou)
Nguồn: https://arxiv.org/abs/1412.1193
Chủ đề: Tối ưu hóa bậc hai, natural gradient, Fisher information matrix, Generalized Gauss-Newton matrix
Vị trí trong lộ trình: Bài số 6 trong nhóm 9 bài toán nền (math 01-09). Mở đầu nhóm con ”hình học thông tin / Fisher information” (06-07), đọc sau math 05 (spectral clustering, vẫn thuộc nhóm eigen-decomposition cổ điển), dùng lại khái niệm Hessian và gradient từ math 01 nhưng áp dụng cho một ma trận ”độ cong” khác (Fisher). Đứng trước math 07 (tổng quát hóa thành khung hình học thông tin đầy đủ cho các họ phân phối).

> Ghi chú tập trung vào nội dung chính của bài; các chứng minh kỹ thuật trong Phụ lục A–E chỉ được giới thiệu khái quát.

1. Bối cảnh

Natural gradient descent, được Amari đề xuất vào cuối những năm 1990, là một phương pháp tối ưu đã được áp dụng trong tách nguồn mù, học tăng cường và huấn luyện mạng nơ-ron. James Martens phân tích lại phương pháp này, từ cơ sở hình học thông tin đến lý do nó có thể hoạt động tốt và giới hạn của các cách diễn giải thường gặp.

Đây là một trong những bài có lịch sử công bố khác thường nhất trong batch: bản arXiv đầu tiên (v1) được đăng tháng 12/2014 (mã arXiv 1412.1193), nhưng bài chỉ được nộp chính thức cho JMLR vào tháng 11/2017, qua chỉnh sửa (revised) tháng 6/2020, và xuất bản tháng 8/2020, với phiên bản arXiv cuối cùng là v11 (tổng cộng 11 lần sửa trong 6 năm). Venue cuối cùng là Journal of Machine Learning Research, tập 21 (2020), trang 1-76, do Léon Bottou làm editor. Ghi chú này dùng năm 2014 cho tên file theo quy ước lấy năm của mã arXiv gốc, nhưng cần hiểu rằng nội dung đã được tinh chỉnh qua nhiều năm trước khi thành bài JMLR hoàn chỉnh.

Tóm tắt (abstract) của paper nêu rõ ba đóng góp chính: (1) chỉ ra natural gradient descent có thể xem như một phương pháp bậc hai (2nd-order method), với Fisher information matrix đóng vai trò thay thế cho Hessian; (2) trong nhiều trường hợp quan trọng, Fisher tương đương với Generalized Gauss-Newton matrix (GGN); (3) phân tích tốc độ hội tụ tiệm cận của natural gradient descent ngẫu nhiên (stochastic) trên hàm mục tiêu lồi bậc hai (quadratic), phê bình ”empirical Fisher approximation” hay bị dùng sai trong thực hành, và phân tích tính bất biến (gần đúng) theo tham số hóa.

2. Ý tưởng chính

Natural gradient được định nghĩa là gradient nhân với nghịch đảo của Fisher information matrix F của mô hình:

$$\tilde{\nabla} h = F^{-1} \nabla h$$

trong đó h là hàm mục tiêu (thường là trung bình loss trên tập huấn luyện). Paper trình bày hai cách hiểu bổ sung cho nhau về natural gradient, đây chính là ”insight” trung tâm của bài:

Cách hiểu truyền thống (information geometry): gradient tiêu chuẩn $-\nabla h$ là hướng giảm dốc nhất (steepest descent) khi khoảng cách trong không gian tham số được đo bằng chuẩn Euclid. Natural gradient là hướng giảm dốc nhất khi khoảng cách được đo bằng một ”metric” khác, nội tại với không gian các phân phối xác suất mà mô hình biểu diễn (không gắn với cách tham số hóa cụ thể), cụ thể là xấp xỉ của KL divergence $KL(P_{x,y}(\theta + d)\|P_{x,y}(\theta))$ trong một lân cận nhỏ. Vì khoảng cách KL không phụ thuộc cách tham số hóa, hướng natural gradient (trong không gian phân phối) bất biến với mọi cách đổi tham số trơn, khả nghịch, trong khi gradient thường thì không.

Cách hiểu mới mà paper đề xuất là trọng tâm (bậc hai / 2nd-order): paper lập luận cách hiểu truyền thống trên có một lỗ hổng thực tế: nó chỉ đúng khi bước cập nhật vô cùng nhỏ, còn trong thực hành ta luôn cập nhật một bước rời rạc hữu hạn trong không gian tham số θ, lúc đó lý giải hình học bị phá vỡ (đi thẳng trong không gian θ không tương ứng với đi thẳng trong không gian phân phối). Paper đề xuất cách hiểu thứ hai: natural gradient descent chính là một dạng phương pháp bậc hai (2nd-order method), trong đó Fisher F đóng vai trò ma trận độ cong (curvature matrix) thay cho Hessian H thật. Và quan trọng hơn: trong nhiều trường hợp phổ biến, F lại chính bằng Generalized Gauss-Newton matrix (GGN), một ma trận xấp xỉ Hessian đã được biết đến từ trước trong tối ưu hóa cổ điển, có tính chất luôn nửa xác định dương (PSD). Cách hiểu này giải thích được vì sao các kỹ thuật ”damping” (giảm xóc, ví dụ Tikhonov regularization, trust region) từ tối ưu hóa bậc hai cổ điển lại cần thiết để natural gradient hoạt động tốt và ổn định trong thực hành với bước cập nhật lớn.

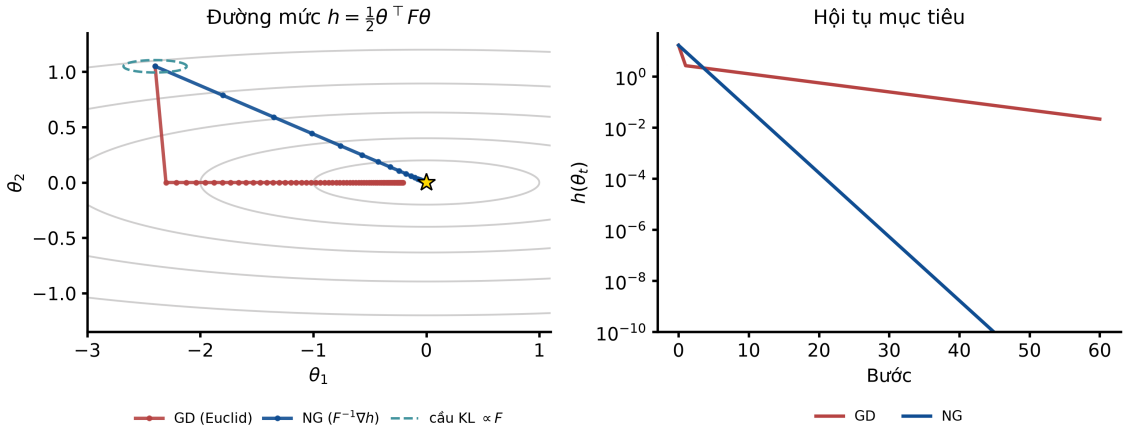
3. Phương pháp chi tiết

3.1. Khung bài toán và KL divergence objective

Paper xét bài toán học có giám sát (supervised learning) tổng quát: mạng neural với tham số $\theta \in \mathbb{R}^n$ tính hàm $f(x, \theta)$, mục tiêu tối thiểu $h(\theta) = \frac{1}{|S|}\sum_{(x,y)\in S} L(y, f(x, \theta))$ với L là hàm loss. Bài báo nhấn mạnh natural gradient có lý thuyết chặt nhất khi $L(y, z) = -\log r(y|z)$ cho một phân phối dự đoán $R_{y|z}$ nào đó (ví dụ: squared error tương ứng phân phối Gauss, cross-entropy tương ứng phân phối multinomial). Khi đó bài toán học tương đương tối thiểu $KL(Q_{x,y}\|P_{x,y}(\theta))$, với $Q_{x,y}$ là phân phối dữ liệu thật, $P_{x,y}(\theta)$ là phân phối mà mô hình biểu diễn. Khi thay Q_x bằng phân phối huấn luyện thực nghiệm $\hat{Q}_x$ và chỉ có một mẫu y cho mỗi x, mục tiêu KL quy về chính hàm negative log-likelihood tiêu chuẩn.

3.2. Fisher information matrix và natural gradient

Fisher của $P_{x,y}(\theta)$ theo θ được định nghĩa:



Natural gradient F⁻¹∇h đi tắt trên bậc hai kéo dài; cầu KL cục bộ tỷ lệ F, khác cầu Euclid. (minh hoạ, tự tính)

$$F = \mathbb{E}_{P_{x,y}}\left[\nabla \log p(x, y|\theta)\, \nabla \log p(x, y|\theta)^\top\right] = -\mathbb{E}_{P_{x,y}}\left[H_{\log p(x,y|\theta)}\right]$$

Dạng thứ nhất (tích ngoài của gradient log-density với chính nó, lấy kỳ vọng) cho thấy ngay F luôn PSD (vì là kỳ vọng của một outer-product, luôn PSD). Dạng thứ hai (âm của kỳ vọng Hessian của log-density) cho thấy F có thể hiểu là "Hessian kỳ vọng dưới phân phối của chính mô hình" (expected Hessian under the model's own predictive distribution), khác với Hessian thật H của h, vốn là kỳ vọng Hessian dưới phân phối thực nghiệm $\hat{Q}_{x,y}$. Natural gradient định nghĩa là $\tilde{\nabla} h = F^{-1}\nabla h$.

3.3. Lý giải hình học (mục 6 của paper)

Paper dùng khai triển Taylor bậc hai của KL divergence:

$$KL(P_{x,y}(\theta + d) \| P_{x,y}(\theta)) = \frac{1}{2} d^\top F d + O(d^3)$$

Dựa trên một kết quả của Arnold và cộng sự (2011) về steepest descent với ma trận PSD A tổng quát, paper cho thấy khi lấy $A = \frac{1}{2}F$, hướng $-F^{-1}\nabla h$ (tỉ lệ bởi hằng số) chính là hướng giảm dốc nhất trong không gian phân phối, đo bằng khoảng cách KL cục bộ. Vì KL chỉ phụ thuộc phân phối dự đoán $P_{y|x}$, không phụ thuộc cách tham số hóa θ, hướng này (trong không gian phân phối) bất biến với đổi tham số hóa, trong khi hướng gradient Euclid thường thì phụ thuộc tham số hóa.

3.4. Phương pháp bậc hai (2nd-order) tổng quát và Generalized Gauss-Newton matrix

Paper định nghĩa khung chung cho các phương pháp bậc hai: cập nhật $\delta_k^* = \arg\min_\delta M_k(\delta)$ với mô hình bậc hai cục bộ $M_k(\delta) = \frac{1}{2}\delta^\top B_k \delta + \nabla h(\theta_k)^\top \delta + h(\theta_k)$, B_k là "ma trận độ cong". Gradient descent ứng với $B_k = \beta I$ (ước lượng độ cong rất thô, một hằng số theo mọi hướng). Newton's method ứng với $B_k = H$ (Hessian thật), chính xác nhất về mặt cục bộ nhưng có vấn đề: H của mạng neural có thể không xác định (indefinite, có hướng cong âm), khiến $M_k(\delta)$ không chặn dưới theo các hướng đó, gây ra bước cập nhật quá lớn, không đáng tin cậy ("model trust" issue).

Generalized Gauss-Newton matrix (GGN), ký hiệu G, giải quyết vấn đề này: với loss $L(y, z)$ lồi theo z,

$$G = \frac{1}{|S|} \sum_{(x,y)\in S} J_f^\top H_L J_f$$

với J_f là Jacobian của $f(x, \theta)$ theo θ, H_L là Hessian của $L(y, z)$ theo z tại $z = f(x, \theta)$. Vì L lồi theo z nên H_L luôn PSD, do đó G luôn PSD, không bao giờ có hướng cong âm. G có thể suy ra bằng cách tuyến tính hóa f (lấy xấp xỉ affine của f quanh θ_k) trước khi lấy Hessian của h theo cách đó; nói cách khác G chính là phần của Hessian thật H còn lại sau khi "bỏ đi" phần đóng góp từ độ cong (phi tuyến) của riêng hàm f (mạng neural), chỉ giữ lại độ cong đến từ hàm loss L. Paper lý luận (mục 8.1, mang tính "suy đoán" theo chính lời bài báo lưu ý): độ cong đến từ L "đáng tin cậy" hơn độ cong đến từ f (vì H_L luôn PSD, ổn định, không đổi dấu bất thường như độ cong của các hàm kích hoạt phi tuyến bên trong mạng), đây là một lời giải thích khả dĩ cho việc GGN thường hoạt động tốt hơn Hessian thật trong huấn luyện mạng neural.

3.5. Mối liên hệ Fisher và GGN (mục 9, kết quả trung tâm của paper)

Paper chỉ ra Fisher cũng có thể viết dưới dạng rất giống GGN:

$$F = \mathbb{E}_{Q_x}\left[J_f^\top F_R J_f\right]$$

với F_R là Fisher của phân phối dự đoán $R_{y|z}$ tại $z = f(x, \theta)$. So với công thức GGN ở trên, hai biểu thức giống nhau hoàn toàn, chỉ khác "ma trận trong" là F_R (Fisher của R) thay cho H_L (Hessian của L). Paper chứng minh (và đây là kết quả định lý trung tâm của bài): $F_R = H_L$ xảy ra khi $R_{y|z}$ thuộc họ phân phối mũ (exponential family) với z là "tham số tự nhiên" (natural parameter), tức $\log r(y|z) = z^\top T(y) - \log Z(z)$.

Các ví dụ cụ thể mà paper nêu: phân phối Gauss đa biến với z tham số hóa trực tiếp trung bình μ (tương ứng loss squared error), và phân phối multinomial với z là input vào softmax (tương ứng loss cross-entropy). Đây là lý do, trong phần lớn các bài toán phân loại/hồi quy tiêu chuẩn của deep learning, Fisher và GGN trùng nhau, nên hai cách lý giải ở mục 2 (hình học thông tin và bậc hai) đều áp dụng được đồng thời.

3.6. Xây dựng phương pháp thực hành và vai trò của damping (mục 10)

Vì bước cập nhật thực tế luôn là một đoạn hữu hạn trong không gian θ (không phải một đường đi mượt vô cùng nhỏ trong không gian phân phối), lý giải hình học ở mục 3.3 không còn chính xác với bước lớn; paper minh họa bằng Hình 1 của paper (đường đi trong không gian tham số do bước natural gradient lớn tạo ra có thể lệch xa so với hướng ban đầu). Thay vào đó, paper đề xuất dựa trên lý giải bậc hai: vì $F \approx G$ trong nhiều trường hợp, cập nhật $-F^{-1}\nabla h$ với hệ số $\alpha = 1$ chính là nghiệm tối ưu của mô hình bậc hai cục bộ dùng G. Khi mô hình bậc hai không còn chính xác ở khoảng cách lớn (và đặc biệt khi F có một số eigenvalue rất nhỏ khiến natural gradient rất lớn theo hướng đó), các kỹ thuật "update damping" quen thuộc trong tối ưu hóa bậc hai cổ điển (Tikhonov regularization, trust region, kiểu Levenberg-Marquardt) nên được dùng, thay vì chỉ đơn giản giảm hệ số bước α một cách đồng nhất cho mọi hướng (vì cách này ảnh hưởng đều lên mọi eigen-direction của F, kể cả những hướng mà natural gradient vốn đã hợp lý).

3.7. Phê bình "empirical Fisher" và ví dụ số minh họa (mục 11, kèm tự tính)

Paper định nghĩa "empirical Fisher" $\bar{F}$ là một xấp xỉ hay bị dùng (và hay bị gọi nhầm là Fisher thật): lấy kỳ vọng tích ngoài gradient log-density dưới phân phối thực nghiệm $\hat{Q}_{x,y}$ (dữ liệu thật) thay vì dưới phân phối dự đoán của chính mô hình $P_{x,y}$:

$$\bar{F} = \frac{1}{|S|} \sum_{(x,y)\in S} \nabla \log p(y|x,\theta)\, \nabla \log p(y|x,\theta)^\top$$

$\bar{F}$ dễ tính hơn F (chỉ cần gradient, không cần lấy thêm kỳ vọng theo phân phối dự đoán của mô hình), nhưng paper lập luận nó khác F theo cách "tinh vi nhưng quan trọng": $\bar{F}$ không có tính chất xấp xỉ Hessian hay tương đương GGN như F có. Paper đưa ra một ví dụ cụ thể để chứng minh điều này (paper chỉ cho công thức tổng quát, không có số liệu cụ thể; phần số liệu dưới đây là tính toán minh họa của người ghi chú, tự tính):

Xét $n = 1$, $f(x,\theta) = \theta$, $R_{y|z} = \mathcal{N}(z,1)$, $S = \{(0,0)\}$ (một mẫu, input và target đều bằng 0). Khi đó $h(\theta) = \frac{1}{2}\theta^2$, $\nabla h = \theta$. Paper tính được: $\bar{F} = \theta^2$ (phụ thuộc θ hiện tại), còn $F = 1$ (hằng số, không đổi theo θ).

Tính tay (tự tính) hai dãy cập nhật với bước cố định $\alpha = 0.5$, bắt đầu từ $\theta_0 = 2$:

Cập nhật dùng Fisher thật: $\theta_{k+1} = \theta_k - \alpha F^{-1}\theta_k = (1-\alpha)\theta_k = 0.5\,\theta_k$. Dãy: $\theta_0 = 2$, $\theta_1 = 1$, $\theta_2 = 0.5$, $\theta_3 = 0.25$, $\theta_4 = 0.125$, giảm đều đặn theo cấp số nhân với tỉ lệ 0.5 mỗi bước, hội tụ nhanh về 0 (đúng như paper dự đoán: với $0 < \alpha < 2$, cập nhật dùng F cho hội tụ tuyến tính ổn định).

Cập nhật dùng empirical Fisher (với $\xi = 1$ như trong công thức tổng quát của paper): $\theta_{k+1} = \theta_k - \alpha(\theta_k^2)^{-1}\theta_k = \theta_k - \dfrac{\alpha}{\theta_k} = \theta_k - \dfrac{0.5}{\theta_k}$. Dãy tính tay: $\theta_0 = 2 \to \theta_1 = 2 - 0.25 = 1.75 \to \theta_2 = 1.75 - 0.5/1.75 \approx 1.464 \to \theta_3 \approx 1.464 - 0.342 \approx 1.123 \to \theta_4 \approx 1.123 - 0.445 \approx 0.677 \to \theta_5 \approx 0.677 - 0.738 \approx -0.061$. Tại bước 5, giá trị vượt qua 0 và trở nên rất nhỏ (gần 0), khiến bước tiếp theo $\theta_6 \approx -0.061 - 0.5/(-0.061) \approx -0.061 + 8.2 \approx 8.14$ nhảy rất xa khỏi nghiệm. Dãy số không hội tụ êm mà dao động với biên độ tăng vọt, đúng như paper cảnh báo: với bước cố định, cập nhật dựa trên empirical Fisher "sẽ không hội tụ về nghiệm tối ưu trừ khi hệ số mũ $\xi < 1$ và bước α_k giảm về 0 đủ nhanh". Đây là minh chứng số cụ thể (tự tính, dựng từ đúng ví dụ và công thức mà paper đưa ra) cho thấy vì sao dùng empirical Fisher làm ma trận độ cong có thể gây mất ổn định mà Fisher thật không gặp phải.

4. Thí nghiệm và kết quả chính

Đây là một bài **lý thuyết hoàn toàn**: toàn bộ 15 mục chính không chứa bất kỳ thí nghiệm huấn luyện mạng neural thực tế nào (không có bảng số liệu, không có hình vẽ loss curve, không có so sánh trên dataset chuẩn như MNIST/CIFAR). "Kết quả chính" của paper là các định lý toán học và các lập luận phân tích:

- **Kết quả 1 (mục 9):** điều kiện chính xác để Fisher F tương đương Generalized Gauss-Newton matrix G: khi $R_{y|z}$ là họ phân phối mũ với z là tham số tự nhiên (điều kiện này đúng cho squared-error loss với Gauss, và cross-entropy loss với multinomial/softmax).
- **Kết quả 2 (mục 11, ví dụ $n = 1$):** một ví dụ phân tích (không phải thí nghiệm số lớn, nhưng là một case study cụ thể, định lượng) cho thấy cập nhật dựa trên empirical Fisher với bước cố định có thể phân kỳ, trong khi cập nhật dựa trên Fisher thật hội tụ tuyến tính ổn định. Mục 3.7 trên đã mở rộng case study này thành một dãy số cụ thể (tự tính) để minh họa trực quan.
- **Kết quả 3 (mục 12, Theorem 1 và Corollary 2):** điều kiện đủ $J_\zeta^\top B_\theta J_\zeta = B_\gamma$ để một ma trận độ cong B cho tính bất biến (gần đúng) theo đổi tham số hóa ζ. Paper chứng minh điều kiện này tự động thỏa mãn cho mọi ma trận độ cong có dạng $\mathbb{E}[J_f^\top A J_f]$ (bao gồm GGN, Fisher, cả empirical Fisher), nhưng **không** thỏa mãn cho Hessian thật (trừ các trường hợp đặc biệt: ζ là hàm affine, hoặc $\nabla h = 0$ tại điểm đang xét).
- **Kết quả 4 (mục 14, Theorem 3, 5, 6, phần trung tâm về lý thuyết hội tụ):** với hàm mục tiêu lồi bậc hai $h(\theta) = \frac{1}{2}(\theta - \theta^*)^\top H^*(\theta - \theta^*)$ và cập nhật ngẫu nhiên (stochastic) dạng $\theta_{k+1} = \theta_k - \alpha_k B^{-1} g_k(\theta_k)$ với B là ma trận độ cong cố định bất kỳ, paper đưa ra biểu thức tiệm cận chính xác cho $\mathbb{E}[h(\theta_k)] - h(\theta^*)$, tách thành hai phần: một phần phụ thuộc nhiễu gradient (noise-dependent, chiếm ưu thế khi $k \to \infty$) và một phần phụ thuộc điểm khởi tạo θ_0 (starting-point-dependent, giảm nhanh hơn theo k). Kết luận gây bất ngờ mà bài báo nêu: **phần phụ thuộc nhiễu không được cải thiện bởi việc dùng ma trận độ cong bậc hai** (natural gradient hay Newton) nếu kết hợp với kỹ thuật Polyak averaging, SGD thường + averaging đạt cùng tốc độ tiệm cận ("Fisher efficient") mà không cần bất kỳ ma trận độ cong nào; lợi ích thực sự của phương pháp bậc hai nằm ở phần phụ thuộc điểm khởi tạo, quan trọng khi số vòng lặp còn hạn chế (chưa đạt chế độ tiệm cận).

Paper không chạy bất kỳ experiment deep learning thực nghiệm nào để kiểm chứng trực tiếp các định lý 3, 5, 6 trên mạng neural thật; mọi kết quả đều là suy diễn giải tích thuần túy cho trường hợp lý tưởng hóa (hàm mục tiêu lồi bậc hai).

5. Phân tích / ablation

So sánh các ma trận độ cong (curvature matrix) theo các tiêu chí mà paper tự đặt ra:

Ma trận	PSD?	Bất biến tham số hóa?	Tương đương Hessian khi nào
Hessian H	Không (có thể âm)	Không, trừ trường hợp đặc biệt	Luôn (chính là H)
GGN G	Luôn PSD	Có (dạng $\mathbb{E}[J_f^\top A J_f]$)	Khi dự đoán tối ưu ($\nabla_z L \approx 0$) hoặc f tuyến tính (mạng tuyến tính)
Fisher F	Luôn PSD	Có	Khi $Q_{x,y} = P_{x,y}(\theta)$ (điều kiện "realizability")
Empirical Fisher $\bar{F}$	Luôn PSD	Có (paper chứng minh riêng ở mục 12)	Chỉ khi "realizability" đúng, nói chung không xấp xỉ tốt H

Phân tích tốc độ hội tụ tiệm cận (mục 14), điểm mấu chốt của ablation lý thuyết: paper chỉ ra rằng kết quả "Fisher efficiency" nổi tiếng của Amari (1998), hàm ý natural gradient descent đạt phương sai nhỏ nhất có thể theo chặn Cramér-Rao, đi kèm bốn điều kiện ngặt mà paper tự liệt kê: (1) phải giả định hội tụ về optimum toàn cục (chỉ hợp lý khi h lồi, không đúng cho mạng neural thường không lồi); (2) Fisher F phải tính trên toàn

bộ tập huấn luyện (full-batch), không cho phép mini-batch; (3) kết quả đo theo sai số tham số $\theta_k - \theta^*$, không trực tiếp theo giá trị hàm mục tiêu; (4) cần điều kiện "realizability" (mô hình phải khớp hoàn toàn phân phối huấn luyện tại θ^*), một điều kiện mà paper gọi là "tinh vi" (subtle) và thường không đúng trong thực hành.

Về vai trò thực sự của phương pháp bậc hai: kết quả 4 ở mục 4 (Theorem 5, 6) cho thấy lợi ích của natural gradient/GGN không nằm ở tốc độ hội tụ tiệm cận dài hạn (nơi SGD với Polyak averaging cũng đạt cùng tốc độ, theo đúng chặn Cramér-Rao, mà không cần ma trận độ cong nào), mà nằm ở việc cải thiện mạnh thành phần phụ thuộc điểm khởi tạo khi Hessian thật H^* bị "ill-conditioned" (tỉ số điều kiện $\kappa(H^*) = \lambda_1(H^*)/\lambda_n(H^*)$ lớn) và số vòng lặp còn hạn chế, chưa đạt chế độ tiệm cận. Đây là một phân tích khá tinh tế: nó giải thích **tại sao** natural gradient có lợi trong thực hành (giai đoạn đầu huấn luyện, hữu hạn bước) nhưng cũng chỉ ra rõ **giới hạn** của lợi ích đó (không giúp gì ở chế độ tiệm cận dài hạn nếu đã dùng averaging).

Về tính bất biến tham số hóa (mục 12): đây là một phân tích "phê bình" đúng nghĩa của paper, không chỉ khẳng định tính bất biến mà còn chỉ rõ nó chỉ là xấp xỉ (approximate), với sai số tỉ lệ $O(\|d_\gamma\|^2)$ phụ thuộc độ cong của hàm đổi tham số hóa ζ và độ lớn bước đi; tính bất biến chính xác chỉ giữ được trong giới hạn bước đi tiến tới 0 (đường đi mượt lý tưởng), không giữ được chính xác cho bước hữu hạn trong thực hành.

6. Giới hạn

Các giới hạn mà **bài báo nêu rõ**, đặc biệt ở mục 15 "Conclusions and Open Questions" (một phần thể hiện paper này khá trung thực/tự phê bình so với nhiều bài khác trong batch):

- **Lý thuyết hội tụ của chính paper không giải thích được hiện tượng thực nghiệm mà nó muốn lý giải.** Paper viết thẳng: các chặn hội tụ ở mục 14 (dựa trên giả định hàm mục tiêu là bậc hai với độ cong không đổi) "luôn được cải thiện khi dùng Hessian thay vì GGN" theo lý thuyết đó, do vậy chính các chặn này **không thể giải thích** được việc GGN quan sát thực nghiệm lại tốt hơn Hessian khi huấn luyện mạng neural, và cũng không giải thích được vì sao kỹ thuật damping lại quan trọng đến vậy trong thực hành. Đây là một giới hạn cốt lõi, bài báo lưu ý và để lại như câu hỏi mở.
- Phần lý giải vì sao GGN có thể tốt hơn Hessian (mục 8.1) được paper tự gọi là "speculation" (suy đoán), không phải kết quả đã chứng minh chặt.
- Kết quả "Fisher efficiency" cổ điển của Amari cần bốn điều kiện ngặt (mục 5 trên đã liệt kê: hội tụ toàn cục, Fisher tính full-batch, đo theo tham số không theo hàm mục tiêu, và điều kiện "realizability"), hầu hết không đúng với huấn luyện mạng neural thực tế.
- Phân tích tốc độ hội tụ (Theorem 5, 6) chỉ áp dụng cho hàm mục tiêu lồi bậc hai với nhiễu gradient có hiệp phương sai không đổi theo θ, một giả định bài báo lưu ý là "somewhat unrealistic" (không thực tế lắm), chỉ xem là gần đúng khi θ đã gần điểm tối ưu.
- Tính bất biến tham số hóa chỉ là xấp xỉ cho bước hữu hạn, không chính xác tuyệt đối (mục 12, đã nêu ở mục 5).
- Paper để lại ba câu hỏi mở tường minh ở cuối bài: có thể chứng minh chặt lợi thế của GGN so với Hessian cho mạng neural không; có tình huống nào Fisher và GGN khác biệt rõ ràng và một trong hai chắc chắn tốt hơn; khi nào lợi thế "trước tiệm cận" (pre-asymptotic) của phương pháp bậc hai so với SGD+averaging thực sự quan trọng trong thực hành.

Quan sát của người ghi chú (không phải paper tự phê bình): toàn bộ bài không có một thí nghiệm huấn luyện mạng neural cụ thể nào để kiểm chứng trực tiếp các định lý; mọi minh họa định lượng (như ví dụ $n = 1$ ở mục 11.1) đều là các case study toán học đơn giản, không phải benchmark thực nghiệm. Đây khác với kỳ vọng thông thường về một bài JMLR dài 76 trang, phần lớn độ dài nằm ở chứng minh toán học (phụ lục A-E, khoảng 1900 dòng) chứ không phải ở phần thực nghiệm.

7. Liên hệ với các bài khác

- **rl content/04_2015_trpo.md:** TRPO (Trust Region Policy Optimization) là ví dụ ứng dụng trực tiếp và nổi tiếng nhất của natural gradient trong RL: TRPO ước lượng bước cập nhật policy bằng cách giải gần đúng $F^{-1}\nabla h$ (natural gradient) thông qua Fisher-vector product và conjugate gradient, chính là kỹ thuật tính matrix-vector product với F mà paper này mô tả ở mục 9.1 (nhân bằng J_f, rồi bằng F_R, rồi bằng $J_f^\top$, giống hệt cách Schraudolph tính matrix-vector product với GGN). Khái niệm "trust region" trong TRPO cũng chính là kỹ thuật damping mà paper này đề xuất ở mục 10 để làm natural gradient ổn định với bước lớn.
- **math 07 (An Elementary Introduction to Information Geometry, bài sau trong batch):** tổng quát hóa chính cách hiểu hình học ở mục 3.3 của bài này (Fisher như một metric tensor Riemannian trên không gian phân phối) thành một khung đầy đủ của information geometry (affine connections, dually flat manifolds, họ phân phối mũ).
- **math 05 (A Tutorial on Spectral Clustering, bài trước trong batch):** cả hai bài đều xoay quanh một ma trận "đặc trưng" của mô hình/dữ liệu (Laplacian của đồ thị ở math 05, Fisher information ở bài này) mà ta cần hiểu rõ cấu trúc riêng (eigenvalue/eigenvector) của nó để thiết kế thuật toán hiệu quả; tuy nhiên hai bài dùng ma trận này theo cách khác nhau hẳn (phân cụm so với tối ưu hóa).
- **math 01 (The Matrix Calculus You Need For Deep Learning, bài đầu batch):** Fisher information matrix và GGN trong bài này đều được xây dựng trực tiếp từ Jacobian J_f của mạng neural theo tham số, đúng loại đối tượng (Jacobian, Hessian của hàm hợp) mà math 01 dạy cách tính bằng chain rule.

8. Gợi ý áp dụng

Natural gradient mô tả bước cập nhật theo hình học của phân phối, nhưng ma trận Fisher đầy đủ có thể rất lớn. Các xấp xỉ như block-diagonal hay K-FAC giảm chi phí bằng cách chấp nhận sai lệch xấp xỉ; empirical Fisher cũng không đồng nhất với Fisher thật.

9. Thí nghiệm gợi ý

1. **Tái tạo ví dụ $n = 1$ của paper (mục 11.1) bằng code:** cài đặt chính xác ví dụ $f(x, \theta) = \theta$, $R_{y|z} = \mathcal{N}(z, 1)$, $h(\theta) = \frac{1}{2}\theta^2$ bằng PyTorch, chạy cả hai dãy cập nhật (dùng $F = 1$ và dùng $\bar{F} = \theta^2$) với vài giá trị α và θ_0 khác nhau, vẽ đồ thị θ_k theo k. Đo: giá trị α tối đa còn giữ được sự hội tụ cho mỗi loại cập nhật. Kỳ vọng: xác nhận lại kết quả tự tính ở mục 3.7 (Fisher ổn định, empirical Fisher dễ phân kỳ với bước cố định).
2. **So sánh SGD, Adam, và một xấp xỉ diagonal Fisher thật (không phải empirical) trên một mạng nhỏ** (ví dụ MLP 2-3 lớp trên MNIST hoặc một bài toán hồi quy toy), ước lượng diagonal Fisher thật bằng cách sample y từ phân phối dự đoán của mô hình rồi lấy gradient theo mẫu đó (đúng kỹ thuật mà paper gợi ý ở mục 11.1, trích dẫn Chapelle & Erhan 2011). Đo tốc độ giảm loss theo số iteration. Kỳ vọng: diagonal Fisher thật hội tụ ổn định hơn diagonal empirical Fisher thô ở learning rate cao.
3. **Kiểm tra tính bất biến tham số hóa gần đúng:** lấy một mạng nhỏ, huấn luyện 2 phiên bản tương đương về mặt hàm số nhưng tham số hóa khác nhau (ví dụ: w trực tiếp so với $w = \exp(u)$ cho một trọng số dương), so sánh đường đi loss khi dùng SGD thường (kỳ vọng khác nhau rõ rệt giữa 2 tham số hóa) so với khi dùng một bước xấp xỉ natural-gradient/diagonal-Fisher (kỳ vọng gần giống nhau hơn, theo đúng Theorem 1/Corollary 2 của paper).
4. **Thử K-FAC đơn giản hóa (per-layer, không Kronecker đầy đủ) trên 1-2 lớp Linear nhỏ nhất của một mô hình nhỏ**, đo thời gian và bộ nhớ cần thêm so với Adam trên cùng GPU, để tự kiểm chứng trực tiếp "rào cản bộ nhớ" đã nêu ở mục 8, từ đó định lượng cụ thể xem natural-gradient-xấp-xỉ có thực sự khả thi ở quy mô lớn hơn một chút (ví dụ vài triệu tham số) trên GPU hay không.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Natural gradient	$\tilde{\nabla} h = F^{-1} \nabla h$
Fisher information matrix	$F = \mathbb{E}_{P_{x,y}}[\nabla \log p(x,y\|\theta) \nabla \log p(x,y\|\theta)^\top] = -\mathbb{E}_{P_{x,y}}[H_{\log p}]$, luôn PSD
Xấp xỉ KL bậc hai	$KL(P(\theta+d)\|P(\theta)) \approx \frac{1}{2} d^\top F d$, cho lý giải hình học của natural gradient
Mô hình bậc hai cục bộ	$M_k(\delta) = \frac{1}{2}\delta^\top B_k \delta + \nabla h(\theta_k)^\top \delta + h(\theta_k)$; $B_k = \beta I$ là gradient descent, $B_k = H$ là Newton
Generalized Gauss-Newton (GGN)	$G = \frac{1}{\|S\|} \sum J_f^\top H_L J_f$, luôn PSD vì L lồi theo z
Điều kiện $F = G$	Đúng khi $R_{y\|z}$ thuộc họ phân phối mũ với z là tham số tự nhiên (ví dụ Gauss với $z = \mu$, softmax/multinomial)
Empirical Fisher	$\bar{F} = \frac{1}{\|S\|} \sum \nabla \log p(y\|x,\theta) \nabla \log p(y\|x,\theta)^\top$, dễ tính nhưng không xấp xỉ tốt H nói chung
Ví dụ tự tính (mục 3.7)	$\theta_0 = 2$, $\alpha = 0.5$: cập nhật bằng $F = 1$ cho dãy $2, 1, 0.5, 0.25, \ldots$ (hội tụ đều); cập nhật bằng $\bar{F} = \theta^2$ cho dãy $2, 1.75, 1.464, 1.123, 0.677, -0.061, 8.14, \ldots$ (phân kỳ)
Điều kiện bất biến tham số hóa	$J_\zeta^\top B_\theta J_\zeta = B_\gamma$; đúng cho $B = \mathbb{E}[J_f^\top A J_f]$ (GGN, Fisher, empirical Fisher), không đúng cho Hessian (trừ ζ affine)
Tốc độ hội tụ tiệm cận (quadratic, SGD bậc hai tổng quát)	$\mathbb{E}[h(\theta_k)] - h(\theta^*)$ tách thành phần phụ thuộc nhiễu (không cải thiện bởi 2nd-order khi có Polyak averaging) và phần phụ thuộc θ_0 (cải thiện mạnh bởi 2nd-order khi H^* ill-conditioned)

11. Kết luận ngắn

Bài báo đưa ra một cách nhìn mới, mang tính thống nhất cho natural gradient: thay vì chỉ là một phép ”chuyển không gian” theo information geometry, natural gradient nên được hiểu là một phương pháp tối ưu bậc hai dùng Fisher information matrix làm ma trận độ cong, và trong rất nhiều trường hợp quan trọng (loss cross-entropy, squared error với các phân phối mũ tương ứng), Fisher trùng với Generalized Gauss-Newton matrix, một ma trận luôn PSD. Cách nhìn này lý giải tại sao các kỹ thuật damping/trust-region của tối ưu hóa bậc hai cổ điển lại cần thiết trong thực hành, và giúp phê bình chính xác điểm yếu của ”empirical Fisher” hay bị dùng nhầm (minh họa bằng ví dụ tính tay ở mục 3.7). Tuy nhiên, điểm đáng chú ý nhất về mặt khoa học là sự trung thực của tác giả ở phần kết luận: chính lý thuyết hội tụ mà paper xây dựng công nhận không giải thích được đầy đủ vì sao GGN/Fisher lại vượt trội so với Hessian trong thực hành huấn luyện mạng neural, để lại đây như câu hỏi mở cho nghiên cứu sau này.

Nguồn gốc: https://arxiv.org/abs/1412.1193

Bài 07

An Elementary Introduction to Information Geometry (2018)

Tác giả: Frank Nielsen (Sony Computer Science Laboratories Inc, Tokyo, Japan)
Venue / arXiv: arXiv:1808.08271 (v2, 06/09/2020); paper không nêu venue hội nghị/tạp chí, phần lời cảm ơn cho biết đây là bài viết dựa trên một bài nói chuyện (invited talk) tại workshop ”Geometry in Machine Learning” (GiMLi) năm 2018
Nguồn: https://arxiv.org/abs/1808.08271
Chủ đề: Hình học thông tin (information geometry), ma trận Fisher information như metric tensor, kết nối đối ngẫu (dual connection), đa tạp dually flat, liên hệ với natural gradient
Vị trí trong lộ trình: Bài số 7 trong 9 bài toán nền (math 01-09). Đọc sau math 06 (natural gradient method), vì bài này tổng quát hóa khung hình học đứng sau khái niệm Fisher information matrix đã gặp ở math 06. Đứng trước math 08 (variational inference), vốn cũng dùng KL divergence nhưng cho mục đích suy luận Bayes xấp xỉ, khác với mục đích xây cấu trúc hình học ở đây.

> Ghi chú giới thiệu các khái niệm chính của hình học thông tin, dựa trên Mục 1–5 và hai phụ lục. Một số chi tiết kỹ thuật, gồm công thức tổng quát của Bregman ball và α-embedding, không được trình bày đầy đủ.

1. Bối cảnh

Bài tổng quan của Frank Nielsen giới thiệu những cấu trúc hình học vi phân nền tảng của hình học thông tin (information geometry, IG), định lý nền tảng của lĩnh vực và một số ứng dụng trong kiểm định giả thuyết Bayes và phân cụm hỗn hợp thống kê. Để giữ bài ngắn gọn, tác giả lược bỏ các chứng minh và dẫn tới các tài liệu chuyên khảo, đặc biệt là hai cuốn của Amari [8, 25].

Theo lời cảm ơn ở cuối bài, đây là bài viết dựa trên một bài nói chuyện mời (invited talk) tại workshop ”Geometry in Machine Learning” (GiMLi) năm 2018, không công bố tại một hội nghị hay tạp chí cụ thể nào trong chính văn bản arXiv này.

Paper có một phần lịch sử khá chi tiết (mục 5.2) đáng chú ý: hình học thông tin bắt nguồn từ việc Harold Hotelling (cuối những năm 1920, ghi chú tay năm 1929) và độc lập C. R. Rao (1945, trong bài báo nổi tiếng nêu ra Cramér-Rao bound) coi ma trận Fisher information là một metric tensor Riemannian trên không gian tham số của một họ phân phối xác suất. Nikolai Chentsov (thập niên 1960-1970, chuyên khảo tiếng Nga) chứng minh Fisher information metric là metric bất biến duy nhất (up to hệ số tỉ lệ) dưới các phép biến đổi Markov. Shun-ichi Amari (từ 1980) phát triển cấu trúc đối ngẫu (dualistic) đầy đủ và đặt tên lĩnh vực là ”information geometry”. Paper trích dẫn trực tiếp định nghĩa của Amari: ”Information geometry is a method of exploring the world of information by means of modern geometry.”

Bài báo gồm năm mục chính và hai phụ lục về ước lượng Monte Carlo cho f-divergence và phân rã canonical của họ Gaussian đa biến.

2. Ý tưởng chính

Một họ tham số các phân phối xác suất $\mathcal{P} = \{p_\theta(x)\}_{\theta \in \Theta}$ có thể được xem là một đa tạp (manifold) M, trong đó mỗi điểm trên đa tạp là một phân phối xác suất cụ thể, và tọa độ địa phương của điểm đó chính là vector tham số θ. Trên một đa tạp trơn như vậy, paper phân biệt rõ hai cấu trúc độc lập có thể đặt lên:

1. **Metric tensor g:** cho phép đo ”độ dài” và góc giữa các vector tiếp xúc (tangent vector) tại mỗi điểm, tức đo khoảng cách vô cùng nhỏ giữa hai phân phối rất gần nhau.

2. **Affine connection ∇:** một toán tử vi phân cho phép định nghĩa đạo hàm hiệp biến (covariant derivative) của một trường vector theo một trường vector khác, từ đó định nghĩa được khái niệm "đường thẳng" tổng quát trên đa tạp (gọi là geodesic, đường tự song song/autoparallel) và cách "vận chuyển song song" (parallel transport) một vector giữa các mặt phẳng tiếp xúc khác nhau.

Điểm khác biệt căn bản giữa hình học thông tin và hình học Riemannian cổ điển: trong hình học Riemannian, với một metric g cho trước, có **duy nhất một** connection thỏa hai điều kiện (không xoắn, tương thích với metric), gọi là connection Levi-Civita ${}^{LC}\nabla$ (Định lý nền tảng của hình học Riemannian, Theorem 1 trong paper). Hình học thông tin đi xa hơn: nó xét **một cặp connection đối ngẫu (conjugate)** (∇, ∇^*) cùng khớp với một metric g (cấu trúc (M, g, ∇, ∇^*)), và connection Levi-Civita chỉ là trường hợp đặc biệt tự đối ngẫu $\nabla = \nabla^* = {}^{LC}\nabla$.

Với một họ tham số thống kê, lựa chọn chính tắc cho metric g là **ma trận Fisher information (FIM)**: paper chứng minh (trích dẫn Chentsov/Amari-Nagaoka) đây là metric **duy nhất** (sai khác một hệ số tỉ lệ) bất biến dưới các phép biến đổi gọi là "Markov embedding" (Theorem 8). Cặp connection đối ngẫu chính tắc tương ứng là connection mũ (exponential connection ∇^e) và connection hỗn hợp (mixture connection ∇^m).

"Định lý nền tảng của hình học thông tin" (Theorem 3): nếu một connection không xoắn ∇ có độ cong hằng κ, thì connection đối ngẫu ∇^* của nó cũng có độ cong hằng κ. Trường hợp đặc biệt và quan trọng nhất trong thực hành là $\kappa = 0$: khi cả ∇ và ∇^* đều "flat" (phẳng, không có độ cong), ta có một **đa tạp dually flat** (Corollary 2). Loại đa tạp này xuất hiện tự nhiên với họ hàm mũ (exponential family, ví dụ Gaussian) và họ hỗn hợp (mixture family), và có cấu trúc đại số rất đẹp: hai hệ tọa độ toàn cục θ và η liên hệ qua biến đổi Legendre-Fenchel, cùng một định lý Pythagore tổng quát (bất đối xứng).

3. Phương pháp chi tiết

3.1. Manifold, metric tensor và affine connection

Một đa tạp trơn D-chiều M là một không gian tôpô mà cục bộ (ở lân cận mỗi điểm) "giống" không gian Euclid $\mathbb{R}^D$. Tại mỗi điểm $p \in M$, có một mặt phẳng tiếp xúc (tangent plane) T_p, là một không gian vector chứa các "hướng đi" khả dĩ tại p. Paper định nghĩa hai cấu trúc trên M:

- **Metric tensor g:** với hai vector tiếp xúc $u, v \in T_p$, $g_p(u, v) = \langle u, v \rangle_p$ là một dạng song tuyến tính đối xứng, xác định dương, cho phép tính độ dài $\|v\| = \sqrt{g(v, v)}$ và tính trực giao ($u \perp v$ khi $g(u, v) = 0$).
- **Affine connection ∇:** cho phép định nghĩa đạo hàm hiệp biến $\nabla_X Y$ của trường vector Y theo trường vector X. Trong hệ tọa độ địa phương, connection được xác định bởi các hệ số Christoffel Γ^k_{ij}, với $\nabla_{\partial_i} \partial_j = \Gamma^k_{ij} \partial_k$.

Từ connection, ta có được: (1) phép vận chuyển song song của vector giữa các mặt tiếp xúc dọc theo một đường cong trơn, (2) các ∇-geodesic (đường tự song song, nghiệm của phương trình vi phân bậc hai $\ddot{\gamma} + \Gamma^k_{ij} \dot{\gamma} \dot{\gamma} = 0$), (3) độ cong và độ xoắn (torsion) nội tại của đa tạp.

Connection gọi là tương thích với metric (metric-compatible) nếu $X\langle Y, Z \rangle = \langle \nabla_X Y, Z \rangle + \langle Y, \nabla_X Z \rangle$ với mọi trường vector X, Y, Z. Định lý nền tảng của hình học Riemannian (Theorem 1 của paper): tồn tại duy nhất một connection không xoắn tương thích với metric, gọi là connection Levi-Civita ${}^{LC}\nabla$, và hình học Riemannian cổ điển (M, g) chính là trường hợp đặc biệt $(M, g, {}^{LC}\nabla)$.

3.2. Cấu trúc đối ngẫu: connection liên hợp (conjugate connection)

Đây là bước mở rộng cốt lõi của hình học thông tin. Paper định nghĩa (Definition 1): connection ∇^* được gọi là **liên hợp (conjugate)** với ∇ đối với metric g nếu với mọi trường vector X, Y, Z:

$$X\langle Y, Z \rangle = \langle \nabla_X Y, Z \rangle + \langle Y, \nabla^*_X Z \rangle$$

So với điều kiện tương thích metric thông thường (mục 3.1), ở đây đạo hàm của Y dùng connection ∇, còn đạo hàm của Z dùng connection khác ∇^*. Tính chất liên hợp là một phép đối hợp: $(\nabla^*)^* = \nabla$. Cấu trúc (M, g, ∇, ∇^*) gọi là **Conjugate Connection Manifold (CCM)**.

Một hệ quả quan trọng (Property 1, công thức 38-39 của paper): phép vận chuyển song song đối ngẫu bảo toàn metric, tức nếu vận chuyển u theo ∇ và v theo ∇^* dọc cùng một đường cong, tích vô hướng $\langle u, v\rangle$ không đổi theo thời gian. Paper cũng chỉ ra (Property 3): connection trung bình $\bar{\nabla} = (\nabla + \nabla^*)/2$ luôn bằng chính connection Levi-Civita, tức ${}^{LC}\nabla$ nằm "ở giữa" cặp connection đối ngẫu.

Từ cặp (∇, ∇^*), paper định nghĩa tensor khối (cubic tensor) Amari-Chentsov, hoàn toàn đối xứng:

$$C_{ijk} := \Gamma^k_{ij} - \Gamma^{*k}_{ij}$$

Cấu trúc (M, g, C) gọi là **statistical manifold** theo định nghĩa của Lauritzen (1987); paper lưu ý tên gọi này có thể gây nhầm, vì đây thuần túy là một cấu trúc hình học, không bắt buộc liên quan đến thống kê (dù luôn có thể tìm một mô hình thống kê tương ứng).

3.3. Họ connection α và định lý nền tảng

Từ một cặp conjugate (∇, ∇^*), paper xây dựng một họ liên tục theo tham số $\alpha \in \mathbb{R}$, gọi là α-connection:

$$\Gamma^\alpha_{ij,k} = \Gamma^0_{ij,k} - \frac{\alpha}{2} C_{ij,k}$$

với Γ^0 là hệ số Christoffel của connection Levi-Civita. Khi $\alpha = 0$ ta có lại Levi-Civita; khi $\alpha = 1$ ta có lại ∇; khi $\alpha = -1$ ta có lại ∇^*.

Định lý nền tảng của hình học thông tin (Theorem 3 của paper, trích dẫn chứng minh từ [25]): nếu một connection không xoắn ∇ có độ cong hằng κ, thì connection liên hợp ∇^* của nó cũng có độ cong hằng κ (y hệt). Hệ quả quan trọng nhất (Corollary 2): đa tạp (M, g, ∇, ∇^*) là ∇-flat (độ cong $\kappa = 0$) khi và chỉ khi nó cũng ∇^*-flat. Trường hợp $\kappa = 0$ này được gọi là **dually flat manifold**, và đây là trường hợp quan trọng nhất trong các ứng dụng thực hành vì có cấu trúc đại số đơn giản (xem mục 3.4).

3.4. Đa tạp dually flat và hình học Bregman

Paper xây dựng đa tạp dually flat từ một hàm sinh (generator) $F(\theta)$ lồi chặt, trơn C^3, gọi là potential function. Từ F, định nghĩa **Bregman divergence**:

$$B_F(\theta : \theta') := F(\theta) - F(\theta') - (\theta - \theta')^\top \nabla F(\theta')$$

Cấu trúc hình học sinh ra từ B_F có các hệ số Christoffel $\Gamma^F_{ij,k}(\theta) = 0$ với mọi θ (Eq. 59 của paper), tức connection này luôn phẳng (flat) theo xây dựng, với **bất kỳ** hàm lồi F nào, không riêng gì các họ phân phối xác suất. Metric tensor tương ứng chính là Hessian của F: $g^F = \nabla^2 F(\theta)$.

Dùng biến đổi Legendre-Fenchel, định nghĩa hàm đối ngẫu $F^*(\eta) := \sup_\theta\{\theta^\top \eta - F(\theta)\}$, và hệ tọa độ đối ngẫu $\eta = \nabla F(\theta)$, $\theta = \nabla F^*(\eta)$. Paper nêu identity của Crouzeix: $\nabla^2 F(\theta)\, \nabla^2 F^*(\eta) = I$. Trên đa tạp dually flat, mọi điểm đều có hai hệ tọa độ toàn cục θ và η, và một cặp điểm P, Q có thể nối bằng geodesic "thẳng theo θ" (∇-geodesic) hoặc "thẳng theo η" (∇^*-geodesic).

Kết quả đẹp nhất của cấu trúc này là **định lý Pythagore đối ngẫu** (Theorem 6, minh họa ở Hình 6 của paper): nếu geodesic đối ngẫu $\gamma^*(P, Q)$ vuông góc với geodesic $\gamma(Q, R)$ (theo metric g) thì:

$$D(P : R) = D(P : Q) + D(Q : R)$$

đây là một dạng tổng quát hóa định lý Pythagore của Euclid cho không gian "cong" nhưng dually flat, với D là divergence (không đối xứng) thay cho khoảng cách Euclid thường.

3.5. Ma trận Fisher information và tính bất biến duy nhất

Cho một họ tham số $\mathcal{P} = \{p_\theta(x)\}_\theta$, với log-likelihood $l(\theta; x) := \log p_\theta(x)$ và score vector $s_\theta = \nabla_\theta l$. Ma trận Fisher information (FIM) được định nghĩa:

$$I(\theta) := \mathbb{E}_\theta[\partial_i l \, \partial_j l]_{ij} \succeq 0$$

Paper chỉ ra FIM là bất biến dưới đổi biến không gian mẫu và biến đổi hiệp biến (covariant) dưới đổi tham số: nếu $\bar{p}(x; \eta) = p(\theta(\eta); x)$, thì $\bar{I}(\eta) = J^\top I(\theta(\eta)) J$ với $J = [\partial\theta_i / \partial\eta_j]$ là ma trận Jacobian (Eq. 81).

Theorem 8 (tính duy nhất của Fisher information metric): Fisher information metric là metric tensor bất biến duy nhất (sai khác một hệ số tỉ lệ) dưới các phép nhúng Markov (Markov embedding). Nói cách khác, trong số vô số metric tensor có thể đặt lên không gian tham số của một mô hình thống kê, chỉ có Fisher metric (và các phiên bản co giãn của nó) thỏa mãn một yêu cầu bất biến tự nhiên trong thống kê. Đây là lý do toán học vì sao Fisher information "phải" được dùng làm metric chuẩn, không phải một lựa chọn tùy ý trong số nhiều lựa chọn tương đương.

Paper cho công thức FIM cho hai loại họ tham số quan trọng:

- **Họ hàm mũ (exponential family)** $p_\theta(x) = \exp(\sum_i t_i(x)\theta_i - F(\theta) + k(x))$ với F là hàm cumulant (log-normalizer) lồi chặt: $I(\theta) = \mathrm{Cov}_{p_\theta}[t(x)] = \nabla^2 F(\theta)$ (Eq. 90). Tức với họ hàm mũ, FIM chính là Hessian của hàm cumulant, và connection sinh ra tự động là dually flat.
- **Họ hỗn hợp (mixture family)** $p_\theta(x) = \sum_i \theta_i F_i(x) + C(x)$: $I(\theta) = \mathbb{E}\left[\dfrac{F_i(x) F_j(x)}{p_\theta(x)^2}\right]$ (Eq. 94).

Ví dụ cụ thể paper cho với Gaussian đơn biến $\mathcal{N}(\mu, \sigma)$: theo tham số hóa $\lambda = (\mu, \sigma)$, $I_\lambda(\lambda) = \mathrm{diag}(1/\sigma^2,\ 2/\sigma^2)$; theo tham số hóa $\lambda' = (\mu, \sigma^2)$, $I_{\lambda'}(\lambda') = \mathrm{diag}(1/\sigma^2,\ 1/(2\sigma^4))$ (Eq. 83-84). Hai ma trận này khác nhau nhưng liên hệ đúng theo công thức biến đổi hiệp biến ở trên, không mâu thuẫn.

3.6. Fisher-Rao distance và hai connection chính tắc cho thống kê

Khi coi FIM là metric Riemannian $g = I(\theta)$ trên $\mathcal{P}$, khoảng cách geodesic Riemannian tương ứng gọi là **Fisher-Rao distance**:

$$D_\rho(p_{\theta_1}, p_{\theta_2}) = \inf_\gamma \int_0^1 \sqrt{\dot\gamma(t)^\top g_{\gamma(t)} \dot\gamma(t)}\, dt$$

Paper cho ba ví dụ cụ thể về hình học sinh ra: họ phân phối rời rạc hữu hạn (categorical) cho hình học cầu (spherical); họ location-scale hai biến cho hình học hyperbolic; họ chỉ-dịch-chuyển (location family, scale cố định) cho hình học Euclid thường.

Hai connection chính tắc gắn với Fisher metric cho một họ tham số thống kê là connection mũ ∇^e và connection hỗn hợp ∇^m (Eq. 96-97 của paper). Paper chỉ ra (Eq. 104): với chính họ hàm mũ E và họ hỗn hợp M, các hệ số Christoffel của cả ∇^e và ∇^m đều triệt tiêu, nghĩa là cả hai loại họ phân phối này tự động là dually flat theo đúng cặp connection chính tắc của chúng, không cần xây dựng gượng ép.

3.7. Ứng dụng natural gradient (liên hệ trực tiếp math 06)

Paper trình bày lại natural gradient trong khung hình học này (mục 4.1). Gradient thường (ordinary gradient descent) không bất biến khi đổi tham số hóa: nếu $\eta = \eta(\theta)$ là một đổi biến trơn, hai dãy gradient descent trên θ và trên η nói chung hội tụ về các điểm khác nhau, vì gradient thường không tính đến hình học (độ cong, metric) của không gian tham số. Paper minh họa bằng ví dụ rất trực quan: tối thiểu hóa một hàm nhiệt độ

theo độ C và theo độ F (cùng một đại lượng vật lý, hai cách đo) có thể cho hai đường đi tối ưu khác nhau nếu dùng gradient thường.

Natural gradient khắc phục vấn đề này bằng cách định nghĩa:

$$^{NG}\nabla L_\theta(\theta) := g_\theta^{-1}(\theta)\,\nabla_\theta L_\theta(\theta)$$

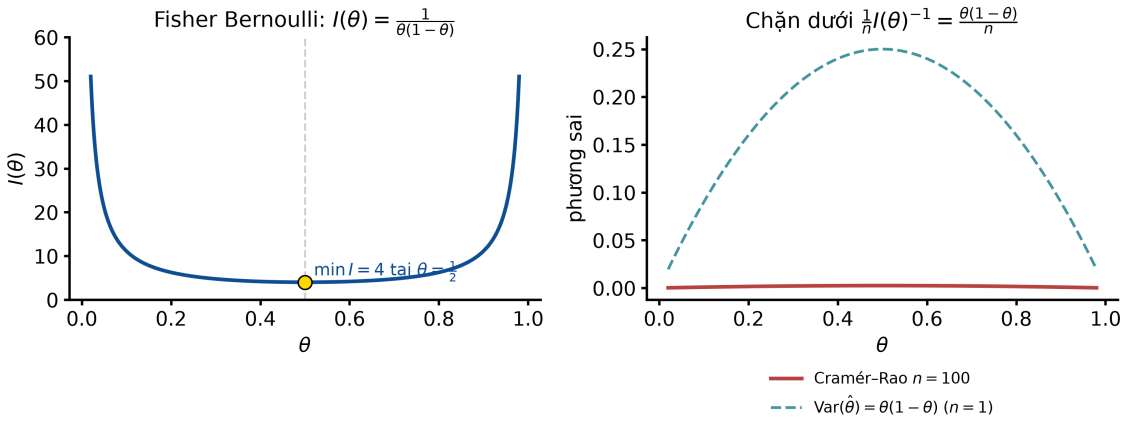
đây xấp xỉ hướng dốc nhất nội tại (intrinsic Riemannian steepest descent) qua một phép xấp xỉ bậc một của ánh xạ mũ Riemannian (Riemannian exponential map), vốn thường không tính được trong thực hành (Property 5 của paper). Paper chứng minh một kết quả đẹp (**Property 7**, suy ra trực tiếp từ chain rule, Eq. 183-186): trên một đa tạp dually flat sinh từ hàm F, với $g_\theta(\theta) = \nabla^2 F(\theta) = \nabla_\theta \eta$, ta có:

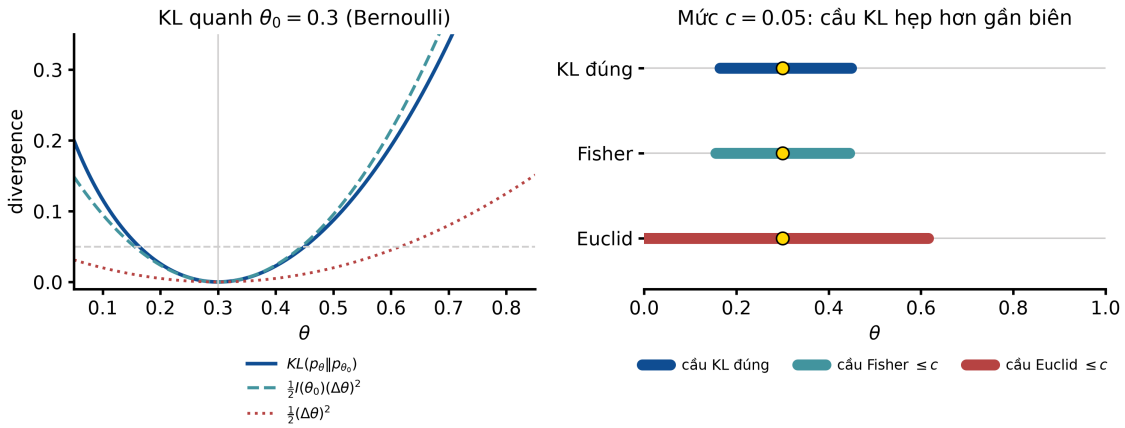
$$\nabla_\theta L_\theta(\theta) = (\nabla_\theta \eta)(\nabla_\eta L_\eta(\eta)) \quad \Rightarrow \quad ^{NG}\nabla L_\theta(\theta) = g_\theta^{-1}(\theta)\nabla_\theta L_\theta(\theta) = (\nabla_\theta \eta)^{-1}(\nabla_\theta \eta)\nabla_\eta L_\eta(\eta) = \nabla_\eta L_\eta(\eta)$$

Nói cách khác: **natural gradient descent trên tham số hóa θ chính là gradient descent thường trên tham số hóa đối ngẫu $\eta = \nabla F(\theta)$**. Đây là một cách diễn đạt rất gọn và sâu sắc cho chính hiện tượng ”bất biến theo tham số hóa” mà bài math 06 (Martens) thảo luận bằng ngôn ngữ đại số tuyến tính (Fisher = Generalized Gauss-Newton).

3.8. Ví dụ số tự tính: Fisher information của phân phối Bernoulli

Paper chỉ cho công thức FIM minh họa cụ thể bằng số cho họ Gaussian (mục 3.5 trên), không tính cho phân phối Bernoulli. Để tự kiểm chứng công thức tổng quát $I(\theta) = \mathbb{E}_\theta[(\partial_\theta l)^2]$ (Eq. 80, trường hợp 1 chiều) bằng một ví dụ khác (tự tính), xét $X \sim \text{Bernoulli}(\theta)$, $p_\theta(x) = \theta^x(1-\theta)^{1-x}$ với $x \in \{0, 1\}$.





Cầu KL đúng / xấp xỉ Fisher / cầu Euclid quanh θ_0=0.3 Bernoulli: gần biên cầu KL hẹp hơn Euclid. (minh hoạ, tự tính)

Fisher Bernoulli I(θ)=1/(θ(1−θ)) cực tiểu 4 tại θ=1/2; Cramér–Rao Var≥θ(1−θ)/n. (minh hoạ, tự tính)

Log-likelihood: $l(\theta; x) = x \log \theta + (1 - x) \log(1 - \theta)$. Đạo hàm theo θ:

$$\partial_\theta l = \frac{x}{\theta} - \frac{1 - x}{1 - \theta}$$

Tại $x = 1$: $\partial_\theta l = 1/\theta$. Tại $x = 0$: $\partial_\theta l = -1/(1 - \theta)$. Áp dụng định nghĩa FIM (lấy kỳ vọng theo $x \sim$ Bernoulli(θ), $P(x = 1) = \theta$, $P(x = 0) = 1 - \theta$):

$$I(\theta) = \theta \cdot \left(\frac{1}{\theta}\right)^2 + (1 - \theta) \cdot \left(\frac{1}{1 - \theta}\right)^2 = \frac{1}{\theta} + \frac{1}{1 - \theta} = \frac{1}{\theta(1 - \theta)}$$

Thay số cụ thể (tự tính): tại $\theta = 0.5$: $I(0.5) = 1/(0.5 \times 0.5) = 4$. Tại $\theta = 0.2$: $I(0.2) = 1/(0.2 \times 0.8) = 1/0.16 = 6.25$. Tại $\theta = 0.05$: $I(0.05) = 1/(0.05 \times 0.95) \approx 21.05$.

Quan sát (của người ghi chú, suy ra từ phép tính trên, không phải trích từ paper): $I(\theta)$ nhỏ nhất tại $\theta = 0.5$ và tăng vọt khi θ tiến về 0 hoặc 1. Điều này có nghĩa là, theo đúng tinh thần "Fisher information đo lượng thông tin về θ mà một quan sát mang lại", khi θ gần biên (0 hoặc 1, kết quả gần như chắc chắn), một mẫu quan sát duy nhất lại mang nhiều thông tin hơn về giá trị chính xác của θ so với khi $\theta = 0.5$ (kết quả ngẫu nhiên nhất). Theo Cramér-Rao bound mà paper nhắc tới (Eq. 88), phương sai của một ước lượng không chệch của θ bị chặn dưới bởi $\frac{1}{n} I(\theta)^{-1} = \frac{\theta(1-\theta)}{n}$, đây đúng là công thức phương sai chuẩn của ước lượng tỉ lệ mẫu (sample proportion) quen thuộc trong thống kê cơ bản, cho thấy công thức FIM tổng quát của paper khớp với một kết quả đã biết.

4. Thí nghiệm và kết quả chính

Đây là một bài tutorial/survey lý thuyết, tự nêu rõ bỏ qua các chứng minh chi tiết, và **không có benchmark thực nghiệm học máy nào** theo nghĩa huấn luyện mô hình, so sánh với baseline, hay đo trên một bộ dữ liệu chuẩn. "Kết quả chính" của bài là các định lý và cấu trúc hình học:

- **Theorem 3 (định lý nền tảng):** độ cong hằng của ∇ và connection liên hợp ∇^* của nó luôn bằng nhau; hệ quả quan trọng nhất là tính dually flat (Corollary 2).
- **Theorem 8:** Fisher information metric là metric bất biến duy nhất dưới Markov embedding.
- **Theorem 6 (Pythagore đối ngẫu)** trên đa tạp dually flat.
- **Property 7:** natural gradient trên θ bằng gradient thường trên tọa độ đối ngẫu η.

Về phần minh họa bằng số, paper có một vài hình vẽ mang tính mô phỏng nhỏ, gần nhất với ”thực nghiệm” trong bài, cần phân biệt rõ đây không phải benchmark có baseline:

- **Hình 8 (minh họa Cramér-Rao bound):** paper lấy các vị trí lưới (μ, σ) của tham số Gaussian, với mỗi vị trí lặp lại 200 lần việc ước lượng MLE của (μ, σ) từ 100 mẫu iid $\sim \mathcal{N}(\mu, \sigma)$, rồi vẽ elip biểu diễn ma trận hiệp phương sai mẫu của các ước lượng MLE (elip đen) so với elip biểu diễn chính ma trận Fisher information tại vị trí lưới đó (elip đỏ). Paper quan sát: elip Fisher (đỏ) luôn có trục song song với hệ tọa độ (vì FIM của Gaussian là ma trận đường chéo theo (μ, σ)), còn elip hiệp phương sai mẫu (đen) của MLE thì không, và tâm của elip đen có thể lệch khỏi đúng vị trí lưới.
- **Hình 14-15 (minh họa phân cụm hỗn hợp thống kê):** paper dựng một họ hỗn hợp (mixture family) bậc $D = 2$ từ 3 phân phối thành phần cho trước (Laplace, Gaussian, Cauchy), minh họa 2 hỗn hợp cụ thể (Hình 14), sau đó áp dụng Bregman k-means trên một đa tạp dually flat xấp xỉ bằng Monte Carlo (vì entropy của hỗn hợp không có dạng đóng) để phân một tập các w-GMM (Gaussian Mixture Model với trọng số khác nhau nhưng cùng tham số thành phần) thành $k = 2$ cụm (Hình 15).

Cả hai minh họa này đều mang tính trực quan hóa khái niệm, không có so sánh định lượng với phương pháp khác hay số liệu bảng biểu kèm theo trong văn bản được đọc.

5. Phân tích / ablation

Vì không có thí nghiệm định lượng, phần này tập trung phân tích các lựa chọn cấu trúc mà paper trình bày và sự khác biệt giữa chúng:

Fisher-Rao distance (metric thật) vs Bregman/KL divergence (không đối xứng): paper tự tổng kết sự khác biệt này trong Hình 10 (được mô tả lại bằng lời ở đây): Fisher-Rao distance là một khoảng cách Riemannian thật (đối xứng, thỏa bất đẳng thức tam giác) nhưng **thường không có dạng đóng** và phải tính xấp xỉ bằng số; còn Bregman divergence (sinh từ đa tạp dually flat) không đối xứng nói chung, nhưng geodesic trên đa tạp dually flat là đường thẳng trong hệ tọa độ phù hợp, nên tính toán rất hiệu quả. Đây là một trade-off cốt lõi của toàn bộ khung lý thuyết: đổi tính đối xứng/đúng nghĩa ”khoảng cách” để lấy khả năng tính toán khép kín.

Vai trò của tham số α: họ α-connection cho một cách nội suy liên tục giữa connection Levi-Civita tự đối ngẫu ($\alpha = 0$) và cặp connection ”cực trị” (∇, ∇^*) ($\alpha = \pm 1$). Với họ hàm mũ và họ hỗn hợp, $\alpha = \pm 1$ (connection mũ/hỗn hợp) cho dually flat; các giá trị α khác nói chung không cho connection phẳng.

Fisher information thực nghiệm có thể không đường chéo dù Fisher information lý thuyết là đường chéo: đây là một quan sát rất cụ thể mà bài báo nêu ở mục 4.1.4 (ứng dụng Natural Evolution Strategies): với Gaussian (μ, σ^2), FIM lý thuyết (kỳ vọng, ”expected FIM”) là ma trận đường chéo $\mathrm{diag}(1/\sigma^2, 1/(2\sigma^4))$, nhưng FIM **thực nghiệm** (trung bình mẫu của tích ngoài score vector, Eq. 189 của paper) ”các phần tử ngoài đường chéo gần 0 nhưng thường không chính xác bằng 0” (nguyên văn gần đúng ý paper). Paper gợi ý một cách thực hành: chỉ ước lượng FIM cho tới khi các phần tử ngoài đường chéo nhỏ hơn một ngưỡng ϵ cho trước. Đây là một điểm quan trọng, liên hệ trực tiếp tới phê bình ”empirical Fisher approximation” ở bài math 06.

So sánh hình học Fisher-Rao và hình học optimal transport (nêu ở phần kết luận, mục 5.1 của paper): paper đưa ra một quan sát thú vị (trích dẫn công trình khác, không tự chứng minh trong bài): với cùng một họ location-scale, hình học Fisher-Rao cho độ cong **âm** (hyperbolic), còn hình học Wasserstein/optimal-transport cho độ cong **dương**. Đây là một minh chứng rõ ràng rằng ”chọn cách đo khoảng cách nào giữa các phân phối” (Fisher-Rao hay Wasserstein) dẫn tới các cấu trúc hình học rất khác nhau, dù cùng áp dụng cho một họ phân phối.

6. Giới hạn

Các giới hạn **bài báo nêu hoặc có thể nhận thấy rõ từ cách trình bày**:

- Paper tự tuyên bố ngay từ tóm tắt: ”proofs are omitted for brevity”. Hầu hết các định lý quan trọng (Theorem 3, Theorem 8, các công thức Pythagore) được trích dẫn từ các chuyên khảo khác của Amari và các đồng nghiệp, không được chứng minh lại trong chính bài. Người đọc muốn kiểm chứng chặt chẽ phải tra cứu tài liệu gốc.
- Fisher-Rao distance (khoảng cách Riemannian thật) ”thường không có dạng đóng” (paper nêu rõ, mục 2.5) và cần phương pháp số như geodesic shooting để tính; đây là một hạn chế thực hành đáng kể nếu muốn dùng chính Fisher-Rao distance (thay vì Bregman/KL divergence) trong ứng dụng.
- Entropy của một hỗn hợp thống kê (ví dụ Gaussian mixture) nói chung **không có dạng đóng** (paper nêu rõ ở mục 4.4, do có tổng log bên trong tích phân), nên việc dựng đa tạp dually flat cho họ hỗn hợp trong thực hành phải dựa vào xấp xỉ Monte Carlo, không phải công thức chính xác.
- Không phải mọi α-geometry đều tự nhiên là dually flat: paper nêu ví dụ họ phân phối Cauchy là một trường hợp α-geometry không dually flat theo cấu trúc tự nhiên, cần một hàm sinh đặc biệt (”Bregman-Tsallis generator”) để ép về dạng dually flat.
- Bài không cung cấp mã nguồn, không có thí nghiệm có thể tái lập theo nghĩa thông thường của học máy (không huấn luyện mô hình, không báo cáo độ chính xác/tốc độ trên dữ liệu thật).
- Mật độ ký hiệu và khái niệm hình học vi phân (connection, curvature, tensor, biconjugation Legendre-Fenchel) rất cao đối với người mới; paper giả định người đọc đã quen với đại số tuyến tính và có thể theo dõi các định nghĩa hình thức, dù tự gọi là ”elementary introduction”.

7. Liên hệ với các bài khác

- **math 06 (New Insights and Perspectives on the Natural Gradient Method, Martens):** liên hệ trực tiếp và sâu nhất. Mục 4.1 của bài này trình bày lại chính natural gradient bằng ngôn ngữ hình học vi phân, và Property 7 (mục 3.7 ở trên) chứng minh một cách rất gọn tính bất biến theo tham số hóa mà math 06 lập luận qua đẳng thức Fisher = Generalized Gauss-Newton. Ngoài ra, cả hai bài đều nêu đúng cùng một vấn đề thực hành: ma trận Fisher information **thực nghiệm/xấp xỉ** (empirical Fisher) có thể không giữ đúng cấu trúc (ví dụ tính đường chéo) của Fisher information lý thuyết, dù dùng hai cách diễn đạt khác nhau (math 07 minh họa cụ thể bằng ví dụ Gaussian ở §4.1.4; math 06 phê bình tổng quát hơn).
- **math 08 (Variational Inference: A Review for Statisticians, bài sau):** cùng dùng công cụ KL divergence nhưng cho mục đích hoàn toàn khác, suy luận Bayes xấp xỉ bằng tối ưu hóa (tối đa ELBO), không phải xây cấu trúc hình học của không gian tham số. Công thức KL giữa hai Gaussian ở Phụ lục B của bài này (Eq. 235) là chính công thức sẽ cần dùng lại khi tính ELBO với họ biến phân Gaussian ở math 08/09.
- **math 12 (Computational Optimal Transport):** bài báo nêu ở phần kết luận (mục 5.1) một sự đối lập cụ thể giữa hình học Fisher-Rao và hình học Wasserstein/optimal-transport trên cùng một họ location-scale (độ cong âm so với dương). Math 12 sẽ đi sâu vào chính loại hình học ”khoảng cách giữa phân phối” thứ hai này.

8. Gợi ý áp dụng

Hình học thông tin giúp diễn giải các divergence và natural gradient trong không gian phân phối. Chiều của KL cần được nêu rõ: đổi thứ tự hai phân phối nói chung làm thay đổi giá trị và hành vi tối ưu.

9. Thí nghiệm gợi ý

Các thí nghiệm nhỏ dưới đây chạy được trên GPU (PyTorch/NumPy):

1. **Tái hiện ý tưởng Hình 8 của paper cho Bernoulli:** với θ cố định (ví dụ 0.3), lấy mẫu n quan sát Bernoulli, tính MLE $\hat{\theta}$ (tỉ lệ mẫu), lặp lại 200 lần, đo phương sai thực nghiệm của $\hat{\theta}$ qua 200 lần, so

sánh với cận Cramér-Rao $\theta(1-\theta)/n$ tính từ công thức $I(\theta) = 1/(\theta(1-\theta))$ ở mục 3.8 (tự tính). Kỳ vọng: phương sai thực nghiệm xấp xỉ cận lý thuyết khi n đủ lớn.

2. **Fisher information thực nghiệm vs lý thuyết cho một lớp softmax nhỏ:** lấy một mô hình phân loại nhỏ (vài lớp), tính FIM thực nghiệm (trung bình ngoài tích của gradient log-likelihood trên một batch) theo đúng công thức Eq. 189 của paper, so sánh với cấu trúc đường chéo kỳ vọng (nếu mô hình tương đương một họ hàm mũ đơn giản như softmax). Đo mức độ lệch khỏi đường chéo khi tăng kích thước batch dùng để ước lượng.
3. **Kiểm tra tính bất biến của natural gradient (Property 5/7):** cài một bài toán tối ưu nhỏ (ví dụ logistic regression) với 2 cách tham số hóa khác nhau cho một tham số dương (ví dụ θ và $\log\theta$), chạy gradient descent thường trên cả hai và quan sát đường đi khác nhau; chạy natural gradient descent (dùng Fisher xấp xỉ diagonal) trên cả hai tham số hóa và kiểm tra đường đi có ”giống nhau hơn” (ít nhất về quỹ đạo của phân phối dự đoán) hay không.
4. **Tính Bregman/KL divergence giữa hai Gaussian đa biến** dùng công thức đóng ở Phụ lục B của paper (Eq. 235), áp dụng cho embedding/activation của hai checkpoint mô hình nhỏ khác nhau (giả định gần Gaussian), xem khoảng cách này biến đổi ra sao khi nén mô hình nhiều hay ít.
5. **Mô phỏng Chernoff information đơn giản (mục 4.3 của paper):** với hai Gaussian 1 chiều cụ thể, tính Bhattacharyya distance B_α theo công thức paper cho một lưới giá trị $\alpha \in (0,1)$, tìm α^* tối thiểu hóa bằng grid search hoặc tối ưu số, minh họa khái niệm ”best error exponent” của bài toán kiểm định giả thuyết.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Metric tensor g	$g_p(u,v) = \langle u,v\rangle_p$, đo độ dài/góc vector tiếp xúc
Affine connection ∇	Xác định đạo hàm hiệp biến $\nabla_X Y$, geodesic, parallel transport
Connection liên hợp ∇^*	$X\langle Y,Z\rangle = \langle \nabla_X Y, Z\rangle + \langle Y, \nabla^*_X Z\rangle$
Tensor Amari-Chentsov C	$C_{ijk} = \Gamma^k_{ij} - \Gamma^{*k}_{ij}$, hoàn toàn đối xứng
α-connection	$\Gamma^\alpha = \Gamma^0 - \frac{\alpha}{2}C$; $\alpha=0$: Levi-Civita; $\alpha=\pm1$: cặp (∇,∇^*)
Định lý nền tảng IG	∇ độ cong hằng $\kappa \Leftrightarrow \nabla^*$ độ cong hằng κ; $\kappa=0$: dually flat
Bregman divergence	$B_F(\theta:\theta') = F(\theta) - F(\theta') - (\theta-\theta')^\top \nabla F(\theta')$, F lồi chặt
Tọa độ đối ngẫu	$\eta = \nabla F(\theta)$, $\theta = \nabla F^*(\eta)$ (Legendre-Fenchel)
Pythagore đối ngẫu	$\gamma^*(P,Q) \perp \gamma(Q,R) \Rightarrow D(P:R) = D(P:Q) + D(Q:R)$
Fisher information matrix	$I(\theta) = \mathbb{E}_\theta[\partial_i l\, \partial_j l]$; duy nhất bất biến (Theorem 8)
Natural gradient	${}^{NG}\nabla L_\theta(\theta) = g_\theta^{-1}(\theta)\nabla_\theta L_\theta(\theta)$; bằng $\nabla_\eta L_\eta(\eta)$ trên đa tạp dually flat
FIM Gaussian 1D, tham số (μ,σ)	$\mathrm{diag}(1/\sigma^2,\ 2/\sigma^2)$
Ví dụ tự tính: FIM Bernoulli	$I(\theta) = 1/(\theta(1-\theta))$; $I(0.5)=4$, $I(0.2)=6.25$, $I(0.05)\approx 21.05$

11. Kết luận ngắn

Bài khảo sát lại toàn bộ khung lý thuyết của information geometry: coi một họ phân phối xác suất như một đa tạp, đặt lên nó một metric (chính tắc là Fisher information, metric bất biến duy nhất theo Theorem 8) và một cặp connection đối ngẫu (chính tắc là connection mũ/hỗn hợp). Kết quả trung tâm là định lý nền tảng (độ cong của connection và đối ngẫu của nó luôn bằng nhau) và trường hợp đặc biệt dually flat, nơi tồn tại hai hệ tọa độ đối ngẫu liên hệ qua Legendre-Fenchel và một định lý Pythagore tổng quát. Một liên hệ quan trọng ở mục 4.1 là: natural gradient trên θ chính là gradient thường trên tọa độ đối ngẫu η của một đa tạp dually flat, một cách diễn đạt hình học rất gọn cho cùng hiện tượng bất biến tham số hóa đã gặp ở math 06. Giới hạn lớn nhất của bài, như bài báo lưu ý, là tính chất ”introduction” của nó: hầu hết chứng minh bị bỏ qua, và nhiều

kết quả đẹp (dually flat, Pythagore đối ngẫu) chỉ áp dụng trơn tru cho các họ phân phối có cấu trúc đặc biệt (hàm mũ, hỗn hợp), không phải cho không gian tham số tổng quát của một mạng neural lớn.

Nguồn gốc: https://arxiv.org/abs/1808.08271

Bài 08

Variational Inference: A Review for Statisticians (2016)

Tác giả: David M. Blei (Đại học Columbia, Khoa Khoa học Máy tính và Khoa Thống kê), Alp Kucukelbir (Đại học Columbia, Khoa Khoa học Máy tính), Jon D. McAuliffe (Đại học California, Berkeley, Khoa Thống kê)
Venue / arXiv: arXiv:1601.00670 (bản v9, ngày 11/05/2018); văn bản arXiv không tự nêu tên hội nghị hay tạp chí công bố nào; theo kiến thức chung (ngoài paper, không xác nhận được trong chính văn bản arXiv) bài này được xuất bản trên Journal of the American Statistical Association (JASA) năm 2017
Nguồn: https://arxiv.org/abs/1601.00670
Chủ đề: Variational inference (suy luận biến phân/suy luận xấp xỉ qua tối ưu hóa), ELBO, mean-field, CAVI, suy luận biến phân ngẫu nhiên (stochastic variational inference, SVI) dùng natural gradient
Vị trí trong lộ trình: Bài số 8 trong 9 bài toán nền (math 01-09). Đọc sau math 06 (natural gradient method) và math 07 (information geometry) vì mục 4.3 của bài này (SVI) dùng trực tiếp khái niệm natural gradient đã học ở hai bài đó. Đứng trước math 09 (variational autoencoder), nơi khung ELBO/variational family được áp dụng cho mô hình sinh sâu (deep generative model) với kỹ thuật reparameterization mà chính bài này cũng nhắc tới ở mục 5.3 như một hướng mở rộng ngoài phạm vi chính của bài.

> Ghi chú diễn giải bài tổng quan suy luận biến phân, gồm nội dung chính và ví dụ từ các phụ lục về hồi quy Bayes, GMM và LDA.

1. Bối cảnh

Bài tổng quan này dành cho các nhà thống kê đã quen với suy luận Bayes bằng Markov chain Monte Carlo (MCMC) và muốn tìm hiểu suy luận biến phân (variational inference, VI). Bài trình bày có hệ thống các phương pháp VI, kèm ví dụ mô hình hỗn hợp Gaussian (Gaussian mixture model, GMM).

Bối cảnh lịch sử theo paper (mục 1): ý tưởng ”mean field” (trường trung bình) có gốc từ vật lý thống kê, được đưa vào học máy/thống kê qua Peterson và Anderson (1987, ”A mean field theory learning algorithm for neural networks”) và Hinton và Van Camp (1993, ”Keeping the neural networks simple by minimizing the description length of the weights”). VI sau đó được hệ thống hóa cho mô hình đồ thị (graphical model) bởi Jordan, Ghahramani, Jaakkola, Saul (1999, ”Introduction to variational methods for graphical models”). Đối trọng truyền thống của VI là các phương pháp MCMC (Metropolis và cộng sự 1953; Hastings 1970; Gibbs sampling của Geman và Geman 1984): MCMC xây một xích Markov có phân phối dừng chính là posterior cần tính, còn VI biến bài toán suy luận thành một bài toán tối ưu hóa.

Paper không tự nêu tên hội nghị/tạp chí công bố trong chính văn bản arXiv (chỉ có từ khóa ”Algorithms; Statistical Computing; Computationally Intensive Methods” ở đầu bài). Việc bài này sau đó xuất bản ở JASA 2017 là thông tin ngoài paper, nêu ở đây chỉ để định vị, không dùng làm cơ sở cho nội dung kỹ thuật.

Ghi chú này đọc toàn bộ nội dung chính (mục 1 đến 5) và cả ba phụ lục (A, B, C), vì ba phụ lục này chứa chi tiết suy luận toán học cho hai thí nghiệm thực sự quan trọng của bài (so sánh với Stan ở Phụ lục B, mô hình LDA ở Phụ lục C).

2. Ý tưởng chính

Bài toán Bayes tổng quát: có dữ liệu quan sát $x = x_{1:n}$ và biến ẩn/tham số $z = z_{1:m}$, mô hình xác định mật độ chung $p(z, x)$. Ta muốn tính posterior

$$p(z \mid x) = \frac{p(z, x)}{p(x)}, \qquad p(x) = \int p(z, x)\, dz.$$

Vấn đề: với nhiều mô hình, $p(x)$ (evidence, "bằng chứng") không tính được ở dạng đóng. Ví dụ paper dùng xuyên suốt bài là mô hình hỗn hợp Gaussian Bayes (Bayesian GMM, mục 2.1): với K cụm,

$$\mu_k \sim \mathcal{N}(0, \sigma^2), \quad k = 1, \ldots, K; \qquad c_i \sim \text{Categorical}\left(\tfrac{1}{K}, \ldots, \tfrac{1}{K}\right); \qquad x_i \mid c_i, \mu \sim \mathcal{N}(c_i^\top \mu, 1).$$

Ở đây c_i là vector chỉ số (indicator) cho biết điểm i thuộc cụm nào. Để tính $p(x)$ phải lấy tổng trên mọi cách gán cụm c có thể, tức K^n số hạng (với n điểm dữ liệu): không thể tính trực tiếp khi n lớn.

Ý tưởng của VI: thay vì tính posterior đúng, đặt ra một họ phân phối xấp xỉ "dễ" $\mathcal{Q} = \{q(z; \nu)\}$ (gọi là variational family), rồi tìm thành viên q^* trong họ đó gần posterior thật nhất, theo nghĩa cực tiểu hóa Kullback-Leibler divergence

$$q^* = \arg\min_{q \in \mathcal{Q}} \mathrm{KL}\big(q(z) \,\|\, p(z \mid x)\big).$$

Vấn đề con: biểu thức $\mathrm{KL}(q \| p(z \mid x))$ tự nó chứa $\log p(x)$ (xem mục 3), nên không tính trực tiếp được. Paper giải quyết bằng cách tối ưu một đại lượng khác, tương đương về mặt tối ưu nhưng tính được: ELBO (evidence lower bound). Đây chính là chuyển bài toán suy luận thành bài toán tối ưu hóa, điểm khác biệt căn bản so với MCMC (một họ thuật toán lấy mẫu).

Để ELBO tính được, paper giả định họ biến phân có dạng mean-field: z chia thành m nhóm/khối $z_1, \ldots, z_m$, và

$$q(z) = \prod_{j=1}^{m} q_j(z_j),$$

tức giả định các nhóm biến ẩn độc lập với nhau dưới q (nhưng không giả định dạng hàm cụ thể nào cho mỗi q_j; dạng của q_j sẽ "lộ" ra từ chính bài toán tối ưu).

3. Phương pháp chi tiết

3.1 ELBO và liên hệ với KL

Paper định nghĩa (mục 2.2):

$$\mathrm{ELBO}(q) = \mathbb{E}_q[\log p(z, x)] - \mathbb{E}_q[\log q(z)].$$

Khai triển $\mathrm{KL}(q(z) \| p(z \mid x))$ và dùng $p(z \mid x) = p(z, x)/p(x)$:

$$\mathrm{KL}(q(z) \| p(z \mid x)) = \mathbb{E}_q[\log q(z)] - \mathbb{E}_q[\log p(z, x)] + \log p(x) = -\mathrm{ELBO}(q) + \log p(x).$$

Sắp xếp lại:

$$\log p(x) = \mathrm{ELBO}(q) + \mathrm{KL}\big(q(z) \,\|\, p(z \mid x)\big).$$

Vì KL ≥ 0 luôn luôn, nên ELBO$(q) \leq \log p(x)$: ELBO là chặn dưới của log-evidence (đúng như tên gọi). Vì $\log p(x)$ không phụ thuộc q, cực đại hóa ELBO(q) theo q đúng bằng cực tiểu hóa KL$(q\|p(z \mid x))$ theo q, mà không cần biết $\log p(x)$. Paper cũng chỉ ra liên hệ với thuật toán EM (Expectation-Maximization): ELBO chính là đại lượng được cực đại hóa ở bước E của EM; VI tổng quát hóa ý tưởng này khi chính posterior có điều kiện đầy đủ (complete conditional) cũng không tính được dạng đóng.

3.2 Mean-field và minh họa Hình 1 của paper

Mục 2.3 của paper minh họa (Hình 1, chỉ là hình vẽ định tính, không có số liệu) rằng khi posterior thật là một Gaussian hai chiều có tương quan mạnh, xấp xỉ mean-field (tích hai Gaussian một chiều độc lập) khớp đúng vị trí trung tâm (mean) nhưng cho ra một "khối" xấp xỉ nhỏ hơn nhiều so với hình elip thật, tức đánh giá thấp phương sai biên (marginal variance).

Ví dụ (tự tính): paper không cho con số cụ thể nào ở Hình 1, nên ở đây tính một ví dụ số cụ thể để định lượng hiện tượng đó. Đây là tính toán độc lập của người ghi chú dựa trên một kết quả toán học chuẩn về xấp xỉ mean-field Gaussian theo KL ngược (không phải điều paper tự chứng minh, nhưng nhất quán với Hình 1 và với kết quả lý thuyết ở mục 5.2 dưới đây).

Giả sử posterior thật là $p(z_1, z_2) = \mathcal{N}(0, \Sigma)$ với

$$\Sigma = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}, \quad \rho = 0.8.$$

Xấp xỉ mean-field $q(z_1, z_2) = q_1(z_1)q_2(z_2)$, tối thiểu KL$(q\|p)$ (đúng chiều KL mà VI luôn dùng). Với p là Gaussian có ma trận hiệp phương sai nghịch đảo (precision) $\Lambda = \Sigma^{-1}$, kết quả chuẩn là: q_j^* có trung bình đúng bằng trung bình thật, còn phương sai bằng $1/\Lambda_{jj}$ (nghịch đảo của phần tử chéo của ma trận precision), không phải Σ_{jj} (phần tử chéo của hiệp phương sai thật).

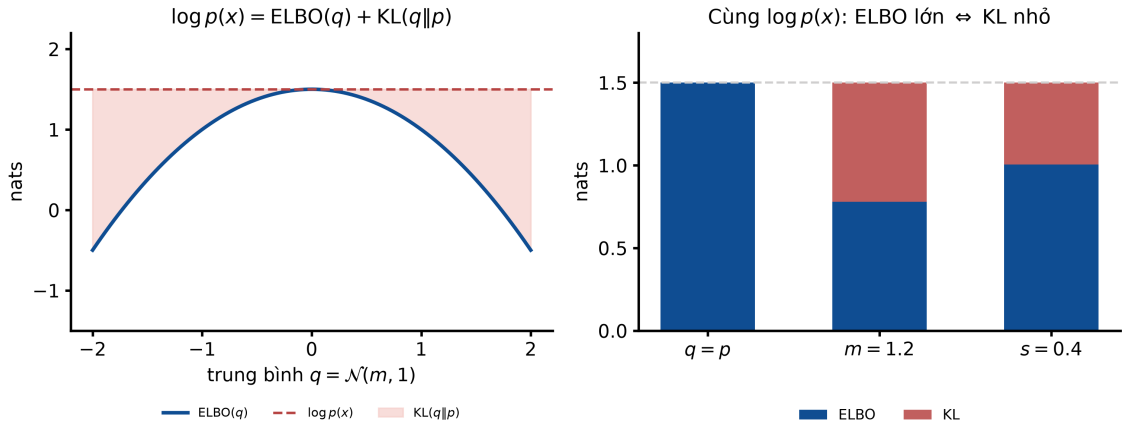
Tính Λ:

$$\Sigma^{-1} = \frac{1}{1-\rho^2}\begin{pmatrix} 1 & -\rho \\ -\rho & 1 \end{pmatrix} = \frac{1}{1-0.64}\begin{pmatrix} 1 & -0.8 \\ -0.8 & 1 \end{pmatrix} = \frac{1}{0.36}\begin{pmatrix} 1 & -0.8 \\ -0.8 & 1 \end{pmatrix}.$$

Vậy $\Lambda_{11} = \Lambda_{22} = 1/0.36 \approx 2.778$. Phương sai của q_j^* là $1/\Lambda_{jj} = 0.36$, trong khi phương sai biên thật $\Sigma_{jj} = 1$. Tỉ lệ $0.36/1 = 0.36$: xấp xỉ mean-field chỉ giữ lại 36% phương sai thật, đánh giá thấp phương sai tới 64% trong ví dụ này. Khi $\rho \to 1$ (tương quan gần tuyệt đối), $1 - \rho^2 \to 0$ nên phương sai mean-field $\to 0$: VI mean-field trở nên "tự tin" quá mức một cách không giới hạn khi độ tương quan tăng. Đây là minh họa số cụ thể cho câu paper nói ở Hình 1 và cho kết quả lý thuyết ở mục 5.2.

3.3 CAVI (Coordinate Ascent Variational Inference)

Với giả định mean-field, paper dẫn ra (mục 2.4, công thức tổng quát) quy tắc cập nhật tối ưu cho từng nhân tử, giữ các nhân tử khác cố định:



log p(x)=ELBO(q)+KL(q‖p): khi q lệch khỏi hậu nghiệm, ELBO giảm đúng bằng phần KL. (minh hoạ, tự tính)

$$q_j^*(z_j) \;\propto\; \exp\Big\{\mathbb{E}_{-j}\big[\log p(z_j \mid z_{-j}, x)\big]\Big\} \;\propto\; \exp\Big\{\mathbb{E}_{-j}\big[\log p(z, x)\big]\Big\},$$

trong đó $\mathbb{E}_{-j}$ là kỳ vọng lấy theo tích các nhân tử q_i với $i \neq j$ (giữ cố định ở vòng cập nhật hiện tại). Thuật toán CAVI (Algorithm 1 của paper): khởi tạo các q_j, sau đó lặp qua $j = 1, \ldots, m$, mỗi lần cập nhật q_j theo công thức trên rồi tính lại ELBO, lặp tới khi ELBO tăng rất ít (hội tụ). Vì mỗi bước chỉ tăng hoặc giữ nguyên ELBO (không giảm), CAVI là một phương pháp coordinate ascent đúng nghĩa, bảo đảm hội tụ tới một điểm tối ưu cục bộ (không bảo đảm toàn cục, vì ELBO như hàm của toàn bộ $(q_1, \ldots, q_m)$ nói chung không lồi).

Paper nêu rõ hai vấn đề thực hành (mục 2.5): (1) nhạy với điểm khởi tạo và tối ưu cục bộ (minh họa thực nghiệm ở mục 4 dưới); (2) cần kỹ thuật log-sum-exp để tránh tràn số (underflow) khi chuẩn hóa các nhân tử dạng categorical (ví dụ xác suất gán cụm trong GMM).

3.4 Ví dụ đầy đủ: GMM

Mục 3 của paper suy ra chi tiết CAVI cho mô hình GMM ở mục 2.1. Nhân tử biến phân cho biến gán cụm c_i là categorical với "trách nhiệm" (responsibility) φ_{ik} tính theo softmax của $\mathbb{E}[\mu_k]\, x_i - \mathbb{E}[\mu_k^2]/2$; nhân tử cho mỗi μ_k là Gaussian với trung bình $m_k = \dfrac{\sum_i \varphi_{ik} x_i}{1/\sigma^2 + \sum_i \varphi_{ik}}$ và phương sai $s_k^2 = \dfrac{1}{1/\sigma^2 + \sum_i \varphi_{ik}}$. Công thức ELBO đầy đủ cho mô hình này là Eq. (21) của paper; Algorithm 2 là CAVI cụ thể hóa cho GMM (lặp cập nhật φ rồi (m, s^2)); Eq. (35) cho mật độ dự đoán của một điểm dữ liệu mới bằng cách trộn K thành phần đã khớp.

3.5 Mở rộng: họ hàm mũ và SVI

Mục 4.1: nếu mỗi complete conditional $p(z_j \mid z_{-j}, x)$ thuộc một họ hàm mũ (exponential family) với tham số tự nhiên $\eta_j(z_{-j}, x)$, thì nhân tử tối ưu q_j^* cũng thuộc đúng họ hàm mũ đó, với tham số tự nhiên bằng kỳ vọng của $\eta_j(z_{-j}, x)$ dưới các q_i khác. Mục 4.2 hình thức hóa lớp mô hình "conditionally conjugate" phù hợp cho dữ liệu lớn: có biến toàn cục β (chung cho toàn bộ dữ liệu, ví dụ tham số mô hình) và biến địa phương z_i (riêng từng điểm dữ liệu, ví dụ biến gán cụm), mỗi z_i chỉ phụ thuộc β và (x_i, z_i).

Mục 4.3 trình bày Stochastic Variational Inference (SVI), đóng góp phương pháp chính của bài để xử lý dữ liệu quy mô lớn. Với cấu trúc conditionally conjugate trên, natural gradient của ELBO theo tham số biến phân toàn cục λ (tham số của $q(\beta)$) có dạng rất gọn (Eq. 51-52 của paper):

$$\nabla_\lambda^{\text{nat}} \,\text{ELBO} = F(\lambda)^{-1} \nabla_\lambda \,\text{ELBO},$$

và với $q(\beta)$ thuộc họ hàm mũ, công thức này rút gọn thành một biểu thức dễ tính (không cần nghịch đảo ma trận Fisher một cách rõ ràng), vì Fisher information của họ hàm mũ theo tham số tự nhiên chính là Hessian của hàm log-partition, triệt tiêu gọn với các số hạng chain-rule trong gradient thường. SVI tận dụng điều này: ước lượng natural gradient bằng cách lấy ngẫu nhiên một (hoặc một minibatch) điểm dữ liệu, nhân với hệ số tỉ lệ $N/|\text{minibatch}|$ để không chệch (unbiased), rồi cập nhật

$$\lambda_{t+1} = \lambda_t + \rho_t \, \widehat{\nabla_\lambda^{\text{nat}} \text{ELBO}}_t,$$

với dãy bước học ρ_t thỏa điều kiện Robbins-Monro (Robbins và Monro, 1951): $\sum_t \rho_t = \infty$ và $\sum_t \rho_t^2 < \infty$ (ví dụ $\rho_t = (t + t_0)^{-\kappa}$ với $\kappa \in (0.5, 1]$), đảm bảo hội tụ hầu chắc chắn tới một điểm tối ưu cục bộ. Đây là Algorithm 3 của paper, và chính là áp dụng cụ thể của natural gradient (math 06) kết hợp với tối ưu ngẫu nhiên (stochastic optimization).

4. Thí nghiệm và kết quả chính

Paper có ba khảo sát thực nghiệm thật sự (không chỉ là ví dụ minh họa trên giấy), ở mục 3.4 và 4.3:

(a) Mô phỏng (mục 3.4): dữ liệu tổng hợp từ hỗn hợp 5 Gaussian hai chiều ($K = 5$). Hình 3 của paper cho thấy đường cong ELBO theo số vòng lặp CAVI có hình dạng ”khuỷu tay” (elbow): có những đoạn gần như đi ngang (ELBO tăng rất chậm) rồi đột ngột nhảy lên, tương ứng với lúc thuật toán ”thoát” khỏi một cấu hình gán cụm chưa tốt sang cấu hình tốt hơn về chất.

(b) So sánh với Stan/HMC trên dữ liệu ảnh thật (mục 3.4, Phụ lục B): dùng tập dữ liệu imageCLEF, mỗi ảnh được biểu diễn bằng histogram màu RGB 576 chiều, với 10.000 ảnh huấn luyện và 10.000 ảnh kiểm tra. Khớp một GMM đường chéo (diagonal covariance) với $K = 30$ cụm bằng CAVI, so với lấy mẫu bằng Stan (dùng Hamiltonian Monte Carlo/NUTS). Siêu tham số cụ thể ghi ở Phụ lục B: $a_0 = 1/K$, $m_0 = 0$, $b_0 = 1$, $\alpha_0 = 1$, $\beta_0 = 1$; Phụ lục B cũng cho toàn bộ mã Stan dùng trong so sánh (Hình 8 của paper). Kết luận của paper: CAVI nhanh hơn Gibbs sampling và HMC/NUTS ”nhiều cấp độ lớn” (orders of magnitude), khớp được mô hình cho 10.000 ảnh trong chưa đầy một phút. Paper tự có một chú thích (footnote) nhấn mạnh đây chỉ là một nghiên cứu trường hợp cụ thể, không phải một khẳng định tổng quát rằng VI luôn nhanh hơn mọi biến thể MCMC.

(c) LDA trên 1,8 triệu bài báo New York Times (mục 4.3, Phụ lục C): áp dụng SVI cho mô hình Latent Dirichlet Allocation (LDA, Blei và cộng sự 2003), dựa trên suy luận CAVI cụ thể cho LDA ở Phụ lục C (cập nhật Eq. 59-61, Algorithm 4, theo Hoffman và cộng sự 2013): nhân tử $q(\beta_k; \lambda_k)$ cho mỗi chủ đề (Dirichlet V chiều), $q(\theta_d; \gamma_d)$ cho tỉ lệ chủ đề mỗi văn bản (Dirichlet K chiều), $q(z_{dn}; \phi_{dn})$ cho gán chủ đề mỗi từ (multinomial). Hình 7 của paper hiển thị 15 chủ đề trích ra được (dưới dạng các từ có xác suất cao trong mỗi chủ đề); vì đây là một hình ảnh minh họa trong bản PDF chứ không phải bảng số liệu dạng văn bản, ghi chú này không trích lại danh sách từ cụ thể của 15 chủ đề (không có trong văn bản text trích xuất được một cách đáng tin cậy). Bài báo nhấn mạnh (diễn giải lại, không trích nguyên văn chính xác từng chữ do giới hạn khi chuyển PDF sang text): nếu không có SVI, một phân tích ở quy mô 1,8 triệu văn bản như vậy sẽ không thể thực hiện được bằng CAVI theo lô (batch) thông thường.

Ba thí nghiệm này là bằng chứng thực nghiệm thật, không phải chỉ lý thuyết, cho hai tuyên bố chính của paper: CAVI nhanh hơn MCMC trên bài toán cỡ vừa (thí nghiệm b), và SVI mở ra khả năng xử lý dữ liệu cỡ rất lớn mà CAVI theo lô không làm được (thí nghiệm c).

5. Phân tích / ablation

Paper không chạy một bảng ablation có kiểm soát theo nghĩa thường gặp trong các bài học máy hiện đại (không có bảng so sánh hệ thống nhiều lựa chọn siêu tham số với số liệu đầy đủ). Thay vào đó, phần ”phân tích” gần nhất gồm:

- **Độ nhạy với điểm khởi tạo (mục 2.5, Hình 2):** chạy CAVI cho GMM với 10 điểm khởi tạo ngẫu nhiên khác nhau trên cùng dữ liệu, cho ra các kết quả hội tụ khác nhau (tối ưu cục bộ khác nhau). Đây là minh họa thực nghiệm thật, không phải chỉ cảnh báo lý thuyết.
- **Khảo sát lý thuyết về sai số của VI (mục 5.2):** paper tổng hợp (không tự chứng minh lại) một kết quả quan trọng của Wang và Titterington (2005): với một lớp mô hình nhất định, ma trận hiệp phương sai posterior theo mean-field VI khác với hiệp phương sai "đúng" (theo ước lượng hợp lý cực đại/nghịch đảo Fisher information) bởi một ma trận nửa xác định dương bị trừ đi, nghĩa là VI **luôn đánh giá thấp** phương sai posterior theo hướng đó. Đây là cơ sở lý thuyết (không phải thực nghiệm của chính paper này) cho hiện tượng đã minh họa định lượng ở mục 3.2 của ghi chú này (ví dụ tự tính $\rho = 0.8$).
- Paper cũng tổng hợp (không tự chứng minh) các kết quả về tính nhất quán (consistency) của ước lượng VI khi $n \to \infty$ từ nhiều tác giả khác, nhưng tự nhận lý thuyết về độ chính xác thống kê đầy đủ của VI (ví dụ phân phối tiệm cận, khoảng tin cậy) còn là vấn đề mở (xem mục 6).

6. Giới hạn

Giới hạn bài báo nêu rõ:

- **Đánh giá thấp phương sai một cách có hệ thống:** do giả định mean-field (độc lập giữa các nhóm biến ẩn) và do chọn chiều KL $\mathrm{KL}(q\|p)$ (không phải $\mathrm{KL}(p\|q)$), xấp xỉ mean-field về cấu trúc không thể tái tạo tương quan giữa các nhóm biến khác nhau, và có xu hướng cho posterior "tự tin quá mức" (xem mục 3.2 và 5.2).
- **Chỉ tìm tối ưu cục bộ:** CAVI/SVI đều là các thuật toán tối ưu cục bộ trên một hàm mục tiêu (ELBO) không lồi nói chung; nhạy với điểm khởi tạo (Hình 2).
- **Yêu cầu cấu trúc conditional conjugacy để có công thức đóng:** phần suy luận chi tiết (CAVI, SVI với công thức tường minh) của paper chỉ áp dụng trực tiếp khi mỗi complete conditional thuộc họ hàm mũ nhận biết được. Với mô hình không liên hợp (non-conjugate), ví dụ hồi quy logistic Bayes mà paper nêu ở mục 5.3, cần kỹ thuật khác (Monte Carlo gradient, "black box" VI của Ranganath, Gerrish, Blei 2014) mà paper chỉ khảo sát sơ lược, không suy diễn chi tiết như với CAVI/SVI.
- **Mục 5.4 (Open problems) paper tự liệt kê là vấn đề mở, không phải đã giải quyết:** lựa chọn phân kỳ (KL hay một họ phân kỳ khác, ví dụ α-divergence) ảnh hưởng tới hành vi của nghiệm (mode-seeking so với mode-covering) nhưng chưa có hướng dẫn tổng quát; expectation propagation được paper mô tả như "dùng KL divergence theo chiều ngược lại"; giao diện giữa VI và MCMC (kết hợp hai họ phương pháp) còn là hướng nghiên cứu đang mở; lý thuyết thống kê đầy đủ cho ước lượng VI (tính nhất quán, phân phối tiệm cận, định lượng độ không chắc chắn khi suy luận sai) còn chưa hoàn chỉnh.

Quan sát riêng của người ghi chú (không phải khẳng định của paper): bài không đưa ra một bảng số liệu thời gian chạy (giây/phút) cụ thể, có thể tái lập từng con số, cho so sánh imageCLEF ở mục 4(b) trên đây; các câu như "nhanh hơn nhiều cấp độ lớn" hay "dưới một phút" là phát biểu định tính trong văn bản chính, không phải một bảng benchmark với số liệu từng cấu hình. Người muốn tái lập chính xác cần tự chạy lại dựa trên mã Stan ở Phụ lục B.

7. Liên hệ với các bài khác

- **math 06 (New Insights and Perspectives on the Natural Gradient Method):** SVI ở mục 4.3 của bài này là một ứng dụng trực tiếp của natural gradient đã học ở math 06; công thức $\nabla_\lambda^{\text{nat}}\text{ELBO} = F(\lambda)^{-1}\nabla_\lambda\text{ELBO}$ chính là đối tượng trung tâm của math 06, áp dụng cho một bài toán cụ thể (tối ưu ELBO).
- **math 07 (An Elementary Introduction to Information Geometry):** lý do natural gradient của SVI rút gọn gọn gàng khi $q(\beta)$ thuộc họ hàm mũ chính là cấu trúc tọa độ đối ngẫu (tham số tự nhiên và tham số kỳ vọng) đã trình bày ở math 07 (Property 7 ở đó: natural gradient theo tọa độ tự nhiên bằng

gradient thường theo tọa độ đối ngẫu). Công thức Eq. 51 của bài này là một trường hợp cụ thể của sự kiện tổng quát đó.

- **math 09 (An Introduction to Variational Autoencoders, bài tiếp theo):** chính paper này, ở mục 5.3 (”Beyond conditional conjugacy”), trích dẫn trực tiếp Kingma và Welling (2014, ”Auto-encoding variational Bayes”) và Rezende, Mohamed, Wierstra (2014, ”Stochastic backpropagation and approximate inference in deep generative models”) như ví dụ về suy luận biến phân cho mô hình không liên hợp, dùng gradient kiểu Monte Carlo (tiền đề của reparameterization trick ở math 09). Đây là liên hệ được chính văn bản paper xác nhận, không phải suy diễn của người ghi chú.
- **math 10 (A Conceptual Introduction to Hamiltonian Monte Carlo):** mục 1 và mục 3.4(b) của bài này so sánh trực tiếp VI/CAVI với HMC (được triển khai qua Stan) như đối trọng chính cần cân nhắc khi chọn phương pháp suy luận Bayes xấp xỉ; math 10 trình bày chi tiết phía HMC của sự so sánh đó.

8. Gợi ý áp dụng

Suy luận biến phân biến bài toán xấp xỉ hậu nghiệm thành tối ưu ELBO. Chất lượng kết quả phụ thuộc họ phân phối xấp xỉ và lựa chọn divergence; một nghiệm ELBO tốt trong họ đã chọn không bảo đảm tái hiện mọi đặc điểm của hậu nghiệm thật.

9. Thí nghiệm gợi ý

Các thí nghiệm dưới đây dùng các mô hình nhỏ:

1. **CAVI so với EM trên GMM tổng hợp nhỏ.** Sinh dữ liệu tổng hợp 2 chiều từ 3-5 Gaussian (vài nghìn điểm), cài CAVI theo Algorithm 2 của paper (numpy/PyTorch, CPU hoặc GPU đều được) và so với `sklearn.mixture.GaussianMixture` (EM). Đo: log-likelihood cuối cùng, thời gian chạy, số vòng lặp tới hội tụ. Kỳ vọng: chất lượng khớp tương đương, cả hai nhanh hơn hẳn một phương pháp MCMC đầy đủ (ví dụ chạy thêm Gibbs sampler thủ công để so sánh thời gian). Diễn giải: xác nhận trực tiếp hiện tượng paper mô tả ở mục 4(b).
2. **Định lượng đánh giá thấp phương sai của mean-field trên một posterior Gaussian tương quan.** Lặp lại ví dụ tự tính ở mục 3.2 (toy $\rho = 0.8$) bằng mô phỏng số: chạy coordinate-ascent thật (không dùng công thức đóng) cho q_1, q_2 Gaussian xấp xỉ $p = \mathcal{N}(0, \Sigma)$, kiểm tra phương sai hội tụ về $1/\Lambda_{jj}$ như tính tay. Thử nhiều giá trị $\rho \in \{0, 0.3, 0.6, 0.8, 0.95\}$ và vẽ đồ thị tỉ lệ phương sai mean-field/phương sai thật theo ρ. Kỳ vọng: tỉ lệ giảm về 0 khi $\rho \to 1$. Diễn giải: minh họa trực quan, có số liệu, cho giới hạn ở mục 6.
3. **SVI so với CAVI theo lô trên LDA cỡ nhỏ.** Dùng một corpus nhỏ (vài nghìn văn bản, ví dụ một phần của 20 Newsgroups), cài CAVI theo lô cho LDA (Phụ lục C, Algorithm 4) và SVI (Algorithm 3) với vài kích thước minibatch và vài giá trị $\kappa \in \{0.5, 0.7, 1.0\}$ cho bước học $\rho_t = (t + t_0)^{-\kappa}$. Đo: perplexity trên tập giữ lại (held-out) theo thời gian chạy thực (wall-clock), không theo số vòng lặp. Kỳ vọng: SVI đạt perplexity tương đương CAVI nhanh hơn khi corpus đủ lớn để amortize chi phí overhead, nhưng có thể chậm hơn CAVI trên corpus rất nhỏ (do nhiễu từ việc lấy mẫu minibatch). Diễn giải: tái hiện ở quy mô nhỏ bài học từ thí nghiệm LDA 1,8 triệu bài báo của paper.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Posterior mục tiêu	$p(z \mid x) = p(z, x)/p(x)$, với $p(x) = \int p(z, x)\, dz$ thường không tính được
ELBO	$\text{ELBO}(q) = \mathbb{E}_q[\log p(z, x)] - \mathbb{E}_q[\log q(z)]$
Liên hệ ELBO và KL	$\log p(x) = \text{ELBO}(q) + \text{KL}(q(z) \Vert p(z \mid x))$, nên cực đại ELBO ≡ cực tiểu KL
Họ mean-field	$q(z) = \prod_{j=1}^{m} q_j(z_j)$ (các nhóm biến ẩn độc lập dưới q)

Khái niệm	Công thức / ý nghĩa
Cập nhật CAVI	$q_j^*(z_j) \propto \exp\{\mathbb{E}_{-j}[\log p(z, x)]\}$, lặp qua j tới hội tụ
Nhân tử họ hàm mũ	Nếu complete conditional thuộc họ hàm mũ, q_j^* cũng thuộc họ đó, tham số = kỳ vọng tham số tự nhiên của conditional
Natural gradient của SVI	$\nabla_\lambda^{\text{nat}}\text{ELBO} = F(\lambda)^{-1}\nabla_\lambda\text{ELBO}$, rút gọn khi $q(\beta)$ thuộc họ hàm mũ
Cập nhật SVI	$\lambda_{t+1} = \lambda_t + \rho_t \widehat{\nabla_\lambda^{\text{nat}}\text{ELBO}}_t$, ước lượng không chệch từ một (mini)batch dữ liệu
Điều kiện Robbins-Monro	$\sum_t \rho_t = \infty$, $\sum_t \rho_t^2 < \infty$ (ví dụ $\rho_t = (t + t_0)^{-\kappa}$, $\kappa \in (0.5, 1]$)
Hạn chế đã biết	Mean-field đánh giá thấp phương sai posterior (Wang và Titterington, 2005); chỉ tìm tối ưu cục bộ

11. Kết luận ngắn

Bài tổng quan trình bày ELBO, CAVI, và SVI có ví dụ GMM suy luận đầy đủ từng bước, và quan trọng là có hai thực nghiệm thật của chính nhóm tác giả (so sánh CAVI với Stan/HMC trên dữ liệu ảnh imageCLEF, và SVI trên 1,8 triệu bài báo New York Times bằng LDA) để minh chứng cho lợi ích tốc độ của VI, không chỉ dừng ở lý thuyết. Vì đây là bài review, phần lớn ”kết quả chính” là trình bày lại/hệ thống hóa các kết quả đã biết (CAVI, SVI của Hoffman và cộng sự 2013) hơn là đề xuất phương pháp hoàn toàn mới, ngoại trừ chính hai thí nghiệm thực nghiệm so sánh nêu trên do nhóm tác giả tự thực hiện. Paper cũng rất thẳng thắn về giới hạn của chính họ phương pháp (mean-field đánh giá thấp phương sai, chỉ tìm tối ưu cục bộ, cần cấu trúc conjugate để có công thức đóng), và liệt kê rõ các vấn đề mở (mục 5.4) thay vì tuyên bố VI đã giải quyết xong mọi bài toán suy luận Bayes. Ghi chú này đã đọc toàn bộ mục 1-5 và cả ba phụ lục A, B, C; phần danh mục tham khảo chỉ đọc lướt để xác nhận đó là trích dẫn thuần túy.

Nguồn gốc: https://arxiv.org/abs/1601.00670

Bài 09

An Introduction to Variational Autoencoders (2019)

Tác giả: Diederik P. Kingma (Google), Max Welling (Universiteit van Amsterdam; Qualcomm)
Venue / arXiv: arXiv:1906.02691 (bản v3, ngày 11/12/2019); bài báo nêu venue ngay trên trang đầu: ”Foundations and Trends® in Machine Learning”, dù số volume/issue ghi là ”xx” (placeholder, có thể do đây là phiên bản tiền in trước khi có số trang tạp chí chính thức)
Nguồn: https://arxiv.org/abs/1906.02691
Chủ đề: Variational autoencoder (VAE), ELBO, reparameterization trick, amortized inference, normalizing flow (inverse autoregressive flow), mô hình sinh biến ẩn sâu (deep latent variable model)
Vị trí trong lộ trình: Bài số 9/9 trong nhóm bài toán nền (math 01-09), bài cuối cùng của nhóm này. Đọc sau math 08 (Variational Inference: A Review for Statisticians) vì VAE dùng đúng khung ELBO/KL của math 08 nhưng thay biến phân cục bộ theo từng điểm dữ liệu bằng một mạng nơ-ron suy luận dùng chung (amortized inference). Cũng dùng lại chain rule/automatic differentiation của math 01-02 (để lan truyền gradient qua phép lấy mẫu) và có liên hệ trực tiếp với PCA ở math 03 (giới hạn tuyến tính không nhiễu của VAE chính là PCA).

> Ghi chú diễn giải các chương 1–5 và Phụ lục A của chuyên khảo về mô hình sinh biến ẩn sâu.

1. Bối cảnh

Chuyên khảo năm 2019 của Kingma và Welling mở rộng bài báo “Auto-Encoding Variational Bayes” (AEVB, 2014), công trình giới thiệu bộ tự mã hóa biến phân (variational autoencoder, VAE). Bài trình bày kỹ hơn nền tảng của VAE và bổ sung các phát triển sau năm 2014, như inverse autoregressive flow và mô hình biến ẩn nhiều tầng.

Động lực theo paper (mục 1.1): phân biệt mô hình sinh (generative, học phân phối chung trên mọi biến) với mô hình phân biệt (discriminative, chỉ học ánh xạ dự đoán). VAE là một khung giúp học đồng thời một mô hình sinh biến ẩn sâu (deep latent-variable model, DLVM) và một mô hình suy luận tương ứng, bằng stochastic gradient descent (SGD). Paper mô tả VAE gồm hai mô hình ghép cặp nhưng tham số hóa độc lập: encoder (mô hình nhận dạng/suy luận) và decoder (mô hình sinh). Theo paper, đóng góp lớn nhất của khung VAE có thể là nhận ra rằng có thể giảm nhiễu trong gradient bằng ”reparameterization trick” (mẹo đổi biến). VAE được paper mô tả là lấy cảm hứng từ Helmholtz Machine (Dayan và cộng sự, 1995) nhưng khác ở điểm quan trọng: Helmholtz Machine dùng thuật toán wake-sleep, tối ưu hai hàm mục tiêu riêng biệt không tương ứng với một bound chung nào của log-likelihood, còn VAE tối ưu một mục tiêu duy nhất.

Paper giả định người đọc có kiến thức cơ bản về đại số, giải tích và lý thuyết xác suất, và nói rõ đây không nhằm là một review toàn diện mọi công trình liên quan, mà là mở rộng có chọn lọc của bài 2014.

2. Ý tưởng chính

Mô hình biến ẩn sâu (DLVM): Có dữ liệu quan sát x và biến ẩn z (không quan sát được), mô hình sinh $p_\theta(x, z) = p_\theta(z)\, p_\theta(x \mid z)$, trong đó $p_\theta(z)$ là prior và $p_\theta(x \mid z)$ (decoder) được tham số hóa bằng mạng nơ-ron. Likelihood biên (marginal likelihood/evidence)

$$p_\theta(x) = \int p_\theta(x, z)\, dz$$

không tính được dạng đóng khi decoder là một mạng nơ-ron phi tuyến (mục 1.8 của paper). Vì $p_\theta(z \mid x) = p_\theta(x, z)/p_\theta(x)$, posterior cũng không tính được khi $p_\theta(x)$ không tính được, và ngược lại.

Ý tưởng VAE: đưa vào một mô hình suy luận (encoder) $q_\phi(z \mid x)$ xấp xỉ posterior thật, $q_\phi(z \mid x) \approx p_\theta(z \mid x)$, cũng tham số hóa bằng mạng nơ-ron, ví dụ

$$(\mu, \log \sigma) = \text{EncoderNeuralNet}_\phi(x), \qquad q_\phi(z \mid x) = \mathcal{N}(z; \mu, \text{diag}(\sigma^2)).$$

Khác biệt quan trọng với variational inference ”cổ điển” (chính là math 08): ở VI cổ điển, mỗi điểm dữ liệu có một bộ tham số biến phân riêng, tối ưu lặp đi lặp lại cho từng điểm (như CAVI ở math 08). VAE dùng MỘT mạng encoder chung cho toàn bộ tập dữ liệu: tham số ϕ (trọng số mạng) được chia sẻ giữa tất cả điểm dữ liệu, mỗi lần suy luận chỉ cần một lượt truyền xuôi (feedforward). Paper gọi đây là ”amortized inference” (suy luận khấu hao/chia sẻ chi phí): tránh phải tối ưu lặp cho từng điểm riêng lẻ, đổi lại phải chịu nhiễu từ việc lấy mẫu z trong quá trình huấn luyện.

Mục tiêu tối ưu: vẫn là ELBO như ở math 08, nhưng bây giờ tối ưu đồng thời cả θ (decoder) và ϕ (encoder) bằng SGD, nhờ một kỹ thuật cho phép lan truyền gradient qua bước lấy mẫu ngẫu nhiên $z \sim q_\phi(z \mid x)$: đó là reparameterization trick, viết $z = g(\varepsilon, \phi, x)$ với ε là nhiễu độc lập với ϕ, x, để ”ngoại hóa” phần ngẫu nhiên và lan truyền gradient qua phần còn lại (hàm g) bằng backpropagation thông thường.

3. Phương pháp chi tiết

3.1 ELBO (dẫn theo cách của paper, không qua bất đẳng thức Jensen)

Paper chọn một cách dẫn ELBO khác với cách dùng bất đẳng thức Jensen thường thấy, để ”cho thấy rõ độ chặt (tightness) của bound” (mục 2.2). Với $q_\phi(z \mid x)$ bất kỳ:

$$\log p_\theta(x) = \mathbb{E}_{q_\phi(z|x)}\Big[\log \frac{p_\theta(x,z)}{q_\phi(z \mid x)}\Big] + \mathbb{E}_{q_\phi(z|x)}\Big[\log \frac{q_\phi(z \mid x)}{p_\theta(z \mid x)}\Big] = \mathcal{L}_{\theta,\phi}(x) + D_{KL}(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)),$$

trong đó

$$\mathcal{L}_{\theta,\phi}(x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x,z) - \log q_\phi(z \mid x)]$$

là ELBO. Vì $D_{KL} \geq 0$, $\mathcal{L}_{\theta,\phi}(x) \leq \log p_\theta(x)$. Bài báo nhấn mạnh điểm ”two for one” (mục 2.2.1): cực đại ELBO theo cả θ và ϕ cùng lúc vừa gần đúng cực đại likelihood biên $p_\theta(x)$ (mô hình sinh tốt hơn), vừa cực tiểu $D_{KL}(q_\phi(z \mid x) \| p_\theta(z \mid x))$ (mô hình suy luận tốt hơn): một hàm mục tiêu, hai mục tiêu cùng đạt được.

3.2 Gradient theo θ dễ, theo ϕ khó

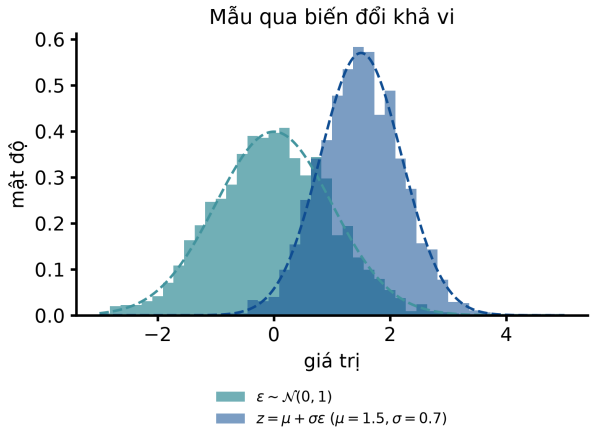
Gradient theo θ đơn giản vì có thể đổi chỗ kỳ vọng và gradient (kỳ vọng không phụ thuộc θ):

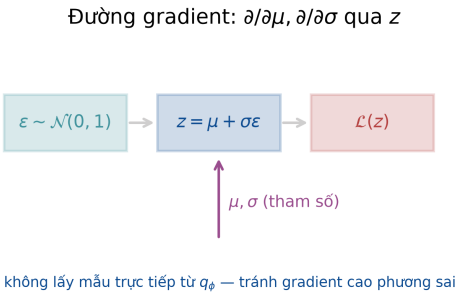
$$\nabla_\theta \mathcal{L}_{\theta,\phi}(x) = \mathbb{E}_{q_\phi(z|x)}[\nabla_\theta \log p_\theta(x,z)] \approx \nabla_\theta \log p_\theta(x,z)$$

(ước lượng Monte Carlo một mẫu). Nhưng gradient theo ϕ khó vì kỳ vọng được lấy theo chính $q_\phi(z \mid x)$, một phân phối phụ thuộc ϕ: không thể đổi chỗ ∇_ϕ và $\mathbb{E}_{q_\phi(z|x)}[\cdot]$ một cách trực tiếp.

3.3 Reparameterization trick

Với biến ẩn liên tục và encoder/decoder khả vi, paper viết lại $z \sim q_\phi(z \mid x)$ thành một phép biến đổi khả vi, khả nghịch của một biến nhiễu độc lập:





Reparameterization z=μ+σϵ với ϵ∼N(0,1): lấy mẫu qua biến đổi khả vi để gradient chảy về μ,σ. (minh hoạ, tự tính)

$$z = g(\varepsilon, \phi, x), \qquad \varepsilon \sim p(\varepsilon) \text{ (không phụ thuộc } \phi, x).$$

Khi đó $\mathbb{E}_{q_\phi(z|x)}[f(z)] = \mathbb{E}_{p(\varepsilon)}[f(z)]$, và kỳ vọng/gradient theo ϕ giao hoán được:

$$\nabla_\phi \mathbb{E}_{q_\phi(z|x)}[f(z)] = \mathbb{E}_{p(\varepsilon)}[\nabla_\phi f(z)] \approx \nabla_\phi f(z).$$

ELBO viết lại: $\mathcal{L}_{\theta,\phi}(x) = \mathbb{E}_{p(\varepsilon)}[\log p_\theta(x,z) - \log q_\phi(z \mid x)]$ với $z = g(\varepsilon, \phi, x)$, và ước lượng một mẫu $\tilde{\mathcal{L}}_{\theta,\phi}(x) = \log p_\theta(x,z) - \log q_\phi(z \mid x)$. Thuật toán tổng quát (Algorithm 1 của paper) gọi là Auto-Encoding Variational Bayes (AEVB); vì nhiễu đến từ cả việc lấy minibatch và việc lấy mẫu ε, đây là một thủ tục "doubly stochastic" (ngẫu nhiên hai lớp). Paper chứng minh tính không chệch (unbiasedness) của gradient này bằng đại số trực tiếp (Eq. 2.30-2.32 của paper).

Để tính $\log q_\phi(z \mid x)$, paper dùng công thức đổi biến với định thức Jacobian:

$$\log q_\phi(z \mid x) = \log p(\varepsilon) - \log d_\phi(x, \varepsilon), \qquad \log d_\phi(x, \varepsilon) = \log \left| \det \frac{\partial z}{\partial \varepsilon} \right|.$$

3.4 Posterior Gaussian factorized và full-covariance

Với encoder Gaussian factorized $q_\phi(z \mid x) = \mathcal{N}(z; \mu, \text{diag}(\sigma^2))$, paper dùng $\varepsilon \sim \mathcal{N}(0, I)$, $z = \mu + \sigma \odot \varepsilon$ ($\odot$ là tích từng phần tử); Jacobian là $\text{diag}(\sigma)$, log-định thức $= \sum_i \log \sigma_i$; mật độ posterior

$$\log q_\phi(z \mid x) = \sum_i [\log \mathcal{N}(\varepsilon_i; 0, 1) - \log \sigma_i].$$

Mở rộng sang full-covariance Gaussian (mục 2.5.1): $z = \mu + L\varepsilon$ với L là ma trận tam giác (phân rã Cholesky $\Sigma = LL^\top$), xây dựng qua $L \leftarrow L_{\text{mask}} \odot L_0 + \text{diag}(\sigma)$ (L_{mask} là ma trận mặt nạ 0 trên/trên đường chéo, 1 dưới đường chéo); định thức Jacobian vẫn đơn giản vì L tam giác: $\log |\det L| = \sum_i \log |L_{ii}|$. Paper cho thuật toán cụ thể (Algorithm 2) tính ước lượng ELBO một mẫu cho một VAE ví dụ: encoder Gaussian full-covariance, decoder Bernoulli factorized (cho dữ liệu nhị phân).

Ví dụ (tự tính): paper cho công thức tổng quát ở Algorithm 2 nhưng không có con số cụ thể nào được tính sẵn trong văn bản chính. Dưới đây tính tay một trường hợp đơn giản (rút gọn về encoder factorized Gaussian

một chiều thay vì full-covariance, để dễ tính tay) theo đúng công thức của Algorithm 2, với dữ liệu Bernoulli 2 chiều.

Giả sử với một điểm dữ liệu $x = (1, 0)$, encoder cho ra $\mu = 0.5$, $\log \sigma = -0.5$ (vậy $\sigma = e^{-0.5} \approx 0.6065$), và lấy mẫu $\varepsilon = 1.0$:

$$z = \mu + \sigma\varepsilon = 0.5 + 0.6065 \times 1.0 = 1.1065.$$

Tính $\tilde{\mathcal{L}}_{\log q(z)} = -(\frac{1}{2}\varepsilon^2 + \frac{1}{2}\log(2\pi) + \log\sigma) = -(0.5 + 0.91894 - 0.5) = -0.91894$.

Tính $\tilde{\mathcal{L}}_{\log p(z)} = -(\frac{1}{2}z^2 + \frac{1}{2}\log(2\pi)) = -(0.5 \times 1.1065^2 + 0.91894) = -(0.61215 + 0.91894) = -1.53109$.

Giả sử decoder cho $p = (0.7, 0.3)$ (xác suất Bernoulli cho hai chiều của x). Với $x = (1, 0)$:

$$\tilde{\mathcal{L}}_{\log p(x|z)} = [1 \cdot \log 0.7 + 0 \cdot \log 0.3] + [0 \cdot \log 0.3 + 1 \cdot \log(1-0.3)] = \log 0.7 + \log 0.7 = 2 \times (-0.356675) = -0.71335.$$

ELBO ước lượng một mẫu: $\tilde{\mathcal{L}} = \tilde{\mathcal{L}}_{\log p(x|z)} + \tilde{\mathcal{L}}_{\log p(z)} - \tilde{\mathcal{L}}_{\log q(z)} = -0.71335 - 1.53109 - (-0.91894) = -1.3255$ nat.

So sánh với công thức đóng (ngoài phạm vi các công thức viết rõ trong các chương đã đọc của paper, nhưng là kết quả chuẩn, thường thấy ở phụ lục của bài Kingma và Welling 2014 gốc): với $q = \mathcal{N}(\mu, \sigma^2)$ và prior $\mathcal{N}(0, 1)$, có công thức đóng

$$D_{KL}(q\|p) = \tfrac{1}{2}(\sigma^2 + \mu^2 - 1 - \log\sigma^2).$$

Với $\mu = 0.5$, $\sigma^2 = e^{-1} \approx 0.3679$: $D_{KL} = 0.5 \times (0.3679 + 0.25 - 1 - (-1)) = 0.5 \times 0.6179 = 0.30895$, tức $\mathbb{E}[\log p(z) - \log q(z)] = -0.30895$ (giá trị kỳ vọng đúng). Số tính bằng một mẫu ở trên cho $-\tilde{\mathcal{L}}_{\log q(z)} + \tilde{\mathcal{L}}_{\log p(z)} = -0.61215$, khác với -0.30895. Đây không phải sai số tính toán, mà minh họa đúng bản chất "ước lượng Monte Carlo một mẫu không chệch trong kỳ vọng, nhưng có nhiễu với từng mẫu cụ thể" mà bài báo nhấn mạnh ở mục 2.4.3 (tính không chệch chỉ đúng khi lấy trung bình trên nhiều mẫu ε, không đúng cho một mẫu đơn lẻ).

3.5 Vượt ra ngoài Gaussian: normalizing flow và IAF

Chương 3 của paper thảo luận hai hướng làm posterior linh hoạt hơn: biến ẩn phụ trợ (auxiliary latent variables) và normalizing flow. Normalizing flow (Rezende và Mohamed, 2015) xây một chuỗi biến đổi khả nghịch $z_t = f_t(z_{t-1}, x)$ bắt đầu từ $\varepsilon_0 \sim p(\varepsilon)$ đơn giản; định thức Jacobian của cả chuỗi là tích các định thức từng bước, nên log-định thức là tổng. Paper đề xuất (dựa trên Kingma và cộng sự, 2016) Inverse Autoregressive Flow (IAF): mỗi bước là một biến đổi tự hồi quy nghịch (inverse autoregressive), có hai tính chất quan trọng: (1) có thể tính song song trên các chiều (khác với việc lấy mẫu từ một mô hình tự hồi quy thường, vốn phải tính tuần tự); (2) Jacobian tam giác với định thức đơn giản là tích các σ_i. IAF ghép T bước như vậy (Algorithm 3), cho phép posterior linh hoạt hơn nhiều so với Gaussian factorized, đặc biệt quan trọng khi không gian biến ẩn có chiều cao. Hình 3.2 của paper minh họa trực quan: trên một tập đồ chơi 4 điểm dữ liệu, posterior factorized Gaussian cho ra các "đốm" tròn đều, còn posterior IAF cho ra hình dạng lệch, phi-Gaussian, khớp tốt hơn với posterior thật.

Chương 4 bàn về mô hình sinh sâu hơn: nhiều tầng biến ẩn $z = \{z_1, ..., z_K\}$ (lấy mẫu tuần tự theo thứ tự topo), và lựa chọn thứ tự topo giữa inference model và generative model (cùng chiều "top-down" hay ngược chiều "bottom-up"); paper trích nhiều công trình (Salimans 2016; Sønderby và cộng sự 2016a; Kingma và cộng sự 2016) cho thấy dùng cùng thứ tự top-down cho cả hai giúp chia sẻ tham số, học nhanh hơn.

4. Thí nghiệm và kết quả chính

Đây là một bài mang tính chuyên khảo/sư phạm hơn là một bài công bố thực nghiệm mới; phần lớn các ”kết quả” được trích dẫn lại từ các bài khác mà paper tổng hợp, cần phân biệt rõ với phần thực nghiệm (nếu có) của chính paper này:

- **Hình 3.2 (thực nghiệm minh họa của chính paper hoặc kế thừa trực tiếp từ Kingma và cộng sự 2016, cùng nhóm tác giả):** trên một tập dữ liệu đồ chơi 4 điểm, so sánh posterior factorized Gaussian với posterior IAF; kết quả hình ảnh cho thấy IAF linh hoạt hơn rõ rệt.
- Mục 3.2.1 có câu ”In our experiment with CIFAR-10, we make use of multiple layers of stochastic variables” (trong thí nghiệm của chúng tôi với CIFAR-10, chúng tôi dùng nhiều tầng biến ngẫu nhiên) nhưng không kèm bảng số liệu định lượng cụ thể nào trong các chương chính đã đọc; đây là một câu nhắc tới thực nghiệm mà không trình bày chi tiết kết quả trong văn bản.
- **Mục 2.9.1 (kết quả định lượng trích dẫn từ một bài khác):** paper trích Kucukelbir và cộng sự (2017) cho biết ước lượng gradient kiểu score-function (REINFORCE) cần ”hai cấp độ lớn” (two orders of magnitude) nhiều mẫu hơn ước lượng theo reparameterization để đạt phương sai tương đương. Đây là một con số định lượng thật, nhưng là kết quả của một bài khác được paper này trích dẫn lại, không phải thực nghiệm tự paper này chạy.
- **Mục 4.5.1 (các kết quả trích dẫn từ nhiều bài follow-up khác):** ví dụ Maaløe và cộng sự (2016) đạt dưới 1% lỗi phân loại trên MNIST khi chỉ dùng 10 ảnh có nhãn mỗi lớp (hơn 99.8% nhãn bị bỏ) bằng kỹ thuật bán giám sát dựa trên VAE; Pu và cộng sự (2016) so sánh GoogLeNet huấn luyện với 1% ảnh ImageNet có nhãn (khoảng 5% độ chính xác top-1) với một cách tiếp cận bán giám sát dùng VAE (khoảng 45% độ chính xác) trên cùng thiết lập. Cả hai là kết quả thật, có số liệu cụ thể, nhưng đều là kết quả của các bài khác, được chương ”Follow-Up Work” của paper này tổng hợp lại, không phải thực nghiệm gốc của chính Kingma và Welling trong bài 2019 này.
- Các ứng dụng khác được liệt kê (thiết kế hóa học của Gómez-Bombarelli và cộng sự 2018, sinh câu văn của Bowman và cộng sự 2015, mô phỏng ảnh thiên văn của Ravanbakhsh và cộng sự 2017, ”smile vector” của White 2016) đều là ví dụ minh họa từ các bài khác, trình bày lại dưới dạng hình ảnh trong paper, không phải số liệu định lượng của chính paper 2019 này.

Nhận xét của người ghi chú: khác với math 08 (có ít nhất hai thực nghiệm thật tự tác giả chạy: so sánh CAVI với Stan, và LDA trên 1,8 triệu bài báo), bài VAE 2019 này gần như không có một bảng số liệu thực nghiệm mới nào do chính Kingma và Welling tự chạy và báo cáo đầy đủ trong văn bản chính; vai trò chính của bài là trình bày lại khung lý thuyết (ELBO, reparameterization, IAF) và tổng hợp có hệ thống các kết quả thực nghiệm từ rất nhiều bài khác.

5. Phân tích / ablation

Không có bảng ablation có kiểm soát do chính paper này tự thực hiện. Các phần gần nhất với ”phân tích” là:

- Mục 2.9.1: so sánh định tính và trích một con số định lượng (hai cấp độ lớn số mẫu) giữa hai loại ước lượng gradient (reparameterization và score function), giải thích bằng trực giác ”information bottleneck”: ước lượng score function chỉ dùng giá trị vô hướng $\log p_\theta(x, z)$, còn ước lượng reparameterization dùng được cả vector gradient $\nabla_z \log p_\theta(x, z)$ (rộng hơn, nhiều thông tin hơn), nên phương sai thấp hơn.
- Mục 2.8.1: khoảng giá trị free-bits $\lambda \in \{0.125, 0.25, 0.5, 1, 2\}$ được báo cáo là ”hoạt động tốt cho một khoảng khá rộng giá trị”, cải thiện log-likelihood trên một bộ benchmark, nhưng đây là kết quả trích từ Kingma và cộng sự (2016), không phải khảo sát mới của bài 2019 này.
- Hình 3.2 là phép so sánh định tính (trực quan, không có bảng số liệu) giữa hai lựa chọn kiến trúc posterior (factorized Gaussian và IAF).

6. Giới hạn

Giới hạn/nhược điểm bài báo nêu rõ:

- **Vấn đề tối ưu/"posterior collapse" (mục 2.8.1):** paper báo cáo (nhất quán với Bowman và cộng sự 2015 và Sønderby và cộng sự 2016a) rằng tối ưu ngẫu nhiên với ELBO nguyên bản có thể bị kẹt ở một điểm cân bằng ổn định không mong muốn, nơi $q(z \mid x) \approx p(z)$ (biến ẩn gần như bị "bỏ qua") ngay từ đầu huấn luyện, vì khi đó số hạng likelihood $\log p(x \mid z)$ còn yếu. Giải pháp (KL annealing, hoặc "free bits" của Kingma và cộng sự 2016) chỉ là các biện pháp khắc phục thực nghiệm (heuristic), không phải một sửa đổi có cơ sở lý thuyết đầy đủ.
- **"Blurriness" (mục 2.7-2.8.2):** vì ELBO tương đương cực tiểu $D_{KL}(q_{D,\phi}(x,z)\|p_\theta(x,z))$ theo một chiều cụ thể của KL (zero-avoiding, không phải zero-forcing), nếu không khớp hoàn hảo được, mô hình sinh p_θ có xu hướng trải rộng xác suất hơn phân phối dữ liệu thật (vì bị phạt rất nặng nếu bỏ sót vùng dữ liệu thật có xác suất, nhưng bị phạt nhẹ nếu "lan" xác suất ra vùng không có dữ liệu thật). Đây là lời giải thích của chính paper cho hiện tượng ảnh sinh ra từ VAE thường bị mờ/nhòe so với ảnh thật.
- **Posterior factorized/Gaussian đơn giản hạn chế độ linh hoạt:** động lực cho cả chương 3 (normalizing flow, IAF); ngay cả flow tổng quát cũng bị giới hạn bởi yêu cầu khả nghịch và Jacobian dễ tính (planar/radial flow của Rezende và Mohamed 2015 được bài báo lưu ý xét là "không rõ cách mở rộng lên không gian biến ẩn có chiều rất cao").
- **IWAE (mục 2.6):** bài báo lưu ý xét ước lượng importance-weighted có "scaling rất tệ" (notoriously bad scaling properties) khi không gian biến ẩn có chiều cao.
- **Reparameterization trick chỉ áp dụng cho biến ẩn liên tục:** biến ẩn rời rạc cần ước lượng kiểu score function/REINFORCE (mục 2.9.1), có phương sai cao hơn hẳn theo chính kết quả paper trích dẫn.
- **Tuyên bố mạnh ở phần kết luận (mục 5):** bài báo lưu ý "VAE vẫn là khung duy nhất trong tài liệu cho phép cả biến quan sát rời rạc và liên tục, cho phép suy luận biến ẩn khấu hao hiệu quả và tổng hợp (synthesis) nhanh, và có thể đạt hiệu năng gần trạng thái tiên tiến nhất về log-likelihood dữ liệu". **Quan sát riêng của người ghi chú:** đây là đánh giá của chính các tác giả tại thời điểm viết (bản v3, cuối 2019); đây là một tuyên bố mạnh, mang tính chủ quan của tác giả về vị trí của phương pháp so với các họ mô hình khác, không phải một kết quả đo lường được, nên cần đọc như một nhận định có bối cảnh thời điểm, không phải một sự thật toán học.

7. Liên hệ với các bài khác

- **math 08 (Variational Inference: A Review for Statisticians):** paper này dùng đúng khung ELBO/KL của math 08 nhưng khác biệt cốt lõi được chính văn bản paper nêu rõ (mục 2.1): VAE dùng "amortized inference" (một mạng suy luận dùng chung cho mọi điểm dữ liệu), trong khi "các phương pháp variational inference truyền thống" (chính là CAVI ở math 08) dùng tham số biến phân riêng cho từng điểm dữ liệu, kém hiệu quả với tập dữ liệu lớn. Phụ lục A.2.2 của paper này còn viết lại rõ công thức variational EM với tham số cục bộ $\phi^{(i)}$ (đúng là công thức CAVI của math 08) để làm nền so sánh.
- **math 01/02 (Matrix Calculus / Automatic Differentiation):** reparameterization trick (mục 2.4) hoạt động được chính là nhờ lan truyền gradient qua phép biến đổi $z = g(\varepsilon, \phi, x)$ bằng backpropagation/automatic differentiation; Phụ lục A.3 của paper này gọi thẳng đây là "reverse-mode automatic differentiation (còn gọi là thuật toán backpropagation)", đúng chủ đề math 02; chain rule dùng trong suy luận gradient chính là chủ đề math 01.
- **math 03 (PCA Tutorial):** mục 2.9 của paper trích dẫn Roweis (1998) chỉ ra rằng ở giới hạn nhiễu decoder tiến về 0 của một VAE linear-Gaussian ($p(z) = \mathcal{N}(0, I)$, $p(x \mid z) = \mathcal{N}(x; Wz, \epsilon I)$), nghiệm maximum-likelihood của W chính là PCA, và posterior trở thành một phân phối Dirac delta (không còn độ không chắc chắn). Đây là liên hệ trực tiếp, được chính văn bản xác nhận, không phải suy diễn của người ghi chú.
- **math 10 (A Conceptual Introduction to Hamiltonian Monte Carlo):** Phụ lục A.2.3 ("MCMC-EM") của paper này thảo luận một cách khác để tối ưu likelihood bằng EM kết hợp MCMC, trích dẫn Neal

(2011) ”MCMC Using Hamiltonian Dynamics”, đúng chủ đề cốt lõi của math 10, và bài báo lưu ý xét nhược điểm của hướng này là cần chạy một xích MCMC độc lập cho mỗi điểm dữ liệu (khác hẳn với việc dùng một mạng encoder chung của VAE).

8. Gợi ý áp dụng

VAE cung cấp một ví dụ cụ thể của suy luận biến phân trong mô hình sinh. Khi áp dụng, cần theo dõi cả chất lượng tái tạo và vai trò của biến tiềm ẩn; ELBO không tự bảo đảm mỗi chiều tiềm ẩn có một ý nghĩa riêng biệt.

9. Thí nghiệm gợi ý

Các thí nghiệm nhỏ sau chạy được trên GPU:

1. **VAE nhỏ trên dữ liệu ảnh đơn giản.** Cài một VAE nhỏ (encoder/decoder vài tầng fully-connected hoặc conv nhỏ, latent dim 2-20) trên MNIST hoặc một tập ký tự/glyph nhỏ, dùng đúng Algorithm 1 (reparameterization trick). Đo: đường cong ELBO theo epoch, chất lượng ảnh tái tạo (reconstruction), và chất lượng ảnh sinh từ mẫu prior $z \sim \mathcal{N}(0, I)$ thuần túy (không qua encoder). Kỳ vọng: ảnh tái tạo rõ hơn ảnh sinh từ prior thuần túy (nhất quán với mục 2.8.2 về blurriness khi sinh không điều kiện).
2. **So sánh posterior factorized Gaussian và IAF trên dữ liệu đồ chơi.** Lặp lại tinh thần Hình 3.2: một tập vài điểm dữ liệu 2D tổng hợp, huấn luyện một encoder Gaussian factorized và một encoder có thêm 1-2 bước IAF, trực quan hóa hình dạng posterior của từng điểm. Đo: ELBO cuối cùng của hai cấu hình. Kỳ vọng: IAF cho ELBO cao hơn (bound chặt hơn) nhờ posterior linh hoạt hơn.
3. **So sánh phương sai gradient: reparameterization và score function.** Trên một mô hình nhỏ cố định (ví dụ 1 tầng ẩn), tính gradient ELBO theo ϕ bằng cả hai ước lượng (reparameterization và REINFORCE/score function, mục 2.9.1 và 2.72-2.73 của paper) trên nhiều lần lấy mẫu lại, đo phương sai thực nghiệm của mỗi ước lượng. Kỳ vọng: ước lượng score function có phương sai lớn hơn đáng kể, kiểm chứng lại con số ”hai cấp độ lớn” mà paper trích dẫn từ Kucukelbir và cộng sự (2017), ở quy mô nhỏ hơn.
4. **Tái hiện posterior collapse và cách khắc phục.** Huấn luyện một VAE với decoder đủ mạnh (ví dụ decoder có nhiều tham số hơn hẳn encoder) trên dữ liệu tuần tự tổng hợp đơn giản, không dùng KL annealing; đo KL trung bình mỗi chiều latent theo epoch để kiểm tra có bị ”collapse” về 0 không (dấu hiệu $q(z \mid x) \approx p(z)$). Lặp lại với KL annealing hoặc free-bits (λ trong khoảng paper gợi ý, ví dụ 0.5) và so sánh KL trung bình, chất lượng tái tạo. Kỳ vọng: không annealing dễ bị collapse (KL gần 0), có annealing/free-bits giữ được KL dương ổn định hơn.
5. **Kiểm chứng liên hệ VAE tuyến tính và PCA (liên hệ trực tiếp với math 03).** Huấn luyện một VAE ”tuyến tính” (encoder và decoder đều là hàm tuyến tính, không có lớp phi tuyến) trên một tập dữ liệu nhỏ, với nhiễu decoder rất nhỏ; so sánh không gian con học được bởi ma trận W của decoder với các thành phần chính (principal components) tính trực tiếp bằng eigen-decomposition của ma trận hiệp phương sai (như ở math 03). Kỳ vọng: khi nhiễu decoder càng nhỏ, không gian con của W càng gần không gian các thành phần chính đầu, xác nhận số liệu cho liên hệ Roweis (1998) mà paper trích ở mục 2.9.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Mô hình sinh biến ẩn sâu (DLVM)	$p_\theta(x, z) = p_\theta(z)\, p_\theta(x \mid z)$, likelihood biên $p_\theta(x) = \int p_\theta(x, z)\, dz$ thường không tính được
Encoder / mô hình suy luận	$q_\phi(z \mid x) \approx p_\theta(z \mid x)$, tham số ϕ dùng chung cho mọi điểm dữ liệu (amortized inference)
ELBO	$\mathcal{L}_{\theta,\phi}(x) = \mathbb{E}_{q_\phi(z\mid x)}[\log p_\theta(x, z) - \log q_\phi(z \mid x)]$

Khái niệm	Công thức / ý nghĩa
Liên hệ ELBO - log likelihood	$\log p_\theta(x) = \mathcal{L}_{\theta,\phi}(x) + D_{KL}(q_\phi(z \mid x) \| p_\theta(z \mid x)) \geq \mathcal{L}_{\theta,\phi}(x)$
Reparameterization trick	$z = g(\varepsilon, \phi, x)$, $\varepsilon \sim p(\varepsilon)$ độc lập ϕ, x; cho phép $\nabla_\phi \mathbb{E}_{q_\phi}[f(z)] \approx \nabla_\phi f(z)$
Log-mật độ qua đổi biến	$\log q_\phi(z \mid x) = \log p(\varepsilon) - \log\lvert\det(\partial z/\partial\varepsilon)\rvert$
Gaussian factorized	$z = \mu + \sigma \odot \varepsilon$, log-định thức Jacobian $= \sum_i \log \sigma_i$
Gaussian full-covariance	$z = \mu + L\varepsilon$, L tam giác (Cholesky của $\Sigma = LL^\top$), log-định thức $= \sum_i \log\lvert L_{ii}\rvert$
Normalizing flow / IAF	Chuỗi biến đổi khả nghịch $z_t = f_t(z_{t-1}, x)$, log-định thức Jacobian toàn chuỗi là tổng log-định thức từng bước
KL Gaussian-Gaussian (công thức đóng, không nằm trong các chương chính đã đọc)	$D_{KL}(\mathcal{N}(\mu, \sigma^2) \| \mathcal{N}(0,1)) = \frac{1}{2}(\sigma^2 + \mu^2 - 1 - \log\sigma^2)$
Hạn chế đã biết	Posterior collapse (mục 2.8.1); ”blurriness” do chiều KL (mục 2.8.2); reparameterization chỉ cho biến liên tục

11. Kết luận ngắn

Bài này là bản mở rộng có tính sư phạm của bài AEVB gốc (Kingma và Welling, 2014), trình bày lại ELBO, reparameterization trick, và các mở rộng (normalizing flow/IAF, mô hình nhiều tầng biến ẩn) có chứng minh từng bước (ví dụ tính không chệch của gradient reparameterization). Khác với math 08, bài này gần như không có thực nghiệm số liệu mới do chính tác giả chạy và báo cáo chi tiết trong văn bản chính; phần lớn ”kết quả” ở chương 4 là tổng hợp lại các con số thật nhưng lấy từ rất nhiều bài follow-up khác (bán giám sát trên MNIST/ImageNet, thiết kế hóa học, sinh câu văn, v.v.), cần đọc với sự phân biệt rõ nguồn gốc. Về mặt lý thuyết, đóng góp cốt lõi (reparameterization trick, amortized inference) được trình bày chặt chẽ, và paper cũng thẳng thắn nêu các hạn chế đã biết của VAE (posterior collapse, blurriness, giới hạn của reparameterization với biến rời rạc). Ghi chú này đã đọc toàn bộ 5 chương và Phụ lục A; phần danh mục tham khảo chỉ đọc lướt để xác nhận đó là trích dẫn thuần túy.

Nguồn gốc: https://arxiv.org/abs/1906.02691

Bài 10

A Conceptual Introduction to Hamiltonian Monte Carlo (2017)

Tác giả: Michael Betancourt (Applied Statistics Center, Columbia University; phần lớn bài viết hoàn thành khi là Research Fellow tại Centre for Research in Statistical Methodology, Đại học Warwick).
Venue / arXiv: arXiv:1701.02434 (v2, 16/07/2018); paper không nêu venue hội nghị hay tạp chí cụ thể trên bản arXiv.
Nguồn: https://arxiv.org/abs/1701.02434
Chủ đề: giải thích trực giác hình học (tránh dùng vi phân hình học hình thức) cho việc tại sao Hamiltonian Monte Carlo (HMC) khám phá hiệu quả các phân phối xác suất nhiều chiều, cách chỉnh các bậc tự do của HMC (kinetic energy, thời gian tích phân, symplectic integrator) để triển khai tốt, và cách chẩn đoán khi phương pháp thất bại.
Vị trí trong lộ trình: đọc sau math 08 (Variational Inference: A Review for Statisticians), vì cả hai là họ phương pháp suy luận xấp xỉ hậu nghiệm Bayes nhưng theo hướng ngược nhau (lấy mẫu MCMC so với tối ưu biến phân); đọc trước math 11 (Stochastic Gradient Descent as Approximate Bayesian Inference), vì bài 11 dùng chính ngôn ngữ ”động lực học” tương tự (Langevin dynamics, nhiệt độ, nhiễu) để biến SGD thành một dạng lấy mẫu Bayes gần đúng và có nhắc tới HMC như một họ phương pháp liên quan. Cũng liên quan tới math 06 (natural gradient) qua khái niệm Riemannian metric.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (bản arXiv); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc.

1. Bối cảnh

Một bài toán trong thống kê tính toán là ước lượng kỳ vọng $\mathbb{E}_\pi[f]$ của hàm f theo phân phối mục tiêu π trên không gian tham số D chiều Q, chẳng hạn một hậu nghiệm Bayes. Khi không tính được kỳ vọng bằng giải tích, ta có thể dùng Markov chain Monte Carlo (MCMC): xây dựng một xích Markov có phân phối dừng là π rồi lấy trung bình f trên các mẫu của xích.

Khó khăn nằm ở **hình học của không gian nhiều chiều**. Paper chỉ ra (Hình 1, Hình 2 trong bài) rằng khi chia không gian D chiều thành các hộp quanh mode, số hộp lân cận mode là $3^D - 1$: với $D = 1$ có 2 hộp lân cận, $D = 3$ đã có 26. Thể tích bên ngoài áp đảo thể tích quanh mode theo cấp lũy thừa khi D tăng, trong khi mật độ xác suất $\pi(q)$ lại lớn nhất tại mode. Kỳ vọng được tính bằng tích phân $\int \pi(q) f(q)\, dq$, mà tích phân này là **tích của mật độ và thể tích**; vùng mật độ lớn thì thể tích nhỏ, vùng thể tích lớn thì mật độ nhỏ, nên vùng đóng góp chính cho kỳ vọng là một ”vành” hẹp giữa hai cực gọi là **typical set**. Khi D tăng, typical set càng mỏng và càng xa mode (hiện tượng ”concentration of measure”). Hệ quả: lưới đều (quadrature) cần N^D điểm để phủ không gian nhưng hầu như không điểm nào rơi vào typical set, nên quadrature thất bại theo cấp lũy thừa với D. Mọi thuật toán lấy mẫu hiệu quả phải tập trung tính toán vào typical set.

Khái niệm cần nắm trước khi đọc (paper giả định người đọc chỉ cần xác suất và giải tích cơ bản, không cần hình học vi phân):

- **Markov transition** $T(q' \mid q)$: xác suất chuyển từ điểm q sang q'. Transition ”bảo toàn” π nếu $\pi(q) = \int dq'\, \pi(q') T(q \mid q')$.
- **Hiệu quả (effective sample size, ESS):** $\text{ESS} = N / (1 + 2 \sum_{l=1}^{\infty} \rho_l)$ với ρ_l là tự tương quan trễ l của f theo lịch sử xích; ESS đo số mẫu độc lập ”tương đương” chứa trong xích.

- **Metropolis-Hastings (MH):** cơ chế đề xuất $Q(q' \mid q)$ rồi chấp nhận với xác suất $a(q' \mid q) = \min\left(1, \frac{Q(q|q')\pi(q')}{Q(q'|q)\pi(q)}\right)$.

2. Ý tưởng chính

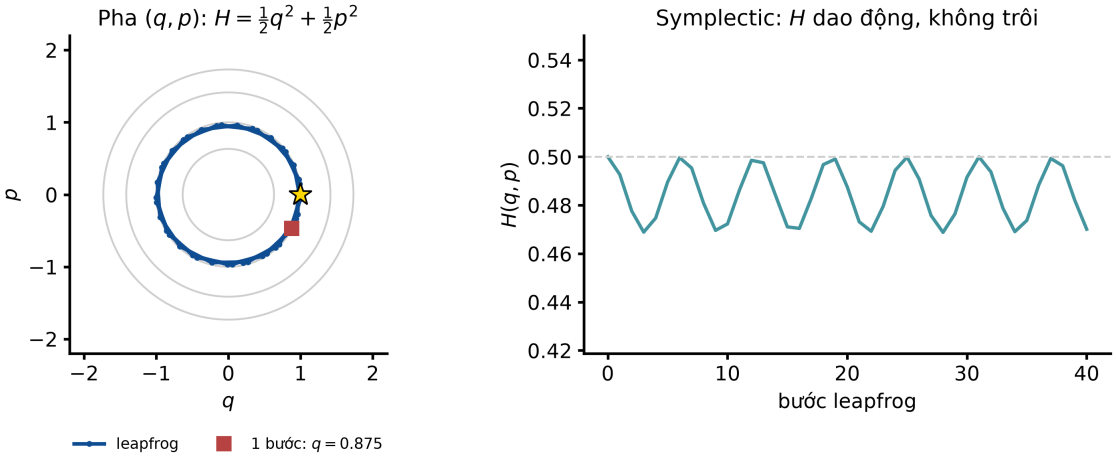
Random Walk Metropolis (RWM, đề xuất Gaussian $Q(q' \mid q) = \mathcal{N}(q' \mid q, \Sigma)$) là thuật toán MCMC cổ điển nhất, nhưng nó chỉ ”đoán và kiểm tra” ngẫu nhiên, không dùng thông tin cục bộ của π. Trong không gian nhiều chiều, hầu hết các đề xuất rơi ra ngoài typical set (bị density thấp từ chối) hoặc phải co nhỏ bước đi để được chấp nhận (nhưng khi đó xích di chuyển quá chậm). Paper lập luận: để mở rộng MCMC lên các bài toán nhiều chiều thực tế, ta cần **tận dụng thông tin hình học của typical set**, cụ thể là gradient $\nabla \log \pi(q)$.

Ý tưởng chính (Mục 3.1): một vector field thẳng theo gradient sẽ kéo ta về **mode** (nơi gradient bằng 0), không phải về typical set (vì gradient phụ thuộc cách tham số hoá, còn typical set thì không). Để ”bẻ” gradient cho thẳng hàng với typical set, paper dùng một phép tương tự vật lý: nghĩ mode như một hành tinh, gradient như lực hấp dẫn, và typical set như quỹ đạo quay quanh hành tinh đó. Một vật ở trạng thái nghỉ sẽ rơi vào hành tinh (giống như một bước gradient thuần sẽ rơi vào mode); nhưng nếu trang bị đúng **động lượng (momentum)** vuông góc với lực hấp dẫn, vật sẽ duy trì quỹ đạo ổn định quanh hành tinh. Cách duy nhất để thêm động lượng có cấu trúc xác suất đúng (bảo toàn thể tích pha, volume-preserving) chính là **Hamiltonian Monte Carlo**: mở rộng không gian tham số q thành không gian pha (q, p) với p là biến động lượng phụ trợ, định nghĩa một hàm Hamiltonian $H(q, p) = -\log \pi(q, p)$, và dùng phương trình Hamilton để sinh ra các quỹ đạo bám theo typical set của phân phối mở rộng $\pi(q, p)$, sau đó chiếu ngược lại không gian q để được mẫu từ $\pi(q)$.

3. Phương pháp chi tiết

3.1 Không gian pha và phương trình Hamilton (Mục 3.2-3.3)

Mở rộng mỗi chiều q_n thành cặp (q_n, p_n), được không gian pha $2D$ chiều. Phân phối hậu nghiệm được ”nâng” lên phân phối chính tắc (canonical distribution) trên không gian pha bằng cách chọn một phân phối có điều kiện $\pi(p \mid q)$ cho động lượng:



HMC leapfrog trên H=q²/2+p²/2: quỹ đạo pha gần mức năng lượng; H dao động không trôi (ε=0.5, q_0=1). (minh hoạ, tự tính)

$$\pi(q, p) = \pi(p \mid q)\, \pi(q).$$

Vì $\pi(q, p)$ không phụ thuộc cách tham số hoá, ta viết nó qua hàm **Hamiltonian** (bất biến):

$$\pi(q,p) = e^{-H(q,p)}, \qquad H(q,p) = -\log \pi(q,p) = \underbrace{-\log \pi(p \mid q)}_{K(p,q)\ \text{(kinetic energy)}} + \underbrace{-\log \pi(q)}_{V(q)\ \text{(potential energy)}}.$$

$V(q)$ hoàn toàn do phân phối mục tiêu quyết định; $K(p,q)$ là bậc tự do do người triển khai chọn. Vector field sinh ra quỹ đạo khám phá là **phương trình Hamilton**:

$$\frac{dq}{dt} = \frac{\partial H}{\partial p} = \frac{\partial K}{\partial p}, \qquad \frac{dp}{dt} = -\frac{\partial H}{\partial q} = -\frac{\partial K}{\partial q} - \frac{\partial V}{\partial q}.$$

Vì $\partial V/\partial q = -\nabla \log \pi(q)$, phương trình Hamilton chính là cách "truyền" gradient qua động lượng để tạo chuyển động thẳng hàng với typical set, đúng như ý tưởng ở mục 2. Một Markov transition Hamiltonian lý tưởng gồm 3 bước: (1) lấy mẫu động lượng $p \sim \pi(p \mid q)$ (nâng điểm hiện tại lên không gian pha), (2) tích phân phương trình Hamilton trong thời gian t: $(q,p) \to \phi_t(q,p)$, (3) chiếu $(q,p) \to q$ (bỏ động lượng). Vì phương trình Hamilton bảo toàn H (do đó bảo toàn thể tích pha, shear transformation), quỹ đạo luôn ở trên một **mức năng lượng** (energy level set) $H^{-1}(E) = \{(q,p) : H(q,p) = E\}$.

3.2 Phân rã vi chính tắc (microcanonical decomposition, Mục 4.1)

Không gian pha phân tầng (foliate) thành các mức năng lượng lồng nhau. Mỗi điểm được xác định bởi năng lượng E và vị trí θ_E trên mức đó. Phân phối chính tắc phân rã thành:

$$\pi(q,p) = \pi(\theta_E \mid E)\, \pi(E),$$

gọi $\pi(\theta_E \mid E)$ là **phân phối vi chính tắc (microcanonical)** và $\pi(E)$ là **phân phối năng lượng biên (marginal energy distribution)**. Một xích Hamiltonian Markov chia thành hai pha xen kẽ: **tất định** (quỹ đạo Hamiltonian khám phá một mức năng lượng) và **ngẫu nhiên** (lấy lại mẫu động lượng tạo một bước random-walk giữa các mức năng lượng). Hiệu năng pha tất định phụ thuộc thời gian tích phân và độ đều của mức năng lượng; hiệu năng pha ngẫu nhiên phụ thuộc việc phân phối chuyển tiếp năng lượng $\pi(E \mid q)$ (sinh ra khi lấy lại mẫu p tại một điểm q) có "khớp" với $\pi(E)$ hay không. Nếu $\pi(E \mid q)$ hẹp hơn nhiều so với $\pi(E)$, random walk giữa các mức năng lượng rất chậm.

3.3 Chọn kinetic energy (Mục 4.2)

Euclidean-Gaussian: nếu không gian tham số có một metric Euclid g (ma trận $D \times D$), ta định nghĩa $M = R\,S\,g\,S^{\mathsf{T}}R^{\mathsf{T}}$ ("mass matrix"), và chọn $\pi(p \mid q) = \mathcal{N}(p \mid 0, M)$, cho kinetic energy:

$$K(q,p) = \tfrac{1}{2}\, p^{\mathsf{T}} M^{-1} p + \log |M| + \text{const.}$$

Lựa chọn tối ưu (Mục 4.2.1) là đặt nghịch đảo metric bằng hiệp phương sai mục tiêu, $M^{-1} = \mathbb{E}_\pi[(q-\mu)(q-\mu)^{\mathsf{T}}]$, ước lượng thực nghiệm trong giai đoạn warm-up (chạy thử, cập nhật M, chạy lại). Việc này tương đương xoay/co giãn không gian tham số để "khử tương quan" phân phối mục tiêu, làm các mức năng lượng đều hơn.

Riemannian-Gaussian (Mục 4.2.2, Girolami và Calderhead 2011): nếu phân phối mục tiêu có độ cong thay đổi theo vị trí (không Gaussian toàn cục), dùng metric biến thiên $\Sigma(q)$: $\pi(p \mid q) = \mathcal{N}(p \mid 0, \Sigma(q))$, $K(q,p) = \frac{1}{2} p^{\mathsf{T}} \Sigma^{-1}(q) p + \frac{1}{2} \log |\Sigma(q)|$. Nếu $\Sigma(q)$ gần giống Hessian của $-\log \pi(q)$ (ví dụ SoftAbs metric, Betancourt 2013a), các mức năng lượng được "sửa cục bộ" nên đều hơn ở mọi vùng. Paper nhận xét non-Gaussian kinetic energy thường kém hơn thực nghiệm, vì khi D lớn, $\pi(E)$ có xu hướng tiến về Gaussian theo định lý giới hạn trung tâm (CLT trên tổng nhiều biến), nên chỉ kinetic energy Gaussian mới khớp tốt.

3.4 Thời gian tích phân tối ưu và NUTS (Mục 4.3)

Với thời gian tích phân T cố định, kỳ vọng dọc theo quỹ đạo E_T hội tụ về kỳ vọng không gian E_ϕ trên "quỹ đạo kín" (orbit) khi $T \to \infty$ (gọi là "dynamic ergodicity"). Giai đoạn đầu hội tụ siêu tuyến tính (mỗi bước mới đều khám phá vùng chưa thăm), sau đó chuyển sang giai đoạn tiệm cận chậm (quỹ đạo quay lại vùng đã thăm). Với họ mục tiêu 1 chiều $\pi_\beta(q) \propto e^{-|q|^\beta}$ và kinetic energy chuẩn $\mathcal{N}(0,1)$, thời gian tích phân tối ưu tỉ lệ:

$$T_{\text{optimal}}(q,p) \propto H(q,p)^{\frac{2-\beta}{2\beta}}.$$

Với $\beta < 2$ (đuôi nặng hơn Gaussian), T_{optimal} tăng khi năng lượng (tức đi sâu vào đuôi) tăng, nên một T tĩnh không bao giờ tối ưu cho mọi vùng; chỉ $\beta = 2$ (đúng Gaussian) mới có T tối ưu hằng số. Đây là động lực cho tiêu chí dừng động (dynamic): **No-U-Turn Sampler (NUTS)** (Hoffman và Gelman, 2014; Betancourt 2013b) dừng quỹ đạo khi nó bắt đầu "quay đầu" lại điểm xuất phát, chi tiết thuật toán nằm ở Phụ lục A.4-A.5 của paper.

3.5 Symplectic integrator và sửa lỗi MH (Mục 5)

Không thể giải chính xác phương trình Hamilton (trừ vài trường hợp đặc biệt), nên phải tích phân số. Bộ giải số thông thường bị "trôi" (drift) tích luỹ lỗi theo thời gian; **symplectic integrator** (ví dụ **leapfrog**) thì bảo toàn thể tích pha đúng như hệ thật, nên quỹ đạo số chỉ dao động quanh mức năng lượng đúng, không trôi xa. Với kinetic energy không phụ thuộc vị trí, leapfrog với bước ϵ:

$$p_{n+\frac{1}{2}} \leftarrow p_n - \tfrac{\epsilon}{2}\frac{\partial V}{\partial q}(q_n), \qquad q_{n+1} \leftarrow q_n + \epsilon\, p_{n+\frac{1}{2}}, \qquad p_{n+1} \leftarrow p_{n+\frac{1}{2}} - \tfrac{\epsilon}{2}\frac{\partial V}{\partial q}(q_{n+1}).$$

Vì leapfrog có sai số nhỏ nhưng khác 0, cần một bước sửa Metropolis-Hastings để có mẫu đúng từ phân phối chính tắc. Vì quỹ đạo số là tất định và không "nghịch" (không thể đề xuất lùi), đề xuất trần (chỉ lấy điểm cuối q_L, p_L) luôn có xác suất chấp nhận MH bằng 0 (Hình 30 của paper). Cách sửa: thêm bước lật dấu động lượng $(q,p) \to (q,-p)$ để đề xuất trở thành nghịch được, cho xác suất chấp nhận:

$$a((q_L,-p_L) \mid (q_0,p_0)) = \min\Big(1,\ \exp\big(-H(q_L,-p_L) + H(q_0,p_0)\big)\Big).$$

Vì leapfrog gần bảo toàn H, xác suất chấp nhận này suy giảm rất ít khi D tăng, khác hẳn RWM. Paper cũng trình bày cách tổng quát hoá: đề xuất đa điểm trên cả quỹ đạo (không chỉ điểm cuối) bằng lấy mẫu đa thức (multinomial) có trọng số $\propto \pi(z)$ trên quỹ đạo, xây dựng tiến dần (progressive sampling) bằng cách gộp hai nửa quỹ đạo cũ/mới theo trọng số $w_{\text{old}}/(w_{\text{old}} + w_{\text{new}})$ (Mục A.2-A.3, công thức (1)-(2) trong paper). Khi độ dài quỹ đạo là lũy thừa của 2, việc mở rộng tương đương xây cây nhị phân cân đối theo đệ quy (Hình 37-38), đây là cấu trúc được Stan dùng cho NUTS hiện đại (Phụ lục A.5).

Cuối cùng, cấu hình symplectic integrator gồm **bước** ϵ và **bậc** K (số lần tính gradient mỗi bước). Bước nhỏ hơn chính xác hơn nhưng tốn nhiều bước hơn ($L = T/\epsilon$); bậc cao chính xác hơn nhưng tốn hơn mỗi bước. Paper dùng hình học của "Hamiltonian bị điều chỉnh" (modified/shadow Hamiltonian, mà symplectic integrator giải đúng) để suy ra cận trên/dưới giữa chi phí và xác suất chấp nhận MH trung bình, từ đó đề xuất **điều chỉnh ϵ tự động trong warm-up để đạt xác suất chấp nhận trung bình trong khoảng 0.6 đến 0.8** (Mục 5.3).

4. Thí nghiệm và kết quả chính

Đây là bài **tổng quan khái niệm (conceptual review)**, không có thí nghiệm thực nghiệm kiểu deep learning (không có dataset, không có bảng so sánh accuracy trên nhiều mô hình). ”Kết quả chính” của paper là các kết luận giải tích/định lý/ví dụ minh hoạ, cụ thể:

Kết quả	Nội dung	Vị trí trong paper
Thể tích vùng lân cận mode	$3^D - 1$ hộp lân cận trong D chiều; tỉ trọng thể tích hộp trung tâm là $1/3, 1/9, 1/27$ với $D = 1, 2, 3$	Hình 1, Mục 1.3
Thời gian tích phân tối ưu (giải tích, 1D)	$T_{\text{optimal}} \propto H^{(2-\beta)/(2\beta)}$ cho họ $\pi_\beta(q) \propto e^{-\lvert q\rvert^\beta}$	Mục 4.3
Cận chi phí / xác suất chấp nhận	Cận trên và dưới giữa chi phí tính toán và xác suất chấp nhận MH trung bình, khuyến nghị huấn luyện ϵ để đạt chấp nhận 0.6-0.8	Mục 5.3, Hình 33
E-BFMI (Bayesian fraction of missing information cho năng lượng)	Chỉ số chẩn đoán kinetic energy kém; thực nghiệm cho thấy giá trị dưới 0.3 là dấu hiệu có vấn đề	Mục 6.1
Phân kỳ (divergences) của symplectic integrator	Xảy ra khi quỹ đạo số bay ra vô cùng ở vùng độ cong cao; dùng như chẩn đoán trực tiếp pathology hình học	Mục 6.2, Hình 29, 35
Phạm vi geometric ergodicity	HMC đơn giản đã geometric ergodic trên lớp phân phối lớn hơn RWM (Livingstone et al. 2016)	Mục 6

Paper không chạy benchmark thực nghiệm trên dữ liệu thật trong bản review này; các khẳng định thực nghiệm (ví dụ ngưỡng E-BFMI 0.3, khoảng chấp nhận 0.6-0.8) được paper trích dẫn là quan sát thực nghiệm đã biết trong các công trình trước (Betancourt và đồng nghiệp), không phải thí nghiệm mới chạy trong bài này.

Ví dụ tính tay (tự tính): xét mục tiêu Gaussian chuẩn 1 chiều $\pi(q) = \mathcal{N}(0, 1)$, $V(q) = q^2/2$, $\partial V/\partial q = q$. Lấy $p_0 \sim \mathcal{N}(0, 1)$, giả sử $q_0 = 1$, $p_0 = 0$, kinetic energy Euclidean-Gaussian $K = p^2/2$. Chạy leapfrog với $\epsilon = 0.5$ một bước:

$$p_{0.5} = p_0 - \tfrac{0.5}{2} \cdot q_0 = 0 - 0.25 \cdot 1 = -0.25,$$

$$q_1 = q_0 + 0.5 \cdot p_{0.5} = 1 + 0.5 \cdot (-0.25) = 0.875,$$

$$p_1 = p_{0.5} - \tfrac{0.5}{2} \cdot q_1 = -0.25 - 0.25 \cdot 0.875 = -0.46875.$$

Năng lượng ban đầu $H_0 = V(q_0) + K(p_0) = 0.5 + 0 = 0.5$. Năng lượng sau 1 bước $H_1 = V(q_1) + K(p_1) = 0.875^2/2 + (-0.46875)^2/2 \approx 0.3828 + 0.1099 = 0.4927$ (tự tính). Sai số $|H_1 - H_0| \approx 0.0073$, rất nhỏ so với $\epsilon = 0.5$, minh hoạ tính chất ”dao động quanh mức năng lượng đúng” của symplectic integrator (không trôi xa như integrator thường) đúng như Hình 27-28 mô tả định tính.

5. Phân tích / ablation

Vì đây là bài lý thuyết, ”ablation” ở đây là các so sánh định tính paper tự trình bày giữa các lựa chọn thiết kế:

- **RWM so với HMC (Mục 2.4 so với Mục 3):** RWM dùng đề xuất Gaussian không thông tin, xác suất chấp nhận suy biến theo cấp lũy thừa với D vì gần như mọi đề xuất rơi ra ngoài typical set hoặc phải co cực nhỏ. HMC dùng gradient qua động lượng, xác suất chấp nhận chỉ suy giảm nhẹ với D vì bản chất bảo toàn thể tích pha của symplectic integrator.

- **Euclidean so với Riemannian kinetic energy (Mục 4.2):** Euclidean chỉ sửa được tương quan toàn cục (một ma trận hiệp phương sai cố định); Riemannian sửa cục bộ theo vị trí, cần thiết cho các mô hình phân cấp (hierarchical model) nơi độ cong biến đổi mạnh theo vùng (ví dụ ”funnel” của Neal), nhưng tốn chi phí tính Hessian/metric mỗi bước.
- **Thời gian tích phân tĩnh so với động (NUTS) (Mục 4.3):** tĩnh chỉ tối ưu cho một mức năng lượng; đuôi nặng ($\beta < 2$) cần T lớn hơn ở vùng đuôi nên một T cố định sẽ làm chậm toàn chuỗi. NUTS thích ứng độ dài quỹ đạo theo từng điểm khởi đầu.
- **Lấy mẫu đều (uniform) so với lệch (biased) progressive sampling (Phụ lục A.3.2):** biến thể lệch ưu tiên lấy mẫu từ nửa quỹ đạo mới, tạo ”phản tương quan” (anti-correlation) giữa các mẫu liên tiếp của xích, có thể cải thiện hiệu năng nhưng paper không đưa ra số liệu định lượng cụ thể trong phần chính (chỉ chứng minh tính đúng đắn, phân phối dừng được bảo toàn).
- **Chẩn đoán kép (E-BFMI và divergences):** cả hai là điều kiện *cần* nhưng không *đủ* cho geometric ergodicity (Mục 6.3); paper nói rõ không có đảm bảo hình thức loại trừ hết các pathology tinh vi hơn.

6. Giới hạn

Do tác giả nêu:

- Không có lý thuyết hình thức toàn diện cho geometric ergodicity của HMC; các kết quả (Livingstone et al. 2016) chỉ là ”preliminary results” trên một lớp phân phối, chưa phải tổng quát.
- E-BFMI và divergence chỉ là điều kiện cần, không đủ (Mục 6.3): một phân phối có thể có pathology không bị hai chẩn đoán này phát hiện.
- SoftAbs metric cho Riemannian-HMC là ”heuristic regularization” (Mục 4.2.2), chưa có lý thuyết hình thức hoá đầy đủ.
- Higher-order symplectic integrator có thêm bậc tự do có thể làm hỏng hiệu năng nếu chọn sai, và việc tinh chỉnh các bậc tự do này vẫn là hướng nghiên cứu mở (Mục 5.3).
- Non-Gaussian kinetic energy ”không có nhiều lý thuyết hỗ trợ” và thực nghiệm cho thấy chúng thường kém hơn, nhưng paper không đưa số liệu cụ thể trong bài (chỉ dẫn trực giác qua CLT).

Nhận xét:

- Bài không có bảng số liệu định lượng nào từ thực nghiệm chạy trong bài; mọi ”kết quả thực nghiệm” (ngưỡng 0.3, khoảng 0.6-0.8) là trích dẫn công trình khác, không tự kiểm chứng lại trong văn bản này.
- Ví dụ giải tích 1D (họ π_β) minh họa trực giác tốt nhưng không tổng quát hoá trực tiếp sang không gian nhiều chiều với tương tác phức tạp giữa các toạ độ.
- Phần kỹ thuật NUTS đầy đủ (bao gồm cả xử lý khi quỹ đạo phân kỳ giữa cây nhị phân) nằm ở Phụ lục A.4-A.5 với độ phức tạp cao; bài báo tự nhận ”a more thorough discussion of the technical details is available in Appendix A” cho thấy phần thân bài chỉ là giản lược khái niệm, Stan thực tế dùng phiên bản phức tạp hơn (xử lý đa chiều khác biệt, điều kiện dừng theo nhiều hướng).

7. Liên hệ với các bài khác

- **math 06** (New insights and perspectives on the natural gradient method, 1412.1193): cùng dùng khái niệm Riemannian metric để sửa hình học không gian tham số; natural gradient dùng Fisher information metric cho bước tối ưu hoá, còn Riemannian-HMC (Mục 4.2.2 của bài này) dùng metric tương tự (có thể là Fisher metric) cho kinetic energy khi lấy mẫu.
- **math 11** (Stochastic Gradient Descent as Approximate Bayesian Inference, 1704.04289): bài tiếp theo trong lộ trình, dùng SGD như một xấp xỉ liên tục (continuous-time, Langevin-type dynamics tương tự Hamiltonian dynamics nhưng có nhiễu và ma sát) để suy luận Bayes gần đúng mà không cần tích phân Hamilton chính xác như HMC.

- **math 35** (A Tutorial on Thompson Sampling, 1707.02038): cùng nằm trong nhóm phương pháp Bayes thực hành (ở đây là bandit), tuy ứng dụng khác HMC nhưng cùng triết lý ”suy luận hậu nghiệm làm nền cho quyết định”, đáng đọc nếu quan tâm thêm về suy luận Bayes ứng dụng.

8. Gợi ý áp dụng

HMC thích hợp để khảo sát hậu nghiệm khi có thể tính gradient của log mật độ. Mỗi đề xuất cần nhiều lần tính gradient, vì vậy hiệu quả thực tế phụ thuộc chi phí mỗi lần tính và cách chọn tham số mô phỏng.

9. Thí nghiệm gợi ý

1. **So sánh RWM và HMC trên Gaussian 2 chiều tương quan cao:** tạo $\pi(q) = \mathcal{N}(0, \Sigma)$ với Σ có tương quan 0.95 giữa 2 chiều; chạy RWM (đề xuất Gaussian đẳng hướng) và HMC leapfrog thủ công (vài chục bước, $L = 10$-20, ϵ nhỏ) bằng NumPy/PyTorch trên CPU. Đo effective sample size (ESS) trên cùng số lần tính hàm mục tiêu. Kỳ vọng: HMC có ESS/evaluation cao hơn RWM rõ rệt khi tương quan lớn, minh họa trực tiếp Mục 2.4 và 3.
2. **Đo xác suất chấp nhận MH theo step size ϵ:** với cùng target Gaussian chuẩn, chạy leapfrog $L = 20$ bước với nhiều $\epsilon \in \{0.01, 0.05, 0.1, 0.3, 0.5, 1.0\}$, vẽ xác suất chấp nhận trung bình theo ϵ. Kỳ vọng: xác suất chấp nhận giảm đơn điệu khi ϵ tăng; xác định ϵ cho chấp nhận quanh 0.65 để kiểm chứng khuyến nghị 0.6-0.8 ở Mục 5.3.
3. **Tạo ”Neal’s funnel” (phân phối phân cấp điển hình) và quan sát divergence:** $v \sim \mathcal{N}(0, 3^2)$, $q \mid v \sim \mathcal{N}(0, e^v)$ (tham số hoá ”centered”); chạy HMC/leapfrog đơn giản, log số bước phân kỳ (năng lượng bùng nổ) khi v nhỏ (vùng cổ hẹp của funnel). Kỳ vọng: số divergence tăng mạnh ở vùng cổ, minh hoạ trực tiếp Mục 6.2 và gợi ý kỹ thuật ”non-centered parameterization” paper nhắc tới.
4. **Tính E-BFMI tự chế trên một xích HMC toy:** log năng lượng $E_n = H(q_n, p_n)$ qua các bước, tính $\widehat{\text{E-BFMI}} = \sum(E_n - E_{n-1})^2 / \sum(E_n - \bar{E})^2$ theo công thức Mục 6.1; thử với kinetic energy ”đúng” (mass matrix = hiệp phương sai target) và ”sai” (mass matrix = identity khi target có tương quan mạnh). Kỳ vọng: cấu hình sai cho E-BFMI thấp hơn (gần hoặc dưới 0.3), khớp ngưỡng paper nêu.
5. **Minh hoạ ba giai đoạn của xích Markov (Hình 7):** chạy một xích RWM từ điểm khởi tạo xa mode, vẽ $|\hat{f}_N - \mathbb{E}_\pi[f]|$ theo số bước; quan sát pha hội tụ ban đầu (bias lớn), pha ”sojourn” đầu tiên (bias giảm nhanh), và pha tinh chỉnh chậm. Kỳ vọng: đường cong có 3 đoạn dốc khác nhau rõ rệt, minh họa trực tiếp Mục 2.2.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Kỳ vọng mục tiêu	$\mathbb{E}_\pi[f] = \int_Q dq\, \pi(q) f(q)$
Hiệu quả xích Markov	$\text{ESS} = N / \left(1 + 2\sum_{l \geq 1} \rho_l\right)$
Metropolis-Hastings	$a(q' \mid q) = \min\left(1, \frac{Q(q \mid q')\pi(q')}{Q(q' \mid q)\pi(q)}\right)$
Phân phối chính tắc	$\pi(q,p) = e^{-H(q,p)}$, $H = K(p,q) + V(q)$
Phương trình Hamilton	$\dot{q} = \partial K / \partial p$, $\dot{p} = -\partial K / \partial q - \partial V / \partial q$
Kinetic energy Euclidean-Gaussian	$K = \frac{1}{2} p^\intercal M^{-1} p + \log\lvert M\rvert$, $M^{-1} \approx \text{Cov}_\pi[q]$
Kinetic energy Riemannian-Gaussian	$K = \frac{1}{2} p^\intercal \Sigma^{-1}(q) p + \frac{1}{2}\log\lvert\Sigma(q)\rvert$
Thời gian tích phân tối ưu (1D, $\pi_\beta \propto e^{-\lvert q\rvert^\beta}$)	$T_{\text{optimal}} \propto H^{(2-\beta)/(2\beta)}$
Leapfrog	$p_{+\frac{\epsilon}{2}} = p - \frac{\epsilon}{2}\nabla V(q)$; $q' = q + \epsilon p_{+\frac{\epsilon}{2}}$; $p' = p_{+\frac{\epsilon}{2}} - \frac{\epsilon}{2}\nabla V(q')$
MH sửa lỗi integrator (momentum flip)	$a = \min\left(1, \exp(-H(q_L, -p_L) + H(q_0, p_0))\right)$
Xác suất chấp nhận tối ưu khuyến nghị	khoảng 0.6 đến 0.8

Khái niệm	Công thức / ý nghĩa
E-BFMI	$\widehat{\text{E-BFMI}} = \sum_n (E_n - E_{n-1})^2 / \sum_n (E_n - \bar{E})^2$; ngưỡng cảnh báo thực nghiệm < 0.3

11. Kết luận ngắn

Bài này xây dựng trực giác hình học cho Hamiltonian Monte Carlo mà không cần vi phân hình học chính thức: không gian nhiều chiều khiến khối lượng xác suất tập trung vào một ”typical set” mỏng và xa mode; các phương pháp ”đoán-kiểm tra” như Random Walk Metropolis thất bại vì không khai thác được hình học này, còn HMC tận dụng gradient qua một hệ động lực học Hamiltonian (động lượng phụ trợ, bảo toàn thể tích pha) để tạo quỹ đạo bám theo typical set. Phần còn lại của bài là cách chỉnh các bậc tự do thực tế (kinetic energy, thời gian tích phân/NUTS, symplectic integrator) và cách chẩn đoán khi các giả định hình học bị vi phạm (E-BFMI, divergent transitions). Giá trị chính cho người đọc không chuyên Bayes là bộ khung khái niệm (typical set, phase space, năng lượng) hữu ích khi đọc các bài liên quan về suy luận Bayes gần đúng (math 11) và các phương pháp tối ưu dùng metric hình học (math 06).

Nguồn gốc: https://arxiv.org/abs/1701.02434

Bài 11

Stochastic Gradient Descent as Approximate Bayesian Inference (2017)

Tác giả: Stephan Mandt (Data Science Institute, Columbia University), Matthew D. Hoffman (Adobe Research), David M. Blei (Columbia University).
Venue / arXiv: arXiv:1704.04289 (v2, 19/01/2018); paper ghi rõ venue là Journal of Machine Learning Research (JMLR) 18 (2017) 1-35, nộp 4/2017, sửa 10/2017, xuất bản 12/2017.
Nguồn: https://arxiv.org/abs/1704.04289
Chủ đề: xem SGD với learning rate hằng số (constant SGD) như một quá trình ngẫu nhiên liên tục có phân phối dừng Gaussian, từ đó chỉnh learning rate/minibatch/preconditioner để phân phối dừng này xấp xỉ hậu nghiệm Bayes; phân tích thêm momentum, SGLD, SGFS, và Polyak iterate averaging như một bộ lấy mẫu MCMC.
Vị trí trong lộ trình: đọc sau math 10 (A Conceptual Introduction to Hamiltonian Monte Carlo), vì cả hai cùng dùng ngôn ngữ ”động lực học ngẫu nhiên” (Hamiltonian/Langevin) để suy luận Bayes gần đúng, nhưng bài 10 tích phân chính xác một hệ Hamiltonian còn bài này xấp xỉ SGD rời rạc bằng một phương trình vi phân ngẫu nhiên (SDE) liên tục; đọc trước math 12 (Computational Optimal Transport) vì sau khi nắm các công cụ suy luận xấp xỉ (sampling, biến phân), phần tiếp theo của lộ trình chuyển sang đo khoảng cách giữa phân phối (OT, Wasserstein). Cũng liên quan math 08 (Variational Inference review) vì bài này tự nhận là ”một dạng variational inference” theo nghĩa rộng.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (bản arXiv); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc.

1. Bối cảnh

Với lịch learning rate giảm dần và các điều kiện thích hợp, SGD có thể hội tụ tới một điểm tối ưu cục bộ của hàm mất mát. Khi learning rate được giữ hằng số, các bước cập nhật tiếp tục dao động quanh vùng lân cận nghiệm. Bài báo phân tích hành vi này qua một quá trình Markov có phân phối dừng và xét cách chọn learning rate ϵ, kích thước minibatch S và ma trận tiền điều kiện H để phân phối đó xấp xỉ hậu nghiệm Bayes $p(\theta \mid x) = \exp\{\log p(\theta, x) - \log p(x)\}$ (công thức (1) của bài).

Đây khác với các phương pháp **stochastic gradient MCMC (SG-MCMC)** đã có trước đó như SGLD (Stochastic Gradient Langevin Dynamics, Welling và Teh 2011), SG-HMC (Chen et al. 2014), SG thermostats (Ding et al. 2014), SGFS (Stochastic Gradient Fisher Scoring, Ahn et al. 2012): các phương pháp này cố tình thêm nhiễu để **tiệm cận chính xác** hậu nghiệm (nhưng mixing chậm khi learning rate phải giảm về 0). Bài báo đi theo hướng ngược lại: chấp nhận một xấp xỉ **không chính xác tuyệt đối** nhưng **mixing nhanh hơn**, bằng cách chạy SGD thông thường (không cần thêm nhiễu đặc biệt) với learning rate hằng số đã được tinh chỉnh.

Công cụ toán học chính: xấp xỉ quá trình rời rạc của SGD bằng một **phương trình vi phân ngẫu nhiên (SDE)** liên tục thời gian, cụ thể là **quá trình Ornstein-Uhlenbeck (OU)** đa biến, một quá trình ngẫu nhiên có nghiệm dừng Gaussian giải tích được. Công cụ thứ hai: **variational inference** theo nghĩa tối thiểu hoá Kullback-Leibler (KL) divergence giữa phân phối dừng (vai trò q) và hậu nghiệm (vai trò f hoặc p); khác biệt so với VI thông thường là ở đây ta không tối ưu tham số của q trực tiếp, mà tối ưu **siêu tham số của thuật toán tối ưu hoá** (learning rate, preconditioner) để định hình q.

2. Ý tưởng chính

Với hàm mất mát $L(\theta) = \frac{1}{N}\sum_{n=1}^{N} \ell_n(\theta)$ (ví dụ $\ell_n(\theta) = -\log p(x_n \mid \theta) - \frac{1}{N}\log p(\theta)$, công thức (3), chính là MAP loss), SGD cập nhật $\theta(t+1) = \theta(t) - \epsilon\, \hat{g}_S(\theta(t))$ với gradient ước lượng trên minibatch S mẫu. Bốn giả định (Assumption 1-4, Mục 3.2) biến quá trình rời rạc này thành một SDE:

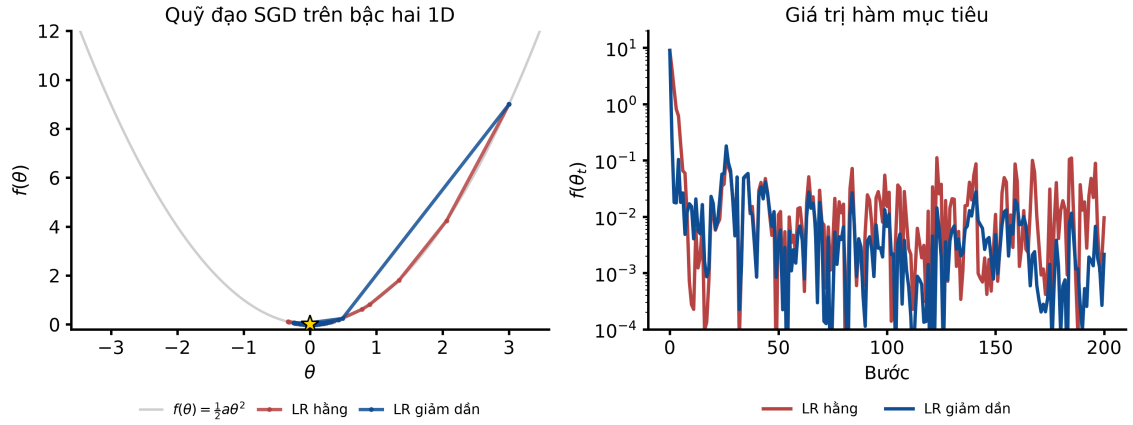
1. **A1:** theo định lý giới hạn trung tâm, nhiễu gradient minibatch là Gaussian: $\hat{g}_S(\theta) \approx g(\theta) + \frac{1}{\sqrt{S}}\Delta g(\theta)$, $\Delta g(\theta) \sim \mathcal{N}(0, C(\theta))$.
2. **A2:** $C(\theta) \approx C = BB^{\mathsf{T}}$ (hằng số, không đổi theo θ trong vùng quanh optimum).
3. **A3:** xấp xỉ sai phân $\Delta\theta(t) = \theta(t+1) - \theta(t)$ bằng SDE liên tục $d\theta = -g(\theta)dt + \frac{\epsilon}{\sqrt{S}} B\, dW(t)$.
4. **A4:** gần optimum, loss là hàm bậc hai $L(\theta) = \frac{1}{2}\theta^{\mathsf{T}} A\theta$ (A là Hessian tại optimum, giả sử optimum ở $\theta = 0$).

Bốn giả định này biến SGD thành một **quá trình Ornstein-Uhlenbeck đa biến**: $d\theta(t) = -A\theta(t)\,dt + \frac{1}{\sqrt{S}} B\, dW(t)$ (công thức (11)), có phân phối dừng Gaussian $q(\theta) \propto \exp(-\frac{1}{2}\theta^{\mathsf{T}}\Sigma^{-1}\theta)$ với hiệp phương sai Σ nghiệm của phương trình Lyapunov $\Sigma A + A\Sigma = \frac{\epsilon}{S} BB^{\mathsf{T}}$ (công thức (13)). Ý tưởng chính: Σ **tỉ lệ thuận với** ϵ **và tỉ lệ nghịch với** A**,** S, nên ta có thể "vặn" ϵ, S, hoặc thêm ma trận tiền điều kiện H để Σ khớp với hiệp phương sai hậu nghiệm thật $N^{-1}A^{-1}$ (xấp xỉ Laplace/Bernstein-von Mises của hậu nghiệm, công thức (14)). Việc khớp này được thực hiện bằng cách **tối thiểu hoá KL**$(q\|f)$ giải tích (vì cả hai đều Gaussian).

3. Phương pháp chi tiết

3.1 Learning rate và preconditioner tối ưu (Mục 4.1)

Định lý 1 (constant SGD, công thức (15)): learning rate hằng số tối thiểu hoá $\mathrm{KL}(q\|f)$ là



SGD trên bậc hai 1D: learning rate giảm dần hội tụ về cực tiểu, còn learning rate hằng dao động quanh nghiệm (phân phối dừng). (minh hoạ, tự tính)

$$\epsilon^* = \frac{2\,S\,D}{N\,\mathrm{Tr}(BB^{\mathsf{T}})},$$

với D là số chiều của θ. Chứng minh (paper, Mục 4.1): $\mathrm{KL}(q\|f) = \frac{c\,\epsilon N}{2S}\mathrm{Tr}(BB^{\mathsf{T}}) - D\log(\epsilon/S)$ (công thức (16)); lấy đạo hàm theo ϵ/S và giải cho 0 ra công thức (15). Diễn giải: learning rate nên **tỉ lệ nghịch với độ lớn trung bình nhiễu gradient** ($\mathrm{Tr}(BB^{\mathsf{T}})$) và **tỉ lệ thuận với** S/N.

Định lý 2 (preconditioned constant SGD, công thức (17)): nếu dùng cập nhật $\theta_{t+1} = \theta_t - H\hat{g}_S(\theta_t)$ với ma trận H xác định dương thay cho ϵ vô hướng, preconditioner tối ưu là

$$H^* = \frac{2S}{N}(BB^\mathsf{T})^{-1}.$$

Hệ quả 3: nếu hạn chế H chéo (diagonal, cần thiết khi D lớn vì H đầy đủ tốn $O(D^2)$), preconditioner chéo tối ưu là $H^*_{kk} = \dfrac{2S}{N\,(BB^\mathsf{T})_{kk}}$.

Hệ quả 4: nếu dùng đúng H^* đầy đủ (công thức (17)), phân phối dừng trở thành **chính xác** hậu nghiệm ($\Sigma = A^{-1}/N$), không chỉ là xấp xỉ KL-tối ưu.

Để áp dụng thực tế cần ước lượng $C = BB^\mathsf{T}$ trực tuyến (online). Paper dùng ước lượng dựa trên hiệu giữa gradient của 1 mẫu và gradient minibatch đầy đủ, cập nhật đệ quy (công thức (21)):

$$C_t = (1-\kappa_t)\,C_{t-1} + \kappa_t\,(\hat{g}_{1,t} - \hat{g}_{S,t})(\hat{g}_{1,t} - \hat{g}_{S,t})^\mathsf{T},$$

với κ_t giảm dần ($\kappa_t \sim 1/t$). Chi phí: $O(D)$ cho xấp xỉ chéo (tương đương AdaGrad/RMSProp), $O(D^2)$ cho preconditioner đầy đủ (paper nhận xét "impractical in many real-word setups" cho D lớn).

3.2 Variational EM cho siêu tham số (Mục 4.2)

Với mô hình có siêu tham số λ (ví dụ hệ số regularization), paper đề xuất chạy **đồng thời hai learning rate**: $\theta_{t+1} = \theta_t - \epsilon^*\nabla_\theta L(\theta_t, \lambda_t)$ (hằng số, để θ lấy mẫu từ hậu nghiệm xấp xỉ) và $\lambda_{t+1} = \lambda_t - \rho_t \nabla_\lambda L(\theta_t, \lambda_t)$ (giảm dần, để λ hội tụ về một điểm tối ưu), công thức (22). Đây là một dạng **variational EM (VEM)**: bước E (lấy mẫu θ từ phân phối dừng của constant SGD, đóng vai trò $q(\theta)$) và bước M (tối ưu λ theo gradient của log-joint kỳ vọng) diễn ra đan xen trong cùng một vòng lặp SGD kép, không cần cross-validation.

3.3 SGD với Momentum (Mục 4.3)

Thêm biến động lượng v: $v(t+1) = (1-\mu)v(t) - \epsilon\,\hat{g}_S(\theta(t))$, $\theta(t+1) = \theta(t) + v(t+1)$, với hệ số hãm (damping) $\mu \in [0,1]$ ($\mu = 1$ là SGD thường, không có "quán tính"). Dùng cùng 4 giả định ở trên cho hệ 2 biến (θ, v), giải hệ phương trình mô-men bậc 2 (chi tiết Phụ lục D), paper thu được quan hệ năng lượng kiểu định lý "fluctuation-dissipation" (công thức (24)):

$$\mathbb{E}[vv^\mathsf{T}] = \tfrac{\epsilon}{2}\mathbb{E}[\theta\theta^\mathsf{T}]A + \tfrac{\epsilon}{2}A\,\mathbb{E}[\theta\theta^\mathsf{T}], \qquad \mu\,\mathbb{E}[vv^\mathsf{T}] = \tfrac{\epsilon^2}{2S}BB^\mathsf{T}.$$

Kết hợp lại cho (công thức (25)):

$$\Sigma A + A\Sigma = \frac{\epsilon}{\mu S}BB^\mathsf{T},$$

giống hệt công thức (13) của SGD không momentum, chỉ thay ϵ/S bằng $\epsilon/(\mu S)$. Hệ quả quan trọng: **momentum không đổi hình dạng của phân phối dừng, chỉ đổi tỉ lệ (scale)** theo hệ số $1/\mu$; chỉ tổ hợp $\epsilon/(\mu S)$ mới ảnh hưởng đến KL divergence, nên "không có một learning rate tối ưu duy nhất" khi có momentum, mà có cả một họ (ϵ, μ, S) cho cùng kết quả xấp xỉ (nhưng tốc độ hội tụ động lực học thì khác nhau).

3.4 SGLD và SGFS với learning rate không tiệm cận 0 (Mục 5)

SGLD (Welling-Teh 2011) thêm nhiễu Wiener $V(t)$ vào update: $\theta_{t+1} = \theta_t - \frac{\epsilon}{2}N\hat{\nabla}L(\theta_t) + \sqrt{\epsilon}\,V(t)$. Phân tích OU cho phương trình dừng: $\frac{1}{2}\epsilon N(A\Sigma + \Sigma A) = I + \frac{\epsilon N}{2S}BB^\mathsf{T}$. Khi $\epsilon \to 0$: $\Sigma^{-1} = NA$ đúng hậu nghiệm. Nhưng với ϵ hữu hạn (như dùng trong thực tế), các hệ số hiệu chỉnh dương khiến **SGLD hệ thống hoá đánh giá quá cao (overestimate) hiệp phương sai hậu nghiệm**, hệ quả của learning rate không tiệm cận 0.

SGFS (Ahn et al. 2012) dùng update $\theta_{t+1} = \theta_t - H\hat{g}(\theta_t) + \sqrt{H}EW(t)$ với H là preconditioner và EE^T là hiệp phương sai nhiễu nhân tạo. **Định lý 5 (công thức (27)):** $H^* = \frac{2}{N}(BB^\mathsf{T} + EE^\mathsf{T})^{-1}$ tối thiểu hoá KL **và đưa KL về đúng 0** (nghĩa là, nếu biết chính xác BB^T, SGFS với H^* tạo mẫu đúng từ hậu nghiệm, không chỉ là xấp xỉ). Paper cũng đưa ra bản chéo/vô hướng (Hệ quả 6) và một thủ thuật ổn định số: chọn nhiễu nhân tạo E sao cho $H_{kk} \equiv h_{\max}$ cho mọi k (tiêm nhiễu Gaussian vào các chiều có gradient-noise quá nhỏ để tránh learning rate hiệu dụng quá lớn ở chiều đó); bài báo nhấn mạnh thủ thuật này **ngược lại** khuyến nghị gốc của Ahn et al. (2012) là chọn B tỉ lệ với E.

3.5 Polyak iterate averaging như một bộ lấy mẫu (Mục 6)

Tối ưu hoá: trung bình các iterate $\hat{\mu}_{t+1} = \frac{t}{t+1}\hat{\mu}_t + \frac{1}{t+1}\theta_{t+1}$ (công thức (28)) đạt tốc độ hội tụ **tốt nhất có thể trong số các thuật toán chỉ dùng stochastic-gradient oracle** (kết quả gốc của Polyak và Juditsky 1992); paper chứng minh lại ngắn hơn bằng công cụ OU, cho hiệp phương sai của ước lượng $D \approx \frac{1}{TS}A^{-1}BB^\mathsf{T}(A^{-1})^\mathsf{T}$ (công thức được paper rút gọn), **không phụ thuộc learning rate hay preconditioner** (vì preconditioning thay đổi cả phân phối dừng và ma trận tự tương quan theo cách triệt tiêu lẫn nhau).

Lấy mẫu: với cửa sổ trung bình $T = N/S$ (một epoch), dưới giả định bổ sung **A5** (định lý Bernstein-von Mises áp dụng với N lớn, và dữ liệu sinh đúng từ model với $\theta^* = 0$, nên $A = BB^\mathsf{T}$ chính là ma trận thông tin Fisher), paper suy ra thuật toán **Iterate Averaging Stochastic Gradient sampler (IASG, Algorithm 1 trong paper):** chạy SGD learning rate hằng, cứ mỗi $T = N/S$ bước thì tính trung bình T iterate gần nhất thành một mẫu hậu nghiệm. Kết quả lý thuyết nổi bật: **IASG cần đúng khoảng N lượt gọi gradient (một lượt qua hết dữ liệu) để sinh một mẫu hậu nghiệm xấp xỉ "hiệu dụng độc lập"**, và vì iterate averaging đạt chặn Cramér-Rao tối ưu, **không thuật toán SG-MCMC "hộp đen" nào có thể sinh mẫu hậu nghiệm nhanh hơn $O(ND)$ (thời gian tuyến tính theo kích thước dữ liệu nhân số chiều)**, trừ khi khai thác thêm thông tin về nguồn nhiễu (như SVRG, SAGA, được paper dẫn ra ở chú thích).

4. Thí nghiệm và kết quả chính

Đây là bài có **thí nghiệm thực nghiệm thật** (không phải thuần lý thuyết), chia 3 phần (Mục 7):

7.1 Kiểm tra hiệp phương sai dừng trên dữ liệu thật. Ba tập dữ liệu UCI: Wine Quality ($N = 4\,898$, 11 đặc trưng, hồi quy tuyến tính), Protein Tertiary Structure ($N = 45\,730$, 8 đặc trưng, hồi quy tuyến tính), Skin Segmentation ($N = 245\,057$, 3 đặc trưng, hồi quy logistic). Minibatch $S = 100, 100, 10\,000$ theo thứ tự; regularizer bậc 2 bằng 1; learning rate theo công thức (15). Paper báo cáo hình ảnh (Hình 1-2) cho thấy hiệp phương sai thực nghiệm khớp tốt với dự đoán từ OU; sai khác đáng kể nhất ở Wine (do tập nhỏ hơn nên hậu nghiệm rộng hơn, cần learning rate lớn hơn, hiệu ứng rời rạc hoá lớn hơn).

Bảng 1 (KL divergence giữa hậu nghiệm xấp xỉ bằng Laplace và phân phối dừng của từng thuật toán, cột là 3 tập dữ liệu Wine/Skin/Protein):

Phương pháp	Wine	Skin	Protein
constant SGD	18.7	0.471	1000.9
constant SGD-d (chéo)	14.0	0.921	678.4
constant SGD-f (đầy đủ)	**0.7**	**0.005**	**1.8**
SGLD (Welling-Teh)	2.9	0.905	4.5
SGFS-d	12.8	0.864	597.4
SGFS-f	0.8	**0.005**	1.3
BBVI	44.7	5.74	478.1

Nhận xét của paper: preconditioner **đầy đủ** (SGD-f, SGFS-f) áp sát hậu nghiệm tốt nhất (KL nhỏ nhất); phiên bản chéo (hay dùng trong thực hành vì rẻ) kém hơn hẳn; **SGD hằng số chỉnh đúng có thể cạnh tranh với các**

thuật toán MCMC chuyên dụng (SGLD, SGFS) và tốt hơn BBVI (black-box VI dùng reparameterization gradient) trên cả 3 tập; paper kết luận yếu tố quan trọng nhất không phải nhiễu nhân tạo thêm vào mà là **chất lượng xấp xỉ ma trận tiền điều kiện**.

7.2 Iterate averaging như MCMC. Dữ liệu tổng hợp đúng theo model: $N = 10\,000$ covariates $D = 10$ chiều từ Gaussian, prior precision $\lambda = 1$. Thí nghiệm xác nhận (Hình 3): chạy constant SGD $\epsilon = 0.005$, $10 \cdot D \cdot N = 10^6$ bước, $S = 1$, cửa sổ trung bình $N/S = 10^4$; hiệp phương sai của iterate đã trung bình hoá **giống hệt** hiệp phương sai hậu nghiệm thật. So sánh tốc độ hội tụ (Hình 4-5) giữa IASG, SGLD, và NUTS (trên cùng bài toán hồi quy tuyến tính): $S = 10$, cửa sổ $N/S = 1000$, $\epsilon_{\text{IASG}} = 0.003$; SGLD dùng lịch Robbins-Monro $\epsilon_t = \epsilon_0/\sqrt{1000 + t}$ với $\epsilon_0 = 10^{-3}$; NUTS tự động chỉnh bước và dùng gradient đầy đủ (không stochastic). Kết quả: khi khởi tạo ngẫu nhiên (không phải tại MAP), **IASG hội tụ nhanh hơn và ổn định hơn SGLD và NUTS** trong việc ước lượng phương sai biên lớn nhất/nhỏ nhất của hậu nghiệm; SGLD có độ lệch chuẩn ước lượng lớn hơn hẳn do learning rate giảm dần làm chậm cân bằng ở các bước sau; khi khởi tạo tại MAP (loại bỏ pha hội tụ ban đầu) cả 3 thuật toán đều hoạt động hợp lý, nhưng IASG vẫn ổn định nhanh nhất, tuy có chút lệch (biased) ở phương sai nhỏ nhất.

7.3 Tối ưu siêu tham số. Mô hình hồi quy logistic đa lớp (softmax) với prior Gaussian, hệ số regularization λ (công thức (35)). Dữ liệu thật: **MNIST** (60.000 huấn luyện, 10.000 kiểm tra, 784 đặc trưng) và **Covertype** (500.000 huấn luyện, 81.012 kiểm tra, 54 đặc trưng). Hình 6: vẽ validation loss theo λ (đường cong liên tục), đánh dấu điểm λ mà VEM dựa trên constant-SGD (Mục 4.2) chọn, so với điểm λ mà BBVI chọn. Kết quả: hai phương pháp chọn λ **gần giống nhau** và đều gần đáy đường cong validation loss; paper kết luận constant SGD-VEM là **giải pháp rẻ hơn cross-validation** để chọn siêu tham số trong mô hình xác suất phức tạp.

5. Phân tích / ablation

- **Preconditioner đầy đủ so với chéo (Bảng 1):** chéo luôn kém hơn đầy đủ một khoảng đáng kể (ví dụ Protein: 678.4 so với 1.8 cho SGD), xác nhận trực giác lý thuyết ở Hệ quả 3/4: chỉ preconditioner đầy đủ $\propto (BB^{\mathsf{T}})^{-1}$ mới cho mẫu đúng hậu nghiệm; bản chéo là một xấp xỉ có hệ thống (biased).
- **SGD hằng số so với SGLD/SGFS/BBVI:** với preconditioning đúng, constant SGD **không hề kém hơn** các thuật toán MCMC chuyên dụng, dù không chủ đích ”được thiết kế để là một sampler”. Điểm nhấn của paper: sự khác biệt chính giữa các phương pháp không phải là ”có thêm nhiễu Bayes hay không” mà là **chất lượng xấp xỉ ma trận hiệp phương sai nhiễu gradient**.
- **Momentum:** không đổi hình dạng phân phối dừng, chỉ đổi tỉ lệ theo $1/\mu$; về mặt lấy mẫu Bayes, dùng momentum tương đương dùng một constant SGD ”hiệu dụng” với learning rate ϵ/μ, nhưng paper trích Sutskever et al. (2013) rằng μ nhỏ hơn có thể giúp **ổn định số học** của động lực học rời rạc hơn với cùng ϵ/μ.
- **Khởi tạo ngẫu nhiên so với tại MAP (Hình 4-5):** IASG robust hơn SGLD/NUTS trong trường hợp khởi tạo ngẫu nhiên (gần với thực tế hơn), cho thấy ưu điểm thực dụng của một sampler dùng learning rate hằng không cần ”ủ nhiệt” (annealing) phức tạp.
- **Biên hiệu quả lý thuyết:** kết quả ”không SG-MCMC nào nhanh hơn $O(ND)$” (Mục 6.2 cuối) là một dạng ”ablation lý thuyết” ngược: paper tự chỉ ra giới hạn trên của chính lớp phương pháp mình đang phát triển, dựa trên optimality đã biết của Polyak averaging.

6. Giới hạn

Do tác giả nêu:

- Giả định A1-A4 chỉ hợp lý trong pha ”đã hội tụ gần optimum, nhiễu nhỏ so với gradient trung bình” (Mục 3.2, ”Discussion of Assumptions”); trong pha tìm kiếm ban đầu (search phase) các giả định bị vi phạm và paper ”không nói được gì chung” về hành vi thuật toán ở pha đó.

- Nếu hiệp phương sai nhiễu gradient C không có hạng đầy đủ (full rank), Σ cũng không đầy hạng, làm toàn bộ phân tích Mục 4 phức tạp hơn; paper đề xuất thêm nhiễu Gaussian đẳng hướng nhỏ làm biện pháp khắc phục nhưng không phân tích sâu tác động của biện pháp này.
- KL ngược $KL(f\|q)$ (Phụ lục C) chỉ giải được dạng đóng khi A và Σ giao hoán, điều **hiếm khi đúng trong thực tế**; do đó paper chỉ dùng $KL(q\|f)$ trong toàn bộ phần chính.
- Kết quả "chặn dưới $O(ND)$" dựa trên Giả định A5 (Bernstein-von Mises áp dụng, N lớn, và mô hình đúng/well-specified: dữ liệu sinh thật từ model với $\theta^* = 0$ nên $A = BB^\mathsf{T}$); đây là "các giả định lạc quan" (optimistic assumptions), và bài báo lưu ý "có thể dễ dàng xây dựng tình huống mà các giả định 1-5 bị vi phạm", chỉ lập luận (không chứng minh) rằng các tình huống vi phạm không dễ hơn.
- Yêu cầu dataset N đủ lớn so với số điều kiện (condition number) $\lambda_{\text{max}}/\lambda_{\text{min}}$ của Hessian A để IASG hoạt động tốt (Mục 6.2, "Approximation error"); nếu không, cần preconditioner $H \approx A^{-1}$ thay learning rate vô hướng.

Nhận xét:

- Thí nghiệm 7.1/7.3 đều dùng mô hình tuyến tính/log-linear (hồi quy tuyến tính, logistic, softmax); paper **không thử nghiệm trên mạng nơ-ron sâu phi tuyến** nào trong văn bản chính, nên tính tổng quát của phương pháp cho các loss không lồi/không xấp xỉ bậc hai tốt (như transformer) chỉ là suy luận từ lý thuyết, chưa được paper kiểm chứng thực nghiệm.
- Việc xem A (Hessian) là hằng số quanh một điểm tối ưu (Assumption 4) rất mạnh cho mạng sâu hiện đại, nơi loss landscape có nhiều điểm yên ngựa/minimum phẳng và Hessian biến đổi mạnh theo vị trí (liên hệ math 18, Shattered Gradients, và math 34).
- Bảng 1 không có cột ghi thời gian chạy thực tế (wall-clock) cho mỗi phương pháp, nên khó so sánh trực tiếp "chi phí tính toán thực" giữa SGD-f (preconditioner $O(D^2)$) và SGD-d (chéo, $O(D)$) ngoài con số KL.

7. Liên hệ với các bài khác

- **math 10** (A Conceptual Introduction to Hamiltonian Monte Carlo, đã viết, `10_2017_hamiltonian_monte_carlo.md`): cùng chủ đề "dùng động lực học liên tục để lấy mẫu Bayes"; HMC tích phân Hamilton chính xác (symplectic integrator), bài này xấp xỉ SGD bằng SDE Ornstein-Uhlenbeck; phần SGD-with-momentum (Mục 3.3 ở trên) có cấu trúc năng lượng động/tĩnh rất giống Mục 3.2 của bài 10 (động năng $\mathbb{E}[vv^\mathsf{T}]$, "nhiệt độ" BB^T).
- **math 08** (Variational Inference: A Review for Statisticians, 1601.00670): bài này tự gọi phương pháp của mình là một dạng VI (tối thiểu hoá $\text{KL}(q\|f)$), nhưng khác VI cổ điển ở chỗ không tối ưu trực tiếp tham số của q mà tối ưu siêu tham số thuật toán; nên đọc math 08 trước để hiểu rõ khung KL-divergence chuẩn.
- **math 06** (New insights and perspectives on the natural gradient method, 1412.1193): cả hai bài đều liên quan đến vai trò ma trận Fisher information/preconditioner; $H^* = (2S/N)(BB^\mathsf{T})^{-1}$ ở Định lý 2 có cùng vai trò với Fisher information trong natural gradient (khi $A = BB^\mathsf{T}$, tức Giả định A5).
- Trong thư mục mới cùng mã 'math': **opt 06** (Adam, 1412.6980) và **opt 08** (On the Convergence of Adam and Beyond, 1904.09237) nên đọc cùng, vì công thức $\epsilon^* \propto 1/\text{Tr}(BB^\mathsf{T})$ ở Định lý 1 chính là cơ sở lý thuyết (từ góc nhìn Bayes) cho trực giác "learning rate nên nghịch với độ lớn nhiễu gradient" mà Adam/RMSProp dùng theo hướng thực dụng (empirical).

8. Gợi ý áp dụng

Cách nhìn SGD như một quá trình ngẫu nhiên giúp phân tích mối liên hệ giữa nhiễu gradient và phân phối tham số. Các kết luận định lượng phụ thuộc giả thiết xấp xỉ trong bài, nên không thể mặc nhiên xem mọi bộ tối ưu hoặc lịch learning rate là bộ lấy mẫu hậu nghiệm.

9. Thí nghiệm gợi ý

1. **Kiểm chứng Định lý 1 trên hồi quy logistic nhỏ:** tạo dữ liệu tổng hợp 2 lớp, $D = 5$-10 chiều, chạy constant SGD với ϵ tính theo công thức (15) (ước lượng BB^T bằng công thức (21)); so hiệp phương sai thực nghiệm của các iterate với hiệp phương sai Laplace của hậu nghiệm thật (tính bằng Hessian nghịch đảo). Kỳ vọng: hai hiệp phương sai gần nhau, xác nhận Hình 1-2 của paper trên quy mô nhỏ hơn.
2. **So sánh preconditioner chéo so với đầy đủ (tái tạo thu nhỏ Bảng 1):** cùng bài toán trên, so KL divergence giữa hậu nghiệm và phân phối dừng của (a) constant SGD không preconditioner, (b) preconditioner chéo (Hệ quả 3), (c) preconditioner đầy đủ (Định lý 2). Kỳ vọng: (c) < (b) < (a) về KL, khớp xu hướng Bảng 1.
3. **Đo ảnh hưởng của batch size tới "nhiệt độ" lấy mẫu:** cố định ϵ, chạy constant SGD với $S \in \{1, 10, 100\}$ trên cùng mô hình tuyến tính nhỏ; đo phương sai thực nghiệm của iterate ở trạng thái dừng. Kỳ vọng: phương sai tỉ lệ nghịch với S (từ $\Sigma \propto \epsilon/S$), minh hoạ trực tiếp cho gợi ý Mục 8.1 về batch size nhỏ trên GPU hạn chế.
4. **Tái tạo thu nhỏ IASG so với SGLD (Hình 4-5):** trên hồi quy tuyến tính $D = 5$, dữ liệu sinh đúng từ model (Assumption 5), so tốc độ hội tụ của phương sai biên ước lượng bởi IASG (cửa sổ N/S) so với SGLD (learning rate giảm theo Robbins-Monro) khi khởi tạo ngẫu nhiên. Kỳ vọng: IASG ổn định nhanh hơn SGLD, giống xu hướng paper báo cáo.
5. **Mô phỏng ảnh hưởng của momentum lên "nhiệt độ hiệu dụng":** với cùng mô hình, chạy SGD-momentum với (ϵ, μ) khác nhau nhưng giữ ϵ/μ không đổi; đo hiệp phương sai dừng. Kỳ vọng: hiệp phương sai gần như không đổi giữa các cấu hình (theo công thức (25)), xác nhận "momentum chỉ đổi tỉ lệ, không đổi hình dạng".

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
SDE xấp xỉ SGD (OU process)	$d\theta = -A\theta\, dt + \frac{1}{\sqrt{S}} B\, dW$, (11)
Phân phối dừng	$q(\theta) \propto \exp(-\frac{1}{2}\theta^\mathsf{T}\Sigma^{-1}\theta)$, $\Sigma A + A\Sigma = \frac{\epsilon}{S} BB^\mathsf{T}$
Learning rate tối ưu (Định lý 1)	$\epsilon^* = \dfrac{2SD}{N\,\mathrm{Tr}(BB^\mathsf{T})}$
Preconditioner đầy đủ tối ưu (Định lý 2)	$H^* = \dfrac{2S}{N}(BB^\mathsf{T})^{-1}$
Preconditioner chéo tối ưu (Hệ quả 3)	$H^*_{kk} = \dfrac{2S}{N\,(BB^\mathsf{T})_{kk}}$
Ước lượng online noise covariance	$C_t = (1-\kappa_t)C_{t-1} + \kappa_t(\hat{g}_{1,t} - \hat{g}_{S,t})(\hat{g}_{1,t} - \hat{g}_{S,t})^\mathsf{T}$, (21)
Momentum, hiệp phương sai dừng	$\Sigma A + A\Sigma = \frac{\epsilon}{\mu S} BB^\mathsf{T}$, (25)
SGFS preconditioner tối ưu (Định lý 5)	$H^* = \frac{2}{N}(BB^\mathsf{T} + EE^\mathsf{T})^{-1}$
Hiệp phương sai iterate averaging	$D \approx \frac{1}{TS} A^{-1}BB^\mathsf{T}(A^{-1})^\mathsf{T}$, không phụ thuộc preconditioner
Cửa sổ IASG	$T = N/S$ (một epoch cho 1 mẫu hậu nghiệm hiệu dụng)
Chặn dưới SG-MCMC	không thuật toán nào nhanh hơn $O(ND)$ cho 1 mẫu hiệu dụng độc lập (dưới Giả định A1-A5)

Ví dụ tính tay (tự tính): với $D = 2$, $N = 1000$, $S = 10$, $\mathrm{Tr}(BB^\mathsf{T}) = 4$, công thức (15) cho $\epsilon^* = \dfrac{2 \cdot 10 \cdot 2}{1000 \cdot 4} = \dfrac{40}{4000} = 0.01$. Nếu tăng batch lên $S = 100$ (giữ $\mathrm{Tr}(BB^\mathsf{T})$ không đổi, một xấp xỉ vì nhiễu thực ra giảm theo S nhưng ở đây $\mathrm{Tr}(BB^\mathsf{T})$ định nghĩa là hiệp phương sai của **một mẫu đơn**, không đổi theo S), ϵ^* tăng 10 lần lên 0.1, khớp trực giác "batch lớn thì chịu được learning rate lớn hơn" thường thấy trong thực hành huấn luyện mạng sâu (tự tính, minh hoạ công thức, không phải số trong paper).

11. Kết luận ngắn

Bài báo chỉ ra rằng SGD với learning rate hằng số, khi xấp xỉ bằng một quá trình Ornstein-Uhlenbeck liên tục, có phân phối dừng Gaussian mà ta có thể chỉnh (qua learning rate, minibatch, preconditioner) để tối thiểu hoá KL divergence tới hậu nghiệm Bayes, biến SGD thuần thành một bộ suy luận Bayes gần đúng rẻ tiền. Phân tích mở rộng sang momentum (chỉ đổi tỉ lệ, không đổi hình dạng phân phối), SGLD/SGFS (định lượng được độ chệch khi learning rate không tiệm cận 0, và SGFS với preconditioner đúng cho mẫu chính xác), và Polyak iterate averaging (tối ưu cho cả tối ưu hoá và lấy mẫu, nhưng kéo theo chặn dưới $O(ND)$ cho tốc độ sinh mẫu của mọi SG-MCMC). Thực nghiệm trên hồi quy tuyến tính/logistic xác nhận các dự đoán lý thuyết và cho thấy constant SGD với preconditioner tốt cạnh tranh được với MCMC chuyên dụng.

Nguồn gốc: https://arxiv.org/abs/1704.04289

Bài 12

Computational Optimal Transport (2018)

Tác giả: Gabriel Peyré, Marco Cuturi
Venue / arXiv: arXiv:1803.00567 (v4, 18/03/2020); bản tạp chí ghi trong bibtex ở đầu tài liệu là *Foundations and Trends in Machine Learning*, tập 11, số 5-6, trang 355-607, năm 2019; tuy nhiên dòng footer ở đầu tài liệu lại ghi ”now Publishers Inc. Computational Optimal Transport. Foundations and Trends in Computer Graphics and Vision, vol. XX, no. XX, pp. 1-205, 2020” - **đây là một điểm paper tự mâu thuẫn giữa hai dòng metadata** (tên tạp chí và số trang khác nhau ở hai chỗ trong cùng tài liệu), ghi chú này không tự chọn một trong hai, chỉ nêu rõ cả hai như paper viết.
Nguồn: https://arxiv.org/abs/1803.00567
Chủ đề: Một tài liệu chuyên khảo (monograph) dài khoảng 250 trang tổng hợp lý thuyết và thuật toán của Optimal Transport (OT, vận tải tối ưu), từ bài toán Monge/Kantorovich cổ điển tới chính quy hoá entropy (entropic regularization) và thuật toán Sinkhorn, cùng các ứng dụng trong khoa học dữ liệu.
Vị trí trong lộ trình: nên đọc trước math 13 (Wasserstein GAN) vì bài đó dùng trực tiếp đối ngẫu Kantorovich-Rubinstein mà không tự chứng minh; có thể đọc sau math 10/11 (Hamiltonian Monte Carlo, SGD Bayesian) vì không có liên hệ kỹ thuật trực tiếp nhưng cả ba đều thuộc nhóm ”nền tảng toán học định lượng” của lộ trình; nên đọc trước hoặc cùng lúc với math 17 (Spectral Normalization for GANs) vì cả hai cùng xoay quanh ràng buộc Lipschitz.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (bản arXiv); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc.

> **LƯU Ý QUAN TRỌNG VỀ PHẠM VI ĐỌC**: đây là một tài liệu chuyên khảo khoảng 250 trang (10 chương, mục lục đầy đủ: 1. Introduction, 2. Theoretical Foundations, 3. Algorithmic Foundations, 4. Entropic Regularization of Optimal Transport, 5. Semidiscrete Optimal Transport, 6. W_1 Optimal Transport, 7. Dynamic Formulations, 8. Statistical Divergences, 9. Variational Wasserstein Problems, 10. Extensions of Optimal Transport), không thể đọc và tóm tắt toàn bộ trong một ghi chú có độ dài hợp lý. Theo hướng dẫn của brief cho các bài khảo sát/sách dài, ghi chú này **đọc đầy đủ và kỹ**: Chương 1 (Introduction), Chương 2 (Theoretical Foundations, toàn bộ §2.1-2.6), và Chương 4 (Entropic Regularization, trọng tâm §4.1, §4.2, một phần §4.5-4.6, vì đây là thuật toán thực hành quan trọng nhất cho machine learning). Ghi chú **đọc một phần** Chương 8 (§8.2, phần Integral Probability Metrics, vì liên hệ trực tiếp tới đối ngẫu Kantorovich-Rubinstein dùng trong WGAN). Ghi chú **KHÔNG đọc chi tiết** Chương 3 (Algorithmic Foundations: network simplex, thuật toán auction, các giải thuật linear-programming cổ điển cho OT rời rạc), Chương 5 (Semidiscrete Optimal Transport), Chương 6 (W_1 Optimal Transport trên không gian metric/đồ thị), Chương 7 (Dynamic Formulations, công thức Benamou-Brenier), phần còn lại của Chương 8, Chương 9 (Variational Wasserstein Problems: Wasserstein barycenter, gradient flow, minimum Kantorovich estimator, chỉ được nhắc tên không đọc chi tiết công thức), và Chương 10 (Extensions: multimarginal, unbalanced OT, sliced Wasserstein, Gromov-Wasserstein) - các chương này chỉ được nêu tên và chủ đề chính trong mục 6 (Giới hạn) để người đọc biết chúng tồn tại, không được tóm tắt nội dung kỹ thuật vì chưa đọc kỹ.

1. Bối cảnh

Vận tải tối ưu (optimal transport, OT) có thể được hình dung qua bài toán Monge: chuyển một đống cát thành hình dạng cho trước với tổng chi phí nhỏ nhất. Trong cách diễn đạt xác suất, hai đống cát tương ứng với hai phân phối; chi phí của một phương án được xác định từ chi phí vận chuyển giữa từng cặp vị trí. OT nghiên cứu phương án ít tốn kém nhất, cách tính phương án đó và các cấu trúc hình học thu được trên không gian phân phối.

OT đã được ”khám phá lại” (re-discovered) nhiều lần dưới nhiều hình thức khác nhau: công trình khởi đầu của Monge xuất phát từ một bài toán kỹ thuật; Tolstoi (những năm 1920) và Hitchcock, Kantorovich, Koopmans

(những năm 1940) thiết lập ý nghĩa của nó với logistics và kinh tế học; Dantzig giải nó bằng số vào năm 1949 trong khung quy hoạch tuyến tính (linear programming); OT được các nhà phân tích toán học xem lại vào những năm 1990, đặc biệt là Brenier; và gần đây gắn liền với tầm nhìn máy tính dưới tên "earth mover's distance". Những năm gần đây chứng kiến một cuộc "cách mạng" khác nhờ sự xuất hiện của các bộ giải xấp xỉ (approximate solvers) có thể mở rộng (scale) tới các bài toán chiều lớn, khiến OT ngày càng được dùng trong xử lý ảnh, đồ hoạ, và machine learning (hồi quy, phân loại, mô hình sinh).

Tài liệu tự định vị mục tiêu: "đây là một tổng quan về OT nghiêng về các phương pháp số, bao quát các tính chất lý thuyết của OT có thể hướng dẫn thiết kế thuật toán mới, tập trung vào làn sóng thuật toán hiệu quả gần đây giúp OT có ý nghĩa trong khoa học dữ liệu" (dòng 132-135, diễn giải). Tài liệu cũng tự nói rõ về cấu trúc 3 tầng độ khó: phần chính của mỗi chương (Chương 2, 3, 4, 9, 10) chỉ xét hình học trên không gian các vector xác suất/histogram rời rạc (đủ để hiện thực thuật toán); các khung xám nhạt (light gray box) trình bày công thức cho **đo rời rạc** (discrete measures, có vị trí hỗ trợ trong không gian metric liên tục); các khung xám đậm (dark gray box) trình bày cho **đo tổng quát** (arbitrary measures, có thể có mật độ). Chương 5-8 nhắm tới độc giả thiên về toán học hơn.

2. Ý tưởng chính

Ý tưởng cốt lõi của toàn bộ OT: thay vì so sánh hai phân phối bằng các phân kỳ cổ điển (như Kullback-Leibler, chỉ so sánh **mật độ điểm-theo-điểm**), OT so sánh bằng cách tìm **kế hoạch vận chuyển rẻ nhất** biến một phân phối thành phân phối khác, dựa trên một "chi phí nền" (ground cost) $c(x, y)$ đo khoảng cách/chi phí di chuyển khối lượng từ điểm x sang điểm y. Vì OT "nâng" (lift) một khoảng cách nền giữa các điểm lên thành một khoảng cách giữa các phân phối trên các điểm đó, nó giải quyết được một hạn chế căn bản của các phân kỳ cổ điển: khi hai phân phối có **support rời nhau** hoặc không chồng lấp (ví dụ hai phân phối rời rạc đặt tại các điểm khác nhau), KL divergence hoặc total variation có thể không định nghĩa được hoặc cho giá trị không phản ánh "khoảng cách hình học" thực sự giữa chúng, còn khoảng cách OT (Wasserstein) vẫn phản ánh đúng mức độ gần/xa về vị trí.

Ý tưởng thứ hai: bài toán gốc của Monge (tìm một **ánh xạ** xác định/deterministic biến mỗi điểm nguồn thành một điểm đích duy nhất) rất khó giải vì không lồi (nonconvex) và có thể không có nghiệm. Kantorovich đưa ra ý tưởng **nới lỏng (relaxation)** mang tính đột phá: thay ánh xạ xác định bằng một **coupling** (phân phối đồng thời π hay ma trận ghép đôi P) cho phép "chia khối lượng" (mass splitting) từ một điểm nguồn tới nhiều điểm đích. Bài toán Kantorovich là một **quy hoạch tuyến tính (linear program)** lồi, dễ phân tích và giải hơn hẳn bài toán Monge gốc.

Ý tưởng thứ ba, mang tính thực hành quan trọng nhất của toàn chương trình nghiên cứu được tài liệu này tổng hợp: giải trực tiếp quy hoạch tuyến tính Kantorovich có độ phức tạp cao (với n điểm, cần xử lý ma trận $n \times n$ qua các thuật toán linear-programming cổ điển). Thêm một **số hạng chính quy hoá entropy (entropic regularization)** vào bài toán biến nó thành một bài toán **lồi chặt (strictly convex)**, có nghiệm duy nhất dạng đóng theo hai vector tỉ lệ (scaling vector), giải được bằng một thuật toán lặp rất đơn giản và song song hoá tốt trên GPU: **thuật toán Sinkhorn**. Đây là ý tưởng đã "làm sống lại" OT trong machine learning hiện đại.

3. Phương pháp chi tiết

3.1 Histogram, đo rời rạc, và bài toán Monge (§2.1-2.2)

Một **histogram** (hay vector xác suất) a là một phần tử của simplex xác suất $\Sigma_n = \{a \in \mathbb{R}_+^n : \sum_i a_i = 1\}$. Một **đo rời rạc** (discrete measure) với trọng số a và vị trí $x_1, \dots, x_n \in \mathcal{X}$ được viết $\alpha = \sum_{i=1}^n a_i \delta_{x_i}$ (công thức 2.1), với δ_x là khối Dirac tại x (một đơn vị khối lượng tập trung vô hạn tại vị trí x). Tài liệu cũng định nghĩa **đo tổng quát (general measure)** $\alpha \in \mathcal{M}(\mathcal{X})$ không nhất thiết rời rạc, truy cập được bằng cách "lấy tích phân" (test) với hàm liên tục $f \in C(\mathcal{X})$.

Bài toán gán tối ưu (optimal assignment problem): cho ma trận chi phí $(C_{i,j})$, với $n = m$, tìm phép hoán vị (permutation) σ tối thiểu $\frac{1}{n}\sum_i C_{i,\sigma(i)}$ (công thức 2.2). Tài liệu nhấn mạnh độ khó tính toán trực tiếp: tập hoán vị có kích thước $n!$, "có hơn 10^{100} phần tử khi n chỉ là 70" (dòng 432-433, trích Dantzig 1983).

Bài toán Monge (giữa hai đo rời rạc $\alpha = \sum a_i \delta_{x_i}$, $\beta = \sum b_j \delta_{y_j}$): tìm ánh xạ T sao cho $T_\#\alpha = \beta$ (push-forward, nghĩa là với mọi j: $b_j = \sum_{i:T(x_i)=y_j} a_i$, công thức 2.4), tối thiểu hoá $\sum_i c(x_i, T(x_i))$ (công thức 2.5). Tài liệu chỉ rõ hạn chế: khi $n \neq m$ "các ánh xạ Monge có thể không tồn tại giữa một đo rời rạc và đo khác, điều này luôn xảy ra khi đo đích có nhiều điểm hơn đo nguồn". Bài toán Monge nói chung **không lồi** (nonconvex) vì tập khả thi của ánh xạ không có cấu trúc lồi, khiến việc giải trực tiếp khó khăn.

3.2 Nới lỏng Kantorovich (§2.3)

Kantorovich (1942) nới lỏng bản chất "xác định" của vận chuyển: thay vì khối lượng tại x_i chỉ được gán cho một điểm đích duy nhất, khối lượng tại x_i có thể được "phân tán" (dispatched) tới nhiều vị trí. Thay cho phép hoán vị σ hoặc ánh xạ T, dùng một **ma trận coupling** $P \in \mathbb{R}_+^{n\times m}$, với $P_{i,j}$ là lượng khối lượng chảy từ bin i sang bin j. Tập coupling khả thi:

$$U(a,b) := \{P \in \mathbb{R}_+^{n\times m} : P\mathbf{1}_m = a \text{ và } P^\top \mathbf{1}_n = b\} \tag{2.10}$$

Tài liệu chỉ ra $U(a,b)$ là một **đa diện lồi bị chặn (bounded convex polytope)**, và khác với Monge (bất đối xứng), công thức Kantorovich **luôn đối xứng**: $P \in U(a,b) \Leftrightarrow P^\top \in U(b,a)$. Bài toán Kantorovich:

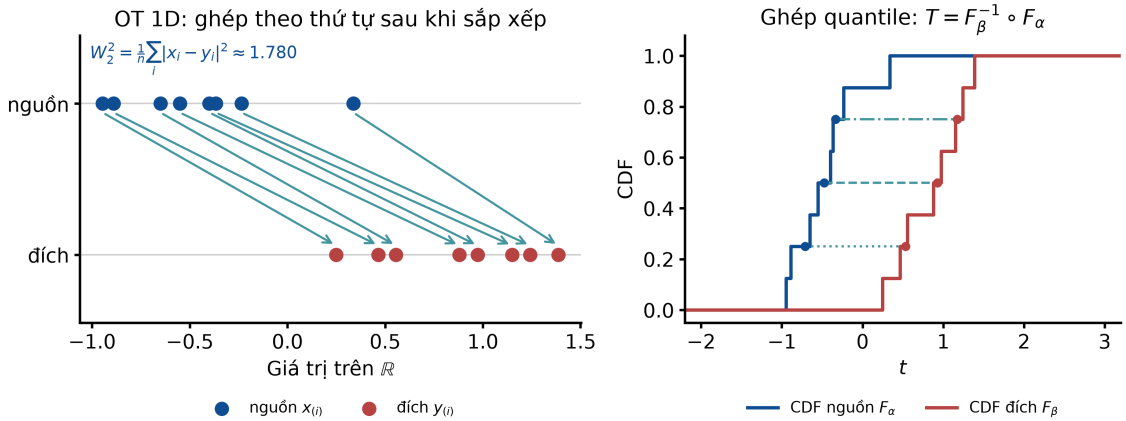
$$L_C(a,b) := \min_{P\in U(a,b)} \langle C, P\rangle = \sum_{i,j} C_{i,j}P_{i,j} \tag{2.11}$$

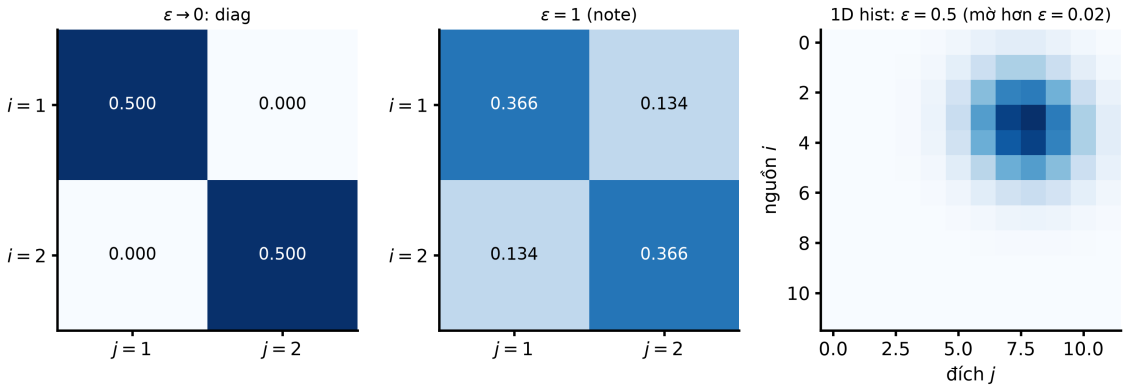
một **quy hoạch tuyến tính**. Tài liệu minh hoạ bằng ví dụ "nhà kho và nhà máy" (Remark 2.10): một người điều hành n nhà kho, m nhà máy, cần vận chuyển nguyên liệu với chi phí đơn vị $C_{i,j}$, giải (2.11) để tìm kế hoạch vận chuyển rẻ nhất.

Khi $n = m$ và $a = b = \mathbf{1}_n/n$ (phân phối đều), **Proposition 2.1 (Kantorovich cho matching)** chứng minh nghiệm tối ưu của (2.11) có thể đạt được tại một ma trận hoán vị (dùng định lý Birkhoff: tập điểm cực trị của $U(\mathbf{1}_n/n, \mathbf{1}_n/n)$ chính là tập ma trận hoán vị; và một định lý cơ bản của linear programming nói cực tiểu một hàm tuyến tính trên một đa diện đạt tại điểm cực trị). Nghĩa là: nới lỏng Kantorovich là **"tight"** (chặt, không mất tính tối ưu) trên bài toán gán, dù không gian tìm kiếm lớn hơn hẳn (coupling thay vì hoán vị).

3.3 Tính chất khoảng cách / khoảng cách Wasserstein (§2.4)

Proposition 2.2: nếu $n = m$ và $C = D^p$ với D là một khoảng cách trên $\{1,\dots,n\}$ (đối xứng, $D_{i,j} = 0 \Leftrightarrow i = j$, thoả bất đẳng thức tam giác), thì:





Sinkhorn: ε → 0 cho P đường chéo; ε=1 làm mờ coupling (ví dụ note ≈0.366/0.134); histogram 1D với ε=0.5. (minh hoạ, tự tính)

Vận tải tối ưu một chiều: sau khi sắp xếp, ghép x_{(i)} với y_{(i)}; tương đương ghép quantile qua CDF. (minh hoạ, tự tính)

$$W_p(a,b) := L_{D^p}(a,b)^{1/p} \tag{2.17}$$

định nghĩa **khoảng cách p-Wasserstein** trên Σ_n: đối xứng, dương, $W_p(a,b) = 0 \Leftrightarrow a = b$, và thoả bất đẳng thức tam giác. Chứng minh bất đẳng thức tam giác dùng kỹ thuật "gluing lemma" (bổ đề ghép nối): nếu P là coupling tối ưu giữa a, b và Q là coupling tối ưu giữa b, c, xây dựng $S = P\,\mathrm{diag}(1/\tilde{b})\,Q$ (với $\tilde{b}_j = b_j$ nếu $b_j > 0$, $= 1$ nếu không) là một coupling khả thi giữa a, c (không nhất thiết tối ưu), từ đó dùng bất đẳng thức Minkowski suy ra $W_p(a,c) \le W_p(a,b) + W_p(b,c)$. Đây là một chứng minh **cấu trúc**, không trừu tượng, phù hợp cho đo rời rạc.

Điểm đáng chú ý nhất về tính chất "khoảng cách yếu" (weak distance) của Wasserstein (Remark 2.18): với mọi $p > 0$, $W^p(\delta_x, \delta_y) = d(x,y)$ (vì $U(\delta_x, \delta_y) = \{\delta_{(x,y)}\}$ chỉ có một coupling khả thi duy nhất), nên $W^p(\delta_x, \delta_y) \to 0$ khi $x \to y$. Đây chính là tính chất mà "các khoảng cách cổ điển không có": chuẩn L^2 chỉ áp dụng được cho đo liên tục có mật độ, trong khi Wasserstein **luôn định nghĩa được** và phản ánh đúng khoảng cách hình học, ngay cả giữa hai khối Dirac. Tính chất này tương đương với **hội tụ yếu (weak convergence)**: $\alpha_k \rightharpoonup \alpha \Leftrightarrow W_p(\alpha_k, \alpha) \to 0$ (Villani 2009, Theorem 6.8).

3.4 Bài toán đối ngẫu (§2.5)

Proposition 2.4: bài toán Kantorovich (2.11) có đối ngẫu:

$$L_C(a,b) = \max_{(f,g)\in R(C)} \langle f, a\rangle + \langle g, b\rangle \tag{2.20}$$

với $R(C) = \{(f,g) \in \mathbb{R}^n \times \mathbb{R}^m : \forall (i,j),\, f_i + g_j \le C_{i,j}\}$ (công thức 2.21). (f,g) gọi là "thế vị Kantorovich" (Kantorovich potentials).

Tài liệu dùng ẩn dụ "thuê ngoài logistics" (Remark 2.21) để giải thích trực giác: một nhà vận chuyển (vendor) đề nghị tính giá "thu gom" f_i tại mỗi nhà kho i và giá "giao hàng" g_j tại mỗi nhà máy j, độc lập với tuyến đường cụ thể; tổng giá là $\langle f, a\rangle + \langle g, b\rangle$. Để người điều hành chấp nhận đề nghị, cần $f_i + g_j \le C_{i,j}$ với mọi cặp (i,j) (nếu không, tự vận chuyển rẻ hơn). Nhà vận chuyển muốn tối đa hoá giá nhận được dưới ràng buộc này - chính là bài toán đối ngẫu (2.20). Tài liệu chứng minh (bằng Lagrangian duality, trao đổi min-max) rằng giá tối ưu của nhà vận chuyển **bằng chính xác** chi phí vận chuyển tối ưu $L_C(a,b)$ - đây là **strong duality** của quy hoạch tuyến tính.

Định lý Brenier (Theorem 2.1), một kết quả cổ điển quan trọng được tài liệu trình bày với sơ đồ chứng minh: trong trường hợp $\mathcal{X} = \mathcal{Y} = \mathbb{R}^d$, $c(x, y) = \|x - y\|^2$, nếu ít nhất một trong hai đo (gọi là α) có mật độ so với đo Lebesgue, thì coupling tối ưu π của bài toán Kantorovich là **duy nhất** và được hỗ trợ trên đồ thị của một "ánh xạ Monge" $T : \mathbb{R}^d \to \mathbb{R}^d$. Hơn nữa, T được xác định duy nhất là **gradient của một hàm lồi** φ: $T(x) = \nabla\varphi(x)$. Ý nghĩa: trong trường hợp này, **bài toán Kantorovich và bài toán Monge là tương đương** (nới lỏng là chặt), đây là phiên bản liên tục của Proposition 2.1 (trường hợp rời rạc, đều). Tài liệu cũng liên hệ φ với phương trình Monge-Ampère $\det(\partial^2\varphi(x))\rho_\beta(\nabla\varphi(x)) = \rho_\alpha(x)$ (công thức 2.32, Remark 2.25), một PDE phi tuyến khó giải số trực tiếp (toán tử Monge-Ampère được tài liệu gọi là "Laplacian suy biến phi tuyến").

3.5 Trường hợp đặc biệt: 1 chiều (§2.6)

Khi $\mathcal{X} = \mathbb{R}$ (một chiều), OT có **công thức đóng đơn giản**: nếu $\alpha = \frac{1}{n}\sum \delta_{x_i}$, $\beta = \frac{1}{n}\sum \delta_{y_j}$ với $x_1 \le \cdots \le x_n$ và $y_1 \le \cdots \le y_n$ đã **sắp thứ tự**, thì:

$$W_p(\alpha, \beta)^p = \frac{1}{n}\sum_{i=1}^{n} |x_i - y_i|^p \tag{2.33}$$

nghĩa là chỉ cần **sắp xếp (sort)** hai tập điểm rồi ghép theo thứ tự, không cần giải quy hoạch tuyến tính đầy đủ. Tài liệu minh hoạ ứng dụng thực tế: **cân bằng histogram ảnh (histogram equalization)**, dùng OT 1D để tìm phép tương ứng giữa mức xám của hai ảnh (Remark 2.29), và công cụ hàm phân phối tích luỹ nghịch (quantile function, công thức 2.34-2.35) để tổng quát hoá cho đo liên tục bất kỳ trên $\mathbb{R}$.

3.6 Chính quy hoá Entropy (§4.1)

Entropy rời rạc của một coupling: $H(P) := -\sum_{i,j} P_{i,j}(\log(P_{i,j}) - 1)$ (công thức 4.1), là hàm **1-lõm mạnh (1-strongly concave)**. Ý tưởng chính quy hoá entropy:

$$L_C^\varepsilon(a, b) := \min_{P \in U(a,b)} \langle P, C\rangle - \varepsilon H(P) \tag{4.2}$$

Vì mục tiêu là hàm ε**-lồi mạnh (ε-strongly convex)**, bài toán (4.2) có nghiệm **duy nhất**. Tài liệu lưu ý nguồn gốc lịch sử: ý tưởng này "có thể truy về các mô hình trong lý thuyết giao thông [Wilson, 1969]: các mẫu giao thông thực tế không khớp với dự đoán của bài toán vận tải tối ưu, vì giao thông thực tế phân tán (diffuse) hơn, trong khi OT có xu hướng dựa vào một vài tuyến đường do tính thưa (sparsity) của coupling tối ưu" (dòng 2694-2698, diễn giải).

Proposition 4.1 (hội tụ theo ε): nghiệm duy nhất P_ε của (4.2) thoả:

- $\varepsilon \to 0$: $P_\varepsilon \to \arg\min\{-H(P) : P \in U(a,b), \langle P, C\rangle = L_C(a,b)\}$, nghĩa là hội tụ về nghiệm **có entropy lớn nhất** trong tập nghiệm tối ưu của bài toán gốc không chính quy hoá, và $L_C^\varepsilon(a,b) \to L_C(a,b)$.
- $\varepsilon \to \infty$: $P_\varepsilon \to a \otimes b = ab^\top$ (tích tensor, coupling của hai biến **độc lập**).

Diễn giải trực giác (tự tổng hợp từ tài liệu): ε nhỏ cho nghiệm gần giống bài toán OT gốc (thưa, "sắc nét"); ε lớn cho nghiệm giống hệt trường hợp hai biến độc lập hoàn toàn (không còn thông tin vận chuyển, chỉ còn tích hai marginal). Giữa hai cực đó, nghiệm **trở nên ít thưa hơn** (less sparse) khi ε tăng, điều này vừa giúp tăng tốc thuật toán (mục 3.7) vừa cải thiện tốc độ hội tụ thống kê (Chương 8, không đọc chi tiết trong ghi chú này).

Một cách biểu diễn khác của (4.2): định nghĩa phân kỳ KL giữa hai ma trận $KL(P|K) := \sum_{i,j} P_{i,j}\log(P_{i,j}/K_{i,j}) - P_{i,j} + K_{i,j}$ (công thức 4.6), và kernel Gibbs $K_{i,j} := e^{-C_{i,j}/\varepsilon}$, thì nghiệm P_ε chính là **phép chiếu KL** của K lên $U(a,b)$: $P_\varepsilon = \text{Proj}^{KL}_{U(a,b)}(K) = \arg\min_{P \in U(a,b)} KL(P|K)$ (công thức

4.7). Bài toán này, với đo tổng quát, được gọi là ”bài toán Schrödinger tĩnh” (static Schrödinger problem), có nguồn gốc từ vật lý thống kê (Schrödinger 1931).

3.7 Thuật toán Sinkhorn (§4.2)

Proposition 4.3: nghiệm của (4.2) có dạng $P_{i,j} = u_i K_{i,j} v_j$ cho hai vector tỉ lệ (scaling) $(u, v) \in \mathbb{R}_+^n \times \mathbb{R}_+^m$ (công thức 4.12). Chứng minh: đưa vào 2 biến đối ngẫu f, g cho ràng buộc marginal, điều kiện bậc nhất (first-order condition) cho $\frac{\partial}{\partial P_{i,j}} \left[\langle P, C\rangle - \varepsilon H(P) - \langle f, P\mathbf{1}_m - a\rangle - \langle g, P^\top \mathbf{1}_n - b\rangle\right] = 0$ cho ra $C_{i,j} + \varepsilon \log(P_{i,j}) - f_i - g_j = 0$, tức $P_{i,j} = e^{f_i/\varepsilon} e^{-C_{i,j}/\varepsilon} e^{g_j/\varepsilon}$, đặt $u_i = e^{f_i/\varepsilon}$, $v_j = e^{g_j/\varepsilon}$ cho dạng trên.

Viết dưới dạng ma trận: $P = \mathrm{diag}(u) K \mathrm{diag}(v)$. Ràng buộc marginal $P\mathbf{1}_m = a$, $P^\top \mathbf{1}_n = b$ trở thành hệ phương trình phi tuyến (bài toán ”matrix scaling” trong giải tích số):

$$u \odot (Kv) = a \quad \text{và} \quad v \odot (K^\top u) = b \tag{4.14}$$

($\odot$ là phép nhân theo từng phần tử). **Thuật toán Sinkhorn** giải hệ này bằng lặp xen kẽ:

$$u^{(\ell+1)} := \frac{a}{K v^{(\ell)}}, \qquad v^{(\ell+1)} := \frac{b}{K^\top u^{(\ell+1)}} \tag{4.15}$$

khởi tạo $v^{(0)} = \mathbf{1}_m$ (phép chia giữa hai vector hiểu theo từng phần tử). Tài liệu nhấn mạnh: dù (u, v) không duy nhất (chỉ xác định tới một hằng số nhân: nếu (u, v) thoả (4.13) thì $(\lambda u, v/\lambda)$ cũng thoả với mọi $\lambda > 0$), **coupling diag**$(u) K$ **diag**(v) **thu được là duy nhất** và các lần lặp đều hội tụ về nó.

Remark 4.5 (lịch sử): các lần lặp (4.15) ”xuất hiện đầu tiên trong [Yule 1912, Kruithof 1937], sau đó được biết là thủ tục iterative proportional fitting (IPFP) và phương pháp RAS. Chứng minh hội tụ được gán cho Sinkhorn [1964], do đó có tên thuật toán này... quan tâm trở lại trong khoa học dữ liệu sau [Cuturi 2013], người chỉ ra thuật toán Sinkhorn cung cấp một xấp xỉ hiệu quả và có thể mở rộng của OT, nhờ song song hoá dễ dàng (đặc biệt trên GPU), và đại lượng chính quy hoá này cũng định nghĩa, khác với công thức quy hoạch tuyến tính, một hàm mất mát khả vi” (dòng 2994-3011, diễn giải).

Remark 4.6 (độ phức tạp tổng thể): Altschuler et al. [2017] chứng minh với $\varepsilon = \frac{4\log(n)}{\tau}$, cần $O(\|C\|_\infty^3 \log(n) \tau^{-3})$ lần lặp Sinkhorn (kèm một bước làm tròn/rounding để đưa về coupling khả thi $\hat{P} \in U(a, b)$) để đảm bảo $\langle \hat{P}, C\rangle \le L_C(a, b) + \tau$. Điều này kéo theo Sinkhorn tính được nghiệm τ**-xấp xỉ** của OT không chính quy hoá trong $O(n^2 \log(n) \tau^{-3})$ phép toán.

Remark 4.7 (ổn định số học): tài liệu cảnh báo ”trong thực hành, hội tụ Sinkhorn thường không bị chậm đi vì một lý do đơn giản hơn: thuật toán Sinkhorn thường **không kết thúc được** ngay khi một số phần tử của kernel K trở nên quá nhỏ để lưu dưới dạng số dương trong bộ nhớ, và trở thành 0 thay vào đó”, dẫn tới phép chia cho 0. Giải pháp: tính toán trên các số nhân tử u, v trong **log-domain** (Remark 4.23, không đọc chi tiết trong ghi chú này).

3.8 Ví dụ số nhỏ tính tay: một bước Sinkhorn (tự tính)

Để minh hoạ cơ chế thuật toán Sinkhorn, xét bài toán rất nhỏ: $n = m = 2$, $a = (0.5, 0.5)$, $b = (0.5, 0.5)$ (hai histogram đều trên 2 bin), ma trận chi phí $C = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ (chi phí 0 nếu cùng bin, 1 nếu khác bin), và $\varepsilon = 1$.

Kernel Gibbs: $K_{i,j} = e^{-C_{i,j}/\varepsilon}$, nên $K = \begin{pmatrix} e^0 & e^{-1} \\ e^{-1} & e^0 \end{pmatrix} = \begin{pmatrix} 1 & 0.3679 \\ 0.3679 & 1 \end{pmatrix}$ (tự tính: $e^{-1} \approx 0.3679$).

Khởi tạo $v^{(0)} = (1, 1)$. Bước 1: $u^{(1)} = a/(Kv^{(0)})$. Tính $Kv^{(0)} = \begin{pmatrix} 1 \times 1 + 0.3679 \times 1 \\ 0.3679 \times 1 + 1 \times 1 \end{pmatrix} = \begin{pmatrix} 1.3679 \\ 1.3679 \end{pmatrix}$ (tự tính). Vậy $u^{(1)} = (0.5/1.3679,\ 0.5/1.3679) = (0.3655,\ 0.3655)$ (tự tính, $0.5/1.3679 \approx 0.3655$).

Bước 2: $v^{(1)} = b/(K^\top u^{(1)})$. Vì K đối xứng, $K^\top = K$. $Ku^{(1)} = \begin{pmatrix} 1 \times 0.3655 + 0.3679 \times 0.3655 \\ 0.3679 \times 0.3655 + 1 \times 0.3655 \end{pmatrix} = \begin{pmatrix} 0.3655 \times 1.3679 \\ 0.3655 \times 1.3679 \end{pmatrix} = \begin{pmatrix} 0.5000 \\ 0.5000 \end{pmatrix}$ (tự tính: $0.3655 \times 1.3679 \approx 0.5000$). Vậy $v^{(1)} = (0.5/0.5,\ 0.5/0.5) = (1, 1)$ (tự tính).

Vì $v^{(1)} = v^{(0)} = (1, 1)$, thuật toán đã **hội tụ ngay sau 1 vòng lặp** đối với bài toán đối xứng đặc biệt này (do tính đối xứng của $a = b$ và C). Coupling tối ưu: $P_{i,j} = u_i K_{i,j} v_j$, với $u = (0.3655, 0.3655)$, $v = (1, 1)$:

$$P = \begin{pmatrix} 0.3655 \times 1 \times 1 & 0.3655 \times 0.3679 \times 1 \\ 0.3655 \times 0.3679 \times 1 & 0.3655 \times 1 \times 1 \end{pmatrix} = \begin{pmatrix} 0.3655 & 0.1345 \\ 0.1345 & 0.3655 \end{pmatrix}$$

(tự tính: $0.3655 \times 0.3679 \approx 0.1345$). Kiểm tra marginal: tổng hàng 1 = $0.3655 + 0.1345 = 0.5000$, khớp $a_1 = 0.5$ (tự tính), xác nhận $P \in U(a, b)$. Nhận xét tự rút ra: nghiệm P có khối lượng tập trung nhiều hơn trên đường chéo ($0.3655 > 0.1345$), phản ánh đúng trực giác "chi phí $C_{1,1} = C_{2,2} = 0$ rẻ hơn $C_{1,2} = C_{2,1} = 1$", nhưng **không hoàn toàn tập trung** trên đường chéo (khác với nghiệm OT không chính quy hoá $\varepsilon = 0$, lẽ ra sẽ cho $P = \text{diag}(0.5, 0.5)$ hoàn toàn trên đường chéo vì đây chính là trường hợp $a = b$ nên coupling tối ưu không chính quy hoá là ma trận đơn vị nhân 0.5), đúng với lý thuyết Proposition 4.1 rằng chính quy hoá entropy "làm mờ" (blur) nghiệm ra khỏi các đỉnh thưa của đa diện.

3.9 Hàm mất mát khả vi và Sinkhorn divergence (§4.5)

Vì $L_C^\varepsilon(a, b)$ là hàm **lồi chung (jointly convex)** theo cả a, b khi $\varepsilon \geq 0$, và khi $\varepsilon > 0$ gradient của nó bằng chính các thế vị đối ngẫu tối ưu: $\nabla L_C^\varepsilon(a, b) = (f^\star, g^\star)$ (Proposition 4.6, chọn $f^\star, g^\star$ sao cho tổng toạ độ bằng 0). Điều này rất quan trọng cho machine learning: L_C^ε **có thể dùng trực tiếp làm hàm mất mát khả vi** trong các bài toán biến phân (Chương 9, không đọc chi tiết).

Tài liệu định nghĩa **Sinkhorn divergence** (nguyên và đối ngẫu, Definition 4.1): $P_C^\varepsilon(a, b) := \langle C, P^\star \rangle$ và $D_C^\varepsilon(a, b) := \langle f^\star, a \rangle + \langle g^\star, b \rangle$, thoả quan hệ $D_C^\varepsilon(a, b) \leq L_C^\varepsilon(a, b) \leq P_C^\varepsilon(a, b)$ và $P_C^\varepsilon(a, b) - D_C^\varepsilon(a, b) = \varepsilon(H(P^\star) + 1)$ (Proposition 4.7, công thức 4.47). Tài liệu tự lưu ý một điểm thực tế quan trọng: các chặn này "chỉ đúng khi Sinkhorn đã hội tụ hoàn toàn và do đó không bao giờ thực sự hữu dụng trong thực hành" (dòng 3879-3881, diễn giải), vì trong thực hành số vòng lặp luôn bị giới hạn (ngưỡng dừng sớm). Khi dừng sau L vòng lặp cố định, $D_C^{(L)}(a, b)$ (dùng $f^{(L)}, g^{(L)}$ chưa hội tụ) vẫn cho một **chặn dưới hợp lệ** của L_C^ε (Proposition 4.8), nhưng tài liệu cũng cảnh báo (Remark 4.27) rằng "khác với L_C^ε, Sinkhorn divergence hữu hạn $D_C^{(L)}(a, b)$ nói chung **không phải hàm lồi** theo tham số của nó (có thể kiểm tra dễ dàng bằng số)" - một điểm hạn chế lý thuyết quan trọng khi dùng Sinkhorn divergence làm loss trong tối ưu không lồi.

3.10 Liên hệ Integral Probability Metric và Kantorovich-Rubinstein (§8.2, đọc một phần)

Tài liệu định nghĩa khung tổng quát **dual norm**/Integral Probability Metric (IPM): cho tập lồi đối xứng B các hàm đo được, $\|\alpha\|_B := \max_f \{\int f \, d\alpha : f \in B\}$ (công thức 8.10). **Example 8.7** xác nhận trực tiếp: khoảng cách W^1 (Wasserstein-1, định nghĩa ở Chương 6 mà ghi chú này không đọc chi tiết) chính là một trường hợp đặc biệt của dual norm này với $B = \{f : \text{Lip}(f) \leq 1\}$ (tập hàm 1-Lipschitz) - đây chính là công thức đối ngẫu Kantorovich-Rubinstein mà math 13 (Wasserstein GAN) dùng trực tiếp làm nền tảng. Tài liệu còn định nghĩa **flat norm** (còn gọi là chuẩn "Kantorovich-Rubinstein" theo Hanin 1992, dùng $B = \{f : \|\nabla f\|_\infty \leq 1, \|f\|_\infty \leq 1\}$, công thức 8.11) và **Maximum Mean Discrepancy (MMD)** như một trường hợp IPM khác

(dùng B là hình cầu đơn vị của một không gian Hilbert tái tạo nhân, RKHS) - những biến thể này không được đọc chi tiết trong ghi chú này.

4. Thí nghiệm và kết quả chính

Vì đây là một tài liệu khảo sát/chuyên khảo lý thuyết-thuật toán, không phải một bài báo thực nghiệm theo nghĩa "so sánh hệ thống mới với baseline trên benchmark cố định". Phần "thực nghiệm" chủ yếu là các hình minh hoạ số học để làm rõ hành vi của các khái niệm lý thuyết, trong các chương mà ghi chú này đã đọc kỹ:

- Figure 2.8 (đã đọc): minh hoạ coupling tối ưu $P^\star$ và thế vị đối ngẫu $f^\star, g^\star$ cho một bài toán OT cụ thể giữa hai đo rời rạc 2D với chi phí Euclid bình phương, trực quan hoá "giá tương đối" tại mỗi điểm.
- Figure 2.9 (đã đọc): minh hoạ coupling tối ưu 1D (công thức 2.33) dưới dạng mũi tên nối các điểm đã sắp xếp.
- Figure 4.1-4.3 (đã đọc): minh hoạ ảnh hưởng của ε lên nghiệm chính quy hoá entropy trên simplex Σ_3 (tam giác 2D) và giữa hai đo liên tục/rời rạc 1D-2D, xác nhận trực quan Proposition 4.1 (nghiệm dịch từ góc đa diện về "trung tâm entropy" khi ε tăng).
- Figure 4.5 (đã đọc): minh hoạ sự tiến triển của coupling Sinkhorn qua các vòng lặp ℓ, và tốc độ hội tụ (đo bằng log của vi phạm ràng buộc marginal) với các giá trị $\varepsilon = 10, 0.1, 10^{-3}$ - xác nhận trực quan rằng ε nhỏ hơn làm Sinkhorn hội tụ chậm hơn.

Vì ghi chú này không đọc chi tiết các chương thực nghiệm ứng dụng cụ thể hơn (ví dụ các ví dụ ảnh màu/texture trong Chương 2 Remark 2.12, hay các ứng dụng Wasserstein barycenter trong Chương 9), không có bảng số liệu định lượng kiểu "accuracy/FID" nào để báo cáo ở mục này - bản chất tài liệu này là lý thuyết và thuật toán, không phải một paper thực nghiệm đơn lẻ.

5. Phân tích / ablation

Tương tự mục 4, không có "ablation" theo nghĩa thực nghiệm máy học thông thường. Thay vào đó, phần có giá trị phân tích gần nhất với "ablation" là các **so sánh chế độ cực hạn của tham số** ε (Proposition 4.1, đã trình bày ở mục 3.6): $\varepsilon \to 0$ (gần với OT gốc, nghiệm thưa/sắc nét, entropy lớn nhất trong tập nghiệm tối ưu) và $\varepsilon \to \infty$ (nghiệm hội tụ về tích hai marginal độc lập, hoàn toàn "mờ"). Đây là một phân tích lý thuyết có vai trò tương đương một "ablation tham số" giúp người đọc hiểu rõ đánh đổi (trade-off) giữa độ chính xác xấp xỉ OT gốc và tốc độ/ổn định tính toán (Remark 4.6-4.7: ε nhỏ hơn cho xấp xỉ tốt hơn nhưng Sinkhorn hội tụ chậm hơn và dễ gặp vấn đề số học).

Một phân tích thứ hai có giá trị: so sánh chặn trên/dưới của Sinkhorn divergence ($D_C^\varepsilon \le L_C^\varepsilon \le P_C^\varepsilon$, với khoảng cách giữa chúng chính xác bằng $\varepsilon(H(P^\star)+1)$) cho thấy rõ **chi phí chính xác của việc chính quy hoá**: sai số giữa bài toán chính quy hoá và OT gốc có thể định lượng trực tiếp qua ε và entropy của nghiệm.

6. Giới hạn

Các giới hạn **bài báo nêu** trong các phần ghi chú này đã đọc:

1. Nghịch lý thực tế của Sinkhorn: lý thuyết nói tốc độ hội tụ chậm đi khi $\varepsilon \to 0$, nhưng tài liệu tự nói "trong thực hành, sự chậm đi này hiếm khi được quan sát vì một lý do đơn giản hơn: thuật toán sẽ **không hội tụ được** (not terminate) khi phần tử kernel K quá nhỏ để lưu dưới dạng số dương, dẫn tới chia cho 0" (Remark 4.7) - một hạn chế số học thực tế nghiêm trọng hơn cả vấn đề tốc độ hội tụ lý thuyết.
2. Chặn lý thuyết của Sinkhorn divergence ($D_C^\varepsilon \le L_C^\varepsilon \le P_C^\varepsilon$) "chỉ đúng khi đã hội tụ hoàn toàn, và do đó không bao giờ thực sự hữu dụng trong thực hành" - bài báo nêu một kết quả lý thuyết đẹp nhưng **không áp dụng được trực tiếp** khi dừng sớm (early stopping), cần công thức thay thế (Proposition 4.8) chỉ cho chặn một chiều.

3. Sinkhorn divergence hữu hạn bước $D_C^{(L)}$ **không lồi** nói chung theo tham số (Remark 4.27) - khác với L_C^ε lý thuyết (lồi chung theo a, b), khiến việc tối ưu các mô hình dùng Sinkhorn divergence làm loss (ví dụ khi L nhỏ để tiết kiệm tính toán) mất đi đảm bảo lồi.
4. Bài toán Monge gốc có thể **không có nghiệm** khi $n \neq m$ và trọng số không tương thích (Remark 2.4 cuối) - nới lỏng Kantorovich luôn có nghiệm (vì tập khả thi compact và không rỗng), nhưng điều này đồng nghĩa Kantorovich và Monge **không luôn tương đương**; định lý Brenier chỉ đảm bảo tương đương trong trường hợp đặc biệt (một đo có mật độ, chi phí Euclid bình phương).
5. Về mặt phạm vi đọc của ghi chú này (giới hạn do người viết chọn, không phải do bài báo nêu): các chương không đọc chi tiết chứa những nội dung quan trọng khác của lý thuyết OT hiện đại mà ghi chú này **không thể đánh giá/tóm tắt chính xác**, cụ thể: Chương 3 (các thuật toán linear-programming cổ điển cho OT rời rạc: network simplex, auction algorithm - các thuật toán chính xác, không xấp xỉ, có độ phức tạp đa thức nhưng chậm hơn Sinkhorn trong thực hành theo hiểu biết chung của lĩnh vực, ”ngoài paper, kiến thức chung”); Chương 5 (Semidiscrete OT, khi một đo rời rạc một đo liên tục); Chương 6 (W^1 trên không gian metric/đồ thị, có công thức đóng nhanh hơn Sinkhorn trong một số trường hợp); Chương 7 (công thức động Benamou-Brenier, biến OT thành bài toán PDE phụ thuộc thời gian); Chương 9 (Wasserstein barycenter, gradient flow trong không gian Wasserstein, minimum Kantorovich estimator - các ứng dụng biến phân quan trọng cho machine learning mà ghi chú này chỉ nêu tên); Chương 10 (OT đa biên/multimarginal, OT không cân bằng/unbalanced khi hai đo không cùng tổng khối lượng, sliced Wasserstein distance, khoảng cách Gromov-Wasserstein cho so sánh cấu trúc giữa hai không gian metric khác nhau).

7. Liên hệ với các bài khác

- **math 13 (Wasserstein GAN, 1701.07875)**: WGAN dùng trực tiếp đối ngẫu Kantorovich-Rubinstein (công thức 2.20 của tài liệu này, hoặc tương đương IPM với B = hàm 1-Lipschitz, Example 8.7 mục 3.10) như một ”hộp đen” lý thuyết không tự chứng minh; tài liệu hiện tại là nơi chứng minh/trình bày đầy đủ hơn kết quả đối ngẫu đó (Proposition 2.4, 2.5).
- **math 17 (Spectral Normalization for GANs, 1802.05957)**: Cả hai bài (math 13, math 17) đều cần ép ràng buộc Lipschitz lên một hàm thực hiện bởi mạng neural; tài liệu hiện tại cung cấp nền tảng lý thuyết **tại sao** ràng buộc 1-Lipschitz quan trọng (chính là định nghĩa của W^1 qua IPM).
- **ml basics content/14_2014_gan.md, ml basics content/15_2013_vae_aevb.md**: GAN gốc dùng Jensen-Shannon divergence; VAE dùng KL divergence trong ELBO. Cả hai đều là các ”phân kỳ cổ điển” mà OT/Wasserstein (tài liệu này) được đề xuất như một thay thế có tính chất hình học tốt hơn khi hai phân phối không chồng lấp.

8. Gợi ý áp dụng

Optimal transport so sánh các phân phối có xét đến chi phí vận chuyển giữa các điểm. Entropic regularization cho phép dùng thuật toán Sinkhorn, đổi lại nghiệm phụ thuộc hệ số regularization và ma trận chi phí có thể tốn bộ nhớ khi số điểm lớn.

9. Thí nghiệm gợi ý

1. **Tái tạo ví dụ Sinkhorn tay (mục 3.8) bằng code và mở rộng lên kích thước lớn hơn**: viết hàm Python/PyTorch hiện thực chính xác công thức (4.15) cho một bài toán $n = m = 50$ (thay vì 2), với a, b ngẫu nhiên trên simplex và ma trận chi phí C là khoảng cách Euclid bình phương giữa các điểm ngẫu nhiên trong $\mathbb{R}^2$, chạy trên GPU hoặc CPU (bài toán rất nhẹ). Đo số vòng lặp cần để vi phạm marginal ($\|P\mathbf{1}_m - a\|_1$) nhỏ hơn một ngưỡng, với vài giá trị ε khác nhau (10, 1, 0.1, 0.01). Kết quả kỳ vọng: số vòng lặp cần tăng mạnh khi ε giảm, có thể gặp lỗi số học (NaN/0) ở ε rất nhỏ nếu không dùng log-domain. Diễn giải: xác nhận trực tiếp Remark 4.6-4.7 của tài liệu (đánh đổi giữa độ chính xác xấp xỉ và tốc độ/ổn định hội tụ) trên dữ liệu tự tạo.

2. **So sánh Sinkhorn divergence vs KL divergence khi hai phân phối ít chồng lấp**: tạo hai phân phối rời rạc nhỏ (ví dụ 10 bin) có support hoàn toàn hoặc gần như rời nhau (ví dụ a đặt khối lượng ở bin 1-3, b ở bin 8-10), tính cả Sinkhorn divergence (công thức ở mục 3.9, dùng chi phí $|i-j|$ giữa chỉ số bin) và KL divergence tiêu chuẩn, thay đổi dần để hai phân phối "xích lại gần nhau" (bin chồng lấp tăng dần). Kết quả kỳ vọng: KL có thể nhảy giá trị đột ngột hoặc không định nghĩa khi support rời hoàn toàn (giống Example 1 của WGAN, math 13), còn Sinkhorn divergence thay đổi mượt theo khoảng cách bin. Diễn giải: minh hoạ trực tiếp động lực toán học cho mục 8 (lý do Wasserstein/Sinkhorn có thể ổn định hơn KL khi phân phối ít chồng lấp).

3. **Kiểm chứng định lý Brenier bằng số trên ví dụ 2D nhỏ**: với hai phân phối Gaussian 2D (một có mật độ, theo đúng giả định định lý Brenier), tính OT bằng Sinkhorn với ε rất nhỏ, trích xuất coupling, và kiểm tra bằng số liệu xem ánh xạ "trung bình có điều kiện" $\hat{T}(x) = \mathbb{E}[y|x]$ suy ra từ coupling có gần với một ánh xạ affine (gradient của hàm lồi bậc 2, vì với hai Gaussian ánh xạ Brenier có dạng tuyến tính đã biết trong kiến thức chung) hay không. Kết quả kỳ vọng: với ε đủ nhỏ, ánh xạ suy ra gần với nghiệm affine lý thuyết (ngoài paper, kiến thức chung: với 2 Gaussian, ánh xạ Monge-Brenier tối ưu là một phép biến đổi tuyến tính cụ thể theo ma trận hiệp phương sai). Diễn giải: kiểm chứng thực nghiệm trực tiếp định lý 2.1 (Brenier) trên một trường hợp đơn giản có thể kiểm tra độc lập.

10. Tóm tắt công thức / khái niệm

- Khoảng cách p-Wasserstein (rời rạc): $W_p(a,b) = L_{D^p}(a,b)^{1/p}$ với $L_C(a,b) = \min_{P\in U(a,b)}\langle C, P\rangle$ (công thức 2.11, 2.17).
- Tập coupling: $U(a,b) = \{P \geq 0 : P\mathbf{1}_m = a, P^\top \mathbf{1}_n = b\}$ (công thức 2.10).
- Đối ngẫu Kantorovich: $L_C(a,b) = \max_{(f,g):f_i+g_j\leq C_{i,j}} \langle f, a\rangle + \langle g, b\rangle$ (công thức 2.20).
- Định lý Brenier: với $c = \|x-y\|^2$, một đo có mật độ, coupling tối ưu hỗ trợ trên đồ thị $T = \nabla\varphi$, φ lồi.
- Công thức đóng 1D: $W_p(\alpha,\beta)^p = \frac{1}{n}\sum_i |x_i - y_i|^p$ với các điểm đã sắp thứ tự (công thức 2.33).
- Entropy: $H(P) = -\sum_{i,j} P_{i,j}(\log P_{i,j} - 1)$ (công thức 4.1).
- OT chính quy hoá entropy: $L_C^\varepsilon(a,b) = \min_{P\in U(a,b)}\langle P, C\rangle - \varepsilon H(P)$ (công thức 4.2); $\varepsilon \to 0$ cho nghiệm entropy lớn nhất của OT gốc, $\varepsilon \to \infty$ cho $P \to ab^\top$.
- Nghiệm Sinkhorn: $P_{i,j} = u_i K_{i,j} v_j$, $K_{i,j} = e^{-C_{i,j}/\varepsilon}$ (công thức 4.12).
- Lặp Sinkhorn: $u^{(\ell+1)} = a/(Kv^{(\ell)})$, $v^{(\ell+1)} = b/(K^\top u^{(\ell+1)})$ (công thức 4.15).
- Độ phức tạp Sinkhorn (τ-xấp xỉ): $O(n^2 \log(n)\tau^{-3})$ phép toán (Altschuler et al. 2017).
- Gradient của OT chính quy hoá: $\nabla L_C^\varepsilon(a,b) = (f^\star, g^\star)$ (Proposition 4.6).
- IPM tổng quát: $\|\alpha\|_B = \max_{f\in B}\int f\, d\alpha$ (công thức 8.10); W^1 là trường hợp B=hàm 1-Lipschitz (Example 8.7), chính là Kantorovich-Rubinstein duality dùng trong WGAN.
- Ví dụ tự tính (mục 3.8): bài toán 2×2 đối xứng, $\varepsilon = 1$, hội tụ sau 1 vòng lặp, $P = \begin{pmatrix} 0.3655 & 0.1345 \\ 0.1345 & 0.3655 \end{pmatrix}$, kiểm tra marginal khớp $a = b = (0.5, 0.5)$ (tự tính).

11. Kết luận ngắn

Tài liệu này tổng hợp nền tảng lý thuyết (Monge, nới lỏng Kantorovich, tính chất khoảng cách Wasserstein, đối ngẫu, định lý Brenier) và thuật toán thực hành quan trọng nhất (chính quy hoá entropy và thuật toán Sinkhorn) của Optimal Transport, với trọng tâm hướng tới ứng dụng khoa học dữ liệu/machine learning. Ý tưởng cốt lõi: OT cho một cách "nâng" khoảng cách nền giữa các điểm thành khoảng cách giữa các phân phối, khắc phục hạn chế của các phân kỳ cổ điển khi hai phân phối không chồng lấp; chính quy hoá entropy biến bài toán quy hoạch tuyến tính khó thành một bài toán lồi chặt có nghiệm duy nhất, giải được bằng thuật toán Sinkhorn đơn giản, song song hoá tốt, và **khả vi** - tính chất khả vi này là cầu nối trực tiếp tới các ứng dụng machine learning hiện đại như WGAN (dùng đối ngẫu Kantorovich-Rubinstein). Ghi chú này đọc đầy đủ Chương 1, 2, phần lớn Chương 4, và một phần Chương 8 của một tài liệu dài khoảng 250 trang; các chương về thuật

toán linear-programming cổ điển (Chương 3), semidiscrete OT (Chương 5), W^1 trên không gian metric/đồ thị (Chương 6), công thức động Benamou-Brenier (Chương 7), các ứng dụng biến phân (Chương 9), và các mở rộng (multimarginal, unbalanced OT, sliced Wasserstein, Gromov-Wasserstein - Chương 10) không được đọc chi tiết và chỉ được nêu tên.

Nguồn gốc: https://arxiv.org/abs/1803.00567

Bài 13

Wasserstein GAN (2017)

Tác giả: Martin Arjovsky, Soumith Chintala, Léon Bottou
Venue / arXiv: arXiv:1701.07875 (v3, 06/12/2017); paper không nêu venue hội nghị/journal cụ thể trên trang đầu (chỉ ghi tên tác giả và affiliation: Courant Institute of Mathematical Sciences, Facebook AI Research); đây là một trong các paper nền tảng được biết rộng rãi trong cộng đồng, công bố qua arXiv.
Nguồn: https://arxiv.org/abs/1701.07875
Chủ đề: Đề xuất thay khoảng cách Jensen-Shannon (JS, dùng trong GAN gốc) bằng khoảng cách Earth-Mover/Wasserstein-1 để đo sự khác biệt giữa phân phối mô hình (generator) và phân phối dữ liệu thật, giải quyết vấn đề gradient biến mất và mode collapse trong huấn luyện GAN, thông qua một thuật toán thực hành gọi là WGAN dùng Kantorovich-Rubinstein duality và weight clipping.
Vị trí trong lộ trình: nên đọc sau ml basics 14 (Generative Adversarial Networks, Goodfellow et al. 2014) để hiểu GAN gốc trước; nên đọc cùng lúc hoặc trước math 12 (Computational Optimal Transport) vì bài này dùng trực tiếp lý thuyết Kantorovich-Rubinstein duality (một kết quả cốt lõi của optimal transport) mà không tự chứng minh chi tiết; nên đọc trước math 17 (Spectral Normalization for GANs) vì cả hai cùng giải quyết vấn đề ”ép ràng buộc Lipschitz cho discriminator/critic” nhưng math 17 là một cải tiến kỹ thuật thay cho weight clipping của bài này.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (bản arXiv); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc.

1. Bối cảnh

Bài báo bắt đầu từ câu hỏi học một phân phối xác suất có nghĩa là gì. Một cách tiếp cận là chọn họ mật độ có tham số $(P_\theta)_{\theta\in\mathbb{R}^d}$ và cực đại hóa likelihood trên dữ liệu: $\max_{\theta\in\mathbb{R}^d} \frac{1}{m}\sum_{i=1}^{m} \log P_\theta(x^{(i)})$. Nếu phân phối dữ liệu thật P_r có mật độ, trong giới hạn $m \to \infty$ việc này tương đương với tối thiểu hóa phân kỳ Kullback–Leibler $KL(P_r\|P_\theta)$.

Vấn đề: cách này cần **mật độ mô hình P_θ tồn tại**. Điều này không đúng trong tình huống phổ biến khi phân phối được hỗ trợ (supported) trên một **manifold chiều thấp** (low dimensional manifold) nhúng trong không gian chiều cao (ví dụ không gian ảnh). Khi đó, ”khó có khả năng manifold của mô hình và support của phân phối thật có giao nhau đáng kể”, khiến khoảng cách KL ”không được định nghĩa (hoặc đơn giản là vô hạn)”. Cách khắc phục thông thường là thêm nhiễu (noise) vào phân phối mô hình, nhưng paper chỉ ra chi phí thực tế của việc này: trích dẫn một paper khác cho thấy ”độ lệch chuẩn tối ưu của nhiễu thêm vào mô hình khi tối đa hoá likelihood là khoảng 0.1 cho mỗi pixel của ảnh sinh ra, khi pixel đã được chuẩn hoá về $[0, 1]$” - ”đây là một lượng nhiễu rất lớn, đến mức khi các paper báo cáo mẫu sinh ra từ mô hình, họ không thêm hạng tử nhiễu mà họ dùng để tính likelihood”. Paper nhận xét thẳng: ”hạng tử nhiễu thêm vào rõ ràng không đúng cho bài toán, nhưng cần thiết để phương pháp maximum likelihood hoạt động”.

Cách thay thế: thay vì ước lượng mật độ P_r (có thể không tồn tại), định nghĩa một biến ngẫu nhiên Z với phân phối cố định $p(z)$, cho qua một hàm tham số $g_\theta : \mathcal{Z} \to \mathcal{X}$ (thường là một mạng neural) để **sinh trực tiếp mẫu** theo một phân phối P_θ nào đó. VAE (Variational Auto-Encoders) và GAN (Generative Adversarial Networks) là hai ví dụ nổi tiếng của cách tiếp cận này. Vì VAE tập trung vào likelihood xấp xỉ, nó vẫn chia sẻ hạn chế cần thêm nhiễu; GAN cho phép linh hoạt hơn trong việc định nghĩa hàm mục tiêu (bao gồm Jensen-Shannon và các f-divergence khác), nhưng ”huấn luyện GAN được biết rộng rãi là tinh vi và bất ổn, với lý do được điều tra về mặt lý thuyết” trong một paper trước đó của cùng tác giả thứ nhất (dòng 70-71, trích [1] Arjovsky & Bottou 2017).

Trọng tâm của paper: các cách khác nhau để đo "độ gần" giữa phân phối mô hình và phân phối thật, tức là các cách định nghĩa khoảng cách hoặc phân kỳ $\rho(P_\theta, P_r)$. Sự khác biệt cơ bản nhất giữa các khoảng cách này là **ảnh hưởng tới sự hội tụ của dãy phân phối xác suất**: một dãy (P_t) hội tụ nếu và chỉ nếu tồn tại P_∞ sao cho $\rho(P_t, P_\infty) \to 0$, và điều này phụ thuộc vào cách định nghĩa ρ. Paper nói đại khái (informally): "một khoảng cách ρ tạo ra một tô-pô (topology) yếu hơn khi nó làm cho một dãy phân phối dễ hội tụ hơn". Mục tiêu then chốt: cần phép ánh xạ $\theta \mapsto P_\theta$ **liên tục** để hàm mất mát $\theta \mapsto \rho(P_\theta, P_r)$ liên tục (có thể dùng gradient descent hiệu quả); khoảng cách yếu hơn giúp ánh xạ này dễ liên tục hơn.

2. Ý tưởng chính

Ý tưởng cốt lõi: thay khoảng cách Jensen-Shannon (JS, dùng trong GAN gốc) bằng **khoảng cách Earth-Mover (EM)**, còn gọi là **khoảng cách Wasserstein-1**, để đo sự khác biệt giữa phân phối mô hình P_θ và phân phối thật P_r. Lý do: Wasserstein là một khoảng cách **yếu hơn hẳn** JS (paper chứng minh bằng Theorem 2, xem mục 3), nghĩa là nhiều dãy phân phối hội tụ dưới Wasserstein nhưng không hội tụ dưới JS/KL/TV. Cụ thể, với hai phân phối có **support rời nhau** (disjoint support, tình huống rất phổ biến khi dữ liệu nằm trên manifold chiều thấp), Wasserstein vẫn cho một khoảng cách **liên tục và có gradient khả dụng theo tham số**, còn JS/KL/TV cho giá trị hằng số (không đổi, gradient = 0) hoặc vô hạn.

Vì tính toán trực tiếp định nghĩa gốc của khoảng cách EM (một bài toán tối ưu vận tải, infimum trên mọi joint distribution) là **cực kỳ khó** (intractable), paper dùng **đối ngẫu Kantorovich-Rubinstein (Kantorovich-Rubinstein duality)**, một định lý từ lý thuyết optimal transport, để biến đổi bài toán infimum khó thành bài toán **supremum trên các hàm 1-Lipschitz**, dễ xấp xỉ bằng một mạng neural (gọi là **critic**, không còn gọi là "discriminator" vì nó không còn là bộ phân loại nhị phân, mà xuất ra một giá trị thực vô hướng).

Ý tưởng thứ hai, mang tính thực hành: để mạng neural xấp xỉ hàm 1-Lipschitz, paper dùng một kỹ thuật đơn giản là **weight clipping** (cắt trọng số về một khoảng cố định $[-c, c]$ sau mỗi lượt cập nhật gradient). Bài báo nêu đây là "một cách rõ ràng là tệ (clearly terrible way)" để ép ràng buộc Lipschitz, nhưng chọn vì đơn giản và "đã cho hiệu năng tốt", đồng thời khuyến khích nghiên cứu tiếp để cải thiện (điều mà math 17, Spectral Normalization, thực hiện sau đó một năm).

3. Phương pháp chi tiết

3.1 Các khoảng cách/phân kỳ cơ bản (§2)

Paper định nghĩa chính thức (với $\mathcal{X}$ là tập metric compact, ví dụ không gian ảnh $[0,1]^d$, Σ là tập các Borel subset của $\mathcal{X}$, Prob$(\mathcal{X})$ là không gian các độ đo xác suất trên $\mathcal{X}$):

- **Total Variation (TV)**: $\delta(P_r, P_g) = \sup_{A \in \Sigma} |P_r(A) - P_g(A)|$.
- **Kullback-Leibler (KL)**: $KL(P_r \| P_g) = \int \log\left(\frac{P_r(x)}{P_g(x)}\right) P_r(x)\, d\mu(x)$, giả định cả hai có mật độ theo cùng một độ đo μ. Bài báo nhấn mạnh: "KL nổi tiếng là bất đối xứng và có thể vô hạn khi có điểm $P_g(x) = 0$ mà $P_r(x) > 0$".
- **Jensen-Shannon (JS)**: $JS(P_r, P_g) = KL(P_r \| P_m) + KL(P_g \| P_m)$, với $P_m = (P_r + P_g)/2$. "Phân kỳ này đối xứng và luôn được định nghĩa vì có thể chọn $\mu = P_m$".
- **Earth-Mover (EM) / Wasserstein-1**:

$$W(P_r, P_g) = \inf_{\gamma \in \Pi(P_r, P_g)} \mathbb{E}_{(x,y)\sim\gamma}[\|x - y\|] \tag{1}$$

  với $\Pi(P_r, P_g)$ là tập mọi phân phối đồng thời (joint distribution) $\gamma(x, y)$ có hai biên (marginal) tương ứng là P_r và P_g. Giải thích trực giác (paper tự nói, dòng 164-167): "$\gamma(x, y)$ cho biết bao nhiêu 'khối lượng' (mass) cần được vận chuyển từ x sang y để biến đổi P_r thành P_g. Khoảng cách EM là 'chi phí' của kế hoạch vận tải tối ưu".

3.2 Ví dụ minh hoạ (Example 1, ”Learning parallel lines”)

Paper đưa ra một ví dụ cụ thể để minh hoạ sự khác biệt giữa các khoảng cách. Cho $Z \sim U[0,1]$ (phân phối đều trên đoạn đơn vị). P_0 là phân phối của $(0, Z) \in \mathbb{R}^2$ (một đường thẳng dọc đi qua gốc toạ độ). Đặt $g_\theta(z) = (\theta, z)$ với θ là một tham số thực đơn (đường thẳng dọc dịch sang phải/trái theo θ). Paper tính trực tiếp:

- $W(P_0, P_\theta) = |\theta|$.
- $JS(P_0, P_\theta) = \log 2$ nếu $\theta \neq 0$, $= 0$ nếu $\theta = 0$.
- $KL(P_\theta \| P_0) = KL(P_0 \| P_\theta) = +\infty$ nếu $\theta \neq 0$, $= 0$ nếu $\theta = 0$.
- $\delta(P_0, P_\theta) = 1$ nếu $\theta \neq 0$, $= 0$ nếu $\theta = 0$.

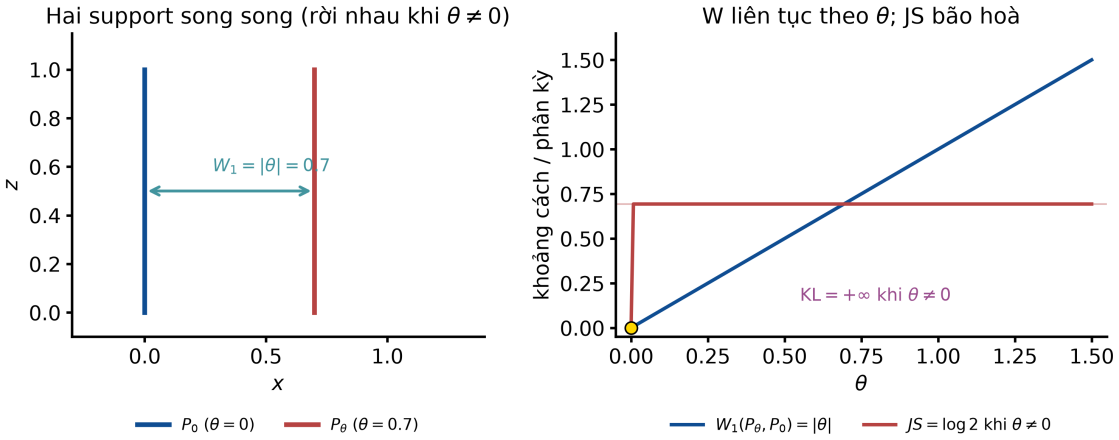
Khi $\theta_t \to 0$, dãy (P_{θ_t}) **hội tụ dưới khoảng cách EM** (vì $|\theta_t| \to 0$), nhưng **không hội tụ chút nào** dưới JS, KL, KL ngược, hoặc TV (vì các giá trị này nhảy đột ngột từ một hằng số dương về 0 chỉ tại $\theta = 0$, không có gì ”liên tục” khi θ_t tiến dần về 0). Paper tự kiểm chứng lại bằng tay (tự tính, kiểm tra tính hợp lý): với $\theta = 0.1$ (rất gần 0 nhưng khác 0), $W = 0.1$ (nhỏ, phản ánh đúng trực giác ”hai đường thẳng gần nhau”), nhưng $JS = \log 2 \approx 0.693$ (giống hệt trường hợp $\theta = 1$ rất xa, không phân biệt được mức độ gần) - đây chính là minh hoạ bằng số cụ thể cho việc JS ”bão hoà” (saturate) ngay khi hai phân phối có support rời nhau, bất kể chúng gần nhau bao nhiêu về khoảng cách Euclid.

Đây là một ví dụ có **support rời nhau** (hai đường thẳng khác θ không giao nhau trừ khi $\theta = 0$); paper nói thêm ”kết luận tương tự vẫn đúng khi support có giao nhau nhưng giao đó nằm trong một tập có độ đo 0”, tức là tình huống tổng quát hơn (hai manifold chiều thấp giao nhau ở vị trí tổng quát/general position) vẫn gặp cùng vấn đề.

3.3 Theorem 1 và Theorem 2 (tính liên tục và quan hệ thứ bậc giữa các khoảng cách)

Theorem 1: cho P_r cố định, $g_\theta(z)$ là một hàm (thường là mạng neural). Thì:

1. Nếu g liên tục theo θ, thì $W(P_r, P_\theta)$ cũng liên tục.
2. Nếu g liên tục Lipschitz cục bộ (locally Lipschitz) và thoả một giả định chính quy (Assumption 1, không đi sâu chi tiết kỹ thuật trong ghi chú này), thì $W(P_r, P_\theta)$ liên tục ở mọi nơi, và khả vi hầu khắp nơi (differentiable almost everywhere).
3. Hai phát biểu (1), (2) **sai** đối với $JS(P_r, P_\theta)$ và mọi KL.



Hai support song song: $W_1 = |\theta|$ liên tục theo θ, trong khi JS bão hoà ở log 2 và KL=+∞ khi θ≠0. (minh hoạ, tự tính)

Corollary 1 (hệ quả quan trọng cho thực hành): nếu g_θ là bất kỳ mạng neural feedforward (định nghĩa: ”hợp các phép biến đổi affine và phi tuyến điểm-theo-điểm mượt và Lipschitz như sigmoid, tanh, elu, softplus”, chú thích 4, dòng 229-232; paper ghi chú thêm ”phát biểu cũng đúng với rectifier nonlinearity nhưng chứng

minh kỹ thuật hơn nên bỏ qua"), và $p(z)$ là một prior sao cho $\mathbb{E}_{z\sim p(z)}[\|z\|] < \infty$ (ví dụ Gaussian, uniform), thì $W(P_r, P_\theta)$ liên tục ở mọi nơi và khả vi hầu khắp nơi. Đây chính là cơ sở lý thuyết cho việc "an toàn" khi dùng gradient descent trên Wasserstein distance với một mạng neural bất kỳ làm generator.

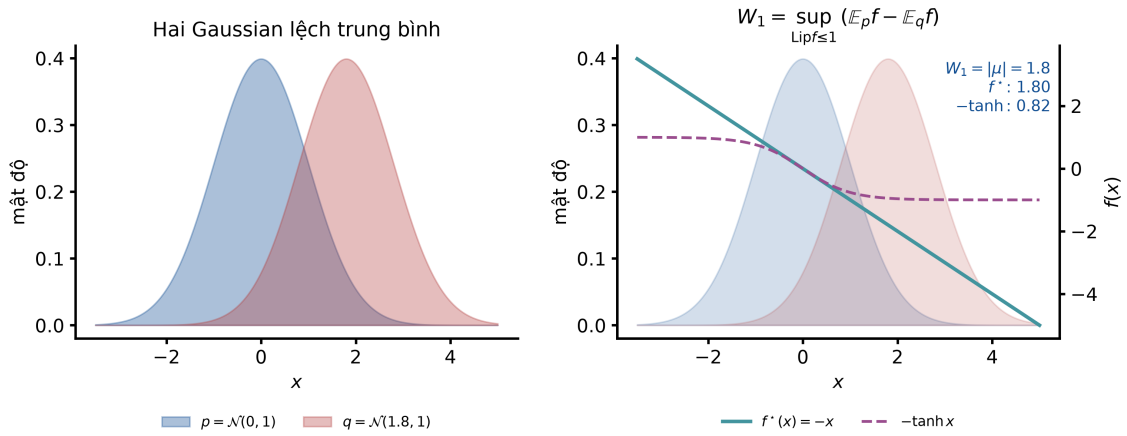
Theorem 2 mô tả **thứ bậc độ mạnh (relative strength)** của các tô-pô sinh ra bởi các khoảng cách này, "KL mạnh nhất, tiếp theo là JS và TV, và EM yếu nhất":

1. $\delta(P_n, P) \to 0 \Leftrightarrow JS(P_n, P) \to 0$ (TV và JS tương đương).
2. $W(P_n, P) \to 0 \Leftrightarrow P_n \xrightarrow{D} P$ (hội tụ Wasserstein tương đương hội tụ theo phân phối, convergence in distribution).
3. $KL(P_n\|P) \to 0$ hoặc $KL(P\|P_n) \to 0 \Rightarrow$ phát biểu (1).
4. Phát biểu (1) $\Rightarrow$ phát biểu (2).

Kết luận rút ra (paper tự nói, dòng 260-262): "điều này nhấn mạnh rằng KL, JS, TV không phải là hàm mất mát hợp lý khi học các phân phối được hỗ trợ trên manifold chiều thấp. Nhưng khoảng cách EM hợp lý trong tình huống đó".

3.4 Đối ngẫu Kantorovich-Rubinstein và thuật toán WGAN (§3)

Vì infimum trong định nghĩa (1) "cực kỳ khó tính" (highly intractable, dòng 269), paper dùng **đối ngẫu Kantorovich-Rubinstein** (trích dẫn [22], không chứng minh trong bài này, dẫn người đọc tới Appendix A để hiểu nguồn gốc trực giác):



Đối ngẫu Kantorovich–Rubinstein: W$_1$=sup_{Lip f≤1}(E_p f−E_q f); f*=−x đạt |μ|, tanh kém hơn. (minh hoạ, tự tính)

$$W(P_r, P_\theta) = \sup_{\|f\|_L \le 1} \mathbb{E}_{x\sim P_r}[f(x)] - \mathbb{E}_{x\sim P_\theta}[f(x)] \tag{2}$$

trong đó supremum lấy trên mọi hàm **1-Lipschitz** $f : \mathcal{X} \to \mathbb{R}$. Nếu thay $\|f\|_L \le 1$ bằng $\|f\|_L \le K$ (K-Lipschitz), ta được $K \cdot W(P_r, P_g)$ (chỉ khác một hằng số nhân). Nếu có một họ hàm tham số $\{f_w\}_{w\in\mathcal{W}}$ đều là K-Lipschitz với K nào đó, có thể giải bài toán:

$$\max_{w\in\mathcal{W}} \mathbb{E}_{x\sim P_r}[f_w(x)] - \mathbb{E}_{z\sim p(z)}[f_w(g_\theta(z))] \tag{3}$$

Nếu supremum trong (2) đạt được tại một $w \in \mathcal{W}$ nào đó (một giả định mạnh, "tương tự giả định khi chứng minh tính nhất quán (consistency) của một ước lượng", dòng 286-287), quá trình này cho một tính toán của $W(P_r, P_\theta)$ (sai một hằng số nhân).

Theorem 3: cho P_r bất kỳ, P_θ là phân phối của $g_\theta(Z)$ thoả Assumption 1. Thì tồn tại nghiệm f cho bài toán cực đại $\max_{\|f\|_L \leq 1} \mathbb{E}_{x\sim P_r}[f(x)] - \mathbb{E}_{x\sim P_\theta}[f(x)]$, và:

$$\nabla_\theta W(P_r, P_\theta) = -\mathbb{E}_{z\sim p(z)}[\nabla_\theta f(g_\theta(z))]$$

khi cả hai vế được định nghĩa tốt. Đây là cơ sở lý thuyết cho việc **lan truyền ngược trực tiếp** qua công thức đối ngẫu để huấn luyện generator.

Thuật toán WGAN cụ thể (Algorithm 1) - để f_w nằm trong không gian compact $\mathcal{W}$ (điều kiện cần cho Lipschitz với hằng số K phụ thuộc $\mathcal{W}$, không phụ thuộc trọng số cụ thể), paper dùng cách đơn giản nhất: **cắt (clamp) trọng số về một hộp cố định** $\mathcal{W} = [-c, c]^l$ sau mỗi lượt cập nhật gradient. Mã giả (paraphrase từ Algorithm 1, dòng 329-351), với giá trị mặc định dùng cho mọi thí nghiệm trong bài: learning rate $\alpha = 0.00005$, clipping parameter $c = 0.01$, batch size $m = 64$, $n_{critic} = 5$ (số lượt cập nhật critic cho mỗi lượt cập nhật generator):

Trong khi θ chưa hội tụ:

- Với $t = 0, \ldots, n_{critic}$: lấy mẫu batch dữ liệu thật $\{x^{(i)}\}$ và batch prior $\{z^{(i)}\}$; tính $g_w \leftarrow \nabla_w \left[\frac{1}{m}\sum f_w(x^{(i)}) - \frac{1}{m}\sum f_w(g_\theta(z^{(i)}))\right]$; cập nhật $w \leftarrow w + \alpha \cdot \text{RMSProp}(w, g_w)$; cắt $w \leftarrow \text{clip}(w, -c, c)$.
- Lấy mẫu batch prior mới; tính $g_\theta \leftarrow -\nabla_\theta \frac{1}{m}\sum f_w(g_\theta(z^{(i)}))$; cập nhật $\theta \leftarrow \theta - \alpha \cdot \text{RMSProp}(\theta, g_\theta)$.

Lưu ý quan trọng: critic được cập nhật dùng **gradient tăng** (ascent, dấu $+$), còn generator dùng **gradient giảm** trên âm của hàm mục tiêu (dấu $-\nabla_\theta$) - khác biệt dấu này phản ánh đúng cấu trúc $\max_w \min_\theta$ (hoặc tương đương) của bài toán gốc.

Paper **tự phê bình thẳng** kỹ thuật weight clipping ngay sau khi trình bày: ”Weight clipping rõ ràng là một cách tệ để ép ràng buộc Lipschitz. Nếu tham số clipping lớn, có thể mất nhiều thời gian để bất kỳ trọng số nào đạt tới giới hạn của nó, khiến việc huấn luyện critic tới tối ưu khó hơn. Nếu clipping nhỏ, điều này có thể dễ dẫn tới gradient biến mất khi số lớp lớn, hoặc khi không dùng batch normalization (như trong RNN)”. Paper cũng nói đã thử ”các biến thể đơn giản (như chiếu trọng số lên một mặt cầu) với ít khác biệt, và giữ nguyên weight clipping vì sự đơn giản và hiệu năng đã tốt”, đồng thời ”để lại chủ đề ép ràng buộc Lipschitz trong bối cảnh mạng neural cho nghiên cứu sau này, và tích cực khuyến khích các nhà nghiên cứu quan tâm cải thiện phương pháp này” - đây chính là lời ”mời gọi” trực tiếp mà các paper sau (như Spectral Normalization, math 17) đáp lại.

3.5 Lý do huấn luyện critic tới tối ưu lại có ích (§3, cuối)

Vì khoảng cách EM ”liên tục và khả vi hầu khắp nơi” (continuous and differentiable a.e.), paper lập luận ”ta có thể (và nên) huấn luyện critic tới tối ưu. Lý giải đơn giản: càng huấn luyện critic nhiều, gradient của Wasserstein mà ta nhận được càng đáng tin cậy”. Ngược lại, với JS, ”khi discriminator tốt hơn, gradient trở nên đáng tin cậy hơn nhưng gradient thật lại bằng 0 vì JS bị bão hoà cục bộ (locally saturated), dẫn tới vanishing gradient”. Figure 2 của paper minh hoạ: khi học phân biệt hai phân phối Gaussian, ”discriminator học rất nhanh cách phân biệt fake và real, và như dự đoán không cung cấp thông tin gradient đáng tin cậy. Critic, tuy nhiên, không thể bão hoà, và hội tụ về một hàm tuyến tính cho gradient rất sạch ở mọi nơi”. Paper giải thích lý do kỹ thuật: ”việc ràng buộc trọng số giới hạn mức tăng trưởng tối đa của hàm là tuyến tính ở các phần khác nhau của không gian, buộc critic tối ưu phải có hành vi này”.

Một lợi ích thứ hai, ”có thể quan trọng hơn”: việc huấn luyện critic tới tối ưu làm **không thể xảy ra mode collapse** khi làm vậy, vì ”mode collapse xuất phát từ việc generator tối ưu cho một discriminator cố định là một tổng các hàm delta tại các điểm discriminator gán giá trị cao nhất” (dòng 370-373, trích dẫn hiện tượng đã quan sát trong [4] Goodfellow 2014 gốc và nhấn mạnh lại trong [11]).

4. Thí nghiệm và kết quả chính

Thực nghiệm sinh ảnh trên **LSUN-Bedrooms** (bộ sưu tập ảnh phòng ngủ), so sánh với baseline **DCGAN** (GAN tích chập huấn luyện theo thủ tục GAN chuẩn dùng mẹo $-\log D$). Mẫu sinh ra là ảnh 3 kênh màu 64x64 pixel. Dùng đúng hyperparameter mặc định ở Algorithm 1 cho mọi thí nghiệm.

Tuyên bố hai lợi ích chính (§4): (a) một thước đo loss có ý nghĩa, tương quan với sự hội tụ của generator và chất lượng mẫu; (b) cải thiện độ ổn định của quá trình tối ưu.

4.2 Thước đo loss có ý nghĩa (Figure 3, 4): vì WGAN huấn luyện critic f tương đối tốt trước mỗi lượt cập nhật generator, hàm loss tại điểm đó là một ước lượng của khoảng cách EM (sai một hằng số nhân phụ thuộc cách ràng buộc Lipschitz của f). Paper thử 3 kiến trúc: DCGAN tích chập, generator là MLP 4 lớp ẩn 512 unit mỗi lớp (critic DCGAN), và cả generator+critic đều là MLP (learning rate cao để huấn luyện thất bại có chủ đích). Kết quả (Figure 3): ”các biểu đồ cho thấy rõ các đường cong này tương quan tốt với chất lượng trực quan của mẫu sinh ra”. Bài báo lưu ý đây là phát hiện đáng chú ý: ”theo hiểu biết của chúng tôi, đây là lần đầu tiên trong văn liệu GAN một thuộc tính như vậy được chỉ ra, nơi loss của GAN thể hiện tính chất hội tụ”. Ngược lại, Figure 4 cho JS estimate của GAN chuẩn: ”cả hai đường cong đều có lỗi tăng dần. Mẫu tốt hơn cho DCGAN nhưng ước lượng JS tăng hoặc giữ không đổi, cho thấy không có tương quan đáng kể giữa chất lượng mẫu và loss”; ”ước lượng JS thường vẫn gần $\log 2 \approx 0.69$, giá trị cao nhất mà khoảng cách JS đạt được” (dòng 469-471, con số $\log 2 \approx 0.693$ khớp chính xác với giá trị tính ở Example 1 mục 3.2, tự kiểm chứng: $\ln 2 \approx 0.6931$).

Paper tự cảnh báo giới hạn của tuyên bố này ngay sau: ”chúng tôi không khẳng định đây là một phương pháp mới để đánh giá định lượng mô hình sinh. Hằng số tỷ lệ phụ thuộc kiến trúc của critic khiến khó so sánh các mô hình có critic khác nhau. Hơn nữa, trong thực tế việc critic không có capacity vô hạn khiến khó biết chính xác ước lượng của chúng ta gần khoảng cách EM thật đến mức nào”.

Một kết quả tiêu cực paper tự báo cáo: ”huấn luyện WGAN trở nên bất ổn đôi lúc khi dùng một optimizer dựa trên momentum như Adam (với $\beta_1 > 0$) trên critic, hoặc khi dùng learning rate cao. Vì loss cho critic không ổn định theo thời gian (nonstationary), các phương pháp dựa trên momentum có vẻ hoạt động kém hơn”. Paper xác định nguyên nhân khả dĩ bằng quan sát thực nghiệm cụ thể: ”khi loss nổ lên và mẫu xấu đi, cosine giữa bước Adam và gradient thường trở nên âm. Nơi duy nhất cosine này âm là trong các tình huống bất ổn này” - đây là một chẩn đoán thực nghiệm cụ thể, dẫn tới quyết định ”chuyển sang RMSProp, được biết hoạt động tốt ngay cả trên các bài toán rất nonstationary”.

4.3 Cải thiện độ ổn định (Figure 5, 6, 7): thử 3 kiến trúc generator: (1) DCGAN tích chập chuẩn, (2) DCGAN tích chập **không** batch normalization và số filter không đổi qua các lớp, (3) MLP ReLU 4 lớp 512 unit. Hai kiến trúc cuối ”được biết hoạt động rất kém với GAN”. Kết quả: với kiến trúc (2), ”GAN chuẩn thất bại trong việc học trong khi WGAN vẫn có thể tạo mẫu” (dòng 522-523, chú thích Figure 6). Với kiến trúc (3) MLP, ”phương pháp WGAN vẫn có thể tạo mẫu, chất lượng thấp hơn DCGAN, và cao hơn MLP của GAN chuẩn. Lưu ý mức độ mode collapse đáng kể trong GAN MLP” (dòng 531-533, chú thích Figure 7). Kết luận tổng quát mà bài báo nhấn mạnh bằng chữ đậm trong văn bản gốc: ”**Trong không thí nghiệm nào chúng tôi thấy bằng chứng của mode collapse cho thuật toán WGAN**”.

5. Phân tích / ablation

Paper không có một bảng ”ablation” theo nghĩa thống kê định lượng (so sánh số liệu qua nhiều cấu hình trong một bảng), nhưng có hai dạng phân tích tương đương:

1. **So sánh kiến trúc generator (độ bền kiến trúc)**: Figure 5, 6, 7 chính là một ablation theo kiến trúc - cố định critic/discriminator (DCGAN) và thay đổi generator qua 3 mức độ ”khó” tăng dần (DCGAN chuẩn → DCGAN không BN → MLP thuần). Kết quả cho thấy WGAN **bền (robust)** hơn hẳn GAN

chuẩn khi kiến trúc generator trở nên ”khó” hơn, trong khi GAN chuẩn thất bại hoàn toàn ở kiến trúc (2) và mode-collapse nặng ở kiến trúc (3).

2. **So sánh optimizer (momentum vs RMSProp)**: phát hiện ở dòng 479-487 (đã nêu ở mục 4) là một dạng ablation về lựa chọn optimizer, cho thấy tính nonstationary của bài toán WGAN nhạy với momentum, dẫn tới khuyến nghị thực hành cụ thể (dùng RMSProp, tránh Adam với $\beta_1 > 0$ hoặc learning rate cao).

Phần §5 ”Related Work” cũng đóng vai trò như một phân tích lý thuyết đối chiếu (không phải ablation thực nghiệm): paper định nghĩa khung tổng quát **Integral Probability Metric (IPM)**: $d_{\mathcal{F}}(P_r, P_\theta) = \sup_{f \in \mathcal{F}} \mathbb{E}_{P_r}[f] - \mathbb{E}_{P_\theta}[f]$ (công thức 4), và chỉ ra các lựa chọn khác nhau của $\mathcal{F}$ cho các khoảng cách rất khác nhau: $\mathcal{F}$ = hàm 1-Lipschitz → Wasserstein (qua Kantorovich-Rubinstein); $\mathcal{F}$ = hàm đo được bị chặn trong $[-1, 1]$ → **total variation** δ (bài báo nhấn mạnh: ”điều này cho thấy đi từ 1-Lipschitz sang 1-Bounded thay đổi mạnh tô-pô của không gian”, dòng 560-562); Energy-based GAN (EBGAN) được paper chứng minh (Appendix D, không đọc chi tiết trong ghi chú này) tương đương với cách tiếp cận sinh dựa trên total variation, nên ”sẽ gặp cùng vấn đề của GAN cổ điển về việc không thể huấn luyện discriminator tới tối ưu”; Maximum Mean Discrepancy (MMD) là trường hợp $\mathcal{F} = \{f \in \mathcal{H} : \|f\|_\infty \leq 1\}$ cho một không gian Hilbert tái tạo nhân (RKHS) - paper phân tích chi tiết chi phí tính toán MMD tăng theo bậc hai với số mẫu, và tính toán cụ thể ”cho ảnh chuẩn 64x64, cần minibatch kích thước ít nhất 4096... chi phí mỗi vòng lặp là 4096^2, hơn 5 bậc độ lớn so với một vòng lặp GAN dùng batch size chuẩn 64” (dòng 620-625, tự kiểm chứng: $4096^2 = 16.777.216$, so với $64^2 = 4096$ của MMD ở batch 64 chuẩn, tỷ lệ $4096/1 \approx 4096$ lần, xấp xỉ $\log_{10}(4096) \approx 3.6$ bậc độ lớn chỉ riêng từ kích thước batch, cộng thêm hệ số khác theo paper nói ”hơn 5 bậc”; tự tính).

6. Giới hạn

Các giới hạn **bài báo nêu rõ**, không phải suy diễn của người ghi chú:

1. Weight clipping ”rõ ràng là một cách tệ” để ép ràng buộc Lipschitz: clipping parameter lớn → chậm đạt giới hạn, khó huấn luyện critic tới tối ưu; clipping nhỏ → dễ vanishing gradient khi mạng sâu hoặc không dùng batch normalization (ví dụ RNN).
2. Bài báo lưu ý: ”chúng tôi không khẳng định đây là một phương pháp mới để đánh giá định lượng mô hình sinh” - loss WGAN chỉ tương quan tốt với chất lượng mẫu, không phải một chỉ số tuyệt đối có thể so sánh giữa các mô hình có kiến trúc critic khác nhau, vì hằng số tỷ lệ (scaling constant) phụ thuộc kiến trúc.
3. ”Trong thực tế việc critic không có capacity vô hạn khiến khó biết chính xác ước lượng của chúng ta gần khoảng cách EM thật đến mức nào” - đây là một giới hạn nhận thức luận quan trọng: WGAN chỉ tính được một **xấp xỉ** của Wasserstein distance thật, không phải giá trị chính xác.
4. Huấn luyện WGAN có thể bất ổn khi dùng optimizer dựa trên momentum (Adam với $\beta_1 > 0$) hoặc learning rate cao trên critic (đã nêu ở mục 4, ”kết quả tiêu cực” paper tự báo cáo).
5. Giả định ”supremum trong (2) đạt được tại một $w \in \mathcal{W}$ nào đó” để công thức (3) tính đúng $W(P_r, P_\theta)$ (sai hằng số nhân) được paper tự gọi là ”một giả định khá mạnh, tương tự giả định khi chứng minh tính nhất quán của một ước lượng” - không được chứng minh luôn đúng trong thực hành với mạng neural cụ thể.
6. Về mặt lý thuyết, Corollary 1 chỉ chứng minh cho activation ”mượt và Lipschitz” (sigmoid, tanh, elu, softplus); với rectifier (ReLU), paper nói ”phát biểu cũng đúng nhưng chứng minh kỹ thuật hơn (dù rất tương tự) nên bỏ qua” (chú thích 4, dòng 232-233) - tức chứng minh đầy đủ cho ReLU **không có trong bài báo này**, chỉ được khẳng định bằng lời.
7. Paper tự để lại ”chủ đề ép ràng buộc Lipschitz trong bối cảnh mạng neural cho nghiên cứu sau này” - weight clipping được paper trình bày như một giải pháp tạm, không phải giải pháp cuối cùng.

7. Liên hệ với các bài khác

- **ml basics content/14_2014_gan.md**: GAN gốc (Goodfellow et al. 2014) là nền tảng trực tiếp mà bài này cải tiến; GAN gốc dùng JS divergence (qua công thức cross-entropy chuẩn), bài này thay bằng Wasserstein distance để giải quyết vấn đề gradient biến mất khi discriminator ”quá tốt”.
- **math 12 (Computational Optimal Transport)**: Bài hiện tại dùng trực tiếp đối ngẫu Kantorovich-Rubinstein (công thức 2) như một ”hộp đen” lý thuyết (không tự chứng minh, dẫn người đọc tới Appendix A và trích dẫn [22]); math 12 là nơi lý thuyết optimal transport tổng quát (bao gồm chính định lý Kantorovich-Rubinstein duality) được trình bày đầy đủ hơn.
- **math 17 (Spectral Normalization for GANs, 1802.05957)**: Math 17 chính là lời đáp trực tiếp cho lời ”kêu gọi nghiên cứu tiếp” của bài hiện tại: thay weight clipping (gây rank-deficiency theo phân tích của math 17 §3) bằng chuẩn hoá giá trị kỳ dị lớn nhất, đạt ràng buộc Lipschitz hiệu quả hơn.
- **ml basics content/15_2013_vae_aevb.md**: VAE là mô hình sinh thay thế dùng likelihood xấp xỉ (ELBO), được paper này nhắc tới ở phần Bối cảnh như một ví dụ khác của cách tiếp cận ”sinh mẫu qua hàm g_θ”, chia sẻ hạn chế cần thêm nhiễu mà GAN/WGAN không gặp phải.

8. Gợi ý áp dụng

WGAN minh họa cách lựa chọn khoảng cách giữa các phân phối ảnh hưởng đến tín hiệu tối ưu của mô hình sinh. Giá trị critic chỉ là một ước lượng trong điều kiện ràng buộc và tối ưu cụ thể, không phải thước đo chất lượng tuyệt đối giữa mọi mô hình.

9. Thí nghiệm gợi ý

1. **Tái tạo Example 1 (hai đường thẳng song song) bằng tay và bằng code**: viết hàm Python tính cả 4 khoảng cách (EM/Wasserstein, JS, KL, TV) cho hai phân phối P_0 và P_θ như định nghĩa ở mục 3.2, với θ chạy từ 0.001 đến 1, vẽ biểu đồ 4 đường. Kết quả kỳ vọng: đường Wasserstein là một đường thẳng $|\theta|$ liên tục qua gốc; các đường JS/KL/TV có bước nhảy gián đoạn ngay tại $\theta = 0^+$. Diễn giải: minh hoạ trực quan, tự làm, cho đúng bằng chứng cốt lõi của paper (Theorem 1-2) mà không cần GPU, chạy trên CPU trong vài giây.

2. **Toy GAN 1D/2D: JS GAN vs WGAN trên hai phân phối không chồng lấp**: tạo dữ liệu thật là hỗn hợp 2 Gaussian hẹp cách xa nhau trong không gian 2D, generator là MLP nhỏ, huấn luyện cả hai phiên bản (GAN chuẩn dùng JS, và WGAN dùng critic + weight clipping) trên GPU hoặc CPU (bài toán rất nhỏ). Theo dõi gradient của generator qua các bước đầu huấn luyện. Kết quả kỳ vọng: GAN chuẩn có gradient rất nhỏ/không ổn định ở giai đoạn đầu (khi hai phân phối rời xa), WGAN cho gradient ổn định hơn và giảm đều. Diễn giải: tái hiện trực tiếp phát hiện cốt lõi của Figure 2 trong paper (discriminator bão hoà vs critic cho gradient sạch) ở quy mô tự kiểm soát hoàn toàn.

3. **Kiểm tra tương quan loss-chất lượng mẫu (tái hiện Figure 3-4 ở quy mô nhỏ)**: huấn luyện WGAN và DCGAN chuẩn trên MNIST (nhỏ, nhanh trên GPU), theo dõi đồng thời: (a) giá trị loss/ước lượng khoảng cách của mỗi thuật toán qua các epoch, (b) một chỉ số chất lượng mẫu đơn giản (ví dụ accuracy của một bộ phân loại MNIST có sẵn khi phân loại ảnh sinh ra, hoặc đơn giản là xem mẫu bằng mắt theo epoch). Kết quả kỳ vọng: với WGAN, đường loss giảm dần tương quan với chất lượng mẫu tăng dần; với GAN chuẩn, loss discriminator có thể không tương quan rõ (như Figure 4 của paper). Diễn giải: xác nhận hoặc phản biện tuyên bố ”thước đo loss có ý nghĩa” của paper trên một bài toán nhỏ hơn hẳn (MNIST thay LSUN-Bedrooms).

4. **Ảnh hưởng của clipping constant** c: huấn luyện WGAN trên một bài toán toy (ví dụ MNIST hoặc dữ liệu 2D tổng hợp) với vài giá trị c khác nhau (ví dụ 0.001, 0.01, 0.1, 1.0), đo tốc độ hội tụ và độ ổn định huấn luyện (có ”nổ” loss hay không). Kết quả kỳ vọng: c quá nhỏ làm chậm học (giống cảnh báo ”vanishing gradient” của paper ở dòng 319-320); c quá lớn làm critic mất nhiều lượt cập nhật để ”chạm”

giới hạn (giống cảnh báo dòng 317-318). Diễn giải: xác nhận trực tiếp bằng thực nghiệm chính lời tự phê bình của paper về weight clipping, và cho thấy tại sao cần một kỹ thuật tốt hơn (dẫn tới động lực học math 17).

5. **So sánh Adam vs RMSProp cho critic WGAN**: lặp lại thí nghiệm nhỏ ở mục 2/3, huấn luyện critic bằng Adam (với $\beta_1 = 0.9$, giống mặc định phổ biến) và bằng RMSProp (không momentum), theo dõi độ ổn định (loss có ”nổ” đột ngột hay không), và nếu có điều kiện, tính cosine similarity giữa bước cập nhật Adam và gradient thô như paper đã làm để tìm mối liên hệ giữa cosine âm và thời điểm bất ổn. Kết quả kỳ vọng: cấu hình Adam với $\beta_1 > 0$ dễ bất ổn hơn RMSProp trên bài toán WGAN, nhất là ở learning rate cao. Diễn giải: tái hiện trực tiếp ”kết quả tiêu cực” mà paper tự báo cáo, giúp hiểu rõ bản chất ”nonstationary” của bài toán tối ưu WGAN.

10. Tóm tắt công thức / khái niệm

- Khoảng cách Earth-Mover/Wasserstein-1: $W(P_r, P_g) = \inf_{\gamma \in \Pi(P_r, P_g)} \mathbb{E}_{(x,y)\sim\gamma}[\|x - y\|]$ (công thức 1).
- Đối ngẫu Kantorovich-Rubinstein: $W(P_r, P_\theta) = \sup_{\|f\|_L \le 1} \mathbb{E}_{P_r}[f(x)] - \mathbb{E}_{P_\theta}[f(x)]$ (công thức 2); với K-Lipschitz, kết quả nhân thêm K.
- Mục tiêu thực hành WGAN: $\max_{w \in \mathcal{W}} \mathbb{E}_{x\sim P_r}[f_w(x)] - \mathbb{E}_{z\sim p(z)}[f_w(g_\theta(z))]$ (công thức 3).
- Gradient của Wasserstein theo tham số generator (Theorem 3): $\nabla_\theta W(P_r, P_\theta) = -\mathbb{E}_{z\sim p(z)}[\nabla_\theta f(g_\theta(z))]$.
- Weight clipping: $w \leftarrow \text{clip}(w, -c, c)$ sau mỗi lượt cập nhật, giữ $w \in \mathcal{W} = [-c, c]^l$ compact (siêu tham số mặc định $c = 0.01$, $\alpha = 0.00005$, $m = 64$, $n_{critic} = 5$).
- Integral Probability Metric (IPM) tổng quát: $d_{\mathcal{F}}(P_r, P_\theta) = \sup_{f\in\mathcal{F}} \mathbb{E}_{P_r}[f] - \mathbb{E}_{P_\theta}[f]$ (công thức 4); $\mathcal{F}$ = 1-Lipschitz cho Wasserstein, $\mathcal{F}$ = bị chặn $[-1, 1]$ cho total variation.
- Ví dụ tự kiểm chứng (Example 1 của paper, mục 3.2): với $\theta = 0.1$, $W = 0.1$ nhưng $JS = \log 2 \approx 0.6931$ (giống $\theta = 1$), minh hoạ JS ”bão hoà” bất kể mức độ gần của hai phân phối có support rời nhau (tự tính xác nhận giá trị $\ln 2 \approx 0.6931$ khớp số paper nêu ≈ 0.69).
- Thứ bậc độ mạnh tô-pô (Theorem 2): KL (mạnh nhất) $\Rightarrow$ (TV $\Leftrightarrow$ JS) $\Rightarrow$ (Wasserstein $\Leftrightarrow$ hội tụ theo phân phối, yếu nhất).

11. Kết luận ngắn

Bài báo đề xuất thay khoảng cách Jensen-Shannon trong GAN chuẩn bằng khoảng cách Earth-Mover/Wasserstein-1, chứng minh bằng lý thuyết (Theorem 1-2, Example 1) rằng Wasserstein tạo ra một tô-pô yếu hơn hẳn, giữ được tính liên tục/khả vi của hàm mất mát ngay cả khi hai phân phối có support rời nhau (tình huống phổ biến khi học manifold chiều thấp). Nhờ đối ngẫu Kantorovich-Rubinstein, bài toán tối thiểu hoá Wasserstein khó tính trực tiếp được chuyển thành bài toán tối đa hoá trên hàm 1-Lipschitz, xấp xỉ bằng một mạng ”critic” và ép Lipschitz bằng weight clipping (kỹ thuật mà bài báo lưu ý là chưa tối ưu). Thực nghiệm trên LSUN-Bedrooms xác nhận hai lợi ích chính: loss có ý nghĩa tương quan với chất lượng mẫu (lần đầu trong văn liệu GAN theo bài báo lưu ý), và độ ổn định cải thiện rõ rệt, không quan sát thấy mode collapse trong bất kỳ thí nghiệm nào, dù vẫn còn hạn chế về độ ổn định khi dùng optimizer có momentum.

Nguồn gốc: https://arxiv.org/abs/1701.07875

Bài 14

Deep Variational Information Bottleneck (2016)

Tác giả: Alexander A. Alemi, Ian Fischer, Joshua V. Dillon, Kevin Murphy (Google Research).
Venue / arXiv: arXiv:1612.00410 (v7, 23/10/2019); paper tự ghi ”Published as a conference paper at ICLR 2017” ở đầu mỗi trang.
Nguồn: https://arxiv.org/abs/1612.00410
Chủ đề: xây dựng một cận dưới biến phân (variational lower bound) khả vi cho mục tiêu Information Bottleneck (IB) của Tishby et al. (1999), cho phép huấn luyện bằng mạng nơ-ron sâu và reparameterization trick; mục tiêu là học một encoding ngẫu nhiên Z vừa nén thông tin dư thừa từ input X vừa giữ thông tin cần cho nhãn Y.
Vị trí trong lộ trình: đọc sau math 11 (SGD as Approximate Bayesian Inference) và math 10 (Hamiltonian Monte Carlo), vì ba bài cùng nằm trong nhóm ”suy luận xấp xỉ/biến phân cho deep learning”, nhưng bài này áp mục tiêu biến phân vào một hàm mất mát huấn luyện mới (IB) chứ không áp vào việc lấy mẫu hậu nghiệm của tham số; đọc cùng nhóm với ml basics 15 (Auto-Encoding Variational Bayes, VAE) vì VIB dùng lại nguyên khối ”encoder ngẫu nhiên + reparameterization trick + KL divergence tới prior” của VAE nhưng trong bài toán có giám sát (supervised) thay vì sinh dữ liệu (generative); nên đọc cùng `pruning content/` và `interp content/` vì khái niệm ”nén biểu diễn trung gian nhưng giữ thông tin cho nhãn” là một khung lý thuyết hữu ích để nghĩ về việc cắt bớt token/kênh trong VLM.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (bản arXiv); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc.

1. Bối cảnh

Bài báo xét mạng nơ-ron sâu dưới góc nhìn lý thuyết thông tin: biểu diễn của một lớp trung gian là mã hóa ngẫu nhiên Z của đầu vào X, sinh bởi encoder có tham số $p(z|x;\theta)$. Thông tin tương hỗ $I(\cdot,\cdot)$ đo lượng bất định về một biến được giảm đi khi biết biến còn lại. Công thức (1) của bài là:

$$I(Z,Y;\theta) = \int dx\, dy\; p(z,y|\theta)\; \log \frac{p(z,y|\theta)}{p(z|\theta)p(y|\theta)}.$$

(Ghi chú: trong tích phân trên z cũng được tích hợp ẩn, công thức paper viết hơi rút gọn; ý nghĩa là $I(Z,Y) = \mathbb{E}_{p(z,y)} \log \frac{p(z,y)}{p(z)p(y)}$, tích phân trên cả y và z.)

Mục tiêu tưởng chừng hợp lý: học encoding Z **tối đa hoá** $I(Z,Y;\theta)$, thông tin tương hỗ giữa encoding và nhãn Y. Nhưng paper chỉ ra một cái bẫy: theo *data processing inequality* (thông tin không thể tăng khi đi qua một hàm xử lý) và tính bất biến của mutual information qua các phép biến đổi khả nghịch (reparameterization), nếu chỉ tối đa hoá $I(Z,Y)$ mà không có ràng buộc gì khác, cách ”tối ưu” nhất đơn giản là lấy $Z = X$ (encoding đồng nhất, identity encoding): giữ nguyên toàn bộ input. Đây không phải một biểu diễn hữu ích (không nén được gì).

Do đó cần thêm một **ràng buộc về độ phức tạp (complexity constraint)** của encoding: giới hạn $I(X,Z) \le I_c$, với I_c gọi là information constraint. Mục tiêu đầy đủ (công thức 2):

$$\max_{\theta} I(Z,Y;\theta) \quad \text{s.t.} \quad I(X,Z;\theta) \le I_c.$$

Dùng Lagrange multiplier β, bài toán ràng buộc trên tương đương tối đa hoá (công thức 3):

$$R_{IB}(\theta) = I(Z, Y; \theta) - \beta I(Z, X; \theta).$$

Đây chính là mục tiêu **Information Bottleneck (IB)** gốc của Tishby et al. (1999): số hạng thứ nhất khuyến khích Z dự đoán tốt Y (expressive), số hạng thứ hai khuyến khích Z "quên" X (compressive); $\beta \geq 0$ kiểm soát độ đánh đổi (tradeoff). Trực giác: Z bị ép trở thành một dạng **minimal sufficient statistic** của X cho việc dự đoán Y (thống kê đủ, tối thiểu).

Lưu ý quy ước dấu (paper, chú thích 3): ở đây β lớn cho ra encoding **nén nhiều hơn** (compressed nhiều); một số tài liệu IB khác viết mục tiêu theo chiều ngược (min $I(Z, X) - \beta I(Z, Y)$), lúc đó β lớn lại tương ứng *ít* nén hơn. Đây là một điểm dễ nhầm giữa các paper IB khác nhau, không phải mâu thuẫn nội tại của bài này.

Vấn đề tính toán: mutual information nói chung khó tính (computationally challenging). Paper chỉ nêu hai trường hợp đã biết giải được trước đây: (a) khi X, Y, Z đều rời rạc (như trong Tishby et al. 1999, dùng để gom cụm từ - cluster discrete data như words), giải bằng thuật toán lặp **Blahut-Arimoto**; (b) khi X, Y, Z cùng Gaussian liên hợp (Chechik et al., 2005). Cả hai đều giới hạn nghiêm trọng lớp mô hình có thể học. Blahut-Arimoto cũng được nêu ở Mục 2 (Related Work) là "infeasible to apply to deep neural networks" vì tính lặp của nó.

Đóng góp của bài: dùng **variational inference** để xây một cận dưới (lower bound) khả tính cho R_{IB} ở công thức (3), rồi dùng **reparameterization trick** (Kingma & Welling, 2014) để lấy gradient Monte Carlo không chệch (unbiased), tối ưu bằng SGD với mạng nơ-ron sâu, xử lý được dữ liệu liên tục chiều cao (ảnh) mà không cần giả định rời rạc hay Gaussian liên hợp.

2. Ý tưởng chính

Ý tưởng cốt lõi gồm ba bước:

1. **Chặn dưới số hạng** $I(Z, Y)$ (số hạng muốn tối đa hoá) bằng cách đưa vào một **decoder biến phân** $q(y|z)$ xấp xỉ phân phối $p(y|z)$ thật (vốn bất khả thi vì phải tích phân qua X). Vì $KL[p(Y|Z), q(Y|Z)] \geq 0$ luôn đúng, ta có $I(Z, Y) \geq \mathbb{E}_{p(x,y,z)}[\log q(y|z)] + H(Y)$ (bỏ qua entropy nhãn $H(Y)$ vì không phụ thuộc tham số tối ưu).
2. **Chặn trên số hạng** $I(Z, X)$ (số hạng muốn tối thiểu hoá) bằng cách đưa vào một **marginal biến phân** $r(z)$ xấp xỉ marginal thật $p(z) = \int dx\, p(z|x)p(x)$ (cũng bất khả thi tính trực tiếp vì phải tích phân qua toàn bộ X). Vì $KL[p(Z), r(Z)] \geq 0$, ta có $I(Z, X) \leq \mathbb{E}_{p(x,z)}[\log \frac{p(z|x)}{r(z)}]$.
3. **Ghép hai chặn lại** thành một cận dưới duy nhất L cho toàn bộ R_{IB} (vì trừ đi một chặn trên thì được một chặn dưới), rồi dùng **reparameterization trick** để lấy mẫu $z = f(x, \epsilon)$ khả vi theo tham số encoder, cho phép backpropagate qua mẫu ngẫu nhiên và tối ưu bằng SGD.

Khác với VAE (ml basics 15), ở đây không có số hạng likelihood cho X (vì bài toán là discriminative, có giám sát, không cần tái tạo lại input), và vai trò của β không cố định bằng 1 mà là một siêu tham số cần quét để kiểm soát độ nén.

3. Phương pháp chi tiết

3.1 Giả định cấu trúc (Mục 3, công thức 5)

Paper giả định phân phối chung factorizes theo **Markov chain** $Y \leftrightarrow X \leftrightarrow Z$:

$$p(X, Y, Z) = p(Z|X, Y)p(Y|X)p(X) = p(Z|X)p(Y|X)p(X),$$

nghĩa là $p(Z|X,Y) = p(Z|X)$: **encoding Z không được phép phụ thuộc trực tiếp vào nhãn Y**, chỉ qua X. Ràng buộc này mở đường cho việc học biểu diễn không giám sát (Phụ lục B). Thành phần duy nhất cần mô hình hoá là encoder $p(Z|X)$; mọi phân phối khác suy ra từ nó và cấu trúc Markov chain.

3.2 Chặn dưới cho $I(Z,Y)$

Viết lại (công thức 6):

$$I(Z,Y) = \int dy\, dz\, p(y,z) \log \frac{p(y,z)}{p(y)p(z)} = \int dy\, dz\, p(y,z) \log \frac{p(y|z)}{p(y)}.$$

$p(y|z)$ được xác định đầy đủ bởi encoder và Markov chain (công thức 7): $p(y|z) = \int dx\, \frac{p(y|x)p(z|x)p(x)}{p(z)}$, nhưng **bất khả thi tính trực tiếp** vì mẫu số $p(z)$ cũng cần tích phân qua X. Đặt $q(y|z)$ (decoder, một mạng nơ-ron riêng) là xấp xỉ biến phân cho $p(y|z)$. Dùng $KL[p(Y|Z), q(Y|Z)] \geq 0$ (bất đẳng thức Gibbs), paper suy ra (công thức 8-11):

$$I(Z,Y) \geq \int dy\, dz\, p(y,z) \log q(y|z) + H(Y).$$

Thay $p(y,z) = \int dx\, p(x)p(y|x)p(z|x)$ (dùng giả định Markov), ta được cận dưới chỉ cần mẫu từ dữ liệu và từ encoder (công thức 12):

$$I(Z,Y) \geq \int dx\, dy\, dz\; p(x)p(y|x)p(z|x)\; \log q(y|z).$$

3.3 Chặn trên cho $I(Z,X)$

Viết lại (công thức 13): $I(Z,X) = \int dz\, dx\, p(x,z) \log p(z|x) - \int dz\, p(z) \log p(z)$. Số hạng thứ hai cần marginal $p(z) = \int dx\, p(z|x)p(x)$, cũng khó tính. Đặt $r(z)$ là xấp xỉ biến phân cho marginal này; dùng $KL[p(Z), r(Z)] \geq 0$ để thay $p(z)$ bằng $r(z)$ ở chiều bất lợi (tạo ra một **chặn trên**, công thức 14):

$$I(Z,X) \leq \int dx\, dz\; p(x)p(z|x) \log \frac{p(z|x)}{r(z)}.$$

3.4 Cận dưới toàn bộ và hàm mất mát thực dụng

Ghép hai chặn (công thức 15):

$$I(Z,Y) - \beta I(Z,X) \geq \int dx\, dy\, dz\; p(x)p(y|x)p(z|x) \log q(y|z) - \beta \int dx\, dz\; p(x)p(z|x) \log \frac{p(z|x)}{r(z)} = L.$$

Thay phân phối dữ liệu thật $p(x,y)$ bằng phân phối thực nghiệm (empirical distribution) trên N mẫu huấn luyện, $p(x,y) = \frac{1}{N} \sum_n \delta_{x_n}(x)\delta_{y_n}(y)$ (công thức 16):

$$L \approx \frac{1}{N} \sum_{n=1}^{N} \int dz \Big[p(z|x_n) \log q(y_n|z) - \beta\, p(z|x_n) \log \frac{p(z|x_n)}{r(z)} \Big].$$

Lưu ý: số hạng thứ hai trong công thức này chính là $\beta\, KL[p(Z|x_n), r(Z)]$ viết dưới dạng tích phân.

Tham số hoá cụ thể: giả sử encoder dạng Gaussian $p(z|x) = \mathcal{N}(z \mid f_e^{\mu}(x), f_e^{\Sigma}(x))$, với f_e là một MLP xuất ra cả mean μ (K chiều) và ma trận hiệp phương sai Σ ($K \times K$). Áp dụng **reparameterization trick** (Kingma

& Welling, 2014): viết $p(z|x)\,dz = p(\epsilon)\,d\epsilon$ với $z = f(x,\epsilon)$ là hàm xác định (deterministic) của x và biến ngẫu nhiên Gaussian ϵ. Vì nhiễu ϵ độc lập với tham số mô hình, có thể lấy gradient dễ dàng qua z.

Giả sử $p(z|x)$ và $r(z)$ chọn sao cho KL divergence giữa chúng có dạng giải tích (analytic KL, ví dụ cả hai Gaussian), paper đưa ra hàm mất mát cần **tối thiểu hoá** (công thức 17):

$$J_{IB} = \frac{1}{N}\sum_{n=1}^{N} \mathbb{E}_{\epsilon\sim p(\epsilon)}\big[-\log q(y_n \mid f(x_n,\epsilon))\big] + \beta\, KL\big[p(Z|x_n), r(Z)\big].$$

Số hạng thứ nhất là cross-entropy giữa nhãn thật và dự đoán của decoder qua một mẫu ngẫu nhiên $z = f(x_n,\epsilon)$ (giống loss phân loại chuẩn, nhưng qua một bước lấy mẫu); số hạng thứ hai là KL divergence giữa phân phối encoding và prior $r(z)$, nhân với β: **chính xác cùng cấu trúc với loss VAE** (reconstruction + β·KL), nhưng số hạng "reconstruction" ở đây là phân loại nhãn y thay vì tái tạo x.

Chú thích 4 của paper: nếu encoder/prior không cho KL giải tích, vẫn có thể reparameterize qua một mẫu của chính divergence đó (Kingma & Welling, 2014; Blundell et al., 2015), nên công thức (17) không bắt buộc phải Gaussian.

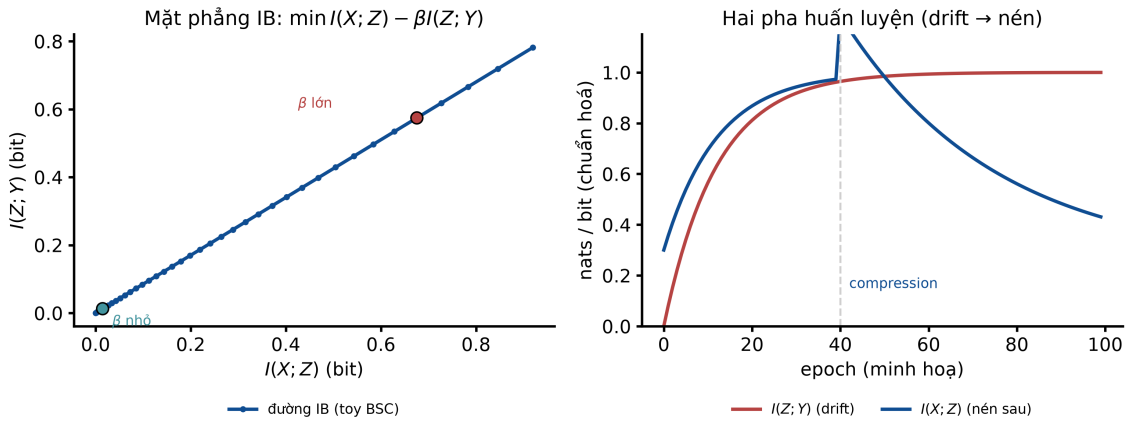
Ví dụ số nhỏ minh hoạ reparameterization trick (tự tính): giả sử với một ảnh x_n, encoder cho ra $\mu = 2.0$, $\sigma = 0.5$ (trường hợp $K = 1$ chiều để đơn giản). Lấy một mẫu nhiễu chuẩn $\epsilon = 0.3$ (từ $\mathcal{N}(0,1)$), reparameterization cho $z = \mu + \sigma\cdot\epsilon = 2.0 + 0.5\times 0.3 = 2.15$ (tự tính). Vì z là hàm khả vi của μ,σ (tham số encoder) với ϵ cố định/không phụ thuộc tham số, gradient $\partial z/\partial\mu = 1$ và $\partial z/\partial\sigma = \epsilon = 0.3$ (tự tính) tính được trực tiếp bằng quy tắc chuỗi (chain rule) qua decoder, không cần đạo hàm qua một phép lấy mẫu "không khả vi".

Ví dụ nhỏ minh hoạ mutual information rời rạc (tự tính): xét Z, Y rời rạc nhị phân với phân phối chung $p(z=0,y=0) = 0.4$, $p(z=0,y=1) = 0.1$, $p(z=1,y=0) = 0.1$, $p(z=1,y=1) = 0.4$ (tự tính, ví dụ minh hoạ, không phải số trong paper). Marginal: $p(z=0) = p(z=1) = 0.5$, $p(y=0) = p(y=1) = 0.5$. Mutual information: $I(Z,Y) = \sum_{z,y} p(z,y)\log_2\frac{p(z,y)}{p(z)p(y)} = 2\times 0.4\log_2\frac{0.4}{0.25} + 2\times 0.1\log_2\frac{0.1}{0.25}$ $\approx 2\times 0.4\times 0.678 + 2\times 0.1\times(-1.322) \approx 0.542 - 0.264 = 0.278$ bit (tự tính). Số này minh hoạ cách tính $I(Z,Y)$ ở công thức (1) khi Z, Y rời rạc hữu hạn, trường hợp mà paper nói đã giải được bằng Blahut-Arimoto nhưng không mở rộng được cho deep network.

4. Thí nghiệm và kết quả chính

4.1 MNIST, bottleneck $K = 256$ (Mục 4.1.1)

Kiến trúc: MLP $784-1024-1024-10$, ReLU, giống kiến trúc trong (Pereyra et al., 2017) để có "headroom" so sánh. Baseline (không regularization) đạt lỗi **1.38%**. Encoder VIB: MLP $784-1024-1024-2K$ ($K = 256$), nửa đầu output là μ, nửa sau là σ (qua softplus). Decoder: hồi quy logistic (softmax) đơn giản từ z ra 10 lớp. $r(z)$ cố định là Gaussian cầu (spherical) K chiều, $\mathcal{N}(0,I)$. Huấn luyện 200 epoch, TensorFlow, Adam, learning rate 10^{-4} (chi tiết đầy đủ ở Phụ lục A).



Mặt phẳng IB I(Z;Y) vs I(X;Z) và hai pha huấn luyện drift → nén (I(X;Z) giảm sau khi I(Z;Y) bão hoà). (minh hoạ, tự tính)

Bảng 1 (lỗi phân loại trên test set MNIST permutation-invariant, $K = 256$):

Model	error
Baseline	1.38%
Dropout	1.34%
Dropout (Pereyra et al., 2017)	1.40%
Confidence Penalty	1.36%
Confidence Penalty (Pereyra et al., 2017)	1.17%
Label Smoothing	1.40%
Label Smoothing (Pereyra et al., 2017)	1.23%
VIB ($\beta = 10^{-3}$)	**1.13%**

Bài báo nêu rõ: sự khác biệt giữa kết quả Confidence Penalty/Label Smoothing của chính họ và số trong (Pereyra et al., 2017) là **do siêu tham số khác nhau** (không phải lỗi tái tạo). VIB đạt lỗi thấp nhất trong bảng (1.13%), nhưng bài báo lưu ý "các kết quả này không phải state of the art, mục tiêu chính không phải chứng minh VIB là regularizer tốt nhất", chỉ cho thấy VIB hoạt động như một regularizer hợp lý khi dùng cùng kiến trúc cho mọi so sánh.

Hình 1(a) vẽ lỗi train/test theo β (trung bình 5 lần chạy độc lập, có error bar), cho cả "1 shot eval" (1 mẫu posterior z) và "avg eval" (trung bình $S = 12$ mẫu Monte Carlo, $p(y|x) = \frac{1}{S}\sum_s q(y|z^s)$). Quan sát của paper:

- Lỗi tăng vọt khi β vượt ngưỡng tới hạn $\beta \sim 10^{-2}$, tương ứng với $I(X,Z)$ nhỏ hơn $\log_2(10)$ bit: không đủ thông tin để phân biệt 10 lớp.
- Với β nhỏ, test error cao hơn train error (overfitting), vì network học $\sigma \approx 0$ (gần xác định/deterministic), làm mất hiệu lực regularization.
- Ở vùng β trung gian, Monte Carlo averaging (12 mẫu) giúp cải thiện kết quả; vùng β cho kết quả tốt nhất gần trùng với vùng mà lợi ích của averaging biến mất, gợi ý một cách chọn β chỉ dựa vào thống kê trên training set (paper nói rõ **chưa khảo sát sâu** khả năng này).

Hình 1(c) vẽ "IB curve" ($I(Z,Y)$ so với $I(Z,X)$ khi biến β): giảm β (cho qua nhiều thông tin hơn) tăng $I(Z,Y)$ trên **training set**, nhưng không nhất thiết trên test set.

Hình 1(d) vẽ $I(Z,X)$ (trục log) theo β: vùng β tốt nhất (10^{-3} đến 10^{-2}) tương ứng $I(Z,X)$ chỉ khoảng **10 đến 100 bit**.

4.2 MNIST, bottleneck $K = 2$ (Mục 4.1.2)

Dùng full-covariance Gaussian ($K = 2$, mạng dự đoán Cholesky decomposition của Σ). Hình 1(b) cho thấy classification kém hơn (do bottleneck hẹp hơn nhiều), nhưng xu hướng chung giống trường hợp $K = 256$; gap train-test lớn hơn.

Hình 2 trực quan hoá posterior $p(z|x)$ như ellipse tin cậy 95% cho 1000 ảnh test, màu theo nhãn, nền là entropy của $q(y|z)$. Ba giá trị β minh hoạ với lỗi kèm theo (đọc từ chú thích Hình 2): $\beta = 10^{-3}$: $err_{mc} = 3.18\%$, $err_1 = 3.24\%$; $\beta = 10^{-1}$: $err_{mc} = 3.44\%$, $err_1 = 4.32\%$; $\beta = 10^{0}$: $err_{mc} = 33.82\%$, $err_1 = 62.81\%$ (ở đây err_1 dùng 1 mẫu, err_{mc} dùng 12 mẫu). Khi β tăng, covariance của embedding tăng tương đối, các lớp bắt đầu chồng lấp; qua một ngưỡng tới hạn, encoding ”sụp” (collapse) và mất hầu hết thông tin về lớp.

4.3 Robustness với adversarial examples (Mục 4.2)

Paper dùng hai loại tấn công: **FGS (Fast Gradient Sign)** của Goodfellow et al. (2015): một bước theo chiều gradient; và **L2 optimization attack** của Carlini & Wagner (2016): tấn công tối ưu hoá, cho perturbation nhỏ nhất đã công bố tới lúc đó, ”capable of fooling the target network 100% of the time”. Paper trích lại rằng trên MNIST, FGS gốc đạt khoảng 90% tỉ lệ đánh lừa mạng maxout với $\epsilon = 0.25$ (số do Goodfellow et al. báo cáo, không phải thí nghiệm của bài này).

Thiết lập (Mục 4.2.3): cùng kiến trúc $K = 256$ ở Mục 4.1; so sánh baseline deterministic, dropout, và VIB các β. Với VIB, dùng 12 mẫu posterior khi tính $p(y|x)$ cho cả việc tấn công và việc đánh giá; paper cũng thử chạy ở ”mean mode” ($\sigma = 0$) và thấy **không khác biệt đáng kể**, nên mọi kết quả báo cáo dùng stochastic evaluation 12 mẫu.

Kết quả MNIST, Carlini-Wagner L2 (0 → 1), Mục 4.2.4: lấy 10 chữ số ”0” đầu tiên trong test set, tấn công buộc phân loại thành ”1”. Hình 3-4 cho thấy mô hình deterministic bị lừa dễ bằng perturbation nhỏ, còn VIB với β đủ lớn thường làm tấn công **thất bại** (trong số bước lặp cho phép); khi tấn công thành công, perturbation phải lớn hơn. Hình 4: khi β tăng, L_0 (số pixel bị thay đổi) giảm nhưng L_2 và L_∞ tăng: kẻ tấn công bị buộc đổi ít pixel hơn nhưng với biên độ lớn hơn mỗi pixel.

Hình 5 (FGS, 1000 ảnh test đầu tiên, 3 lần lặp mỗi β): tất cả model test đạt trên 98.4% accuracy trên test set sạch (không bị méo bởi khác biệt accuracy nền). Accuracy tương đối của VIB (so baseline) tăng mạnh theo β. Baseline deterministic đạt khoảng 1% accuracy (adversarial) khi $\epsilon = 0.5$ đến khoảng 5% khi $\epsilon = 0.35$. Dropout khá hơn baseline (khoảng 5%-16% cùng dải ϵ) nhưng kém VIB.

Hình 6 (Carlini-Wagner L2, cả targeted và untargeted, trên toàn 1000 ảnh mọi lớp): **deterministic và dropout đều đạt accuracy 0%** trên cả targeted và untargeted attack (khớp với số 0% mà Carlini & Wagner (2016) báo cáo cho một ConvNet huấn luyện trên MNIST). VIB cho accuracy dương đáng kể trên một dải β rộng; ngay cả khi $\beta \to 0$ (thực nghiệm dùng $\beta = 10^{-11}$, ”indistinguishable” so với $\beta = 0$), VIB vẫn robust hơn hẳn 0%. Paper giải thích hiện tượng này (dù β rất nhỏ) bằng khung lý thuyết của Fawzi et al. (2016): phân loại **quadratic** có khả năng robust cao hơn phân loại **linear**; Phụ lục C chứng minh gần đúng rằng bộ encoder/decoder Gaussian/softmax của VIB xấp xỉ một hàm quadratic với mọi $\beta < \infty$.

4.4 ImageNet (Mục 4.2.5)

Dùng checkpoint pretrained công khai của **Inception ResNet V2** (Szegedy et al., 2016) trên ImageNet (Deng et al., 2009), đạt **80.4%** classification accuracy trên validation set. Trích đặc trưng (feature) ở tầng penultimate (1536 chiều), whitened, rồi gắn một kiến trúc giống thiết lập MNIST: 2 lớp fully-connected 1024 unit, sau đó lớp encoding ngẫu nhiên Gaussian cầu 1024 chiều, rồi decoder là hồi quy logistic.

Kết quả classification: accuracy tốt nhất đạt được là **80.12%** tại $\beta = 0.01$: ”gần như bằng” 80.4% gốc nhưng chỉ cần $I(X, Z) \sim 45$ bit (paper gọi là ”a surprisingly small amount of information”). Ngược lại, khi $\beta = 0$ (không nén, trên 10,000 bit theo paper), accuracy chỉ **78.87%**. Baseline deterministic (cùng kiến trúc nhưng không VIB loss, lớp linear 1024 chiều thay encoding ngẫu nhiên) chỉ đạt **78.75%**. Bài báo lưu ý các

con số thấp hơn 80.4% gốc ”likely due to suboptimal hyperparameters settings or inadequate training” (không phải do bản chất VIB).

Kết quả robustness (Mục 4.2.5, ”Adversarial Robustness”): dùng tấn công Carlini targeted L2. Hình 7-8 và Bảng 2 so sánh 3 kiến trúc: Determ (baseline deterministic trên precomputed feature), IRv2 (Inception ResNet V2 gốc không chỉnh sửa), VIB(0.01) ($\beta = 0.01$). Mục tiêu tấn công là nhãn 222 (”soccer ball”), trên 30/40 ảnh đầu validation set mà VIB phân loại đúng.

Bảng 2 (metric, Determ, IRv2, VIB(0.01)):

Metric	Determ	IRv2	VIB(0.01)
Successful target	1.0	1.0	0.567
L_2	6.45	14.43	43.27
L_∞	0.18	0.44	0.92

”Successful target” là tỉ lệ ảnh bị tấn công thành công ép về đúng nhãn mục tiêu (thấp hơn tốt hơn). Cả Determ và IRv2 đều bị tấn công thành công **100%**; VIB(0.01) chỉ bị thành công **56.7%**. Trong văn bản chính (không phải bảng), paper diễn giải cùng thí nghiệm theo cách khác: ”chỉ 17/30 cuộc tấn công thành công” và ”VIB model can resist about 43.3% of the attacks”. Hai cách nói này **nhất quán với nhau** ($17/30 \approx 0.567$ là tỉ lệ thành công, $1 - 0.567 \approx 0.433$ là tỉ lệ chống được), nhưng phần caption Hình 7 lại ghi ”in 13 out of 30 examples the attack fails” ($13/30 \approx 0.433$, khớp phần ”resist 43.3%”): tức ba cách diễn đạt (bảng 0.567, văn bản 17/30, caption 13/30 fail) **đều tự nhất quán khi quy đổi qua lại**, không phải mâu thuẫn thật, nhưng người đọc cần chú ý bảng ghi ”successful” còn văn bản đôi khi ghi theo ”fail”/”resist” nên dễ nhầm dấu khi đọc lướt.

L_2 và L_∞ trung bình của perturbation cần để tấn công thành công đều **lớn hơn nhiều** cho VIB (43.27, 0.92) so với Determ (6.45, 0.18) và IRv2 (14.43, 0.44): Determ và IRv2 chỉ cần perturbation nhỏ không thể nhận ra bằng mắt, còn với VIB, trong nhiều trường hợp perturbation **có thể nhận thấy được** (paper tự nói ”the perturbations for the VIB network can be perceived”, Hình 7-8).

5. Phân tích / ablation

- $\beta = 0$ **như giới hạn deterministic:** khi $\beta \to 0$, quá trình tối ưu VIB có xu hướng làm $f_e^{\Sigma}(x) \to 0$, khiến network gần như xác định (deterministic); paper cũng huấn luyện riêng một model deterministic tường minh (bỏ hẳn lớp Gaussian, chỉ dùng $z = f_e^{\mu}(x)$) để so sánh, và công thức (18) (J_{IB_0}) là mục tiêu tương ứng trường hợp $\beta = 0$.
- **Số mẫu Monte Carlo khi đánh giá (1 shot so với 12 mẫu):** ở vùng β ”tốt” (cho kết quả gần tối ưu), một mẫu đơn (1 shot) đã gần sát kết quả trung bình 12 mẫu (Hình 2 caption: ”For ’good’ values of β, a single sample suffices”); lợi ích của việc trung bình nhiều mẫu rõ nhất ở vùng β trung gian, và paper quan sát vùng lợi ích này biến mất gần đúng tại vùng β cho kết quả tốt nhất: một ”ablation tự nhiên” gợi ý cách chọn β không cần validation set riêng (nhưng chưa khảo sát kỹ).
- **Dropout so với VIB trên adversarial robustness:** dropout cải thiện robustness so với baseline deterministic (ví dụ 5%-16% so với 1%-5% trên FGS), nhưng vẫn kém rõ VIB, đặc biệt khi β tăng (Hình 4b, 5b). Đây là so sánh trực tiếp giữa hai cơ chế ”tiêm nhiễu” khác nhau: dropout tắt ngẫu nhiên neuron, VIB tiêm nhiễu Gaussian có kiểm soát bởi mục tiêu thông tin.
- **”Mean mode” so với stochastic evaluation (Mục 4.2.3):** paper thử tắt nhiễu khi đánh giá ($\sigma \to 0$) và thấy ”no noticeable impact”, nghĩa là lợi ích robustness của VIB không chỉ đến từ việc *đánh giá* ngẫu nhiên (mà có thể đến từ việc *huấn luyện* ngẫu nhiên định hình decision boundary).

- L_0 **so với** L_2/L_∞ **khi** β **tăng (Hình 4, MNIST):** một hiện tượng tinh tế: β tăng khiến L_0 (số pixel) của perturbation thành công **giảm** nhưng L_2, L_∞ (biên độ) **tăng**: attacker bị ép tập trung vào ít pixel hơn nhưng phải ”đục” sâu hơn ở các pixel đó.
- **Robustness khi** $\beta \to 0$ **vẫn cao hơn 0% (Hình 6):** đây là quan sát bất ngờ nhất của bài, được paper giải thích không phải bởi lượng nén (vì β gần 0 gần như không nén) mà bởi **dạng hàm quadratic** của bộ encoder/decoder Gaussian/softmax (Phụ lục C): một cơ chế robustness khác, độc lập một phần với cơ chế ”nén thông tin”.

6. Giới hạn

Do tác giả nêu:

- Kết quả MNIST (Bảng 1) ”không phải state of the art”, và bài không chủ đích chứng minh VIB là phương pháp regularization tốt nhất, chỉ dùng cùng kiến trúc để so sánh công bằng các dạng regularization khác nhau; cần ”much more experimentation” để khẳng định tổng quát.
- Kết quả ImageNet thấp hơn baseline gốc 80.4% ở mọi β thử nghiệm; bài báo lưu ý nguyên nhân có thể là ”inadequate training time or suboptimal hyperparameters”, không loại trừ khả năng VIB có thể làm tốt hơn nữa nếu tối ưu kỹ hơn.
- Phụ lục C chỉ đưa ra một **xấp xỉ** (dùng Taylor series expansion bậc 2 cho hàm log-sum-exp), không phải một chặn chặt (tight bound) hoàn chỉnh; paper ghi rõ ”using the TSE is sufficient to sketch the derivation”, và có thể làm chặt hơn qua kỹ thuật của Browne & McNicholas (2015) nhưng ”merely changes the scaling... the result is still log-quadratic” (không đổi bản chất kết luận).
- Mục ”Future Directions” (Mục 5) tự liệt hàng loạt hướng **chưa làm** trong bài này: đặt mục tiêu VIB ở nhiều/mọi lớp của network (bài chỉ dùng một bottleneck); test trên ảnh thật đa dạng hơn (not fully explored); dùng $r(z)$ phong phú hơn thay vì cố định $\mathcal{N}(0, I)$; liên hệ với differential privacy (chỉ trích dẫn, chưa làm thực nghiệm); mở rộng sang open-universe classification; và mở rộng sang sequence prediction/predictive information (chỉ nêu ý tưởng).

Nhận xét:

- Thí nghiệm ImageNet không huấn luyện toàn bộ mạng từ đầu mà **đóng băng** backbone Inception ResNet V2 và chỉ gắn/huấn luyện một đầu (head) nhỏ phía trên feature trích sẵn; do đó kết luận về classification/robustness trên ImageNet chỉ áp dụng cho setting ”fine-tune head nhỏ trên feature cố định”, chưa chứng minh được VIB hoạt động tốt khi huấn luyện end-to-end một mạng lớn từ đầu trên dữ liệu ảnh thật.
- Số liệu adversarial robustness ImageNet (Bảng 2, Hình 7-8) chỉ dựa trên một tập rất nhỏ (30 ảnh, một nhãn mục tiêu cố định ”soccer ball”); đây là mẫu nhỏ so với quy mô 1000 lớp của ImageNet, nên độ tin cậy thống kê của tỉ lệ 56.7%/43.3% khá hạn chế (không có error bar hay nhiều lần lặp như thí nghiệm MNIST ở Hình 5-6).
- Phần so sánh với Confidence Penalty/Label Smoothing (Bảng 1) dùng lại kiến trúc của Pereyra et al. (2017) nhưng bài báo lưu ý siêu tham số khác, khiến phép so sánh ”VIB tốt hơn Confidence Penalty của chính paper này” không hoàn toàn tương đương phép so sánh ”VIB tốt hơn số 1.17% trong paper gốc của Pereyra”: hai mức kết quả Confidence Penalty (1.36% so với 1.17%) chênh khá lớn, cho thấy độ nhạy cao với tuning, một yếu tố cần cẩn trọng khi diễn giải ”VIB thắng” ở Bảng 1.

7. Liên hệ với các bài khác

- **ml basics 15** (Auto-Encoding Variational Bayes, đã tồn tại tại `ml basics content/15_2013_vae_aevb.md`): paper tự nói ở Mục 2 (Related Work) rằng VAE ”là một trường hợp đặc biệt của một phiên bản không giám sát (unsupervised) của VIB, nhưng với β cố định ở 1.0” (Phụ lục B chứng minh chi tiết). VIB dùng lại nguyên cơ chế reparameterization trick + KL-to-prior của VAE, nhưng cho bài toán

có giám sát và cho phép β thay đổi tự do. Nên đọc VAE trước để hiểu rõ nguồn gốc của kỹ thuật reparameterization và cấu trúc "encoder + KL-regularizer" trước khi đọc VIB.

- **math 11** (SGD as Approximate Bayesian Inference, đã viết, `math content/11_2017_sgd_approximate_bayesian_inference.md`): cả hai bài cùng nhóm "làm cho suy luận xấp xỉ/biến phân chạy được bằng SGD trên mạng sâu", nhưng math 11 áp KL divergence vào việc chọn **siêu tham số của bộ tối ưu** (learning rate, preconditioner) để mô phỏng một hậu nghiệm Bayes trên **tham số mô hình** θ, còn VIB áp KL divergence trực tiếp vào **hàm mất mát** để kiểm soát biểu diễn trung gian Z; cùng công cụ (biến phân, reparameterization) nhưng áp vào hai đối tượng khác nhau (tham số mô hình so với biểu diễn trung gian).
- **math 10** (Hamiltonian Monte Carlo, đã viết): cùng nhóm "suy luận xấp xỉ cho deep learning" trong lộ trình, tuy không liên hệ kỹ thuật trực tiếp (HMC không xuất hiện trong bài VIB).
- Trong thư mục mới cùng mã 'math': paper tự trích dẫn **Higgins et al. (2017), beta-VAE** (ICLR 2017, cùng ý tưởng đổi trọng số KL term trong VAE nhưng từ góc nhìn neuroscience) là công trình "đã khám phá đúng thiết lập này" với động cơ khác; nếu thư mục `math` có bài về beta-VAE trong tương lai (numbering chưa xác định ở đây), nên đọc cùng nhóm.

8. Gợi ý áp dụng

Information bottleneck cung cấp một cách đặt câu hỏi về sự đánh đổi giữa nén biểu diễn và giữ thông tin phục vụ dự đoán. Mục tiêu biến phân là một xấp xỉ có thể tối ưu; các đại lượng trong mục tiêu không tự chứng minh rằng biểu diễn đã học là tối ưu theo mọi tiêu chí.

9. Thí nghiệm gợi ý

1. **Tái tạo thu nhỏ Bảng 1 trên MNIST với K nhỏ hơn:** MLP $784-256-256-2K$ với $K=32$ hoặc 64, so sánh baseline deterministic, VIB với vài giá trị $\beta \in \{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}\}$. Đo: test error, và $I(Z,X)$ ước lượng qua $KL[p(Z|x), r(Z)]$ trung bình trên test set. Kỳ vọng: tái lập hình dạng chữ U của lỗi theo β (Hình 1a) và quan hệ tỉ lệ thuận giữa β nhỏ-$I(Z,X)$ lớn.
2. **Visualize bottleneck 2 chiều (tái tạo thu nhỏ Hình 2):** dùng $K=2$ như Mục 4.1.2, vẽ ellipse $p(z|x)$ cho 200-500 ảnh test với 3-4 giá trị β. Đo: tỉ lệ các lớp chồng lấp (ví dụ bằng diện tích overlap ước lượng) theo β. Kỳ vọng: overlap tăng đơn điệu theo β, khớp mô tả "classes start to overlap" của paper.
3. **Đo adversarial robustness cơ bản với FGS trên MNIST (quy mô nhỏ):** huấn luyện baseline deterministic và VIB ($\beta = 10^{-2}$ chẳng hạn) trên MNIST đầy đủ, tấn công FGS với $\epsilon \in \{0.1, 0.2, 0.3\}$ trên 200 ảnh test. Đo: accuracy trên ảnh adversarial của hai model. Kỳ vọng: VIB giữ accuracy cao hơn deterministic, khớp xu hướng Hình 5 (không kỳ vọng khớp số chính xác vì quy mô thí nghiệm nhỏ hơn).
4. **Kiểm tra ảnh hưởng của số mẫu Monte Carlo khi đánh giá (1 shot so với trung bình):** với model VIB đã huấn luyện ở thí nghiệm 1, so test error khi dùng 1 mẫu so với trung bình 12 mẫu, ở vài giá trị β khác nhau. Kỳ vọng: khác biệt lớn ở β trung gian, gần như không khác biệt ở vùng β tốt nhất, khớp quan sát Mục 4.1.1 (Hình 1a, caption Hình 2).
5. **Minh hoạ "collapse" của encoding khi β quá lớn:** quét β từ 10^{-4} đến 10 trên cùng thiết lập $K=2$, đo entropy trung bình của $q(y|z)$ đánh giá tại $z=\mu(x)$ cho các ảnh test. Kỳ vọng: entropy tăng mạnh (dự đoán trở nên bất định) khi β vượt một ngưỡng, minh hoạ hiện tượng "encoding collapses" mô tả ở Mục 4.1.2.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / ý nghĩa
Mục tiêu Information Bottleneck	$R_{IB}(\theta) = I(Z,Y;\theta) - \beta I(Z,X;\theta)$, (3)

Khái niệm	Công thức / ý nghĩa
Markov chain giả định	$Y \leftrightarrow X \leftrightarrow Z$, $p(Z\|X,Y) = p(Z\|X)$, (5)
Chặn dưới $I(Z,Y)$ (decoder biến phân $q(y\|z)$)	$I(Z,Y) \geq \int p(x)p(y\|x)p(z\|x) \log q(y\|z)$, (12)
Chặn trên $I(Z,X)$ (marginal biến phân $r(z)$)	$I(Z,X) \leq \int p(x)p(z\|x) \log \frac{p(z\|x)}{r(z)}$, (14)
Cận dưới toàn bộ L	ghép (12) và (14), (15)
Hàm mất mát thực dụng J_{IB}	$\frac{1}{N}\sum_n \mathbb{E}_\epsilon[-\log q(y_n\|f(x_n,\epsilon))] + \beta\, KL[p(Z\|x_n), r(Z)]$, (17)
Reparameterization	$z = f(x,\epsilon)$, ϵ độc lập tham số, (Kingma & Welling 2014)
Quy ước dấu β	β lớn = nén nhiều hơn (ngược với một số paper IB khác), chú thích 3
Giới hạn deterministic $\beta = 0$	$J_{IB_0} = -\frac{1}{N}\sum_n \mathbb{E}_{z\sim\mathcal{N}(f_e^\mu, f_e^\Sigma)}[\log S(y_n\|f_d(z))]$, (18)
Softplus cho σ (MNIST $K = 256$)	$\sigma = \log(1+\exp(x-5.0))$, (19), Phụ lục A
VAE là trường hợp đặc biệt	VAE = VIB không giám sát với $\beta = 1$ cố định, Phụ lục B
Quadratic bound cho encoder/decoder Gaussian/softmax	xấp xỉ bằng Taylor series expansion bậc 2 của log-sum-exp, Phụ lục C

Ví dụ tính tay minh hoạ chặn KL (tự tính): giả sử $p(z\|x) = \mathcal{N}(1, 0.25)$ (tức $\sigma^2 = 0.25$, $\sigma = 0.5$) và $r(z) = \mathcal{N}(0,1)$ (prior cố định, 1 chiều). Công thức KL giữa hai Gaussian 1 chiều: $KL[\mathcal{N}(\mu,\sigma^2)\|\mathcal{N}(0,1)] = \frac{1}{2}\left(\sigma^2 + \mu^2 - 1 - \log\sigma^2\right)$. Thay số: $\frac{1}{2}(0.25 + 1 - 1 - \log 0.25) = \frac{1}{2}(0.25 - (-1.386)) = \frac{1}{2}(1.636) \approx 0.818$ nat (tự tính). Nhân với β (ví dụ $\beta = 10^{-2}$) cho đóng góp của riêng mẫu này vào số hạng regularization trong công thức (17): $0.818 \times 0.01 \approx 0.00818$ nat (tự tính): minh hoạ cụ thể cách mỗi mẫu x_n góp một số hạng $\beta \cdot KL[p(Z|x_n), r(Z)]$ nhỏ vào tổng loss khi β nhỏ, khớp trực giác Mục 4.1.1 rằng vùng β tốt chỉ cần vài chục bit (ở đây minh hoạ bằng nat, 1 bit $= \ln 2$ nat ≈ 0.693 nat, nên khoảng 10 bit ≈ 6.93 nat).

11. Kết luận ngắn

Deep VIB xây một cận dưới biến phân khả vi cho mục tiêu Information Bottleneck gốc, bằng cách đưa vào một decoder biến phân $q(y|z)$ (chặn dưới $I(Z,Y)$) và một marginal biến phân $r(z)$ (chặn trên $I(Z,X)$), rồi dùng reparameterization trick để huấn luyện bằng SGD trên mạng sâu. Thực nghiệm trên MNIST (lỗi phân loại 1.13% tại $\beta = 10^{-3}$, tốt hơn các regularizer so sánh trong cùng kiến trúc) và ImageNet (80.12% accuracy chỉ với ~ 45 bit thông tin, so với 78.87% khi không nén) cho thấy VIB vừa cải thiện generalization vừa tăng mạnh robustness với adversarial example (Carlini-Wagner L2, FGS), kể cả khi β rất nhỏ, một phần giải thích được bằng tính chất quadratic gần đúng của bộ encoder/decoder Gaussian/softmax. VIB cũng tổng quát hoá VAE (trường hợp đặc biệt không giám sát với $\beta = 1$) và mở đường áp dụng khung IB cho dữ liệu liên tục chiều cao, điều mà các phương pháp IB rời rạc/Gaussian cổ điển không làm được.

Nguồn gốc: https://arxiv.org/abs/1612.00410

Bài 15

Online normalizer calculation for softmax (2018)

Tác giả: Maxim Milakov, Natalia Gimelshein (NVIDIA).
Venue / arXiv: arXiv:1805.02867 (v2, 28/07/2018); paper tự ghi ”Preprint. Work in progress.” ở cuối trang đầu, nên không có venue hội nghị hay tạp chí chính thức.
Nguồn: https://arxiv.org/abs/1805.02867
Chủ đề: một thuật toán tính hàm softmax ”an toàn” (không tràn số) trong một lượt quét duy nhất qua vector đầu vào, bằng cách tính đồng thời giá trị lớn nhất và tổng mũ chuẩn hoá, giảm số lần truy cập bộ nhớ so với cách làm 3-pass tiêu chuẩn, và hợp nhất (fuse) với phép TopK để giảm thêm.
Vị trí trong lộ trình: đọc sau math 14 (Deep Variational Information Bottleneck); đọc trước math 16 (FP8 Formats for Deep Learning) vì cả hai cùng chủ đề số học hiệu năng (numerics) cho deep learning trên phần cứng thật. Liên quan rất chặt tới **llm 44/45** (FlashAttention, FlashAttention-2): công thức đệ quy ở bài này là nền tảng toán học trực tiếp cho kỹ thuật ”online softmax theo block” mà FlashAttention dùng để tránh lưu toàn bộ ma trận attention.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (bản arXiv); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc.

1. Bối cảnh

Hàm softmax biến vector logit $x \in \mathbb{R}^V$, với V là số lớp hoặc kích thước từ vựng, thành một vector xác suất:

$$y_i = \frac{e^{x_i}}{\sum_{j=1}^{V} e^{x_j}}. \qquad (1)$$

Softmax xuất hiện ở lớp cuối của hầu hết mô hình phân loại/ngôn ngữ (sau lớp Projection ”chiếu” biểu diễn ẩn ra không gian từ vựng), và trong hồi quy logistic đa lớp. Paper liệt kê nhiều công trình trước đó cố **tránh phải tính softmax đầy đủ** (Differentiated Softmax, SVD-Softmax, Hierarchical Softmax, Importance Sampling, Noise Contrastive Estimation, Self-Normalized Softmax), nhưng hầu hết vẫn phải chạy hàm softmax gốc trên một vector đầy đủ hoặc rút gọn. Paper này đi theo hướng khác: **không thay đổi thuật toán**, mà làm cho chính phép tính softmax gốc **chạy nhanh hơn trên phần cứng thật**, bằng cách giảm số lần truy cập bộ nhớ (memory access).

Vấn đề kỹ thuật cốt lõi: công thức (1) tính trực tiếp có thể **tràn số (overflow)** nếu x_i lớn (vì e^{x_i} tăng rất nhanh), hoặc **mất độ chính xác (underflow)** nếu x_i rất âm. Giải pháp kinh điển, được mọi framework lớn (TensorFlow v1.7, PyTorch v0.4.0, MXNet v1.1.0, Microsoft CNTK v2.5.1, Chainer v5.0.0a1, theo paper liệt kê) dùng, là trừ đi giá trị lớn nhất trước khi lấy exponent:

$$y_i = \frac{e^{x_i - \max_k x_k}}{\sum_{j=1}^{V} e^{x_j - \max_k x_k}}. \qquad (2)$$

Công thức (2) cần **3 lượt quét** qua vector đầu vào: lượt 1 tìm max, lượt 2 tính tổng mẫu số, lượt 3 tính y_i và ghi ra, tổng cộng **4 lần truy cập bộ nhớ mỗi phần tử** (3 lần đọc + 1 lần ghi). Paper gọi đây là ”Safe Softmax” và muốn cải tiến số lần truy cập này.

2. Ý tưởng chính

Phương pháp gộp việc tính cực đại và tổng mẫu số trong một lượt quét bằng công thức đệ quy trực tuyến, lấy cảm hứng từ thuật toán tính phương sai ổn định số của Welford (1962, tài liệu [18] trong bài). Mỗi khi gặp cực đại mới, ta điều chỉnh tổng đã tích lũy bằng một hệ số mũ trước khi cộng phần tử mới. Nhờ vậy chỉ cần **1 lượt quét để tính xong cả max và tổng mẫu số**, giảm từ 4 xuống **3 lần truy cập bộ nhớ mỗi phần tử** (1 đọc cho lượt tính chuẩn hoá + 1 đọc, 1 ghi cho lượt tính y_i). Paper gọi thuật toán này là **"Online Softmax"**.

Ý tưởng thứ hai, mở rộng từ ý tưởng một: vì công thức đệ quy có thể viết lại thành một **phép toán kết hợp (associative) và giao hoán (commutative)** trên cặp (m, d) (max cục bộ, tổng mẫu số cục bộ), ta có thể **tính song song** (chia vector thành nhiều đoạn, tính từng đoạn độc lập, rồi gộp lại), rất phù hợp với GPU nhiều luồng.

Ý tưởng thứ ba: khi softmax được theo sau bởi **TopK** (ví dụ trong beam search để sinh văn bản tự hồi quy), ta không cần vật chất hoá (materialize) toàn bộ vector xác suất y; chỉ cần giữ K giá trị lớn nhất cùng chỉ số của chúng trong lúc quét, hợp nhất (fuse) luôn vào vòng lặp của Online Softmax, giảm xuống chỉ **1 lần truy cập bộ nhớ mỗi phần tử** cho cả Softmax+TopK.

3. Phương pháp chi tiết

3.1 Thuật toán Online Softmax (Mục 3, Algorithm 3)

Khởi tạo $m_0 = -\infty$, $d_0 = 0$. Với $j = 1, \dots, V$:

$$m_j \leftarrow \max(m_{j-1}, x_j), \qquad d_j \leftarrow d_{j-1} \cdot e^{m_{j-1} - m_j} + e^{x_j - m_j}.$$

Sau lượt quét này, $m_V = \max_k x_k$ và $d_V = \sum_{j=1}^{V} e^{x_j - m_V}$ đúng như công thức (2) cần. Lượt quét thứ hai (bắt buộc phải có, vì y_i cần m_V và d_V cuối cùng, không thể biết trước khi quét hết vector) tính $y_i \leftarrow e^{x_i - m_V} / d_V$.

Định lý 1 (paper, chứng minh bằng quy nạp): các dòng 1-6 của Algorithm 3 tính đúng $m_V = \max_k x_k$ và $d_V = \sum_j e^{x_j - m_V}$. Chứng minh quy nạp: cơ sở $V = 1$ hiển nhiên đúng ($m_1 = x_1$, $d_1 = e^{x_1 - m_1} = 1$); bước quy nạp giả sử đúng tới $S-1$, paper triển khai $d_S = d_{S-1} \cdot e^{m_{S-1} - m_S} + e^{x_S - m_S} = \sum_{j=1}^{S-1} e^{x_j - m_S} + e^{x_S - m_S} = \sum_{j=1}^{S} e^{x_j - m_S}$ (dùng giả thiết quy nạp và tính chất $e^{a-b} \cdot e^{b-c} = e^{a-c}$).

Tính an toàn số học: m_j luôn nằm trong $[\min_k x_k, \max_k x_k]$ nên không thể tràn/underflow; d_j luôn thoả $1 \le d_j \le j$ (vì mỗi số hạng $e^{x_i - m_j} \in (0, 1]$ và có ít nhất một số hạng bằng 1 khi $x_i = m_j$). Với lưu trữ số thực 32-bit, paper tính được d_j không tràn số cho tới 1.7×10^{37} phần tử, một con số "khá lớn" theo paper, nhưng nếu vector lớn hơn mức này cần chuyển sang số thực 64-bit.

3.2 Tính song song (Mục 3.1, công thức (3)-(4))

Định nghĩa phép toán hai ngôi $\oplus : \mathbb{R}^2 \times \mathbb{R}^2 \to \mathbb{R}^2$ trên cặp (max, tổng):

$$\begin{pmatrix} m_i \\ d_i \end{pmatrix} \oplus \begin{pmatrix} m_j \\ d_j \end{pmatrix} = \begin{pmatrix} \max(m_i, m_j) \\ d_i \cdot e^{m_i - \max(m_i, m_j)} + d_j \cdot e^{m_j - \max(m_i, m_j)} \end{pmatrix}. \qquad (4)$$

Áp dụng $\oplus$ liên tiếp từ trái sang phải cho các cặp $\binom{x_1}{1}, \binom{x_2}{1}, \dots, \binom{x_V}{1}$ tương đương chạy thuật toán tuần tự ở trên (công thức (3)). Paper nêu (không chứng minh chi tiết, "we omit the proofs for brevity") rằng $\oplus$ **kết hợp được (associative)** và **giao hoán được (commutative)**, nên có thể chia vector thành nhiều đoạn, tính (m, d) cục bộ cho mỗi đoạn độc lập (song song trên nhiều luồng GPU), rồi gộp bằng $\oplus$ theo cây giảm (tree reduction) theo thứ tự tuỳ ý.

3.3 Hợp nhất với TopK (Mục 4, Algorithm 4)

Hàm TopK(y) trả về K chỉ số và K giá trị lớn nhất của y (công thức (5)). Nếu chạy Safe Softmax rồi TopK tách biệt, tổng cộng tốn 5 lần truy cập bộ nhớ mỗi phần tử (4 của Safe Softmax + 1 đọc của TopK); dùng Online Softmax riêng rồi TopK riêng tốn 4 lần. Thuật toán 4 của paper **hợp nhất cả hai**: trong lúc quét một lượt để cập nhật (m_j, d_j), đồng thời giữ một buffer $K+1$ phần tử đã sắp xếp giảm dần (chèn phần tử mới vào đúng vị trí bằng một vòng lặp chèn, insertion sort cục bộ, dòng 10-15 của Algorithm 4), rồi ở cuối chỉ cần chuẩn hoá K giá trị trong buffer bằng m_V, d_V đã biết. Kết quả: **chỉ 1 lần truy cập bộ nhớ mỗi phần tử** cho toàn bộ Softmax+TopK (không cần ghi ra toàn bộ vector y kích thước V, chỉ cần ghi ra K giá trị).

Ví dụ tính tay (tự tính): xét vector $x = [1, 3, 2, 5]$ ($V = 4$). Theo công thức (2), Safe Softmax: $m = 5$; các số hạng $e^{1-5} = e^{-4} \approx 0.0183$, $e^{3-5} = e^{-2} \approx 0.1353$, $e^{2-5} = e^{-3} \approx 0.0498$, $e^{5-5} = 1$; tổng $d = 0.0183 + 0.1353 + 0.0498 + 1 \approx 1.2034$; $y \approx [0.0152,\ 0.1124,\ 0.0414,\ 0.8310]$ (tổng xấp xỉ 1, tự tính).

Chạy Algorithm 3 (Online Softmax) theo từng bước: $m_0 = -\infty, d_0 = 0$.

- $j = 1, x_1 = 1$: $m_1 = \max(-\infty, 1) = 1$; $d_1 = 0 \cdot e^{-\infty} + e^{1-1} = 1$.
- $j = 2, x_2 = 3$: $m_2 = \max(1, 3) = 3$; $d_2 = 1 \cdot e^{1-3} + e^{3-3} = e^{-2} + 1 \approx 0.1353 + 1 = 1.1353$.
- $j = 3, x_3 = 2$: $m_3 = \max(3, 2) = 3$; $d_3 = 1.1353 \cdot e^{3-3} + e^{2-3} = 1.1353 + e^{-1} \approx 1.1353 + 0.3679 = 1.5032$.
- $j = 4, x_4 = 5$: $m_4 = \max(3, 5) = 5$; $d_4 = 1.5032 \cdot e^{3-5} + e^{5-5} = 1.5032 \cdot e^{-2} + 1 \approx 1.5032 \times 0.1353 + 1 \approx 0.2034 + 1 = 1.2034$.

$d_4 \approx 1.2034$ khớp đúng với d tính trực tiếp theo công thức (2) ở trên (tự tính, kiểm chứng Định lý 1 trên ví dụ số cụ thể). Nhận xét: mỗi lần gặp max mới ($j = 2$ và $j = 4$), tổng cũ bị "chiết khấu" lại bằng hệ số $e^{m_{\text{cũ}} - m_{\text{mới}}} < 1$ trước khi cộng số hạng mới, đúng cơ chế rescale của thuật toán.

4. Thí nghiệm và kết quả chính

Benchmark thực hiện trên **GPU NVIDIA Tesla V100 PCIe 16 GB**, CUDA Toolkit 9.1, ECC bật, persistent mode bật, dùng thư viện CUB v1.8.0 cho phép reduction song song; mã nguồn benchmark công khai tại github.com/NVIDIA/online-softmax (theo paper). Tất cả thí nghiệm dùng kiểu số **fp32**.

Benchmark Softmax đơn (Mục 5.1, Hình 1-2): so 3 thuật toán Naive, Safe, Online trên nhiều kích thước vector V từ 10^2 đến 10^5 (hoặc 10^7 cho batch nhỏ), với 2 kích thước batch: 4.000 vector (mô phỏng huấn luyện/suy luận theo lô lớn) và 10 vector (mô phỏng suy luận trực tuyến từng mẫu nhỏ).

- Batch lớn (4.000): cả 3 thuật toán tương đương tới $V = 1000$; sau đó bộ nhớ cache L1/L2 bị "thrashing" (giao động liên tục), hiệu năng bị giới hạn bởi băng thông DRAM; Online và Naive (đều chỉ cần 3 truy cập) vượt Safe (4 truy cập), đạt khoảng **1.3x** tại $V = 4000$, gần đúng tỉ lệ giảm truy cập bộ nhớ lý thuyết $4/3 \approx 1.33$ (tự tính: $4/3 = 1.333$).
- Batch nhỏ (10 vector): GPU không được dùng hết (chỉ 10 threadblock), hiệu năng tuyệt đối thấp hơn hẳn, hiện tượng latency chi phối; Online/Naive vượt Safe khoảng **1.15x** cho $V \geq 1000$ (thấp hơn mức 1.3x của batch lớn, vì không còn bị giới hạn thuần bởi băng thông).

Benchmark Softmax+TopK (Mục 5.2, Hình 3-4), $K = 5$**:** so Safe Softmax+TopK chạy tách biệt, Safe Softmax+TopK hợp nhất (fused), và Online Softmax+TopK hợp nhất.

- Batch lớn (4.000): cải thiện bắt đầu từ **1.5x**, tăng dần theo V, đạt gần **5x** tại $V = 25000$; paper phân tích 5x này = 2.5x từ việc hợp nhất (fusion, giảm từ 5 xuống ~2 lượt truy cập tổng thể) **nhân** 2x từ chính Online Softmax (giảm thêm so với Safe).
- Batch nhỏ (10 vector): cải thiện **1.5x-2.5x**, không đạt 5x vì vùng latency-bound (GPU chưa bão hoà), nhưng vẫn có lợi.

- Với K lớn hơn, cải thiện giảm dần: **3.5x cho** $K = 10$**, 2x cho** $K = 15$**, 1.4x cho** $K = 30$, vì chi phí chèn-sắp-xếp (insertion sort) buffer TopK là $O(K)$ mỗi phần tử, dần chiếm ưu thế so với chi phí Softmax khi K tăng.

Kết quả tổng hợp (Mục 6): Online Softmax giảm truy cập bộ nhớ 1.33 lần (lý thuyết) cho softmax đơn, đo được **1.15x đến 1.3x** cải thiện tốc độ thực tế tuỳ chế độ batch/kích thước vector; khi hợp nhất với TopK, đạt **1.5x đến 5x** cải thiện; và các cải thiện này **độc lập (orthogonal)** với các kỹ thuật tối ưu softmax khác (Hierarchical Softmax, Importance Sampling, SVD-Softmax) theo nhận định của paper, vì Online Softmax chỉ thay đổi cách tính softmax gốc, không đổi thuật toán cấp cao hơn.

Bài báo **không có bảng số liệu dạng "accuracy/benchmark mô hình học máy"** (không so sánh độ chính xác mô hình ngôn ngữ hay bất kỳ task ML cụ thể nào); toàn bộ "kết quả chính" là đo tốc độ/throughput (elements per second) trên một loại GPU cụ thể, đúng tính chất của một báo cáo kỹ thuật hiệu năng (performance engineering report), không phải nghiên cứu mô hình.

5. Phân tích / ablation

- **Naive so với Safe so với Online:** Naive và Online có cùng số lần truy cập bộ nhớ (3), nên tốc độ tương đương trên benchmark; khác biệt chỉ ở **độ an toàn số học** (Naive có thể tràn số, Online thì không) với "chi phí thêm gần như không đáng kể" (paper dùng từ "negligible additional cost of two operations per vector element"). Đây là điểm bán hàng chính: Online Softmax cho "miễn phí" (gần như không mất tốc độ) sự an toàn số học mà trước đây phải đánh đổi bằng cách chạy Safe Softmax chậm hơn.
- **Batch lớn so với batch nhỏ:** sự khác biệt về mức cải thiện (1.3x so với 1.15x; 5x so với 1.5-2.5x) cho thấy lợi ích của việc giảm truy cập bộ nhớ chỉ **phát huy tối đa khi hệ thống bị giới hạn bởi băng thông bộ nhớ** (memory-bandwidth-bound, batch lớn, V lớn); khi hệ thống bị giới hạn bởi độ trễ (latency-bound, batch nhỏ, GPU chưa bão hoà), lợi ích vẫn có nhưng nhỏ hơn.
- **Ảnh hưởng của** K **trong TopK fusion:** cải thiện giảm đơn điệu khi K tăng (5x ở $K = 5$ xuống 1.4x ở $K = 30$), vì chi phí chèn-sắp-xếp tăng tuyến tính theo K mỗi phần tử trong khi lợi ích từ giảm truy cập bộ nhớ là hằng số; đây là một đánh đổi (trade-off) rõ ràng mà paper trình bày định lượng.
- **Fusion so với chỉ đổi thuật toán:** phân tích 5x = 2.5x (fusion) × 2x (online) cho thấy **hai nguồn cải thiện độc lập có thể cộng gộp theo tích**, không chỉ theo tổng, một quan sát hữu ích khi thiết kế kernel GPU hợp nhất nhiều phép toán.

6. Giới hạn

Do tác giả nêu (Mục 7, Discussion):

- Chỉ benchmark trên **một loại GPU** (Tesla V100), không có thí nghiệm trên CPU; paper suy đoán (không kiểm chứng) rằng nếu mã nguồn được vector hoá tốt thì CPU cũng có thể đạt cải thiện tương tự.
- Online Softmax và Softmax+TopK fused "vẫn bị giới hạn bởi băng thông bộ nhớ" (still limited by the memory bandwidth); paper gợi ý hướng cải thiện tiếp theo là hợp nhất với **lớp đứng trước** (ví dụ phép nhân ma trận chiếu) để tránh một lượt đọc/ghi bộ nhớ round-trip nữa, nhưng tự nhận "this change is more challenging" và không triển khai trong bài.
- Lợi ích của TopK fusion giảm mạnh khi K lớn (xuống 1.4x ở $K = 30$ và "degrades further for bigger Ks"), nên kỹ thuật này chủ yếu có giá trị cho K nhỏ (ví dụ beam search với beam width nhỏ).
- Đây là "Preprint. Work in progress.", không qua bình duyệt chính thức (peer review) tại một venue cụ thể.

Nhận xét:

- Toàn bộ benchmark dùng **fp32**; paper không thử nghiệm với fp16/bf16 (các kiểu số phổ biến hơn trong huấn luyện/suy luận LLM hiện nay), nên các số liệu tốc độ cụ thể (1.3x, 5x) không chắc giữ nguyên trên các kiểu số khác hoặc trên GPU đời mới hơn V100 (bài viết năm 2018).

- Không có thí nghiệm nào mô phỏng workload thực tế của attention trong Transformer (chuỗi dài, nhiều ”hàng” softmax đồng thời như trong self-attention); các con số benchmark chỉ cho một hàm softmax độc lập trên vector/batch vector, chưa tính tới chi phí/đặc thù khi softmax là một bước trong một kernel attention lớn hơn (đây chính là việc FlashAttention giải quyết sau này, nhưng không phải nội dung của bài báo 2018 này).
- Không có phân tích độ phức tạp hình thức (Big-O) cho overhead của thuật toán song song (Mục 3.1) khi số luồng tăng; chỉ nói ”enables parallel evaluation” mà không đo riêng hiệu năng của phiên bản song song so với tuần tự.

7. Liên hệ với các bài khác

- **Attention Is All You Need** (bài 01, phần LLM): attention dùng softmax trên mỗi hàng của ma trận điểm $QK^{\mathsf{T}}/\sqrt{d}$.
- **FlashAttention và FlashAttention-2** (bài 44–45, phần LLM): phép cập nhật cực đại và tổng chuẩn hóa có thể kết hợp qua các khối, giúp tính attention mà không cần lưu toàn bộ ma trận $N \times N$ trong bộ nhớ HBM. Đây là liên hệ về công thức; không hàm ý quan hệ trích dẫn giữa các bài.
- **math 16** (FP8 Formats for Deep Learning, bài tiếp theo cùng thư mục): cùng chủ đề số học hiệu năng (numerics) cho deep learning trên phần cứng thật, nhưng ở tầng khác (định dạng số FP8 cho lưu trữ/tính toán, so với thuật toán tính softmax ổn định ở tầng này).
- **math 18** (The Shattered Gradients Problem): khác chủ đề (ổn định huấn luyện mạng sâu qua gradient, không phải numerics suy luận), nhưng cùng tinh thần ”phân tích chi tiết hành vi số học/thống kê của một phép toán cơ bản trong deep learning”.

8. Gợi ý áp dụng

Online softmax hữu ích khi xử lý dữ liệu theo khối hoặc khi không muốn lưu toàn bộ giá trị trung gian. Cần duy trì cực đại và hệ số chuẩn hóa nhất quán để phép tính ổn định số; cách tổ chức phép tính còn phụ thuộc phần cứng.

9. Thí nghiệm gợi ý

1. **Kiểm chứng tính đúng đắn của Online Softmax trên CPU/GPU nhỏ:** viết 3 hàm Naive/Safe/Online softmax bằng NumPy hoặc PyTorch trên vector ngẫu nhiên (vài nghìn phần tử, giá trị logit lớn để test overflow), so sánh đầu ra Online với `torch.softmax` (Safe); đo sai số tuyệt đối tối đa. Kỳ vọng: sai số ở mức epsilon máy (khoảng 10^{-7} với fp32), xác nhận Định lý 1; thử Naive với logit lớn (ví dụ nhân 100) để quan sát NaN/Inf xuất hiện, minh hoạ lý do cần phiên bản an toàn.
2. **Đo tốc độ fused Softmax+TopK ở mức Python (giới hạn):** implement một vòng lặp Python/PyTorch tính softmax rồi `topk` tách biệt, so với một cách tính ”giả lập hợp nhất” (tính trong cùng một hàm, tránh ghi tensor trung gian ra bộ nhớ nhiều lần qua `torch.no_grad()` và viết thủ công). Đo thời gian với $K \in \{5, 10, 30\}$. Kỳ vọng: cải thiện ở mức Python nhỏ hơn nhiều so với 5x của paper (vì overhead Python/kernel launch chi phối, không phải băng thông bộ nhớ thuần như CUDA kernel chuyên dụng); kết luận: lợi ích thật của kỹ thuật này chỉ hiện rõ ở tầng kernel CUDA viết tay, không phải ở tầng framework cấp cao.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / giá trị
Softmax gốc	$y_i = e^{x_i} / \sum_j e^{x_j}$, (1)
Safe softmax	$y_i = e^{x_i - \max_k x_k} / \sum_j e^{x_j - \max_k x_k}$, (2)
Đệ quy Online Softmax	$m_j = \max(m_{j-1}, x_j)$; $d_j = d_{j-1} e^{m_{j-1} - m_j} + e^{x_j - m_j}$

Khái niệm	Công thức / giá trị
Phép kết hợp song song	$\binom{m_i}{d_i} \oplus \binom{m_j}{d_j} = \binom{\max(m_i,m_j)}{d_i e^{m_i-\max}+d_j e^{m_j-\max}}$, (4)
Số lần truy cập bộ nhớ/phần tử	Naive: 3; Safe: 4; Online: 3; Online+TopK fused: 1
Cận an toàn của d_j	$1 \le d_j \le j$; fp32 an toàn tới 1.7×10^{37} phần tử
Tốc độ Softmax đơn (V100, fp32, batch 4000)	$\approx$ 1.3x so với Safe tại $V = 4000$
Tốc độ Softmax đơn (batch 10)	$\approx$ 1.15x so với Safe tại $V \ge 1000$
Tốc độ Softmax+TopK ($K = 5$, batch 4000)	tới $\approx$ 5x ($= 2.5\times$ fusion $\times\, 2\times$ online) tại $V = 25000$
Suy giảm theo K (Softmax+TopK, batch 4000)	5x ($K = 5$) $\to$ 3.5x ($K = 10$) $\to$ 2x ($K = 15$) $\to$ 1.4x ($K = 30$)

11. Kết luận ngắn

Bài báo kỹ thuật ngắn này giải quyết một vấn đề thực dụng: softmax ”an toàn” tiêu chuẩn cần 3 lượt quét/4 lần truy cập bộ nhớ mỗi phần tử; bằng một công thức đệ quy tính đồng thời max và tổng mẫu số (lấy cảm hứng từ thuật toán phương sai trực tuyến của Welford), Online Softmax giảm xuống 3 lần truy cập mà vẫn giữ an toàn số học, đo được cải thiện tốc độ 1.15x-1.3x trên GPU Tesla V100; khi hợp nhất thêm với TopK (cho beam search), cải thiện lên tới 5x với K nhỏ. Giá trị lâu dài của bài này vượt xa các con số benchmark cụ thể năm 2018: chính công thức đệ quy/phép kết hợp $\oplus$ ở đây là nền tảng toán học cho kỹ thuật ”online softmax theo block” trong FlashAttention, công cụ gần như bắt buộc để chạy attention hiệu quả về bộ nhớ trên các mô hình hiện đại, trong nhiều mô hình hiện đại.

Nguồn gốc: https://arxiv.org/abs/1805.02867

Bài 16

FP8 Formats for Deep Learning (2022)

Tác giả: Paulius Micikevicius, Dusan Stosic, Patrick Judd, John Kamalu, Stuart Oberman, Mohammad Shoeybi, Michael Siu, Hao Wu (NVIDIA); Neil Burgess, Sangwon Ha, Richard Grisenthwaite (Arm); Naveen Mellempudi, Marius Cornea, Alexander Heinecke, Pradeep Dubey (Intel)
Venue / arXiv: arXiv:2209.05433 (v2, 29/09/2022); paper không nêu venue hội nghị/journal cụ thể (là một technical report/whitepaper liên công ty NVIDIA-Arm-Intel, công bố kèm đề xuất chuẩn)
Nguồn: https://arxiv.org/abs/2209.05433
Chủ đề: Đề xuất một chuẩn định dạng số thực dấu phẩy động 8-bit (FP8) gồm hai biến thể E4M3 và E5M2 để huấn luyện và suy luận mạng neural, kèm thực nghiệm trên nhiều kiến trúc và kích cỡ mô hình (đến 175 tỷ tham số).
Vị trí trong lộ trình: đọc sau math 15 (Online normalizer calculation for softmax, cùng chủ đề ”làm toán số học hiệu quả trên phần cứng”), có thể đọc cùng lúc hoặc trước opt 42 (Mixed Precision Training, FP16) vì FP8 là bước tiếp theo tự nhiên của FP16 mixed-precision; nên đọc trước các bài lượng tử hoá LLM (llm 51 LLM.int8, llm 52 SmoothQuant, llm 53 GPTQ, llm 54 AWQ) để hiểu sự khác biệt giữa lượng tử hoá dấu phẩy động (floating-point, không đều) và lượng tử hoá nguyên (int8, đều).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (bản arXiv); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc.

1. Bối cảnh

Một số dấu phẩy động gồm bit dấu (sign, S), phần mũ (exponent, E) và phần định trị (mantissa hay trailing significand, M). Phần mũ ảnh hưởng đến dải giá trị biểu diễn được, còn phần định trị ảnh hưởng đến độ chính xác. Theo cách dùng trong bài và chuẩn IEEE 754, “mantissa” ở đây chỉ trường trailing significand, không tính bit 1 ẩn của số chuẩn hóa. Giá trị của một số chuẩn hóa là:

$$(-1)^S \times 1.M_2 \times 2^{(E_2 - \text{bias})}$$

trong đó M_2 là mantissa đọc theo nhị phân (dạng $1.xxx$), bias là độ lệch mũ để cho phép biểu diễn cả số mũ âm bằng một số nguyên không dấu.

Bối cảnh rộng hơn: các mô hình deep learning (DL) ngày càng lớn (GPT-3, Turing-Megatron, PaLM, OPT được paper trích dẫn ở phần Introduction) cần hàng nghìn processor chạy trong nhiều tuần. Biểu diễn số với độ chính xác thấp hơn (reduced precision) là ”trụ cột” (corner stone, nguyên văn dòng 44 có lỗi đánh máy ”corner stoen”) để tăng tốc huấn luyện/suy luận. Các định dạng phổ biến trước FP8 gồm: FP32 (single precision, IEEE), TF32 (dùng cho Tensor Core NVIDIA), FP16 (IEEE half precision), bfloat16 (BF16, Google). Với suy luận (inference), int8 cố định điểm (fixed-point) là lựa chọn phổ biến, nhưng paper nêu rõ ”trong một số trường hợp int8 gặp khó khăn để duy trì độ chính xác cần cho việc triển khai ứng dụng” (dòng 48-49, trích [1] Facebook inference accelerator deployment). Những hướng cực đoan hơn như mạng 1-bit (binary networks) được paper liệt ra (BinaryConnect, Binarized Neural Networks, XNOR-Net, DoReFa-Net) nhưng ”không thành công trong việc duy trì chất lượng kết quả cần cho nhiều ứng dụng thực tế”.

Các công trình tiền nhiệm trực tiếp mà paper tự liệt ra trong Introduction (đoạn 2 của §1): Wang et al. [22] (NeurIPS 2018) dùng exponent 5-bit để huấn luyện CNN trên CIFAR-10/ILSVRC12; Mellempudi et al. [12] (2019) nghiên cứu định dạng exponent 5-bit cho CNN lớn hơn và mạng dịch ngôn ngữ (RNN, Transformer); cả hai đều nghiên cứu weight update 16-bit và stochastic rounding; Sun et al. [20] (NeurIPS 2019, ”HFP8”)

giới thiệu việc dùng **hai** định dạng FP8 (exponent 4-bit và 5-bit) cho huấn luyện, nghiên cứu nhiều CNN, speech, dịch ngôn ngữ, và cũng khảo sát suy luận FP8 của mạng huấn luyện ở độ chính xác cao hơn, đưa ra kỹ thuật ”returning of batch normalization statistics” để cải thiện độ chính xác; Noune et al. [16] (2022) đề xuất biến thể FP8 dành một encoding duy nhất cho giá trị đặc biệt để tăng dynamic range. Bài hiện tại (2209.05433) là nỗ lực hợp nhất của 3 công ty (NVIDIA, Arm, Intel) để đưa ra **một chuẩn interchange format** duy nhất (không phải chỉ một kỹ thuật nghiên cứu), nhằm các nhà sản xuất phần cứng khác nhau có thể tương thích với nhau khi trao đổi tensor ở dạng FP8.

2. Ý tưởng chính

Bài báo định nghĩa hai định dạng số dấu phẩy động 8 bit, đặt tên theo số bit phần mũ và phần định trị:

- **E4M3**: 1 bit dấu + 4 bit exponent + 3 bit mantissa.
- **E5M2**: 1 bit dấu + 5 bit exponent + 2 bit mantissa.

Lý do cần **hai** định dạng (không chỉ một) là có sự đánh đổi (trade-off) giữa dynamic range (thang đo giá trị biểu diễn được, do số bit exponent quyết định) và độ chính xác (precision, do số bit mantissa quyết định). Tensor trọng số (weight) và activation thường có phân phối giá trị tương đối hẹp quanh 0 nên cần **độ chính xác cao hơn, dynamic range thấp hơn** → dùng E4M3. Tensor gradient trong lan truyền ngược (backward pass) thường có dynamic range rộng hơn rất nhiều (nhiều giá trị rất nhỏ gần 0 và một số giá trị lớn) nên cần **dynamic range cao hơn, có thể chấp nhận độ chính xác thấp hơn** → dùng E5M2. Paper khuyến nghị rõ: ”khuyến nghị sử dụng E4M3 cho tensor trọng số và activation, và E5M2 cho tensor gradient” (dòng 122, ”The recommended use of FP8 encodings is E4M3 for weight and activation tensors, and E5M2 for gradient tensors”), đồng thời nói một số mạng cần cả hai loại cùng lúc.

Ý tưởng thứ hai, mang tính triết lý thiết kế: **giữ tối đa quy ước IEEE 754**, chỉ lệch khỏi chuẩn khi lợi ích cho ứng dụng DL là đáng kể. Cụ thể, E5M2 giữ nguyên hoàn toàn quy ước IEEE (có vô cực, có nhiều mã NaN, có số 0 có dấu), còn E4M3 ”hi sinh” việc biểu diễn số vô cực (infinity) và chỉ giữ **một** mẫu bit cho NaN, để ”lấy lại” không gian mã đó cho các giá trị số bình thường lớn hơn, mở rộng dynamic range. Đây là đánh đổi có chủ đích: trong DL, giá trị tràn số (overflow) thường được xử lý bằng cách bão hoà (saturate) về giá trị lớn nhất biểu diễn được, nên không cần một mã riêng cho ”vô cực” như toán học thuần túy cần.

3. Phương pháp chi tiết

3.1 Bảng mã hoá chi tiết (Table 1 của paper)

Paper dùng ký hiệu S.E.M (ví dụ S.1111.1102) để biểu diễn chuỗi bit: S là bit dấu, nhóm tiếp là exponent (4 hoặc 5 bit), nhóm cuối là mantissa (3 hoặc 2 bit), số có chỉ số dưới 2 là số nhị phân, không có chỉ số là số thập phân. Nguyên văn Table 1:

	E4M3	E5M2
Exponent bias	7	15
Infinities	N/A (không biểu diễn)	$S.11111.00_2$
NaN	$S.1111.111_2$	$S.11111.\{01, 10, 11\}_2$
Zeros	$S.0000.000_2$	$S.00000.00_2$
Max normal	$S.1111.110_2 = 1.75 \times 2^8 = 448$	$S.11110.11_2 = 1.75 \times 2^{15} = 57\,344$
Min normal	$S.0001.000_2 = 2^{-6}$	$S.00001.00_2 = 2^{-14}$
Max subnormal	$S.0000.111_2 = 0.875 \times 2^{-6}$	$S.00000.11_2 = 0.75 \times 2^{-14}$
Min subnormal	$S.0000.001_2 = 2^{-9}$	$S.00000.01_2 = 2^{-16}$

Giải thích các khái niệm trong bảng cho người chưa quen:

- **Exponent bias**: số cộng/trừ để đổi exponent (được lưu dưới dạng số nguyên không âm trong các bit) thành exponent thực (có thể âm). Với E4M3, bias = 7, nên exponent field $E_2 = 1111_2 = 15$ (thập phân) tương ứng exponent thực $15 - 7 = 8$.
- **Số chuẩn hoá (normal)**: có bit ẩn đầu là 1, công thức $(-1)^S \times 1.M \times 2^{E-\text{bias}}$.
- **Số subnormal (hay denormal)**: khi phần exponent toàn 0, bit ẩn đầu trở thành 0 (không phải 1), công thức $(-1)^S \times 0.M \times 2^{1-\text{bias}}$, cho phép biểu diễn các số rất gần 0 (nhỏ hơn min normal) với độ chính xác giảm dần (gradual underflow), đổi lại để tránh "rơi thẳng về 0" đột ngột.
- **Max normal** của E4M3: pattern exponent 1111_2 = 15 nhưng **không** dùng để biểu diễn vô cực (vì E4M3 bỏ hẳn vô cực), nên pattern mantissa 110_2 (không phải 111_2, pattern đó dành cho NaN) với exponent lớn nhất 1110_2... paper viết trực tiếp là S.1111.110_2, nghĩa là exponent field vẫn là 1111 (=15, thực là $15 - 7 = 8$), mantissa 110_2 = 0.75, tổng mantissa đầy đủ $1 + 0.75 = 1.75$, giá trị $= 1.75 \times 2^8 = 1.75 \times 256 = 448$. (tự tính: $1.75 \times 256 = 448$, khớp số paper ghi.)

3.2 Mở rộng dynamic range của E4M3 (§3.1)

Đây là điểm kỹ thuật quan trọng nhất của thiết kế. Với quy ước IEEE 754 chuẩn, pattern exponent toàn 1 (1111_2) được dành riêng cho vô cực (mantissa = 0) và NaN (mantissa $\neq$ 0, có nhiều mẫu NaN khác nhau). Nếu E4M3 tuân theo đúng quy ước này, giá trị lớn nhất biểu diễn được chỉ là khi exponent field = 1110_2 (=14, thực $14 - 7 = 7$) với mantissa lớn nhất 111_2=0.875, tức $1.875 \times 2^7 = 240$ (paper nói rõ: "Giá trị lớn nhất biểu diễn được nếu không có sửa đổi này sẽ là 240", dòng 159).

Bằng cách **bỏ vô cực** và **chỉ giữ một mẫu NaN** (S.1111.111_2), E4M3 "lấy lại" 7 mẫu bit mantissa còn lại ở exponent field = 1111_2 (mantissa từ 000 đến 110, trừ 111 dành cho NaN) để biểu diễn 7 giá trị số bình thường mới: 256, 288, 320, 352, 384, 416, 448 (paper liệt kê chính xác các số này ở dòng 158). Việc này "mở rộng dynamic range thêm một luỹ số 2" (extends the dynamic range by one extra power of 2), từ 17 lên 18 binade. Khái niệm **binade**: một "binade" là một dải giá trị giữa hai luỹ thừa liên tiếp của 2 (ví dụ từ 2^k đến 2^{k+1}); "17 binade" nghĩa là khoảng cách giữa giá trị nhỏ nhất và lớn nhất biểu diễn được trải qua 17 lần nhân đôi liên tiếp.

Paper cũng giải thích lý do **không** đi xa hơn (bỏ luôn việc phân biệt +0/-0 và dùng chung một mã NaN duy nhất, có thể biểu diễn thêm số 480): việc này sẽ phá vỡ tính đối xứng dấu dương/âm cố hữu của IEEE 754, làm hỏng các thuộc tính thực thi quan trọng như "so sánh và sắp xếp số dấu phẩy động bằng phép toán số nguyên" (dòng 163-164: "IEEE floating point formats allow comparison and sorting of floating point values using integer operations"). Lợi ích tăng từ 448 lên 480 "không đáng để đánh đổi" quy ước này.

Ngược lại, **E5M2 giữ đầy đủ quy ước IEEE** cho giá trị đặc biệt (có vô cực S.11111.00_2, nhiều mã NaN). Lý do: E5M2 đã có 32 binade (nhờ 5 bit exponent), lợi ích của việc "lấy lại" không gian mã dành cho đặc biệt chỉ thêm được 3 giá trị số nữa (do mantissa chỉ 2 bit) - "một binade bổ sung ít quan trọng hơn hẳn khi E5M2 đã có 32 (so với E4M3 chỉ có 17 nếu không chỉnh sửa)". Ngoài ra việc giữ đúng IEEE giúp chuyển đổi (convert) thẳng giữa E5M2 và FP16 dễ dàng ("straightforward conversion between E5M2 and IEEE FP16 formats", dòng 147-148), vì E5M2 có thể coi là "FP16 với ít bit mantissa hơn" giống như cách bfloat16/TF32 được coi là "FP32 với ít bit hơn".

3.3 Exponent bias cố định, scaling factor xử lý trong software (§3.2)

Cả hai định dạng **giữ bias kiểu IEEE** (E4M3: bias=7, E5M2: bias=15), không dùng "programmable exponent bias" (bias có thể cấu hình được theo tensor). Lý do paper đưa ra: dùng **scaling factor** (hệ số nhân, lưu ở độ chính xác cao hơn, ví dụ FP32) trước khi chuyển một tensor sang FP8 đạt được hiệu quả tương đương với việc thay đổi bias, nhưng linh hoạt hơn, vì scaling factor "có thể nhận bất kỳ giá trị thực nào" còn programmable bias "chỉ tương đương cho phép các luỹ thừa của 2 làm scaling factor". Cơ chế này tương tự hệt với **loss scaling** trong mixed-precision training FP16 (paper trích dẫn trực tiếp Mixed Precision Training, [14] = arXiv 1710.03740, chính là opt 42 trong lộ trình này): gradient được nhân với một hệ số trước khi ép về FP16/FP8 để tránh bị tràn dưới (underflow) về 0, rồi được chia lại hệ số đó sau khi tính toán ở độ chính xác cao hơn.

Khác biệt quan trọng so với FP16 automatic mixed precision: với FP8, **không nên** dùng chiến lược ”bỏ qua cập nhật trọng số khi tràn số rồi giảm scaling factor” (weight update skipping) như FP16 AMP vẫn làm, vì ”tràn số có khả năng xảy ra nhiều hơn hẳn do dynamic range hẹp hơn, dẫn đến quá nhiều lượt cập nhật bị bỏ qua”. Thay vào đó FP8 dùng **bão hoà (saturation)**: giá trị vượt ngưỡng tối đa bị cắt về giá trị lớn nhất biểu diễn được, không bị coi là lỗi cần bỏ qua bước cập nhật.

Paper cũng mô tả quy trình tính toán thực tế: phép toán toán học trên input FP8 (ví dụ matrix-multiplication/dot-product trong một lớp fully-connected hoặc convolution, gọi chung là **GEMM**, viết tắt General Matrix Multiply) tạo ra **output ở độ chính xác cao hơn** (ví dụ FP32), không viết trực tiếp ra FP8; chỉ khi cần lưu vào bộ nhớ (memory) thì mới convert (tuỳ chọn) về FP8. Các phép toán ít nặng về số học hơn (nonlinearity, normalization, cập nhật trọng số bởi optimizer) thường bị giới hạn bởi băng thông bộ nhớ (memory-bandwidth limited) hơn là bởi tốc độ tính toán, nên ”không nhạy với một phép toán số học bổ sung cho mỗi giá trị” khi phải unscale (nhân nghịch đảo scaling factor).

3.4 Ví dụ số tính tay (tự tính)

Để hiểu rõ cơ chế biểu diễn và sai số làm tròn, ta thử biểu diễn số thực $x = 300$ bằng E4M3.

Bước 1: tìm luỹ thừa của 2 gần nhất không vượt quá 300: $2^8 = 256 \le 300 < 512 = 2^9$. Vậy exponent thực $= 8$, và dạng chuẩn hoá $x = 1.m \times 2^8$ với $1.m = 300/256 = 1.171875$.

Bước 2: $m = 0.171875$. Với 3 bit mantissa, các giá trị m biểu diễn được là các số nguyên $k/8$ với $k = 0, \dots, 7$ (vì $2^3 = 8$ mức). $0.171875 \times 8 = 1.375$. Số nguyên gần nhất là $k = 1$, cho $m = 1/8 = 0.125$, hoặc $k = 2$ cho $m = 2/8 = 0.25$.

- Với $k = 1$: giá trị biểu diễn $= 1.125 \times 256 = 288$.
- Với $k = 2$: giá trị biểu diễn $= 1.25 \times 256 = 320$.

300 cách 288 là 12, cách 320 là 20, nên làm tròn đến số gần nhất (round-to-nearest-even) sẽ chọn 288. Sai số tuyệt đối là $|300 - 288| = 12$ (tự tính), sai số tương đối là $12/300 \approx 4\%$ (tự tính). Đây minh hoạ trực tiếp bảng Table 1: 288 chính là một trong 7 giá trị ”lấy lại” được nhờ bỏ vô cực mà paper liệt ở dòng 158 (256, 288, 320, ...), nên ví dụ này nằm đúng trong vùng dynamic range mở rộng của E4M3.

Nếu dùng E5M2 cho cùng số $x = 300$: mantissa chỉ có 2 bit, các mức $m = k/4$, $k = 0, \dots, 3$. $0.171875 \times 4 = 0.6875$, gần nhất là $k = 1$ ($m = 0.25$, giá trị $= 320$) hoặc $k = 0$ ($m = 0$, giá trị $= 256$). 300 cách 256 là 44, cách 320 là 20, chọn 320, sai số tuyệt đối 20, sai số tương đối $\approx 6.7\%$ (tự tính). Kết luận tự rút ra (suy luận của người ghi chú, không phải paper nói trực tiếp bằng con số này): với cùng một giá trị cụ thể, E4M3 (nhiều bit mantissa hơn) cho sai số làm tròn nhỏ hơn E5M2, đúng với nguyên lý thiết kế của paper rằng E4M3 ưu tiên độ chính xác còn E5M2 ưu tiên dynamic range.

4. Thí nghiệm và kết quả chính

Bài báo nhấn mạnh phương pháp luận thực nghiệm: dùng **FP8 mô phỏng (simulated FP8)**, nghĩa là giá trị tensor được ”cắt” (clip) về đúng các giá trị có thể biểu diễn trong FP8 (bao gồm cả áp dụng scaling factor và bão hoà), nhưng **phép tính số học thực tế vẫn thực hiện ở độ chính xác rộng hơn** (FP16 hoặc bfloat16). Lý do nêu rõ: định dạng interchange là trọng tâm của paper (vì các hãng phần cứng khác nhau có thể chọn cách hiện thực phép toán vector/matrix khác nhau), và việc giả lập số học FP8 thật trên phần cứng chưa hỗ trợ sẽ ”quá chậm cho việc huấn luyện các mô hình lớn”.

Table 2 - Image Classification (ILSVRC12, Top-1 Accuracy validation), paper so Baseline (FP16/bfloat16) với FP8, các mô hình: VGG-16 (71.27 vs 71.11), VGG-16 BN (73.95 vs 73.69), Inception v3 (77.23 vs 77.06), DenseNet 121 (75.59 vs 75.33), DenseNet 169 (76.97 vs 76.83), ResNet18 (70.58 vs 70.12), ResNet34 (73.84 vs 73.72), ResNet50 v1.5 (76.71 vs 76.76, FP8 còn **cao hơn** baseline), ResNet101 v1.5 (77.51 vs 77.48), ResNeXt50 (77.68 vs 77.62), Xception (79.46 vs 79.17), MobileNet v2 (71.65 vs 71.04, chênh lệch lớn nhất

trong bảng, 0.61 điểm, tự tính: $71.65 - 71.04 = 0.61$), DeiT small (80.08 vs 80.02, mô hình Transformer-based duy nhất trong bảng này). Bài báo lưu ý xét: ”ngoại trừ MobileNet v2, độ chính xác đạt được bởi huấn luyện FP8 nằm trong biến thiên run-to-run của huấn luyện độ chính xác cao hơn”, và nói rõ ”chúng tôi tiếp tục nghiên cứu để khôi phục phần độ chính xác còn thiếu cho MobileNet v2”, tức là **bài báo lưu ý một hạn chế cụ thể**, không che giấu.

Table 3 - Language Translation (WMT 2016 English→German, BLEU qua sacreBLEU trên newstest2014): GNMT (RNN-based, 24.83 vs 24.65), Transformer Base (26.87 vs 26.83), Transformer Large (28.43 vs 28.35). Tất cả đều ”trong biên độ biến thiên run-to-run”.

Table 4 - NLP Models, Perplexity (thấp hơn tốt hơn): Transformer-XL Base (22.98 vs 22.99), Transformer-XL Large (17.80 vs 17.75, FP8 tốt hơn), GPT 126M (19.14 vs 19.24), GPT 1.3B (10.62 vs 10.66), GPT 5B (8.94 vs 8.98), GPT 22B (7.21 vs 7.24), **GPT 175B (6.65 vs 6.68)**. Paper ghi chú quan trọng: với mô hình 175B, ”perplexity được báo cáo ở 75% quá trình huấn luyện, vì run baseline bfloat16 chưa hoàn thành” - đây là một **giới hạn thực nghiệm được paper tự khai báo rõ ràng**, không phải so sánh đầy đủ 100% huấn luyện. Dù vậy, ”phiên huấn luyện FP8 đã hoàn thành và loss curve của nó phù hợp với một quá trình huấn luyện thành công” (Figure 1 minh hoạ perplexity theo số iteration chuẩn hoá cho nhiều mô hình GPT-3).

Table 5 - Post-training quantization (PTQ) của model huấn luyện FP16, so int8 vs E4M3: BERT Base trên SQuAD v1.1 (F1): 16-bit FP = 88.19, int8 = 76.89 (giảm mạnh, mất $88.19 - 76.89 = 11.3$ điểm, tự tính), E4M3 = 88.09 (chỉ giảm 0.1 điểm, tự tính). BERT Large: 16-bit = 90.87, int8 = 89.65, E4M3 = 90.94 (FP8 **vượt** baseline). GPT3 126M trên wikitext103 (perplexity): 16-bit = 19.01, int8 = 28.37 (tăng mạnh/tệ hơn hẳn), E4M3 = 19.43. GPT3 1.3B: 16-bit = 10.19, int8 = 12.74, E4M3 = 10.29. GPT3 6.7B: 16-bit = 8.51, int8 = 10.29, E4M3 = 8.41 (FP8 vượt baseline). Kết luận chính của bảng này (paper nói trực tiếp, dòng 293-296): ”BERT trên SQuAD cho thấy FP8 PTQ duy trì độ chính xác trong khi int8 PTQ dẫn tới mất độ chính xác đáng kể... GPT trên wikitext103 cho thấy FP8 PTQ tốt hơn hẳn trong việc giữ độ chính xác so với int8”. Paper cũng thử ”cast tensor sang FP8 mà không áp dụng scaling factor” và thấy ”mất độ chính xác đáng kể, tăng perplexity lên 11.0” - đây là minh chứng trực tiếp cho tầm quan trọng của scaling factor đã nói ở mục 3.3.

5. Phân tích / ablation

Mục §4.3 ”Per-tensor scaling factors” là phần phân tích sâu nhất của paper, dùng Figure 2: đo perplexity suy luận trên wikitext103 khi lấy một mô hình GPT3 1.3B đã huấn luyện bằng bfloat16, rồi cast weight/activation sang E4M3 với **các giá trị exponent bias khác nhau**, không dùng calibration, không dùng per-tensor scaling riêng (tức là dùng đúng cơ chế ”bias cố định” thay cho ”scaling factor mềm” để xem khi nào nó đủ dùng).

Kết quả: khi chỉ quantize các tensor input cho phép toán GEMM (bao gồm cả hai phép nhân ma trận theo batch trong attention), có ”một vài lựa chọn exponent bias trong khoảng [7, 10] cho kết quả khớp baseline bfloat16” - tức là tồn tại một vùng bias hoạt động tốt, không chỉ một giá trị duy nhất, cho thấy có biên độ dung sai (margin). Nhưng khi **quantize thêm cả tensor residual connection** (input cho phép toán Add, giúp giảm thêm áp lực lưu trữ/băng thông bộ nhớ), ”không một giá trị exponent bias đơn lẻ nào cho độ chính xác đủ tốt, thậm chí bias = 7 cho perplexity 12.59”, so với baseline 10.19, tức **tệ hơn baseline** $12.59 - 10.19 = 2.4$ **điểm perplexity** (tự tính). Đây là một phát hiện quan trọng về giới hạn: residual connection có phân phối giá trị khó khớp với bất kỳ bias cố định chung nào.

Giải pháp mà paper đưa ra ngay sau đó: nếu **calibrate riêng từng tensor** (per-channel scaling cho weight, per-tensor scaling cho activation, giống quy ước lượng tử hoá int8 thường dùng, trích dẫn [23] = Integer quantization for deep learning inference), thì đạt perplexity 10.29 (chỉ GEMM) và 10.44 (GEMM + residual), rất gần baseline 10.19. Đây chính là bằng chứng thực nghiệm trực tiếp ủng hộ lựa chọn thiết kế ở §3.2: giữ bias cố định kiểu IEEE và đẩy việc scaling sang software linh hoạt, thay vì cố gắng làm ”programmable bias” trong phần cứng.

Về mức độ ”ablation” theo nghĩa cắt bớt thành phần: paper **không** có một bảng ablation kiểu ”bật/tắt từng thành phần của thuật toán” như nhiều paper deep learning khác (vì đây là một đề xuất định dạng số, không phải một pipeline nhiều bước); phân tích gần nhất với ablation là việc so sánh ”chỉ GEMM” vs ”GEMM + residual” vừa nêu, và việc so sánh E4M3 vs int8 trong Table 5 (coi int8 như một ”ablation” của thiết kế dấu phẩy động, thay bằng cố định điểm).

6. Giới hạn

Các giới hạn mà **bài báo nêu** (không phải suy luận của người ghi chú):

1. MobileNet v2 chưa khớp hoàn toàn baseline, còn một phần độ chính xác cần ”khôi phục”, paper nói đang tiếp tục nghiên cứu.
2. Kết quả GPT 175B dựa trên so sánh với baseline bfloat16 **mới hoàn thành 75%** quá trình huấn luyện tại thời điểm viết paper, nên con số so sánh ở Table 4 cho mô hình lớn nhất chưa phải so sánh đầy đủ 100% các bước huấn luyện.
3. Residual connection không thể quantize tốt bằng exponent bias cố định chung, cần calibration riêng từng tensor (per-tensor/per-channel scaling factor) (§4.3, đã phân tích ở mục 5).
4. Phạm vi tensor được quantize trong thí nghiệm huấn luyện chỉ là ”activation, weight, và activation gradient tensor là input cho GEMM”; **tensor output để ở độ chính xác cao hơn** vì chúng thường được tiêu thụ bởi phép toán non-GEMM (nonlinearity, normalization) và ”việc chuyển thêm nhiều tensor sang FP8 là đề tài nghiên cứu tương lai” - paper tự nói rõ phạm vi áp dụng FP8 trong bài **chưa phải toàn bộ** pipeline huấn luyện.
5. Thí nghiệm dùng **FP8 mô phỏng**: số học thực tế vẫn chạy ở FP16/bfloat16, chỉ giá trị tensor bị ”cắt” về lưới giá trị FP8; paper không báo cáo tốc độ (throughput/latency) thực đo trên phần cứng FP8 thật, vì bài báo là một **đề xuất định dạng** đi trước phần cứng hỗ trợ đầy đủ, không phải một báo cáo benchmark hiệu năng. Đây là điểm quan trọng người đọc dễ hiểu lầm: paper không chứng minh tốc độ thực tế tăng bao nhiêu lần, chỉ chứng minh **độ chính xác được giữ nguyên** khi ép tensor về lưới giá trị FP8.
6. Rounding mode (round-to-nearest-even, stochastic rounding, v.v.) được paper nói ”là lựa chọn của việc hiện thực, phần mềm và có thể phần cứng, để tối đa linh hoạt” - nghĩa là **không được chuẩn hoá** trong đề xuất này, có thể dẫn tới kết quả khác nhau giữa các hiện thực khác nhau của cùng một chuẩn FP8.
7. Paper không thảo luận chi tiết heuristic chọn scaling factor (”vượt quá phạm vi của bài báo này”, dòng 94-95), chỉ nói ý tưởng chung là chọn sao cho giá trị lớn nhất trong tensor gần với giá trị lớn nhất biểu diễn được của định dạng.

7. Liên hệ với các bài khác

- **math 15 (Online normalizer calculation for softmax, 1805.02867)**: Cả hai bài đều thuộc nhóm ”làm toán số học hiệu quả và ổn định trên phần cứng tăng tốc deep learning”: math 15 quan tâm tới **thứ tự tính toán** (giảm số lần truy cập bộ nhớ, giữ ổn định số học khi tính softmax theo từng khối), còn math 16 quan tâm tới **cách biểu diễn số** (bao nhiêu bit, định dạng nào); hai hướng tối ưu này độc lập nhưng bổ sung nhau khi thiết kế kernel GPU hiệu năng cao như FlashAttention.
- **opt 42 (Mixed Precision Training, arXiv 1710.03740)**: bài báo FP8 trích dẫn trực tiếp công trình này [14] làm nguồn cho kỹ thuật ”loss scaling” mà FP8 kế thừa (dòng 92-93, 404-406 trong reference list), và dòng 68 nói rõ FP8 ”match FP16 or bfloat16 training results” - tức FP8 là bước tiếp theo của chính kỹ thuật mixed-precision mà opt 42 khởi xướng cho FP16.
- **llm content/51_2022_llm_int8.md, llm content/52_2023_smoothquant.md, llm content/53_2023_gptq.md, llm content/54_2024_awq.md**: đây là các kỹ thuật lượng tử hoá **int8/int4** (cố định điểm) cho LLM đã huấn luyện. So sánh trực tiếp với kết quả Table 5 của bài này: FP8 PTQ giữ độ chính xác tốt hơn int8 PTQ ”ngây thơ” (naive), nhưng các bài LLM.int8/SmoothQuant/GPTQ/AWQ giải quyết đúng vấn đề ”int8 PTQ mất độ chính xác” bằng các kỹ thuật thông minh hơn (ví dụ SmoothQuant di chuyển độ khó lượng tử hoá từ activation sang weight bằng một hệ số scale theo

channel, GPTQ dùng xấp xỉ Hessian để tối ưu làm tròn). Nói cách khác: nếu phần cứng không hỗ trợ FP8, các kỹ thuật int8 ”thông minh” này là con đường thay thế để đạt độ chính xác tương đương.

8. Gợi ý áp dụng

Hai định dạng FP8 đánh đổi dải giá trị và độ chính xác. Việc chọn định dạng và hệ số scaling cần xét phân phối giá trị của từng phép tính, hiện tượng tràn hoặc làm tròn, cùng hỗ trợ thực tế của phần cứng và thư viện.

9. Thí nghiệm gợi ý

1. **Mô phỏng FP8:** làm tròn tensor float32 về các giá trị biểu diễn được của E4M3 và E5M2 sau khi scale; khử scale và so sánh sai số với tensor ban đầu.
2. **PTQ int8 ”ngây thơ” vs E4M3-mô phỏng trên một bài toán nhỏ**: lấy một mô hình phân loại ảnh nhỏ (ví dụ CNN 3-4 lớp trên MNIST/CIFAR-10, chạy được trên GPU trong vài phút), huấn luyện ở FP32, rồi (a) lượng tử hoá int8 theo kiểu max-calibration đơn giản, (b) lượng tử hoá ”giả-E4M3” theo công thức mục 3.4, so độ giảm accuracy của (a) và (b). Kết quả kỳ vọng: (b) giảm accuracy ít hơn (a), lặp lại định tính phát hiện ở Table 5 của paper (FP8 PTQ tốt hơn int8 PTQ) ở quy mô nhỏ hơn hẳn. Diễn giải: đây là một kiểm tra ”sanity check” rẻ, không cần phần cứng FP8, để tin tưởng hơn vào kết luận định tính trước khi đầu tư công sức triển khai thật.
3. **Đo ảnh hưởng của không dùng scaling factor**: lặp lại thí nghiệm 1 nhưng cố tình đặt scaling factor = 1 (không chuẩn hoá trước khi ép về FP8), so với có scaling factor đúng. Kết quả kỳ vọng: sai số tăng mạnh (tương tự paper báo cáo perplexity tăng lên 11.0 khi bỏ scaling factor, dòng 294-295), đặc biệt nếu giá trị tensor nhỏ hơn hẳn giá trị biểu diễn tối thiểu hoặc lớn hơn hẳn giá trị biểu diễn tối đa của định dạng. Diễn giải: minh hoạ trực quan tại sao scaling factor là bước bắt buộc, không phải chi tiết tối ưu tuỳ chọn.

10. Tóm tắt công thức / khái niệm

- Giá trị số chuẩn hoá (normal): $(-1)^S \times 1.M_2 \times 2^{(E_2-\text{bias})}$.
- Giá trị số subnormal: $(-1)^S \times 0.M_2 \times 2^{(1-\text{bias})}$.
- E4M3: 1 bit dấu, 4 bit exponent (bias = 7), 3 bit mantissa; không có vô cực; 1 mẫu NaN; max normal = 448; min normal = 2^{-6}; max subnormal = 0.875×2^{-6}; min subnormal = 2^{-9}.
- E5M2: 1 bit dấu, 5 bit exponent (bias = 15), 2 bit mantissa; tuân IEEE đầy đủ (có vô cực, nhiều mẫu NaN); max normal = 57 344; min normal = 2^{-14}; max subnormal = 0.75×2^{-14}; min subnormal = 2^{-16}.
- Khuyến nghị sử dụng: E4M3 cho weight/activation (cần độ chính xác), E5M2 cho gradient (cần dynamic range).
- Scaling factor s: trước khi ép tensor x (độ chính xác cao) về FP8, tính $x' = x \times s$ sao cho max $|x'|$ gần giá trị lớn nhất biểu diễn được của định dạng; sau phép toán, khử scale bằng $x'' = y \times s^{-1}$.
- Bão hoà (saturation): giá trị $|x'|$ vượt ngưỡng tối đa bị cắt về giá trị lớn nhất biểu diễn được, thay cho việc bỏ qua cập nhật như FP16 AMP.
- Ví dụ tự tính (mục 3.4): biểu diễn $x = 300$ bằng E4M3 cho kết quả gần nhất 288 (sai số tuyệt đối 12, sai số tương đối $\approx 4\%$); bằng E5M2 cho kết quả gần nhất 320 (sai số tuyệt đối 20, sai số tương đối $\approx 6.7\%$) (tự tính).
- Binade: khoảng giá trị giữa hai luỹ thừa liên tiếp của 2; E4M3 (đã mở rộng) có 18 binade, E5M2 có 32 binade.

11. Kết luận ngắn

Bài báo đề xuất chuẩn FP8 gồm hai encoding E4M3 (ưu tiên độ chính xác, dùng cho weight/activation) và E5M2 (ưu tiên dynamic range, dùng cho gradient), thiết kế bám sát tối đa quy ước IEEE 754 và chỉ lệch khi lợi ích DL rõ ràng (bỏ vô cực và giảm mã NaN ở E4M3 để mở rộng dynamic range). Thực nghiệm trên nhiều kiến trúc (CNN, RNN, Transformer) và kích cỡ (đến GPT 175B) cho thấy huấn luyện FP8 (mô phỏng) khớp kết quả huấn luyện FP16/bfloat16 mà không cần đổi hyperparameter, và suy luận FP8 PTQ giữ độ chính xác tốt hơn int8 PTQ, đặc biệt khi dùng per-tensor/per-channel scaling factor thay cho exponent bias cố định chung, nhất là với tensor khó như residual connection.

Nguồn gốc: https://arxiv.org/abs/2209.05433

Bài 17

Spectral Normalization for Generative Adversarial Networks (2018)

Tác giả: Takeru Miyato, Toshiki Kataoka, Masanori Koyama, Yuichi Yoshida
Venue / arXiv: arXiv:1802.05957 (v1, 16/02/2018); Published as a conference paper at ICLR 2018 (paper tự ghi rõ ở đầu mỗi trang)
Nguồn: https://arxiv.org/abs/1802.05957
Chủ đề: Đề xuất ”spectral normalization”, một kỹ thuật chuẩn hoá trọng số (weight normalization) ép ràng buộc Lipschitz lên discriminator của GAN bằng cách chia mỗi ma trận trọng số cho giá trị kỳ dị (singular value) lớn nhất của nó, nhằm ổn định huấn luyện GAN.
Vị trí trong lộ trình: nên đọc sau math 13 (Wasserstein GAN, 1701.07875, cùng mã ’math’, số 13) vì cả hai cùng giải quyết bài toán ”ép ràng buộc Lipschitz cho discriminator/critic” nhưng bằng hai cách khác nhau (WGAN dùng weight clipping, bài này dùng chuẩn hoá giá trị kỳ dị lớn nhất); có thể đọc cùng lúc hoặc trước math 12 (Computational Optimal Transport) để hiểu rõ hơn vì sao ràng buộc Lipschitz quan trọng với khoảng cách Wasserstein.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (bản arXiv); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc.

1. Bối cảnh

GAN (Generative Adversarial Network, Goodfellow và cộng sự, 2014) huấn luyện hai mạng: generator G tạo mẫu từ phân phối mô hình, còn discriminator D phân biệt các mẫu đó với dữ liệu thật. Hai mạng được cập nhật xen kẽ theo bài toán min-max $\min_G \max_D V(G,D)$, với mục tiêu gốc $V(G,D) = \mathbb{E}_{x\sim q_{data}}[\log D(x)] + \mathbb{E}_{x\prime\sim p_G}[\log(1 - D(x\prime))]$.

Vấn đề kéo dài của GAN là **sự bất ổn (instability) của huấn luyện**. Paper nêu rõ nguyên nhân cụ thể: trong không gian nhiều chiều, việc ước lượng tỷ số mật độ (density ratio) bởi discriminator thường không chính xác và bất ổn trong quá trình huấn luyện, khiến generator không học được cấu trúc đa phương thức (multimodal) của phân phối mục tiêu. Tệ hơn, khi phân phối mô hình và phân phối dữ liệu thật có **support rời nhau** (disjoint support), tồn tại một discriminator có thể phân biệt hoàn hảo hai phân phối (trích dẫn Arjovsky & Bottou, 2017). Khi discriminator đó xuất hiện, quá trình huấn luyện generator ”hoàn toàn ngừng lại, vì đạo hàm của discriminator đó theo input bằng 0”. Đây chính là động lực để **giới hạn tập hợp các discriminator** được phép chọn.

Khái niệm cần giải thích: **hàm Lipschitz liên tục (Lipschitz continuous)**, cụ thể một hàm f là **K-Lipschitz** nếu $\|f(x) - f(x')\|/\|x - x'\| \le K$ với mọi x, x' (dùng chuẩn ℓ_2), tức là đầu ra của hàm không thể thay đổi ”quá nhanh” so với đầu vào. Nhiều công trình trước đó (Uehara et al. 2016; Qi 2017; Gulrajani et al. 2017) đã chỉ ra tầm quan trọng của ràng buộc Lipschitz liên tục để đảm bảo các đại lượng thống kê bị chặn (bounded). Cụ thể, paper dẫn ra discriminator tối ưu của công thức GAN gốc: $D^*_G(x) = \text{sigmoid}(f^*(x))$ với $f^*(x) = \log q_{data}(x) - \log p_G(x)$ (công thức 3), và đạo hàm của nó $\nabla_x f^*(x) = \frac{1}{q_{data}(x)}\nabla_x q_{data}(x) - \frac{1}{p_G(x)}\nabla_x p_G(x)$ (công thức 4) ”có thể không bị chặn hoặc thậm chí không tính được”. Đây là lý do cần một điều kiện chính quy (regularity condition) cho đạo hàm của f.

Các công trình thành công trước đó đã kiểm soát hằng số Lipschitz bằng cách thêm hạng tử điều chuẩn (regularization) được định nghĩa trên các mẫu input cụ thể: weight clipping (Arjovsky et al. 2017, chính là WGAN = math 13 trong lộ trình này) và gradient penalty (Gulrajani et al. 2017, WGAN-GP). Paper chỉ ra

hạn chế chung của các phương pháp "dựa trên mẫu input" (input-based regularization): "chúng không thể áp đặt điều chuẩn trên không gian **bên ngoài** support của phân phối generator và dữ liệu mà không cần những biện pháp mang tính heuristic". Đây là động lực để đề xuất **spectral normalization**: một phương pháp điều chuẩn trực tiếp trên **ma trận trọng số**, không phụ thuộc vào mẫu dữ liệu cụ thể.

2. Ý tưởng chính

Spectral normalization chia mỗi ma trận trọng số W trong discriminator cho giá trị kỳ dị lớn nhất của nó, đưa chuẩn phổ (spectral norm) của ma trận về 1:

$$\bar{W}_{SN}(W) := W/\sigma(W)$$

trong đó $\sigma(W)$ là chuẩn kỳ dị, bằng chính giá trị kỳ dị lớn nhất của W. Lý do việc này đảm bảo tính 1-Lipschitz của toàn bộ discriminator: paper dùng bất đẳng thức cho hợp hàm $\|g_1 \circ g_2\|_{Lip} \leq \|g_1\|_{Lip} \cdot \|g_2\|_{Lip}$, và quan sát rằng với lớp tuyến tính $g(h) = Wh$, chuẩn Lipschitz $\|g\|_{Lip} = \sigma(W)$ (chuẩn kỳ dị chính là hằng số Lipschitz của một phép biến đổi tuyến tính). Nếu hàm kích hoạt (activation) giữa các lớp có $\|a_l\|_{Lip} = 1$ (đúng với ReLU, leaky ReLU), thì toàn mạng f (gồm $L+1$ lớp tuyến tính xen activation) thoả: $\|f\|_{Lip} \leq \prod_{l=1}^{L+1} \sigma(W^l)$ (công thức 7). Nếu mỗi W^l đã được chuẩn hoá để $\sigma(\bar{W}_{SN}(W^l)) = 1$, tích trên $= 1$, nên $\|f\|_{Lip} \leq 1$: toàn bộ discriminator được đảm bảo là 1-Lipschitz, **không cần biết gì về phân phối dữ liệu**, khác hẳn các phương pháp dựa trên mẫu input.

Ý tưởng thứ hai, mang tính thực tiễn: tính đúng giá trị kỳ dị lớn nhất bằng phân rã giá trị kỳ dị (SVD) đầy đủ ở mỗi bước huấn luyện là quá tốn kém tính toán; paper dùng **phương pháp lặp lũy thừa (power iteration method)**, một thuật toán xấp xỉ rẻ, chỉ cần vài phép nhân ma trận-vector mỗi bước, để ước lượng $\sigma(W)$ gần như "miễn phí" so với chi phí forward/backward propagation của toàn mạng.

Ý tưởng thứ ba (phân tích gradient, §2.3): chuẩn hoá phổ không chỉ là một phép biến đổi hình thức mà có một hiệu ứng điều chuẩn tự nhiên (implicit regularization): gradient của hàm mất mát theo W (sau chuẩn hoá) có thêm một hạng tử trừ đi thành phần dọc theo hướng kỳ dị thứ nhất $u_1 v_1^\top$, với hệ số thích nghi λ. Điều này **ngăn không gian cột của W tập trung (concentrate) vào một hướng duy nhất** trong quá trình huấn luyện, khác hẳn một điều chuẩn "tĩnh" như L2 regularization.

3. Phương pháp chi tiết

3.1 Thiết lập discriminator và chuẩn kỳ dị (§2, §2.1)

Paper xét discriminator dạng mạng neural nhiều lớp:

$$f(x, \theta) = W^{L+1} a_L(W^L(a_{L-1}(W^{L-1}(\dots a_1(W^1 x)\dots))))$$

(công thức 1), với $\theta = \{W^1, \dots, W^L, W^{L+1}\}$, $W^l \in \mathbb{R}^{d_l \times d_{l-1}}$, bỏ qua bias term để đơn giản. Đầu ra cuối $D(x, \theta) = A(f(x, \theta))$ với A là hàm kích hoạt tương ứng loại divergence được chọn (ví dụ sigmoid cho GAN gốc).

Định nghĩa chuẩn kỳ dị (spectral norm) của ma trận A:

$$\sigma(A) := \max_{h: h \neq 0} \frac{\|Ah\|_2}{\|h\|_2} = \max_{\|h\|_2 \leq 1} \|Ah\|_2$$

(công thức 6). Khái niệm giải thích cho người mới: $\sigma(A)$ đo "mức giãn (stretch) tối đa" mà phép biến đổi tuyến tính A có thể gây ra lên độ dài của một vector đơn vị bất kỳ; về mặt đại số, $\sigma(A)$ **chính là giá trị kỳ dị**

lớn nhất của A (giá trị kỳ dị, singular value, là căn bậc hai của trị riêng của $A^\top A$). Bằng định nghĩa, chuẩn Lipschitz của một lớp tuyến tính $\|g\|_{Lip} = \sup_h \sigma(\nabla g(h)) = \sigma(W)$.

3.2 Thuật toán lặp lũy thừa (power iteration) để ước lượng $\sigma(W)$ (Appendix A, Algorithm 1)

Paper trình bày thuật toán cụ thể (Algorithm 1 "SGD with spectral normalization"):

- Khởi tạo $\tilde{u}^l \in \mathbb{R}^{d_l}$ cho mỗi lớp $l = 1, \ldots, L$ bằng một vector ngẫu nhiên (lấy mẫu từ phân phối đẳng hướng, isotropic).
- Với mỗi lượt cập nhật và mỗi lớp l:
  1. Áp dụng lặp lũy thừa cho ma trận trọng số **chưa chuẩn hoá** W^l:

$$\tilde{v}_l \leftarrow (W^l)^\top \tilde{u}_l / \|(W^l)^\top \tilde{u}_l\|_2 \quad (20)$$

$$\tilde{u}_l \leftarrow W^l \tilde{v}_l / \|W^l \tilde{v}_l\|_2 \quad (21)$$

  2. Tính $\bar{W}_{SN}$ với chuẩn kỳ dị ước lượng:

$$\bar{W}^l_{SN}(W^l) = W^l / \sigma(W^l), \quad \text{trong đó } \sigma(W^l) = \tilde{u}_l^\top W^l \tilde{v}_l \quad (22)$$

  3. Cập nhật W^l bằng SGD trên mini-batch D_M với learning rate α:

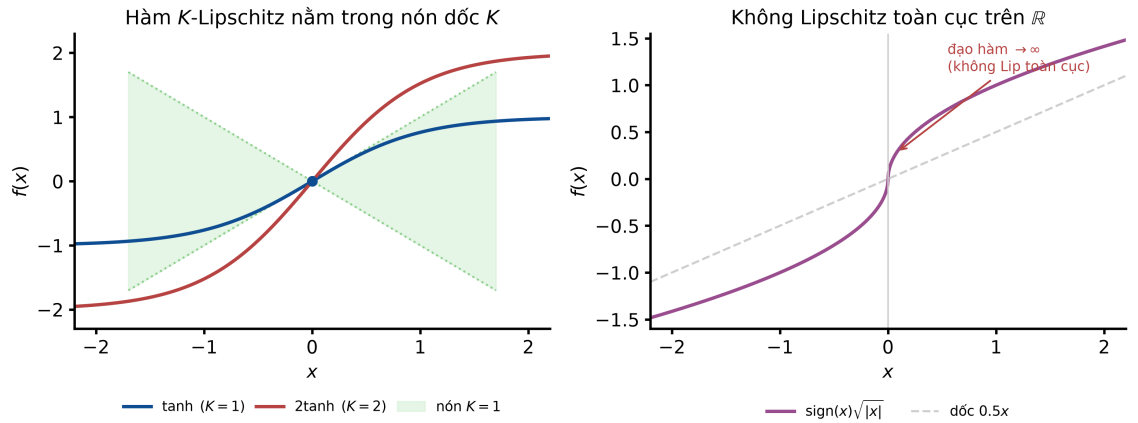
$$W^l \leftarrow W^l - \alpha \nabla_{W^l} \ell(\bar{W}^l_{SN}(W^l), D_M) \quad (23)$$

Điểm mấu chốt về hiệu quả tính toán: paper chỉ chạy **một bước** lặp lũy thừa mỗi lượt cập nhật (không lặp tới khi hội tụ hoàn toàn), và nhận xét "trong thực tế, chúng ta có thể an toàn giả định $\tilde{u}$ sinh từ phân phối đều trên mặt cầu không trực giao với vector kỳ dị thứ nhất, vì điều này chỉ xảy ra với xác suất 0" (chú thích 7, dòng 1105-1107) - vì $\tilde{u}$ được cập nhật liên tục qua các bước huấn luyện (không reset mỗi lần), nó "theo dõi" (track) hướng kỳ dị thứ nhất một cách liên tục, nên chỉ cần 1 bước lặp mỗi lượt vẫn hội tụ tốt theo thời gian huấn luyện. Bài báo nhấn mạnh "thủ tục này rất rẻ về tính toán, thậm chí so với tính forward/backward propagation của mạng neural".

Với trọng số của **lớp convolution** $W \in \mathbb{R}^{d_{out} \times d_{in} \times h \times w}$, paper xử lý bằng cách "coi phép toán như một ma trận 2 chiều kích thước $d_{out} \times (d_{in} hw)$", kèm chú thích rằng vì convolution được thực hiện rời rạc, chuẩn kỳ dị sẽ phụ thuộc vào kích thước stride và padding, nhưng "câu trả lời chỉ khác nhau bởi một hằng số K xác định trước" (dòng 330-331, chú thích 3) - tức đây là một **xấp xỉ có chủ đích**, không phải chuẩn kỳ dị chính xác của phép toán convolution thật.

3.3 Ví dụ số tính tay: lặp lũy thừa trên ma trận 2x2 cụ thể (tự tính)

Để hiểu rõ cơ chế lặp lũy thừa, ta thử trên một ma trận 2×2 cụ thể: $W = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}$. Giá trị kỳ dị của ma trận đường chéo này chính là các giá trị tuyệt đối trên đường chéo: $\sigma_1 = 2$, $\sigma_2 = 1$ (đây là trường hợp đặc biệt dễ kiểm chứng trực tiếp vì $W^\top W = \begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix}$ đã chéo hoá, trị riêng 4 và 1, căn bậc hai cho $\sigma = 2$ và $\sigma = 1$; tự tính).



Hàm K-Lipschitz bị chặn trong nón dốc K; ví dụ sign(x)√|x| không Lipschitz toàn cục vì đạo hàm tại 0 vô hạn. (minh hoạ, tự tính)

Khởi tạo $\tilde{u}_0 = (1,1)^\top/\sqrt{2}$ (vector đơn vị, không trực giao với vector kỳ dị thứ nhất $(1,0)^\top$, đúng như giả định của paper).

Bước 1 (công thức 20): $\tilde{v}_1 \leftarrow W^\top \tilde{u}_0 = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix} = \begin{pmatrix} 2/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix} = \begin{pmatrix} \sqrt{2} \\ 1/\sqrt{2} \end{pmatrix}$ (vì W đối xứng nên $W^\top = W$). Chuẩn hoá: $\|\tilde{v}_1\|_2 = \sqrt{2 + 0.5} = \sqrt{2.5} \approx 1.5811$. Vậy $\tilde{v}_1 \approx (0.8944, 0.3162)$ (tự tính: $\sqrt{2}/1.5811 \approx 0.8944$; $(1/\sqrt{2})/1.5811 \approx 0.3162$).

Bước 2 (công thức 21): $\tilde{u}_1 \leftarrow W\tilde{v}_1 = (2 \times 0.8944,\ 1 \times 0.3162) = (1.7888, 0.3162)$. Chuẩn hoá: $\|\cdot\|_2 = \sqrt{1.7888^2 + 0.3162^2} = \sqrt{3.1998 + 0.1} \approx \sqrt{3.2998} \approx 1.8165$. $\tilde{u}_1 \approx (0.9850, 0.1741)$ (tự tính).

Ước lượng σ sau 1 bước (công thức 22): $\sigma \approx \tilde{u}_1^\top W \tilde{v}_1 = \tilde{u}_1 \cdot (1.7888, 0.3162) = 0.9850 \times 1.7888 + 0.1741 \times 0.3162 \approx 1.7620 + 0.0551 \approx 1.8171$ (tự tính).

Sau chỉ 1 bước lặp, ước lượng $\sigma \approx 1.817$, đã gần với giá trị đúng $\sigma_1 = 2$ (sai số tương đối $\approx (2 - 1.817)/2 = 9.15\%$, tự tính). Nếu lặp thêm một bước nữa (bước 2), $\tilde{u}, \tilde{v}$ sẽ hội tụ nhanh hơn nữa về $(1,0)^\top$ vì tỷ lệ $\sigma_1/\sigma_2 = 2$ (hội tụ theo tốc độ hình học $(\sigma_2/\sigma_1)^k = (1/2)^k$ sau k bước, kiến thức chung về power iteration, ngoài paper). Minh hoạ này cho thấy trực quan tại sao chỉ cần 1 bước mỗi lượt cập nhật vẫn đủ tốt trong thực hành khi $\tilde{u}$ được giữ liên tục qua nhiều lượt cập nhật (không reset), đúng như lý giải của paper ở chú thích 7.

3.4 Phân tích gradient của trọng số đã chuẩn hoá (§2.3)

Paper tính gradient của $\bar{W}_{SN}(W)$ theo từng phần tử W_{ij}:

$$\frac{\partial \bar{W}_{SN}(W)}{\partial W_{ij}} = \frac{1}{\sigma(W)} \left(E_{ij} - [u_1 v_1^\top]_{ij} \bar{W}_{SN} \right) \quad (10)$$

với E_{ij} là ma trận có phần tử (i,j) bằng 1, còn lại bằng 0; u_1, v_1 là vector kỳ dị trái/phải thứ nhất của W. Từ đó, đạo hàm của mục tiêu $V(G,D)$ theo W của discriminator (tính trên mini-batch):

$$\frac{\partial V(G,D)}{\partial W} = \frac{1}{\sigma(W)} \left(\hat{\mathbb{E}}[\delta h^\top] - \lambda u_1 v_1^\top \right) \quad (12)$$

trong đó $\delta := \partial V(G,D)/\partial(\bar{W}_{SN} h)$, $\lambda := \hat{\mathbb{E}}[\delta^\top \bar{W}_{SN} h]$, $\hat{\mathbb{E}}[\cdot]$ là kỳ vọng thực nghiệm trên mini-batch. Paper giải thích ý nghĩa: "hạng tử đầu $\hat{\mathbb{E}}[\delta h^\top]$ giống hệt đạo hàm của trọng số khi không chuẩn hoá. Hạng tử thứ hai có thể coi là một hạng tử điều chuẩn, phạt (penalize) thành phần kỳ dị thứ nhất với hệ số điều chuẩn thích nghi λ. λ dương khi δ và $\bar{W}_{SN} h$ cùng hướng, và điều này ngăn không gian cột của W tập trung vào một

hướng cụ thể trong quá trình huấn luyện". Nói cách khác: spectral normalization **tự động** tạo ra một áp lực chống lại việc ma trận trọng số trở nên "gần hạng-1" (rank-deficient), khác hẳn với một điều chuẩn độc lập dữ liệu như L2.

3.5 So sánh với các kỹ thuật khác (§3)

Paper so sánh kỹ với 3 kỹ thuật thay thế:

1. **Weight normalization (Salimans & Kingma 2016)**: chuẩn hoá ℓ_2-norm của mỗi vector hàng trong W, tương đương với yêu cầu $\sigma_1(\bar{W}_{WN})^2 + \cdots + \sigma_T(\bar{W}_{WN})^2 = d_o$ (công thức 13, gần giống chuẩn Frobenius). Paper chỉ ra: để tối đa hoá $\|\bar{W}_{WN}h\|_2$ cho một vector đơn vị cố định h, cách tốt nhất là đặt $\sigma_1(\bar{W}_{WN}) = \sqrt{d_o}$ và tất cả giá trị kỳ dị còn lại bằng 0, nghĩa là $\bar{W}_{WN}$ **có hạng 1 (rank one)**. Weight clipping (Arjovsky et al. 2017, WGAN = math 13) cũng "gặp cùng một cạm bẫy" (dòng 275-276, "suffers from same pitfall"). Hệ quả: các phương pháp này có **xung đột lợi ích** giữa muốn giữ norm lớn (để discriminator nhạy hơn) và việc giữ norm lớn lại làm giảm hạng ma trận (giảm số đặc trưng hữu dụng), khiến discriminator chỉ phân biệt hai phân phối dựa trên "một vài đặc trưng chọn lọc".

2. **Orthonormal regularization (Brock et al. 2016)**: thêm hạng tử điều chuẩn $\|W^\top W - I\|_F^2$ (công thức 14) vào mục tiêu đối nghịch. Khác biệt cốt lõi mà bài báo nhấn mạnh: orthonormal regularization "phá hủy thông tin về phổ (spectrum) bằng cách đặt TẤT CẢ giá trị kỳ dị bằng 1", còn spectral normalization "chỉ scale (tỷ lệ) phổ sao cho giá trị lớn nhất bằng 1" - giữ nguyên tỷ lệ tương đối giữa các giá trị kỳ dị, không ép tất cả về 1.

3. **Gradient penalty / WGAN-GP (Gulrajani et al. 2017)**: thêm hạng tử điều chuẩn khuyến khích $\|\nabla_{\hat{x}} f\|_2 = 1$ tại các điểm nội suy $\hat{x} = \epsilon\tilde{x} + (1-\epsilon)x$ giữa mẫu sinh và mẫu thật. Paper chỉ ra điểm yếu: phương pháp này "phụ thuộc nặng vào support của phân phối generator hiện tại", mà support đó thay đổi dần trong quá trình huấn luyện, "có thể làm mất ổn định hiệu ứng của điều chuẩn"; paper quan sát thực nghiệm "learning rate cao có thể làm mất ổn định hiệu năng của WGAN-GP". Ngược lại, spectral normalization "điều chuẩn hàm trên không gian toán tử (operator space), hiệu ứng điều chuẩn ổn định hơn với lựa chọn batch", và không dễ mất ổn định với learning rate mạnh. Về chi phí tính toán: "WGAN-GP cần chi phí tính toán nhiều hơn spectral normalization với 1 bước lặp lũy thừa, vì tính $\|\nabla_{\hat{x}} f\|_2$ cần một vòng forward-backward propagation đầy đủ".

4. Thí nghiệm và kết quả chính

Paper thực nghiệm trên CIFAR-10, STL-10, và ILSVRC2012 (ImageNet). Dùng Adam optimizer, thử 6 cấu hình hyperparameter (Table 1): A ($\alpha = 0.0001, \beta_1 = 0.5, \beta_2 = 0.9, n_{dis} = 5$, theo Gulrajani et al. 2017), B ($\alpha = 0.0001, \beta_1 = 0.5, \beta_2 = 0.999, n_{dis} = 1$, theo Warde-Farley & Bengio 2017), C ($\alpha = 0.0002, \beta_1 = 0.5, \beta_2 = 0.999, n_{dis} = 1$, theo Radford et al. 2016), và D, E, F là các cấu hình learning rate "hung hăng" hơn ($\alpha = 0.001$) để kiểm tra độ bền (robustness). n_{dis} là số lượt cập nhật discriminator cho mỗi lượt cập nhật generator.

Kết quả Inception Score và FID (Table 2) trên CIFAR-10 (kiến trúc Standard CNN): Real data = 11.24±.12 (Inception score). Weight clipping = 6.41±.11 (IS), FID 42.6. GAN-GP = 6.93±.08, FID 37.7. WGAN-GP = 6.68±.06, FID 40.2. Batch Norm = 6.27±.10, FID 56.3. Layer Norm = 7.19±.12, FID 33.9. Weight Norm = 6.84±.07, FID 34.7. Orthonormal = 7.40±.12, FID 29.0. **(ours) SN-GANs = 7.42±.08, FID 29.3**. SN-GANs với hinge loss (công thức 17) = 7.58±.12, FID 25.5 (tốt nhất trong nhóm Standard CNN). Trên kiến trúc **ResNet**: Orthonormal với hinge loss = 7.92±.04 (IS), FID 23.8±.58; **SN-GANs với hinge loss = 8.22±.05 (IS), FID 21.7±.21** (tốt nhất toàn bảng cho CIFAR-10 ResNet).

Trên **STL-10** (Standard CNN): Real data = 26.08±.26. Weight clipping = 7.57±.10. WGAN-GP = 8.42±.13. Orthonormal = 8.56±.07. SN-GANs = 8.28±.09 (thấp hơn Orthonormal ở cùng số lượt cập nhật). Nhưng với 2x số lượt cập nhật: Orthonormal (2x) = 8.67±.08, SN-GANs (2x) = 8.69±.09 (SN-GANs vượt nhẹ). Trên

kiến trúc ResNet: Orthonormal với hinge loss = 8.72±.06, **SN-GANs với hinge loss = 9.10±.04** (tốt nhất); FID: Orthonormal = 42.4±.99, SN-GANs = 40.1±.50 (tốt nhất, thấp hơn là tốt hơn).

Bài báo lưu ý một trường hợp SN-GANs **không** vượt trội: ”với cùng số lượt huấn luyện, SN-GANs thua Orthonormal regularization trên STL-10” (dòng 490-491, Standard CNN, không phải ResNet) - đây là điểm paper thẳng thắn thừa nhận không phải lúc nào phương pháp đề xuất cũng thắng tuyệt đối.

Độ bền với hyperparameter (Figure 1, 2): ”spectral normalization tương đối bền (robust) với learning rate và tham số momentum hung hăng. WGAN-GP thất bại trong việc huấn luyện GAN tốt ở learning rate và momentum cao trên cả CIFAR-10 và STL-10. Orthonormal regularization hoạt động kém ở cấu hình E trên STL-10, nhưng hoạt động tốt hơn một chút so với phương pháp của chúng tôi ở cấu hình tối ưu”.

Phân tích giá trị kỳ dị của trọng số đã huấn luyện (Figure 3, §4.1.1): ”giá trị kỳ dị của các lớp 1 đến 5 huấn luyện bằng weight clipping và weight normalization tập trung vào một vài thành phần... Ngược lại, giá trị kỳ dị của ma trận trọng số huấn luyện bằng spectral normalization được phân bố rộng hơn hẳn”. Paper giải thích hệ quả: ”khi mục tiêu là phân biệt một cặp phân phối xác suất trên manifold phi tuyến chiều thấp nhúng trong không gian chiều cao, sự thiếu hạng (rank deficiency) ở các lớp thấp có thể đặc biệt tai hại”, vì đầu ra của các lớp thấp ”chỉ đi qua một vài phép biến đổi rectified linear, nên có xu hướng nằm trên không gian tuyến tính ở phần lớn vùng”.

Chi phí thời gian huấn luyện: ”trên CIFAR-10, SN-GANs chậm hơn một chút so với weight normalization (khoảng 110-120% thời gian tính toán), nhưng nhanh hơn đáng kể so với WGAN-GP”, vì WGAN-GP cần tính gradient-của-gradient $\|\nabla_x D\|_2$.

Kết quả trên ImageNet (§4.2): ”GAN không điều chuẩn và GAN với layer normalization sụp đổ (collapsed) ngay từ đầu huấn luyện và không tạo được ảnh có ý nghĩa. GAN với orthonormal normalization và spectral normalization có thể tạo ảnh. Inception score của orthonormal normalization chững lại (plateau) quanh vòng lặp thứ 20.000, trong khi SN tiếp tục cải thiện sau đó”. Bài báo lưu ý đây là ”nghiên cứu đầu tiên thuộc loại này thành công trong việc tạo ảnh tạm ổn (decent) từ ImageNet với một cặp discriminator-generator duy nhất”. Đo mức độ mode-collapse bằng intra MS-SSIM: ”SN-GANs (intra MS-SSIM = 0.101) chịu mode-collapse ít hơn AC-GANs (intra MS-SSIM $\sim$ 0.25)”.

5. Phân tích / ablation

Mục §4.1.2 là phần ablation/phân tích sâu nhất: so sánh trực tiếp SN-GANs với Orthonormal regularization bằng cách **tăng dần kích thước feature map ở lớp cuối** (lớp conv thứ 7), giữ các thiết lập khác cố định. Kết quả (Figure 4): ”hiệu năng của orthonormal regularization giảm dần khi tăng kích thước feature map ở lớp cuối. SN-GANs của chúng tôi, mặt khác, không hề suy giảm với thay đổi kiến trúc này”. Paper giải thích lý do: orthonormal regularization ”ép discriminator sử dụng TẤT CẢ chiều, bao gồm cả những chiều không cần thiết”, vì nó đặt mọi giá trị kỳ dị về 1 (không được phép có giá trị kỳ dị nhỏ/bằng 0 để ”bỏ qua” một số chiều), nên khi không gian feature lớn hơn mức cần thiết, việc ép dùng hết mọi chiều trở nên có hại. Spectral normalization không gặp vấn đề này vì ”chuẩn kỳ dị độc lập với hạng (rank)”: một ma trận có nhiều giá trị kỳ dị nhỏ/gần 0 vẫn thoả $\sigma(W) = 1$ miễn giá trị **lớn nhất** bằng 1, nên mô hình có ”tự do hơn trong việc chọn số thành phần kỳ dị (đặc trưng) cần dùng”.

Đây là một ablation có giá trị cao vì nó tách bạch rõ **sự khác biệt bản chất toán học** giữa hai phương pháp tưởng chừng tương tự (cả hai đều liên quan tới giá trị kỳ dị của W): orthonormal ép TẤT CẢ giá trị kỳ dị bằng nhau (và bằng 1), còn spectral normalization chỉ ép giá trị kỳ dị **lớn nhất** bằng 1, cho phép các giá trị kỳ dị khác tự do nhỏ hơn (bao gồm gần 0, tương đương ”tắt” một số đặc trưng không cần thiết).

Về mặt ablation hyperparameter (Figure 1-2, Table 1, đã mô tả ở mục 4): paper tự thực hiện một dạng ”stress test” có hệ thống qua 6 cấu hình optimizer khác nhau (từ ”an toàn” A-C đến ”hung hăng” D-F), cho phép so

sánh độ bền (không chỉ hiệu năng tốt nhất) của các phương pháp điều chuẩn khác nhau - đây là một thiết kế ablation chặt chẽ hơn so với nhiều paper GAN cùng thời chỉ báo cáo kết quả tốt nhất.

6. Giới hạn

Các giới hạn **bài báo nêu hoặc tự thừa nhận**:

1. ”Trong công việc tương lai, chúng tôi muốn tiếp tục điều tra vị trí của phương pháp này so với các phương pháp khác trên cơ sở lý thuyết chặt chẽ hơn” (dòng 905-906, phần Conclusion) - bài báo lưu ý phân tích lý thuyết hiện tại **chưa đầy đủ**, đặc biệt về việc so sánh định lượng nghiêm ngặt với các phương pháp khác (ví dụ chưa có một chứng minh hội tụ GAN tổng thể khi dùng spectral normalization).
2. Với cấu hình Standard CNN trên STL-10 (cùng số lượt cập nhật), SN-GANs thua Orthonormal regularization (đã nêu ở mục 4); chỉ khi tăng gấp đôi số lượt cập nhật SN-GANs mới vượt nhẹ.
3. Việc tính chuẩn kỳ dị cho lớp convolution là một **xấp xỉ** (coi phép toán như ma trận 2D $d_{out} \times (d_{in}hw)$), ”câu trả lời chỉ khác nhau bởi một hằng số K xác định trước” - tức không phải chuẩn kỳ dị thực sự chính xác của phép toán convolution (bản chất convolution là một phép biến đổi tuyến tính có cấu trúc đặc biệt, Toeplitz-like, chuẩn kỳ dị chính xác của nó phức tạp hơn một ma trận dày đặc thông thường).
4. Thuật toán chỉ chạy **1 bước** lặp lũy thừa mỗi lượt cập nhật, nên $\sigma(W)$ ước lượng được chỉ là xấp xỉ (không hội tụ hoàn toàn về giá trị kỳ dị lớn nhất thật, xem ví dụ tự tính ở mục 3.3 với sai số $\sim 9\%$ sau 1 bước cho một ma trận cụ thể); paper lý giải điều này ”an toàn” trong thực hành nhờ $\tilde{u}$ được giữ liên tục qua các lượt cập nhật, nhưng đây vẫn là một xấp xỉ, không phải phép tính chính xác tuyệt đối.
5. Kết quả thực nghiệm chỉ trên ảnh (CIFAR-10, STL-10, ImageNet); paper không thử trên các loại dữ liệu khác (văn bản, âm thanh), nên tổng quát hoá sang các loại GAN khác chưa được paper tự kiểm chứng.
6. Paper không khẳng định dứt khoát spectral normalization luôn tốt hơn gradient penalty/WGAN-GP về mặt lý thuyết (chỉ nói ”có thể kết hợp cả hai”, ”can possibly used in combinations”, dòng 904-905), nghĩa là hai phương pháp được paper coi là **bổ sung** nhau hơn là loại trừ lẫn nhau.

7. Liên hệ với các bài khác

- **math 13 (Wasserstein GAN, 1701.07875)**: Cả hai bài cùng giải quyết bài toán ”ép ràng buộc Lipschitz cho discriminator/critic của GAN” nhưng bằng hai kỹ thuật khác nhau: WGAN dùng **weight clipping** (cắt trực tiếp giá trị trọng số về một khoảng $[-c, c]$ cố định) - paper hiện tại chỉ ra weight clipping ”gặp cùng cạm bẫy” với weight normalization là làm ma trận trọng số trở thành gần hạng-1 (rank deficient, dòng 275-276); spectral normalization giải quyết vấn đề này bằng cách chỉ ràng buộc giá trị kỳ dị **lớn nhất**, giữ tự do cho các giá trị kỳ dị khác.
- **math 12 (Computational Optimal Transport)**: WGAN và lý thuyết Kantorovich-Rubinstein duality (chủ đề của math 12) định nghĩa khoảng cách Wasserstein-1 thông qua supremum trên các hàm 1-Lipschitz; spectral normalization của bài này là một kỹ thuật **thực hành cụ thể** để ép ràng buộc 1-Lipschitz đó lên một mạng neural, nên có thể coi là ”công cụ hiện thực hoá” cho chính ràng buộc lý thuyết mà Kantorovich-Rubinstein duality đòi hỏi.
- **distillation content/02_2015_distilling_knowledge_hinton.md**: không liên quan trực tiếp về nội dung kỹ thuật, nhưng cùng chia sẻ một chủ đề meta: cả hai đều quan tâm tới việc kiểm soát ”độ nhạy” (sensitivity) của một mạng neural theo đầu vào (distillation qua temperature trong softmax, spectral normalization qua ràng buộc Lipschitz theo norm trọng số) để đạt một mục tiêu huấn luyện ổn định hơn.

8. Gợi ý áp dụng

Spectral normalization khống chế chuẩn phổ của từng ma trận trọng số để kiểm soát độ nhạy của mạng. Hiệu quả phụ thuộc phép xấp xỉ giá trị kỳ dị lớn nhất và cấu trúc toàn mạng; ràng buộc từng lớp không tự giải quyết mọi bất ổn trong huấn luyện GAN.

9. Thí nghiệm gợi ý

1. **Cài thuật toán power iteration và kiểm chứng trên ma trận ngẫu nhiên**: viết hàm Python/PyTorch hiện thực chính xác Algorithm 1 (công thức 20-22) cho một ma trận ngẫu nhiên kích thước vài trăm chiều, trên GPU (hoặc CPU, vì bài toán rất nhẹ). So sánh ước lượng $\sigma(W)$ sau 1, 5, 20 bước lặp với giá trị kỳ dị lớn nhất tính bằng `torch.linalg.svd` (chính xác). Kết quả kỳ vọng: sai số giảm theo cấp số nhân theo tỷ lệ $(\sigma_2/\sigma_1)^k$; sau khoảng 10-20 bước, sai số nhỏ hơn 1%. Diễn giải: xác nhận trực tiếp cơ chế hội tụ của power iteration đã minh hoạ bằng tay ở mục 3.3, với một ma trận lớn hơn và nhiều bước hơn.

2. **Toy GAN có/không spectral normalization trên dữ liệu 2D**: huấn luyện một GAN rất nhỏ (generator/discriminator vài lớp fully-connected) để học một phân phối 2D đơn giản (ví dụ hỗn hợp Gaussian nhiều mode), chạy nhanh trên CPU/trên GPU. So sánh độ ổn định huấn luyện (ví dụ dao động của loss, có mode collapse hay không) giữa phiên bản có và không có spectral normalization trên discriminator, với learning rate "hung hăng" cố ý (giống thiết lập D-F của paper). Kết quả kỳ vọng: phiên bản không chuẩn hoá dễ mất ổn định/mode collapse hơn ở learning rate cao, giống quan sát ở Figure 1-2 của paper (dù ở quy mô nhỏ hơn hẳn). Diễn giải: tái hiện trực tiếp phát hiện cốt lõi của paper (SN giúp bền hơn với learning rate cao) ở quy mô kiểm soát được.

3. **So sánh weight clipping vs spectral normalization trên cùng một discriminator nhỏ**: huấn luyện hai phiên bản của cùng một GAN nhỏ (toy, trên MNIST hoặc dữ liệu 2D), một dùng weight clipping (cắt trọng số về $[-0.01, 0.01]$ như WGAN gốc), một dùng spectral normalization, đo và so sánh phân phối giá trị kỳ dị của ma trận trọng số discriminator sau huấn luyện (giống Figure 3). Kết quả kỳ vọng: weight clipping cho giá trị kỳ dị tập trung mạnh vào thành phần đầu (gần hạng-1), spectral normalization cho phân phối rộng hơn. Diễn giải: xác nhận trực tiếp tuyên bố cốt lõi của paper ở §3 (weight clipping gây rank deficiency, spectral normalization không) bằng thực nghiệm tự làm, không chỉ dựa vào lý thuyết.

10. Tóm tắt công thức / khái niệm

- Chuẩn kỳ dị (spectral norm): $\sigma(A) = \max_{h \neq 0} \|Ah\|_2 / \|h\|_2$ = giá trị kỳ dị lớn nhất của A.
- Spectral normalization: $\bar{W}_{SN}(W) = W/\sigma(W)$, đảm bảo $\sigma(\bar{W}_{SN}) = 1$.
- Chặn Lipschitz toàn mạng: $\|f\|_{Lip} \leq \prod_{l=1}^{L+1} \sigma(W^l)$ (công thức 7); sau chuẩn hoá mỗi lớp, $\|f\|_{Lip} \leq 1$.
- Power iteration (Algorithm 1, công thức 20-22): $\tilde{v} \leftarrow W^\top \tilde{u}/\|W^\top \tilde{u}\|_2$; $\tilde{u} \leftarrow W\tilde{v}/\|W\tilde{v}\|_2$; $\sigma(W) \approx \tilde{u}^\top W \tilde{v}$. Chỉ 1 bước mỗi lượt cập nhật, $\tilde{u}$ được giữ liên tục qua các lượt.
- Gradient của trọng số chuẩn hoá (công thức 10, 12): $\frac{\partial V}{\partial W} = \frac{1}{\sigma(W)} \left(\hat{\mathbb{E}}[\delta h^\top] - \lambda u_1 v_1^\top \right)$, với $\lambda = \hat{\mathbb{E}}[\delta^\top \bar{W}_{SN} h]$; hạng tử thứ hai phạt thành phần kỳ dị thứ nhất, chống rank collapse.
- Weight normalization (so sánh, công thức 13): $\sigma_1(\bar{W}_{WN})^2 + \cdots + \sigma_T(\bar{W}_{WN})^2 = d_o$; tối ưu hoá norm dẫn tới ma trận hạng 1.
- Orthonormal regularization (so sánh, công thức 14): hạng tử điều chuẩn $\|W^\top W - I\|_F^2$, ép mọi giá trị kỳ dị về 1 (khác spectral normalization chỉ ép giá trị lớn nhất về 1).
- Ví dụ tự tính (mục 3.3): với $W = \text{diag}(2, 1)$, $\sigma_1 = 2$ đúng; sau 1 bước power iteration từ $\tilde{u}_0 = (1, 1)/\sqrt{2}$, ước lượng $\sigma \approx 1.817$ (sai số $\approx 9.15\%$), minh hoạ tốc độ hội tụ hình học $(\sigma_2/\sigma_1)^k = (1/2)^k$ (tự tính).

- Inception Score (IS): $I(\{x_n\}) = \exp(\mathbb{E}[D_{KL}[p(y|x)\|p(y)]])$, cao hơn tốt hơn; FID (Fréchet Inception Distance): khoảng cách 2nd-order giữa phân phối feature của ảnh thật và ảnh sinh, thấp hơn tốt hơn.

11. Kết luận ngắn

Bài báo đề xuất spectral normalization: chia mỗi ma trận trọng số của discriminator cho giá trị kỳ dị lớn nhất của nó, đảm bảo toàn mạng discriminator là 1-Lipschitz một cách trực tiếp (không cần mẫu dữ liệu), tính toán rẻ nhờ power iteration một bước mỗi lượt cập nhật. Khác biệt cốt lõi so với weight normalization/weight clipping (gây rank deficiency) và orthonormal regularization (phá huỷ thông tin phổ) là spectral normalization chỉ ràng buộc giá trị kỳ dị lớn nhất, giữ tự do cho các giá trị kỳ dị khác, cho phép discriminator dùng nhiều đặc trưng hơn. Thực nghiệm trên CIFAR-10, STL-10, ImageNet cho thấy SN-GANs đạt Inception Score/FID tốt hơn hoặc tương đương các kỹ thuật ổn định hoá GAN trước đó, bền hơn với learning rate cao, và là nghiên cứu đầu tiên (theo bài báo lưu ý) tạo được ảnh chất lượng tạm ổn từ ImageNet với một cặp generator-discriminator duy nhất.

Nguồn gốc: https://arxiv.org/abs/1802.05957

Bài 18

The Shattered Gradients Problem: If resnets are the answer, then what is the question? (2017)

Tác giả: David Balduzzi, Marcus Frean, Lennox Leary, JP Lewis, Kurt Wan-Duo Ma, Brian McWilliams
Venue / arXiv: arXiv:1702.08591 (v2, 06/06/2018); Proceedings of the 34th International Conference on Machine Learning (ICML), Sydney, Australia, PMLR 70, 2017 (paper tự ghi rõ ở trang đầu)
Nguồn: https://arxiv.org/abs/1702.08591
Chủ đề: Phân tích toán học vì sao gradient của mạng feedforward dùng rectifier (ReLU) càng sâu càng giống nhiễu trắng (white noise), làm huấn luyện khó khăn; chứng minh skip-connection (resnet) làm chậm hẳn quá trình này, và đề xuất một cách khởi tạo khác (”looks linear”) giải quyết vấn đề mà không cần skip-connection.
Vị trí trong lộ trình: nên đọc sau opt 37 (Delving Deep into Rectifiers / He init, 1502.01852) và opt 38 (Batch Normalization, 1502.03167) vì bài này dùng trực tiếp giả định khởi tạo của He et al. và phân tích tương tác với batch normalization; liên hệ với math 20 (Benefits of depth in neural networks) về số lượng vùng tuyến tính (linear region) tăng theo độ sâu, là nguyên nhân gốc của hiện tượng ”vỡ gradient” theo chính paper này.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (bản arXiv); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc.

1. Bối cảnh

Khi lan truyền ngược qua nhiều lớp, gradient có thể thu nhỏ hoặc tăng rất lớn, làm việc cập nhật trọng số khó khăn. Bài báo nhận xét rằng khởi tạo cẩn thận (Glorot & Bengio 2010; He và cộng sự 2015) cùng batch normalization (BN; Ioffe & Szegedy 2015) đã giảm đáng kể vấn đề này. Theo phân tích được bài dẫn, He initialization giữ phương sai activation qua các lớp rectifier trong điều kiện thích hợp, còn BN làm cho lan truyền ngược ít phụ thuộc vào thang của trọng số.

Nhưng có một nghịch lý thực nghiệm: dù đã dùng khởi tạo tốt và BN, **kiến trúc có skip-connection** (như highway network và resnet) vẫn hoạt động tốt hơn hẳn kiến trúc feedforward chuẩn khi mạng đủ sâu (He et al. 2016a;b). Câu hỏi mà paper đặt ra ngay từ tiêu đề: ”Nếu resnet là câu trả lời, thì câu hỏi là gì?” (”If resnets are the answer, then what is the question?”). Nói cách khác: nếu vanishing/exploding gradient đã được giải quyết bằng khởi tạo + BN, thì **tại sao resnet vẫn cần thiết và vẫn tốt hơn**? Paper xác định một vấn đề mới, trực giao (orthogonal) với vanishing/exploding gradient, gọi là **vấn đề gradient bị vỡ (shattered gradients problem)**.

Khái niệm cần giải thích trước khi đọc tiếp: **rectifier** (ReLU, Rectified Linear Unit) là hàm kích hoạt $\rho(a) = \max(0, a)$, không mượt (not smooth) tại $a = 0$ (có một điểm gãy, đạo hàm không liên tục). Paper phân biệt rõ giữa **đầu ra thực (real-valued output)** của rectifier và **activation nhị phân (binary activation)**: activation = 1 nếu đầu ra dương, = 0 nếu không (dòng 39-40 phần Terminology). Đây là phân biệt quan trọng cho phần phân tích sau: mỗi neuron rectifier, tại một điểm dữ liệu cụ thể, ”đang hoạt động” (active, output > 0, hành xử như tuyến tính) hoặc ”không hoạt động” (inactive, output = 0).

2. Ý tưởng chính

Ý tưởng cốt lõi: **tương quan (correlation) giữa gradient tại các điểm dữ liệu khác nhau** trong mạng feedforward chuẩn **suy giảm theo hàm mũ (exponentially) với độ sâu**, khiến gradient (xét như một hàm theo input, hoặc xét giữa các mini-batch) trông giống **nhiễu trắng (white noise)**: không có cấu trúc không gian, giá trị tại các điểm gần nhau không còn tương quan với nhau. Ngược lại, trong kiến trúc có skip-connection

(resnet, highway network), tương quan này **chỉ suy giảm theo tốc độ dưới tuyến tính (sublinear)**, cụ thể là theo $1/\sqrt{L}$ (L là độ sâu), khiến gradient giữ được nhiều cấu trúc hơn, gần giống **nhiễu nâu (brown noise, tức chuyển động Brown / Brownian motion)** hơn là nhiễu trắng.

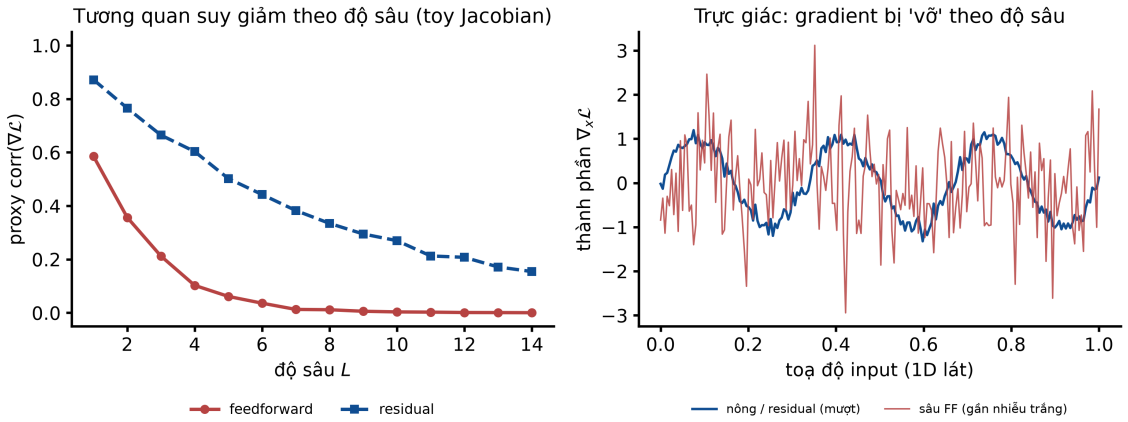
Hệ quả thực tiễn: khi gradient giống nhiễu trắng, các thuật toán tối ưu giả định "gradient tại các điểm gần nhau là giống nhau" (ví dụ phương pháp dùng momentum, phương pháp tăng tốc/accelerated methods) sẽ hoạt động kém, vì "ảnh hưởng của một neuron lên đầu ra mạng (tăng trọng số làm đầu ra tăng hay giảm) trở nên cực kỳ không ổn định, khiến việc học khó khăn" (dòng 183-186, "becomes extremely unstable making learning difficult"). Paper cũng nhấn mạnh phân tích này là **tại thời điểm khởi tạo (at initialization)**, chưa xét quá trình huấn luyện: "shattering nên giảm trong quá trình huấn luyện. Hiểu cách shattering ảnh hưởng huấn luyện là một vấn đề mở quan trọng" (dòng 34-35 phần Abstract).

Ý tưởng thứ hai, mang tính kỹ thuật phân tích: gradient của mạng rectifier có thể biểu diễn như **tổng theo các đường đi (path-sum)** qua mạng, trong đó mỗi "đường đi" (path) chỉ góp phần nếu TẤT CẢ neuron trên đường đi đó đang "hoạt động" (active). Khi mạng sâu hơn, số lượng đường đi tăng theo hàm mũ với độ sâu, nhưng tỷ lệ đường đi "cùng hoạt động" ở hai điểm dữ liệu khác nhau giảm nhanh hơn, dẫn đến tương quan giữa gradient giảm theo hàm mũ.

3. Phương pháp chi tiết

3.1 Mô hình tối giản và quan sát trực quan (§1.1)

Để loại bỏ các yếu tố gây nhiễu (gradient phụ thuộc cả loss và mẫu dữ liệu ngẫu nhiên, và có số chiều rất lớn), paper xây dựng **mô hình tối giản**: mạng $f_W : \mathbb{R} \to \mathbb{R}$ nhận số thực vào số thực (không phải dữ liệu thật), mỗi lớp ẩn có $N = 200$ neuron rectifier. Paper quan sát đạo hàm $\frac{df_W}{dx}(x^{(i)})$ với $x^{(i)} \in [-2, 2]$ trên lưới $M = 256$ "điểm dữ liệu" (Eq. 1).



Shattered gradients (toy): proxy tương quan Jacobian giảm nhanh theo độ sâu ở FF, chậm hơn với residual; trực giác nhiễu trắng. (minh hoạ, tự tính)

Quan sát trực quan (Figure 1 của paper, mô tả bằng lời vì không đọc được hình trực tiếp từ text, nhưng chú thích hình được đọc đầy đủ): mạng feedforward 1 lớp cho gradient có cấu trúc giống **nhiễu nâu (brown noise)** (so sánh trực tiếp với xấp xỉ rời rạc của chuyển động Brown $B^N(t) = \sum_{s=1}^{t} W_s$ với $W_s \sim N(0, 1/N)$, công thức dòng 182-184); mạng feedforward 24 lớp cho gradient giống **nhiễu trắng (white noise, lấy mẫu $W_k \sim N(0, 1)$)**; mạng resnet 50 lớp (dùng batch normalization với $\beta = 0.1$) cho gradient có cấu trúc ***ở giữa*** hai cực đó, "nhiều cấu trúc hơn hẳn mạng feedforward tương đương".

Paper giải thích cơ sở lý thuyết của sự tương đồng với Brownian motion: "mục A1 áp dụng định lý Donsker để chứng minh gradient hội tụ về chuyển động Brown khi $N \to \infty$". Về mặt trực giác: nếu tất cả neuron là tuyến tính (linear), gradient sẽ là một đường ngang (constant theo x); nhưng rectifier không mượt nên gradient

có các điểm gián đoạn (discontinuous) tại các ngưỡng bias của từng neuron, và việc tích luỹ nhiều điểm gián đoạn ngẫu nhiên (độc lập) qua nhiều lớp tạo ra hành vi giống bước ngẫu nhiên (random walk).

Để đo lường định lượng cấu trúc không gian của gradient, paper dùng **hàm tự tương quan (Autocorrelation Function, ACF)**, so sánh ACF của gradient mạng feedforward/resnet ở các độ sâu khác nhau với ACF của nhiễu trắng và nhiễu nâu (Figure 2). Kết quả định tính: "ACF của mạng nông giống ACF nhiễu nâu. Khi mạng sâu hơn, ACF nhanh chóng giống ACF nhiễu trắng" cho feedforward; nhưng với resnet, "mặc dù gradient trở nên kém có cấu trúc hơn dần, chúng không 'trắng hoá' (whiten) đến mức độ của mạng feedforward chuẩn... vẫn còn tương quan trong resnet 50 lớp trong khi ở mạng feedforward tương đương, gradient không thể phân biệt được với nhiễu trắng". Paper cũng chỉ ra hiệu ứng của β-**rescaling** (Szegedy et al. 2016): "hiệu ứng ấn tượng của β-rescaling mới được đề xuất: ACF của mạng 50 lớp giống nhiễu nâu" khi dùng β nhỏ.

3.2 Định nghĩa toán học chính thức (§3)

Paper định nghĩa (Definition 1): cho mỗi input $x^{(i)} \in D$, $\nabla_i := \sum_{p=1}^{P} \frac{\partial f_p}{\partial n}(x^{(i)})$ là đạo hàm của (tổng các toạ độ đầu ra) f_p theo đầu ra n của một neuron cụ thể, tính tại input $x^{(i)}$. Vì trọng số được lấy mẫu từ phân phối trung bình 0, ∇_i có kỳ vọng 0. Hiệp phương sai (covariance) và tương quan (correlation) giữa gradient tại hai điểm dữ liệu được định nghĩa:

$$C(i,j) = \mathbb{E}[\nabla_i \nabla_j] \quad \text{và} \quad R(i,j) = \frac{\mathbb{E}[\nabla_i \nabla_j]}{\sqrt{\mathbb{E}[\nabla_i^2] \cdot \mathbb{E}[\nabla_j^2]}}$$

(kỳ vọng lấy theo phân phối của trọng số). Lưu ý quan trọng paper tự giải thích: theo quy tắc chuỗi (chain rule), cập nhật trọng số $\frac{\partial f_W}{\partial w_{ij}} = \frac{\partial f_W}{\partial n_j} \cdot \frac{\partial n_j}{\partial w_{ij}}$; thành phần $\frac{\partial f_W}{\partial n_j}$ (đạo hàm đầu ra mạng theo đầu ra neuron ở một lớp, tức là chính đại lượng ∇_i mà paper phân tích) là thành phần **phụ thuộc vào cấu trúc mạng (kiến trúc, độ sâu)**, còn các thành phần khác không phụ thuộc. Đây là lý do paper tập trung phân tích ∇_i thay vì toàn bộ gradient trọng số.

Giả định điểm dữ liệu điển hình (Assumption 1, "typical datapoints"): hai điểm dữ liệu $x^{(i)}, x^{(j)}$ được coi là "điển hình" nếu một nửa neuron mỗi lớp đang hoạt động cho mỗi điểm, và một phần tư neuron mỗi lớp cùng hoạt động cho cả hai điểm (tức là xác suất "cùng active" của hai neuron độc lập là $0.5 \times 0.5 = 0.25$, tự tính, khớp với giả định "một phần tư"). Giả định này được paper kiểm chứng thực nghiệm bằng Figure 3: với batch normalization, tỷ lệ "cùng hoạt động" (coactivation) của một cặp input phân biệt là **đúng 1/4** (dòng 324, "neurons are co-active for 1/4 of distinct pairs of inputs, which is what would happen if activations were decided by unbiased coin flips"), còn **không có** batch normalization, tỷ lệ cùng hoạt động "tăng dần theo độ sâu, cho thấy phản hồi neuron ngày càng dư thừa (redundant)" - tức là giả định "điểm dữ liệu điển hình" chỉ đúng tốt khi có BN.

3.3 Định lý 1: mạng feedforward (§3.1)

Giả định trọng số được khởi tạo với phương sai $\sigma^2 = 2/N$ theo He et al. (2015). Với một neuron n cách đầu ra L lớp, **Định lý 1 (hiệp phương sai gradient trong mạng feedforward)**:

a) Phương sai gradient tại $x^{(i)}$: $C^{fnn}(i) = 1$.

b) Hiệp phương sai: $C^{fnn}(i,j) = \frac{1}{2^L}$.

Paper giải thích ý nghĩa: phần (a) "khôi phục lại quan sát của He et al. (2015) rằng đặt $\sigma^2 = 2/N$ giữ nguyên phương sai qua các lớp trong mạng rectifier" - tức đây là một kiểm chứng nhất quán với lý thuyết He init đã biết. Phần (b) là **phát hiện mới** (dòng 389, "Part (b) is new"), giải thích trực tiếp quan sát thực nghiệm ở Figure 2a rằng gradient trong mạng feedforward "trắng hoá" theo độ sâu: "trực giác, gradient trắng hoá vì số đường đi (path) qua mạng tăng theo hàm mũ nhanh hơn hẳn phần đường đi cùng hoạt động (co-active)".

Vì $C^{fnn}(i) = 1$ (hằng số, không đổi theo L) nhưng $C^{fnn}(i,j) = 1/2^L \to 0$ theo hàm mũ, **tương quan** $R^{fnn}(i,j) = C^{fnn}(i,j)/C^{fnn}(i) = 1/2^L$ cũng suy giảm theo hàm mũ với L.

3.4 Định lý 2: resnet không có batch normalization (§3.2)

Module residual gốc (He et al. 2016a): $x_l = x_{l-1} + W_l\rho_{BN}(V_l\rho_{BN}(x_{l-1}))$, trong đó $\rho_{BN}(a) = \rho(BN(a))$. Paper phân tích phiên bản đơn giản hoá (bỏ V_l, không làm thay đổi bản chất phân tích):

$$x_l = \alpha \cdot x_{l-1} + \beta \cdot W_l\rho_{BN}(x_{l-1}) \tag{2}$$

với α, β là hệ số tái chuẩn hoá (rescaling factor). **Định lý 2 (hiệp phương sai gradient trong resnet)**: xét resnet **không** dùng batch normalization, $\alpha = \beta = 1$, $\sigma^2 = 2/N$ như trên:

a) Phương sai: $C^{res}(i) = 2^L$.

b) Hiệp phương sai: $C^{res}(i,j) = \left(\frac{3}{2}\right)^L$.

Suy ra tương quan $R^{res}(i,j) = \left(\frac{3}{4}\right)^L$ (tự tính: $(3/2)^L/2^L = (3/4)^L$, khớp với công thức paper ghi trực tiếp ở dòng 439). Paper chỉ rõ **hệ quả tiêu cực**: định lý này ngụ ý resnet không có BN có **hai vấn đề**: (i) phương sai gradient tăng (theo 2^L, tức là *nổ*/explode theo độ sâu), và (ii) tương quan vẫn suy giảm theo hàm mũ (theo $(3/4)^L$, dù chậm hơn $(1/2)^L$ của feedforward vì $3/4 > 1/2$). Cả hai vấn đề này "đều quan sát được thực nghiệm".

3.5 Rescaling và Định lý 3: resnet có batch normalization (§3.3)

Giải pháp cho phương sai nổ: rescale bằng $\alpha = 1/\sqrt{2}$, cho $C^{res}_{\alpha=1/\sqrt{2}}(i) = 1$ (ổn định phương sai) nhưng $R^{res}_{\alpha=1/\sqrt{2}}(i,j) = (3/4)^L$ **vẫn giữ nguyên tốc độ suy giảm theo hàm mũ** của tương quan - tức rescaling α đơn thuần không giải quyết được vấn đề "vỡ gradient", chỉ giải quyết vấn đề phương sai nổ. Trong thực tế, α-rescaling không được dùng; thay vào đó activation được tái chuẩn hoá bằng batch normalization, kết hợp đặt $\beta \in [0.1, 0.3]$ theo Szegedy et al. (2016).

Định lý 3 (hiệp phương sai gradient trong resnet có BN và β-rescaling): với resnet có batch normalization và β-rescaling,

a) Phương sai: $C^{res}_{\beta,BN}(i) = \beta^2(L-1) + 1$.

b) Hiệp phương sai: $C^{res}_{\beta,BN}(i,j) \sim \beta\sqrt{L}$.

c) Tương quan: $R^{res}_{\beta,BN}(i,j) \sim \frac{1}{\beta\sqrt{L}}$ (kèm chú thích [1] trỏ tới mục A3.4 phụ lục để xem tính toán chính xác, paper không chứng minh đầy đủ trong thân bài).

Đây là kết quả **then chốt** của paper: tốc độ suy giảm tương quan đổi từ **hàm mũ** $1/2^L$ (feedforward) sang **dưới tuyến tính** $1/\sqrt{L}$ (resnet có BN). Paper giải thích trực giác: "phương sai đầu ra của các lớp tăng tuyến tính, nên batch normalization tái chuẩn hoá chúng theo các lượng khác nhau. Rescale bằng β thêm một hệ số hằng".

3.6 Ví dụ số minh hoạ từ chính paper, và tính tay kiểm chứng (tự tính)

Paper tự đưa ra một ví dụ số cụ thể: "mô hình dự đoán khi dùng batch normalization với $\beta = 0.1$ trên resnet 100 lớp cho tương quan điển hình $R^{res}_{0.1,BN}(i,j) = 0.7$. Đặt $\beta = 1.0$ cho $R^{res}_{1.0,BN}(i,j) = 0.1$. Ngược lại, mạng feedforward 100 lớp có tương quan không thể phân biệt được với 0."

Kiểm tra lại bằng công thức $R \sim 1/(\beta\sqrt{L})$ (tự tính): với $\beta = 0.1$, $L = 100$: $1/(0.1 \times \sqrt{100}) = 1/(0.1 \times 10) = 1/1 = 1.0$. Với $\beta = 1.0$, $L = 100$: $1/(1.0 \times 10) = 0.1$. Nhận xét tự rút ra (suy luận của người

ghi chú): công thức "$\sim$" (xấp xỉ tỉ lệ, không phải đẳng thức chính xác, ký hiệu paper dùng ở Theorem 3b/3c) khớp đúng với trường hợp $\beta = 1.0$ (ra đúng 0.1 như paper nói), nhưng với $\beta = 0.1$ công thức xấp xỉ cho ra 1.0 trong khi paper nói con số cụ thể là 0.7; đây không phải mâu thuẫn mà là bản chất của ký hiệu tỷ lệ bậc lớn ($\sim$, tương tự $O(\cdot)$ trong giải tích), không phải công thức chính xác tuyệt đối - công thức chính xác đầy đủ được paper nói "xem mục A3.4" (phụ lục, không đọc chi tiết công thức đóng chính xác trong ghi chú này, chỉ đọc phần kết quả §3 chính). Đây là điểm cần lưu ý khi dùng công thức Theorem 3 cho mục đích tính toán cụ thể: chỉ nên dùng để hiểu xu hướng (order of magnitude), không nên coi là công thức đóng chính xác.

3.7 Highway network (§3.4)

Highway network (Srivastava et al. 2015) có dạng $x_l = (1 - T(x_{l-1})) \cdot x_{l-1} + T(x_{l-1}) \cdot H(x_{l-1})$ với T, H là cổng (gate) và đặc trưng (feature) học được. Paper xét biến thể đơn giản hoá với cổng là **số vô hướng (scalar)** cố định γ_1, γ_2 thoả $\gamma_1^2 + \gamma_2^2 = 1$: $x_l = \gamma_1 \cdot x_{l-1} + \gamma_2 \cdot W_l \rho(x_{l-1})$.

Corollary 1 (hiệp phương sai gradient trong highway network): a) phương sai $C_\gamma^{HN}(i) = 1$; b) tương quan $R_\gamma^{HN}(i,j) = \left(\gamma_1^2 + \frac{1}{2}\gamma_2^2\right)^L$.

Điểm đặc biệt: nếu chọn $\gamma_1 = \sqrt{1 - 1/L}$ và $\gamma_2 = \sqrt{1/L}$ (tức là các hệ số phụ thuộc vào chính độ sâu L), thì khi $L \to \infty$:

$$\lim_{L\to\infty} R_\gamma^{HN}(i,j) = \frac{1}{\sqrt{e}}$$

(tự kiểm chứng bằng giới hạn toán học: $\left(1 - \frac{1}{L} + \frac{1}{2L}\right)^L = \left(1 - \frac{1}{2L}\right)^L \to e^{-1/2} = 1/\sqrt{e} \approx 0.6065$ khi $L \to \infty$, tự tính, khớp với công thức paper nêu). Đây là kết quả ấn tượng nhất về mặt lý thuyết: tương quan **không suy giảm theo độ sâu** (tiến tới một hằng số dương $1/\sqrt{e} \approx 0.6065$ chứ không về 0), đạt "độ sâu độc lập" (depth-independent correlation structure, đúng như tiêu đề Corollary 1). Đánh đổi (trade-off) mà paper nêu ngay sau: "sự đánh đổi là đóng góp của các lớp trở nên ngày càng tầm thường (gần với identity) khi $L \to \infty$" - vì $\gamma_2 = \sqrt{1/L} \to 0$, mỗi lớp gần như không biến đổi gì cả khi mạng rất sâu.

4. Thí nghiệm và kết quả chính

4.1 Hiệu quả neuron và batch normalization (§2)

Paper quan sát (Figure 3, 100 mạng fully-connected rectifier với 100 unit ẩn mỗi lớp, trung bình trên 100 lần chạy): tỷ lệ neuron "hoạt động" (activation) trung bình luôn khoảng 1/2 bất kể có/không có BN, nhưng **phân phối trở nên lưỡng cực (bimodal) theo độ sâu** khi không có BN: "ở lớp 50 trong mạng feedforward không BN, neuron hoặc luôn hoạt động hoặc luôn không hoạt động". BN làm "hầu hết neuron hoạt động cho đúng một nửa input". Paper rút ra: "neuron luôn hoạt động thì cũng như tuyến tính; neuron luôn không hoạt động thì cũng như không tồn tại. Do đó batch normalization tăng hiệu quả sử dụng tính phi tuyến rectifier" - đây là một đóng góp phụ của paper, độc lập với phân tích chính về gradient.

Figure 4 minh hoạ bằng raster-plot: activation của mạng feedforward không BN "giống nhiễu tĩnh truyền hình (static television noise), cấu trúc không gian bị phá huỷ hoàn toàn"; resnet "thể hiện một sự thoả hiệp: neuron được sử dụng hiệu quả nhưng cấu trúc không gian cũng được bảo toàn phần nào", đo bằng "contiguity histogram" (biểu đồ đếm độ dài các dãy hoạt động liên tục).

4.2 Gradient vỡ trong convnet trên CIFAR-10 (§4)

Paper kiểm tra các kết quả lý thuyết (vốn chứng minh cho mạng fully-connected) có đúng với **convnet trên dữ liệu thật** (CIFAR-10) không. Thiết lập: mạng feedforward và resnet với 2, 4, 10, 24, 50 lớp, dùng biến thể kiến trúc "bottleneck" (He et al. 2016a), chèn một skip-connection cho mỗi hai lớp convolution, cả hai kiến trúc đều dùng batch normalization.

Figure 5a-b: so sánh ma trận hiệp phương sai của gradient ở lớp đầu tiên, minibatch 256 mẫu ngẫu nhiên từ CIFAR-10, độ sâu 2 và 50. Kết quả: ”cấu trúc không gian hiện rõ ở cả hai mạng 2 lớp, dù rõ hơn ở resnet. Trong mạng feedforward, cấu trúc nhanh chóng biến mất theo độ sâu. Trong resnet, cấu trúc vẫn hiện rõ ở 50 lớp”.

Figure 5c: đo **relative effective rank** (hạng hiệu dụng tương đối), định nghĩa $r(\Delta) = \mathrm{tr}(\Delta^\top \Delta)/\|\Delta\|_2^2$ (đo ”chiều nội tại” của ma trận Δ có các cột là gradient theo input, so với một ma trận ”trắng” Y cùng kích thước có phần tử Gauss độc lập). ”Relative effective rank (trung bình trên 30 minibatch) tăng nhanh hơn hẳn theo độ sâu cho mạng không có skip-connection. Với resnet, tham số β làm chậm tốc độ tăng của effective rank, đúng như dự đoán của Theorem 3”.

Figure 5d: norm ℓ_2 trung bình của gradient trên mỗi toạ độ (chuẩn hoá theo độ lệch chuẩn tính trên minibatch). ”Đại lượng này giảm nhanh hơn hẳn theo độ sâu cho mạng feedforward. Điều này do hiệu ứng lấy trung bình gradient ngày càng trắng hoá trong mỗi minibatch. Nói cách khác, nhiễu trong minibatch lấn át tín hiệu. Hiện tượng này ít rõ rệt hơn hẳn trong resnet”.

4.3 LL-init: khởi tạo ”trông tuyến tính” (§5)

Paper đề xuất một giải pháp **thay thế** cho skip-connection: khởi tạo rectifier sao cho nó ”trông tuyến tính” (looks linear, LL-init) ngay từ đầu. Dùng CReLU (concatenated ReLU, Shang et al. 2016): $x \mapsto \begin{pmatrix} \rho(x) \\ \rho(-x) \end{pmatrix}$. Quan sát chính: khởi tạo trọng số với cấu trúc khối đối xứng (mirrored block structure) $W, -W$ cho đầu ra: $W \cdot \rho(x) - W \cdot \rho(-x) = W \cdot x$ (tự kiểm chứng: $\rho(x) - \rho(-x) = \max(0, x) - \max(0, -x) = x$ với mọi x thực, vì nếu $x > 0$ thì $= x - 0 = x$, nếu $x < 0$ thì $= 0 - (-x) = x$, đúng với mọi x; tự tính). Nghĩa là **tại thời điểm khởi tạo**, toàn mạng hành xử đúng như một mạng tuyến tính (không có gradient vỡ, vì mạng tuyến tính không có vấn đề này - xem nhận xét ngay sau Eq. 1), nhưng vẫn giữ được tính phi tuyến *tiềm năng* một khi hai khối trọng số phân kỳ (diverge) trong quá trình huấn luyện.

Thí nghiệm CIFAR-10 (Figure 6, Adam optimizer, auto-scheduler giảm learning rate khi loss training chững lại 5 lần đo liên tiếp trong 10 epoch gần nhất, data augmentation dịch tối đa 4 pixel + lật ngang xác suất 0.5): so sánh 5 kiến trúc ở các độ sâu 6, 14, 30, 54, 102, 198 lớp - ”Linear” (baseline tuyến tính thuần), ”ReLU” (feedforward chuẩn), ”CReLU w/o LL” (CReLU nhưng khởi tạo thường), ”Resnet”, ”CReLU w/ LL” (đề xuất của paper). Kết quả: ”mạng ReLU và CReLU hoạt động ngày càng kém theo độ sâu; mạng ReLU hoạt động kém hơn cả baseline tuyến tính (40%) ở độ sâu tối đa 198. Mạng feedforward với LL-init hoạt động **tương đương resnet**, cho thấy gradient vỡ là một phần lớn của vấn đề khi huấn luyện mạng rất sâu”. Đây là bằng chứng thực nghiệm mạnh nhất ủng hộ giả thuyết trung tâm của paper: nếu một kiến trúc **không có skip-connection nhưng không bị vỡ gradient** (nhờ LL-init) cũng huấn luyện tốt như resnet, thì ”shattered gradients” đúng là nguyên nhân cốt lõi khiến mạng feedforward sâu khó huấn luyện, không phải bản thân việc thiếu skip-connection.

Paper lưu ý rõ mục tiêu thí nghiệm **không phải** đạt state-of-the-art (”The aim was not to match the state-of-the-art, but rather to test the hypothesis”, dòng 641-642) - đây là thí nghiệm kiểm định giả thuyết (hypothesis test) có kiểm soát, không phải thí nghiệm benchmark.

5. Phân tích / ablation

Về bản chất, các Định lý 1-3 và Corollary 1 chính là ”ablation lý thuyết”: mỗi định lý xét một biến thể kiến trúc (feedforward thường, resnet không BN, resnet có BN, highway network) và tách bạch ảnh hưởng của từng thành phần:

- So Định lý 1 (feedforward, $R \sim 1/2^L$) với Định lý 2 (resnet không BN, $R \sim (3/4)^L$): thêm skip-connection (không BN) làm chậm tốc độ suy giảm tương quan (vì $3/4 > 1/2$) nhưng **vẫn là hàm mũ**, và gây ra vấn đề mới là phương sai nổ (2^L).

- So Định lý 2 với §3.3 (α-rescaling, R vẫn $(3/4)^L$): α-rescaling giải quyết phương sai nổ nhưng **không** đổi được bản chất hàm mũ của suy giảm tương quan - cho thấy vấn đề "tương quan suy giảm" và vấn đề "phương sai nổ" là hai hiện tượng tách biệt, cần cơ chế khác nhau để giải quyết.
- So §3.3 với Định lý 3 (BN thay cho α-rescaling): đổi hẳn dạng hàm từ mũ ($(3/4)^L$) sang dưới tuyến tính ($1/\sqrt{L}$) - đây là "thành phần" quan trọng nhất, cho thấy **chính batch normalization** (không phải chỉ bản thân phép cộng residual) là yếu tố làm thay đổi bản chất tốc độ suy giảm.
- Corollary 1 (highway network với γ phụ thuộc L) đẩy xa hơn nữa: tương quan có thể **không suy giảm theo độ sâu**, đổi lại mỗi lớp góp phần ngày càng nhỏ.

Phần thực nghiệm §4.3 (LL-init) là một "ablation" theo nghĩa khác: giữ cố định việc không có skip-connection, chỉ thay đổi cách khởi tạo, để tách bạch xem "gradient vỡ" (chứ không phải "thiếu residual path" per se) có phải nguyên nhân chính khiến mạng sâu khó huấn luyện.

Paper tự ghi nhận một điểm chưa giải thích được (một dạng "giới hạn của phân tích ablation"): "các thí nghiệm sơ bộ (không trình bày) cho thấy LL-init hiệu quả hơn trên kiến trúc dựa CReLU so với PReLU. Lý do chưa rõ" (dòng 584-588, "The reason is unclear"). Đây là một khoảng trống trong hiểu biết mà paper **tự thừa nhận công khai**, không cố gắng giải thích thêm.

6. Giới hạn

Các giới hạn **bài báo nêu rõ**:

1. Phân tích Định lý 1-3 và Corollary 1 chỉ chứng minh cho **mạng fully-connected** với số neuron bằng nhau ở mỗi lớp; "mở rộng sang convnet cần thêm công việc sổ sách (bookkeeping) đáng kể". Phần thực nghiệm §4 kiểm chứng định tính trên convnet thật (CIFAR-10) nhưng không có định lý toán học tổng quát cho convnet.
2. Giả định "điểm dữ liệu điển hình" (Assumption 1) "sẽ không đúng cho mọi cặp điểm dữ liệu", chỉ được kiểm chứng **trung bình** đúng khi có batch normalization (Figure 3); không có BN, giả định này sai dần theo độ sâu (tỷ lệ cùng hoạt động tăng, không giữ ở 1/4).
3. Toàn bộ phân tích là **tại thời điểm khởi tạo (at initialization)**, chưa phải trong suốt quá trình huấn luyện: "shattering nên giảm trong quá trình huấn luyện. Hiểu cách shattering ảnh hưởng huấn luyện là một vấn đề mở quan trọng" (dòng 34-35, trích lại từ Abstract) - bài báo lưu ý đây là câu hỏi chưa trả lời.
4. Công thức Theorem 3 (b, c) chỉ là xấp xỉ tỷ lệ bậc lớn ($\sim$), không phải đẳng thức chính xác (xem mục 3.6: ví dụ số với $\beta = 0.1$ không khớp hoàn toàn khi dùng công thức xấp xỉ, dù paper nói công thức chính xác đầy đủ có trong phụ lục A3.4 mà ghi chú này không đọc chi tiết).
5. Thí nghiệm LL-init (§5.1) dùng kiến trúc "mỏng" (thin, số filter nhỏ, từ 8 đến 64) cố ý để "đặc biệt khó cho gradient lan truyền ở độ sâu cao" - đây là một lựa chọn thiết kế thí nghiệm có chủ đích làm nổi bật hiệu ứng, không phải kiến trúc thực tế dùng trong sản xuất; paper tự nói rõ mục tiêu "không phải đạt state-of-the-art".
6. Lý do LL-init hiệu quả hơn trên CReLU so với PReLU "chưa rõ" (đã nêu ở mục 5), paper không giải thích được.
7. Paper không đề xuất một giải pháp duy nhất "tốt nhất"; LL-init được trình bày như một hướng thay thế khả thi (alternate solution), không phải thay thế hoàn toàn skip-connection trong thực hành (paper gợi ý hướng tương lai là "kết hợp LL-init với skip-connection", dòng 661-662, chưa thử nghiệm).

7. Liên hệ với các bài khác

- **opt 37 (Delving Deep into Rectifiers / He initialization, 1502.01852)**: được trích dẫn trực tiếp và dùng làm nền tảng giả định $\sigma^2 = 2/N$ xuyên suốt Định lý 1-3 của bài này (He et al. 2015 trong reference list). Bài hiện tại chỉ ra rằng He init giải quyết đúng vấn đề nó nhắm tới (giữ phương sai activation qua các lớp) nhưng **không** giải quyết vấn đề "vỡ tương quan gradient" mà paper này mới phát hiện.

- **opt 38 (Batch Normalization, 1502.03167)**: là đối tượng phân tích trung tâm của §2 và Định lý 3; phát hiện bất ngờ của bài này (”An alternate solution... an unexpected side-effect of batch normalization”, dòng 657-659) là BN không chỉ giảm covariate shift như mục tiêu gốc mà còn **thay đổi hẳn tốc độ suy giảm tương quan gradient** từ hàm mũ sang $1/\sqrt{L}$ khi kết hợp với resnet.
- **math 20 (Benefits of depth in neural networks, 1602.04485)**: Phần Kết luận của bài hiện tại trích trực tiếp Montufar et al. (2014) và Telgarsky (2016): ”số vùng tuyến tính (linear region) có thể tăng theo hàm mũ với độ sâu... một hệ quả ít được nhận ra của sự tăng hàm mũ này là sự gia tăng các điểm gián đoạn trong gradient của mạng rectifier” - đây là **liên hệ nhân quả trực tiếp** mà bài báo nêu: chính hiện tượng ”số vùng tuyến tính tăng theo hàm mũ” (chủ đề của math 20) là nguyên nhân sâu xa của ”shattered gradients problem”.
- **llm content/01_2017_attention_is_all_you_need.md**: Transformer dùng residual connection quanh mỗi tầng con attention và FFN; phân tích của bài này về lý do residual connection giúp gradient ”ít vỡ hơn” khi mạng rất sâu cung cấp nền tảng lý thuyết cho lựa chọn kiến trúc đó, dù Attention Is All You Need không tự phân tích theo góc độ tương quan gradient.
- **pruning content/31_2024_shortgpt.md, pruning content/33_2024_shortened_llama.md, pruning content/34_2024_sleb.md**: các bài này về cắt bớt lớp (layer pruning) của LLM đã huấn luyện xong, khác bối cảnh với bài hiện tại (bài hiện tại nói về gradient *lúc khởi tạo*, trước huấn luyện); liên hệ gián tiếp (suy luận): khi cắt một lớp residual giữa mạng đã huấn luyện, phần còn lại của mạng (đặc biệt các lớp sau điểm cắt) có thể gặp một dạng ”sốc” tương tự về cấu trúc tương quan tín hiệu lan truyền, dù đây không phải cùng cơ chế gradient-tại-khởi-tạo mà paper này phân tích.

8. Gợi ý áp dụng

Phân tích tương quan gradient giúp giải thích một khó khăn khi huấn luyện mạng rất sâu và vai trò của kết nối residual. Các kết quả định lượng được suy ra trong những giả thiết cụ thể về mạng và khởi tạo, không nên chuyển trực tiếp sang kiến trúc khác.

9. Thí nghiệm gợi ý

2. **Kiểm chứng Định lý 1 bằng Monte Carlo**: với mô hình tối giản trên, lấy L cố định (ví dụ $L = 5, 10, 15$), khởi tạo lại mạng (resample trọng số) nhiều lần (ví dụ 200 lần), mỗi lần tính ∇_i tại hai điểm dữ liệu cố định $x^{(i)}, x^{(j)}$ cách nhau một khoảng cố định, ước lượng hiệp phương sai thực nghiệm $\widehat{C}(i,j) = \frac{1}{200}\sum \nabla_i \nabla_j$. So với dự đoán lý thuyết $1/2^L$. Kết quả kỳ vọng: giá trị ước lượng giảm theo đúng tốc độ hàm mũ 2^{-L} khi vẽ trên thang log. Diễn giải: kiểm chứng định lượng trực tiếp công thức toán học của paper bằng mô phỏng, giúp hiểu rõ bản chất ”suy giảm theo hàm mũ” không chỉ là lời văn mà là một con số đo được.

10. Tóm tắt công thức / khái niệm

- Định nghĩa hiệp phương sai/tương quan gradient: $C(i,j) = \mathbb{E}[\nabla_i \nabla_j]$, $R(i,j) = \dfrac{\mathbb{E}[\nabla_i \nabla_j]}{\sqrt{\mathbb{E}[\nabla_i^2]\mathbb{E}[\nabla_j^2]}}$.
- Định lý 1 (feedforward, He init $\sigma^2 = 2/N$): $C^{fnn}(i) = 1$; $C^{fnn}(i,j) = 1/2^L$; suy ra $R^{fnn}(i,j) = 1/2^L$ (suy giảm hàm mũ).
- Định lý 2 (resnet không BN, $\alpha = \beta = 1$): $C^{res}(i) = 2^L$ (phương sai nổ); $C^{res}(i,j) = (3/2)^L$; $R^{res}(i,j) = (3/4)^L$ (tự tính từ tỉ số hai công thức trên, khớp paper).
- Rescale $\alpha = 1/\sqrt{2}$: ổn định phương sai ($C = 1$) nhưng tương quan vẫn $(3/4)^L$ (không giải quyết vỡ gradient).
- Định lý 3 (resnet có BN và β-rescaling): $C^{res}_{\beta,BN}(i) = \beta^2(L-1)+1$; $C^{res}_{\beta,BN}(i,j) \sim \beta\sqrt{L}$; $R^{res}_{\beta,BN}(i,j) \sim 1/(\beta\sqrt{L})$ (suy giảm dưới tuyến tính, chậm hơn hẳn hàm mũ).

- Corollary 1 (highway network, cổng vô hướng γ_1, γ_2, $\gamma_1^2 + \gamma_2^2 = 1$): $C_\gamma^{HN}(i) = 1$; $R_\gamma^{HN}(i,j) = (\gamma_1^2 + \frac{1}{2}\gamma_2^2)^L$; với $\gamma_1 = \sqrt{1 - 1/L}$, $\gamma_2 = \sqrt{1/L}$: $\lim_{L\to\infty} R_\gamma^{HN}(i,j) = 1/\sqrt{e} \approx 0.6065$ (tự tính bằng giới hạn $(1 - \frac{1}{2L})^L \to e^{-1/2}$).
- Ví dụ số tự kiểm tra (mục 3.6): công thức $R \sim 1/(\beta\sqrt{L})$ với $\beta = 1.0, L = 100$ cho 0.1 (khớp đúng số paper nêu); với $\beta = 0.1, L = 100$ cho 1.0 (paper nêu số thực tế là 0.7, chênh lệch do "$\sim$" chỉ là xấp xỉ bậc lớn, không phải đẳng thức chính xác) (tự tính).
- LL-init (CReLU, khởi tạo khối đối xứng $W, -W$): $W\rho(x) - W\rho(-x) = Wx$ với mọi x (tự kiểm chứng đại số), khiến mạng hành xử tuyến tính tại khởi tạo dù dùng rectifier.
- Relative effective rank: $r(\Delta) = \text{tr}(\Delta^\top \Delta)/\|\Delta\|_2^2$, đo "chiều nội tại" của ma trận gradient, dùng để định lượng mức độ "trắng hoá".

11. Kết luận ngắn

Bài báo xác định "vấn đề gradient vỡ" (shattered gradients problem), trực giao với vanishing/exploding gradient đã biết: tương quan giữa gradient tại các điểm dữ liệu khác nhau suy giảm theo hàm mũ với độ sâu trong mạng feedforward rectifier chuẩn (Định lý 1), khiến gradient giống nhiễu trắng; skip-connection kết hợp batch normalization đổi tốc độ suy giảm này thành dưới tuyến tính $1/\sqrt{L}$ (Định lý 3), giải thích tại sao resnet huấn luyện được ở độ sâu lớn hơn hẳn mạng feedforward dù cả hai đều dùng khởi tạo và BN tốt. Thực nghiệm trên CIFAR-10 (cả fully-connected và convnet) xác nhận các dự đoán lý thuyết, và một khởi tạo thay thế (LL-init, không cần skip-connection) đạt hiệu quả gần bằng resnet, ủng hộ mạnh giả thuyết rằng chính "gradient vỡ", không phải sự thiếu vắng đường tắt per se, là nguyên nhân cốt lõi khiến mạng feedforward rất sâu khó huấn luyện.

Nguồn gốc: https://arxiv.org/abs/1702.08591

Bài 19

Measuring the Intrinsic Dimension of Objective Landscapes (2018)

Tác giả: Chunyuan Li (Duke University), Heerad Farkhoor, Rosanne Liu, Jason Yosinski (Uber AI Labs).
Venue / arXiv: arXiv:1804.08838 (v1, 24/04/2018); venue ghi trong paper: ”Published as a conference paper at ICLR 2018”.
Nguồn: https://arxiv.org/abs/1804.08838
Chủ đề: đo ”intrinsic dimension” (số chiều nội tại) của objective landscape bằng phương pháp random subspace training; dùng nó để so sánh độ khó bài toán giữa các domain (supervised, RL) và để nén mạng.
Vị trí trong lộ trình: đọc sau math 18 (Shattered Gradients, cùng khảo sát ”objective landscape” nhưng theo hướng gradient thay đổi hỗn loạn theo độ sâu) và trước math 22-25 (NNGP, NTK, Principles of Deep Learning Theory, cùng đặt câu hỏi ”cần bao nhiêu chiều/tham số hiệu dụng để mạng hoạt động tốt”), và math 24 (Lazy Training, cũng dùng khái niệm ”số chiều hiệu dụng” của mô hình quanh điểm khởi tạo). Trong thư mục pruning, liên hệ trực tiếp ”pruning 08” (Lottery Ticket Hypothesis) và ”pruning 12” (state of pruning).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (bản arXiv); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc.

1. Bối cảnh

Với một mạng có D tham số, hàm mất mát có thể được hình dung như một mặt địa hình trên không gian D chiều: mỗi điểm $\theta \in \mathbb{R}^D$ tương ứng với một giá trị loss. Hình dung này dễ vẽ khi $D = 2$, nhưng trực giác từ không gian thấp chiều không nhất thiết còn đáng tin khi số tham số lên đến hàng nghìn hay hàng triệu.

Trước bài này, một số công trình đã chỉ ra các tính chất ”lạ” của landscape nhiều chiều: Dauphin et al. (2014) cho thấy các điểm tới hạn gần như luôn là saddle point (yên ngựa) chứ không phải valley (hõm cục bộ) như trực giác 2D hay gợi ý; Goodfellow et al. (2015) cho thấy đường đi thẳng từ điểm khởi tạo tới điểm tối ưu thường giảm loss một cách đơn điệu. Những quan sát này gợi ý rằng landscape nhiều chiều ”đơn giản” hơn ta nghĩ theo một số nghĩa.

Câu hỏi trung tâm của paper: một mạng có D tham số, nhưng thực chất cần bao nhiêu ”độ tự do” (degrees of freedom) để giải được một bài toán cụ thể? Số tham số D là thước đo thô (rough gauge) cho độ khó bài toán, nhưng paper muốn có một thước đo định lượng tốt hơn, áp dụng được cho cả supervised learning và reinforcement learning (RL), để so sánh độ khó giữa các bài toán khác nhau một cách công bằng.

Khái niệm cần nhớ: **random subspace** là một không gian con d chiều ($d < D$) được chọn ngẫu nhiên và cố định (không học) bên trong không gian tham số đầy đủ $\mathbb{R}^D$; **codimension** của một tập con $S \subset \mathbb{R}^D$ là D trừ số chiều của S (số chiều ”vuông góc” với S).

2. Ý tưởng chính

Paper định nghĩa **intrinsic dimension** d_{int} của một ”objective landscape” (gắn với một bài toán + mô hình cụ thể) là **codimension của tập nghiệm** (solution set) bên trong không gian tham số đầy đủ $\mathbb{R}^D$. Nếu gọi s là số chiều của tập nghiệm (bao nhiêu hướng di chuyển mà vẫn giữ nghiệm tốt), thì:

$$D = d_{int} + s \tag{1}$$

Ý tưởng đo d_{int} trong thực tế: thay vì huấn luyện trực tiếp trong $\mathbb{R}^D$ (gọi là **direct method**), ta huấn luyện trong một không gian con ngẫu nhiên d chiều của $\mathbb{R}^D$ (gọi là **subspace method**), tăng dần d, và ghi nhận giá trị d nhỏ nhất mà tại đó mô hình đạt hiệu năng đủ tốt (ngưỡng 90% so với baseline huấn luyện trực tiếp). Giá trị d đó được định nghĩa là d_{int90} (đo được, ước lượng của d_{int} lý thuyết).

Hai hệ quả bất ngờ mà bài báo nhấn mạnh:

- Nhiều bài toán có d_{int} **thấp hơn nhiều** so với D (ví dụ mạng fully-connected (FC) cho MNIST có $D = 199.210$ nhưng chỉ cần khoảng 750 chiều để đạt 90% hiệu năng), cho thấy landscape ”đơn giản” hơn số tham số gợi ý.
- Khi tăng D (thêm layer rộng hơn, sâu hơn) cho **cùng một bài toán**, d_{int} gần như **không đổi**: paper diễn giải điều này là ”mỗi tham số thêm vào chỉ làm tăng số chiều dư thừa s của tập nghiệm, không làm bài toán trở nên khó hơn hay dễ hơn về bản chất”. Đây là cách nhìn định lượng cho hiện tượng ”big models waste capacity” đã được quan sát từ trước (Dauphin & Bengio, 2013).

Phương pháp đo d_{int} cũng dẫn tới một hệ quả phụ: vì chỉ cần lưu d số thực (cộng 2 random seed) để tái tạo mạng, đây là một cách **nén mạng** đơn giản, và d_{int} cho một **chặn trên (upper bound) của Minimum Description Length (MDL)** của nghiệm.

3. Phương pháp chi tiết

3.1 Huấn luyện trong không gian con ngẫu nhiên

Gọi $\theta^{(D)} \in \mathbb{R}^D$ là vector tham số đầy đủ, $\theta_0^{(D)}$ là điểm khởi tạo ngẫu nhiên (ví dụ He init hay Glorot init), P là ma trận chiếu ngẫu nhiên $D \times d$ (cột xấp xỉ trực chuẩn, chuẩn hoá độ dài 1), và $\theta^{(d)} \in \mathbb{R}^d$ là tham số thực sự được học, khởi tạo bằng 0. Tham số đầy đủ dùng để forward/backward là:

$$\theta^{(D)} = \theta_0^{(D)} + P\,\theta^{(d)} \tag{2}$$

P và $\theta_0^{(D)}$ được sinh ngẫu nhiên và **đóng băng** (không cập nhật); chỉ $\theta^{(d)}$ được huấn luyện bằng gradient descent. Vì $\theta^{(d)}$ khởi tạo bằng 0, mạng bắt đầu đúng tại điểm khởi tạo ”chuẩn” $\theta_0^{(D)}$ (vẫn hưởng lợi từ các kỹ thuật init tốt).

Hai trường hợp biên: nếu $d = D$ và P là ma trận đơn vị, ta có đúng bài toán huấn luyện trực tiếp; nếu $d = D$ và P là ma trận xoay ngẫu nhiên, ta có bài toán xoay (với optimizer ”rotation-invariant” như SGD thì kết quả như cũ, nhưng với Adam/RMSProp thì có thể khác vì các optimizer này giả định các trục toạ độ có ý nghĩa riêng).

Ví dụ toy (chính paper dùng để minh hoạ, không phải tự tính): $D = 1000$, bài toán ép tổng 100 phần tử đầu bằng 1, 100 phần tử tiếp bằng 2, ..., tổng cộng 10 nhóm với 10 ràng buộc tổng. Tập nghiệm là siêu phẳng (hyperplane) 990 chiều (990 hướng di chuyển vẫn giữ cost bằng 0), nên theo công thức (1): $d_{int} = D - s = 1000 - 990 = 10$, khớp với số ràng buộc đặt ra. Khi quét d từ 0 lên, performance đạt 90% và 100% cùng tại $d = 10$.

Tự tính (minh hoạ nhỏ hơn, số khác với paper): xét một bài toán toy $D = 10$ với 2 ràng buộc tuyến tính độc lập (ví dụ 5 phần tử đầu phải tổng 1, 5 phần tử sau phải tổng 2). Theo công thức (1): $s = D - d_{int} = 10 - 2 = 8$ chiều dư thừa trong tập nghiệm, $d_{int} = 2$ (đúng bằng số ràng buộc). Đây chỉ là áp dụng công thức (1) cho một bộ số nhỏ hơn để kiểm tra trực giác, không phải số trong paper.

3.2 Định nghĩa d_{int90} và d_{int100}

Vì loss hiếm khi về đúng 0 với dữ liệu thật, paper dùng ngưỡng hiệu năng tương đối so với baseline (mô hình huấn luyện trực tiếp tốt nhất): d_{int100} là chiều tại đó hiệu năng ”không thể phân biệt thống kê” với baseline; d_{int90} là chiều tại đó hiệu năng đạt **tối thiểu 90%** của baseline. Paper nhận thấy d_{int100} dao động mạnh (có thể

gần D nếu baseline được tune rất kỹ, hoặc giảm mạnh nếu subspace training vô tình có tác dụng regularize), nên chọn d_{int90} làm thước đo chính, vì nó ổn định hơn với nhiễu huấn luyện. Ngưỡng 90% được chọn bằng quan sát thực nghiệm trên nhiều đồ thị performance-vs-d (paper lưu ý: ngưỡng khác đi thì kết luận định tính không đổi).

Thước đo hiệu năng: validation accuracy cho supervised classification; tổng reward đã dịch về không âm cho RL.

3.3 Ba phương pháp chiếu ngẫu nhiên (để mở rộng quy mô)

Khi D lớn (hàng triệu), việc lưu và nhân ma trận P dày đặc $D \times d$ tốn $O(Dd)$ thời gian và không gian, không khả thi. Paper dùng 3 cách:

1. **Dense**: P là ma trận chuẩn Gauss dày đặc, $O(Dd)$ thời gian/không gian.
2. **Sparse** (Li et al., 2006): mỗi phần tử khác 0 với xác suất $1/\sqrt{D}$, cho $O(\sqrt{Dd})$ thời gian/không gian (xấp xỉ, thực nghiệm chỉ nhanh hơn dense khoảng hằng số 10x do giới hạn cài đặt TensorFlow SparseTensor, không đạt đúng hệ số lý thuyết $\sqrt{D}$).
3. **Fastfood transform** (Le et al., 2013): phân tích $M = HG\Pi HB$ (Hadamard, Gauss chéo, hoán vị, dấu ngẫu nhiên), cho $O(D \log d)$ thời gian và $O(D)$ không gian, không cần lưu ma trận P hiển thị. Đây là phương pháp cho phép scale tới ImageNet/SqueezeNet ($D \approx 1,25$ triệu) và Atari Pong ($D \approx 1$ triệu).

Bảng S4 (đo thời gian forward+backward qua M, $d = 1\%$ của D): Dense tại $D = 10^5$ là 0,0169s, tại $D = 10^6$ là 1,0742s (ước lượng, hết nhớ ở $D = 6 \times 10^7$); Sparse tại $D = 10^5$ là 0,0002s, tại $D = 10^6$ là 0,0019s; Fastfood tại $D = 10^5$ là 0,0181s, tại $D = 10^6$ là 0,0195s, và vẫn chạy được ở $D = 6 \times 10^7$ (0,7949s), trong khi Dense/Sparse hết bộ nhớ ở quy mô đó.

4. Thí nghiệm và kết quả chính

Đây là paper thực nghiệm (không phải lý thuyết thuần); mục này tóm tắt các bảng/số liệu chính, lấy trực tiếp từ Bảng 1, Bảng S5 và các hình trong text.

MNIST (FC và LeNet). Mạng FC 784-200-200-10 ($D = 199.210$) đạt $d_{int90} \approx 750$ (chỉ dùng 0,4% số tham số để đạt 90% hiệu năng). LeNet ($D = 44.426$) đạt $d_{int90} = 290$ (hệ số nén khoảng 150 lần). Quét 20 mạng FC với độ sâu $\{1, ..., 5\}$ lớp ẩn và độ rộng $\{50, 100, 200, 400\}$: D thay đổi theo hệ số 24,1 lần nhưng d_{int90} chỉ thay đổi theo hệ số 1,33 lần (dùng baseline toàn cục 100%) hoặc 1,42 lần (dùng baseline riêng từng mô hình) **(Hình 3, Hình S7)**, minh chứng chính cho luận điểm "d_{int} ổn định khi D tăng".

MNIST với dữ liệu xáo trộn. Xáo trộn pixel (shuffled pixels): d_{int90} của FC không đổi (750, vì FC bất biến với hoán vị input), nhưng của LeNet tăng từ 290 lên 1400 (convnet mất lợi thế cấu trúc cục bộ). Xáo trộn label (shuffled labels, ghi nhớ nhãn ngẫu nhiên hoàn toàn trên toàn bộ 50.000 ảnh train, đo bằng training accuracy): $d_{int90} = 190.000$, tương đương 3,8 số thực/nhãn được ghi nhớ. Khi giảm cỡ tập ghi nhớ xuống 50% và 10%, d_{int90} giảm xuống 130.000 và 90.000, tức là **chi phí ghi nhớ mỗi nhãn tăng lên (5,2 và 18,0 số thực/nhãn)** khi tập dữ liệu nhỏ hơn, gợi ý có "chia sẻ hạ tầng" (shared infrastructure) giúp ghi nhớ hiệu quả hơn khi dữ liệu lớn.

CIFAR-10 và ImageNet. Theo Bảng 1 (văn bản chính): FC $D = 656.810 \rightarrow d_{int90} = 9000$; LeNet $D = 62.006 \rightarrow d_{int90} = 2900$; để đạt $> 50\%$ accuracy toàn cục, FC/LeNet/ResNet cần d_{int90} xấp xỉ 9k/2,9k/1k tương ứng, cho thấy **ResNet hiệu quả tham số hơn** FC và LeNet trên CIFAR-10. ImageNet (SqueezeNet, $D = 1.248.424$): do hạn chế thời gian/bộ nhớ, chỉ xác nhận được $d_{int90} > 500.000$ (chưa đo được giá trị chính xác; tại $d = 500k$ accuracy mới đạt 38,95%, dưới ngưỡng 90% của baseline 55,5%).

Reinforcement learning (ES và DQN). Dùng Evolution Strategies (ES): InvertedPendulum $d_{int100} = d_{int90} = 4$ (chỉ 4 tham số đủ giải bài toán!); Humanoid (đi bộ) $d_{int90} = 700$ (tương đương độ khó MNIST-FC,

thấp hơn nhiều so với trực giác ”đi bộ khó hơn phân loại ảnh”); Atari Pong (từ pixel) $d_{int90} = 6000$ (cùng bậc với CIFAR-10). Dùng DQN cho CartPole/Pole/Cart: $d_{int90} = 25/23/7$.

Bảng tổng hợp (Bảng 1, văn bản chính, trích các giá trị chính):

Bài toán	Mạng	D	d_{int90}
MNIST	FC	199.210	750
MNIST	LeNet	44.426	290
MNIST (shuffled pixel)	LeNet	44.426	1.400
MNIST (shuffled label, 100%)	FC	959.610	190.000
CIFAR-10	FC	656.810	9.000
CIFAR-10	LeNet	62.006	2.900
ImageNet	SqueezeNet	1.248.424	> 500.000
Inverted Pendulum	FC	562	4
Humanoid	FC	166.673	700
Atari Pong	ConvNet	1.005.974	6.000

Ablation kiến trúc convnet (Mục S10): so 4 biến thể LeNet (Standard, Untied weights, FC-tied kernel toàn ảnh, FC hoàn toàn) trên MNIST: $d_{int90} = 290, 600, 425, 2000$; trên CIFAR-10: $d_{int90} = 1000, 2750, 2500, 35000$. Kết luận: **cả weight-tying và local receptive field đều quan trọng**, loại bỏ một trong hai làm tăng d_{int90} đáng kể (tăng độ khó bài toán theo thước đo này).

5. Phân tích / ablation

Vai trò regularizer (Mục S8, Hình S15-S17). So sánh subspace training với weight decay (ℓ_2 penalty) và dropout trên FC ($L = 2, W = 200$) ở CIFAR-10: cả hai regularizer truyền thống đều giảm khoảng cách train-test accuracy; subspace training tự nó cũng có tác dụng regularize mạnh (đặc biệt khi d nhỏ). Khi $\ell_2 < 5 \times 10^{-4}$ hoặc dropout $< 0,1$ (tức ít regularize truyền thống), subspace training vượt direct training về test NLL, cho thấy nó có thể là một **regularizer thay thế** hiệu quả trong vùng ít regularize khác.

Ổn định huấn luyện mạng sâu (Mục S5.3, Hình S9). Với FC lên tới 10 lớp, SGD step 0,1, ReLU + He init: **direct training luôn thất bại khi độ sâu** $L > 5$ (nhiều lần thất bại ở $L = 4$, luôn thất bại ở $L > 4$), trong khi **subspace training ổn định ở mọi độ sâu được thử**. Paper giải thích: mỗi cơ sở ngẫu nhiên của P ”nhìn” qua toàn bộ D tham số trực tiếp, nên bài toán tối ưu trong không gian con có thể có điều kiện số (conditioning) tốt hơn hẳn so với không gian gốc (nơi các tham số có scale rất khác nhau theo layer).

Vai trò optimizer (Mục S5.4, Hình S10). So SGD (lr 0,1) và Adam (lr 0,001): Adam cho d_{int90} cao hơn SGD nói chung, vì Adam đạt baseline cao hơn (đặc biệt khi dùng baseline riêng từng mô hình).

Hai cách nhìn khác về d_{int} (Mục S5.1, S5.2, S9): d_{int} đo với baseline toàn cục khác với baseline riêng từng mô hình (số liệu lệch nhẹ: hệ số thay đổi 1,33 so với 1,42); với ImageNet, do hạn chế thời gian (mỗi lần đo tốn 6-7 ngày trên 4 GPU), kết luận chỉ là ”chặn dưới $> 500k$”, không có giá trị hội tụ.

6. Giới hạn

Bài báo nêu một số giới hạn: (a) phương pháp subspace training **không giúp tăng tốc inference** (khác với các phương pháp nén khác có kèm lợi ích tính toán khi dự đoán); (b) d_{int} chỉ là **chặn trên** của MDL thực sự, độ ”chặt” của chặn này chưa rõ, có thể khác nhau giữa các bài toán; (c) kết luận ”ResNet tốt hơn FC cho MNIST” (dựa trên d_{int} thấp hơn) là dễ đoán, nhưng với mô hình lớn/phức tạp/không đồng nhất hơn, kết luận từ d_{int} sẽ không hiển nhiên như vậy; (d) không nên khẳng định mọi FC-net trên MNIST đều có $d_{int} \approx 750$,

chỉ nên hiểu đó là một ”cao nguyên” (plateau) giá trị gần đúng cho một họ kiến trúc/siêu tham số cụ thể đã thử.

Hai điểm paper tự mâu thuẫn giữa văn bản chính và phụ lục (người ghi chú phát hiện khi đọc toàn văn, không phải bài báo nêu):

- Bảng 1 (văn bản chính) ghi CIFAR-10/FC có $D = 656.810$; Bảng S5 (phụ lục) lại ghi CIFAR-10/FC có $D = 1.055.610$ cho cùng cấu hình ”FC” ($d_{int90} = 9000$ ở cả hai bảng). Hai số D này khác nhau đáng kể (chênh khoảng 61%) mà không có giải thích trong text; có thể do khác kiến trúc FC cụ thể (số layer/width) giữa khi lập hai bảng, nhưng paper không nói rõ.
- Mục 3.2 (văn bản chính, Hình 2 caption) ghi MNIST/LeNet có $d_{int90} = 290$; Bảng S5 (phụ lục) lại ghi MNIST/LeNet có $d_{int90} = 275$. Chênh lệch nhỏ (15 đơn vị) nhưng là số khác nhau cho cùng một cấu hình được nêu ở hai nơi.

Hai điểm trên không ảnh hưởng tới kết luận định tính của paper (landscape thường có $d_{int} \ll D$, và d_{int} ổn định khi D tăng), nhưng người đọc kỹ số liệu nên biết có sự không khớp này giữa bảng chính và bảng phụ lục.

7. Liên hệ với các bài khác

- **math 23 (Neural Tangent Kernel)** và **math 22 (Deep NN as Gaussian Processes)**: cùng đặt câu hỏi ”mạng có bao nhiêu độ tự do hiệu dụng”, nhưng theo hướng lý thuyết infinite-width thay vì đo thực nghiệm bằng random subspace.
- **math 24 (On Lazy Training)**: dùng khái niệm gần giống ”số chiều hiệu dụng quanh điểm khởi tạo” (qua rank của $Dh(w_0)$ và độ lớn κ_h) để xác định khi nào mạng rơi vào chế độ tuyến tính hoá; bổ sung góc nhìn ”tĩnh” (landscape) của bài 19 bằng góc nhìn ”động” (quỹ đạo huấn luyện).
- **math 25 (Principles of Deep Learning Theory)**: khái niệm ”principle of sparsity” và tỉ lệ $r = L/n$ trong sách này cũng nói về việc mạng có nhiều tham số ”dư” theo nghĩa thống kê, cùng tinh thần với nhận xét ”extra parameters chỉ tăng s, không tăng d_{int}” của bài 19, dù công cụ toán khác hẳn (effective theory/RG flow so với đo thực nghiệm).
- **pruning 08 (Lottery Ticket Hypothesis)**: cùng tinh thần ”mạng dùng ít tham số hơn số D gốc vẫn giải được bài toán”, nhưng lottery ticket tìm **sub-network thưa** (một tập con trục toạ độ gốc), còn bài 19 tìm **subspace tuyến tính ngẫu nhiên** (tổ hợp tuyến tính của mọi tham số); đây là hai cách khác nhau để lượng hoá ”độ dư thừa tham số” của cùng một hiện tượng.
- **pruning 12 (state of neural network pruning)**: khảo sát các phương pháp đo độ dư thừa tham số/nén mạng nói chung, là bối cảnh rộng hơn cho kỹ thuật nén-bằng-subspace của bài 19 (paper 19 tự nhận chỉ là một ”cách nén đơn giản”, không cạnh tranh về tỉ lệ nén với các phương pháp group-sparsity như Louizos et al. 2017).

8. Gợi ý áp dụng

Intrinsic dimension được đo bằng cách tối ưu trong một không gian con ngẫu nhiên và xét kích thước cần để đạt một ngưỡng hiệu năng. Con số này phụ thuộc tác vụ, tham số hóa, phân phối không gian con và ngưỡng chọn; nó không phải một thuộc tính cố định của riêng kiến trúc.

9. Thí nghiệm gợi ý

1. **Đo d_{int90} cho một MLP nhỏ trên tập con MNIST** (ví dụ 5.000 ảnh, mạng 784-100-100-10) bằng random subspace training với P dense, quét $d \in \{10, 50, 100, ..., 1000\}$ trên GPU. Kỳ vọng: đường cong performance-vs-d dạng chữ S giống Hình 2 của paper, d_{int90} cỡ vài trăm. Diễn giải: xác nhận lại hiện tượng cơ bản của paper ở quy mô nhỏ, kiểm tra code trước khi áp dụng ý tưởng cho mô hình thật.

2. **So sánh d_{int90} của cùng một MLP ở 3 độ rộng khác nhau** (width 50/100/200, cùng 2 lớp ẩn) trên cùng tập dữ liệu. Đo: D tăng theo width nhưng d_{int90} có đổi nhiều không? Kỳ vọng: giống Hình 3 của paper, d_{int90} gần như không đổi dù D tăng vài lần. Diễn giải: nếu đúng, xác nhận ”extra width chỉ tăng độ dư thừa”; nếu sai nhiều, có thể do mạng/dữ liệu toy quá nhỏ để quan sát hiệu ứng.
3. **Đo d_{int90} khi fine-tune một adapter nhỏ (giống LoRA) trên một mô hình ngôn ngữ nhỏ (ví dụ GPT-2 nhỏ) cho một tác vụ phân loại văn bản đơn giản**, dùng chính kỹ thuật random subspace training của paper (chiếu ngẫu nhiên không gian tham số của adapter, không phải toàn mô hình, để tiết kiệm bộ nhớ). Đo: d_{int90} của adapter cho tác vụ đó là bao nhiêu so với rank LoRA thường chọn theo kinh nghiệm (ví dụ rank 8/16/32)? Diễn giải: nếu d_{int90} đo được nhỏ hơn rank thường dùng, gợi ý có thể giảm rank mà không mất hiệu năng; nếu lớn hơn, gợi ý rank hiện tại đang hạn chế hiệu năng.
4. **Thí nghiệm ghi nhớ nhãn ngẫu nhiên (shuffled label) trên tập con nhỏ** (ví dụ 1.000 ảnh MNIST, nhãn xáo trộn hoàn toàn), đo d_{int90} (theo training accuracy, không phải validation vì validation sẽ luôn ở mức ngẫu nhiên). Kỳ vọng: d_{int90} gần với số tham số cần để ghi nhớ hoàn toàn (xấp xỉ vài số thực mỗi nhãn, tăng khi tập nhỏ hơn, giống Bảng S2 của paper). Diễn giải: minh hoạ trực quan sự khác biệt giữa ”học” (generalization, d_{int} thấp) và ”nhớ” (memorization, d_{int} cao gần D).
5. **So sánh thời gian huấn luyện của 2 phương pháp chiếu (dense vs. sparse ngẫu nhiên, bỏ Fastfood vì phức tạp cài đặt hơn) trên một mạng CNN nhỏ** với D cỡ 10^5-10^6 trên GPU, đo thời gian mỗi bước forward+backward. Kỳ vọng: sparse nhanh hơn dense ở D lớn, giống xu hướng Bảng S4 của paper (có thể không đạt đúng hệ số lý thuyết $\sqrt{D}$ do giới hạn thư viện, giống paper tự ghi nhận). Diễn giải: hiểu chi phí thực tế của kỹ thuật này trước khi áp dụng cho mô hình lớn hơn.

10. Tóm tắt công thức / khái niệm

- **Intrinsic dimension qua codimension:** $D = d_{int} + s$ (1), với s là số chiều của tập nghiệm.
- **Huấn luyện trong subspace:** $\theta^{(D)} = \theta_0^{(D)} + P\,\theta^{(d)}$ (2), $P \in \mathbb{R}^{D\times d}$ cố định ngẫu nhiên, chỉ $\theta^{(d)}$ được học, khởi tạo $\theta^{(d)} = 0$.
- d_{int90} / d_{int100}: chiều subspace nhỏ nhất mà hiệu năng đạt tương ứng 90%/100% so với baseline trực tiếp (ngưỡng đo trên validation accuracy hoặc reward đã dịch chuẩn).
- **3 phương pháp chiếu và độ phức tạp:** Dense $O(Dd)$; Sparse (mật độ $1/\sqrt{D}$) lý thuyết $O(\sqrt{Dd})$; Fastfood $O(D \log d)$ thời gian, $O(D)$ không gian, dùng phân tích $M = HG\Pi HB$.
- **Hệ số nén (tự tính, theo ví dụ MNIST-FC của paper):** $D/d_{int90} = 199.210/750 \approx 265,6$ lần (paper nói ”260x”, khớp xấp xỉ với tự tính ở đây, chênh nhỏ do cách quy đổi byte/bit). Kích thước lưu trữ ước tính $d_{int90} \times 4$ byte (float32) $= 750 \times 4 = 3.000$ byte $\approx 2,93$ KB (paper ghi ”3,2kB”; tự tính ra 2,93KB, chênh nhỏ có thể do paper tính thêm vài byte cho 2 random seed hoặc quy đổi KB kiểu khác, không ảnh hưởng tới kết luận định tính về hệ số nén rất lớn).
- **MDL diễn giải:** d_{int} (đo bằng số bậc tự do, không phải bit) là chặn trên xây dựng được (constructive upper bound) cho Minimum Description Length của nghiệm, theo tinh thần Rissanen (1978) và Hinton & Van Camp (1993): với độ chính xác bằng nhau, mô hình MDL ngắn hơn được xem là ”tốt hơn” theo nguyên lý này.

11. Kết luận ngắn

Bài này đưa ra một định nghĩa đo được (random subspace training) cho ”intrinsic dimension” của objective landscape, cho thấy nhiều bài toán deep learning có d_{int} thấp hơn số tham số D rất nhiều, và d_{int} khá ổn định khi mô hình được mở rộng (width/depth) cho cùng một bài toán, tức tham số thêm vào chủ yếu làm tăng độ dư thừa của tập nghiệm chứ không thay đổi độ khó cốt lõi. Phát hiện phụ: d_{int} cho phép so sánh độ khó giữa supervised và RL trên cùng một thước đo, và mở ra một kỹ thuật nén mạng đơn giản (lưu seed + d số thực) dù không có lợi ích tốc độ suy luận. Người ghi chú phát hiện hai điểm số liệu không khớp giữa bảng chính và bảng phụ lục (xem mục 6), nên khi trích dẫn số cụ thể của paper này cần đối chiếu lại nguồn gốc bảng.

Nguồn gốc: https://arxiv.org/abs/1804.08838

Bài 20

Benefits of Depth in Neural Networks (2016)

Tác giả: Matus Telgarsky (University of Michigan, Ann Arbor; email ghi trong paper là @cs.ucsd.edu, giữ nguyên như bản in).
Venue / arXiv: arXiv:1602.04485 (v2, 27/05/2016); venue in trong paper: ”JMLR: Workshop and Conference Proceedings vol 49:1-23, 2016” (đây chính là kỷ yếu COLT 2016, nhưng paper chỉ in dòng JMLR W&CP; tên ”COLT 2016” là ”(ngoài paper, kiến thức chung)”).
Nguồn: https://arxiv.org/abs/1602.04485
Chủ đề: chứng minh tồn tại các hàm tính được bởi mạng ReLU rất sâu (Θ(k^3) lớp, Θ(1) node mỗi lớp) mà không mạng nào nông hơn (O(k) lớp) có thể xấp xỉ tốt, trừ khi mạng nông đó cực lớn (Ω(2^k) node); kết quả tổng quát cho một lớp gate rộng gọi là ”semi-algebraic gate” (ReLU, max, piecewise polynomial, decision tree).
Vị trí trong lộ trình: đọc cùng cặp với math 21 (Expressive Power of Neural Networks: A View from the Width) để so sánh hai câu hỏi đối ngẫu ”độ sâu giúp gì” và ”độ rộng giúp gì”. Nên đọc sau khi đã quen các khái niệm xấp xỉ hàm (không bắt buộc bài trước). Sau cặp 20-21, có thể đọc math 25 (Principles of Deep Learning Theory) vì bài đó định nghĩa tỉ số khía cạnh (aspect ratio) L/n (depth-to-width) làm trục trung tâm, một cách nhìn định lượng hơn cho câu hỏi ”sâu bao nhiêu, rộng bao nhiêu là đủ” mà bài 20/21 đặt ra dưới dạng định lý tồn tại/không tồn tại.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (bản arXiv); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc.

1. Bối cảnh

Mạng nơ-ron có thể được mô tả bằng một đồ thị có hướng không chu trình: mỗi nút nhận các giá trị thực, tính một hàm rồi truyền kết quả cho các nút tiếp theo. Trong phân tích của bài, độ sâu là số lớp phép toán được ghép liên tiếp; một số mô hình khác như SVM hay boosted decision stump tương ứng với mạng có số lớp cố định.

Định lý xấp xỉ tổng quát cổ điển (universal approximation, Cybenko 1989, Hornik et al. 1989, không trích dẫn chi tiết trong paper này nhưng được nhắc tới gián tiếp qua các công trình liên quan) nói rằng mạng có độ sâu cố định nhỏ (ví dụ 1 lớp ẩn) nhưng **đủ rộng** đã có thể xấp xỉ bất kỳ hàm liên tục trên miền compact. Nhưng độ rộng cần thiết có thể tăng theo cấp số nhân với số chiều đầu vào. Câu hỏi đặt ra: tăng độ sâu có giúp mạng biểu diễn được các hàm ”khó” với kích thước (số node, số tham số) nhỏ hơn hẳn so với mạng nông không?

Công trình gần nhất trước đó, Eldan và Shamir (2015), chỉ ra sự tách biệt giữa mạng 2 lớp và 3 lớp (trường hợp riêng k=2) khi số chiều đầu vào d lớn: có hàm 3 lớp không thể xấp xỉ bởi mạng 2 lớp subexponential-size. Paper này (Telgarsky 2016) tổng quát hoá cho **mọi k nguyên dương** và cho **nhiều loại gate** (không chỉ ReLU chuẩn), đồng thời làm việc với d cố định bất kỳ (bao gồm d=1), một chế độ mà các công trình trước không xử lý triệt để.

Các khái niệm cần nắm trước khi đọc mục 2-3:

- **ReLU**: $\sigma_R(z) := \max\{0, z\}$.
- **Semi-algebraic gate** (t, α, β)-sa: một hàm $f : \mathbb{R}^k \to \mathbb{R}$ được ghép từ t đa thức bậc $\leq \alpha$ (dùng làm ”ngưỡng” chia vùng) và các đa thức bậc $\leq \beta$ trên mỗi vùng. Khung này gộp được ReLU, max/min-gate, piecewise polynomial, và decision tree vào một định nghĩa duy nhất.

- **Khoảng cách L1**: sai lệch điểm $|f(x) - g(x)|$ lấy trung bình (tích phân) trên khối $[0,1]^d$.

2. Ý tưởng chính

Kết quả trung tâm là **Định lý 1.1**: với mọi số nguyên $k \geq 1$ và số chiều $d \geq 1$, tồn tại hàm $f : \mathbb{R}^d \to \mathbb{R}$ tính được bởi một mạng ReLU chuẩn có $2k^3 + 8$ lớp, $3k^3 + 12$ node, và $4 + d$ tham số riêng biệt (một tham số có thể xuất hiện ở nhiều cạnh), sao cho

$$\inf_{g \in \mathcal{C}} \int_{[0,1]^d} |f(x) - g(x)|\, dx \geq \frac{1}{64},$$

trong đó $\mathcal{C}$ là hợp của (a) các hàm tính bởi mạng (t, α, β)-semi-algebraic với $\leq k$ lớp và $\leq 2^k/(t\alpha\beta)$ node, và (b) các hàm là tổ hợp tuyến tính của $\leq t$ decision tree, mỗi cây $\leq 2^{k^3}/t$ node.

Nói cách khác: có một hàm cần $\Theta(k^3)$ lớp nhưng chỉ $\Theta(1)$ node mỗi lớp để tính chính xác, mà bất kỳ mạng nông hơn ($\leq k$ lớp, tức là giảm độ sâu đi hệ số k^2) muốn xấp xỉ nó trong sai số L1 nhỏ hơn 1/64 đều phải có kích thước **lớn theo cấp số nhân** ($\Omega(2^k)$ node, hoặc với decision tree thì $\Omega(2^{k^3})$ tổng số node cây, một phân biệt mạnh hơn).

Ý tưởng chứng minh dựa trên quan sát về **dao động (oscillation)**: ghép (compose) các hàm "sóng tam giác" (triangle wave) qua nhiều lớp làm số lần hàm cắt ngang ngưỡng 1/2 tăng theo **cấp số nhân** với số lớp, còn việc chỉ thêm node trong một lớp (addition, không tăng độ sâu) chỉ làm số lần cắt tăng theo cấp số nhân với độ sâu hiện có (bị chặn bởi một hàm e mũ theo depth, chứ không mũ theo chính depth đó lần nữa) - nói đơn giản: composition "thắng" addition trong việc tạo dao động, giống cách ghép đa thức (degree nhân nhau) tạo nhiều nghiệm hơn cộng đa thức (degree chỉ lấy max).

Paper cũng đưa ra **kết quả đồng hành** (companion, Định lý 1.2 và Lemma 1.3): số lượng hàm "khó" như Định lý 1.1 không nhiều - mạng $\Theta(k^3)$ lớp, dù biểu diễn được hàm dao động mạnh, có **VC dimension** bị chặn khá chặt, nên trên n điểm dữ liệu với nhãn ngẫu nhiên đồng xác suất, mạng đó cũng chỉ fit đúng được khoảng một nửa nhãn (sai số $\geq 1/4$) khi n đủ lớn so với số tham số. Điều này cho thấy "mạng sâu mạnh" không có nghĩa là "mạng sâu fit được mọi nhãn ngẫu nhiên".

3. Phương pháp chi tiết

3.1 Semi-algebraic gate và khung ký hiệu mạng

Định nghĩa 2.1: $f : \mathbb{R}^k \to \mathbb{R}$ là (t, α, β)-sa nếu có t đa thức $(q_i)_{i=1}^t$ bậc $\leq \alpha$ và m bộ ba (U_j, L_j, p_j) với p_j bậc $\leq \beta$, sao cho $f(v) = \sum_j p_j(v) \prod_{i \in L_j} \mathbb{1}[q_i(v) < 0] \prod_{i \in U_j} \mathbb{1}[q_i(v) \geq 0]$. Số hạng m không ảnh hưởng tới các chặn độ phức tạp trong bài, nên không xuất hiện trong tên gọi.

Các trường hợp riêng quan trọng (Lemma 2.3, đã kiểm trong text):

- ReLU chuẩn $v \mapsto \sigma_R(\langle a, v\rangle + b)$ là $(1, 1, 1)$-sa.
- max/min gate trên r đa thức bậc $\leq \alpha$ là $(r(r-1), \alpha, \alpha)$-sa.
- Decision tree k node là $(k, 1, 0)$-sa; boosted decision tree t cây mỗi cây $\leq k$ node là $(tk, 1, 0)$.

Ký hiệu mạng: lớp 0 là input (d node, node i là $x \mapsto x_i$); $\mathcal{N}_d((m_i, t_i, \alpha_i, \beta_i)_{i=1}^l)$ là lớp hàm tính bởi mạng có l lớp (không tính lớp 0), lớp i có $\leq m_i$ node (t_i, α_i, β_i)-sa.

3.2 Đếm dao động (crossing number)

Với hàm 1 biến $f : \mathbb{R} \to \mathbb{R}$, đặt $\tilde{f}(x) := \mathbb{1}[f(x) \geq 1/2]$ và **crossing number** $\mathrm{Cr}(f) := |I_f|$ với I_f là phân hoạch $\mathbb{R}$ thành các đoạn mà $\tilde{f}$ không đổi (ví dụ $\mathrm{Cr}(\sigma_R) = 2$).

Lemma 3.1 (xấp xỉ qua đếm dao động): nếu $\mathrm{Cr}(f) \gg \mathrm{Cr}(g)$ thì

$$\frac{1}{\mathrm{Cr}(f)} \sum_{U \in I_f} \mathbb{1}[\forall x \in U,\ \tilde{f}(x) \neq \tilde{g}(x)] \ \geq\ \frac{1}{2}\Big(1 - \frac{2\,\mathrm{Cr}(g)}{\mathrm{Cr}(f)}\Big).$$

Ý nghĩa: nếu g dao động ít hơn f rất nhiều, thì trên phần lớn các đoạn mà $\tilde{f}$ dao động, $\tilde{g}$ không theo kịp nên khác dấu với $\tilde{f}$, dẫn tới sai số xấp xỉ lớn.

Lemma 3.2 (ít lớp, ít dao động - mấu chốt của "benefits of depth"): với h là phép affine $\mathbb{R} \to \mathbb{R}^d$ và $f \in \mathcal{N}_d((m_i, t_i, \alpha_i, \beta_i)_{i=1}^{l})$, đặt $\alpha = \max_i \alpha_i$, $\beta = \max_i \beta_i$, $t = \max_i t_i$, $m = \sum_i m_i$:

$$\mathrm{Cr}(f \circ h) \leq 2\Big(\frac{2tm\alpha}{l}\Big)^{l} \beta^{l^2}.$$

Đây là **điểm mấu chốt**: số lớp l nằm ở **số mũ**, còn số node m nằm ở **cơ số**. Muốn tăng crossing number theo cấp số nhân mà chỉ tăng m (số node, giữ l cố định) thì cần m tăng theo cấp số nhân; nhưng nếu tăng l (độ sâu) thì crossing number tăng theo cấp số nhân của l với cơ số nhỏ.

Việc chứng minh Lemma 3.2 dựa trên một chuỗi bổ đề về việc ghép hàm semi-algebraic với hàm piecewise-polynomial (Lemma 3.3-3.6): hợp các phân hoạch của k hàm $(t, \cdot)$-poly thành một phân hoạch mịn hơn có $\leq kt$ đoạn (Lemma 3.4), từ đó suy luận quy nạp theo lớp rằng node ở lớp i là $(2^i T_i A_i C_{i-1} M_{i-1}, B_i)$-poly với $T_i, A_i, \ldots$ là tích lũy các $t_j, \alpha_j, \ldots$ của các lớp trước.

3.3 Xây dựng hàm nhiều dao động: sóng tam giác

Hàm $f(z) := \sigma_R(2\sigma_R(z) - 4\sigma_R(z - 1/2)) = \min\{\sigma_R(2z), \sigma_R(2 - 2z)\}$ (Lemma 3.10) là một mạng ReLU **1 lớp, 2 node** ($\in \mathcal{N}_1(2, 1, 1, 1, 1)$), đồ thị là một tam giác từ $(0, 0)$ lên $(1/2, 1)$ xuống $(1, 0)$.

Định nghĩa "triangle" (3.7): f là $(t, [0, 1])$-triangle nếu liên tục, có thể chia $[0, 1]$ thành $2t$ đoạn xen kẽ tăng/giảm, giá trị tại các mốc bằng 0 hoặc 1 xen kẽ.

Lemma 3.8: nếu f là $(s, [0, 1])$-triangle và g là $(t, [0, 1])$-triangle thì $f \circ g$ là $(2st, [0, 1])$-triangle. Ghép (composition) nhân số "đỉnh" lại.

Corollary 3.9: với $f \in \mathcal{N}_1(m, l, t, \alpha, \beta)$ là $(t, [0, 1])$-triangle, thì f^k (ghép k lần) $\in \mathcal{N}_1(m, kl, t, \alpha, \beta)$ là $(2^{k-1}t^k, [0, 1])$-triangle với $\mathrm{Cr}(f^k) = (2t)^k + 1$.

Áp dụng cho f ở trên ($t = 1$): f^k chỉ cần k lớp (vì f gốc 1 lớp) nhưng có $\mathrm{Cr}(f^k) = 2^k + 1$ - **dao động tăng theo cấp số nhân với số lớp**, đúng như trực giác composition-thắng-addition ở mục 2.

3.4 Chứng minh Định lý 1.1 (phác thảo đã kiểm trong text)

Đặt $f_0(z) := h^{k^3+4}(z)$ với h là hàm tam giác ReLU ở trên; theo Corollary 3.9, $\mathrm{Cr}(f_0) = 2^{k^3+4} + 1$. Định nghĩa $f(x) = f_0(x_1)$ (chỉ phụ thuộc toạ độ đầu).

Với mạng nông g (semi-algebraic, $\leq k$ lớp, $m \leq 2^k/(t\alpha\beta)$ node), áp Lemma 3.2 cho $g \circ p_y$ (với $p_y(z) := (z, y)$ chiếu lên 1 chiều):

$$\mathrm{Cr}(g \circ p_y) \leq 2\Big(\frac{2tm\alpha}{k}\Big)^{k^2} \beta^{k^3} \leq 4(tm\alpha\beta)^{k^2} \leq 2^{k^3+2}$$

(nếu g là (t, k)-bdt thay vào đó thì $\mathrm{Cr}(g \circ p_y) \leq 2t\, 2^{k^3}/t \leq 2^{k^3+2}$ tương tự).

Áp Lemma 3.1 với $\mathrm{Cr}(f) = 2^{k^3+4} + 1 \gg \mathrm{Cr}(g) \leq 2^{k^3+2}$, kết hợp với tính "đều" (regularity) của hàm tam giác ReLU để chuyển từ bất đẳng thức trên tập chỉ số sang tích phân L1 thật, paper suy ra sai số $\geq 1/64$ (chi tiết số học nằm ở việc chia hằng số 1/16, 1/32 qua các bước).

Ví dụ tự tính (minh hoạ, $k = 2$, không phải số trong bảng paper vì paper không có bảng số cho Định lý 1.1): kiến trúc mạng sâu có $2k^3 + 8 = 24$ lớp, $3k^3 + 12 = 36$ node, $4 + d$ tham số (lấy $d = 1$: 5 tham số). Hàm f_0 tương ứng với ghép $k^3 + 4 = 12$ lần, crossing number $\mathrm{Cr}(f_0) = 2^{12} + 1 = 4097$ (tự tính). Mạng nông bị hạn chế $\leq k = 2$ lớp, giả sử $t = \alpha = \beta = 1$ (ReLU gate thuần) thì $m \leq 2^2/1 = 4$ node mỗi phía, và theo công thức trên $\mathrm{Cr}(g \circ p_y) \leq 2^{k^3+2} = 2^{10} = 1024$ (tự tính). Tỉ số $\mathrm{Cr}(f)/\mathrm{Cr}(g) \approx 4097/1024 \approx 4$ (tự tính): mạng nông dao động kém hơn mạng sâu khoảng 4 lần ngay ở $k = 2$, và khoảng cách này tăng theo cấp số nhân với k (ở số mũ k^3) - minh hoạ trực quan cho "benefits of depth", dù phép tính bằng tay này chỉ mang tính minh hoạ cơ chế, không phải số liệu paper công bố.

3.5 Hạn chế dao động: VC dimension (Mục 4 của paper)

Lemma 4.2: mạng $\leq p$ tham số, $\leq l$ lớp, $\leq m$ node (t, α, β)-sa có VC $\leq 6p(l+1)\big(\ln(2p(l+1)) + \ln(8emt\alpha) + l\ln\beta\big)$.

Lemma 4.1 (kiểu Gilbert-Varshamov qua bất đẳng thức Hoeffding): với n điểm và nhãn ngẫu nhiên đều, xác suất $\geq 1 - \delta$ là mọi f trong lớp hàm có growth function $\mathrm{Sh}(\mathcal{F}; n)$ đều fit sai $\geq \frac{1}{2}\big(1 - \sqrt{(\ln \mathrm{Sh}(\mathcal{F}; n) + \ln(1/\delta))/(2n)}\big)$ tỉ lệ nhãn.

Kết hợp hai bổ đề này (Định lý 1.2): với $n \geq 8pl^2 \ln(8emt\alpha\beta p(l+1)) + 4\ln(1/\delta)$ điểm, xác suất $\geq 1 - \delta$, mạng $\Theta(k^3)$-lớp ở Định lý 1.1 fit sai $\geq 1/4$ tỉ lệ nhãn ngẫu nhiên trên $\Theta(k^9)$ điểm - dù mạng này biểu diễn được hàm dao động cực mạnh, nó **không** fit được nhãn ngẫu nhiên tốt hơn một mạng nhỏ nhiều.

4. Thí nghiệm và kết quả chính

Đây là paper lý thuyết túy; **không có thí nghiệm thực nghiệm số nào** trong toàn văn (đã đọc hết Mục 1-5 và Phụ lục A, không có hình minh hoạ chạy số, không có bảng số liệu huấn luyện). "Kết quả chính" ở đây là các định lý toán học:

Định lý	Nội dung	Loại chặn
1.1	Hàm $\Theta(k^3)$-lớp, $\Theta(1)$-node/lớp không xấp xỉ được (sai L1 $\geq 1/64$) bởi mạng $\leq k$ lớp trừ khi $\Omega(2^k)$ node (hoặc $\Omega(2^{k^3})$ node cây cho boosted tree)	tồn tại (lower bound cho depth efficiency)
1.2 / Lemma 1.3	VC dimension mạng semi-algebraic $O(pl\log(\cdot))$	chặn trên, hạn chế khả năng fit nhãn ngẫu nhiên
3.12	Tổng quát 1.1 cho đo rời rạc/liên tục khác, hằng số sai số khác (1/32, 1/8...)	mở rộng kỹ thuật
Corollary A.1	Tồn tại đa thức bậc 2^k không xấp xỉ được bởi đa thức bậc $\leq 2^{k-3}$ (sai $\geq 1/32$)	hệ quả phụ, liên hệ lý thuyết đa thức cổ điển (Weierstrass, Kolmogorov, Warren)

Vì không có benchmark thực nghiệm, Mục 4 của ghi chú này chỉ tóm tắt các định lý/chặn như trên theo đúng yêu cầu brief cho paper lý thuyết.

5. Phân tích / ablation

Paper không có ablation theo nghĩa thực nghiệm; dưới đây là phân tích cấu trúc của các chặn, dựa hoàn toàn trên nội dung đã đọc:

- **Khoảng giữa depth và width trong chặn chính**: hàm khó cần $\Theta(k^3)$ lớp để xấp xỉ bởi mạng k lớp cần $\Omega(2^k)$ node - đây là **tách biệt theo hệ số đa thức bậc 3 về độ sâu** đổi lấy **tách biệt theo cấp số nhân về kích thước**. Tác giả không claim đây là chặn chặt tối ưu (ví dụ không chứng minh được việc giảm độ sâu đi 1 đơn vị, từ $k+1$ xuống k, đã đòi hỏi cỡ lớn theo cấp số nhân; chặn chỉ áp dụng khi giảm độ sâu đi hệ số k^2, từ $\Theta(k^3)$ xuống $O(k)$).

- **Vai trò của decision tree**: với boosted decision tree, chặn mạnh hơn ($\Omega(2^{k^3})$ tổng số node cây so với $\Omega(2^k)$ cho mạng neural nông) - paper lý giải vì cấu trúc cây ”cộng tuyến tính nhiều cây nông” vốn đã yếu hơn khi so composition.
- **Vai trò của chiều dữ liệu** d: Định lý 1.1 đúng với d cố định bất kỳ (bao gồm $d = 1$), khác với Eldan-Shamir (2015) cần d lớn. Đây là điểm bài báo nhấn mạnh là cải tiến kỹ thuật (Mục 5, ”Bibliographic notes”).
- **Bổ đề đếm dao động là công cụ 1 chiều**: toàn bộ chứng minh quy về hàm 1 biến (sau khi chiếu qua affine map p_y) rồi lấy trung bình trên các hướng chiếu còn lại $y \in \mathbb{R}^{d-1}$ - kỹ thuật ”random projection” mà bài báo lưu ý là ”rất thô sơ” (extremely primitive form) so với kỹ thuật dùng trong depth-hierarchy cho mạch boolean (Håstad 1986; Rossman et al. 2015).
- **Lemma 1.3/4.2 không làm yếu Định lý 1.1**: VC dimension nhỏ của mạng sâu không mâu thuẫn với khả năng biểu diễn hàm dao động mạnh của nó, vì VC dimension đo khả năng fit nhãn tùy ý trên **n điểm cho trước**, còn Định lý 1.1 nói về một hàm **cụ thể** (không phải nhãn ngẫu nhiên) cần nhiều lớp để xấp xỉ.

6. Giới hạn

- Chỉ là **kết quả tồn tại** (existence): paper chứng minh có một hàm cụ thể khó xấp xỉ, không mô tả được ”có bao nhiêu” hàm như vậy hay chúng có xuất hiện trong các hàm mục tiêu thực tế (dữ liệu ảnh, văn bản...) hay không. Tác giả tự nêu trong Mục 5: hướng mở là đặc trưng hoá **tất cả** các hàm khó xấp xỉ bởi độ sâu nhỏ, không chỉ một ví dụ.
- Hàm xây dựng (sóng tam giác lặp) có cấu trúc rất đặc biệt (dao động đều, tần số tăng dần theo cấp số nhân) - không có bằng chứng hàm như vậy xuất hiện tự nhiên trong các bài toán học máy thực tế.
- Chặn chỉ so sánh $\Theta(k^3)$ lớp với $O(k)$ lớp (một khoảng cách bậc 3 theo k, không phải $k + 1$ so với k); do đó không trả lời được câu hỏi ”giảm đúng 1 lớp có luôn cần tăng kích thước theo cấp số nhân không” - đây là giới hạn mà Eldan-Shamir (2015) với trường hợp $k = 2$ (so 2 và 3 lớp) từng trả lời được cho trường hợp riêng khi d lớn, nhưng paper 20 không mở rộng điều đó cho k tổng quát.
- VC dimension bound (Định lý 1.2/Lemma 4.2) dùng kỹ thuật của Warren (1968) cho đa thức, chỉ là ”điều chỉnh nhỏ” (minor departure) theo lời tác giả, không tối ưu hoá riêng cho semi-algebraic gate.
- Không có thí nghiệm số nào để kiểm chứng trực giác (khác với bài 21 có bảng thực nghiệm).

7. Liên hệ với các bài khác

- **math 21** (The Expressive Power of Neural Networks: A View from the Width): câu hỏi đối ngẫu của chính bài này - thay vì hỏi ”sâu giúp gì” thì hỏi ”rộng giúp gì”. Bài 21 trích dẫn trực tiếp bài 20 (”Benefits of depth in neural networks (Telgarsky, 2016)”) làm nền depth-efficiency để so sánh với width-efficiency họ chứng minh (đã kiểm: trong references của 1709.02540 có ”[14] Matus Telgarsky. Benefits of depth in neural networks. COLT 2016”).
- **math 22 / math 23** (Deep Neural Networks as Gaussian Processes; Neural Tangent Kernel): nhìn mạng neural theo hướng hoàn toàn khác - không phải ”mạng nào biểu diễn được hàm nào” mà là hành vi thống kê của mạng **rất rộng** (width $\to \infty$) tại khởi tạo và trong huấn luyện. Khái niệm ”rộng” trong 2 bài đó là lợi thế (đưa về Gaussian process/kernel), trái với tinh thần bài 20 là ”sâu mới là lợi thế biểu diễn” - một tương phản thú vị giữa lý thuyết xấp xỉ (approximation theory, bài 20/21) và lý thuyết huấn luyện (training theory, bài 22/23).
- **math 25** (The Principles of Deep Learning Theory): định nghĩa tỉ số khía cạnh $r = L/n$ (depth-to-width) làm biến điều khiển chính của lý thuyết hiệu dụng (effective theory); khi $r \to 0$ mạng ứng xử như vô hạn rộng (mất khả năng học biểu diễn), khi $r \gg 1$ mạng ”hỗn loạn” (chaotic, không tính được). Bài 20 góp một mảnh ghép định tính cho phía ”tăng L (độ sâu)” của tỉ số này từ góc độ biểu diễn hàm, chứ không phải từ góc độ huấn luyện như math 25.

- **pruning content/32_2024_unreasonable_ineffectiveness_deeper_layers.md**: thực nghiệm cho thấy nhiều lớp sâu trong LLM hiện đại có thể bị xoá mà độ chính xác giảm ít - một quan sát thực nghiệm dường như ”mâu thuẫn” với trực giác lý thuyết của bài 20 (sâu hơn luôn biểu diễn được hàm khó hơn); cách dung hoà hợp lý (suy luận của người ghi chú, không phải trong 2 paper): bài 20 chỉ nói có **tồn tại** hàm cần nhiều lớp, không nói **mọi** lớp của **mọi** mạng đã huấn luyện đều cần thiết cho hàm mục tiêu thực tế mà mạng đó đang biểu diễn.
- **pruning content/38_2023_sheared_llama.md**: pruning theo chiều sâu (loại bỏ layer) là một trong các chiều cắt được dùng thực tế; bài 20 cho trực giác lý thuyết (yếu) về việc giảm độ sâu có thể ”cắt cầu” một số hàm dao động mạnh mà mô hình đã học, cần đền bù bằng việc tăng mạnh kích thước các lớp còn lại.

8. Gợi ý áp dụng

Các kết quả về lợi ích của độ sâu cho thấy có những hàm biểu diễn hiệu quả hơn bằng mạng sâu trong lớp hàm được nghiên cứu. Chúng không chứng minh rằng tăng độ sâu luôn cải thiện huấn luyện hay tổng quát hóa trên mọi tác vụ.

9. Thí nghiệm gợi ý

1. **Tái dựng hiệu ứng dao động theo độ sâu (toy, trên GPU, thậm chí chạy CPU)**: xây mạng ReLU 1 chiều ghép k lần hàm tam giác $f(z) = \min\{2z, 2 - 2z\}$ (PyTorch, vài dòng), vẽ đồ thị f^k với $k = 1, ..., 6$, đếm số lần cắt ngang 0.5 bằng code, so với công thức $\text{Cr}(f^k) = 2^k + 1$ (kỳ vọng khớp đúng - nếu không khớp, kiểm tra lại điều kiện biên).
2. **Đo sai số xấp xỉ mạng nông vs mạng sâu cùng số tham số**: cố định tổng số tham số (ví dụ 200), chia cho 2 mạng: (a) sâu (nhiều lớp, ít node/lớp) ghép hàm tam giác nhiều lần, (b) nông (ít lớp, nhiều node/lớp), train mạng nông để fit lại hàm f^k của mạng sâu bằng MSE loss, đo sai số L1 trên $[0, 1]$. Kỳ vọng: sai số mạng nông tăng nhanh khi k tăng, minh hoạ Định lý 1.1 trong môi trường có train (không phải existence proof) - một phép kiểm định kinh nghiệm, không phải chứng minh lại định lý.
3. **Depth-pruning một mạng ReLU nhỏ đã huấn luyện fit hàm dao động**: huấn luyện mạng 12 lớp fit $f^6(z)$ (hàm tam giác ghép 6 lần) trên $[0, 1]$, sau đó xoá dần từng layer (giữ nguyên các trọng số còn lại, có thể fine-tune nhẹ), đo lại sai số MSE theo số layer còn lại. So sánh với cắt rộng (giảm số node mỗi lớp xuống một nửa, giữ đủ layer). Kỳ vọng: cắt sâu cho hàm dao động nhiều này sẽ giảm chất lượng nhanh hơn cắt rộng tương đương về số tham số, đúng trực giác Mục 8 (nhưng đây là hàm tổng hợp đặc biệt, không đại diện cho toàn bộ không gian hàm mục tiêu thực tế).
4. **Kiểm chứng VC dimension/khả năng fit nhãn ngẫu nhiên**: lấy mạng ReLU 12 lớp nhỏ (≤ 50 tham số), gán nhãn ngẫu nhiên nhị phân cho 200 điểm ngẫu nhiên trong $[0, 1]$, train tới khi hội tụ (nếu có thể), đo train accuracy. Kỳ vọng theo Định lý 1.2: với n điểm đủ lớn so với tham số, train accuracy khó vượt quá khoảng 75% (sai $\geq 1/4$) dù mạng có thể biểu diễn hàm dao động rất mạnh - minh hoạ rằng biểu diễn được hàm khó không đồng nghĩa fit được nhãn ngẫu nhiên tùy ý.

10. Tóm tắt công thức / khái niệm

Khái niệm/công thức	Ý nghĩa
$\sigma_R(z) = \max\{0, z\}$	ReLU
(t, α, β)-sa	semi-algebraic gate: t đa thức ngưỡng bậc $\leq \alpha$, giá trị là đa thức bậc $\leq \beta$ trên mỗi vùng
$\text{Cr}(f) = \lvert I_f \rvert$	crossing number: số đoạn mà $\tilde{f} = \mathbb{1}[f \geq 1/2]$ không đổi
Lemma 3.1	$\text{Cr}(f) \gg \text{Cr}(g) \Rightarrow$ sai số xấp xỉ $\geq \frac{1}{2}(1 - 2\text{Cr}(g)/\text{Cr}(f))$ (dạng rời rạc)
Lemma 3.2	$\text{Cr}(f \circ h) \leq 2(2tm\alpha/l)^l \beta^{l^2}$: số lớp l ở số mũ, số node m ở cơ số

Khái niệm/công thức	Ý nghĩa
$(t, [0,1])$-triangle, Cor. 3.9	ghép k lần hàm tam giác t-đỉnh cho $\mathrm{Cr}(f^k) = (2t)^k + 1$
Định lý 1.1	$\exists f$: $2k^3{+}8$ lớp, $3k^3{+}12$ node, $4{+}d$ tham số, không mạng $\leq k$ lớp và $o(2^k)$ node nào xấp xỉ sai $< 1/64$
Định lý 1.2 / Lemma 1.3	$\mathrm{VC}(\mathcal{N}(G)) \leq 6p(l{+}1)\big(\ln(2p(l{+}1)) + \ln(8emt\alpha) + l\ln\beta\big)$
Corollary A.1	tồn tại đa thức bậc 2^k không xấp xỉ được bởi đa thức bậc $\leq 2^{k-3}$, sai $\geq 1/32$
Ví dụ tự tính ($k = 2$)	$\mathrm{Cr}(f_0) = 2^{12} + 1 = 4097$; mạng nông ≤ 2 lớp: $\mathrm{Cr}(g) \leq 2^{10} = 1024$; tỉ số ≈ 4

11. Kết luận ngắn

Bài báo chứng minh bằng công cụ ”đếm dao động” (crossing number) rằng có những hàm ReLU cần số lớp tăng theo k^3 nhưng số node mỗi lớp chỉ là hằng số, trong khi bất kỳ mạng nông hơn ($\leq k$ lớp, hoặc boosted decision tree tương đương) muốn xấp xỉ gần đúng hàm đó đều cần kích thước tăng theo cấp số nhân với k. Cơ chế cốt lõi: composition (ghép lớp) nhân dao động theo cấp số nhân, còn addition (thêm node) chỉ cộng dao động theo cấp số nhân với **độ sâu sẵn có** - số lớp nằm ở số mũ của chặn trên dao động (Lemma 3.2), đó là lý do toán học cho ”benefits of depth”. Đồng thời, paper cũng chỉ ra (qua VC dimension) rằng khả năng biểu diễn hàm dao động mạnh không đồng nghĩa với khả năng fit nhãn ngẫu nhiên tùy ý. Đây là kết quả tồn tại (existence), không phải đặc trưng hoá đầy đủ, và không có thực nghiệm số - phần thực nghiệm kiểm tra trực giác tương tự (có train) được bài 21 (math 21) thực hiện cho câu hỏi đối ngẫu về độ rộng.

Nguồn gốc: https://arxiv.org/abs/1602.04485

Bài 21

The Expressive Power of Neural Networks: A View from the Width (2017)

Tác giả: Zhou Lu, Hongming Pu, Feicheng Wang, Zhiqiang Hu, Liwei Wang (Peking University: Department of Mathematics; Key Laboratory of Machine Perception, MOE, School of EECS; Center for Data Science, Beijing Institute of Big Data Research).
Venue / arXiv: arXiv:1709.02540 (v3, 01/11/2017); venue in trong paper: ”31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.”
Nguồn: https://arxiv.org/abs/1709.02540
Chủ đề: câu hỏi đối ngẫu của ”benefits of depth”: độ rộng (width) của mạng ReLU ảnh hưởng thế nào tới khả năng biểu diễn hàm; chứng minh định lý xấp xỉ tổng quát cho mạng width-bounded, một ”phase transition” ở width = n (số chiều đầu vào), và một chặn dưới đa thức (không phải mũ) cho ”width efficiency”.
Vị trí trong lộ trình: đọc cùng cặp với math 20 (Benefits of Depth, Telgarsky 2016) mà bài này trích dẫn trực tiếp làm nền so sánh depth-efficiency; đọc trước khi vào các bài infinite-width (math 22 Gaussian Process, math 23 NTK) để thấy rằng ”rộng” ở đây (xấp xỉ hàm, width hữu hạn nhỏ) và ”rộng” ở các bài sau (width vô hạn, hành vi thống kê khi khởi tạo) là hai khái niệm khác hẳn nhau dù dùng chung từ ”width”.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (bản arXiv); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc.

1. Bối cảnh

Định lý xấp xỉ tổng quát cổ điển (Cybenko 1989; Hornik, Stinchcombe và White 1989; Funahashi 1989; Barron 1994) cho biết mạng depth-2 có một lớp ẩn, hàm kích hoạt phù hợp và độ rộng đủ lớn có thể xấp xỉ một hàm liên tục trên miền compact với sai số tùy ý. Tuy nhiên, độ rộng cần thiết có thể tăng theo cấp số nhân với số chiều đầu vào. Vì vậy, ngoài khả năng xấp xỉ, cần xét kích thước mạng cần dùng để đạt độ chính xác mong muốn.

Hướng nghiên cứu ”depth efficiency” (độ sâu giúp giảm kích thước) đã có nhiều kết quả: Eldan và Shamir (2015) cho trường hợp riêng 2 vs 3 lớp khi chiều dữ liệu lớn; Cohen, Sharir, Shashua (2016, dùng phân tích tensor) cho mạng tích chập; và **Telgarsky (2016)** (= math 20 trong thư mục này, trích dẫn trực tiếp là tài liệu tham khảo [14]) chứng minh tồn tại mạng $O(k^3)$ lớp không thể xấp xỉ bởi mạng $O(k)$ lớp trừ khi kích thước mũ 2^k.

Paper này đặt câu hỏi **đối ngẫu**: độ **rộng** (width) có vai trò gì cho khả năng biểu diễn? Nếu tráo vai trò độ sâu và độ rộng trong các định lý depth-efficiency mà vẫn đúng, thì width có ”sức mạnh” tương đương depth về mặt biểu diễn. Paper nghiên cứu mạng **fully-connected ReLU**, với **depth** h = số lớp (tính cả output, không tính input) và **width** d_m = số node tối đa trong một lớp; số chiều đầu vào là n.

2. Ý tưởng chính

Ba kết quả chính trong Mục 3–4 của bài báo là:

Định lý 1 (Universal Approximation cho mạng width-bounded): với mọi hàm khả tích Lebesgue $f : \mathbb{R}^n \to \mathbb{R}$ và mọi $\varepsilon > 0$, tồn tại mạng ReLU fully-connected **width** $\leq n + 4$ (độ sâu không hạn chế) sao cho $\int_{\mathbb{R}^n} |f(x) - F_{\mathcal{A}}(x)|\, dx < \varepsilon$. Đây là bản đối ngẫu của universal approximation cổ điển: thay vì cố định depth=2 và cho width tùy ý lớn, bài này cố định width nhỏ ($n + 4$, không phụ thuộc ε) và cho depth tùy ý lớn.

Định lý 2 và 3 (phase transition tại width = n): Định lý 2 nói rằng, trừ một tập đo Lebesgue bằng 0, **không** hàm khả tích nào có thể được xấp xỉ bởi mạng width $\leq n$ trên toàn $\mathbb{R}^n$ (sai số chỉ có thể là 0 hoặc $+\infty$, không có "gần đúng"). Định lý 3 làm chặt hơn trên miền bị chặn $[-1,1]^n$: với hàm liên tục không hằng theo bất kỳ hướng nào, mạng width $\leq n-1$ có sai số dưới chặn dương $\varepsilon^* > 0$ cố định (không giảm được dù mạng sâu bao nhiêu). Hai định lý này cho thấy **width = n là ngưỡng chuyển pha**: trên ngưỡng (n+4) xấp xỉ được tùy ý tốt, dưới ngưỡng (n-1 hoặc n) thì không bao giờ xấp xỉ tốt được dù tăng độ sâu vô hạn.

Định lý 4 (width efficiency, chặn dưới đa thức): với số nguyên $k \geq n+4$, tồn tại mạng rộng $F_{\mathcal{A}}$ (width $2k^2$, depth 3) sao cho **không** mạng hẹp $F_{\mathcal{B}}$ nào (width $\leq k^{1.5}$, depth $\leq k+2$, tham số bị chặn trong $[-b,b]$) có thể đạt $\int (F_{\mathcal{A}} - F_{\mathcal{B}})^2\, dx < \epsilon$ cho một $\epsilon > 0$ nào đó. Đây là chặn dưới cho "width efficiency" - nhưng **chỉ là đa thức** ($k^{1.5}$ so với k^2), khác hẳn chặn **mũ** (2^k) của depth efficiency (bài 20). Tác giả kết luận (lời paper, Mục 5): "kết quả cung cấp bằng chứng toàn diện hơn rằng độ sâu có thể hiệu quả hơn độ rộng cho tính biểu diễn của mạng ReLU" - nhưng đây là so sánh một chặn dưới mũ (depth) với một chặn dưới đa thức (width) đã chứng minh được, **không phải** chứng minh width-efficiency chỉ có thể là đa thức (chặn trên mũ cho width chưa ai chứng minh, và chặn trên đa thức cũng chưa chứng minh - đây là open problem tác giả tự nêu).

Paper còn tiến hành **thực nghiệm** (Mục 4.1) để gợi ý rằng chặn dưới đa thức của Định lý 4 có thể gần với chặn trên thật (tức là mạng hẹp hơn một chút so với chặn dưới đã đủ xấp xỉ tốt mạng rộng) - xem Mục 4 của ghi chú này.

3. Phương pháp chi tiết

3.1 Khung ký hiệu

Mạng ReLU fully-connected: depth h (số lớp tính cả output, không tính input), width d_m (số node tối đa 1 lớp), input dimension n. $F_{k,\mathcal{A}}$ là hàm của lớp k, $R_{k,\mathcal{A}}$ là hàm sau ReLU của k lớp đầu, $R_{0,\mathcal{A}}$ là lớp input.

3.2 Chứng minh Định lý 1 (phác thảo đã kiểm trong Phụ lục A.1)

Ý tưởng 3 bước:

1. Với f khả tích Lebesgue, chọn N đủ lớn để phần "đuôi" $|x_i| \geq N$ có tích phân $|f|$ nhỏ (bỏ qua $< \varepsilon/2$); tách $f = f_1 - f_2$ (phần dương/âm trên khối $E = [-N,N]^n$); phủ vùng dưới đồ thị f_1, f_2 bằng hữu hạn khối $(n+1)$-chiều (Lebesgue covering), quy hàm về **tổng có trọng số của các hàm chỉ báo (indicator) trên khối n-chiều**.
2. Xây một **block** (khối) mạng ReLU width $n+4$, depth $4n+1$, gọi là **Single ReLU Unit (SRU)**, có khả năng: (a) "chặt" (chop) từng chiều của một khối n-chiều để tạo ra hàm chỉ báo xấp xỉ trên khối đó, (b) lưu lại (nhớ) tổng các khối đã xử lý trước, (c) cộng dồn kết quả hiện tại vào bộ nhớ. Trong $n+4$ node mỗi lớp: n node "truyền" tọa độ input không đổi suốt mạng, 2 node lưu phần dương/âm tích lũy, 2 node còn lại dùng để "chặt" dần từng chiều ($k = 1,\ldots,n$) bằng hàm bậc thang trơn hóa qua tham số δ nhỏ (xem công thức $R_{1,n+3,A_k}$, $R_{4,n+3,A_k}$ trong Phụ lục A.1 - đây chính là cách chuyển một "lát cắt" hộp n-chiều thành một dãy ReLU 4 lớp lặp n lần).
3. Ghép (concatenate) các block cho từng khối indicator, dùng 2 node nhớ để cộng dồn; chọn δ đủ nhỏ để sai số tổng các bước cộng lại nhỏ hơn ε (các hằng số $1/2, 1/4, 1/8, 3/4$ được paper phân bổ cụ thể qua các bất đẳng thức (7)-(26) trong Phụ lục A.1).

3.3 Chứng minh Định lý 2 và 3 (lập luận hình học, Phụ lục A.2-A.3)

Ý tưởng mấu chốt khi $d_m \leq n$ (ma trận trọng số lớp đầu là ánh xạ $\mathbb{R}^n \to \mathbb{R}^m$ với $m < n$): ánh xạ tuyến tính này **không đơn (not injective)**, nên tồn tại hướng $x_0 \neq 0$ sao cho mọi điểm dịch theo x_0 cho cùng giá trị lớp 1, do đó hàm toàn mạng **hằng số dọc theo tia** x_0 (Lemma trong A.3). Nếu hàm bị chặn tích phân hữu hạn trên toàn $\mathbb{R}^n$ (Định lý 2) mà lại hằng số không đổi trên một tia vô hạn với giá trị khác 0, tích phân trên tia đó đã là $+\infty$ (mâu thuẫn) - nên hàm phải bằng 0 trên tia đó; lập luận quy nạp theo từng lớp (Lemma 1-3 trong A.2, dùng tính Lipschitz của hàm ReLU-mạng và định lý hội tụ bị chặn - Dominated Convergence Theorem)

mở rộng điều này ra toàn miền, suy ra hàm $\equiv 0$ nếu tích phân hữu hạn - tức **không xấp xỉ được** hàm khác 0 có tích phân hữu hạn khác 0, trừ khi hàm đó cũng gần 0 (tập đo 0 các trường hợp biên).

Định lý 3 (miền bị chặn) dùng lập luận tương tự nhưng đơn giản hơn: nếu $d_m \leq n-1$, tồn tại hướng x_0 cố định (không phụ thuộc input cụ thể, chỉ phụ thuộc tham số mạng) mà hàm hằng số dọc x_0; với f liên tục không hằng theo hướng đó trên $[-1,1]^n$, khoảng cách L1 giữa f và bất kỳ hàm hằng-theo-x_0 có chặn dưới dương ε^* xác định bởi tính liên tục đều (uniform continuity) của f trên miền compact.

3.4 Chứng minh Định lý 4 (lập luận đo độ - measure zero, Phụ lục A.4)

Với $n=1$: chọn $2k^4$ điểm rời rạc cụ thể $x^{(1)},\ldots,x^{(2k^4)}$; định nghĩa tập E_0 các vector giá trị "hợp lệ" (Lemma 4) sao cho **mọi** vector trong E_0 đều có thể được một mạng rộng (width $2k^2$, depth 3) biểu diễn đúng tại các điểm đó - tức $E_0 \subset E_w$ (tập giá trị khả đạt của mạng rộng). Mặt khác (Lemma 5), ánh xạ từ không gian tham số của mạng hẹp (width $\leq k^{1.5}$, depth $\leq k+2$, có **số chiều tự do** $= d^2(h-2)+d(h-1)+2d+1 \leq k^4+k^3 < 2k^4$) sang không gian giá trị $\mathbb{R}^{2k^4}$ có **miền ảnh đo-0** theo định lý biến đổi Jacobian (vì chiều miền xác định nhỏ hơn chiều không gian đích $2k^4$). Do E_0 có đo dương, E_0 giao với miền ảnh đó là tập con "bỏ qua được" (negligible) trong E_0 - nghĩa là hầu hết hàm trong lớp mạng rộng **không thể** khớp đúng bởi mạng hẹp tại các điểm đã chọn. Lemma 6 dùng tính liên tục + compact (dãy con hội tụ, định lý hội tụ bị chặn) để chuyển từ "không khớp đúng tại điểm rời rạc" sang "có khoảng cách dương $\epsilon_j > 0$ đều" cho MỖI hình dạng mạng hẹp cụ thể, từ đó lấy $\epsilon = \min_j \epsilon_j$ trên tập hữu hạn các hình dạng khả dĩ để có hằng số chung. Mở rộng từ $n=1$ sang $n>1$: cho mạng rộng chỉ phụ thuộc x_1 (trọng số từ $x_2,\ldots,x_n$ gán 0), lập luận tương tự áp dụng trực tiếp.

4. Thí nghiệm và kết quả chính

Khác với bài 20 (không có thực nghiệm), bài này **có** một thực nghiệm thật (Mục 4.1), nhằm kiểm tra liệu chặn dưới đa thức của Định lý 4 có "gần" chặn trên thật không.

Thiết lập: với $n \in \{1,2\}$, $k \in \{3,4,5\}$: mạng mục tiêu (target network) là mạng ReLU ngẫu nhiên, input dim n, output dim 1, **width** $2k^2$**, depth 3** (trọng số lấy theo chuẩn tắc standard normal, bias theo uniform $[-1,1)$). Mạng xấp xỉ (approximator network) có **width** $3k^{1.5}$**, depth** $k+2$ (kiến trúc theo đúng tỉ lệ nêu trong Định lý 4), huấn luyện bằng mini-batch **AdaDelta, learning rate 1.0**, 100 epoch (với $n=1$) hoặc 200 epoch ($n=2$), trên 20000 điểm input ($n=1$) hoặc 40000 điểm ($n=2$) lấy đều trong $[-1,1)^n$ (một nửa làm train, đo sai số trên phần còn lại theo mô tả trong Mục 4.1 - "half of all the test inputs... constitute the training set"), lặp lại 50 lần lấy mẫu tham số mạng mục tiêu, báo cáo MSE (mean square error) trường hợp xấu nhất (worst) và trung bình (average).

Bảng 1 (nguyên văn số liệu trong paper):

n	k	width mục tiêu	depth mục tiêu	width xấp xỉ	depth xấp xỉ	MSE xấu nhất	MSE trung bình
1	3	18	3	16	5	0.002248	0.000345
1	4	36	3	24	6	0.003263	0.000892
1	5	50	3	34	7	0.005643	0.001296
2	3	18	3	16	5	0.008729	0.001990
2	4	36	3	24	6	0.018852	0.006251
2	5	50	3	34	7	0.030114	0.007984

Kiểm tra khớp công thức (tự tính): width mục tiêu $2k^2$: $k=3 \to 2\cdot 9 = 18$ ✓, $k=4 \to 2\cdot 16 = 32$... khoan, paper ghi 36 cho $k=4$: $2k^2 = 2\cdot 16 = 32 \neq 36$. Kiểm tra lại: có thể $2k^2$ với $k=4$ nên là 32, nhưng bảng paper in "36" (đã đọc trực tiếp từ bảng, dòng "1 4 36 3 24 6..."). Đây là **điểm paper tự mâu thuẫn giữa**

công thức Định lý 4 ($2k^2$) và số liệu bảng (ghi rõ ở Mục 6 "Giới hạn" dưới đây, không tự sửa). Với $k = 3$: $2k^2 = 18$ khớp; $k = 5$: $2k^2 = 50$ khớp; chỉ riêng dòng $k = 4$ cho 36 là lệch so với $2 \cdot 4^2 = 32$ (có thể lỗi đánh máy trong bảng gốc, hoặc công thức áp dụng khác cho $k = 4$ - không đủ thông tin trong text để kết luận chắc).

Kiểm tra width xấp xỉ $3k^{1.5}$ (tự tính): $k = 3$: $3 \times 3^{1.5} = 3 \times 5.196 = 15.59 \approx 16$ (paper ghi 16, khớp làm tròn lên); $k = 4$: $3 \times 4^{1.5} = 3 \times 8 = 24$ (khớp đúng, paper ghi 24); $k = 5$: $3 \times 5^{1.5} = 3 \times 11.18 = 33.54 \approx 34$ (paper ghi 34, khớp làm tròn lên). Depth xấp xỉ $k + 2$: $3 \to 5$, $4 \to 6$, $5 \to 7$ - khớp đúng cả 3.

Kết luận thực nghiệm (lời paper): sai số MSE rất nhỏ (từ 3×10^{-4} đến 3×10^{-2}) trong khi giá trị hàm mục tiêu dao động với biên độ ~ 10 (paper tự ghi), nên sai số tuyệt đối này được xem là nhỏ một cách hợp lý; kết quả "cho thấy một gợi ý rằng độ sâu lớn hơn theo tỉ lệ đa thức có thể đủ để mạng hẹp xấp xỉ mạng rộng" - đây **chỉ là gợi ý thực nghiệm trên vài cấu hình nhỏ ($n \leq 2$, $k \leq 5$)**, không phải chứng minh chặn trên đa thức (vẫn là open problem theo lời paper).

5. Phân tích / ablation

Paper không có ablation nghiêm ngặt (không thử các tỉ lệ width/depth khác với công thức Định lý 4), nhưng có thể phân tích các điểm sau từ chính nội dung:

- **"Khoảng trống" giữa Định lý 1 và Định lý 2/3**: Định lý 1 chứng minh xấp xỉ được với width $n + 4$; Định lý 2/3 chứng minh **không** xấp xỉ được với width $\leq n$ (toàn miền) hoặc $\leq n - 1$ (miền bị chặn). Vùng width $\in \{n, n+1, n+2, n+3\}$ (toàn miền) hoặc $\{n\}$ (miền bị chặn) **chưa được xác định** trong paper - đây không phải lỗi mà là khoảng chưa giải quyết, paper không claim đã có chặn chặt khít (tight).
- **Đối xứng kỹ thuật nhưng không đối xứng kết luận**: cấu trúc chứng minh Định lý 1 (dùng khối SRU) là bản "lật ngược vai trò" khá trực tiếp của universal approximation cổ điển (ở đó depth=2, width lớn; ở đây width=$n + 4$, depth lớn) - nhưng Định lý 4 (width efficiency) chỉ đạt chặn **đa thức**, không đối xứng với chặn **mũ** của depth efficiency (bài 20). Bản thân paper thừa nhận (Mục 4) điều này "không loại trừ khả năng có chặn mũ cho width efficiency" - nghĩa là kết luận "depth hiệu quả hơn width" của paper dựa trên **so sánh chặn dưới đã biết, không phải chặn dưới chặt nhất có thể**.
- **Vai trò thực nghiệm**: vì chặn trên đa thức không chứng minh được bằng lý thuyết, thực nghiệm Mục 4.1 được dùng như "bằng chứng gián tiếp" (indirect evidence) - đây là điểm khác biệt về chuẩn mực chứng minh giữa phần lý thuyết (chặt, định lý) và phần này (gợi ý, không tổng quát hoá được).
- **Lemma 2 (trong A.2) có phát biểu "ngược đời"**: nếu mạng hẹp ($d_m \leq n$) thoả mãn tích phân toàn miền hữu hạn (Assumption 1) thì hàm mạng đó **phải bằng 0 khắp nơi** - tác giả tự ghi chú "kết luận này có thể lạ lúc đầu vì $F_{\mathcal{A}}$ giống hàm tuyến tính" (trích dẫn gần nguyên văn từ text), nhấn mạnh rằng kết luận chỉ đúng dưới giả thiết tích phân hữu hạn - một cách khác để nói "hàm khác 0 có tích phân hữu hạn thì không thể biểu diễn bởi mạng width $\leq n$".

6. Giới hạn

- **Mâu thuẫn số liệu trong Bảng 1** đã nêu ở Mục 4: dòng $k = 4$ ghi width mục tiêu = 36 nhưng công thức Định lý 4 là $2k^2 = 32$; không rõ là lỗi đánh máy của paper hay cách tính khác, paper không giải thích - ghi nhận ở đây, không tự sửa theo brief.
- Định lý 1 dùng khoảng cách **L1** trên hàm **khả tích Lebesgue toàn không gian** $\mathbb{R}^n$, khác định lý xấp xỉ cổ điển dùng **L∞** trên hàm **liên tục** miền **compact** - hai phát biểu không hoàn toàn so sánh ngang hàng được dù cùng gọi "universal approximation" (bài báo nêu rõ sự khác biệt này ở cuối Mục 3).
- Định lý 4 chỉ xây dựng **một họ cụ thể** (random weight target network cấu hình cố định), không phải đặc trưng hoá mọi hàm khó xấp xỉ bởi mạng hẹp.
- Thực nghiệm chỉ chạy $n \in \{1, 2\}$, $k \in \{3, 4, 5\}$ - quy mô rất nhỏ, không có thực nghiệm trên dữ liệu hoặc kiến trúc thực tế (CNN, Transformer).

- Chặn trên đa thức (narrow network chỉ cần tăng depth đa thức để xấp xỉ wide network) và chặn dưới mũ cho width efficiency đều là **open problem** tự paper nêu (Mục 4, cuối): "Currently, neither the exponential lower bound nor the polynomial upper bound seems within the reach."
- Không bàn tới hàm kích hoạt khác ReLU, không bàn tới kiến trúc không fully-connected (CNN, attention).

7. Liên hệ với các bài khác

- **math 20** (Benefits of Depth in Neural Networks, Telgarsky 2016): nền tảng depth-efficiency mà bài này trích dẫn trực tiếp (reference [14]) để đối chiếu; cùng dùng khái niệm "efficiency" (kích thước cần tăng bao nhiêu khi giảm chiều kia) nhưng đo bằng công cụ khác (bài 20: crossing number/oscillation; bài 21: đo độ Lebesgue trên không gian tham số).
- **math 22 / math 23** (Deep Neural Networks as Gaussian Processes; Neural Tangent Kernel): khái niệm "width" ở hai bài này là **width** $\to \infty$ theo nghĩa thống kê tại khởi tạo/huấn luyện (không phải width hữu hạn nhỏ như $n + 4$ ở đây); hai hướng nghiên cứu (approximation theory của width hữu hạn vs. infinite-width statistics) phát triển khá độc lập trong lý thuyết deep learning, dù cùng dùng từ "width".
- **math 25** (The Principles of Deep Learning Theory): dùng tỉ số $r = L/n$ (depth-to-width) làm biến trung tâm; bài 21 góp phần định tính cho "mẫu số" n của tỉ số đó từ góc độ xấp xỉ hàm hữu hạn-width, trong khi math 25 phân tích $n \to \infty$ với L cố định rồi mở rộng nhiễu loạn theo $1/n$.
- **pruning content/32_2024_unreasonable_ineffectiveness_deeper_layers.md**: quan sát thực nghiệm rằng LLM hiện đại chịu được xoá khá nhiều layer sâu; liên hệ (suy luận của người ghi chú): nếu width của mỗi layer còn lại trong các LLM thực tế đã rất lớn hơn ngưỡng phase-transition n (số chiều input/hidden dimension) của bài 21, thì mạng còn nhiều "dư thừa" width để hấp thụ việc giảm depth, khớp với quan sát hiện tượng loại bỏ layer ít ảnh hưởng.
- **pruning content/20_2023_depgraph.md**: framework cắt theo đồ thị phụ thuộc giữa các thành phần mạng (có thể cắt kênh/width một cách có cấu trúc); bài 21 cho cảnh báo lý thuyết (yếu, trên mạng ReLU đơn giản) rằng cắt width xuống dưới ngưỡng n (số chiều input cục bộ của một khối) có nguy cơ gây "sập" khả năng biểu diễn chứ không suy giảm từ từ - một điều nên lưu ý khi tự động hoá quyết định tỉ lệ cắt theo width.

8. Gợi ý áp dụng

Các định lý về độ rộng làm rõ rằng khả năng xấp xỉ phụ thuộc cả kiến trúc, hàm kích hoạt lẫn miền xét. Ngưỡng chứng minh cho mạng ReLU fully-connected không tự chuyển thành ngưỡng thực hành cho kiến trúc khác.

9. Thí nghiệm gợi ý

1. **Tái dựng phase transition của Định lý 2/3 (toy, CPU hoặc 1 GPU nhỏ)**: với $n = 3$, huấn luyện mạng ReLU fit một hàm mục tiêu cố định (ví dụ mạng ReLU ngẫu nhiên khác, width 20 depth 3) bằng mạng xấp xỉ có width quét qua $\{1, 2, 3, 4, 5, 6, 7\}$ (quanh ngưỡng $n = 3$), depth cố định lớn (ví dụ 10), đo MSE. Kỳ vọng: MSE giảm mạnh ("rơi vực") khi width vượt qua $n = 3$ sang $n + 1 = 4$ trở lên, minh hoạ trực quan Định lý 2/3 (lưu ý: đây không phải chứng minh lại định lý, vì định lý nói về sai số **0 hoặc vô hạn trên toàn không gian**, còn thực nghiệm đo trên miền bị chặn hữu hạn điểm).
2. **Tái lập đúng Bảng 1 ở quy mô nhỏ**: dựng target network width $2k^2$ depth 3 cho $k = 3$ (width 18), approximator width $3k^{1.5} \approx 16$ depth $k + 2 = 5$, train bằng Adam hoặc AdaDelta lr nhỏ, so khớp MSE với số liệu paper (0.002248 worst, 0.000345 average cho $n = 1$). Nếu số liệu lệch nhiều, kiểm tra lại cách sinh tham số target (standard normal weight, uniform $[-1, 1)$ bias) và cách chia train/test.

10. Tóm tắt công thức / khái niệm

Khái niệm/công thức	Ý nghĩa
depth h, width d_m, input dim n	số lớp (không tính input), số node tối đa/lớp, số chiều đầu vào
Định lý 1	$\forall f$ khả tích Lebesgue, $\exists$ mạng ReLU width $\leq n+4$ xấp xỉ L1 sai $< \varepsilon$ tùy ý
Định lý 2	width $\leq n$: trừ tập đo 0, sai số L1 trên $\mathbb{R}^n$ chỉ là 0 hoặc $+\infty$
Định lý 3	miền $[-1,1]^n$, width $\leq n-1$: sai số L1 $\geq \varepsilon^* > 0$ cố định với hàm không hằng theo hướng nào
Định lý 4	$\exists$ mạng rộng (width $2k^2$, depth 3) không xấp xỉ L2 được bởi mạng hẹp (width $\leq k^{1.5}$, depth $\leq k+2$)
SRU (Single ReLU Unit)	khối ReLU width $n+4$ depth $4n+1$ dùng để ”chặt” + ”nhớ” + ”cộng” khi dựng chứng minh Định lý 1
Bảng 1 (tự tính kiểm khớp)	width xấp xỉ $\approx 3k^{1.5}$: $k=3\to 16$, $k=4\to 24$, $k=5\to 34$; depth xấp xỉ $= k+2$
MSE Bảng 1, $n=1, k=3$	worst 0.002248, average 0.000345 (AdaDelta lr=1.0, 100 epoch, 20000 điểm)
Số chiều tự do mạng hẹp	$d^2(h-2)+d(h-1)+2d+1$ (dùng trong Lemma 5, lập luận đo-0)

11. Kết luận ngắn

Bài báo mở ra hướng ”width efficiency” đối ngẫu với ”depth efficiency” (bài 20): chứng minh mạng ReLU width $n+4$ (depth tùy ý) là universal approximator theo L1, trong khi width $\leq n$ (toàn miền) hoặc $\leq n-1$ (miền bị chặn) gần như không bao giờ xấp xỉ tốt - một **phase transition rõ rệt** quanh width = số chiều đầu vào. Về hiệu quả định lượng, chặn dưới cho width efficiency (Định lý 4) chỉ là **đa thức** ($k^{1.5}$ so k^2), yếu hơn hẳn chặn **mũ** của depth efficiency, và thực nghiệm nhỏ (Mục 4.1, với một điểm số liệu không khớp công thức ở $k=4$ cần lưu ý) gợi ý depth tăng đa thức có thể đủ bù width hẹp. Cả chặn trên đa thức cho width và chặn dưới mũ cho width đều là bài toán mở. Kết luận thực hành của paper: depth và width đều quan trọng và cần tinh chỉnh cùng nhau, không nên chỉ tối ưu một chiều.

Nguồn gốc: https://arxiv.org/abs/1709.02540

Bài 22

Deep Neural Networks as Gaussian Processes (2017)

Tác giả: Jaehoon Lee, Yasaman Bahri (hai tác giả đóng góp ngang nhau), Roman Novak, Samuel S. Schoenholz, Jeffrey Pennington, Jascha Sohl-Dickstein (Google Brain, Google AI Residency Program)
Venue / arXiv: ”Published as a conference paper at ICLR 2018”; bản đọc arXiv:1711.00165v3 (3/3/2018)
Nguồn: arXiv:1711.00165
Chủ đề: Lý thuyết mạng nơ-ron vô hạn rộng (infinite-width), tương ứng với Gaussian Process, suy luận Bayes cho deep learning
Vị trí trong lộ trình: math 22/35 trong ”math content/”, cùng cặp lý thuyết NN vô hạn rộng với math 23 (Neural Tangent Kernel)

> Ghi chú này là bản tóm tắt, diễn giải lại theo cách hiểu của người viết sau khi đọc trực tiếp bài báo gốc, không phải bản dịch nguyên văn.

1. Bối cảnh

Neal (1994) chỉ ra rằng một mạng nơ-ron có một lớp ẩn hội tụ theo luật về Gaussian process (GP) khi độ rộng tiến tới vô hạn và trọng số, bias được khởi tạo Gaussian độc lập cùng phân phối. Lee, Bahri và đồng nghiệp mở rộng câu hỏi này sang mạng có nhiều lớp ẩn.

Bối cảnh cụ thể: xét mạng fully-connected (FC) có L lớp ẩn, độ rộng (số unit) mỗi lớp ẩn là $N_1, \dots, N_L$, input $x \in \mathbb{R}^{d_{in}}$, output $z^L \in \mathbb{R}^{d_{out}}$. Pre-activation và post-activation của lớp l là z_i^l và $x_i^l = \phi(z_i^l)$ với ϕ là hàm phi tuyến. Trọng số W_{ij}^l i.i.d. zero-mean, variance σ_w^2/N_l; bias b_i^l i.i.d. zero-mean, variance σ_b^2. Câu hỏi: khi $N_1, \dots, N_{L-1} \to \infty$ (lấy giới hạn tuần tự, lớp này rồi lớp kia), các output function $z_i^L(\cdot)$ có còn là GP không, và nếu có, kernel (hàm hiệp phương sai) của GP đó là gì?

Động lực thực tế: nếu trả lời được câu hỏi trên, ta có thể làm suy luận Bayes ”đúng” (closed-form, không cần SGD) cho một họ mạng deep rất rộng, và so sánh trực tiếp hiệu năng của suy luận Bayes này với mạng hữu hạn huấn luyện thông thường.

2. Ý tưởng chính

Ý tưởng cốt lõi gồm 3 phần:

1. **Tương ứng NN vô hạn rộng vs. GP (NNGP):** bằng cách áp dụng Định lý giới hạn trung tâm (CLT) lặp lại qua từng lớp (lấy $N_1 \to \infty$, rồi $N_2 \to \infty$, ...), output của mạng deep FC hội tụ về một GP có kernel $K^L(x, x')$ được tính **đệ quy** từ lớp này sang lớp kia, chỉ phụ thuộc vào $\sigma_w^2, \sigma_b^2, \phi$ và kernel của lớp trước.
2. **Suy luận Bayes đóng (closed-form):** một khi đã có kernel K^L, bài toán ”huấn luyện” mạng vô hạn rộng này trở thành bài toán GP regression chuẩn: mean và covariance dự đoán tại điểm test có công thức đại số tuyến tính rõ ràng, không cần gradient descent.
3. **Thực nghiệm:** so sánh NNGP (suy luận Bayes đúng) với mạng FC hữu hạn huấn luyện bằng Adam trên MNIST/CIFAR-10, và liên hệ hiệu năng NNGP với lý thuyết signal propagation (ranh giới ”critical line” giữa các pha ordered/chaotic hoặc bounded/unbounded).

Điểm thú vị nhất: NNGP không học biểu diễn (representation learning) theo cách người ta thường gán công cho sức mạnh của deep learning, nó dùng một kernel cố định (fixed basis function). Vậy mà nó thường **không hề thua**, thậm chí vượt mạng SGD-trained ở nhiều kích thước tập huấn luyện.

3. Phương pháp chi tiết

3.1 Trường hợp 1 lớp ẩn (ôn lại, mục 2.2 của bài)

Với $L = 1$: $z_i^1(x) = b_i^1 + \sum_j W_{ij}^1 x_j^1(x)$. Vì các số hạng là i.i.d. và tổng trên $N_1 \to \infty$ số hạng, theo CLT $z_i^1 \sim GP(0, K^1)$ với

$$K^1(x, x') \equiv \mathbb{E}[z_i^1(x) z_i^1(x')] = \sigma_b^2 + \sigma_w^2 \, \mathbb{E}[x_i^1(x) x_i^1(x')] \equiv \sigma_b^2 + \sigma_w^2 C(x, x') \quad (2)$$

3.2 Trường hợp deep (mục 2.3), đệ quy qua các lớp

Lấy giới hạn tuần tự $N_1 \to \infty$, rồi $N_2 \to \infty$,... Giả sử z_j^{l-1} đã là GP (giống và độc lập theo j). Sau $l-1$ bước, $z_i^l(x) = b_i^l + \sum_j W_{ij}^l x_j^l(x)$ với $x_j^l(x) = \phi(z_j^{l-1}(x))$, và lại là tổng i.i.d. nên $z_i^l \sim GP(0, K^l)$ với

$$K^l(x, x') \equiv \mathbb{E}[z_i^l(x) z_i^l(x')] = \sigma_b^2 + \sigma_w^2 \, \mathbb{E}_{z^{l-1} \sim GP(0, K^{l-1})} \big[\phi(z_i^{l-1}(x)) \phi(z_i^{l-1}(x')) \big] \quad (4)$$

Kỳ vọng trong (4) chỉ phụ thuộc vào 3 đại lượng $K^{l-1}(x, x'), K^{l-1}(x, x), K^{l-1}(x', x')$, nên bài báo viết gọn thành một hàm đệ quy F_ϕ:

$$K^l(x, x') = \sigma_b^2 + \sigma_w^2 \, F_\phi\big(K^{l-1}(x, x'),\, K^{l-1}(x, x),\, K^{l-1}(x', x')\big) \quad (5)$$

Trường hợp gốc (base case), với $W_{ij}^0 \sim N(0, \sigma_w^2 / d_{in})$, $b_j^0 \sim N(0, \sigma_b^2)$:

$$K^0(x, x') = \sigma_b^2 + \sigma_w^2 \, \frac{x \cdot x'}{d_{in}} \quad (6)$$

Bài báo nhấn mạnh: công thức đệ quy (5)-(6) **trùng** với công thức mean-field signal propagation của Poole et al. (2016) và Schoenholz et al. (2017) (math 26, math 27), và cũng xuất hiện trong văn liệu "compositional kernel" (Cho & Saul 2009; Daniely et al. 2016). Với ReLU, (5) có dạng đóng, đó là "arccosine kernel" (Cho & Saul 2009):

$$\begin{aligned} K^l(x, x') = \sigma_b^2 + \frac{\sigma_w^2}{2\pi} \sqrt{K^{l-1}(x, x) K^{l-1}(x', x')} \\ \Big[\sin \theta_{x,x'}^{l-1} + (\pi - \theta_{x,x'}^{l-1}) \cos \theta_{x,x'}^{l-1} \Big], \\ \theta_{x,x'}^l = \cos^{-1}\left(\frac{K^l(x, x')}{\sqrt{K^l(x, x) K^l(x', x')}} \right) \end{aligned} \quad (11)$$

Ví dụ tự tính (dùng đúng công thức (6) và (11)): lấy $d_{in} = 2$, $\sigma_w^2 = 1, \sigma_b^2 = 0$, hai input trực giao $x_1 = (1, 0)$, $x_2 = (0, 1)$, ReLU.

- Lớp 0 (công thức 6): $K^0(x_1, x_1) = \frac{1 \cdot 1 + 0 \cdot 0}{2} = 0.5$; tương tự $K^0(x_2, x_2) = 0.5$; $K^0(x_1, x_2) = \frac{1 \cdot 0 + 0 \cdot 1}{2} = 0$.
- Góc ở lớp 0: $\theta_{x_1, x_2}^0 = \cos^{-1}(0 / \sqrt{0.5 \cdot 0.5}) = \cos^{-1}(0) = \pi/2$. Vậy $\sin \theta^0 = 1, \cos \theta^0 = 0$, ngoặc vuông $= 1 + (\pi - \pi/2) \cdot 0 = 1$.
- Lớp 1, cặp khác nhau (công thức 11): $K^1(x_1, x_2) = 0 + \frac{1}{2\pi} \sqrt{0.5 \cdot 0.5} \cdot 1 = \frac{0.5}{2\pi} = \frac{1}{4\pi} \approx 0.0796$.
- Lớp 1, cùng điểm: $\theta_{x_1, x_1}^0 = \cos^{-1}(1) = 0 \Rightarrow \sin 0 = 0, \cos 0 = 1$, ngoặc vuông $= 0 + \pi \cdot 1 = \pi$. $K^1(x_1, x_1) = 0 + \frac{1}{2\pi} \cdot 0.5 \cdot \pi = 0.25$ (tương tự $K^1(x_2, x_2) = 0.25$).
- Hệ số tương quan sau 1 lớp: $K^1(x_1, x_2) / \sqrt{K^1(x_1, x_1) K^1(x_2, x_2)} = 0.0796 / 0.25 \approx 0.318$.

Nhận xét từ phép tự tính này: hai input trực giao ở lớp 0 (tương quan 0) sau một bước ReLU-NNGP đã có tương quan dương (≈ 0.318), minh họa trực quan cho hiện tượng ”trộn tương quan theo chiều sâu” mà mục 3.2/Phân tích bên dưới sẽ khai triển ở quy mô lớn (critical line).

3.3 Suy luận Bayes với GP prior (mục 2.4)

Với dữ liệu $D = \{(x_1, t_1), ..., (x_n, t_n)\}$ và nhiễu quan sát Gaussian variance σ_ϵ^2, phân phối posterior dự đoán tại điểm test x^* là Gaussian $z^*|D, x^* \sim N(\bar{\mu}, \bar{K})$ với

$$\bar{\mu} = K_{x^*,D}(K_{D,D} + \sigma_\epsilon^2 I_n)^{-1} t \quad (8)$$

$$\bar{K} = K_{x^*,x^*} - K_{x^*,D}(K_{D,D} + \sigma_\epsilon^2 I_n)^{-1} K_{x^*,D}^T \quad (9)$$

Đây là công thức GP regression chuẩn (Rasmussen & Williams 2006), không cần lan truyền ngược, chỉ cần dựng được K^L.

3.4 Thuật toán tính kernel hiệu quả (mục 2.5)

Với nonlinearity không có dạng đóng, tích phân Gauss trong (4) phải tính số. Cách ”ngây thơ” (tính riêng từng cặp điểm, từng lớp) có chi phí $O(n_g^2 L(n_{train}^2 + n_{train} n_{test}))$. Bài báo giảm chi phí bằng một bảng tra (lookup table) dựng trước trên lưới cố định của pre-activation u, variance s và hệ số tương quan c:

$$F_{ij} = \frac{\sum_{ab} \phi(u_a)\phi(u_b) \exp\big(-\frac{1}{2}(u_a, u_b)\,\Sigma^{-1}\,(u_a, u_b)^T\big)}{\sum_{ab} \exp\big(-\frac{1}{2}(u_a, u_b)\,\Sigma^{-1}\,(u_a, u_b)^T\big)}, \quad \Sigma = \begin{pmatrix} s_i & s_i c_j \\ s_i c_j & s_i \end{pmatrix} \quad (10)$$

sau đó tính $K^l(x, x')$ cho mọi cặp điểm bằng nội suy song tuyến tính (bilinear interpolation) vào bảng này, chi phí còn lại chỉ là $O(n_g^2 n_v n_c + L(n_{train}^2 + n_{train} n_{test}))$. Cài đặt mã nguồn mở: github.com/brain-research/nngp.

4. Thí nghiệm và kết quả chính

Thiết lập: MNIST và CIFAR-10 dạng ”permutation invariant” (làm phẳng ảnh, bỏ cấu trúc không gian), mạng baseline FC cùng độ rộng ở mọi lớp ẩn, loss MSE (nhãn lớp mã hóa one-hot dạng regression target: -0.1 cho lớp sai, 0.9 cho lớp đúng), optimizer Adam, siêu tham số (learning rate, weight decay, σ_w, σ_b, batch size) dò bằng Google Vizier, trung bình khoảng 250 trial cho mỗi cấu hình (kích thước tập train, depth, width, nonlinearity). Không dùng dropout. Chia dữ liệu: MNIST 50k/10k/10k (train/val/test), CIFAR-10 45k/5k/10k. Với GP, quét lưới 30×30 điểm cho ($\sigma_w^2 \in [0.1, 5.0],\ \sigma_b^2 \in [0, 2.0]$), tức 900 lần đánh giá, chọn cấu hình tốt nhất trên validation.

Bảng 1 (số liệu nguyên văn, test accuracy):

Tập train	NN (ReLU)	Acc	GP (ReLU)	Acc	NN (Tanh)	Acc	GP (Tanh)	Acc
MNIST: 1k	NN-2-5000-3.19-0.00	0.9252	GP-20-1.45-0.28	0.9279	NN-2-1000-0.60-0.00	0.9254	GP-20-1.96-0.62	0.9266
MNIST: 10k	NN-2-2000-0.42-0.16	0.9771	GP-7-0.61-0.07	0.9765	NN-2-2000-2.41-1.84	0.9745	GP-2-1.62-0.28	0.9773
MNIST: 50k	NN-2-2000-0.60-0.44	0.9864	GP-1-0.10-0.48	0.9875	NN-2-5000-0.28-0.34	0.9857	GP-1-1.28-0.00	0.9879

Tập train	NN (ReLU)	Acc	GP (ReLU)	Acc	NN (Tanh)	Acc	GP (Tanh)	Acc
CIFAR:10k	NN-5-500-1.29-0.28	0.3225	GP-7-1.28-0.00	0.3608	NN-1-200-1.45-0.12	0.3378	GP-50-2.97-0.97	0.3702
CIFAR:10k	NN-5-2000-1.60-1.07	0.4545	GP-5-2.97-0.28	0.4780	NN-1-500-1.48-1.59	0.4429	GP-7-3.48-2.00	0.4766
CIFAR:45k	NN-3-5000-0.53-0.01	0.5313	GP-3-3.31-1.86	0.5566	NN-2-2000-1.05-2.08	0.5034	GP-3-3.48-1.52	0.5558

(Ký hiệu model: NN là "depth-width-σ_w^2-σ_b^2", GP là "depth-σ_w^2-σ_b^2". Bảng 2 ở Appendix là bản mở rộng của Bảng 1 với nhiều kích thước tập train hơn (100 đến 50k cho MNIST, 100 đến 45k cho CIFAR), theo đúng mẫu hình đã thấy ở Bảng 1.)

Hình 1: accuracy/MSE của NNGP tốt nhất so với NN tốt nhất theo từng width, vẽ theo kích thước tập train; NN càng rộng thì hiệu năng càng tiến gần NNGP.

Hình 2: generalization gap của 180 mạng FC 5 lớp ẩn trên CIFAR-10 (random hyperparameter), lọc lấy các mạng đạt 100% train accuracy (125 mạng Tanh, 55 mạng ReLU) theo bề rộng (width); mạng rộng nhất luôn generalize tốt nhất.

Hình 3 (uncertainty): vì NNGP là Bayes, mỗi điểm test có một ước lượng phương sai dự đoán (công thức 9); phương sai này tương quan cao với MSE thực tế. Số liệu tương quan nguyên văn: MNIST-50k, Tanh-corr: 0.9330, ReLU-corr: 0.9573; CIFAR-45k, Tanh-corr: 0.7428, ReLU-corr: 0.8223 (hyperparameter ví dụ: depth=3, $\sigma_w^2 = 2.0$, $\sigma_b^2 = 0.2$).

Thời gian tính toán (Appendix D): dựng lưới (grid generation, $n_g = n_v = 501, n_c = 500$) tốn 440-460s trên 6 CPU; dựng $K_{D,D}$ đầy đủ cho MNIST 50k tốn 90-140s trên 64 CPU; giải Cholesky cho 1000 điểm test tốn 180-220s; noise mặc định $\sigma_\epsilon^2 = 10^{-10}$, tăng gấp 10 khi Cholesky thất bại.

5. Phân tích / ablation

Ba quan sát phân tích quan trọng rút ra trực tiếp từ bài:

1. **NNGP thường thắng NN nhưng không phải luôn luôn.** Nhìn kỹ Bảng 1: GP thắng ở 11/12 cặp so sánh, nhưng ở MNIST:10k với ReLU, NN (0.9771) nhích hơn GP (0.9765) một chút. Đây là ngoại lệ thật trong số liệu của chính bài báo, không phải lỗi đọc, cho thấy ưu thế của NNGP là một xu hướng mạnh ở dữ liệu nhỏ/kernel method phù hợp, không phải định lý tuyệt đối cho mọi (kích thước tập, nonlinearity).
2. **Mức tương quan uncertainty-error phụ thuộc độ khó bài toán.** Tương quan cao hơn ở MNIST (0.93-0.96) so với CIFAR-10 (0.74-0.82). Điều này gợi ý: GP uncertainty là tín hiệu định lượng tốt khi bài toán "gần tuyến tính hoá được" bởi kernel cố định, còn với bài toán khó hơn (CIFAR), phần sai số không giải thích được bởi kernel cố định tăng lên.
3. **Hiệu năng theo cấu trúc "critical line" (mục 3.2).** Bài báo liên hệ trực tiếp đến lý thuyết signal propagation (Poole et al. 2016; Schoenholz et al. 2017): với Tanh có hai pha "ordered" ($K^l(x, x') \to q^*$ với mọi cặp, mọi input "quên" sự khác biệt) và "chaotic" ($K^l(x, x) \to q^*$ nhưng $K^l(x, x') \to q^* c^*$ với $c^* < 1$ khi $x \neq x'$); với ReLU có hai pha "bounded" (q^* hữu hạn, khác 0) và "unbounded" (q^* tiến vô cùng hoặc về 0). Trên đường biên giữa hai pha (critical line), tốc độ hội tụ về điểm cố định là phi mũ (chậm hơn hẳn). Heatmap accuracy của NNGP theo lưới (σ_w^2, σ_b^2) (Hình 4, và Hình 9 ở Appendix cho nhiều depth khác nhau) bám rất sát critical line này; xa critical line, độ chính xác rơi về mức đoán ngẫu nhiên, và hiện tượng này xảy ra nhanh hơn khi depth L tăng, vì lượng thông tin về dữ liệu chỉ còn nằm

ở phần sai khác $K^L(x, x') - K^\infty(x, x')$, phần này suy giảm theo cấp số nhân với depth và cuối cùng bị nuốt bởi lỗi số (numerical precision). Bài báo còn chỉ ra (Hình 10, Appendix E) rằng hyperparameter tốt nhất của NNGP tập trung quanh critical line một cách nhất quán qua nhiều kích thước tập train khác nhau, cho thấy đây không phải trùng hợp ngẫu nhiên của một thí nghiệm đơn lẻ.

6. Giới hạn

- Chỉ xét mạng fully-connected, dữ liệu được dùng ở dạng ”permutation invariant” (làm phẳng ảnh), bỏ hoàn toàn cấu trúc không gian của ảnh; bài không thử nghiệm kiến trúc convolutional.
- NNGP mô tả **phân phối tại khởi tạo và suy luận Bayes đúng**, không mô tả quá trình SGD thực tế huấn luyện một mạng hữu hạn. Việc SGD có ”xấp xỉ” suy luận Bayes hay không vẫn là câu hỏi mở mà bài chỉ nêu ra (trích dẫn Mandt et al. 2017, tức math 11 trong lộ trình) chứ không trả lời, đây chính là khe hở mà math 23 (NTK) lấp vào theo một hướng khác (mô tả động lực gradient descent bằng kernel, thay vì chỉ mô tả trạng thái tại khởi tạo).
- Suy luận GP đúng cần giải hệ phương trình tuyến tính cỡ $n_{train} \times n_{train}$ (Cholesky), chi phí $O(n^3)$ theo kích thước tập train; bài chỉ dừng ở quy mô MNIST/CIFAR (tối đa 50k/45k điểm) và nêu các phương pháp GP có thể mở rộng quy mô (Quinonero-Candela & Rasmussen 2005; Hensman et al. 2013) là hướng tương lai, chưa triển khai trong bài này.
- Chỉ dùng loss MSE (hồi quy one-hot) để có thể so sánh trực tiếp với công thức GP; chưa đánh giá với cross-entropy loss (bài nêu rõ đây là việc để lại cho công việc tương lai). Không dùng dropout.
- Đã kiểm tra văn bản của bản v3 (3/3/2018, ”Published as a conference paper at ICLR 2018”) được đọc cho ghi chú này: không thấy ghi chú erratum/correction nào trong nội dung đã đọc.

7. Liên hệ với các bài khác

- **math 23 (Jacot, Gabriel, Hongler, ”Neural Tangent Kernel”):** đây là bài nối tiếp trực tiếp và hay bị nhầm lẫn với bài hiện tại. NNGP mô tả kernel **tại khởi tạo** ($\Sigma^{(L)}$ trong ký hiệu của math 23, tương đương K^L ở đây) và dùng nó để làm suy luận Bayes tĩnh (công thức 8-9), không nói gì về việc gradient descent sẽ đi thế nào. NTK mô tả một kernel khác (ký hiệu $\Theta^{(L)}$), xuất hiện khi lấy đạo hàm của hàm mạng theo tham số, và chứng minh rằng **toàn bộ quá trình gradient descent** của mạng vô hạn rộng tương đương kernel gradient descent theo $\Theta_\infty^{(L)}$ cố định. Tóm lại: NNGP = kernel-tại-khởi-tạo cho suy luận Bayes; NTK = kernel-trong-suốt-huấn-luyện cho gradient descent. Hai kernel này khác nhau về bản chất toán học (một là hiệp phương sai của hàm, một là tích các đạo hàm riêng), tuy công thức đệ quy của $\Sigma^{(L)}$ trong NTK trùng với K^L ở đây.
- **math 24 (”On Lazy Training in Differentiable Programming”):** mở rộng ý tưởng của NTK, chỉ ra ”chế độ lazy” (lazy regime, gần tuyến tính hoá quanh khởi tạo) không chỉ xảy ra vì mạng vô hạn rộng mà còn vì cách tham số hoá/scale của mô hình; liên hệ trực tiếp đến NTK (math 23) hơn là NNGP, nhưng cùng một họ ý tưởng ”mạng rất rộng hành xử như một đối tượng tuyến tính hoá/kernel”.
- **math 26 (Poole et al., ”Exponential expressivity in deep neural networks through transient chaos”) và math 27 (Schoenholz et al., ”Deep Information Propagation”):** đây chính là nguồn gốc của công thức đệ quy (5)-(6) và khái niệm ”critical line”/pha ordered-chaotic mà mục 3.2 của bài hiện tại dùng để giải thích tại sao NNGP hoạt động tốt gần critical line. Về mặt công thức, $K^l(x, x')$ ở đây và q^{ab} trong lý thuyết signal propagation của hai bài đó là một.
- (Tham khảo thêm, không bắt buộc đọc): math 25 (”The Principles of Deep Learning Theory”) là một tài liệu tổng hợp muộn hơn, mở rộng NNGP sang các hiệu chỉnh độ rộng hữu hạn (finite-width corrections), nằm ngoài phạm vi của bài 1711.00165 nên không được khai triển ở đây.

8. Gợi ý áp dụng

Giới hạn Gaussian process cho mạng rộng cung cấp một đối chứng để nghiên cứu tác động của độ sâu và hàm kích hoạt. Với mạng hữu hạn, đây là xấp xỉ cần được đánh giá trong điều kiện cụ thể, không phải đồng nhất thức tự động.

9. Thí nghiệm gợi ý

1. **Tái lập mini-Table 1:** lấy một tập con nhỏ của MNIST (ví dụ 1.000-5.000 ảnh), dựng kernel NNGP bằng công thức arccosine ReLU (11) tính đệ quy qua vài lớp, giải GP regression bằng (8)-(9) (dùng numpy/Cholesky), so sánh test accuracy với một mạng FC nhỏ huấn luyện bằng Adam trên cùng tập con. Chạy hoàn toàn trên CPU hoặc trên GPU, không cần huấn luyện lớn.
2. **Tái lập Hình 3 ở quy mô nhỏ:** với cùng kernel ở thí nghiệm 1, tính phương sai dự đoán GP (công thức 9) cho một batch điểm test, đối chiếu với sai số bình phương thực tế, tính hệ số tương quan, kiểm tra xem có nằm trong khoảng tương tự báo cáo của bài (0.7-0.97) không.
3. **Mini phase-diagram:** huấn luyện vài chục mạng FC nhỏ (2-4 lớp ẩn, vài trăm unit) trên một tập con MNIST với lưới thô (σ_w^2, σ_b^2) cho khởi tạo, đo accuracy sau một số bước cố định, vẽ heatmap và xem có xuất hiện dải "tốt" giống critical line của Hình 4/9 không. Rất rẻ, chạy được trên GPU với batch nhỏ.
4. **Kiểm tra tính nửa xác định dương của kernel tự tính:** mở rộng phép tính tay ở mục 3 (hai điểm x_1, x_2 trực giao) ra 4-5 điểm ngẫu nhiên trong $\mathbb{R}^2$, tính ma trận K^0 rồi K^1 bằng công thức (6)/(11) bằng code ngắn, kiểm tra eigenvalue không âm, một bài tập kiểm chứng công thức rất nhẹ, không cần GPU.

10. Tóm tắt công thức / khái niệm

- **NNGP:** mạng FC vô hạn rộng với trọng số/bias Gaussian hội tụ về $GP(0, K^L)$ khi lấy giới hạn tuần tự theo từng lớp.
- Công thức (2): kernel lớp 1, $K^1 = \sigma_b^2 + \sigma_w^2 C(x, x')$.
- Công thức (4)-(5): đệ quy kernel lớp l qua kỳ vọng Gaussian của ϕ ở lớp $l-1$, viết gọn bằng hàm F_ϕ.
- Công thức (6): kernel gốc (lớp 0), tuyến tính theo $x \cdot x'/d_{in}$.
- Công thức (7)-(9): suy luận Bayes GP chuẩn, mean/covariance posterior dạng đóng.
- Công thức (10): lookup table tính F_ϕ số học, dùng nội suy song tuyến tính để giảm độ phức tạp.
- Công thức (11): arccosine kernel, dạng đóng của đệ quy (5) cho ReLU, có ví dụ tự tính ở mục 3.
- **Critical line / pha ordered-chaotic (Tanh), bounded-unbounded (ReLU):** ranh giới hyperparameter (σ_w^2, σ_b^2) nơi tốc độ hội tụ của K^l về điểm cố định là phi mũ (chậm), tương ứng vùng NNGP hoạt động tốt nhất.
- **Permutation invariant MNIST/CIFAR-10:** cách đóng khung ảnh thành vector phẳng để dùng được với mạng FC, bỏ cấu trúc không gian.

11. Kết luận ngắn

Bài báo chứng minh (và kiểm chứng thực nghiệm) rằng mạng FC deep vô hạn rộng tương ứng chính xác với một Gaussian Process, với kernel tính đệ quy qua từng lớp; suy luận Bayes dùng kernel này (NNGP) thường đạt hiệu năng ngang hoặc vượt mạng hữu hạn huấn luyện bằng SGD trên MNIST/CIFAR-10, và độ không chắc chắn của GP tương quan tốt với sai số thực tế. Hiệu năng NNGP còn được giải thích bằng lý thuyết signal propagation (critical line). Đây là nền tảng lý thuyết đầu tiên của một nhánh "lý thuyết NN vô hạn rộng" mà math 23 (NTK) tiếp nối theo hướng mô tả động lực huấn luyện, và math 24, math 25 mở rộng thêm.

Nguồn gốc: https://arxiv.org/abs/1711.00165

Bài 23

Neural Tangent Kernel: Convergence and Generalization in Neural Networks (2018)

Tác giả: Arthur Jacot, Franck Gabriel, Clément Hongler (École Polytechnique Fédérale de Lausanne; Franck Gabriel đồng thời thuộc Imperial College London)
Venue / arXiv: ”32nd Conference on Neural Information Processing Systems (NIPS 2018), Montréal, Canada.”; bản đọc arXiv:1806.07572v4 (10/2/2020, bản chỉnh sửa sau hội nghị NIPS 2018)
Nguồn: arXiv:1806.07572
Chủ đề: Neural Tangent Kernel (NTK), động lực huấn luyện mạng nơ-ron vô hạn rộng bằng gradient descent, kernel gradient descent
Vị trí trong lộ trình: math 23/35 trong ”math content/”, cùng cặp lý thuyết NN vô hạn rộng với math 22 (Deep Neural Networks as Gaussian Processes)

> Ghi chú này là bản tóm tắt, diễn giải lại theo cách hiểu của người viết sau khi đọc trực tiếp bài báo gốc, không phải bản dịch nguyên văn.

1. Bối cảnh

Tối ưu mạng nơ-ron là bài toán không lồi; các điểm yên ngựa có thể làm chậm hội tụ, như những công trình được bài báo dẫn ra đã phân tích. Dù vậy, mạng quá tham số hóa vẫn thường được huấn luyện hiệu quả và tổng quát hóa tốt trên một số tác vụ. Bài báo cũng liên hệ hiện tượng này với các phương pháp kernel.

Nền tảng đã biết trước bài này (trích dẫn [16; 4; 7; 13; 6], trong đó [13] chính là Lee, Bahri, Novak, Schoenholz, Pennington, Sohl-Dickstein, ”Deep neural networks as gaussian processes”, ICLR 2018, tức math 22 trong lộ trình này): tại khởi tạo, ANN vô hạn rộng tương đương một Gaussian Process. Câu hỏi còn bỏ ngỏ mà bài này đặt ra: **trong khi huấn luyện** (không chỉ tại khởi tạo), hành vi của ANN vô hạn rộng có thể mô tả được bằng một đối tượng tương tự kernel không? Tác giả gọi đối tượng đó là Neural Tangent Kernel (NTK).

Thiết lập: ANN fully-connected, lớp đánh số từ 0 (input) đến L (output), mỗi lớp có $n_0, \dots, n_L$ neuron, nonlinearity $\sigma : \mathbb{R} \to \mathbb{R}$ Lipschitz, khả vi hai lần với đạo hàm bậc hai bị chặn (giả thiết làm đơn giản chứng minh, bài ghi rõ có thể không cần thiết nghiêm ngặt nhưng không chứng minh điều đó). Hàm mạng $F^{(L)} : \mathbb{R}^P \to \mathcal{F}$ biến tham số θ thành hàm f_θ, với $P = \sum_{l=0}^{L-1}(n_l + 1)n_{l+1}$ tham số gồm ma trận $W^{(l)} \in \mathbb{R}^{n_l \times n_{l+1}}$ và bias $b^{(l)} \in \mathbb{R}^{n_{l+1}}$, khởi tạo i.i.d. $N(0, 1)$.

2. Ý tưởng chính

Ba khối ý tưởng chính:

1. **Kernel gradient descent:** huấn luyện f_θ bằng gradient descent trên θ tương đương (trong hàm không gian $\mathcal{F}$) với kernel gradient descent theo một kernel nào đó, ngay cả khi hàm cost hợp thành $C \circ F^{(L)}$ không lồi theo θ, trong khi cost C theo $f \in \mathcal{F}$ có thể lồi. Kernel ở đây chính là NTK $\Theta^{(L)}(\theta) = \sum_p \partial_{\theta_p} F^{(L)}(\theta) \otimes \partial_{\theta_p} F^{(L)}(\theta)$, ngẫu nhiên và biến đổi theo $\theta(t)$ trong trường hợp chung (vì $F^{(L)}$ không tuyến tính theo θ).
2. **Giới hạn vô hạn rộng:** khi $n_1, \dots, n_{L-1} \to \infty$ (tuần tự), NTK hội tụ về một kernel giới hạn **xác định (deterministic)** $\Theta_\infty^{(L)}$ tại khởi tạo, và quan trọng hơn, **giữ nguyên không đổi suốt quá trình huấn luyện**. Đây là khác biệt căn bản so với math 22 (NNGP): NNGP chỉ nói về phân phối của f_θ **tại khởi tạo**; bài này nói về **toàn bộ động lực huấn luyện**.
3. **Hệ quả cho least-squares regression:** khi cố định kernel $\Theta_\infty^{(L)}$, động lực của $f_\theta(t)$ trong giới hạn vô hạn rộng trở thành một phương trình vi phân **tuyến tính**, giải được bằng khai triển theo các thành phần

chính (kernel PCA) của $\Theta_\infty^{(L)}$; tốc độ hội tụ theo mỗi thành phần tỉ lệ với eigenvalue tương ứng, điều này cho một lý do lý thuyết cho early stopping.

3. Phương pháp chi tiết

3.1 Tham số hoá NTK (khác tham số hoá cổ điển, Remark 1)

Hàm mạng được định nghĩa theo chuẩn tham số hoá "NTK parametrization":

$$\tilde{\alpha}^{(0)}(x;\theta) = x, \qquad \tilde{\alpha}^{(l+1)}(x;\theta) = \frac{1}{\sqrt{n_l}} W^{(l)} \alpha^{(l)}(x;\theta) + \beta b^{(l)}, \qquad \alpha^{(l)}(x;\theta) = \sigma(\tilde{\alpha}^{(l)}(x;\theta))$$

và $f_\theta(x) := \tilde{\alpha}^{(L)}(x;\theta)$. Khác với tham số hoá cổ điển (không có hệ số $1/\sqrt{n_l}$ và β, dùng khởi tạo LeCun $W_{ij}^{(l)} \sim N(0, 1/n_l)$), tập hàm biểu diễn được là như nhau giữa hai cách tham số hoá, nhưng **đạo hàm** của $F^{(L)}$ theo $W^{(l)}$ và $b^{(l)}$ bị scale khác nhau ($1/\sqrt{n_l}$ và β tương ứng). Hệ số β dùng để cân bằng ảnh hưởng của bias so với trọng số khi n_l lớn (vì $1/\sqrt{n_l}$ làm giảm mạnh ảnh hưởng của trọng số khi n_l lớn). Trong thực nghiệm của bài, $\beta = 0.1$ với learning rate 1.0 (tương đương một mạng cổ điển rộng 100 với learning rate 0.01).

3.2 Kernel gradient descent (mục 3)

Một kernel đa chiều $K : \mathbb{R}^{n_0} \times \mathbb{R}^{n_0} \to \mathbb{R}^{n_L \times n_L}$ (đối xứng $K(x,x') = K(x',x)^T$) sinh một dạng song tuyến tính $\langle f, g \rangle_K := \mathbb{E}_{x,x' \sim p_{in}}[f(x)^T K(x,x') g(x')]$. K được gọi là xác định dương (positive definite) theo $\|\cdot\|_{p_{in}}$ nếu $\|f\|_{p_{in}} > 0 \Rightarrow \|f\|_K > 0$. Với $\mathcal{F}^*$ là đối ngẫu của $\mathcal{F}$ theo p_{in}, ánh xạ $\Phi_K : \mathcal{F}^* \to \mathcal{F}$ biến $\mu = \langle d, \cdot \rangle_{p_{in}}$ thành hàm f_μ với $f_{\mu,i}(x) = \langle d, K_{i,\cdot}(x,\cdot) \rangle_{p_{in}}$.

Với cost C chỉ phụ thuộc giá trị của f tại các điểm dữ liệu, đạo hàm hàm $\partial_{fin} C|_{f_0} \in \mathcal{F}^*$ có phần tử đối ngẫu $d|_{f_0}$. **Kernel gradient** $\nabla_K C|_{f_0} := \Phi_K(\partial_{fin} C|_{f_0})$, với dạng tường minh trên tập hữu hạn N điểm:

$$\nabla_K C|_{f_0}(x) = \frac{1}{N} \sum_{j=1}^{N} K(x, x_j)\, d|_{f_0}(x_j)$$

Hàm $f(t)$ theo "kernel gradient descent" với K thoả $\partial_t f(t) = -\nabla_K C|_{f(t)}$, và cost tiến hoá theo $\partial_t C|_{f(t)} = -\|d|_{f(t)}\|_K^2$: cost giảm chặt trừ khi $\|d|_{f(t)}\|_{p_{in}} = 0$. Nếu K xác định dương và C lồi, bị chặn dưới, $f(t)$ hội tụ về cực tiểu toàn cục.

3.3 Trực giác: random functions approximation (mục 3.1)

Lấy P hàm ngẫu nhiên độc lập $f^{(p)}$ với hiệp phương sai K ($\mathbb{E}[f_k^{(p)}(x) f_{k'}^{(p)}(x')] = K_{kk'}(x,x')$), tham số hoá tuyến tính $f_\theta^{lin} = \frac{1}{\sqrt{P}} \sum_p \theta_p f^{(p)}$. Gradient descent trên θ của cost $C \circ F^{lin}$ tương đương kernel gradient descent với "tangent kernel" ngẫu nhiên $\tilde{K} = \frac{1}{P} \sum_p f^{(p)} \otimes f^{(p)}$, và khi $P \to \infty$, $\tilde{K} \to K$ (luật số lớn). Đây là mô hình đồ chơi giải thích trực quan tại sao, với $F^{(L)}$ (phi tuyến theo θ) thay vì F^{lin} (tuyến tính theo θ), người ta kỳ vọng một hiện tượng tương tự xảy ra trong giới hạn vô hạn rộng.

3.4 Định nghĩa NTK và kết quả tại khởi tạo (mục 4, 4.1)

$$\Theta^{(L)}(\theta) = \sum_{p=1}^{P} \partial_{\theta_p} F^{(L)}(\theta) \otimes \partial_{\theta_p} F^{(L)}(\theta)$$

Mệnh đề 1 (Proposition 1): tại khởi tạo, với $n_1, \ldots, n_{L-1} \to \infty$ (tuần tự), $f_{\theta,k}$ ($k = 1, \ldots, n_L$) tiến về i.i.d. GP tâm 0, hiệp phương sai $\Sigma^{(L)}$ xác định đệ quy:

$$\Sigma^{(1)}(x,x') = \frac{1}{n_0}x^T x' + \beta^2, \qquad \Sigma^{(L+1)}(x,x') = \mathbb{E}_{f\sim N(0,\Sigma^{(L)})}[\sigma(f(x))\sigma(f(x'))] + \beta^2$$

Định lý 1 (Theorem 1): trong cùng giới hạn, NTK hội tụ theo xác suất về kernel giới hạn xác định: $\Theta^{(L)} \to \Theta_\infty^{(L)} \otimes \mathrm{Id}_{n_L}$, với kernel vô hướng $\Theta_\infty^{(L)}$ xác định đệ quy:

$$\Theta_\infty^{(1)}(x,x') = \Sigma^{(1)}(x,x'), \qquad \Theta_\infty^{(L+1)}(x,x') = \Theta_\infty^{(L)}(x,x')\,\dot\Sigma^{(L+1)}(x,x') + \Sigma^{(L+1)}(x,x')$$

trong đó $\dot\Sigma^{(L+1)}(x,x') = \mathbb{E}_{f\sim N(0,\Sigma^{(L)})}[\dot\sigma(f(x))\dot\sigma(f(x'))]$, $\dot\sigma$ là đạo hàm của σ. $\Theta_\infty^{(L)}$ chỉ phụ thuộc σ, depth và variance khởi tạo (ở đây là 1).

Bài chú giải ý nghĩa (Remark 4): trong công thức đệ quy, số hạng $\Sigma^{(L+1)}$ đại diện cho phần học của **lớp cuối**, còn số hạng $\Theta_\infty^{(L)}\dot\Sigma^{(L+1)}$ đại diện cho phần học của **các lớp thấp hơn**. Dù biến động của từng activation riêng lẻ ở lớp ẩn co lại khi width lớn, hiệu ứng tập thể (collective) qua rất nhiều neuron vẫn đáng kể, nên các lớp thấp vẫn học được.

Ví dụ tự tính (trường hợp cơ sở $L = 1$, không có lớp ẩn, đúng theo chứng minh của Định lý 1 trong Appendix A.1): với $L = 1$, $f_\theta(x) = \frac{1}{\sqrt{n_0}}W^{(0)}x + \beta b^{(0)}$. Lấy $n_0 = 2$, $\beta = 0.1$, hai điểm $x = (1,2)$, $x' = (3,1)$.

- $\partial_{W_1} f(x) = x_1/\sqrt{n_0} = 1/\sqrt{2}$, $\partial_{W_2} f(x) = x_2/\sqrt{n_0} = 2/\sqrt{2}$, $\partial_b f(x) = \beta = 0.1$ (tương tự cho x': $3/\sqrt{2}, 1/\sqrt{2}, 0.1$).
- $\Theta(x,x') = \partial_{W_1} f(x)\partial_{W_1} f(x') + \partial_{W_2} f(x)\partial_{W_2} f(x') + \partial_b f(x)\partial_b f(x') = \frac{1\cdot 3}{2} + \frac{2\cdot 1}{2} + 0.1\cdot 0.1 = 1.5 + 1.0 + 0.01 = 2.51$.
- Đối chiếu công thức tổng quát: $\Sigma^{(1)}(x,x') = \frac{x\cdot x'}{n_0} + \beta^2 = \frac{(1\cdot 3 + 2\cdot 1)}{2} + 0.01 = \frac{5}{2} + 0.01 = 2.51$. Khớp đúng với tính trực tiếp từ đạo hàm, xác nhận công thức cơ sở của Định lý 1 bằng tay cho trường hợp đơn giản nhất ($L = 1$, không hidden layer).

3.5 Kết quả trong khi huấn luyện (mục 4.2)

Tham số cập nhật theo hướng huấn luyện $d_t \in \mathcal{F}$ (tổng quát hơn gradient descent thường, bao gồm cả GAN): $\partial_t \theta_p(t) = \langle \partial_{\theta_p} F^{(L)}(\theta(t)), d_t\rangle_{p_{in}}$, với gradient descent thì $d_t = -d|_{f_{\theta(t)}}$.

Định lý 2 (Theorem 2): nếu σ Lipschitz, khả vi hai lần, đạo hàm hai bị chặn, và $\int_0^T \|d_t\|_{p_{in}} dt$ bị chặn ngẫu nhiên, thì khi $n_1, \ldots, n_{L-1} \to \infty$ (tuần tự), đều trên $t \in [0,T]$: $\Theta^{(L)}(t) \to \Theta_\infty^{(L)} \otimes \mathrm{Id}_{n_L}$ (NTK giữ nguyên không đổi suốt huấn luyện). Khi đó động lực của f_θ là:

$$\partial_t f_{\theta(t)} = \Phi_{\Theta_\infty^{(L)}\otimes \mathrm{Id}_{n_L}}(\langle d_t, \cdot\rangle_{p_{in}})$$

Chứng minh (Appendix A.2) dựa trên quy nạp theo depth, chặn biến động của activation/weight theo tốc độ $O(1/\sqrt{n_l})$ bằng bất đẳng thức Grönwall (Lemma 1, Appendix A.3): mỗi activation riêng lẻ biến đổi rất ít khi width lớn, nhưng tổng hợp qua nhiều neuron vẫn tạo ra học tập có ý nghĩa ở các lớp thấp.

Mệnh đề 2 (Proposition 2, xác định dương): với σ Lipschitz, không phải đa thức, giới hạn $\Theta_\infty^{(L)}$ hạn chế trên mặt cầu đơn vị S^{n_0-1} là xác định dương nếu $L \geq 2$. Chứng minh (Appendix A.4) dùng kỹ thuật khai triển Hermite và hàm "dual activation" (Daniely et al. 2016) cùng một định lý tổng quát hoá kết quả Schönberg (Gneiting 2013) về tính xác định dương của kernel dạng $f(x^T x')$ trên mặt cầu, dựa trên dấu các hệ số khai triển chuỗi lũy thừa của f.

3.6 Least-squares regression và early stopping (mục 5)

Với cost $C(f) = \frac{1}{2}\|f - f^*\|^2_{p_{in}}$, động lực $\partial_t f_t = \Phi_K(\langle f^* - f, \cdot\rangle_{p_{in}})$ có nghiệm $f_t = f^* + e^{-t\Pi}(f_0 - f^*)$ với $\Pi : f \mapsto \Phi_K(\langle f, \cdot\rangle_{p_{in}})$. Π có tối đa Nn_L eigenfunction dương $f^{(1)}, \ldots, f^{(Nn_L)}$ (chính là thành phần chính kernel PCA của dữ liệu theo K), eigenvalue λ_i. Phân rã $f^* - f_0 = \Delta_f^0 + \sum_i \Delta_f^i$, nghiệm trở thành:

$$f_t = f^* + \Delta_f^0 + \sum_{i=1}^{Nn_L} e^{-t\lambda_i}\Delta_f^i$$

Hội tụ **nhanh hơn** theo các thành phần có eigenvalue lớn hơn; early stopping do đó tập trung hội tụ vào các thành phần chính quan trọng nhất, tránh khớp các thành phần "nhiễu" (eigenvalue nhỏ, ví dụ ứng với tần số cao nếu K là RBF kernel). Khi $t \to \infty$ (và K xác định dương trên dữ liệu, tức ma trận Gram $\tilde{K}$ khả nghịch):

$$f_{\infty,k}(x) = \kappa_{x,k}^T \tilde{K}^{-1} y^* + \left(f_0(x) - \kappa_{x,k}^T \tilde{K}^{-1} y_0\right)$$

Số hạng đầu (mean) có ý nghĩa thống kê: đó là ước lượng MAP với Gaussian prior $f_k \sim N(0, \Theta_\infty^{(L)})$ và điều kiện $f_k(x_i) = f_k^*(x_i)$, tương đương kernel ridge regression khi hệ số regularize $\lambda \to 0$. Số hạng sau là một Gaussian tâm 0 có variance tiến về 0 tại các điểm dữ liệu.

4. Thí nghiệm và kết quả chính

Mục 6 làm 3 thí nghiệm số, tất cả FC-ANN cùng width $n = n_1 = \cdots = n_{L-1}$ ở các lớp ẩn, ReLU, $n_L = 1$, $\beta = 0.1$.

6.1 Sự hội tụ của NTK: mạng depth $L = 4$, width $n = 500$ và $n = 10000$, input trên vòng tròn đơn vị $x = (\cos\gamma, \sin\gamma)$, $x_0 = (1, 0)$ cố định. Vẽ $\Theta^{(4)}(x_0, x)$ cho 10 lần khởi tạo độc lập mỗi width, tại $t = 0$ và sau 200 bước gradient descent (lr $= 1.0$), với mục tiêu hồi quy $f^*(x) = x_1 x_2$ trên input $N(0, 1)$ ngẫu nhiên. Kết quả: mạng rộng hơn có NTK ít phương sai hơn, mượt hơn; kỳ vọng của NTK gần như giống nhau ở cả hai width; sau huấn luyện, NTK có xu hướng "phình to" (inflate), hiệu ứng này nhỏ hơn rõ rệt ở mạng rộng ($n = 10000$).

6.2 Kernel regression: so sánh phân phối Gaussian lý thuyết tại $t \to \infty$ với phân phối thực nghiệm của mạng hữu hạn tại $T = 1000$ (lr $= 1.0$), width $n = 50, 1000$, 10 lần khởi tạo, huấn luyện trên 4 điểm trên vòng tròn đơn vị. Đồng thời xấp xỉ $\Theta_\infty^{(4)}$ và $\Sigma^{(4)}$ bằng mạng rất rộng ($n = 10000$) để vẽ percentile thứ 10, 50, 90 của phân phối giới hạn. Kết quả: phân phối của cả hai width khớp khá tốt với phân phối giới hạn; ngay cả $n = 50$ cũng cho chỉ dấu tốt về phân phối tại $t \to \infty$.

6.3 Hội tụ theo thành phần chính, trên MNIST ($n_0 = 784$): tính 3 thành phần chính đầu của NTK (width $n = 10000$) trên batch $N = 512$ ảnh, bằng power iteration. Eigenvalue thu được: $\lambda_1 = 0.0457$, $\lambda_2 = 0.00108$, $\lambda_3 = 0.00078$. Vì kernel PCA không centering, thành phần đầu gần như là hàm hằng, giải thích khoảng cách lớn giữa eigenvalue 1 và 2. Đặt mục tiêu $f^* = f_{\theta(0)} + 0.5 f^{(2)}$, phân rã $f_{\theta(t)} - f^*$ thành g_t (tỉ lệ với $f^{(2)}$) và h_t (trực giao). Trong giới hạn vô hạn rộng: $\|g_t\|_{p_{in}} = 0.5e^{-\lambda_2 t}$, $h_t = 0$. Thử với width $n = 100, 1000, 10000$: mạng rộng hơn lệch khỏi đường thẳng lý thuyết ít hơn (Hình 3b); quỹ đạo theo thành phần chính thứ 2 hội tụ về giới hạn lý thuyết (Hình 3c) khi width tăng.

Chú thích (footnote 3): nếu chọn $\beta = 1.0$ thay cho $\beta = 0.1$ (khuyến nghị), khoảng cách giữa eigenvalue 1 và 2 lớn hơn khoảng 10 lần, khiến việc huấn luyện khó hơn.

5. Phân tích / ablation

- **Quan sát "nghịch lý" về tốc độ hội tụ theo số bước:** mạng nhỏ hơn có vẻ hội tụ nhanh hơn theo số bước (Hình 3c), điều này giải thích được bởi hiệu ứng "inflation" của NTK quan sát ở thí nghiệm 6.1:

nhân NTK với một hệ số a tương đương nhân learning rate với a; nhưng vì NTK của mạng rộng ổn định hơn trong khi huấn luyện, về nguyên tắc có thể chọn learning rate lớn hơn cho mạng rộng. Tác giả cảnh báo: phải cẩn thận khi so sánh tốc độ hội tụ theo số bước (steps) so với theo thời gian liên tục t, vì cả hiệu ứng inflation và learning rate đều ảnh hưởng.

- **Ablation về β (footnote 3):** tăng β từ 0.1 lên 1.0 làm khoảng cách giữa eigenvalue thứ nhất và thứ hai của NTK lớn hơn khoảng 10 lần, và bài báo nói rõ điều này "khiến huấn luyện khó hơn" (đây là một ví dụ ablation thật có trong bài, không phải suy luận thêm), cho thấy tầm quan trọng của việc cân bằng ảnh hưởng bias/trọng số đúng cách trong tham số hoá NTK.
- **Độ rộng và độ ổn định của kernel:** xuyên suốt cả ba thí nghiệm, mẫu số chung là "mạng rộng hơn luôn hành xử gần giới hạn lý thuyết hơn" theo mọi tiêu chí được đo (phương sai NTK ở 6.1, khớp phân phối ở 6.2, lệch khỏi đường thẳng lý thuyết ở 6.3); đây chính là bằng chứng thực nghiệm trực tiếp cho hai định lý chính (Định lý 1, Định lý 2), không chỉ là minh họa định tính.

6. Giới hạn

- Các định lý chính (Mệnh đề 1, Định lý 1, Định lý 2) được chứng minh khi lấy giới hạn width **tuần tự** ($n_1 \to \infty$ rồi $n_2 \to \infty$, ...), không phải lấy đồng thời $\min(n_1, \ldots, n_{L-1}) \to \infty$; bài báo tự ghi rõ trong Appendix A: "kết quả của chúng tôi về nguyên tắc có thể được làm mạnh hơn cho thiết lập chung hơn" nhưng không chứng minh điều đó trong bài này.
- Giả thiết σ Lipschitz, khả vi hai lần, đạo hàm bậc hai bị chặn, được dùng cho toàn bộ máy chứng minh (đặc biệt Định lý 2 và Lemma 1 dùng bất đẳng thức Grönwall); ReLU **không khả vi hai lần theo nghĩa cổ điển** (đạo hàm bậc hai không tồn tại tại 0 theo nghĩa thông thường), nhưng thí nghiệm số (mục 6) vẫn dùng ReLU, tức nằm ngoài giả thiết chứng minh chặt của Định lý 2; bài không bàn thêm về khoảng cách này giữa lý thuyết và thực nghiệm.
- Mệnh đề 2 (xác định dương) chỉ chứng minh cho dữ liệu nằm trên **mặt cầu đơn vị** và σ không phải đa thức, với $L \geq 2$; không bao quát phân phối input tổng quát.
- Chứng minh hội tụ tuyến tính, nghiệm f_t dạng đóng, và lý giải early stopping (mục 5) chỉ được triển khai đầy đủ cho **least-squares regression**; với cost tổng quát (ví dụ cross-entropy cho phân loại), bài chỉ đưa ra khung kernel gradient descent ở mức mục 3 (không có bảo đảm hội tụ tuyến tính tường minh như mục 5).
- Thí nghiệm số có quy mô nhỏ: hai thí nghiệm đầu dùng input 2 chiều trên vòng tròn đơn vị (dữ liệu tổng hợp), thí nghiệm thứ ba dùng MNIST nhưng chỉ một batch 512 ảnh để tính kernel PCA; không có thí nghiệm ở quy mô dữ liệu/mạng lớn như thực tế hiện đại.
- Đã rà soát toàn văn bản v4 (10/2/2020) được đọc cho ghi chú này: không tìm thấy ghi chú erratum/correction nào trong nội dung, mặc dù đây là bản sửa đổi sau hội nghị NIPS 2018 gốc; không thể xác định cụ thể những gì đã được sửa giữa các phiên bản vì chỉ đọc được văn bản của phiên bản v4 duy nhất.

7. Liên hệ với các bài khác

- **math 22 (Lee et al., "Deep Neural Networks as Gaussian Processes"):** đây là liên kết **trực tiếp, được chính bài này trích dẫn** (reference [13], "Deep neural networks as gaussian processes, ICLR 2018"). Phân biệt rõ: NNGP (math 22) nói về $\Sigma^{(L)}$, kernel hiệp phương sai của hàm mạng **tại khởi tạo**, dùng cho suy luận Bayes tĩnh (công thức GP regression). Bài hiện tại (math 23) nói về $\Theta^{(L)}$, kernel sinh từ tích các đạo hàm riêng của hàm mạng theo tham số, mô tả **toàn bộ quỹ đạo huấn luyện bằng gradient descent**. Về công thức, $\Sigma^{(L)}$ ở hai bài gần như cùng một đệ quy (chỉ khác ký hiệu σ_w^2, σ_b^2 so với tham số hoá $1/\sqrt{n_l}, \beta$ của bài này), nhưng $\Theta_\infty^{(L)}$ là một đối tượng hoàn toàn khác, không xuất hiện trong math 22. Đây chính là điểm hay bị người học nhầm lẫn nhất giữa hai bài: NNGP không nói gì về động lực gradient descent, NTK mới là bài nói về điều đó.

- **math 24 ("On Lazy Training in Differentiable Programming"):** mở rộng và hình thức hoá thêm ý tưởng "chế độ lazy" (lazy regime) mà NTK ngầm giả định (mạng tuyến tính hoá quanh khởi tạo trong giới hạn rộng), chỉ ra hiện tượng lazy training không chỉ đến từ việc width lớn mà còn từ cách scale tham số hoá (gần giống đúng nhận xét ở mục 3.1 của bài hiện tại về vai trò của β và $1/\sqrt{n_l}$).
- **math 34 (Mei, Montanari, Nguyen, "A Mean Field View of the Landscape of Two-Layers Neural Networks"):** cũng được trích dẫn trực tiếp trong bài này (reference [15]), ở đúng câu "phân tích động lực huấn luyện trong giới hạn rộng cho mạng nông (shallow) đã có tiến triển gần đây" (một hướng nghiên cứu song song, dùng công cụ mean-field PDE thay vì kernel, cho mạng 2 lớp).
- **math 33 ("Gradient Descent Finds Global Minima of Deep Neural Networks"):** cùng chủ đề lộ trình (không phải trích dẫn trực tiếp trong bài 1806.07572, mà là liên kết theo mạch lý thuyết hội tụ toàn cục của deep network dựa trên các tính chất giống NTK/positive-definite Gram matrix), nên được xếp gần math 23 trong lộ trình nghiên cứu hội tụ của gradient descent trên mạng rộng.

8. Gợi ý áp dụng

NTK giúp phân tích động lực học huấn luyện trong giới hạn độ rộng lớn. Cần phân biệt chế độ kernel gần cố định với quá trình học đặc trưng khi biểu diễn thay đổi đáng kể; kết quả giới hạn không tự bảo đảm mô hình hữu hạn hoạt động giống hệt.

9. Thí nghiệm gợi ý

1. **Tái lập mini-6.1 (hội tụ/inflation của NTK):** với mạng toy depth 4 trên input vòng tròn đơn vị, so sánh hai width nhỏ (ví dụ 200 và 2000) thay vì 500/10000 của bài, tính NTK bằng autograd (Jacobian theo tham số rồi nhân ngoài), quan sát hiện tượng "phình to" sau vài trăm bước gradient descent. Rất rẻ, chạy trên CPU hoặc trên GPU trong vài giây đến vài phút.
2. **Tái lập mini-6.3 (kernel PCA trên MNIST):** lấy batch nhỏ (128-512 ảnh) MNIST, tính empirical NTK của một mạng width vài nghìn bằng autograd, trích 3 eigenvalue đầu bằng power iteration, so sánh thứ tự độ lớn với $\lambda_1 = 0.0457, \lambda_2 = 0.00108, \lambda_3 = 0.00078$ của bài (không kỳ vọng khớp tuyệt đối vì kiến trúc/seed khác, chỉ kỳ vọng khớp về thứ tự độ lớn và khoảng cách lớn giữa λ_1 và λ_2).
3. **Kiểm chứng tốc độ hội tụ theo thành phần chính:** đặt $f^* = f_{\theta(0)} + 0.5 f^{(2)}$ như mục 6.3, huấn luyện mạng nhỏ, đo $\|g_t\|$ theo thời gian, fit tốc độ suy giảm mũ, so sánh với λ_2 ước lượng ở thí nghiệm 2; làm với vài width để xem độ lệch khỏi $0.5e^{-\lambda_2 t}$ giảm khi width tăng, như Hình 3b/3c của bài.
4. **Ablation β:** lặp lại thí nghiệm 2 hoặc 3 với $\beta = 0.1$ và $\beta = 1.0$, đo khoảng cách $\lambda_1 - \lambda_2$ ở mỗi trường hợp, kiểm tra có thấy xu hướng tăng rõ rệt giống ghi chú của bài (footnote 3) không.

10. Tóm tắt công thức / khái niệm

- **NTK:** $\Theta^{(L)}(\theta) = \sum_p \partial_{\theta_p} F^{(L)}(\theta) \otimes \partial_{\theta_p} F^{(L)}(\theta)$, ngẫu nhiên tại khởi tạo, biến đổi theo $\theta(t)$ nói chung.
- **Kernel gradient descent:** $\partial_t f(t) = -\nabla_K C|_{f(t)}$, với $\nabla_K C|_{f_0}(x) = \frac{1}{N}\sum_j K(x, x_j) d|_{f_0}(x_j)$; cost giảm theo $\partial_t C|_{f(t)} = -\|d|_{f(t)}\|_K^2$.
- **Mệnh đề 1:** $\Sigma^{(1)}(x, x') = \frac{1}{n_0} x^T x' + \beta^2$; $\Sigma^{(L+1)} = \mathbb{E}_{f\sim N(0,\Sigma^{(L)})}[\sigma(f(x))\sigma(f(x'))] + \beta^2$ (GP tại khởi tạo).
- **Định lý 1:** $\Theta^{(L)} \to \Theta_\infty^{(L)} \otimes \mathrm{Id}_{n_L}$; $\Theta_\infty^{(L+1)} = \Theta_\infty^{(L)} \dot{\Sigma}^{(L+1)} + \Sigma^{(L+1)}$, với $\dot{\Sigma}^{(L+1)} = \mathbb{E}[\dot\sigma(f(x))\dot\sigma(f(x'))]$ (kernel giới hạn tại khởi tạo).
- **Định lý 2:** $\Theta^{(L)}(t) \to \Theta_\infty^{(L)} \otimes \mathrm{Id}_{n_L}$ đều trên $[0, T]$ (NTK không đổi suốt huấn luyện); động lực $\partial_t f_{\theta(t)} = \Phi_{\Theta_\infty^{(L)} \otimes \mathrm{Id}}(\langle d_t, \cdot\rangle_{p_{in}})$.
- **Mệnh đề 2:** $\Theta_\infty^{(L)}$ hạn chế trên mặt cầu đơn vị là xác định dương nếu σ không phải đa thức, $L \geq 2$.
- **Nghiệm least-squares:** $f_t = f^* + \Delta_f^0 + \sum_i e^{-t\lambda_i} \Delta_f^i$ (hội tụ nhanh theo eigenvalue lớn, cơ sở cho early stopping); f_∞ = MAP/kernel ridge regression khi $\lambda \to 0$.

- **Chế độ lazy:** vùng tham số hoá/width mà $\Theta^{(L)}(t) \approx \Theta^{(L)}_{\infty}$ gần như không đổi, mạng hành xử gần tuyến tính hoá quanh khởi tạo (liên hệ math 24).

11. Kết luận ngắn

Bài báo chứng minh rằng ANN vô hạn rộng, trong khi huấn luyện bằng gradient descent, tuân theo kernel gradient descent với một kernel xác định, cố định gọi là Neural Tangent Kernel; điều này biến bài toán huấn luyện không lồi theo tham số thành bài toán kernel method lồi theo hàm, cho phép phân tích hội tụ (qua tính xác định dương của NTK) và sinh ra một lý giải lý thuyết rõ ràng cho early stopping trong hồi quy least-squares. Đây là bước tiếp nối trực tiếp của math 22 (chuyển từ ”tại khởi tạo” sang ”trong suốt huấn luyện”), và là nền tảng cho các phân tích lazy training sâu hơn như math 24.

Nguồn gốc: https://arxiv.org/abs/1806.07572

Bài 24

On Lazy Training in Differentiable Programming (2018)

Tác giả: Lénaïc Chizat (CNRS, Université Paris-Sud), Edouard Oyallon (CentraleSupélec, INRIA), Francis Bach (INRIA, ENS, PSL Research University).
Venue / arXiv: arXiv:1812.07956 (v5, 07/01/2020, bản mở rộng từ bản đầu "A Note on Lazy Training in Supervised Differential Programming" online 12/2018); venue ghi trong paper: "33rd Conference on Neural Information Processing Systems (NeurIPS 2019), Vancouver, Canada."
Nguồn: https://arxiv.org/abs/1812.07956
Chủ đề: định nghĩa và phân tích hiện tượng "lazy training" (mô hình phi tuyến hành xử gần như phiên bản tuyến tính hoá quanh điểm khởi tạo khi được scale lên), chỉ ra đây là hệ quả của cách chọn **scale** (không chỉ của over-parameterization), và thực nghiệm cho thấy CNN hiệu năng cao không nằm trong chế độ này.
Vị trí trong lộ trình: đọc sau math 22 (Deep NN as Gaussian Processes) và math 23 (Neural Tangent Kernel), vì bài này dùng lại khái niệm tangent kernel của NTK nhưng đặt trong khung tổng quát hơn (differentiable programming, không chỉ neural network) và đưa thêm bằng chứng thực nghiệm phản biện. Nên đọc trước math 25 (Principles of Deep Learning Theory), nơi "lazy"/infinite-width limit cũng xuất hiện như một trường hợp biên ($r = L/n \to 0$). Liên hệ math 19 (Intrinsic Dimension, cùng hỏi "mô hình cần di chuyển bao nhiêu trong không gian tham số để học").

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (bản arXiv); các phép tính đánh dấu "tự tính" là của người ghi chú. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc.

1. Bối cảnh

Các công trình khoảng năm 2018–2019 cho thấy trong những điều kiện nhất định, mạng nơ-ron quá tham số hóa được huấn luyện bằng gradient descent có thể đạt loss huấn luyện bằng 0 với tốc độ hội tụ tuyến tính, trong khi tham số thay đổi ít so với lúc khởi tạo. Bài này gọi một chế độ như vậy là lazy training: mô hình phi tuyến hoạt động gần với phép tuyến tính hóa quanh điểm khởi tạo.

Câu hỏi paper đặt ra: hiện tượng này có thực sự là lý do khiến mạng nơ-ron hoạt động tốt trong thực tế hay không, hay chỉ là một chế độ huấn luyện đặc biệt, dễ phân tích về lý thuyết nhưng không phải là nguyên nhân thành công thực sự?

Khái niệm cần nhớ: cho một mô hình $h : \mathbb{R}^p \to \mathcal{F}$ (ví dụ mạng nơ-ron, $\mathcal{F}$ là không gian Hilbert các hàm dự đoán), **tuyến tính hoá (linearization)** của h quanh điểm khởi tạo w_0 là

$$\bar{h}(w) = h(w_0) + Dh(w_0)(w - w_0),$$

tức khai triển Taylor bậc 1. Nếu h là mạng nơ-ron, $Dh(w_0)$ chính là **Jacobian** của mạng theo tham số tại điểm khởi tạo, và $\bar{h}$ tương đương một **mô hình kernel method tuyến tính** với kernel $K(x, x') = D_w f(w_0, x) D_w f(w_0, x')^\top$ (gọi là **tangent kernel**, cùng khái niệm Jacot et al. 2018 dùng cho NTK, nhưng ở đây công thức cho mạng hữu hạn chiều, không lấy giới hạn width $\to \infty$).

2. Ý tưởng chính

Bài báo lập luận rằng lazy training không chỉ xuất hiện ở mạng quá tham số hóa: nó còn phụ thuộc vào lựa chọn thang đo, kể cả những lựa chọn ngầm định. Paper chứng minh: với hầu như **bất kỳ** mô hình tham số hoá trơn h, nếu đầu ra ban đầu gần 0 ($h(w_0) \approx 0$) và ta nhân đầu ra với một hệ số α đủ lớn, mô hình sẽ rơi vào chế độ lazy (gần tuyến tính hoá) khi $\alpha \to \infty$, bất kể số tham số p là bao nhiêu.

Để định lượng "mức độ lazy", paper định nghĩa **tỉ số nghịch đảo scale tương đối (inverse relative scale)** $\kappa_h(w_0)$; với hàm loss bình phương $R(y) = \frac{1}{2}\|y - y^\star\|^2$:

$$\kappa_h(w_0) := \|h(w_0) - y^\star\| \cdot \frac{\|D^2 h(w_0)\|}{\|Dh(w_0)\|^2} \tag{1}$$

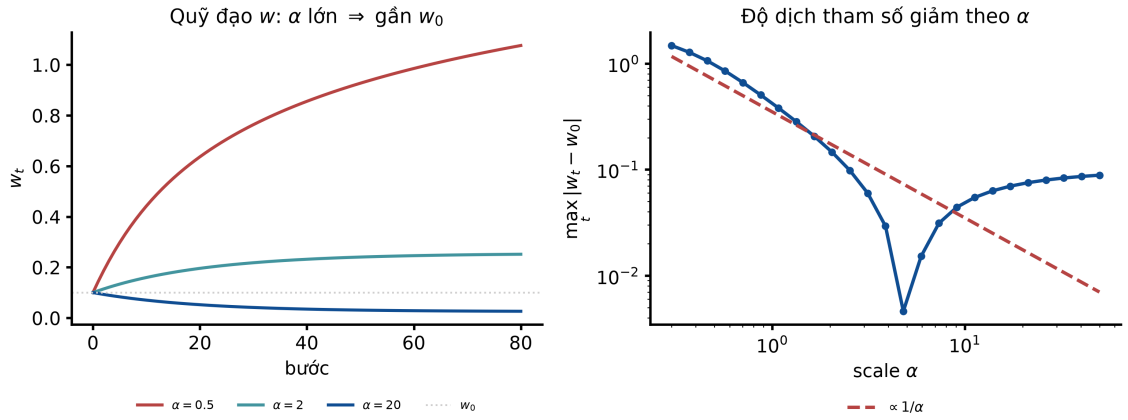
$\kappa_h(w_0) \ll 1$ là điều kiện **đủ** (không phải cần) để lazy training xảy ra (Định lý 2.3 định lượng chính xác hơn). Trực giác: κ_h nhỏ khi (a) điểm khởi tạo đã gần đích ($\|h(w_0) - y^\star\|$ nhỏ, qua việc scale α lớn), hoặc (b) mô hình "gần tuyến tính" cục bộ ($\|D^2 h\|$ nhỏ so với $\|Dh\|^2$).

Hệ quả quan trọng paper rút ra bằng thực nghiệm: CNN hiệu năng cao trong thực tế (VGG, ResNet trên CIFAR-10) **không** nằm trong, và khi bị **ép vào** chế độ lazy (bằng cách tăng α), hiệu năng **giảm mạnh**, không vượt qua được một số phương pháp kernel/feature cố định đơn giản. Điều này cho thấy lazy training **không giải thích** được vì sao mạng nơ-ron thực tế hoạt động tốt; thành công thực sự nằm ở chế độ phi tuyến, nơi "neuron di chuyển" (feature learning), điều mà tại thời điểm paper viết (2018-2019) vẫn **chưa có lý thuyết đầy đủ**.

3. Phương pháp chi tiết

3.1 Thiết lập và tiêu chí lazy training

Cho mô hình $h : \mathbb{R}^p \to \mathcal{F}$, loss $R : \mathcal{F} \to \mathbb{R}_+$, mục tiêu $F(w) := R(h(w))$ và mục tiêu tuyến tính hoá $\bar{F}(w) := R(\bar{h}(w))$. **Lazy training** là tình huống (không hiển nhiên) mà quỹ đạo tối ưu của F và $\bar{F}$ (xuất phát cùng w_0) **vẫn gần nhau cho tới khi thuật toán dừng**, chứ không chỉ gần nhau ở vài bước đầu (điều này luôn đúng với mọi mô hình trơn).



Lazy training: scale α lớn giữ w gần w_0; độ dịch max|w $-w_0$| giảm theo ∼1/α. (minh hoạ, tự tính)

Tiêu chí hình thức (Mục 1.2): xét một bước gradient descent $w_1 = w_0 - \eta \nabla F(w_0)$. Thay đổi tương đối của loss là $\Delta(F) \approx \eta \|\nabla F(w_0)\|^2 / F(w_0)$; thay đổi tương đối của Jacobian là $\Delta(Dh) \lesssim \eta \, \|\nabla F(w_0)\| \cdot \|D^2 h(w_0)\| / \|Dh(w_0)\|$. Lazy training xảy ra khi $\Delta(F) \gg \Delta(Dh)$ (loss giảm mạnh mà Jacobian gần như không đổi), dẫn tới điều kiện $\|\nabla F(w_0)\| / F(w_0) \gg \|D^2 h(w_0)\| / \|Dh(w_0)\|$, và với loss bình phương dẫn tới định nghĩa $\kappa_h(w_0)$ ở công thức (1).

3.2 Mô hình được scale lại (rescaled models)

Với hệ số scale $\alpha > 0$, xét $\kappa_{\alpha h}(w_0) = \frac{1}{\alpha}\|\alpha h(w_0) - y^\star\| \cdot \|D^2 h(w_0)\| / \|Dh(w_0)\|^2$. Nếu $\|\alpha h(w_0) - y^\star\|$ bị chặn (ví dụ $h(w_0) = 0$ nên không đổi theo α), thì $\kappa_{\alpha h}(w_0)$ giảm theo α^{-1}, tức α **lớn luôn dẫn tới lazy training**, không cần over-parameterization.

Mô hình thuần nhất bậc q (homogeneous): nếu $h(\lambda w) = \lambda^q h(w)$ với mọi $\lambda > 0$ (đúng với mạng q-layer gồm toàn phép tuyến tính không affine và phi tuyến thuần nhất, ví dụ ReLU), thì nhân khởi tạo với λ tương đương nhân scale α với λ^q: $\kappa_h(\lambda w_0) = \lambda^{-q}\|\lambda^q h(w_0) - y^\star\| \cdot \|D^2 h(w_0)\|/\|Dh(w_0)\|^2$. Vậy **tăng phương sai khởi tạo** cũng đẩy mạng vào chế độ lazy, miễn đầu ra ban đầu có norm bị chặn.

Mạng 2 lớp với m neuron: $h_m(w) = \alpha(m)\sum_{i=1}^m \phi(\theta_i)$, khởi tạo i.i.d. với $\mathbb{E}[\phi(\theta_i)] = 0$. Paper chứng minh (Phụ lục A.2, dựa trên luật số lớn):

$$\mathbb{E}[\kappa_{h_m}(w_0)] \lesssim m^{-1/2} + (m\,\alpha(m))^{-1} \qquad \text{(bound trong Mục 1.2)}$$

Vậy nếu $m\,\alpha(m) \to \infty$ khi $m \to \infty$ (ví dụ $\alpha(m) = 1/\sqrt{m}$), mạng **chắc chắn** rơi vào lazy khi m lớn, vì norm đầu ra ban đầu trở nên "không đáng kể" so với scale, do triệt tiêu thống kê (statistical cancellation) từ giả thiết trung bình 0. Ngược lại, **scale tới hạn** $\alpha(m) = 1/m$ cho giới hạn **mean-field** (khi $m \to \infty$) không suy biến, mô tả bởi một phương trình đạo hàm riêng, và **không** rơi vào lazy training (đây là chế độ "feature learning" đối lập, liên hệ math 34 "mean field view of landscape").

3.3 Các định lý chặn khoảng cách quỹ đạo

Gọi $w_\alpha(t)$ là gradient flow của $F_\alpha(w) := \frac{1}{\alpha^2}R(\alpha h(w))$ (scale lại objective để α lớn tương ứng tốc độ "đúng" của động lực học), và $\bar{w}_\alpha(t)$ là gradient flow của $\bar{F}_\alpha$ tương tự với mô hình tuyến tính hoá.

Định lý 2.2 (tổng quát, không cần lồi): nếu $h(w_0) = 0$, với mọi khoảng thời gian hữu hạn $[0, T]$:

$$\sup_{t\in[0,T]} \|w_\alpha(t) - w_0\| = O(1/\alpha), \quad \sup_{t\in[0,T]} \|w_\alpha(t) - \bar{w}_\alpha(t)\| = O(1/\alpha^2), \quad \sup_{t\in[0,T]} \|\alpha h(w_\alpha(t)) - \alpha\bar{h}(\bar{w}_\alpha(t))\| = O(1/\alpha).$$

Định lý 2.3 (loss bình phương, định lượng): với K đóng vai trò "số bước lặp" ($T = K/\mathrm{Lip}(h)^2$):

$$\frac{\|\alpha h(w_\alpha(T)) - \alpha\bar{h}(\bar{w}_\alpha(T))\|}{\|\alpha h(w_0) - y^\star\|} \le \frac{K^2\,\mathrm{Lip}(Dh)}{\alpha\,\mathrm{Lip}(h)^2}\,\|\alpha h(w_0) - y^\star\|$$

với điều kiện $\alpha \ge K\|\alpha h(w_0) - y^\star\|/(r\,\mathrm{Lip}(h))$. Vế trái là sai số tương đối giữa động lực phi tuyến và tuyến tính hoá; vế phải tỉ lệ nghịch với α, nên α **lớn làm sai số nhỏ**, nhưng hệ số K^2 cho thấy sai số **tăng theo bình phương số bước huấn luyện**.

Định lý 2.4 (over-parameterized, rank $Dh(w_0)$ **đầy/surjective):** với R là m-lồi mạnh, M-trơn (số điều kiện $\kappa = M/m$), nếu σ_{min} (giá trị kỳ dị nhỏ nhất của $Dh(w_0)^\top$) dương và α đủ lớn, $\alpha h(w_\alpha(t))$ hội tụ **tuyến tính** (hàm mũ) về $y^\star$ với tốc độ $\exp(-m\sigma_{min}^2 t/4)$ (sai số nhân với $\sqrt{\kappa}$), đồng thời $\sup_t \|w_\alpha(t) - \bar{w}_\alpha(t)\| = O(\log\alpha/\alpha^2)$.

Định lý 2.5 (under-parameterized): nếu F vô hạn chiều (ví dụ population risk) và rank $Dh(w)$ không đổi quanh w_0, tồn tại α_0 sao cho với $\alpha \ge \alpha_0$, gradient flow hội tụ với **tốc độ hình học không phụ thuộc** α về một **cực tiểu cục bộ** (không nhất thiết toàn cục) của F_α. Điểm quan trọng: kết quả này **không mâu thuẫn** với kết quả hội tụ toàn cục của Chizat & Bach (2018, mean-field, $p \to \infty$), vì Định lý 2.5 giả định p **hữu hạn**.

3.4 Kernel tương đương cho mạng 2 lớp

Với mạng 2 lớp $f_m(w, x) = \alpha(m)\sum_{j=1}^m b_j\,\sigma(a_j \cdot x)$, chọn $\alpha(m) = 1/\sqrt{m}$ (giới hạn không suy biến), tangent kernel $K_m = K_m^{(a)} + K_m^{(b)}$ (một phần cho mỗi lớp) hội tụ (khi $m \to \infty$) về kernel ngẫu nhiên-đặc trưng (random feature kernel). Với σ là ReLU và phân bố trọng số a_j đối xứng xoay (rotation invariant):

$$K^{(a)}(x,x') = \frac{(x\cdot x')\,\mathbb{E}(b^2)}{2\pi}(\pi-\varphi), \qquad K^{(b)}(x,x') = \frac{\|x\|\|x'\|\,\mathbb{E}(\|a\|^2)}{2\pi d}\big((\pi-\varphi)\cos\varphi+\sin\varphi\big) \quad (8)$$

với φ là góc giữa x, x'. $K^{(b)}$ (đạo hàm theo lớp output) là kernel truyền thống liên kết mạng nơ-ron với random features (Rahimi & Recht); $K^{(a)}$ (đạo hàm theo lớp ẩn) là phần mới hơn, đặc trưng cho tangent kernel đầy đủ.

4. Thí nghiệm và kết quả chính

Paper có 2 nhóm thí nghiệm số: (a) mạng 2 lớp trên dữ liệu tổng hợp (teacher-student), (b) CNN sâu (VGG-11, ResNet) trên CIFAR-10. Đây không phải paper lý thuyết thuần, có minh hoạ số thật.

Mạng 2 lớp, teacher-student (Hình 1, Hình 2). Student ReLU 2 lớp, teacher cùng kiến trúc với 3 neuron ẩn. Hình 1: với τ (độ lệch chuẩn khởi tạo) nhỏ ($\tau = 0,1$), gradient descent "tìm ra" đúng 3 hướng neuron giáo viên (non-lazy); với $\tau = 2$ (lazy), neuron chỉ di chuyển rất ít, nội suy được train loss về gần 0 nhưng **generalization xấu dần khi τ tăng** (Hình 1c, không gian 100 chiều), đạt một plateau trùng khớp với hiệu năng của mô hình tuyến tính hoá. Hình 2a: quét số neuron m với 2 cách scale $\alpha(m) = 1/\sqrt{m}$ (dẫn tới lazy, test error tăng theo m) so với $\alpha(m) = 1/m$ (mean-field, test error giữ gần 0 khi m tăng).

CNN trên CIFAR-10 (Mục 3.2, Hình 3). VGG-11 chuẩn (centered để $h(w_0) = 0$), huấn luyện objective $R(\alpha h(w))/\alpha^2$ với $\alpha \in \{10^0, ..., 10^7\}$, 70 epoch, batch 128. Kết quả chính (Bảng trong Hình 3b, số liệu lấy đúng từ paper):

Mô hình	Train acc. (%)	Test acc. (%)
ResNet wide, linearized ($\alpha = 10^7$)	55,0	56,7
VGG-11 wide, linearized ($\alpha = 10^7$)	61,0	61,7
Prior features (scattering transform, không học)	-	82,3
Random features (không học)	-	84,2
VGG-11 wide, standard ($\alpha = 1$)	99,9	89,7
ResNet wide, standard ($\alpha = 1$)	99,4	91,0

Nhận xét trực tiếp từ bảng: **mô hình linearized (lazy, $\alpha = 10^7$) kém hơn cả các phương pháp feature cố định không học (random features, scattering/prior features)**, và kém xa mô hình standard cùng kiến trúc. "Stability of activations" (tỉ lệ neuron ReLU giữ đúng dấu kích hoạt từ đầu đến cuối huấn luyện) tiến gần 100% khi α tăng, xác nhận mô hình thực sự tuyến tính hoá khi α lớn (Hình 3a).

Phân tích phổ tangent kernel (Phụ lục C.3, Hình 7). Lý do linearized VGG train accuracy không về 0 dù over-parameterized: phổ giá trị riêng σ_i^2 của $Dh(w_0)Dh(w_0)^\top$ có **khoảng cách 1 bậc độ lớn giữa 0,2% giá trị riêng lớn nhất và phần còn lại**, sau đó giảm theo $O(1/i)$ (quan sát giống nhau trên ảnh CIFAR-10 thật và ảnh pixel ngẫu nhiên, cho thấy đây là tính chất nội tại của kiến trúc VGG tuyến tính hoá, không phụ thuộc dữ liệu), gây ra hiện tượng **điều kiện số kém (ill-conditioning)**.

5. Phân tích / ablation

Vai trò over- vs. under-parameterization. Định lý 2.4 (over-parameterized) cho hội tụ **toàn cục**; Định lý 2.5 (under-parameterized, ví dụ population risk vô hạn chiều) chỉ cho hội tụ về **cực tiểu cục bộ**, minh chứng thực nghiệm ở Hình 2b: với $d = 100$, $m = 50$ (dưới tham số hoá), SGD trên population loss hội tụ tốt khi τ nhỏ nhưng mắc ở cực tiểu cục bộ khi τ lớn (lazy); có cả vùng trung gian (hatched area trong Hình 2b) hội tụ rất chậm, chưa kết thúc khi dừng thí nghiệm.

Widening không cứu được lazy regime. Thử mở rộng CNN (x8 kênh cho VGG, x7 cho ResNet) để tăng over-parameterization trong chế độ lazy ($\alpha = 10^7$): hiệu năng linearized nhúc nhích lên (VGG: 61,7% như bảng trên) nhưng **vẫn kém đáng kể** so với random/prior features và kém xa non-lazy, cho thấy vấn đề không chỉ là thiếu tham số mà là **điều kiện số của tangent kernel cố định** (phổ riêng lệch mạnh, xem Hình 7).

Ghi chú về khởi tạo chuẩn (standard init). Paper trích Jacot et al. (2018, NTK): mạng khởi tạo chuẩn (phương sai $O(1/n_\ell)$ ở lớp ℓ) **tất yếu** tiến tới lazy khi mọi lớp rộng vô hạn, và cách khởi tạo này **gần như bắt buộc** để tránh gradient nổ/biến mất ở mạng rất sâu (trích He et al. 2015, Hanin & Rolnick 2018). Bài báo nhấn mạnh: **không phủ nhận** mạng rộng có xu hướng lazy, chỉ khẳng định các mạng **hoạt động tốt trong thực tế còn cách xa giới hạn tiệm cận đó**.

6. Giới hạn

Bài báo nêu một số giới hạn: (a) các chặn lý thuyết cho $\kappa_h(w_0)$ chỉ là điều kiện **đủ**, không phải cần, nên lazy training "có thể xảy ra ngay cả khi κ_h lớn" (ghi chú footnote 3); (b) chặn của Định lý 2.2 phụ thuộc **hàm mũ** vào horizon T (hằng số ẩn tăng theo $\exp(T)$), chỉ Định lý 2.3 (loss bình phương) loại bỏ được sự phụ thuộc này; (c) Định lý 2.5 (under-parameterized) chỉ dừng ở phát biểu **định tính** (tồn tại α_0), không cho công thức tường minh vì liên quan tới giá trị kỳ dị dương nhỏ nhất của $Dh(w_0)$, "khó kiểm soát" theo lời paper; (d) câu hỏi "liệu có thể thiết kế kiến trúc/pipeline khác để linearized CNN cạnh tranh được với non-lazy" được paper để **mở**, chỉ trích một kết quả khác (Arora et al. 2019, kiến trúc đặc biệt giải quyết ill-conditioning, đạt 77,4% không data augmentation) nhưng không tự thử nghiệm.

Một lưu ý khi đọc: đoạn Phụ lục C.3 trong bản PDF gốc dùng dấu gạch ngang dài kiểu typeset (nguyên văn có đoạn chêm ý kiểu "...which causes the ill conditionning, and then a decrease...", bản gốc dùng ký hiệu chêm câu khác) để chêm ý trong câu; đây là văn phong gốc của paper, không phải lỗi, nhưng ghi chú này diễn đạt lại bằng dấu phẩy/ngoặc theo quy tắc không dùng em dash.

7. Liên hệ với các bài khác

- **math 23 (Neural Tangent Kernel)**: bài 24 dùng lại đúng khái niệm tangent kernel $K(x, x') = D_w f(w_0, x) D_w f(w_0, x')^\top$ của bài 23, nhưng đặt câu hỏi khác: bài 23 hỏi "giới hạn width vô hạn thì NTK có tính chất gì", còn bài 24 hỏi "với mạng hữu hạn, khi nào và vì sao mô hình rơi vào chế độ giống NTK (lazy), và chế độ đó có tốt không". Bài 24 cũng chỉ ra rằng bản thân khái niệm "lazy" rộng hơn neural network, áp dụng cho mọi "differentiable programming".
- **math 22 (Deep NN as Gaussian Processes)**: paper trích dẫn [20,23] (Lee et al., Matthews et al.) về việc phân phối $h(w_0)$ hội tụ về Gaussian process khi mạng over-parameterized với khởi tạo chuẩn, dùng để giải thích nguồn gốc ngẫu nhiên của "offset" $h(w_0)$ trong kernel method tương đương (Mục A.1).
- **math 19 (Measuring the Intrinsic Dimension of Objective Landscapes)**: cả hai bài cùng quan tâm "mạng cần di chuyển/dùng bao nhiêu độ tự do để học", nhưng bài 19 đo tĩnh (subspace cố định, không phụ thuộc động lực học), còn bài 24 đo động (quỹ đạo gradient flow di chuyển bao xa khỏi w_0, cụ thể $\|w_\alpha(t) - w_0\| = O(1/\alpha)$ trong chế độ lazy).
- **math 25 (Principles of Deep Learning Theory)**: sách này mô tả chính xác chế độ "$r = L/n \to 0$" (infinite-width) là nơi mạng hành xử tuyến tính và **mất khả năng học biểu diễn** (representation learning), đúng là định nghĩa "lazy" của bài 24 nhìn từ góc độ width/depth thay vì scale α của output; sách cũng dùng khai triển $1/n$ để mô tả "độ lệch khỏi lazy" tương tự cách bài 24 dùng khai triển $1/\alpha$.

8. Gợi ý áp dụng

Khung lazy training cho phép so sánh mô hình phi tuyến với tuyến tính hóa quanh điểm khởi tạo. Mức độ gần đúng phụ thuộc tham số hóa và thang đo; theo dõi sự thay đổi tham số hoặc biểu diễn có thể hỗ trợ khảo sát chế độ huấn luyện.

9. Thí nghiệm gợi ý

1. **Tái lập Hình 1 của paper ở quy mô nhỏ hơn:** mạng ReLU 2 lớp, $n = 15$ điểm dữ liệu tổng hợp, $m = 20$ neuron, teacher 3 neuron, quét $\tau \in \{0,1;\ 0,5;\ 1;\ 2;\ 5\}$. Đo: quỹ đạo $|a_j(t)|b_j(t)$ trong không gian 2D và test loss cuối cùng. Kỳ vọng: τ nhỏ tìm đúng hướng teacher (non-lazy), τ lớn chỉ dịch chuyển nhẹ quanh gốc (lazy), test loss tăng theo τ. Diễn giải: xác nhận hiện tượng cơ bản trước khi tin vào các kết luận CNN phức tạp hơn.
2. **Đo $\kappa_h(w_0)$ (công thức 1) trực tiếp bằng autograd cho một MLP nhỏ** (2-3 lớp, vài trăm tham số) ở vài mức phương sai khởi tạo τ^2, dùng Hessian-vector product để ước lượng $\|D^2h(w_0)\|$ (hoặc xấp xỉ bằng finite difference của Jacobian). Đo: κ_h có giảm theo $1/\tau^q$ (q là bậc thuần nhất) như công thức Mục 3.2 dự đoán không? Diễn giải: kiểm tra trực tiếp công thức lý thuyết bằng số, một bài tập autograd hữu ích.
3. **Huấn luyện VGG nhỏ (hoặc ResNet-8) trên CIFAR-10 với vài giá trị $\alpha \in \{1, 10, 100, 1000\}$** trên GPU (batch nhỏ, vài epoch), đo train/test accuracy và "stability of activations" (tỉ lệ neuron ReLU giữ dấu kích hoạt từ đầu đến cuối). Kỳ vọng: giống xu hướng Hình 3a của paper (accuracy giảm, stability tăng khi α tăng), dù quy mô nhỏ hơn nhiều so với paper (không cần tới $\alpha = 10^7$ hay widen x7-x8 để thấy xu hướng). Diễn giải: xác nhận hiện tượng ở quy mô nhỏ.
4. **Fine-tune một adapter nhỏ (LoRA rank thấp) trên một mô hình ngôn ngữ nhỏ cho 1 tác vụ, với 2 learning rate khác nhau (rất nhỏ vs. bình thường)**, đo khoảng cách tham số $\|w(T) - w_0\|$ và hiệu năng cuối. Kỳ vọng: learning rate rất nhỏ giữ tham số gần w_0 (giống "lazy" quanh checkpoint) nhưng có thể đạt hiệu năng tốt nếu checkpoint ban đầu đã gần tối ưu cho tác vụ, trái với kết luận "lazy luôn xấu" của paper (vì bối cảnh khác: không phải khởi tạo ngẫu nhiên). Diễn giải: minh hoạ sự khác biệt giữa "lazy quanh init ngẫu nhiên" (xấu, theo paper) và "ít thay đổi quanh checkpoint đã huấn luyện tốt" (có thể tốt, tuỳ tác vụ).

10. Tóm tắt công thức / khái niệm

- **Linearization:** $\bar{h}(w) = h(w_0) + Dh(w_0)(w - w_0)$; tangent kernel $K(x, x') = D_w f(w_0, x) D_w f(w_0, x')^\top$ (7).
- **Inverse relative scale (điều kiện đủ cho lazy, loss bình phương):** $\kappa_h(w_0) = \|h(w_0) - y^\star\| \cdot \|D^2h(w_0)\| / \|Dh(w_0)\|^2$ (1); $\kappa_h \ll 1 \Rightarrow$ lazy.
- **Rescaled objective:** $F_\alpha(w) = \frac{1}{\alpha^2} R(\alpha h(w))$ (2); $\kappa_{\alpha h}(w_0)$ giảm $\propto \alpha^{-1}$ khi $\alpha \to \infty$ (nếu $\|\alpha h(w_0) - y^\star\|$ bị chặn).
- **Mô hình thuần nhất bậc q:** nhân khởi tạo λ tương đương nhân scale α với λ^q; mạng q-layer ReLU/homogeneous rơi vào lazy khi phương sai khởi tạo tăng.
- **Chặn chính (Định lý 2.2, 2.3):** $\sup_t \|w_\alpha(t) - \bar{w}_\alpha(t)\| = O(1/\alpha^2)$ (horizon hữu hạn, tổng quát); với loss bình phương, sai số tương đối $\leq (K^2 \mathrm{Lip}(Dh)/\alpha \,\mathrm{Lip}(h)^2) \cdot \|\alpha h(w_0) - y^\star\|$.
- **Hội tụ toàn cục (over-parameterized, Định lý 2.4):** $\|\alpha h(w_\alpha(t)) - y^\star\| \leq \sqrt{\kappa}\, \|\alpha h(w_0) - y^\star\| \exp(-m\sigma_{min}^2 t/4)$.
- **Tự tính (ví dụ số nhỏ minh hoạ công thức 1):** giả sử một mô hình toy có $\|h(w_0) - y^\star\| = 2$, $\|D^2h(w_0)\| = 0,5$, $\|Dh(w_0)\| = 10$. Khi đó $\kappa_h(w_0) = 2 \times 0,5/10^2 = 1/100 = 0,01 \ll 1$, nên theo tiêu chí (1), mô hình này (với các số giả định trên) đang ở chế độ lazy mạnh. Nếu scale h lên $\alpha = 5$ (giữ $y^\star$ cố định), theo công thức Mục 3.2 ta có $\kappa_{\alpha h}(w_0) = \kappa_h(w_0)/\alpha = 0,01/5 = 0,002$, càng nhỏ hơn, tức lazy hơn nữa. Đây là số tự chọn để minh hoạ cách dùng công thức, không phải số đo thực từ paper.

11. Kết luận ngắn

Bài này tách rõ "lazy training" (mô hình phi tuyến hành xử như tuyến tính hoá quanh điểm khởi tạo) ra khỏi "over-parameterization": lazy chủ yếu do **lựa chọn scale** (ngầm định qua cách khởi tạo/chuẩn hoá), và có thể xảy ra ở mọi kích thước mô hình khi scale đủ lớn. Bằng cả lý thuyết (các chặn với horizon hữu hạn và vô

hạn, cho cả trường hợp over- và under-parameterized) và thực nghiệm (CNN thật trên CIFAR-10), bài báo đưa ra một kết luận mang tính phản biện quan trọng cho cả nhánh lý thuyết NTK: **chế độ lazy, dù dễ phân tích, không giải thích được vì sao mạng nơ-ron hoạt động tốt trong thực tế**; chìa khoá thành công thực sự nằm ở chế độ phi tuyến (neuron di chuyển, feature learning), vẫn còn thiếu lý thuyết đầy đủ tại thời điểm paper được viết.

Nguồn gốc: https://arxiv.org/abs/1812.07956

Bài 25

The Principles of Deep Learning Theory (2021)

Tác giả: Daniel A. Roberts, Sho Yaida (nghiên cứu dựa trên hợp tác với Boris Hanin).
Venue / arXiv: arXiv:2106.10165 (v2, 24/08/2021); paper không nêu venue hội nghị/journal. Sau này nội dung được Cambridge University Press in thành sách, nhưng đó là nhà xuất bản sách, không phải ”venue” theo nghĩa hội nghị khoa học.
Nguồn: https://arxiv.org/abs/2106.10165
Chủ đề: Đây là một cuốn sách giáo trình dài khoảng 470 trang, xây dựng một ”lý thuyết hiệu dụng” (effective theory, mượn từ vật lý) cho mạng nơ-ron sâu tại thời điểm khởi tạo và trong lúc huấn luyện, với tham số trung tâm là tỉ lệ độ sâu trên độ rộng $r = L/n$. **Quan trọng:** ghi chú này CHỈ đọc kỹ các phần sau: Preface, Chương 0 (Initialization), Chương 3 (Effective Theory of Deep Linear Networks, toàn bộ §3.1 đến §3.4), Chương 5 §5.1-5.2 (Criticality Analysis of the Kernel, Criticality for Scale-Invariant Activations), Epilogue, Appendix A (Information in Deep Learning), Appendix B (Residual Learning). KHÔNG đọc Chương 1, 2, 4, 6, 7-9, 10, 11, ∞ (Pretraining, Neural Networks, phần còn lại của Effective Theory of Deep Nonlinear Networks, RG flow, Gradients/NTK chi tiết, Kernel Prediction, Representation Learning, End of Training) và phần References/Index. Đây là lựa chọn đọc ”core parts” cho một cuốn sách rất dài, theo đúng tinh thần brief cho phép với tài liệu khảo sát/sách dài.
Vị trí trong lộ trình: nên đọc sau math 26 (Poole et al., transient chaos) và math 27 (Schoenholz et al., Deep Information Propagation), vì PDLT kế thừa trực tiếp khái niệm criticality/edge-of-chaos của hai bài đó (chính PDLT ghi trong Chương 5 rằng ”initial part of this analysis was first carried out in a series of papers” dùng kỹ thuật khác nhưng tương đương ở giới hạn vô hạn rộng) và hệ thống hoá nó bằng một khung lý thuyết nhiễu loạn chặt chẽ hơn. Cũng nên đọc sau math 22 (Deep Neural Networks as Gaussian Processes, NNGP) và math 23 (Neural Tangent Kernel), vì Chương 0 và Epilogue của PDLT dùng lại đúng hai khái niệm NNGP và NTK làm hai đối tượng trung tâm của giới hạn vô hạn rộng. Nên đọc cùng nhóm với math 19 (Intrinsic Dimension) và math 24 (Lazy Training) vì cả ba đều xoay quanh câu hỏi ”độ phức tạp hiệu dụng của mô hình là gì, nếu không phải là số tham số”.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (bản arXiv); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc.

1. Bối cảnh

Khoảng năm 2021, hai hướng lý thuyết về mạng sâu tại thời điểm khởi tạo được nghiên cứu song song. Ở giới hạn độ rộng vô hạn, đầu ra tiến tới Gaussian process (NNGP, bài 22) và động lực gradient descent được mô tả bằng Neural Tangent Kernel (NTK, bài 23). Hướng lan truyền tín hiệu và edge of chaos của Poole và cộng sự (bài 26), Schoenholz và cộng sự (bài 27) nghiên cứu những giá trị tới hạn của phương sai trọng số và bias giúp tín hiệu tránh tăng hoặc suy giảm theo cấp số nhân theo độ sâu.

PDLT đặt câu hỏi: có thể xây một khung lý thuyết DUY NHẤT, chặt chẽ về mặt toán học, vừa chứa cả hai nhánh trên như các trường hợp riêng, vừa cho phép tính toán có hệ thống các hiệu ứng độ rộng hữu hạn (finite-width) để mạng vẫn ”học được representation” (biểu diễn) thay vì chỉ là một bộ làm mượt tuyến tính (linear smoother) không đổi theo dữ liệu? Cách tiếp cận của sách là mượn nguyên ”effective theory approach” từ vật lý: giống như nhiệt động lực học (thermodynamics) là một lý thuyết ”hiệu dụng” ở quy mô lớn, mô tả máy hơi nước mà không cần theo dõi từng phân tử khí riêng lẻ (đó là việc của cơ học thống kê, statistical mechanics), các tác giả muốn một lý thuyết hiệu dụng mô tả hành vi mạng nơ-ron ở quy mô ”nhiều neuron, nhiều lớp” mà không cần theo dõi từng tham số riêng lẻ.

Thuật ngữ cần nắm trước: **lý thuyết hiệu dụng (effective theory)** là một lý thuyết xấp xỉ, chỉ đúng và chỉ cần đúng trong một phạm vi (scale) nhất định, bỏ qua chi tiết vi mô không ảnh hưởng tới hành vi vĩ mô ở phạm vi đó. **Tới hạn (criticality)**, mượn từ lý thuyết chuyển pha trong vật lý thống kê (ví dụ sắt từ chuyển từ có từ tính sang không có từ tính tại một nhiệt độ tới hạn), ở đây chỉ việc tinh chỉnh siêu tham số khởi tạo (initialization hyperparameters) tới một điểm đặc biệt mà hệ thống không ”đổ” về một trong hai hành vi tầm thường (bùng nổ hoặc triệt tiêu theo cấp số nhân) khi độ sâu tăng.

2. Ý tưởng chính

Toàn bộ sách xoay quanh việc khai triển Taylor hàm output đã huấn luyện $z(x_\delta;\theta^\star)$ quanh điểm khởi tạo θ:

$$z(x_\delta;\theta^\star) = z(x_\delta;\theta) + \sum_{\mu}(\theta^\star-\theta)_\mu \frac{dz_\delta}{d\theta_\mu} + \frac{1}{2}\sum_{\mu,\nu}(\theta^\star-\theta)_\mu(\theta^\star-\theta)_\nu \frac{d^2 z_\delta}{d\theta_\mu d\theta_\nu} + \dots$$

Khai triển này (Epilogue gọi lại là eq ε.2, tương ứng eq 0.5 ở Chương 0) gặp ba ”Vấn đề” được paper nêu rõ ngay từ Chương 0 (eq 0.6, 0.7, 0.8):

- **Vấn đề 1:** về nguyên tắc phải tính vô hạn số hạng đạo hàm $dz/d\theta, d^2z/d\theta^2, d^3z/d\theta^3, \dots$
- **Vấn đề 2:** cần biết phân phối đồng thời (joint distribution) của output z và tất cả các đạo hàm đó, ánh xạ từ phân phối khởi tạo $p(\theta)$ sang phân phối của $\left(z, dz/d\theta, d^2z/d\theta^2, \dots\right)$, nói chung không khả giải (intractable).
- **Vấn đề 3:** điểm hội tụ $\theta^\star$ sau huấn luyện phụ thuộc vào gần như mọi thứ: thuật toán học, learning rate, dữ liệu huấn luyện, v.v.

Sách đưa ra hai cách giải quyết bộ ba vấn đề này, cả hai đều dựa trên một quan sát gọi là **”nguyên lý thưa” (principle of sparsity)**: khi số neuron n mỗi lớp lớn, rất nhiều tương tác giữa các neuron tự triệt tiêu lẫn nhau (tương tự định luật số lớn), làm phân phối trở nên đơn giản hơn hẳn so với khi n nhỏ.

- **Cách 1 (giới hạn vô hạn rộng nghiêm ngặt, $n \to \infty$):** giải quyết triệt để cả 3 vấn đề, cho nghiệm closed-form (dạng Gaussian, đây chính là NNGP và NTK). Tuy nhiên cái giá phải trả: trong giới hạn này các biểu diễn ẩn (hidden representations) ở mọi lớp không thay đổi so với lúc khởi tạo trong suốt quá trình huấn luyện, nghĩa là mạng **không học được representation** theo nghĩa thông thường, chỉ là một bộ hồi quy tuyến tính trên các đặc trưng ngẫu nhiên cố định (random features).
- **Cách 2 (khai triển nhiễu loạn $1/n$, giữ n lớn nhưng hữu hạn):** giữ lại hiệu ứng độ rộng hữu hạn ở bậc dẫn đầu (leading order). Paper viết dưới dạng một khai triển cho phân phối của output đã huấn luyện (eq 0.14-0.15 ở Chương 0):

$$p(f^\star) = p^{\{0\}}(f^\star) + \frac{1}{n}p^{\{1\}}(f^\star) + O\left(\frac{1}{n^2}\right)$$

Số hạng $p^{\{0\}}$ chính là nghiệm Gaussian của Cách 1; số hạng $p^{\{1\}}/n$ là hiệu chỉnh độ rộng hữu hạn đầu tiên, và CHÍNH số hạng này mang lại khả năng học representation thật sự.

Điểm mấu chốt của toàn sách: khi thực hiện khai triển $1/n$ này cho một mạng có L lớp, tham số khai triển thực sự xuất hiện không phải là $1/n$ đơn thuần, mà là tỉ lệ

$$r \equiv \frac{L}{n}$$

gọi là **aspect ratio** (tỉ lệ độ sâu trên độ rộng, eq 0.19 ở Chương 0). Ba miền giá trị của r (thuật ngữ chính xác từ sách, giải thích thêm bằng tiếng Việt):

- $r \to 0$: mạng hành xử "giống vô hạn rộng" (infinite-width-like); dù L có lớn, nếu n lớn hơn nhiều thì $r \approx 0$ và mạng "không thực sự sâu" theo nghĩa hiệu dụng.
- $0 < r \ll 1$: mạng "hiệu dụng là sâu" (effectively deep); đây là vùng mà khai triển $1/n$ cắt ở bậc 1 chính xác, và mạng thực tế hay rơi vào vùng này.
- $r \gg 1$: mạng "quá sâu" (overly deep); các neuron tương tác mạnh (chaotic), không còn mô tả hiệu dụng nào có giá trị thực tế, và theo paper, mạng như vậy "vô dụng" (useless) trong thực hành.

3. Phương pháp chi tiết

Mạng tuyến tính sâu làm mô hình đồ chơi (Chương 3)

Sách gọi deep linear network là "giống dao động điều hòa đơn giản (simple harmonic oscillator) đối với deep learning": một mô hình đồ chơi tối giản nhưng vẫn lộ ra toàn bộ cấu trúc khái niệm (criticality, chaos, aspect ratio) mà các chương sau sẽ tổng quát hoá. Mạng được định nghĩa bởi (eq 3.1, bias đặt bằng 0 trong chương này):

$$z_{i;\alpha}^{(\ell+1)} = \sum_j W_{ij}^{(\ell+1)} z_{j;\alpha}^{(\ell)}$$

với thống kê trọng số (eq 3.4): $E[W_{ij}^{(\ell)}] = 0$ và $E[W_{i_1 j_1}^{(\ell)} W_{i_2 j_2}^{(\ell)}] = \frac{C_W}{n_{\ell-1}} \delta_{i_1 i_2} \delta_{j_1 j_2}$ (chú ý hệ số $1/n_{\ell-1}$: đây là cách khởi tạo "chuẩn hoá theo độ rộng" phổ biến, để phương sai output không phụ thuộc n).

Correlator hai điểm (two-point correlator) $G_{\alpha_1\alpha_2}^{(\ell)} \equiv \frac{1}{n_\ell} \sum_i E[z_{i;\alpha_1}^{(\ell)} z_{i;\alpha_2}^{(\ell)}]$ đo "độ giống nhau" trung bình giữa preactivation của hai mẫu đầu vào α_1, α_2 ở lớp ℓ. Dùng **Wick contraction** (một hệ quả của việc các biến W là Gaussian: kỳ vọng của tích 4 biến Gaussian phân rã thành tổng các tích từng cặp hiệp phương sai), paper suy ra công thức đệ quy đơn giản (eq 3.14): $G_{\alpha_1\alpha_2}^{(\ell+1)} = C_W G_{\alpha_1\alpha_2}^{(\ell)}$, giải chính xác (eq 3.15): $G^{(\ell)} = (C_W)^\ell G^{(0)}$.

Đây là chỗ khái niệm **criticality** xuất hiện lần đầu, dưới dạng đơn giản nhất: nếu $C_W > 1$, $G^{(\ell)}$ bùng nổ theo cấp số nhân khi ℓ tăng (điểm cố định tầm thường $G^\star = \infty$); nếu $C_W < 1$, $G^{(\ell)}$ triệt tiêu theo cấp số nhân ($G^\star = 0$, mọi thông tin phân biệt giữa các mẫu bị xoá sạch). Chỉ tại $C_W = 1$ (gọi là **critical initialization**), correlator được bảo toàn đúng nguyên giá trị ban đầu qua mọi lớp: đây là "điểm cố định không tầm thường" (nontrivial fixed point), tương tự điểm chuyển pha trong vật lý thống kê.

Correlator bốn điểm và dao động (fluctuations, §3.3): ở độ rộng hữu hạn, correlator bậc 4 có đệ quy (eq 3.24): $G_4^{(\ell+1)} = C_W^2 \left(1 + \frac{2}{n_\ell}\right) G_4^{(\ell)}$, giải ra (eq 3.25): $G_4^{(\ell)} = \left[\prod_{\ell'} \left(1 + \frac{2}{n_{\ell'}}\right)\right] \left(G_2^{(\ell)}\right)^2$. Ở giới hạn vô hạn rộng, số hạng $2/n_{\ell'}$ biến mất, cho $G_4 = (G_2)^2$: đây đúng là tính chất của phân phối Gaussian thuần tuý (không có tương quan "dư" giữa các neuron trong cùng lớp). Ở độ rộng hữu hạn, tại đúng điểm tới hạn $C_W = 1$, độ lệch khỏi tính Gaussian (non-Gaussianity) ở bậc dẫn đầu là (eq 3.28):

$$\frac{2(\ell-1)}{n} \left(G_2^{(\ell)}\right)^2 + O\left(\frac{1}{n^2}\right)$$

Bài báo nhấn mạnh đây "có lẽ là chủ đề lặp lại quan trọng nhất của toàn sách": độ lệch này tỉ lệ thuận với độ sâu, tỉ lệ nghịch với độ rộng, nghĩa là tỉ lệ với chính $r = L/n$. Về ý nghĩa: đây chính là tương tác thực sự giữa các neuron trong một lớp (intralayer interaction), và chính nó là thứ cho phép mạng học representation; mạng vô hạn rộng (không có số hạng này) không thể có tương tác neuron-neuron nên không học được representation theo nghĩa phi tầm thường.

Hỗn loạn (chaos, §3.4): tổng quát hoá cho correlator bậc $2m$ bất kỳ, đệ quy có dạng (eq 3.45): $G_{2m}^{(\ell+1)} = c_{2m}(n_\ell) C_W^m G_{2m}^{(\ell)}$ với $c_{2m}(n)$ là một hệ số tổ hợp học (eq 3.46). Hai cách lấy giới hạn cho kết quả khác nhau hẳn, và đây là điểm rất quan trọng: nếu lấy $n \to \infty$ trước (giữ L cố định), $c_{2m} \to 1$, output hoàn toàn Gaussian, và mạng như vậy (theo cách diễn giải của sách) thực chất tương đương một mạng 1 lớp, "không

thực sự sâu". Ngược lại nếu lấy $L \to \infty$ trước (giữ n cố định), ngay cả tại đúng điểm tới hạn $C_W = 1$, các correlator bậc cao vẫn bùng nổ theo cấp số nhân theo L: mạng hoàn toàn hỗn loạn (chaotic), không Gaussian, và theo lời paper là "không dùng được" trên thực tế. Hai giới hạn này không giao hoán (eq 3.51).

Paper giải quyết bằng cách lấy **giới hạn kép** (double-scaling limit): cho cả $n, L \to \infty$ nhưng giữ $r = L/n$ cố định (eq 3.52). Dùng khai triển $c_{2m}(n) = 1 + \frac{m(m-1)}{n} + O(1/n^2)$ (eq 3.53) và công thức giới hạn mũ $\lim_{L\to\infty}(1 + a/L)^L = e^a$ (eq 3.54), kết quả là (eq 3.55):

$$G_{2m}^{(L)} \to e^{m(m-1)r} \left(G_2^{(L)}\right)^m$$

Công thức này nội suy liên tục giữa giới hạn Gaussian ($r \to 0$) và giới hạn hỗn loạn ($r \to \infty$), với r là tham số duy nhất điều khiển vị trí trên "thanh trượt" đó. Tại criticality, correlator liên thông (connected correlator, phần "mới" sau khi trừ đi phần suy ra được từ các correlator bậc thấp hơn) bậc 4 là $(e^{2r} - 1)(G_2^{(0)})^2$, xấp xỉ $2r(G_2^{(0)})^2$ khi r nhỏ (eq 3.56); correlator liên thông bậc 6 xấp xỉ $12r^2(G_2^{(0)})^3$ (eq 3.57). Các correlator liên thông bậc càng cao bị triệt giảm bởi luỹ thừa càng cao của CÙNG MỘT tham số nhỏ r: đây là cấu trúc "phân bậc" (hierarchical) làm cho việc cắt khai triển nhiễu loạn ở một bậc cố định là hợp lý khi r nhỏ.

Tự tính (ví dụ minh họa bằng số): lấy một mạng tuyến tính cụ thể với $L = 20$ lớp, $n = 200$ neuron mỗi lớp, chuẩn hoá $G_2^{(0)} = 1$. Tính $r = L/n = 20/200 = 0.1$ (tự tính). Giá trị đúng của correlator liên thông bậc 4 tại criticality: $e^{2r} - 1 = e^{0.2} - 1$. Khai triển Taylor $e^{0.2} = 1 + 0.2 + \frac{0.2^2}{2} + \frac{0.2^3}{6} + \frac{0.2^4}{24} \approx 1 + 0.2 + 0.02 + 0.001333 + 0.0000667 \approx 1.2214$ (tự tính), nên $e^{0.2} - 1 \approx 0.2214$ (tự tính). Xấp xỉ bậc nhất mà paper dùng cho r nhỏ là $2r = 0.2$. Sai số tương đối giữa hai giá trị: $(0.2214 - 0.2)/0.2214 \approx 9.7\%$ (tự tính). Nhận xét riêng (suy luận của người ghi chú, không phải kết luận paper nêu rõ bằng số): ngay ở $r = 0.1$, một tỉ lệ sâu/rộng nghe có vẻ "an toàn", xấp xỉ tuyến tính bậc nhất theo r đã lệch gần 10% so với giá trị đúng, cho thấy biên an toàn thực tế của khai triển nhiễu loạn hẹp hơn cảm giác "chỉ cần $r < 1$".

Criticality cho activation phi tuyến tổng quát (Chương 5, §5.1-5.2)

Với activation phi tuyến $\sigma(z)$ bất kỳ, kernel (ký hiệu K, chính là giới hạn vô hạn rộng của G) có đệ quy (eq 5.1): $K_{\alpha\beta}^{(\ell+1)} = C_b + C_W \langle \sigma_\alpha \sigma_\beta \rangle_{K^{(\ell)}}$, trong đó $\langle \cdot \rangle_K$ là kỳ vọng Gaussian với ma trận hiệp phương sai K. Khác với mạng tuyến tính, với activation phi tuyến xuất hiện một phức tạp mới: với hai mẫu đầu vào khác nhau $\alpha \neq \beta$, thành phần ngoài đường chéo $K_{\alpha\beta}$ bị trộn với các thành phần trên đường chéo $K_{\alpha\alpha}, K_{\beta\beta}$ (paper minh hoạ bằng ví dụ activation bậc hai $\sigma(z) = z^2$, eq 5.3, cho thấy cần giải 3 phương trình đệ quy phi tuyến liên kết nhau).

Một đầu vào: định nghĩa hàm trợ giúp $g(K) \equiv \langle \sigma(z)\sigma(z) \rangle_K$ (eq 5.5), đệ quy một-đầu-vào là $K_{00}^{(\ell+1)} = C_b + C_W\, g\left(K_{00}^{(\ell)}\right)$ (eq 5.4). Tuyến tính hoá quanh điểm cố định $K_{00}^\star$ (nghiệm của eq 5.7) cho **susceptibility song song (parallel susceptibility)** $\chi_\parallel(K) \equiv C_W g'(K)$ (eq 5.10), và điều kiện tới hạn đầu tiên là $\chi_\parallel(K_{00}^\star) = 1$ (eq 5.11). Paper liệt kê ba kiểu criticality có thể xảy ra tuỳ activation: (i) một đường điểm cố định được bảo toàn chính xác (giống mạng tuyến tính), xảy ra với activation scale-invariant (§5.2 dưới đây); (ii) suy giảm theo luật lũy thừa (power law) về $K^\star = 0$, xảy ra với tanh, sin (§5.3.3, không đọc chi tiết); (iii) suy giảm luật lũy thừa về $K^\star \neq 0$, xảy ra với SWISH, GELU (§5.3.4, không đọc chi tiết).

Hai đầu vào: phân tích ma trận kernel 2×2 (ứng với 2 mẫu) trong cơ sở $\gamma^{[0]}, \gamma^{[1]}, \gamma^{[2]}$ (eq 5.15-5.16), với ý nghĩa vật lý: $K_{[0]}$ là độ lớn trung bình ("khối tâm", eq 5.17), $K_{[1]} = R/2$ là nửa hiệu độ lớn hai đầu vào (eq 5.18), $K_{[2]} = D/4$ liên quan bình phương khoảng cách giữa hai đầu vào sau khi lan truyền qua ℓ lớp (eq 5.19). Sau một phép tính khá dài (khai triển quanh giới hạn hai đầu vào trùng nhau, dùng phân tích trị riêng của kernel), paper thu được ba đệ quy "bootstrapped" (giải lần lượt, cái sau dùng nghiệm cái trước, eq 5.46-5.48):

$$
\begin{aligned}
K_{00}^{(\ell+1)} &= C_b + C_W\, g\Big(K_{00}^{(\ell)}\Big)\,, \\
\delta K_{[1]}^{(\ell+1)} &= \chi_{\parallel}\Big(K_{00}^{(\ell)}\Big)\,\delta K_{[1]}^{(\ell)}\,, \\
\delta\delta K_{[2]}^{(\ell+1)} &= \chi_{\perp}\Big(K_{00}^{(\ell)}\Big)\,\delta\delta K_{[2]}^{(\ell)} + h\Big(K_{00}^{(\ell)}\Big)\,\Big(\delta K_{[1]}^{(\ell)}\Big)^2
\end{aligned}
$$

với **susceptibility vuông góc (perpendicular susceptibility)** mới $\chi_{\perp}(K) \equiv C_W\langle\sigma'(z)\sigma'(z)\rangle_K$ (eq 5.51). Điều kiện tới hạn tổng quát bây giờ cần CẢ HAI điều kiện (eq 5.55): $\chi_{\parallel}(K^\star) = 1$ **và** $\chi_{\perp}(K^\star) = 1$, nhiều hơn một điều kiện so với mạng tuyến tính (chỉ cần $C_W = 1$), vì activation phi tuyến làm $\chi_{\parallel} \neq \chi_{\perp}$ nói chung.

Activation scale-invariant (§5.2): lớp activation thoả $\sigma(\lambda z) = \lambda\sigma(z)$ với mọi $\lambda > 0$ (eq 5.58), có dạng tuyến tính từng đoạn $\sigma(z) = a_+ z$ ($z \geq 0$) hoặc $a_- z$ ($z < 0$) (eq 5.59). Lớp này bao gồm cả activation tuyến tính ($a_+ = a_- = 1$) và ReLU ($a_+ = 1, a_- = 0$) như hai trường hợp riêng. Với lớp này, $g(K) = A_2 K$ (eq 5.60) với $A_2 \equiv (a_+^2 + a_-^2)/2$ (eq 5.61), nghĩa là g tuyến tính đúng trong K, và hệ quả là $\chi_{\parallel} = \chi_{\perp} = A_2 C_W \equiv \chi$ **không phụ thuộc** K (eq 5.62), đơn giản hoá cực lớn. Điểm tới hạn: $C_b = 0, C_W = 1/A_2$ (eq 5.67), cho một ĐƯỜNG điểm cố định (bất kỳ $K_{00}^\star$ nào cũng được, tham số hoá bởi norm đầu vào). Với ReLU, $A_2 = 1/2$, nên $C_W = 2$: đây chính xác là **Kaiming initialization** $(C_b, C_W)^{\text{critical}} = (0, 2)$, ở đây được suy ra từ nguyên lý criticality, khác với lập luận gốc của Kaiming (bảo toàn phương sai forward pass theo kiểu heuristic hơn).

4. Thí nghiệm và kết quả chính

Điểm đặc biệt quan trọng cần nói rõ ngay: trong Preface, các tác giả nói thẳng rằng họ chủ động KHÔNG đưa các biểu đồ kiểm chứng thực nghiệm (experimental verification plots) vào sách, mà giữ các kiểm chứng đó trong ”sổ tay máy tính riêng” của mình. Nói cách khác, cuốn sách (ở các phần đã đọc) hoàn toàn là suy diễn giải tích (closed-form derivations), KHÔNG có benchmark trên dữ liệu thật, KHÔNG có bảng số liệu thực nghiệm kiểu accuracy/loss trên MNIST hay ImageNet. Vì vậy mục này liệt kê các ”kết quả chính” dưới dạng công thức/định lý suy ra được, không phải số liệu đo thực nghiệm:

1. Điều kiện criticality cho mạng tuyến tính: $C_W = 1$ duy nhất (một điều kiện, vì activation tuyến tính không gây mixing giữa các thành phần kernel).
2. Kaiming/He initialization $(C_b, C_W) = (0, 2)$ được tái suy ra từ nguyên lý criticality cho ReLU, không cần giả định heuristic ban đầu của Kaiming et al.
3. Công thức giới hạn kép thống nhất hai chế độ ”Gaussian” và ”hỗn loạn” của mạng tuyến tính: $G_{2m}^{(L)} \to e^{m(m-1)r}(G_2^{(L)})^m$, với $r = L/n$ là tham số liên tục duy nhất.
4. Hai điều kiện criticality tổng quát cho activation phi tuyến bất kỳ: $\chi_{\parallel} = \chi_{\perp} = 1$, thay vì chỉ một điều kiện như mạng tuyến tính.
5. (Epilogue) Công thức độ phức tạp mô hình: mô tả Gaussian vô hạn rộng cần $O(N_D^2)$ số để đặc tả đầy đủ (eq ε.14); mô tả nearly-Gaussian độ rộng hữu hạn (cắt ở bậc $1/n$) cần $O(N_D^4)$ số (eq ε.15); tổng quát, độ chính xác bậc $1/n^k$ cần $O(N_D^{2k})$ số (eq ε.25), với N_D là tổng số mẫu train+test.
6. (Appendix A) Công thức tỉ lệ sâu/rộng tối ưu theo tiêu chí tối đa hoá mutual information giữa hai nhóm neuron: $r^\star = \frac{4}{20+3n_L} \cdot \frac{1}{\nu}$ (eq A.73), với ν là hằng số phụ thuộc activation ($\nu = 2/3$ cho lớp $K^\star = 0$, hoặc $\nu = 3A_4/A_2^2 - 1$ cho lớp scale-invariant, trong đó $A_4 \equiv (a_+^4 + a_-^4)/2$; riêng ReLU cho $\nu = 5$, tuyến tính cho $\nu = 2$).
7. (Appendix B) Với residual network, điều kiện tới hạn mới $C_W(\gamma) = \frac{1}{A_2}(1-\gamma^2)$ (lớp scale-invariant, eq B.20) hoặc $C_W(\gamma) = \frac{1}{\sigma_1^2}(1-\gamma^2)$ (lớp $K^\star = 0$, eq B.21), với γ là hệ số residual (độ mạnh của nhánh identity/skip connection). Lựa chọn ”mặc định” $\gamma = 1$ (không có residual hyperparameter được tinh chỉnh) làm điều kiện tới hạn KHÔNG THỂ thoả mãn, trừ trường hợp suy biến $C_W = 0$.

5. Phân tích / ablation

Sách không chạy ablation thực nghiệm (xem mục 4), nhưng có hai dạng ”ablation giải tích” (so sánh các lựa chọn kiến trúc/activation bằng công thức, không phải bằng số liệu đo):

So sánh các ”universality class” của activation (§5.2 vs §5.3, chỉ §5.2 đọc chi tiết): activation scale-invariant (tuyến tính, ReLU) cho một đường điểm cố định được bảo toàn đúng, trong khi các activation khác (tanh, sin, SWISH, GELU, không đọc chi tiết trong ghi chú này) cho suy giảm theo luật lũy thừa về 0 hoặc về một giá trị khác 0. Paper gọi các activation cùng hành vi tại criticality là cùng một ”universality class” (mượn thuật ngữ nhóm tái chuẩn hoá trong vật lý: nhiều hệ vi mô khác nhau có thể chảy về cùng một lý thuyết hiệu dụng ở quy mô lớn). Hệ quả thực tiễn được paper rút ra: mạng dùng các activation trong cùng một class sẽ có hành vi gần giống nhau ở các lớp sâu, bất kể chi tiết cụ thể của activation.

Ablation theo hệ số residual γ (Appendix B, §B.3): $\gamma = 0$ cho lại mạng MLP thường (vanilla); $\gamma \to 1$ (nhánh identity chiếm ưu thế hoàn toàn) đẩy aspect ratio tối ưu $r^\star(\gamma)$ lên vô hạn (do $\nu(\gamma) \to 0$ đơn điệu khi $\gamma \to 1$, cả cho lớp scale-invariant eq B.29 và lớp $K^\star = 0$ eq B.35), nhưng đồng thời $C_W(\gamma) \to 0$: nhánh trọng số ngẫu nhiên gần như bị ”tắt”. Đây là một phát hiện khá bất ngờ: lựa chọn $\gamma = 1$ ”theo mặc định” (không tinh chỉnh gì thêm), vốn là lựa chọn phổ biến khi mô tả residual connection ở dạng $z^{(\ell+1)} = R(z^{(\ell)}) + z^{(\ell)}$ với trọng số hệ số 1 trước $z^{(\ell)}$, về lý thuyết LÀM HỎNG hoàn toàn điều kiện tới hạn, trừ khi có thêm heuristic khác bù lại (paper nêu rõ layer normalization, dùng trong kiến trúc transformer, là một heuristic thực tế cho mục đích này). Ngược hướng, công thức mutual information bậc $1/n^3$ (eq A.71) có hai số hạng ngược dấu (một dương bậc r^2, một âm bậc r^3), nên tồn tại một điểm tối ưu nội tại: cả mạng quá nông (r quá nhỏ) và mạng quá sâu (r quá lớn) đều kém hơn theo tiêu chí này, không phải ”sâu hơn luôn tốt hơn”.

6. Giới hạn

- **Không có kiểm chứng thực nghiệm trong sách** (đã nêu ở mục 4): mọi kết quả là suy diễn giải tích, các tác giả tự nhận không đưa số liệu thực nghiệm vào văn bản chính.
- **Giới hạn đọc của ghi chú này:** chỉ đọc 6 trong khoảng 14 chương/phụ lục của sách (Preface, Chương 0, Chương 3, Chương 5 §5.1-5.2, Epilogue, Appendix A, Appendix B); KHÔNG đọc Chương 1 (Pretraining), Chương 2 (Neural Networks, mô tả kiến trúc tổng quát hơn MLP), Chương 4, Chương 6 (RG flow), Chương 7-9 (Gradients, Backpropagation, NTK chi tiết), Chương 10 (Kernel Prediction), Chương 11 (Representation Learning), Chương ∞ (End of Training), và References/Index. Do đó ghi chú KHÔNG thể xác nhận hoặc phủ nhận liệu sách có phân tích kiến trúc convolutional hay self-attention ở các chương chưa đọc hay không; không đưa ra khẳng định nào về nội dung những chương đó.
- **Giả định đơn giản hoá tường minh của chính paper:** ở Chương 5, các siêu tham số C_b, C_W được giả định không đổi theo lớp (layer-independent) để giảm số siêu tham số cần xét; đây là lựa chọn mô hình hoá rõ ràng của tác giả, không phải thiếu sót.
- **Caveat do chính tác giả nêu về $r^\star$:** công thức aspect ratio tối ưu ở Appendix A (eq A.72-A.74) dựa HOÀN TOÀN trên phân phối PRIOR (trước huấn luyện), dùng tiêu chí thông tin (mutual information) chứ không phải hiệu năng sau huấn luyện thật. Paper tự ghi chú (footnote 30 của Appendix A) rằng giá trị $r^\star$ tính ra ”có thể trông nhỏ”, và khuyên người đọc đừng quên bản chất tiêu chí này. Số liệu tự tính ở mục 10 dưới đây cho thấy cụ thể $r^\star$ nhỏ tới mức nào.
- **Không phát hiện mâu thuẫn nội bộ rõ ràng** giữa văn bản chính và công thức trong các phần đã đọc (khác với một số bài khác trong series này, ví dụ math 19 hoặc math 21, nơi phát hiện được sai khác số liệu giữa bảng và công thức). Tuy nhiên vì phần lớn cuốn sách không được kiểm tra trực tiếp, đây chỉ là nhận xét trong phạm vi đã đọc, không phải khẳng định toàn sách nhất quán.

7. Liên hệ với các bài khác

- **math 26 (Poole et al., transient chaos) và math 27 (Schoenholz et al., Deep Information Propagation):** là tiền thân trực tiếp của khái niệm criticality/edge-of-chaos mà Chương 3 và Chương 5 của

PDLT hệ thống hoá lại bằng ngôn ngữ correlator/susceptibility chặt chẽ hơn; PDLT tự ghi nhận hai dòng công trình này trong Chương 5.

- **math 22 (Deep Neural Networks as Gaussian Processes, NNGP):** chính là nội dung "Cách 1" (giới hạn vô hạn rộng nghiêm ngặt) ở mục 2 của ghi chú này; PDLT dùng lại trực tiếp kết quả NNGP làm điểm khởi đầu $p^{\{0\}}$ của khai triển $1/n$.
- **math 23 (Neural Tangent Kernel):** xuất hiện trực tiếp trong Epilogue (eq ε.6-ε.9): đạo hàm bậc nhất của output theo tham số, $dz/d\theta$, chính là random feature, và kernel tương ứng với các feature đó chính là NTK Θ.
- **math 24 (Lazy Training, do người viết ghi chú này tự đọc và viết):** liên hệ trực tiếp đến giới hạn vô hạn rộng/NTK ở mục 2: "lazy training" mô tả đúng chế độ mà PDLT gọi là giới hạn vô hạn rộng nghiêm ngặt, nơi mạng gần như tuyến tính hoá quanh khởi tạo và không học representation; khái niệm $\kappa_h(w_0)$ trong math 24 và aspect ratio r trong PDLT đều đóng vai trò "tham số điều khiển mức độ phi tuyến/representation learning", dù định nghĩa khác nhau.
- **math 19 (Intrinsic Dimension) và math 20/21 (Benefits of Depth, Expressive Power of Width):** cùng nhóm câu hỏi "độ phức tạp hiệu dụng của mô hình là gì nếu không phải số tham số"; Epilogue của PDLT (công thức $O(N_D^{2k})$) đưa ra một câu trả lời định lượng khác hẳn với "intrinsic dimension" của math 19 (đếm theo codimension tập nghiệm) nhưng cùng tinh thần phản bác trực giác "đếm tham số thô".
- **pruning content, file `01_1989_1993_optimal_brain_damage_surgeon.md`**: chính Appendix A của PDLT, ở footnote 32, gợi ý liên hệ giữa kết quả tripartite information (thông tin được lưu trữ dư thừa giữa các neuron, eq A.75) với ý tưởng Optimal Brain Damage/Surgeon.
- **pruning content, file `08_2018_lottery_ticket_hypothesis.md`**: cùng được PDLT nhắc tên trong footnote 32 của Appendix A, như một hướng diễn giải khả dĩ cho hiện tượng thông tin dư thừa giữa neuron.
- **llm content, file `01_2017_attention_is_all_you_need.md`**: Appendix B của PDLT, footnote 1, nêu rõ residual connection là thành phần của "transformer-based language models", và §B.4 (Residual Building Blocks) đưa ra công thức tổng quát áp dụng được về nguyên tắc cho bất kỳ residual block nào, bao gồm kiến trúc attention.

8. Gợi ý áp dụng

Khung lý thuyết ở đây giúp tổ chức các câu hỏi về độ rộng, khởi tạo và động lực học của mạng sâu. Khi dùng trực giác từ giới hạn vô hạn, cần kiểm tra các giả thiết và mức độ phù hợp với mạng hữu hạn.

9. Thí nghiệm gợi ý

1. **Khảo sát phương sai theo độ sâu:** khởi tạo nhiều mạng MLP nhỏ với các độ rộng khác nhau, đo phương sai preactivation theo từng lớp và so sánh các lần khởi tạo. Quan sát này không thay thế các giả thiết của lý thuyết trong bài.

10. Tóm tắt công thức / khái niệm

- **Aspect ratio:** $r \equiv L/n$ (eq 0.19/ε.1); tham số nhiễu loạn duy nhất của toàn lý thuyết hiệu dụng.
- **Khai triển $1/n$:** $p(f^\star) = p^{\{0\}}(f^\star) + \frac{1}{n}p^{\{1\}}(f^\star) + O(1/n^2)$ (eq 0.14-0.15); $p^{\{0\}}$ là NNGP/NTK (vô hạn rộng), $p^{\{1\}}/n$ mang lại representation learning.
- **Criticality mạng tuyến tính:** $C_W = 1$ (eq 3.2 vùng lân cận, suy từ 3.14-3.15).
- **Non-Gaussianity bậc dẫn đầu tại criticality (mạng tuyến tính):** $\frac{2(\ell-1)}{n}(G_2^{(\ell)})^2$ (eq 3.28), tỉ lệ với r.
- **Giới hạn kép, hợp nhất Gaussian và hỗn loạn:** $G_{2m}^{(L)} \to e^{m(m-1)r}(G_2^{(L)})^m$ (eq 3.55).
- **Susceptibility song song/vuông góc (activation phi tuyến tổng quát):** $\chi_\parallel(K) = C_W g'(K)$ (eq 5.10), $\chi_\perp(K) = C_W\langle\sigma'\sigma'\rangle_K$ (eq 5.51); điều kiện tới hạn tổng quát $\chi_\parallel = \chi_\perp = 1$ (eq 5.55).

- **Criticality cho activation scale-invariant:** $(C_b, C_W)^{\text{critical}} = (0, 1/A_2)$ với $A_2 = (a_+^2 + a_-^2)/2$ (eq 5.67); ReLU cho $(0, 2)$ = Kaiming init.
- **Độ phức tạp mô hình theo bậc khai triển:** $O(N_D^{2k})$ số để đặc tả đầy đủ phân phối ở độ chính xác $1/n^k$ (eq ε.25); vô hạn rộng ($k = 0$, thực ra $k = 1$ theo cách đếm của paper cho Gaussian) là $O(N_D^2)$ (eq ε.14), $1/n$ (nearly-Gaussian) là $O(N_D^4)$ (eq ε.15).
- **Aspect ratio tối ưu theo mutual information (Appendix A):** $r^\star = \frac{4}{20+3n_L} \cdot \frac{1}{\nu}$ (eq A.73), với $\nu = 2/3$ (lớp $K^\star = 0$) hoặc $\nu = 3A_4/A_2^2 - 1$ (lớp scale-invariant, cho $\nu = 2$ với tuyến tính, $\nu = 5$ với ReLU).
- **Criticality cho residual network:** $C_W(\gamma) = \frac{1-\gamma^2}{A_2}$ (lớp scale-invariant, eq B.20); $r^\star(\gamma)$ tăng đơn điệu theo γ (eq B.36), giải thích vì sao residual connection cho phép huấn luyện mạng sâu hơn.

Tự tính (ví dụ số cho $r^\star$): áp dụng eq A.73 cho ReLU ($\nu = 5$) với một lớp rộng $n_L = 256$ (giả định, chỉ để minh hoạ công thức): $r^\star = \frac{4}{20+3\times256} \times \frac{1}{5} = \frac{4}{20+768} \times \frac{1}{5} = \frac{4}{788} \times \frac{1}{5} \approx 0.005076 \times 0.2 \approx 0.0010152$ (tự tính). Suy ra số lớp ”tối ưu” theo tiêu chí này: $L^\star = r^\star \times n_L \approx 0.0010152 \times 256 \approx 0.26$ lớp (tự tính), tức là CHƯA ĐẦY MỘT LỚP. Đây khớp với đúng caveat mà bài báo nêu ở footnote 30 Appendix A: giá trị $r^\star$ (và do đó $L^\star$) từ tiêu chí mutual-information-trên-prior là rất nhỏ, và không nên hiểu đây là ”số lớp nên dùng trong thực tế”, mà chỉ là một tính chất hình thức của tham số khai triển nhiễu loạn khi tối ưu hoá một tiêu chí thông tin trừu tượng trước khi huấn luyện.

11. Kết luận ngắn

PDLT xây một khung lý thuyết hiệu dụng thống nhất cho mạng nơ-ron sâu bằng cách nhận ra tỉ lệ độ sâu/rộng $r = L/n$ là tham số nhiễu loạn tự nhiên duy nhất, nối liền hai giới hạn trước đó tách rời (vô hạn rộng/NNGP/NTK và edge-of-chaos/signal propagation) thành một phổ liên tục. Trong các phần đã đọc (Chương 0, 3, 5 §5.1-5.2, Epilogue, Appendix A, B), đóng góp cụ thể gồm: tổng quát hoá điều kiện criticality cho activation phi tuyến bất kỳ (hai susceptibility $\chi_\parallel, \chi_\perp$), tái suy Kaiming init từ nguyên lý vật lý, định nghĩa lại ”độ phức tạp mô hình” theo số coupling phụ thuộc dữ liệu thay vì số tham số, và chỉ ra bằng công thức rằng residual connection dịch chuyển vùng ”hiệu dụng là sâu” lên các giá trị r lớn hơn. Hạn chế lớn nhất khi dùng ghi chú này: đây là một cuốn sách rất dài, ghi chú chỉ đọc có chọn lọc khoảng 6/14 phần, và sách tự nhận không có kiểm chứng thực nghiệm, nên mọi kết quả nêu trên cần được hiểu là định lý/công thức giải tích, không phải số liệu đo đạc.

Nguồn gốc: https://arxiv.org/abs/2106.10165

Bài 26

Exponential Expressivity in Deep Neural Networks through Transient Chaos (2016)

Tác giả: Ben Poole (Stanford University), Subhaneil Lahiri (Stanford University), Maithra Raghu (Google Brain and Cornell University), Jascha Sohl-Dickstein (Google Brain), Surya Ganguli (Stanford University).
Venue / arXiv: arXiv:1606.05340 (v2, 17/06/2016); paper không nêu venue hội nghị hay tạp chí nào trong toàn văn (đã kiểm bằng grep các từ khóa ”conference”, ”workshop”, ”published as”, ”proceedings”, ”NIPS”, ”NeurIPS”, ”ICLR”, ”ICML” trên toàn bộ file text trích từ PDF, không tìm thấy kết quả nào).
Nguồn: https://arxiv.org/abs/1606.05340
Chủ đề: lý thuyết mean-field cho sự lan truyền tín hiệu qua mạng nơ-ron ngẫu nhiên (chưa huấn luyện) nhiều lớp; ranh giới order-to-chaos (trật tự - hỗn loạn) theo phương sai trọng số/bias; dùng hình học Riemann (độ cong, độ dài Gauss) để định lượng độ phức tạp của hàm mà mạng tính được theo độ sâu.
Vị trí trong lộ trình: đọc cùng/trước math 27 (Deep Information Propagation), vì math 27 lấy đúng khung lý thuyết mean-field của bài này làm nền (”Background”) rồi mở rộng thêm depth-scale và lý thuyết backprop. Trong nhóm math, bài này còn liên hệ math 18 (Shattered Gradients), math 20 (Benefits of depth), math 21 (Expressive Power, View from Width), math 22 (Deep NN as Gaussian Processes); chi tiết ở mục 7.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (đọc toàn văn bản text trích từ PDF arXiv, bản v2 ngày 17/06/2016, kể cả phụ lục A-F). Các phép tính đánh dấu ”tự tính” là của người ghi chú, không phải số có sẵn trong paper. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc.

1. Bối cảnh

Trước năm 2016, nhiều công trình đã nghiên cứu vì sao mạng sâu có thể biểu diễn một số hàm hiệu quả hơn mạng nông rộng. Các chứng minh thường khai thác cấu trúc của lớp hàm cụ thể: Montufar và cộng sự (2014) đếm vùng tuyến tính của mạng ReLU; Delalleau và Bengio (2011), Martens và cộng sự (2013) xét mạng sum-product; Eldan và Shamir (2015), Telgarsky (2015) xét các họ hàm Boolean hoặc hàm dao động. Điều này đặt ra câu hỏi về khả năng mở rộng phân tích sang các dạng phi tuyến khác.

Bài này chọn một hướng khác, không phụ thuộc dạng phi tuyến cụ thể: xét mạng nơ-ron **ngẫu nhiên, chưa huấn luyện** (random, untrained) như một hệ động lực học, dùng lý thuyết mean-field (vật lý thống kê của hệ hỗn loạn/mất trật tự) kết hợp hình học Riemann để đo độ phức tạp của ánh xạ mà mạng tính được, khi tín hiệu lan truyền qua nhiều lớp. Ý tưởng nền: nếu ngay cả một mạng ngẫu nhiên (không học gì) đã có thể tính hàm phức tạp theo cấp số nhân với độ sâu, thì độ sâu tự nó là nguồn sức mạnh biểu diễn, độc lập với việc huấn luyện.

Thiết lập: mạng truyền thẳng (feedforward) kết nối đầy đủ, có D lớp trọng số $W^1, \dots, W^D$ và $D+1$ lớp hoạt động $x^0, \dots, x^D$, lớp l có N_l nơ-ron. Động lực truyền thẳng: $x^l = \phi(h^l)$, $h^l = W^l x^{l-1} + b^l$, với ϕ là hàm phi tuyến áp dụng theo từng thành phần. Trọng số W^l_{ij} lấy i.i.d. từ Gauss 0 trung bình, phương sai σ_w^2/N_{l-1} (chia theo chiều rộng lớp trước để giữ tổng đầu vào mỗi nơ-ron ở mức $O(1)$ không phụ thuộc N_{l-1}); bias b^l_i lấy i.i.d. Gauss 0 trung bình, phương sai σ_b^2.

Với N_l lớn, phân phối thực nghiệm của h^l_i trên các nơ-ron i hội tụ về Gauss (do là tổng nhiều biến ngẫu nhiên độc lập), nên toàn bộ động lực lan truyền một đầu vào có thể tóm gọn trong một đại lượng vô hướng: độ dài bình phương chuẩn hoá $q^l = \frac{1}{N_l}\sum_i (h^l_i)^2$.

2. Ý tưởng chính

Ý tưởng cốt lõi gồm bốn phần, nối tiếp nhau:

(a) Order-to-chaos cho một cặp điểm. Với hai đầu vào $x^0_{;1}, x^0_{;2}$, mối quan hệ giữa chúng ở lớp l được đo bằng hệ số tương quan c^l_{12}. Khi $l \to \infty$, c^l_{12} hội tụ về điểm cố định $c^*(\sigma_w, \sigma_b)$. Độ ổn định của điểm cố định $c^* = 1$ (hai điểm trùng nhau) được quyết định bởi hệ số giãn (stretch factor) χ_1: nếu $\chi_1 < 1$, mọi cặp điểm hội tụ về nhau khi qua nhiều lớp (**pha trật tự**, ordered); nếu $\chi_1 > 1$, các điểm gần nhau tách xa nhau theo cấp số nhân (**pha hỗn loạn**, chaotic), xuất hiện điểm cố định mới $c^* < 1$. Đường $\chi_1(\sigma_w, \sigma_b) = 1$ là ranh giới pha. $\log \chi_1$ chính là tương tự của số mũ Lyapunov trong lý thuyết hệ động lực học, nhưng vì mạng truyền thẳng chỉ có độ sâu D hữu hạn (bị "cắt cụt"), hiện tượng này được gọi là **transient chaos** (hỗn loạn tạm thời/nhất thời), chứ không phải hỗn loạn đúng nghĩa của hệ lặp vô hạn.

(b) Hình học Riemann cho một đa tạp (manifold). Thay vì chỉ xét 2 điểm rời rạc, bài báo xét một đường cong 1 chiều (ví dụ hình tròn) trong không gian đầu vào, lan truyền qua mạng. Ở mỗi lớp, đường cong có độ dài Euclid L^E và độ cong ngoại tại (extrinsic curvature) $\kappa(\theta)$; kết hợp hai đại lượng này qua ánh xạ Gauss (Gauss map, gửi mỗi điểm tới vector tiếp tuyến đơn vị của nó trên mặt cầu S^{N-1}) cho ra độ dài Grassmannian L^G, một thước đo độ phức tạp "tổng hợp" của đường cong. Kết quả bất ngờ: trong pha hỗn loạn, L^E và L^G tăng theo cấp số nhân với độ sâu, nhưng độ cong κ **không** suy biến về 0 như trong một phép co giãn tuyến tính đơn giản (scale-up tuyến tính giữ L^G không đổi vì độ dài tăng bù cho độ cong giảm); ở đây độ cong hội tụ về một điểm cố định $\bar{\kappa}^*$ khác không. Nói cách khác, đường cong không chỉ "dài ra" mà còn "ngoằn ngoèo" liên tục ở mọi độ sâu, nên độ phức tạp hình học tăng theo cấp số nhân thật sự, không phải chỉ là giãn phẳng.

(c) Mạng nông không thể đạt độ phức tạp này. Định lý 1 của paper chứng minh: với một mạng 1 lớp ẩn rộng N_1, độ dài Euclid tối đa có thể đạt được chỉ tăng tuyến tính theo N_1 (nên độ dài chuẩn hoá theo nơ-ron chỉ tăng như $\sqrt{N_1}$), bất kể chọn trọng số/bias thế nào. Trong khi đó mạng sâu ở pha hỗn loạn đạt tăng trưởng mũ theo độ sâu. Đây là bằng chứng định lượng cho "độ sâu mạnh hơn độ rộng" theo đúng nghĩa hình học.

(d) Ranh giới quyết định cũng cong theo cấp số nhân. Nếu đặt một bộ phân loại tuyến tính ở lớp cuối, ranh giới quyết định (decision boundary) là một siêu phẳng ở lớp cuối, nhưng khi lan truyền ngược (backpropagate) ranh giới này về lớp đầu vào, nó trở thành một đa tạp cong; các độ cong chính (principal curvatures) của đa tạp này tăng theo cấp số nhân với độ sâu. Mạng sâu, ngay cả khi ngẫu nhiên, đã "gấp" không gian đầu vào đủ nhiều để một siêu phẳng đơn giản ở đầu ra tương ứng với một ranh giới rất phức tạp ở đầu vào.

Ngoài bốn ý trên, phụ lục F còn chỉ ra tính đối ngẫu (duality): thay vì cố định trọng số và thay đổi đầu vào, nếu cố định đầu vào và thay đổi (nhiễu) trọng số ở một lớp giữa, hàm mà mạng tính được cũng decorrelate (mất tương quan) theo cấp số nhân với độ sâu tính từ lớp bị nhiễu đến lớp cuối, y hệt cơ chế input-chaos. Hiện tượng này được gọi là **weight chaos**.

3. Phương pháp chi tiết

Length map (bản đồ độ dài). Với một đầu vào, q^l tuân theo công thức lặp:

$$q^l = \mathcal{V}(q^{l-1} \mid \sigma_w, \sigma_b) \equiv \sigma_w^2 \int \mathcal{D}z\, \phi\Big(\sqrt{q^{l-1}}\, z\Big)^2 + \sigma_b^2,$$

với $\mathcal{D}z$ là đo Gauss chuẩn. Với $\sigma_b = 0$, $\sigma_w < 1$: điểm cố định duy nhất là $q^* = 0$ (mạng co tín hiệu về gốc). Với $\sigma_w > 1, \sigma_b = 0$: $q^* = 0$ mất ổn định, xuất hiện điểm cố định không-không ổn định. Với $\sigma_b \neq 0$ bất kỳ: luôn có một điểm cố định ổn định khác không.

Correlation map (bản đồ tương quan). Với 2 đầu vào, hiệp phương sai q^l_{12} (và hệ số tương quan $c^l_{12} = q^l_{12}/\sqrt{q^l_{11}q^l_{22}}$) tuân theo:

$$q^l_{12} = \sigma_w^2 \int \mathcal{D}z_1 \mathcal{D}z_2\, \phi(u_1)\phi(u_2) + \sigma_b^2, \quad u_1 = \sqrt{q^{l-1}_{11}}\, z_1,\ u_2 = \sqrt{q^{l-1}_{22}} \left(c^{l-1}_{12} z_1 + \sqrt{1-(c^{l-1}_{12})^2}\, z_2 \right).$$

Hệ số giãn tại điểm tới hạn $c = 1$:

$$\chi_1 \equiv \left. \frac{\partial c^l_{12}}{\partial c^{l-1}_{12}} \right|_{c=1} = \sigma_w^2 \int \mathcal{D}z \left[\phi'\left(\sqrt{q^*}\, z \right) \right]^2.$$

Nhận xét quan trọng về tốc độ hội tụ (không phải công thức, mà là quan sát thực nghiệm của paper, dùng ở mục 4): bản đồ tương quan nằm gần đường chéo (unity line) hơn bản đồ độ dài, nên c^l_{12} hội tụ về c^* chậm hơn nhiều so với q^l hội tụ về q^*.

Hình học Riemann của đường cong lan truyền. Với đường $h(\theta)$, vector tốc độ $v(\theta) = \partial_\theta h(\theta)$, gia tốc $a(\theta) = \partial_\theta v(\theta)$; đường tròn osculating (tiếp xúc) tại θ có bán kính $R(\theta)$, độ cong $\kappa(\theta) = 1/R(\theta)$. Metric Euclid $g^E = v \cdot v$ cho độ dài $L^E = \int \sqrt{g^E}\, d\theta$. Ánh xạ Gauss gửi θ tới vector tiếp tuyến đơn vị $\hat{v}(\theta) \in S^{N-1}$; metric Gauss $g^G = (\partial_\theta \hat{v}) \cdot (\partial_\theta \hat{v}) = \kappa^2 g^E$ cho độ dài $L^G = \int \sqrt{g^G}\, d\theta$.

Chuẩn hoá theo chiều rộng ($\bar{g}^E = g^E/N$, $\bar{\kappa} = \sqrt{N}\kappa$), paper suy ra công thức lặp:

$$\bar{g}^{E,l} = \chi_1\, \bar{g}^{E,l-1}, \qquad (\bar{\kappa}^l)^2 = \frac{\chi_2}{\chi_1^2} + \frac{1}{\chi_1}(\bar{\kappa}^{l-1})^2, \qquad \chi_2 = \sigma_w^2 \int \mathcal{D}z \left[\phi''\left(\sqrt{q^*}\, z \right) \right]^2.$$

Đọc công thức này: mỗi lớp nhân $\bar{g}^E$ với χ_1 (tăng mũ nếu $\chi_1 > 1$); độ cong $\bar{\kappa}^2$ của lớp trước bị suy giảm hệ số $1/\chi_1$ nhưng được "bù" thêm một lượng χ_2/χ_1^2 sinh ra từ độ cong của chính hàm phi tuyến ϕ (qua ϕ''). Hai hiệu ứng này cân bằng ở một điểm cố định $\bar{\kappa}^* \neq 0$, khác hẳn phép giãn tuyến tính đơn giản (ở đó κ giảm đúng bằng tỉ lệ nghịch với độ giãn, giữ L^G không đổi).

Định lý 1 (chặn trên cho mạng nông). Nếu ϕ đơn điệu không giảm với "dynamic range" $R = \max_h \phi(h) - \min_h \phi(h)$ hữu hạn, và đường cong đầu vào $x^0(\theta)$ có mọi phép chiếu 1 chiều của $\partial_\theta x^0(\theta)$ đổi dấu không quá s lần, thì với **mọi** cách chọn W^1, b^1: $L^E(x^1) \leq N_1(1+s)R$. Với đường tròn ($s = 1$) và $\phi = \tanh$ ($R = 2$), paper rút ra dạng đơn giản cho độ dài chuẩn hoá: $\bar{L}^E \leq 2\sqrt{N_1}$.

Độ cong của ranh giới quyết định. Với phân loại tuyến tính $y = \mathrm{sgn}(\beta \cdot x^D - \beta_0)$, ranh giới ở lớp đầu vào là nghiệm của $G(x^0) = \beta \cdot x^D(x^0) - \beta_0 = 0$; các độ cong chính $\kappa_1 \geq \cdots \geq \kappa_{N-1}$ là trị riêng của Hessian chuẩn hoá $H = \|\nabla G\|^{-1} P\, \partial^2 G/\partial x \partial x^T\, P$ chiếu lên mặt phẳng tiếp tuyến (P là toán tử chiếu). Paper tính số các κ_i này theo độ sâu bằng mô phỏng (không có công thức closed-form đơn giản như length/correlation map).

Weight chaos (phụ lục F). Xét họ mạng tham số hoá bởi $\Delta \in [-1, 1]$, chỉ trọng số lớp 2 thay đổi theo Δ: $W^2(\Delta) = \sqrt{1-|\Delta|}\, W + \sqrt{|\Delta|}\, dW$ (W, dW i.i.d. cùng phương sai σ_w^2). Với một đầu vào cố định (hình tròn ở bán kính q^*), tương quan hàm số $C^l(\Delta)$ giữa mạng tại $\Delta = 0$ và tại Δ tuân theo công thức lặp cùng dạng toán học với correlation map ở phần (b), nên trong pha hỗn loạn, $C^l(\Delta)$ cũng giảm theo cấp số nhân với độ sâu tính từ lớp 2: một nhiễu nhỏ ở trọng số lớp giữa gây ra thay đổi lớn, tăng mũ theo số lớp còn lại, trong không gian hàm mà mạng tính được.

4. Thí nghiệm và kết quả chính

Tất cả thí nghiệm dùng mạng sigmoidal ($\phi = \tanh$), chiều rộng 1000 nơ-ron mỗi lớp, so sánh dự đoán lý thuyết (đường liền) với mô phỏng mạng thật (điểm rời rạc).

- **Hình 1 (length map):** với $\sigma_b = 0{,}3$ cố định và 3 giá trị σ_w khác nhau, đường lý thuyết của bản đồ độ dài khớp tốt với mô phỏng; q^l hội tụ về điểm cố định q^* rất nhanh, **thường chỉ trong khoảng 4 lớp**, không phụ thuộc điều kiện khởi đầu.
- **Hình 2 (correlation map):** cùng $\sigma_b = 0{,}3$ và các σ_w ở Hình 1, mô phỏng với $N_l = 1000$; tương quan c_{12}^l hội tụ về c^* **chậm hơn nhiều, khoảng 20 lớp**, đúng như lý giải ở mục 3 (bản đồ tương quan gần đường chéo hơn). Mặt phẳng (σ_w, σ_b) được chia bởi đường $\chi_1 = 1$ thành vùng trật tự ($\chi_1 < 1$, $c^* = 1$) và vùng hỗn loạn ($\chi_1 > 1$, $c^* < 1$).
- **Hình 3 (lan truyền hình tròn):** một hình tròn (tạo từ 2 vector cơ sở trực chuẩn u_0, u_1) được lan truyền qua 3 mạng ngẫu nhiên với σ_w khác nhau, $\sigma_b = 0{,}3$ cố định. Ở σ_w lớn (chế độ hỗn loạn), phổ trị riêng (singular values) của hình chiếu phẳng dần và đường cong "rối" hơn rõ rệt khi nhìn ở lớp 5 và lớp 10 (chiếu lên 3 thành phần chính đầu). Tự tương quan $c_{12}^l(\Delta\theta)$ đo theo lớp khớp định lượng với lý thuyết lặp từ công thức tương quan ở mục 3 (đường lý thuyết trùng số liệu mô phỏng, kể cả độ lệch chuẩn theo θ).
- **Hình 4 (độ cong/độ dài):** mạng 1000 nơ-ron, xác nhận định lượng rằng $\bar{g}^E$ tăng và $\bar{\kappa}$ hội tụ về điểm cố định khác không (không suy biến về 0), đúng như công thức (8)-(9) ở mục 3; lý thuyết và mô phỏng khớp nhau trên các panel C-E.
- **Hình 5 (sâu thắng rộng):** dùng chế độ hỗn loạn cụ thể $\sigma_b = 0{,}3, \sigma_w = 4{,}0$, mạng rộng 1000, $\phi = \tanh$. (A) hoạt động của 4 nơ-ron đầu ra theo θ ngày càng phức tạp khi mạng sâu hơn. (B) hồi quy tuyến tính lên một hàm mục tiêu ngẫu nhiên khớp tốt hơn rõ rệt với mạng sâu hơn. (C) phân rã lỗi theo tần số: mạng nông không bắt được thành phần tần số cao của hàm ngẫu nhiên dù mạng sâu làm được. (D) **tăng chiều rộng mạng 1 lớp ẩn lên tới 10.000 nơ-ron không làm giảm lỗi ở tần số cao** - xác nhận thực nghiệm trực tiếp cho Định lý 1 (độ rộng chỉ giúp theo $\sqrt{N_1}$, độ sâu giúp theo cấp số nhân).
- **Hình 4G (độ cong ranh giới quyết định):** các độ cong chính lớn nhất và nhỏ nhất (4 cặp $\kappa_{\pm 1}, \ldots, \kappa_{\pm 4}$) của ranh giới quyết định, tính số bằng mô phỏng, tăng theo cấp số nhân khi lan truyền ngược về lớp đầu vào.

5. Phân tích / ablation

Đây là paper lý thuyết trên mạng ngẫu nhiên (không huấn luyện trên dữ liệu thật), nên không có ablation kiểu "bỏ một thành phần kiến trúc rồi đo accuracy" như các paper thực nghiệm khác. Thay vào đó, các "ablation" ở đây là quét tham số một cách có hệ thống:

- **Quét σ_w, σ_b:** Hình 1C/D và Hình 2C/D vẽ q^*, số lớp cần để hội tụ 1%, c^*, và χ_1 như hàm của toàn bộ mặt phẳng (σ_w, σ_b), không chỉ 3 điểm minh hoạ ở Hình 1A/2A, cho thấy biên $\chi_1 = 1$ là một đường cong trơn trong mặt phẳng này, không phải một hiện tượng cô lập ở vài giá trị cụ thể.
- **Quét chiều rộng mạng nông:** Hình 5D là "ablation" rõ nhất về mặt khái niệm: tăng N_1 từ nhỏ lên tới 10.000, lỗi ở tần số cao gần như không giảm, trực tiếp kiểm chứng Định lý 1 (chặn trên $\sqrt{N_1}$, không phải tăng theo độ sâu).
- **Vai trò của bias:** so sánh định tính case $\sigma_b = 0$ (chỉ có điểm cố định $q^* = 0$ khi $\sigma_w < 1$, mạng co tín hiệu về gốc) với case $\sigma_b \neq 0$ (luôn có điểm cố định khác không), cho thấy bias đóng vai trò "chống suy biến" tín hiệu ngay cả khi trọng số nhỏ.
- **Paper không** chạy ablation trên loại phi tuyến ϕ khác tanh một cách hệ thống trong văn bản chính; phần lý thuyết (length/correlation map, χ_1) viết tổng quát cho ϕ đơn điệu bất kỳ, nhưng mọi hình vẽ minh hoạ số liệu đều dùng tanh.

6. Giới hạn

- Toàn bộ phân tích áp dụng cho mạng **ngẫu nhiên, chưa huấn luyện**; paper tự coi đây là một "null model" (mô hình nền) để so sánh với mạng đã huấn luyện, không trực tiếp nói điều gì xảy ra sau khi học.

- Chỉ xét mạng **kết nối đầy đủ** (fully-connected) với trọng số/bias i.i.d. Gauss; không mở rộng cho CNN, RNN hay kiến trúc có trọng số chia sẻ/cấu trúc đặc biệt trong bài này (paper tiếp theo, math 27, cũng tự nêu đúng hạn chế này).
- Lý thuyết dựa trên xấp xỉ mean-field ở giới hạn chiều rộng lớn ($N_l \to \infty$, giả thiết self-averaging); với N_l hữu hạn (ví dụ $N_l = 1000$ trong mô phỏng), chỉ là xấp xỉ tốt, không phải đẳng thức chính xác, dù khớp rất tốt trong các hình vẽ.
- Định lý 1 yêu cầu ϕ đơn điệu với dynamic range R **hữu hạn**; với các hàm không chặn như ReLU, $R = \infty$ nên chặn trên trở nên vô nghĩa theo đúng công thức đã nêu, dù paper không bàn trực tiếp vấn đề này trong văn bản chính (math 27 sau này mới nêu rõ hạn chế tương tự của khung mean-field với "unbounded activations" như ReLU).
- Bài báo lưu ý đây chỉ là "khảo sát sơ bộ" (preliminary exploration); việc đếm số lượng mẫu phân loại đạt được (classification patterns) được nhường hẳn cho bài đồng hành (companion paper) [16] = Raghu, Poole, Kleinberg, Ganguli, Sohl-Dickstein, "On the expressive power of deep neural networks" (arXiv:1606.05336). Cần lưu ý: đây **không** phải là math 21 trong bộ ghi chú này (math 21 là "The Expressive Power of Neural Networks: A View from the Width", arXiv:1709.02540, của Lu, Pu, Wang, Hu, Wang, một nhóm tác giả và nội dung hoàn toàn khác dù tên bài tương tự); ghi chú này không gộp hai bài lại.

7. Liên hệ với các bài khác

- **math 27 (Deep Information Propagation):** là bài kế tiếp trực tiếp, lấy đúng bộ công thức q^l, c_{12}^l, χ_1 của bài này làm "Background" (mục 2 của math 27), rồi mở rộng thêm hai depth-scale ξ_q, ξ_c (trong đó ξ_c phân kỳ đúng tại $\chi_1 = 1$), hiệu ứng dropout phá vỡ điểm tới hạn, và một lý thuyết mean-field song song cho gradient backprop nối trực tiếp $\chi_1 < 1 / > 1$ với vanishing/exploding gradient.
- **math 18 (Shattered Gradients Problem):** khảo sát hiện tượng có liên hệ gần: gradient của mạng ReLU sâu ngẫu nhiên trở nên giống nhiễu trắng (mất tương quan) theo độ sâu nếu không có kiến trúc residual phù hợp; đây là mặt "gradient" của chính hiện tượng chaos (mất tương quan tín hiệu) mà bài 26 nghiên cứu ở chiều truyền thẳng; math 27 xây cầu nối hình thức giữa hai hiện tượng này qua cùng hệ số χ_1.
- **math 21 (Expressive Power of Neural Networks, View from the Width):** cũng nghiên cứu đánh đổi độ sâu/độ rộng cho tính biểu diễn, nhưng bằng kỹ thuật khác (chặn dưới độ rộng cần thiết để mạng ReLU rộng xấp xỉ mạng hẹp sâu), bổ sung cho lập luận hình học/chaos của bài 26 về việc vì sao mạng nông không theo kịp mạng sâu.
- **math 22 (Deep Neural Networks as Gaussian Processes):** dùng chính cơ chế mean-field/Gauss ở giới hạn chiều rộng lớn (công thức lặp cho q^l ở đây về bản chất là công thức lặp cho kernel NNGP) nhưng cho mục đích khác: suy luận Bayes chính xác ở giới hạn vô hạn chiều rộng, thay vì đo độ hỗn loạn/biểu diễn.
- **math 20 (Benefits of depth in neural networks):** đưa lập luận sớm hơn, bằng công cụ lý thuyết xấp xỉ/lớp hàm, cho việc độ sâu có lợi; là một góc nhìn đối chứng hữu ích với lập luận hệ động lực học/hình học của bài 26 cho cùng câu hỏi.

8. Gợi ý áp dụng

Phân tích transient chaos cho thấy một cơ chế tạo độ phức tạp biểu diễn tăng nhanh theo độ sâu trong mô hình được nghiên cứu. Khả năng biểu diễn và khả năng học được hàm bằng một thuật toán tối ưu là hai câu hỏi khác nhau.

9. Thí nghiệm gợi ý

1. **Đo tốc độ hội tụ của length map và correlation map** trên mạng FC tanh ngẫu nhiên nhỏ (rộng 500-1000, sâu tới 50 lớp) tại vài điểm (σ_w, σ_b) trải từ pha trật tự tới pha hỗn loạn, trên GPU (hoặc chỉ CPU

vì mạng nhỏ). Kỳ vọng: q^l hội tụ trong khoảng 4 lớp, c_{12}^l hội tụ trong khoảng 20 lớp, đúng như Hình 1-2 của paper; gần đường $\chi_1 = 1$, tốc độ hội tụ chậm hẳn lại (vì $|\log \chi_1| \to 0$).

2. **Tái lập rút gọn Hình 5:** so sánh một mạng 1 lớp ẩn với chiều rộng tăng dần (100 đến 5.000) với một mạng sâu hẹp ở chế độ hỗn loạn (ví dụ $\sigma_w = 4{,}0, \sigma_b = 0{,}3$, rộng cố định 300-500, độ sâu tăng dần), cùng khớp một hàm mục tiêu ngẫu nhiên nhiều tần số. Đo lỗi theo tần số; kỳ vọng mạng rộng bão hoà còn mạng sâu tiếp tục cải thiện, giống Hình 5C/D.
3. **Đo điểm cố định độ cong** $\bar{\kappa}^*$ cho một hình tròn lan truyền qua mạng tanh nhỏ (rộng 500) ở vài giá trị σ_w trong pha hỗn loạn, so với giá trị lý thuyết giải từ công thức $(\bar{\kappa}^*)^2 = \chi_2/(\chi_1^2 - \chi_1)$ (rút ra từ công thức lặp ở mục 3 khi đặt $\bar{\kappa}^l = \bar{\kappa}^{l-1} = \bar{\kappa}^*$). Kiểm tra độ khớp lý thuyết - mô phỏng.
4. **Thí nghiệm prune-và-rescale đơn giản:** khởi tạo một mạng FC tanh nhỏ gần $\chi_1 \approx 1$, prune ngẫu nhiên 50% nơ-ron mỗi lớp, đo χ_1 của mạng con **trước** và **sau** khi rescale lại σ_w^2 theo chiều rộng mới N_{l-1}. Mục tiêu: kiểm tra trực tiếp giả thuyết ở mục 8, rằng pruning không rescale có thể làm χ_1 trôi khỏi vùng tới hạn.
5. **Đo weight chaos thực nghiệm:** với mạng tanh nhỏ ở pha hỗn loạn, cố định đầu vào (hình tròn ở bán kính q^*), nhiễu trọng số một lớp giữa theo tham số Δ như công thức ở mục 3, đo tương quan hàm $C^l(\Delta)$ ở lớp cuối theo Δ và theo khoảng cách từ lớp bị nhiễu tới lớp cuối; kỳ vọng giảm theo cấp số nhân, khớp với lập luận ở phụ lục F.

10. Tóm tắt công thức / khái niệm

- **Length map:** $q^l = \sigma_w^2 \int \mathcal{D}z\, \phi(\sqrt{q^{l-1}}z)^2 + \sigma_b^2$; điểm cố định q^*.
- **Correlation map và χ_1:** $q_{12}^l = \sigma_w^2 \int \mathcal{D}z_1 \mathcal{D}z_2\, \phi(u_1)\phi(u_2) + \sigma_b^2$; $\chi_1 = \sigma_w^2 \int \mathcal{D}z\, [\phi'(\sqrt{q^*}z)]^2$; $\chi_1 < 1$ là pha trật tự, $\chi_1 > 1$ là pha hỗn loạn, $\chi_1 = 1$ là đường tới hạn.
- **Transient chaos:** $\log \chi_1$ tương tự số mũ Lyapunov, nhưng "cắt cụt" ở độ sâu hữu hạn D (khác hỗn loạn đúng nghĩa của hệ lặp vô hạn).
- **Tiến hoá độ dài/độ cong:** $\bar{g}^{E,l} = \chi_1 \bar{g}^{E,l-1}$; $(\bar{\kappa}^l)^2 = \chi_2/\chi_1^2 + (\bar{\kappa}^{l-1})^2/\chi_1$, với $\chi_2 = \sigma_w^2 \int \mathcal{D}z [\phi''(\sqrt{q^*}z)]^2$; độ cong hội tụ về điểm cố định khác không trong pha hỗn loạn.
- **Định lý 1 (chặn mạng nông):** $L^E \le N_1(1+s)R$; với hình tròn + tanh: $\bar{L}^E \le 2\sqrt{N_1}$.
- **Tốc độ hội tụ quan sát được:** length map hội tụ trong khoảng 4 lớp; correlation map hội tụ trong khoảng 20 lớp (chậm hơn vì gần đường chéo hơn).
- **Tự tính (ví dụ 1, theo đúng công thức Định lý 1 mà paper đã rút gọn cho hình tròn + tanh):** với $N_1 = 1.000$: $\bar{L}^E \le 2\sqrt{1.000} \approx 2 \times 31{,}62 \approx 63{,}2$. Với $N_1 = 10.000$ (như thí nghiệm Hình 5D): $\bar{L}^E \le 2\sqrt{10.000} = 200$, tức tăng độ rộng gấp 10 lần chỉ cho phép chặn trên tăng khoảng $\sqrt{10} \approx 3{,}16$ lần, không phải 10 lần, giải thích tại sao Hình 5D cho thấy lỗi tần số cao gần như không đổi khi tăng N_1.
- **Tự tính (ví dụ 2, minh hoạ tăng trưởng mũ của $\bar{g}^E$):** giả sử $\chi_1 = 1{,}5$ (pha hỗn loạn) và $q^* = 1$; theo công thức $\bar{g}^{E,l} = \chi_1\, \bar{g}^{E,l-1}$, sau 19 lớp: $\bar{g}^{E,20} = 1{,}5^{19} \times 1$. Tính: $1{,}5^{10} \approx 57{,}67$; $1{,}5^9 \approx 38{,}44$; nhân lại $1{,}5^{19} \approx 57{,}67 \times 38{,}44 \approx 2.218$. Vậy sau 20 lớp, độ dài bình phương trên mỗi nơ-ron đã tăng khoảng 2.218 lần chỉ từ $\chi_1 = 1{,}5$, tương ứng độ dài L^E tăng khoảng $\sqrt{2.218} \approx 47$ lần, minh hoạ cụ thể tốc độ tăng trưởng mũ mà paper mô tả bằng lời.

11. Kết luận ngắn

Bài này chỉ ra, bằng lý thuyết mean-field kết hợp hình học Riemann trên mạng ngẫu nhiên chưa huấn luyện, rằng ở pha hỗn loạn ($\chi_1 > 1$) độ dài và độ cong toàn cục của một đa tạp đầu vào tăng theo cấp số nhân với độ sâu, trong khi mạng nông chỉ có thể đạt độ phức tạp tăng theo căn bậc hai của chiều rộng (Định lý 1); ranh giới quyết định cũng cong theo cấp số nhân khi lan truyền ngược về đầu vào, và hiện tượng đối ngẫu "weight chaos" cho thấy nhiễu trọng số nhỏ cũng có thể gây thay đổi hàm lớn ở pha hỗn loạn. Những kết quả này đặt nền cho math 27 mở rộng sang depth-scale và lý thuyết gradient backprop.

Nguồn gốc: https://arxiv.org/abs/1606.05340

Bài 27

Deep Information Propagation (2016)

Tác giả: Samuel S. Schoenholz (Google Brain), Justin Gilmer (Google Brain), Surya Ganguli (Stanford University), Jascha Sohl-Dickstein (Google Brain).
Venue / arXiv: arXiv:1611.01232 (v2, 04/04/2017); venue ghi trong paper: ”Published as a conference paper at ICLR 2017”.
Nguồn: https://arxiv.org/abs/1611.01232
Chủ đề: hai depth-scale (ξ_q, ξ_c) điều khiển lan truyền tín hiệu trong mạng ngẫu nhiên chưa huấn luyện; ảnh hưởng của dropout lên điểm tới hạn order-to-chaos; lý thuyết mean-field cho backprop gradient; thực nghiệm trainability trên MNIST/CIFAR-10 theo độ sâu và dropout.
Vị trí trong lộ trình: đọc ngay sau math 26 (Exponential expressivity in deep neural networks through transient chaos), vì bài này lấy nguyên khung lý thuyết mean-field của math 26 (length map, correlation map, χ_1) làm phần ”Background”, rồi mở rộng thành câu hỏi thực dụng hơn: với tham số nào thì mạng sâu **huấn luyện được**. Trong nhóm math, còn liên hệ math 18, math 20, math 22, math 23; chi tiết ở mục 7.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (đọc toàn văn bản text trích từ PDF arXiv, bản v2 ngày 04/04/2017, gồm cả phụ lục 7.1-7.10). Các phép tính đánh dấu ”tự tính” là của người ghi chú, không phải số có sẵn trong paper. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc.

1. Bối cảnh

Poole và cộng sự (2016, bài 26) và công trình đồng hành của Raghu và cộng sự (2016) nghiên cứu khả năng biểu diễn của mạng ngẫu nhiên chưa huấn luyện qua phân tích mean-field và ranh giới giữa pha trật tự với hỗn loạn. Khả năng biểu diễn một hàm chưa bảo đảm thuật toán có thể học được hàm đó. Bài của Schoenholz, Gilmer, Ganguli và Sohl-Dickstein (ICLR 2017) tiếp tục hỏi những siêu tham số nào cho phép huấn luyện mạng sâu hiệu quả.

Bối cảnh rộng hơn: việc chọn khởi tạo cho mạng sâu là bài toán nửa khoa học nửa ”nghệ thuật”, với nhiều kỹ thuật trực giác hoặc gián tiếp đã tồn tại trước bài này: khởi tạo ”edge of chaos” trực tiếp (Bertschinger et al., 2005; Glorot và Bengio, 2010), hoặc gián tiếp qua các kỹ thuật thúc đẩy lan truyền tín hiệu sâu như batch normalization (Ioffe và Szegedy, 2015), khởi tạo ma trận trực giao (Saxe et al., 2014), khởi tạo random-walk (Sussillo và Abbott, 2014), composition kernel (Daniely et al., 2016), hay kiến trúc residual (He et al., 2015). Bài báo lưu ý điểm mới của mình là hai điều: (1) khung lý thuyết dự đoán được độ sâu khả huấn luyện **ngay cả khi mạng ở xa điểm tới hạn** (ví dụ với dropout, nơi không hề có điểm tới hạn sắc nét); (2) đưa ra một giải thích **hình thức** (formal), không chỉ trực giác, cho hiện tượng ”chỉ huấn luyện được mạng rất sâu gần edge of chaos”.

2. Ý tưởng chính

(a) Hai depth-scale. Khai triển tiệm cận bản đồ độ dài và bản đồ tương quan của math 26 quanh điểm cố định cho ra hai thước đo độ sâu riêng biệt: ξ_q (độ sâu mà thông tin về **biên độ** một đầu vào đơn lẻ có thể lan truyền) và ξ_c (độ sâu mà thông tin về **tương quan** giữa hai đầu vào có thể lan truyền). Khác biệt mấu chốt: ξ_q luôn **hữu hạn và ngắn**, bất kể tham số nào, còn ξ_c **phân kỳ đúng tại đường tới hạn** $\chi_1 = 1$ của math 26. Đây là cách hình thức hoá trực giác ”edge of chaos cho phép mạng vô hạn sâu”: chỉ có thông tin tương quan (không phải thông tin biên độ) mới có thể lan truyền xa vô hạn, và chỉ khi mạng ở đúng điểm tới hạn.

(b) Dropout phá vỡ điểm tới hạn hoàn toàn. Với tỉ lệ dropout $\rho < 1$ bất kỳ, dù rất nhỏ, điểm $c^* = 1$ **không còn là điểm cố định** của bản đồ tương quan nữa (khác hẳn trường hợp không dropout, nơi $c^* = 1$ luôn là

điểm cố định, chỉ có ổn định hay không mới phụ thuộc χ_1). Hệ quả: ξ_c không bao giờ phân kỳ khi có dropout, nên độ sâu khả huấn luyện bị chặn trên một cách nghiêm ngặt, là hàm của ρ.

(c) Đối ngẫu forward-backward cho gradient. Paper xây một lý thuyết mean-field song song cho backprop: phương sai lỗi $\tilde{q}^l_{aa}$ lan truyền ngược qua các lớp tuân theo công thức lặp dùng **đúng** hệ số χ_1 của forward pass. Ba chế độ: pha trật tự ($\chi_1 < 1$) gradient **biến mất** (vanish) theo cấp số nhân; điểm tới hạn ($\chi_1 \to 1$) gradient **ổn định** ở mọi độ sâu; pha hỗn loạn ($\chi_1 > 1$) gradient **nổ** (explode) theo cấp số nhân. Đây được trình bày như lời giải thích hình thức đầu tiên (bằng mean-field) cho hiện tượng vanishing/exploding gradient đã được quan sát từ lâu (Bengio et al., 1993).

(d) Hiệp phương sai gradient cũng bị chặn bởi ξ_c. Không chỉ phương sai gradient của một mẫu, mà hiệp phương sai gradient **giữa hai mẫu đầu vào khác nhau** cũng suy giảm theo cùng depth-scale ξ_c chi phối tương quan tín hiệu forward. Điều này nối trực tiếp "thông tin tương quan có lan truyền được không" với "mạng có học được cấu trúc chung giữa các mẫu huấn luyện không".

(e) Giả thuyết trainability và kiểm chứng thực nghiệm. Từ (a)-(d), paper đưa ra giả thuyết: điều kiện cần để một mạng ngẫu nhiên huấn luyện được là thông tin phải lan truyền được cả hai chiều (forward và backward), nghĩa là độ sâu mạng L không được lớn hơn nhiều so với ξ_c. Giả thuyết này **không phụ thuộc dữ liệu**, chỉ phụ thuộc kiến trúc/siêu tham số, nên được gọi là chặn "phổ quát" (universal). Paper kiểm chứng bằng cách huấn luyện hàng loạt mạng FC sâu (độ sâu, σ_w^2, có/không dropout khác nhau) trên MNIST và CIFAR-10.

3. Phương pháp chi tiết

Nhắc lại khung mean-field (mục 2 của paper, trùng math 26). Mạng FC chưa huấn luyện, $z_i^l = \sum_j W_{ij}^l y_j^l + b_i^l$, $y_i^{l+1} = \phi(z_i^l)$, $W_{ij}^l \sim \mathcal{N}(0, \sigma_w^2/N_l)$, $b_i^l \sim \mathcal{N}(0, \sigma_b^2)$. Phương sai một đầu vào $q_{aa}^l = \sigma_w^2 \int \mathcal{D}z\, \phi^2(\sqrt{q_{aa}^{l-1}} z) + \sigma_b^2$; hiệp phương sai hai đầu vào $q_{ab}^l = \sigma_w^2 \int \mathcal{D}z_1 \mathcal{D}z_2\, \phi(u_1)\phi(u_2) + \sigma_b^2$; hệ số giãn $\chi_1 = \sigma_w^2 \int \mathcal{D}z\, [\phi'(\sqrt{q^*} z)]^2$; $\chi_1 = 1$ là ranh giới order-to-chaos.

Khai triển tiệm cận cho ξ_q. Đặt $q_{aa}^l = q^* + \epsilon_l$, khai triển bậc nhất quanh điểm cố định (xem phụ lục 7.1 của paper):

$$\epsilon_{l+1} = \epsilon_l \left[\chi_1 + \sigma_w^2 \int \mathcal{D}z\, \phi''(\sqrt{q^*} z) \phi(\sqrt{q^*} z) \right] + O(\epsilon_l^2),$$

cho nghiệm tiệm cận $\epsilon_l \sim e^{-l/\xi_q}$ với

$$\xi_q^{-1} = -\log \left[\chi_1 + \sigma_w^2 \int \mathcal{D}z\, \phi''(\sqrt{q^*} z) \phi(\sqrt{q^*} z) \right].$$

Với $\phi = \tanh$, số hạng thứ hai trong dấu ngoặc luôn **âm** (do tính đối xứng lẻ của tanh và ϕ''), nên dù $\chi_1 \to 1$, cả biểu thức trong ngoặc vẫn nhỏ hơn 1 một khoảng đáng kể; đây là lý do ξ_q luôn ngắn, không phân kỳ, kể cả gần điểm tới hạn.

Khai triển tiệm cận cho ξ_c. Đặt $c_{ab}^l = c^* + \epsilon_l$, khai triển tương tự (phụ lục 7.2):

$$\epsilon_{l+1} = \epsilon_l\, \sigma_w^2 \int \mathcal{D}z_1 \mathcal{D}z_2\, \phi'(u_1^*)\phi'(u_2^*) + O(\epsilon_l^2), \qquad \xi_c^{-1} = -\log \left[\sigma_w^2 \int \mathcal{D}z_1 \mathcal{D}z_2\, \phi'(u_1^*)\phi'(u_2^*) \right].$$

Trong pha trật tự ($c^* = 1$), biểu thức trong ngoặc rút gọn đúng thành χ_1, nên $\xi_c^{-1} = -\log \chi_1$: khi $\chi_1 \to 1$ (tức tiến tới ranh giới order-to-chaos), $\xi_c \to \infty$. Đây là kết quả trung tâm của mục này: ξ_c **phân kỳ chính xác** tại điểm tới hạn.

Dropout. Với dropout tỉ lệ ρ, tiền kích hoạt trở thành $z_i^l = \frac{1}{\rho} \sum_j W_{ij}^l p_j^l y_j^l + b_i^l$, $p_j^l \sim \text{Bernoulli}(\rho)$ (chọn độc lập cho mỗi đầu vào, giống cách dropout dùng trong thực tế). Phương sai một đầu vào (phụ lục 7.3): $\bar{q}_{aa}^l = \frac{\sigma_w^2}{\rho} \int \mathcal{D}z\, \phi^2(\sqrt{\bar{q}_{aa}^{l-1}} z) + \sigma_b^2$, tức tương đương việc rescale $\sigma_w^2 \to \sigma_w^2/\rho$ cho trường hợp một đầu vào.

Hiệp phương sai hai đầu vào (phụ lục 7.4) có **cùng dạng công thức** như không dropout. Nhưng khi thay $\bar{c}^l_{ab} = 1$ vào công thức này (phụ lục 7.5), kết quả là:

$$\bar{c}^{l+1}_{ab} = 1 - \frac{1-\rho}{\rho \bar{q}^*} \sigma_w^2 \int \mathcal{D}z\, \phi^2(\sqrt{\bar{q}^*} z),$$

số hạng trừ đi **luôn dương** với mọi $\rho < 1$, nên $\bar{c}^{l+1}_{ab} < 1$ bất kể $\bar{c}^l_{ab} = 1$: nói cách khác, $c^* = 1$ **không còn là điểm cố định nào cả** của động lực này, với mọi $\rho < 1$, dù nhỏ tới đâu.

Mean-field cho gradient. Backprop: $\partial E/\partial W^l_{ij} = \delta^l_i\, \phi(z^{l-1}_j)$, $\delta^l_i = \phi'(z^l_i) \sum_j \delta^{l+1}_j W^{l+1}_{ji}$. Với giả thiết bổ sung (giống feedback alignment, Lillicrap et al. 2014) rằng trọng số dùng ở forward và trọng số ”nhìn thấy” ở backward là độc lập, paper suy ra (phụ lục 7.6-7.7) công thức lặp cho $\tilde{q}^l_{aa} = E[(\delta^l_i)^2]$:

$$\tilde{q}^l_{aa} = \tilde{q}^{l+1}_{aa} \frac{N_{l+1}}{N_l} \chi_1(q^l_{aa}).$$

Với mạng chiều rộng không đổi ($N_{l+1}/N_l = 1$), nghiệm là $\tilde{q}^l_{aa} = \tilde{q}^L_{aa}\, e^{-(L-l)/\xi_\nabla}$, $\xi_\nabla^{-1} = -\log \chi_1$ (đúng cả hai phía của điểm tới hạn, nên ξ_∇ có thể âm hoặc dương). Ba chế độ: $\chi_1 < 1 \Rightarrow \xi_\nabla > 0$ (gradient biến mất theo độ sâu $|\xi_\nabla|$); $\chi_1 \to 1 \Rightarrow \xi_\nabla \to \infty$ (gradient ổn định); $\chi_1 > 1 \Rightarrow \xi_\nabla < 0$ (gradient nổ theo độ sâu $|\xi_\nabla|$).

Hiệp phương sai gradient giữa hai mẫu. Tương tự (phụ lục 7.8-7.9): $\tilde{q}^l_{ab} = \tilde{q}^{l+1}_{ab} \frac{N_{l+1}}{N_{l+2}} \sigma_w^2 \int \mathcal{D}z_1 \mathcal{D}z_2\, \phi'(u_1)\phi'(u_2)$, có nghiệm tiệm cận bậc không với **cùng** ξ_c như ở forward pass.

4. Thí nghiệm và kết quả chính

- **Hình 1 (phase diagram và hội tụ):** (a) biểu đồ pha (σ_w^2, σ_b^2) cho $\phi = \tanh$, phân chia order/chaos. (b)-(c) phần dư $|q^* - q^l_{aa}|$ và $|c^* - c^l_{ab}|$ theo độ sâu, trục log, với $\sigma_b^2 = 0{,}05$ cố định và σ_w^2 từ $0{,}01$ (đỏ) tới $1{,}7$ (tím): cả hai cho thấy **hành vi mũ rõ ràng** qua nhiều bậc độ lớn.
- **Hình 2 (đo depth-scale):** (a) bản đồ tương quan lặp với 3 giá trị $\sigma_w^2 = 0{,}5$ (trật tự, xa), $1{,}8$ (gần edge of chaos), $3{,}5$ (hỗn loạn sâu); đường tuyến tính hoá quanh điểm tới hạn có hệ số góc e^{-1/ξ_c}. (b) ξ_q theo σ_w^2 với σ_b^2 từ $0{,}01$ tới $0{,}3$: lý thuyết (nét đứt) khớp đo đạc (nét liền), ξ_q luôn ngắn và hữu hạn. (c) ξ_c theo σ_w^2, cùng dải σ_b^2: **phân kỳ rõ rệt tại điểm tới hạn**. Phương pháp đo: lặp công thức với $q^0_{aa} = q^0_{bb} = 0{,}8$, $c^0_{ab} = 0{,}6$, $\sigma_w^2 \in [0{,}1, 3{,}0]$, $\sigma_b^2 \in [0{,}01, 0{,}3]$, fit phần dư theo hàm mũ để suy ra depth-scale đo được.
- **Hình 3 (dropout phá điểm tới hạn):** (a) bản đồ tương quan với dropout $\rho = 1{,}0$, $0{,}95$, $0{,}9$ gần điểm tới hạn: $\bar{c}^l_{ab} = 1$ không còn là điểm cố định, điểm cố định mới giảm dần khi ρ giảm. (b) c^* theo σ_w^2 với ρ từ $0{,}8$ đến $1{,}0$: **không còn chuyển pha sắc nét** với mọi $\rho < 1$. (c) ξ_c theo σ_w^2 cùng dải ρ: **không còn phân kỳ** với $\rho < 1$, và ξ_c giảm khi ρ giảm ở cùng σ_w^2.
- **Hình 4 (gradient backprop):** mạng ngẫu nhiên sâu $L = 240$, chiều rộng $N_l = 300$, hàm lỗi cross-entropy trên MNIST. (a) chuẩn-2 gradient theo lớp $\|\nabla W^l_{ab} E\|_2^2$ với σ_w^2 từ $1{,}0$ (xanh) tới $4{,}0$ (đỏ): hành vi mũ rõ rệt qua nhiều bậc độ lớn, gradient biến mất ở pha trật tự, nổ ở pha hỗn loạn. (b) $|\xi_\nabla|$ đo được so với lý thuyết: khớp tốt, lệch chủ yếu gần điểm tới hạn do hiệu ứng kích thước hữu hạn.
- **Mục 5 (thực nghiệm trainability, kết quả thực dụng chính của paper):** mạng FC độ sâu $10 \le L \le 300$, $1 \le \sigma_w^2 \le 4$, $\sigma_b^2 = 0{,}05$; huấn luyện bằng SGD và RMSProp trên MNIST và CIFAR-10. Learning rate: SGD 10^{-3} khi $L \lesssim 200$, 10^{-4} khi L lớn hơn; RMSProp 10^{-5}; các giá trị này chọn bằng grid search từ 10^{-6} đến 10^{-2} theo bước nhân 10. Hình 5: (a) MNIST, SGD, 200 bước; (b) CIFAR-10, SGD, 2000 bước; (c) MNIST, SGD, 14000 bước; (d) MNIST, RMSProp, 300 bước. Ở mọi trường hợp, mạng cuối cùng overfit tới train accuracy 100%, test accuracy 98% trên MNIST và 55% trên CIFAR-10 (mục tiêu thí nghiệm là chứng minh ”huấn luyện được”, không phải tối ưu test accuracy). Overlay các đường $n\xi_c$ ($n = 1$ đến 6) lên panel (a): mạng có vẻ **huấn luyện được khi** $L \lesssim 6\xi_c$ (paper tự ghi ”sẽ thú vị nếu hiểu được tại sao là hệ số 6”); overlay riêng $6\xi_c$ lên các panel (b)-(d) cho thấy ranh giới này **nhất quán giữa dữ liệu, thời gian huấn luyện, và optimizer**, nên được gọi là chặn ”phổ quát”.

- **Hình 6 (trainability với dropout):** cùng khung thí nghiệm trên MNIST với $\rho = 0{,}99, 0{,}98, 0{,}94$. Ngay cả dropout rất nhẹ ($\rho = 0{,}99$) cũng **giới hạn độ sâu khả huấn luyện tối đa xuống khoảng** $L = 100$; ξ_c (tính theo công thức dropout ở mục 3) vẫn dự đoán đúng vùng huấn luyện được ứng với từng mức dropout.
- **Phụ lục 7.10:** Hình 7 (độ chính xác huấn luyện MNIST sau 45/304/2048/13780 bước SGD, lr 10^{-3}) và Hình 8 (tương tự, sau 45/304/2048/13780 bước RMSProp, lr 10^{-5}) cho thấy quan hệ L so với ξ_c giữ nguyên dạng qua các mốc thời gian huấn luyện khác nhau.

5. Phân tích / ablation

Paper quét hệ thống nhiều trục cùng lúc, và chính sự **nhất quán xuyên suốt** các trục này là luận điểm thực nghiệm trung tâm:

- **Độ sâu L:** từ 10 đến 300 lớp.
- **Phương sai trọng số σ_w^2:** từ 1 đến 4 cho thí nghiệm trainability (và rộng hơn, 0,1 đến 3,0, khi đo depth-scale ở Hình 2).
- **Phương sai bias σ_b^2:** cố định $0{,}05$ cho thí nghiệm trainability; quét $0{,}01$ đến $0{,}3$ khi đo depth-scale.
- **Dữ liệu:** MNIST và CIFAR-10.
- **Optimizer:** SGD và RMSProp.
- **Thời gian huấn luyện:** từ 45 đến 14.000 bước.
- **Dropout:** ρ từ $0{,}8$ đến $1{,}0$ cho phân tích lý thuyết; $\rho = 0{,}99/0{,}98/0{,}94$ cho thực nghiệm trainability.

Kết quả ”ablation” đáng chú ý nhất: ranh giới $L \lesssim 6\xi_c$ **không đổi dạng** khi hoán đổi dữ liệu (MNIST/CIFAR-10), optimizer (SGD/RMSProp), hay thời gian huấn luyện (200 đến 14.000 bước), ủng hộ tuyên bố ”chặn phổ quát, không phụ thuộc dữ liệu”. Riêng trục dropout cho một ”ablation” định lượng rõ ràng: $\rho = 0{,}99 \to 0{,}98 \to 0{,}94$ làm độ sâu khả huấn luyện tối đa giảm dần, và ξ_c (công thức dropout) tiếp tục dự đoán đúng từng mức. Cuối cùng, việc so sánh trực tiếp Hình 2(b) và 2(c) (tức ξ_q so với ξ_c trên cùng dải tham số) là một phép ”ablation” khái niệm: cho thấy chính ξ_c, không phải ξ_q, là đại lượng quyết định trainability, vì ξ_q luôn ngắn nên không đổi nhiều theo việc mạng gần hay xa điểm tới hạn.

6. Giới hạn

Bài báo nêu rõ một số hạn chế ở phần Discussion:

- Khung lý thuyết **không áp dụng cho hàm hoạt hoá không chặn** như ReLU, vì có những vùng tham số nơi công thức độ dài (eq. 3, giống math 26) **không có điểm cố định**.
- Phân tích chỉ áp dụng trực tiếp cho **mạng kết nối đầy đủ, truyền thẳng**; bài báo lưu ý cần mở rộng cho kiến trúc có ma trận trọng số có cấu trúc, như CNN.
- Mục tiêu thí nghiệm mục 5 là chứng minh ”huấn luyện được” (trainable), **không phải** tối ưu test accuracy; test accuracy đạt được (98% MNIST, 55% CIFAR-10) chỉ là sản phẩm phụ, không nên dùng để so sánh với mô hình SOTA.
- Hệ số 6 trong ranh giới $L \lesssim 6\xi_c$ là quan sát thực nghiệm, **chưa có suy luận lý thuyết** cho giá trị cụ thể này; paper tự đặt câu hỏi mở này, không trả lời.
- Lý thuyết gradient cần thêm một giả thiết (trọng số forward và ”nhìn thấy” ở backward độc lập, giống feedback alignment) **ngoài** giả thiết mean-field gốc; paper không định lượng sai số của giả thiết này, chỉ kiểm chứng gián tiếp qua khớp số liệu ở Hình 4.
- Công thức gradient (eq. 15) giả định chiều rộng các lớp bằng nhau ($N_{l+1}/N_l = 1$); trường hợp chiều rộng không đều được paper **nhường lại** cho tài liệu khác (Glorot và Bengio, 2010), không tự suy ra trong bài này.

7. Liên hệ với các bài khác

- **math 26 (Exponential expressivity... through transient chaos):** nền tảng trực tiếp; mọi công thức ở mục ”Background” của bài 27 (length map, correlation map, χ_1) là lấy nguyên từ bài 26, chỉ đổi mục tiêu phân tích từ ”biểu diễn được gì” sang ”huấn luyện được không”.
- **math 18 (Shattered Gradients Problem):** cùng nói về gradient sâu trở nên ”giống nhiễu trắng” theo độ sâu nếu không có kiến trúc phù hợp; math 27 cho công cụ định lượng (ξ_∇, χ_1) để hình thức hoá đúng hiện tượng mà math 18 quan sát bằng thực nghiệm trên ReLU/residual network.
- **math 22 (Deep Neural Networks as Gaussian Processes):** paper tự liên hệ trực tiếp trong phần Discussion: hiệp phương sai q^l_{ab} chính là kernel của Gaussian Process cảm sinh (Neal, 2012) khi mạng vô hạn rộng; những trường hợp thông tin tương quan lan truyền được (tức ξ_c lớn) tương ứng chính xác với những trường hợp GP cảm sinh ”well-defined” (có kernel không suy biến).
- **math 23 (Neural Tangent Kernel):** cũng dùng công cụ kernel/mean-field ở giới hạn chiều rộng lớn để phân tích động lực gradient của mạng, nhưng theo hướng khác (kernel NTK không đổi trong suốt huấn luyện ở giới hạn vô hạn rộng), bổ sung góc nhìn cho lý thuyết backprop ”hữu hạn độ sâu, hữu hạn chiều rộng” của bài 27.
- **math 20 (Benefits of depth in neural networks):** cùng câu hỏi lớn ”độ sâu có ích gì”, nhưng bài 27 trả lời ở khía cạnh ”huấn luyện được hay không” thay vì ”biểu diễn được gì”, bổ khuyết cho nhau.

8. Gợi ý áp dụng

Phân tích lan truyền tín hiệu giúp lựa chọn thang khởi tạo để tránh tín hiệu hoặc gradient suy giảm hay bùng nổ theo độ sâu. Các dự đoán dựa trên giả thiết mean-field cần được kiểm tra khi kiến trúc hoặc phân phối trọng số khác mô hình phân tích.

9. Thí nghiệm gợi ý

1. **Tái lập Hình 2(b)(c) ở quy mô nhỏ:** lặp số bản đồ độ dài/tương quan cho $\phi = \tanh$ (hoàn toàn trên CPU, không cần GPU), đo ξ_q, ξ_c theo σ_w^2 ở vài giá trị σ_b^2, xác nhận ξ_c phân kỳ tại $\chi_1 = 1$ còn ξ_q luôn hữu hạn/ngắn.
2. **Lưới trainability thu nhỏ:** huấn luyện mạng FC tanh sâu (độ sâu 10-100, rộng 100-300, vừa trên GPU) trên tập con MNIST, quét σ_w^2 quanh điểm tới hạn, kiểm tra xem ranh giới $L \lesssim 6\xi_c$ có xấp xỉ đúng ở quy mô nhỏ hơn nhiều so với Hình 5 của paper (độ sâu tới 300) hay không.
3. **Tái lập rút gọn Hình 4 (gradient theo lớp):** dựng mạng ngẫu nhiên sâu 100-240 lớp, rộng 200-300 , đo chuẩn-2 gradient theo lớp trên **một batch** MNIST (chỉ cần 1 lần forward + backward, không cần huấn luyện), so tốc độ tăng/giảm đo được với dự đoán lý thuyết $\xi_\nabla^{-1} = -\log \chi_1$.
4. **Trainability với dropout thu nhỏ:** lặp lại thí nghiệm 2 nhưng thêm dropout $\rho = 0{,}99/0{,}95/0{,}9$, kiểm tra độ sâu khả huấn luyện tối đa có giảm dần đúng như Hình 6 của paper hay không, ở quy mô mạng nhỏ hơn (độ sâu 20-150).

10. Tóm tắt công thức / khái niệm

- **Depth-scale biên độ:** $\xi_q^{-1} = -\log\left[\chi_1 + \sigma_w^2 \int \mathcal{D}z\, \phi''(\sqrt{q^*}z)\phi(\sqrt{q^*}z)\right]$; luôn hữu hạn, ngắn.
- **Depth-scale tương quan:** $\xi_c^{-1} = -\log\left[\sigma_w^2 \int \mathcal{D}z_1 \mathcal{D}z_2\, \phi'(u_1^*)\phi'(u_2^*)\right]$; trong pha trật tự bằng $-\log \chi_1$; **phân kỳ tại** $\chi_1 = 1$.
- **Dropout phá điểm tới hạn:** $\tilde{c}_{ab}^{l+1} = 1 - \frac{1-\rho}{\rho \tilde{q}^*}\sigma_w^2 \int \mathcal{D}z\, \phi^2(\sqrt{\tilde{q}^*}z) < 1$ với mọi $\rho < 1$; không còn điểm cố định $c^* = 1$.
- **Depth-scale gradient:** $\xi_\nabla^{-1} = -\log \chi_1$ (cả hai phía điểm tới hạn); $\chi_1 < 1$ gradient biến mất, $\chi_1 > 1$ gradient nổ, $\chi_1 \to 1$ gradient ổn định.
- **Quy tắc trainability thực nghiệm:** mạng huấn luyện được khi độ sâu $L \lesssim 6\xi_c$; với dropout, ngưỡng này co lại nhanh (ví dụ $\rho = 0{,}99$ còn khoảng $L \approx 100$).

- **Tự tính (ví dụ 1, depth-scale gradient):** giả sử $\chi_1 = 0{,}95$ (pha trật tự, hơi dưới điểm tới hạn). $\xi_\nabla = -1/\ln(0{,}95)$. Tính $\ln(0{,}95) \approx -0{,}05129$, nên $\xi_\nabla \approx 1/0{,}05129 \approx 19{,}5$ lớp. Sau $L = 240$ lớp (đúng cấu hình Hình 4 của paper), gradient suy giảm theo hệ số $e^{-240/19{,}5} = e^{-12{,}3} \approx 4{,}5 \times 10^{-6}$ (tự tính), khớp với mô tả ”biến mất qua nhiều bậc độ lớn” của paper ở pha trật tự.
- **Tự tính (ví dụ 2, quy tắc $6\xi_c$):** nếu đo được $\xi_c = 50$ lớp tại một cấu hình (σ_w, σ_b) cho trước (số giả định để minh hoạ cách dùng công thức, không phải số trong paper), quy tắc thực nghiệm của paper dự đoán độ sâu khả huấn luyện tối đa xấp xỉ $6 \times 50 = 300$ lớp (tự tính).

11. Kết luận ngắn

Bài này mở rộng khung mean-field của math 26 bằng cách định nghĩa hai depth-scale ξ_q (luôn ngắn) và ξ_c (phân kỳ đúng tại điểm tới hạn order-to-chaos), chỉ ra dropout phá vỡ điểm tới hạn này với mọi tỉ lệ $\rho < 1$, xây một lý thuyết mean-field đối ngẫu cho gradient backprop nối trực tiếp pha trật tự/hỗn loạn với vanishing/exploding gradient, và kiểm chứng bằng thực nghiệm rằng mạng sâu huấn luyện được chính xác khi độ sâu không vượt quá khoảng $6\xi_c$, một quy tắc có vẻ phổ quát qua dữ liệu, optimizer, và thời gian huấn luyện khác nhau.

Nguồn gốc: https://arxiv.org/abs/1611.01232

Bài 28

Geometric Deep Learning: Grids, Groups, Graphs, Geodesics, and Gauges (2021)

Tác giả: Michael M. Bronstein (Imperial College London / Twitter), Joan Bruna (New York University), Taco Cohen (Qualcomm AI Research), Petar Veličković (DeepMind).
Venue / arXiv: arXiv:2104.13478 (v2, 02/05/2021). Paper không nêu venue hội nghị được bình duyệt: đây là một cuốn sách/monograph dài (phần nội dung chính dài khoảng 127 trang, cộng thêm danh mục tham khảo rất dài phía sau). Lời cảm ơn ở cuối bài có nhắc việc nội dung được trình bày dưới dạng bài keynote tại ICLR 2021, nhưng bản thân văn bản không phải một bài báo hội nghị qua phản biện.
Nguồn: https://arxiv.org/abs/2104.13478
Chủ đề: một khung lý thuyết (gọi là ”Geometric Deep Learning”, GDL) thống nhất phần lớn kiến trúc deep learning phổ biến (CNN, GNN, Deep Sets, Transformer, Mesh CNN, Spherical CNN, RNN/LSTM) dưới bốn nguyên lý hình học chung: đối xứng (nhóm bất biến/tương đương), ổn định biến dạng, tách biệt tỉ lệ, và một ”blueprint” gồm 4 khối xây dựng; áp dụng lên 5 miền hình học gọi là ”5 Gs”: Grids (lưới), Groups (nhóm/không gian đồng nhất), Graphs (đồ thị/tập hợp), Geodesics (đa tạp/metric nội tại), Gauges (trường quy chiếu cục bộ trên bundle). Ghi chú tập trung vào các chương 1–5; các phần sau được giới thiệu chọn lọc.
Vị trí trong lộ trình: nên đọc sau math 01 (matrix calculus) và math 02 (automatic differentiation) để quen ký hiệu đạo hàm/toán tử tuyến tính dùng xuyên suốt bài. Nên đọc cùng lúc hoặc trước llm 01 (Attention is All You Need), llm 29 (ViT), llm 21 (Mamba): bài này cung cấp một ”kính lý thuyết” (nhóm đối xứng, bất biến/tương đương) để nhìn lại ba kiến trúc đó như các trường hợp riêng của cùng một blueprint, chứ không phải ba phát minh độc lập.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu, công thức và số thứ tự phương trình lấy từ bản full text arXiv (tải ngày 09/11/2026); các phép tính đánh dấu ”tự tính” là của người ghi chú, không có trong paper. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc. Đây là một cuốn sách rất dài, ghi chú chỉ chọn phần cốt lõi theo đúng phạm vi đọc đã nêu ở mục ”Chủ đề”.

1. Bối cảnh

CNN, RNN, GNN và Transformer được phát triển cho những miền dữ liệu khác nhau và thường được trình bày bằng các ký hiệu riêng. Bronstein, Bruna, Cohen và Veličković đặt câu hỏi liệu các nguyên lý hình học chung có thể giúp giải thích và hệ thống hóa những kiến trúc này hay không.

Động lực toán học nằm ở ”curse of dimensionality” (lời nguyền chiều cao, chương 2 của paper). Nếu hàm mục tiêu f chỉ được giả định là 1-Lipschitz trên không gian d chiều, số mẫu cần để học f với sai số cố định tăng theo cấp lũy mũ với d: paper định nghĩa độ phân kỳ cực đại (maximum discrepancy) $\kappa(d) \simeq N^{-1/d}$ cho lớp hàm Lipschitz(1), nên để có $\kappa(d) \leq \epsilon$ cần $N = \Theta(\epsilon^{-d})$ mẫu. Thay giả định Lipschitz bằng lớp Sobolev H^s cũng không giúp nhiều: tốc độ hội tụ minimax là $\epsilon^{-d/s}$, chỉ cải thiện đáng kể khi độ mịn s tỉ lệ với d, một giả định phi thực tế. Mạng fully-connected với regularization thưa (sparsity) có thể phá vỡ lời nguyền này (Bach, 2017) nhưng phải giả định f chỉ phụ thuộc vào một số ít tổ hợp tuyến tính chiều thấp của đầu vào, điều cũng phi thực tế với ảnh, âm thanh, hoá học.

Lối thoát mà paper đề xuất là khai thác cấu trúc hình học sẵn có của miền dữ liệu Ω (lưới ảnh, đồ thị xã hội, mặt cầu, đa tạp 3D...) thông qua hai nguyên lý: đối xứng (symmetry) và tách biệt tỉ lệ (scale separation), lấy cảm hứng trực tiếp từ Erlangen Programme của Felix Klein (ngoài paper, kiến thức chung: 1872): ”hình học là nghiên cứu các bất biến dưới một nhóm biến đổi”. Tên ”Geometric Deep Learning” được một trong các tác giả (Bronstein) đặt ra từ 2015; cái tên rút gọn ”4G: Grids, Graphs, Groups, Gauges” do Max Welling đề xuất cho chương trình ELLIS; paper này thêm chữ G thứ 5, Geodesics, để nói về các bất biến metric nội tại trên đa

tạp. Các tác giả tự nhận ”chữ G cuối (Gauges) hơi gượng ép” vì cấu trúc bên dưới thực ra là bundle/manifold chứ không hẳn là ”gauge”, nhưng vẫn giữ tên cho vần điệu.

2. Ý tưởng chính

Bài báo mô tả nhiều kiến trúc học sâu bằng một khung chung gồm bốn khối xây dựng (mục 3.4), với miền hình học Ω và nhóm đối xứng G được lựa chọn theo bài toán. Paper hệ thống hoá 5 miền hình học, gọi tắt ”5 Gs”:

G	Miền Ω	Nhóm đối xứng G	Kiến trúc tiêu biểu
Graphs / Sets	đồ thị, tập hợp không thứ tự	hoán vị Σ_n	GNN, Deep Sets, Transformer (trên đồ thị đầy đủ)
Grids	lưới Euclid (ảnh, chuỗi)	tịnh tiến	CNN
Groups	không gian đồng nhất (mặt cầu...)	$SO(3)$, nhóm Lie khác	Spherical CNN
Geodesics	đa tạp Riemann	đẳng cự (isometry)	Mesh/Intrinsic CNN (phần metric)
Gauges	bundle tiếp xúc / bundle đặc trưng	$SO(2)$ (đối khung quy chiếu)	gauge-equivariant Mesh CNN

Paper còn liệt kê thêm LSTM ứng với miền lưới 1D thời gian và nhóm ”time warping” (không phải nhóm tịnh tiến thường), minh hoạ rằng ngay cả tính bất biến ”lạ” (bất biến lớp, không phải bất biến zero-shot) cũng rơi vào cùng khung lý thuyết.

Hai ý chính cần nắm:

1. **Đối xứng tạo ra lớp hàm equivariant/invariant hẹp hơn, học hiệu quả hơn.** Một hàm f là G-bất biến (invariant) nếu $f(\rho(g)x) = f(x)$, và G-tương đương (equivariant) nếu $f(\rho(g)x) = \rho(g)f(x)$, với $\rho(g)$ là biểu diễn (representation) của nhóm G tác động lên không gian tín hiệu. Ví dụ convolution trong CNN là tịnh tiến-tương đương, không phải tịnh tiến-bất biến; global average pooling sau đó mới tạo ra bất biến.
2. **Bất biến đúng nhất khi kết hợp với tách biệt tỉ lệ (locality + đa tỉ lệ).** Hàm tuyến tính bất biến tuyệt đối chỉ có thể là phép lấy trung bình toàn cục (quá nghèo thông tin), nên cần xen lớp phi tuyến giữa các lớp tương đương cục bộ (local), rồi mới bất biến hoá ở lớp cuối. Đây chính là mô hình ”convolution + nonlinearity + pooling + global pooling” của CNN, tổng quát hoá thành blueprint 4 khối ở mục 3.4.

3. Phương pháp chi tiết

3.1 Nguyên lý 1: đối xứng, biểu diễn nhóm, bất biến và tương đương (chương 3.1-3.2)

Một nhóm G là tập hợp cùng phép hợp thành $\circ$ thoả 4 tiên đề: kết hợp, có phần tử đơn vị e, mọi phần tử có nghịch đảo, đóng kín. Nhóm không bắt buộc giao hoán; nhóm giao hoán ($gh = hg$ với mọi g, h) gọi là nhóm Abel. Một nhóm tác động (group action) lên miền Ω là ánh xạ $(g, u) \mapsto g.u$ thoả $g.(h.u) = (gh).u$. Từ đó suy ra tác động của G lên không gian tín hiệu $\mathcal{X}(\Omega)$ qua phương trình (3) của paper:

$$(g.x)(u) = x(g^{-1}u)$$

Biểu diễn nhóm (group representation) $\rho : G \to \mathbb{R}^{n\times n}$ gán cho mỗi g một ma trận khả nghịch $\rho(g)$ thoả $\rho(gh) = \rho(g)\rho(h)$. Hai định nghĩa trung tâm của toàn bài:

- f là G-bất biến nếu $f(\rho(g)x) = f(x)$ với mọi $g \in G$.
- f là G-tương đương nếu $f(\rho(g)x) = \rho(g)f(x)$ với mọi $g \in G$.

Ví dụ kinh điển: một lớp convolution trong CNN là tịnh tiến-tương đương (dịch ảnh đầu vào thì feature map đầu ra dịch theo cùng lượng), còn hàm phân loại ảnh toàn cục (dán nhãn "mèo"/"chó") cần tịnh tiến-bất biến. Multi-layer Perceptron xấp xỉ được mọi hàm trơn nhưng không có tính tương đương này, một phần lý do các mô hình pattern recognition đầu tiên (thập niên 1970) thất bại trên ảnh.

Chương 3.2 phân biệt **isomorphism** (ánh xạ song ánh bảo toàn cấu trúc giữa hai đối tượng khác nhau, ví dụ hai đồ thị đẳng cấu chỉ khác thứ tự đánh số nút) và **automorphism** (isomorphism từ một đối tượng về chính nó, tức là một đối xứng thật sự). Tuỳ mức cấu trúc cần bảo toàn mà ta có các lớp ánh xạ lồng nhau: song ánh (set) $\supset$ homeomorphism (bảo toàn liên tục, không gian topo) $\supset$ diffeomorphism (bảo toàn vi phân, đa tạp trơn) $\supset$ isometry (bảo toàn khoảng cách, không gian metric). Thêm cấu trúc thì nhóm đối xứng càng nhỏ: $Diff(\Omega) \supset E(2) \supset SE(2)$ (nhóm Euclid đầy đủ rồi tới nhóm chỉ giữ hướng), phản ánh đúng thứ bậc hình học Xạ ảnh $\supset$ Affine $\supset$ Euclid của Erlangen Programme.

3.2 Nguyên lý 2: ổn định biến dạng (chương 3.3)

Yêu cầu bất biến/tương đương chính xác với một nhóm cố định là quá cứng: biến dạng nhỏ (ví dụ hai vật thể trong video di chuyển theo hai hướng khác nhau) không tạo thành nhóm (hợp của nhiều biến dạng nhỏ có thể thành biến dạng lớn, phá vỡ tính đóng kín). Paper thay bằng khái niệm **ổn định hình học** (geometric stability), đo "khoảng cách" $c(\tau)$ từ một phép biến dạng $\tau \in Diff(\Omega)$ tới nhóm đối xứng G ($c(\tau) = 0$ khi $\tau \in G$), và yêu cầu (phương trình 4):

$$\|f(\rho(\tau)x) - f(x)\| \leq C\, c(\tau)\, \|x\|, \quad \forall x \in \mathcal{X}(\Omega)$$

Một lựa chọn phổ biến cho ảnh liên tục là năng lượng Dirichlet $c^2(\tau) := \int_\Omega \|\nabla \tau(u)\|^2 du$, đo độ "đàn hồi" của τ so với trường dịch chuyển hằng. Khi miền Ω chính nó biến dạng (ví dụ đồ thị xã hội thay đổi theo thời gian, hoặc vật thể 3D không cứng), paper định nghĩa khoảng cách domain $d_{\mathcal{D}}(\Omega, \tilde{\Omega})$ (ví dụ graph edit distance, hoặc khoảng cách Gromov-Hausdorff cho đa tạp Riemann) và yêu cầu tương tự (phương trình 5):

$$\|f(x, \Omega) - f(\tilde{x}, \tilde{\Omega})\| \leq C\|x\|\, d_{\mathcal{D}}(\Omega, \tilde{\Omega})$$

Ý quan trọng: biểu diễn Fourier (toàn cục) ổn định với đúng nhóm đối xứng (ví dụ tịnh tiến) nhưng **không** ổn định với biến dạng nhỏ gần nhóm đó: với tịnh tiến gần đúng $\tau(u) = u - \tilde{\tau}(u)$, $\|\nabla \tilde{\tau}\|_\infty \leq \epsilon$ nhỏ bất kỳ, hàm module-Fourier $f(x) = |\hat{x}|$ vẫn có $\|f(\rho(\tau)x) - f(x)\|/\|x\| = O(1)$, không giảm theo ϵ. Ngược lại biểu diễn wavelet (đa tỉ lệ, cục bộ cả không gian và tần số) ổn định hơn: $\|\rho(\tau)(W_\psi x) - W_\psi(\rho(\tau)x)\|/\|x\| = O(\epsilon)$ (Mallat, 2012). Đây là lý do toán học cho việc dùng filter cục bộ (như convolution kernel 3x3) thay vì biến đổi Fourier toàn cục.

3.3 Nguyên lý 3: tách biệt tỉ lệ (chương 3.4)

Ổn định biến dạng thu hẹp không gian hàm hợp lệ nhưng chưa đủ phá lời nguyền chiều cao. Paper đề xuất phân rã Ω thành một dãy miền thô dần $\Omega_1, \ldots, \Omega_J$ (coarsening, ví dụ pooling trong CNN hoặc gộp cụm trong GNN), và nói một hàm f "ổn định cục bộ tại tỉ lệ j" nếu $f \approx f_j \circ P_j$ với $P_j : \mathcal{X}(\Omega) \to \mathcal{X}_j(\Omega_j)$ là một phép rút gọn phi tuyến. Nghĩa là thay vì học tương tác tầm xa trực tiếp, mô hình học tương tác cục bộ trước, rồi truyền lên tỉ lệ thô hơn dần, đúng tinh thần renormalization group trong vật lý thống kê và Fast Multipole Method trong tính toán số.

3.4 Blueprint của Geometric Deep Learning (chương 3.5, "mục quan trọng nhất của paper")

Quan sát mở đầu: nếu f tuyến tính và G-bất biến thì f chỉ có thể là trung bình nhóm $Ax = \frac{1}{\mu(G)} \int_G (g.x)\, d\mu(g)$ (với ảnh và nhóm tịnh tiến, điều này suy biến thành "chỉ dùng màu RGB trung bình của ảnh", quá nghèo). Ngược lại họ hàm tuyến tính G-**tương đương** (không bất biến) phong phú hơn nhiều, và có thể ghép với phi

tuyến σ bất kỳ mà vẫn giữ tương đương: nếu B là G-tương đương và σ áp dụng theo từng điểm thì $\sigma \circ B$ vẫn G-tương đương. Từ đó paper định nghĩa 4 khối xây dựng:

- **Lớp tuyến tính G-tương đương** $B : \mathcal{X}(\Omega, C) \to \mathcal{X}(\Omega', C')$, $B(g.x) = g.B(x)$.
- **Phi tuyến theo điểm** $\sigma : C \to C'$, áp dụng từng toạ độ: $(\sigma(x))(u) = \sigma(x(u))$.
- **Local pooling (coarsening)** $P : \mathcal{X}(\Omega, C) \to \mathcal{X}(\Omega', C)$ với Ω' là bản thu gọn của Ω.
- **Lớp G-bất biến (global pooling)** $A : \mathcal{X}(\Omega, C) \to Y$, $A(g.x) = A(x)$.

Ghép lại thành công thức chung cho toàn bộ "Geometric Deep Learning Blueprint":

$$f = A \circ \sigma_J \circ B_J \circ P_{J-1} \circ \cdots \circ P_1 \circ \sigma_1 \circ B_1$$

Mỗi lớp B_j có thể dùng nhóm G khác nhau. Bảng tương ứng kiến trúc, miền, nhóm mà paper đưa ra ngay sau blueprint này chính là nguồn của bảng 5G ở mục 2; CNN = (Grid, tịnh tiến), Spherical CNN = (mặt cầu, $SO(3)$), Intrinsic/Mesh CNN = (đa tạp, đẳng cự hoặc gauge $SO(2)$), GNN/Deep Sets/Transformer = (đồ thị/tập/đồ thị đầy đủ, hoán vị Σ_n), LSTM = (lưới 1D, time warping).

3.5 Năm miền hình học, "5 Gs" (chương 4.1-4.6)

Graphs và Sets (4.1). Tập hợp là đồ thị không cạnh. Với ma trận đặc trưng $X \in \mathbb{R}^{n \times d}$ (mỗi dòng một nút), phép hoán vị $g \in \Sigma_n$ ứng với ma trận hoán vị P (mỗi dòng/cột đúng một số 1). $f(X)$ bất biến hoán vị nếu $f(PX) = f(X)$; dạng đơn giản nhất (phương trình 6): $f(X) = \phi(\sum_u \psi(x_u))$. $F(X)$ tương đương hoán vị nếu $F(PX) = PF(X)$; phép tuyến tính đơn giản nhất là $F_\Theta(X) = X\Theta$ (phương trình 7). Khi có cạnh, ma trận kề A (phương trình 8) cũng bị hoán vị theo $PAP^\top$, và định nghĩa bất biến/tương đương tổng quát hoá thành $f(PX, PAP^\top) = f(X, A)$ (phương trình 9) và $F(PX, PAP^\top) = PF(X, A)$ (phương trình 10). Với lân cận 1-hop N_u (phương trình 11) và multiset đặc trưng lân cận X_{N_u} (phương trình 12), blueprint cho đồ thị là áp một hàm cục bộ $\phi(x_u, X_{N_u})$ bất biến hoán vị lên từng nút (phương trình 13): $F(X, A) = (\phi(x_1, X_{N_1}), \ldots, \phi(x_n, X_{N_n}))^\top$. Khi ϕ là đơn ánh (injective), một bước của ϕ tương đương một bước của phép kiểm thử đẳng cấu Weisfeiler-Lehman (WL test).

Grids (4.2). Lưới là đồ thị đặc biệt với thứ tự nút cố định (vòng $\mathbb{Z}_n$). Vì lân cận có thứ tự ("trái"/"phải"), ϕ không cần bất biến hoán vị nữa: $\phi(x_{u-1}, x_u, x_{u+1}) = \theta_{-1}x_{u-1} + \theta_0 x_u + \theta_1 x_{u+1}$ tạo ra ma trận circulant $C(\theta)$, và $C(\theta)x = x \star \theta$ chính là convolution rời rạc. Các ma trận circulant giao hoán với nhau và với toán tử dịch (shift) S: $SC(\theta)x = C(\theta)Sx$, tức convolution là tịnh tiến-tương đương; ngược lại, một toán tử tuyến tính tịnh tiến-tương đương bắt buộc phải là convolution. Paper suy ra biến đổi Fourier rời rạc (DFT) từ nguyên lý đối xứng thuần túy: các ma trận circulant giao hoán nên đồng chéo hoá được, và vì toán tử dịch S là circulant, các trị riêng của nó (căn bậc n của đơn vị) chính là cơ sở Fourier $\phi_k = \frac{1}{\sqrt{n}}(1, e^{2\pi i k/n}, \ldots)$. Đây là một trong những đoạn đẹp nhất của paper: Fourier transform không phải "cho sẵn" mà được suy ra từ yêu cầu đồng chéo hoá các toán tử tương đương-tịnh tiến.

Groups và Homogeneous spaces (4.3). Tổng quát hoá convolution: với nhóm G tác động lên Ω và biểu diễn ρ, group convolution (phương trình 14):

$$(x \star \theta)(g) = \langle x, \rho(g)\theta \rangle = \int_\Omega x(u)\, \theta(g^{-1}u)\, du$$

luôn G-tương đương, suy trực tiếp từ $\langle x, \rho(g)\theta \rangle = \langle \rho(g^{-1})x, \theta \rangle$. Với $\Omega = \mathbb{Z}_n$, $G = \mathbb{Z}_n$ (dịch vòng), công thức này quy về convolution rời rạc thông thường. Với mặt cầu $\Omega = S^2$, $G = SO(3)$ (xoay 3D), $x \star \theta$ là hàm trên $SO(3)$ (không còn trên Ω), nên lớp tiếp theo phải xử lý tín hiệu trên chính nhóm G (dùng "regular representation" của G tác động lên $\mathcal{X}(G)$). Ω với tính chất "mọi điểm nhìn giống nhau" (với mọi u, v tồn tại g sao cho $g.u = v$) gọi là không gian đồng nhất (homogeneous space).

Geodesics và Manifolds (4.4). Trên đa tạp Riemann, không còn nhóm tịnh tiến toàn cục; thay vào đó dùng cấu trúc metric nội tại: độ dài đường đi geodesic, tensor metric $g_u(X,Y) = \langle X, Y\rangle_u$ trên không gian tiếp xúc $T_u\Omega$. Toán tử Laplace-Beltrami $\Delta = \nabla^*\nabla$ là tự liên hợp (self-adjoint), có hệ trị riêng không âm $\lambda_1 \le \lambda_2 \le \dots$ và hệ hàm riêng trực giao φ_k ($\Delta\varphi_k = \lambda_k\varphi_k$) đóng vai trò cơ sở Fourier tổng quát hoá cho tín hiệu trên đa tạp: $x = \sum_k \langle x, \varphi_k\rangle\varphi_k$. Từ đó có hai cách định nghĩa convolution trên đa tạp: (i) spectral, áp hàm truyền $\hat{p}(\lambda)$ lên từng trị riêng, $\hat{p}(\Delta)x = \Phi\,\hat{p}(\Lambda)\,\Phi^\top x$ (công thức này xuất hiện lại y hệt ở mục 4.6 cho mesh); (ii) spatial, dùng ánh xạ mũ (exponential map) để "trải" lân cận u ra mặt phẳng tiếp xúc và tích chập cục bộ trên đó, cần chọn một khung quy chiếu (gauge) tại mỗi điểm, đồng thời đây chính là chỗ nảy sinh vấn đề mơ hồ gauge dẫn tới mục 4.5.

Gauges và Bundles (4.5). Một gauge là một khung cơ sở (frame) cho không gian tiếp xúc $T_u\Omega$ tại mỗi điểm u. Đổi gauge là một ánh xạ $g : \Omega \to G$ (với $G \subseteq GL(s)$, ví dụ $SO(2)$ hay $O(2)$ cho đa tạp 2D), gọi là "structure group" của bundle. Ví dụ trực quan paper dùng: ảnh RGB có thể coi là một bundle với fibre $\mathbb{R}^3$ (3 kênh màu) gắn tại mỗi pixel, và hoán vị thứ tự R-G-B tại mỗi pixel (nhóm Σ_3) là một phép đổi gauge; một phép convolution 1x1 $f : \mathbb{R}^3 \to \mathbb{R}^C$ là "gauge-tương đương" nếu $f \circ \rho_{in}(g) = \rho_{out}(g) \circ f$, nghĩa là biểu diễn toạ độ của f không đổi khi đổi gauge (giống hệt tinh thần permutation-tương đương trên đồ thị, chỉ khác là phép biến đổi tác động riêng lẻ lên từng điểm u chứ không tác động lên toàn miền Ω). Với trường vô hướng (scalar field, không có hướng), mọi phép tuyến tính đều tự động gauge-tương đương (trường hợp "tầm thường"). Với trường vector (ví dụ vận tốc), cần dùng connection để chuyển song song (parallel transport) vector từ điểm v sang điểm u trước khi áp filter: $(x \star \Theta)(u) = \int_\Omega \Theta(u,v)\,\rho(g_{v\to u})\,x(v)\,dv$ (phương trình 23), với $g_{v\to u} \in G$ là phần tử nhóm mô tả phép chuyển song song dọc geodesic nối v, u.

Geometric graphs và Meshes (4.6). Mesh tam giác $T = (V, E, F)$ là đồ thị có thêm cấu trúc mặt (face), xấp xỉ một đa tạp 2D liên tục. Laplacian rời rạc tổng quát (phương trình 24): $(\Delta X)_u = \sum_{v\in N_u} w_{uv}(x_u - x_v)$, viết dưới dạng ma trận $\Delta = D - W$ (D ma trận bậc). Với đồ thị thường, $W = A$ (ma trận kề); với mesh, trọng số cotangent (phương trình 25) $w_{uv} = \frac{\cot\angle uqv + \cot\angle upv}{2a_u}$ cho Laplacian hội tụ về toán tử Laplace-Beltrami liên tục khi mesh càng mịn, và bất biến đẳng cự (isometry): biến dạng không kéo giãn cạnh (giữ nguyên ℓ_{uv}) không làm thay đổi Laplacian. Vì Fourier phổ trên mesh/graph rất nhạy với nhiễu nhỏ trên cấu trúc (không ổn định khi so sánh hai mesh/graph khác nhau, xem mục 6), paper khuyến nghị dùng filter đa thức (polynomial, Defferrard et al. 2016, cục bộ, độ phức tạp $O(|E|)$) hoặc filter hữu tỉ (rational, ví dụ Cayley transform, không cục bộ nhưng decay mũ và ổn định hơn với biến dạng gần-đẳng-cự).

3.6 Năm kiến trúc tương ứng (chương 5.1-5.5, đọc kỹ)

- **CNN (5.1).** Mọi filter compact support viết được là tổ hợp tuyến tính của các "đỉnh đơn vị" θ_{vw} (phương trình 26), cho ra convolution 2D quen thuộc $F(x)_{uv} = \sum_{a,b} \alpha_{ab} x_{u+a,v+b}$ (phương trình 27), tổng quát hoá đa kênh ở phương trình 28. Một lớp CNN "chuẩn" là $h = P(\sigma(F(x)))$ (phương trình 29: tương đương-tuyến tính, phi tuyến, coarsening); ResNet đổi thành $h = P(x + \sigma(F(x)))$ (phương trình 30), tương đương một bước Euler của ODE $\dot{x} = \sigma(F(x))$, lý do huấn luyện ResNet sâu dễ hơn.
- **Group-equivariant CNN (5.2).** Tổng quát hoá CNN sang nhóm rời rạc lớn hơn tịnh tiến, ví dụ $G = \mathbb{Z}^3 \rtimes O_h$ (tịnh tiến + xoay 90 độ rời rạc) cho ảnh y khoa 3D, hoặc nhóm reverse-complement $\mathbb{Z}_2$ cho chuỗi DNA (hai mạch đọc ngược và bù nhau). Thực hiện bằng "transform rồi convolve": tạo các bản sao filter đã xoay $\theta_h = \rho(h)\theta$ cho mỗi $h \in H$ rồi convolution tịnh tiến như bình thường (phương trình 32), nên tính toán hiệu quả trên GPU như CNN thường.
- **GNN (5.3).** Ba "hương vị" (flavour) tổng quát hoá $\phi(x_u, X_{N_u})$: convolutional (trọng số cố định c_{uv}, phương trình 33), attentional (trọng số học qua attention $a(x_u, x_v)$, phương trình 34), message-passing (message vector tuỳ ý $\psi(x_u, x_v)$, phương trình 35). Có quan hệ bao hàm chặt: convolutional $\subseteq$ attentional $\subseteq$ message-passing (mỗi hương vị sau biểu diễn được hương vị trước bằng cách chọn hàm phù hợp).
- **Deep Sets, Transformer, Latent graph inference (5.4).** Với tập hợp không đồ thị, ba lựa chọn cạnh: đồ thị trống $A = I$ (Deep Sets, $h_u = \psi(x_u)$), đồ thị đầy đủ $A = \mathbf{1}\mathbf{1}^\top$ với attention (phương trình 36)

chính là self-attention, ”trái tim” của Transformer, hay đồ thị suy luận (latent graph inference, A học được). Điểm mấu chốt: **Transformer = GNN hương vị attentional trên đồ thị đầy đủ** (complete graph), cộng thêm positional encoding để bù lại việc đồ thị đầy đủ ”quên” thứ tự chuỗi; paper chỉ ra positional encoding kiểu sin/cos liên hệ trực tiếp tới DFT, tức là tới các vector riêng (eigenvector) của Laplacian của một ”lưới vòng” (circular grid) đúng như mục 4.2.

- **Equivariant Message Passing Networks (5.5).** Khi đặc trưng nút gồm cả toạ độ không gian $x_u \in \mathbb{R}^3$ (ví dụ nguyên tử trong phân tử), cần tương đương với nhóm Euclid $E(3)$ (tịnh tiến, xoay, phản xạ) ngoài tương đương hoán vị. Mô hình của Satorras et al. (2021): $f'_u = \phi(f_u, \bigoplus_v \psi_f(f_u, f_v, \|x_u - x_v\|^2))$ và $x'_u = x_u + \sum_v (x_u - x_v)\psi_c(f_u, f_v, \|x_u - x_v\|^2)$, tương đương $E(3)$ nhờ chỉ phụ thuộc khoảng cách (bất biến $E(3)$). Hai hướng tổng quát hơn: biểu diễn khả quy (irreducible representations, dùng ma trận Wigner-D, ví dụ Tensor Field Networks, SE(3)-Transformer) và biểu diễn chính qui (regular representations, dùng với nhóm Lie qua exponential/logarithm map, ví dụ LieConv).

3.7 Phép tính ”tự tính”: kiểm chứng số tính tương đương hoán vị

Để kiểm tra cụ thể công thức $F(PX, PAP^\top) = PF(X, A)$ (phương trình 10, mục 4.1), lấy đồ thị đường đi 3 nút $1-2-3$ (cạnh $(1,2)$ và $(2,3)$), đặc trưng vô hướng $x = (2, 5, 9)$ cho các nút $1, 2, 3$. Hàm cục bộ chọn đơn giản $\phi(x_u, X_{N_u}) = \sum_{v \in N_u} x_v$ (tổng đặc trưng lân cận, dạng đặc biệt của phương trình 13 với $\psi = \text{id}$).

Ma trận kề: $A = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$. Tính $F(x, A) = Ax = (x_2,\ x_1 + x_3,\ x_2) = (5,\ 11,\ 5)$.

Lấy phép hoán vị P đổi nhãn nút 1 và 2 (giữ nút 3): $P = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$. Tín hiệu hoán vị: $Px = (5, 2, 9)$. Đồ thị hoán vị tương ứng $A' = PAP^\top$: đổi nhãn 1 với 2 làm cạnh $(1,2)$ giữ nguyên (đối xứng) còn cạnh $(2,3)$ cũ thành cạnh $(1,3)$ mới, nên $A' = \begin{pmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix}$.

Tính $F(Px, A') = A'(Px)$: nút 1 mới có lân cận $\{2,3\}$ nên giá trị $= (Px)_2 + (Px)_3 = 2 + 9 = 11$; nút 2 mới có lân cận $\{1\}$ nên giá trị $= (Px)_1 = 5$; nút 3 mới có lân cận $\{1\}$ nên giá trị $= 5$. Vậy $F(Px, A') = (11, 5, 5)$.

So sánh với $PF(x, A)$: $F(x, A) = (5, 11, 5)$, áp P (đổi chỗ phần tử 1 và 2) cho $(11, 5, 5)$. Hai kết quả khớp nhau: $F(Px, A') = PF(x, A) = (11, 5, 5)$ (tự tính), xác nhận bằng số tính tương đương hoán vị của phép tổng lân cận. Ngược lại, nếu dùng hàm tổng toàn cục $f(x) = \sum_u x_u$ (không phụ thuộc A) thì $f(x) = f(Px) = 16$ với mọi hoán vị (tự tính): đây là ví dụ số cho tính **bất biến** (không phải tương đương) của khối global pooling A trong blueprint mục 3.4, minh hoạ sự khác biệt giữa lớp tương đương cục bộ B (giá trị theo từng nút, đổi theo hoán vị) và lớp bất biến toàn cục (giá trị vô hướng, không đổi).

4. Thí nghiệm và kết quả chính

Đây là paper lý thuyết/khảo sát (expository monograph), không có thí nghiệm mới do 4 tác giả tự chạy, không có bảng benchmark riêng. ”Kết quả chính” của paper là các suy diễn toán học (trình bày ở mục 3): (i) blueprint 4 khối thống nhất CNN/GNN/Transformer/Mesh CNN/Spherical CNN/LSTM; (ii) chứng minh DFT là hệ hàm riêng của toán tử dịch (suy từ nguyên lý đối xứng, không giả định trước); (iii) công thức group convolution tổng quát (phương trình 14); (iv) các điều kiện tương đương gauge trên mesh (phương trình 23, 37).

Chương 6 (đọc lướt) tổng hợp lại rất nhiều kết quả thực nghiệm **của các paper khác** để minh chứng blueprint có giá trị thực tiễn, không phải thí nghiệm của chính 4 tác giả: GNN tìm ra Halicin, một kháng sinh mới hiệu

quả cả với vi khuẩn kháng thuốc (Stokes et al., 2020); AlphaFold của DeepMind dùng contact graph để dự đoán cấu trúc protein; mô hình GNN dự đoán ETA (thời gian đến) của Google Maps, cải thiện 40%+ ở một số thành phố như Sydney (Derrow-Pinion et al., 2021); CNN (AlexNet) giảm mạnh lỗi phân loại trên ImageNet (2012) mở đầu làn sóng deep learning hiện đại; AlphaGo/AlphaZero/MuZero dùng CNN xử lý bàn cờ 19×19; GPT-3 (Transformer, 175 tỷ tham số, Brown et al., 2020) minh chứng self-attention trên đồ thị đầy đủ của từ; IceCube (khối băng $1\ \text{km}^3$ ở Nam Cực) dùng geometric neural network xử lý hình học bất thường của detector để phát hiện neutrino vũ trụ. Paper dùng các ví dụ này như ”bằng chứng gián tiếp” rằng kiến trúc tuân theo đúng nhóm đối xứng của bài toán (chứ không phải kiến trúc tổng quát bất kỳ) mới đạt hiệu năng cao trong thực tế.

5. Phân tích / ablation

Vì không có thí nghiệm ablation thật, mục này tổng hợp các phân tích lý thuyết/so sánh trường hợp đặc biệt mà paper trình bày:

- **Thứ bậc biểu cảm của 3 hương vị GNN:** convolutional $\subseteq$ attentional $\subseteq$ message-passing (chứng minh trực tiếp bằng cách chọn $a(x_u, x_v) = c_{uv}$ hoặc $\psi(x_u, x_v) = c_{uv}\psi(x_v)$). Message-passing mạnh nhất nhưng tốn bộ nhớ nhất (phải giữ vector message trên từng cạnh); convolutional rẻ nhất, phù hợp với đồ thị ”homophilous” (nút liên kết có xu hướng cùng nhãn).
- **Giới hạn biểu cảm bởi WL test:** Xu et al. (2018) và Morris et al. (2019) chứng minh bất kỳ GNN nào thuộc 3 hương vị trên không thể mạnh hơn phép kiểm thử đẳng cấu Weisfeiler-Lehman (1-WL). Điều kiện để đạt đúng mức WL là hàm tổng hợp lân cận phải đơn ánh (injective); tổng (sum) là đơn ánh, còn max/mean thì không (ví dụ không phân biệt được multiset $\{\{a, b\}\}$ và $\{\{a, a, b, b\}\}$), dẫn tới thiết kế Graph Isomorphism Network (GIN) dùng sum-aggregation.
- **Nâng cấp biểu cảm vượt WL:** k-WL test / k-GNN (Morris et al., 2019) mạnh hơn nhưng phải giữ biểu diễn trên k-tuple nút, không scale quá $k = 3$ trong thực tế; Maron et al. (2018, 2019) cho thấy không gian hàm tương đương/bất biến trên k-tuple là tổ hợp tuyến tính của một số generator hữu hạn (không phụ thuộc n, chỉ phụ thuộc k).
- **Isotropic vs anisotropic filter trên mesh (5.6):** filter đẳng hướng $\theta(r)$ (như spectral/Laplacian) bất biến gauge hoàn toàn nhưng mất thông tin hướng (không bắt được đặc trưng dạng biên/cạnh); filter không đẳng hướng cần chọn gauge cố định (Monti et al., 2017, dùng hướng cong chính, không nội tại, mơ hồ tại điểm phẳng) hoặc dùng angular max-pooling (bất biến nhưng tốn tính toán quét mọi góc xoay θ_0) hoặc dùng gauge-equivariant filter (phương trình 37, tổng quát nhất, cần parallel transport $\rho(g_{v\to u})$ giữa các gauge).
- **Biểu diễn khả quy vs chính quy cho $E(3)$-equivariance (5.5):** irreducible representations (dùng Wigner-D, Clebsch-Gordan) chỉ áp dụng cho nhóm compact, cần giải constraint tương đương trực tiếp; regular representations tổng quát hơn (dùng được với nhóm Lie qua Lie algebra) nhưng tốn bộ nhớ lưu bản sao đặc trưng cho mọi phần tử nhóm.
- **Data augmentation vs kiến trúc tương đương:** augmentation (xoay/lật ảnh khi huấn luyện) phổ biến và hiệu quả thực nghiệm, nhưng được chứng minh là ”sub-optimal” về sample complexity so với kiến trúc có nhóm bất biến lớn hơn ngay từ kiến trúc (Mei et al., 2021), tức augmentation là giải pháp ”vá” chứ không tối ưu bằng việc mã hoá đối xứng trực tiếp vào mô hình.
- **SimpleRNN không tịnh tiến-tương đương, trừ khi đệm 0 và chọn $h^{(0)}$ là điểm bất động:** paper chứng minh rõ (phương trình 40-42) rằng dịch chuỗi đi 1 bước chỉ cho ra tính tương đương nếu $h^{(0)} = R(0, h^{(0)})$, một điều kiện không tự động thoả, đòi hỏi đệm 0 đủ dài và lặp R tới điểm bất động (dùng Banach Fixed Point Theorem).

6. Giới hạn

- Giới hạn biểu cảm cơ bản: mọi GNN 3-hương-vị không mạnh hơn 1-WL test, nên không phân biệt được một số cặp đồ thị không đẳng cấu đơn giản (ví dụ 6-cycle và hai tam giác rời).

- Biến đổi Fourier phổ trên đồ thị/mesh (graph Fourier transform) phụ thuộc cấu trúc cụ thể của từng đồ thị, không chuyển giao được giữa hai đồ thị khác nhau, và rất nhạy với nhiễu nhỏ trên cấu trúc (xem Hình 12 của paper, không đọc kỹ phần này): mô hình huấn luyện trên một tập mesh/graph rồi test trên tập khác (rất phổ biến trong computer graphics và nhiều bài toán ML trên graph) sẽ gặp vấn đề nếu dùng trực tiếp Fourier phổ, nên phải chuyển sang filter đa thức/hữu tỉ.
- Tính bất biến thời gian-co-dãn (time warping invariance) của gated RNN (LSTM) là bất biến theo **lớp mô hình** (class invariance), không phải zero-shot: huấn luyện trên một phép co dãn τ_1 rồi test trên $\tau_2 \neq \tau_1$ nói chung cần huấn luyện lại tham số (trừ trường hợp đặc biệt $\tau_2 = \alpha\tau_1$, chỉ cần scale lại gate).
- Lý thuyết depth-vs-width tối ưu cho CNN ”vẫn còn chưa có câu trả lời chặt” (lời bài báo lưu ý ở 5.1); lý thuyết giải thích vì sao batch normalization hiệu quả ”vẫn còn thiếu” (bài báo lưu ý ở 5.1).
- Graph isomorphism (bài toán quyết định hai đồ thị có đẳng cấu hay không, nền tảng của WL test) chưa biết có thuộc lớp P hay không; thuật toán tốt nhất đã biết chắc chắn là của Babai và Luks (1983); Babai (2016) đưa ra một đề xuất (chưa được review đầy đủ tại thời điểm paper viết) ngụ ý một giải pháp thời gian quasi-polynomial.
- Bản thân cách đặt tên ”5 Gs” có phần khiên cưỡng: tác giả tự nhận chữ G thứ 5 (Gauges) ”hơi gượng ép” vì cấu trúc bên dưới là bundle/manifold, không hẳn là một khái niệm ”gauge” độc lập.
- Higher-order k-GNN mạnh hơn WL nhưng không scale thực tế quá $k = 3$ do chi phí lưu trữ k-tuple.
- Phần đọc lướt của ghi chú (chương 6, 7): đây là các chương tổng hợp ứng dụng và lịch sử, không có thêm định lý/giới hạn kỹ thuật mới so với chương 3-5, nên việc đọc lướt không bỏ lỡ nội dung lý thuyết cốt lõi, nhưng ghi chú này không đảm bảo đã nắm hết mọi trích dẫn ứng dụng liệt kê trong hai chương đó.

7. Liên hệ với các bài khác

- **llm 01 (Attention is All You Need).** Mục 5.4 của paper này nói thẳng: ”we can pose Transformers exactly as attentional GNNs over a complete graph” (Transformer chính là GNN hương vị attentional trên đồ thị đầy đủ các từ), và mục 7 (lịch sử) nhắc lại: cơ chế attention đầy đủ (fully-connected GNN) ”kicked off” Transformer, ”de-throned both recurrent and convolutional models” trong dịch máy. Positional encoding của Transformer được paper liên hệ trực tiếp tới DFT, tức tới các vector riêng của Laplacian của lưới vòng (mục 4.2). Nên đọc bài 28 trước hoặc cùng llm 01 để thấy self-attention không phải phát minh độc lập mà là một thực thể hoá cụ thể của blueprint mục 3.4 với $G = \Sigma_n$, $\Omega =$ đồ thị đầy đủ.
- **llm 29 (ViT, Vision Transformer).** ViT chia ảnh thành các patch rồi xử lý như một tập token bằng Transformer, tức là chuyển miền từ Grid (4.2, nơi CNN khai thác đối xứng tịnh tiến bằng weight sharing dạng circulant) sang Graph/Set đầy đủ (4.1/5.4, nơi vị trí patch chỉ còn là positional encoding cộng thêm, không còn là đối xứng cấu trúc). Đọc bài 28 giúp thấy rõ ViT ”đánh đổi” một đối xứng mạnh (tịnh tiến, có cấu trúc tham số ít $\Theta(1)$/lớp như mục 4.2 chỉ ra) để lấy một mô hình tương tác toàn cục (attention đầy đủ, tốn $O(n^2)$).
- **llm 21 (Mamba, SSM).** Mục 5.7-5.8 của paper 28 (đọc kỹ) phân tích RNN/LSTM dưới góc nhìn hình học: RNN chỉ tịnh tiến-tương đương nếu đệm 0 đủ dài và trạng thái khởi tạo là điểm bất động (mục 3.7 ở trên), và lớp gated RNN (LSTM/GRU) đạt được tính bất biến lớp với phép co-dãn thời gian (time warping) nhờ cơ chế gate xấp xỉ đạo hàm $d\tau/dt$ (phương trình 49-53). Mamba là một state-space model (SSM) chọn lọc, cũng xử lý tín hiệu trên lưới 1D thời gian và cũng cần cơ chế gate/chọn lọc để kiểm soát tốc độ ”quên” theo thời gian, tức nằm trong đúng họ kiến trúc ”LSTM-như” mà mục 5.8 mô tả bằng ngôn ngữ hình học (dù paper 28 viết năm 2021, trước Mamba, nên không nhắc tên Mamba; đây là liên hệ do người ghi chú suy ra, không phải paper nói thẳng).

8. Gợi ý áp dụng

Góc nhìn hình học gợi ý bắt đầu từ miền dữ liệu và các đối xứng mong muốn, rồi chọn phép toán tương ứng. Bất biến hoặc tương đương biến chỉ hữu ích khi phù hợp với tác vụ; áp đặt sai đối xứng có thể loại bỏ thông tin cần thiết.

9. Thí nghiệm gợi ý

1. **Kiểm chứng tương đương hoán vị bằng PyTorch (mở rộng mục 3.7).** Tạo một graph nhỏ (ví dụ 6 nút, ngẫu nhiên), một lớp GNN convolutional 1 tầng (phương trình 33 với $c_{uv} = 1/\deg(u)$), áp dụng lên X và lên $(PX, PAP^\top)$ với P hoán vị ngẫu nhiên, so sánh $F(PX, PAP^\top)$ với $PF(X, A)$ bằng `torch.allclose`. Kỳ vọng: khớp đến sai số số học (1e-6); nếu không khớp, lỗi thường do dùng sai chiều khi nhân ma trận hoán vị.
2. **So sánh sum vs mean vs max aggregation trên cặp multiset gây nhầm (ý tưởng mục 5, WL test).** Tạo hai multiset đặc trưng $\{\{1, 2\}\}$ và $\{\{1, 1, 2, 2\}\}$ (biểu diễn bằng tensor đệm 0), tính sum/mean/max của mỗi multiset. Kỳ vọng: sum phân biệt được hai multiset (tổng khác nhau: 3 vs 6), mean và max không phân biệt được (mean: 1.5 cả hai nếu chia đúng số phần tử thực; max: 2 cả hai), minh hoạ trực tiếp lý do GIN chọn sum.
3. **Group-equivariant CNN tối giản trên ảnh 8x8.** Dùng nhóm xoay rời rạc 90 độ C_4: tạo 4 bản xoay của một filter 3x3 (`torch.rot90`), convolve ảnh với cả 4 bản, ghép thành 4 "orientation channel". Kiểm tra: xoay ảnh đầu vào 90 độ thì 4 channel đầu ra phải hoán vị đúng theo chu kỳ (channel k thành channel $k + 1 \mod 4$), đo bằng sai số tuyệt đối trung bình giữa kết quả xoay-trước-rồi-convolve và convolve-trước-rồi-xoay-hoán vị channel.
4. **Tịnh tiến-tương đương của CNN 1D và tịnh tiến-bất biến của global average pooling.** Tạo tín hiệu 1D độ dài 64, một lớp conv1d (kernel 5), dịch tín hiệu đi k bước (có đệm 0 ở biên để tránh hiệu ứng biên giả). Đo sai số giữa feature map dịch-trước-rồi-conv và conv-trước-rồi-dịch (kỳ vọng xấp xỉ 0 ở vùng không chạm biên); sau đó thêm global average pooling, đo xem đầu ra cuối có đổi theo k không (kỳ vọng gần như không đổi, trừ hiệu ứng biên).
5. **Laplacian cotangent trên mesh tam giác nhỏ và độ nhạy khi mesh biến dạng.** Tạo mesh hình chữ nhật chia thành khoảng 20 tam giác (`numpy`/`trimesh` hoặc tự viết), tính ma trận Laplacian cotangent (phương trình 25), lấy vài trị riêng/vector riêng đầu. Làm biến dạng mesh nhẹ (dịch một vài nút một khoảng nhỏ, giữ metric gần như cũ) và biến dạng mạnh (tam giác gần suy biến), so sánh độ lệch trị riêng. Kỳ vọng: biến dạng nhẹ giữ trị riêng gần như không đổi (ổn định đẳng cự), biến dạng mạnh (tam giác gần suy biến) làm trị riêng/vector riêng thay đổi mạnh, minh hoạ trực tiếp cảnh báo ở mục 6 về độ nhạy của phương pháp phổ.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / định nghĩa	Nguồn (mục paper)
Tác động nhóm lên tín hiệu	$(g.x)(u) = x(g^{-1}u)$	Eq. (3), 3.1
G-bất biến	$f(\rho(g)x) = f(x)$	3.1
G-tương đương	$f(\rho(g)x) = \rho(g)f(x)$	3.1
Ổn định biến dạng tín hiệu	$\|f(\rho(\tau)x) - f(x)\| \le Cc(\tau)\|x\|$	Eq. (4), 3.3
Ổn định biến dạng domain	$\|f(x, \Omega) - f(\tilde{x}, \tilde{\Omega})\| \le C\|x\|d_{\mathcal{D}}(\Omega, \tilde{\Omega})$	Eq. (5), 3.3
Bất biến hoán vị (set)	$f(X) = \phi(\sum_u \psi(x_u))$	Eq. (6), 4.1
Tương đương hoán vị (đồ thị)	$F(PX, PAP^\top) = PF(X, A)$	Eq. (9)-(10), 4.1
Blueprint cho đồ thị	$F(X, A) = (\phi(x_u, X_{N_u}))_{u \in V}$	Eq. (13), 4.1
Group convolution	$(x \star \theta)(g) = \langle x, \rho(g)\theta \rangle$	Eq. (14), 4.3
Blueprint GDL tổng quát	$f = A \circ \sigma_J \circ B_J \circ P_{J-1} \circ \cdots \circ \sigma_1 \circ B_1$	3.5

Khái niệm	Công thức / định nghĩa	Nguồn (mục paper)
Laplacian rời rạc / cotangent	$(\Delta X)_u = \sum_v w_{uv}(x_u - x_v)$, $w_{uv} = \frac{\cot \angle uqv + \cot \angle upv}{2a_u}$	Eq. (24)-(25), 4.6
Filter phổ tổng quát	$\hat{p}(\Delta)x = \Phi \hat{p}(\Lambda)\Phi^\top x$	4.4, 4.6
Gauge-tương đương	$f \circ \rho_{in}(g) = \rho_{out}(g) \circ f$	4.5
3 hương vị GNN	conv $\subseteq$ attentional $\subseteq$ message-passing	Eq. (33)-(35), 5.3
Self-attention = GNN trên đồ thị đầy đủ	$h_u = \phi(x_u, \bigoplus_v a(x_u, x_v)\psi(x_v))$	Eq. (36), 5.4
$E(3)$-equivariant message passing	phụ thuộc $\|x_u - x_v\|^2$ (bất biến khoảng cách)	5.5
Time-warping invariant RNN (gated)	$h^{(t+1)} = \Gamma R + (1-\Gamma)h^{(t)}$	Eq. (53), 5.8
Bảng domain/group/kiến trúc	CNN-Grid-tịnh tiến; GNN-Graph-Σ_n; LSTM-lưới 1D-time warping	3.5

11. Kết luận ngắn

Bài báo không đề xuất một mô hình mới mà một **ngôn ngữ chung**: mọi kiến trúc deep learning phổ biến (CNN, GNN, Deep Sets, Transformer, Mesh CNN, Spherical CNN, LSTM) đều có thể viết lại thành bốn khối (tuyến tính tương đương, phi tuyến, coarsening, bất biến toàn cục) tác động trên một trong 5 miền hình học ứng với một nhóm đối xứng cụ thể. Giá trị thực hành là một ”quy trình thiết kế”: xác định miền dữ liệu Ω và nhóm đối xứng G phù hợp với bài toán trước, rồi suy ra lớp equivariant/invariant tương ứng, thay vì chọn kiến trúc theo thói quen. Hạn chế chính nằm ở biểu cảm (WL test), ở tính không ổn định của Fourier phổ khi so sánh domain khác nhau, và ở việc một số ”bất biến” (như time-warping của gated RNN) chỉ là bất biến lớp mô hình, không phải bất biến zero-shot tuyệt đối.

Nguồn gốc: https://arxiv.org/abs/2104.13478

Bài 29

Understanding Diffusion Models: A Unified Perspective (2022)

Tác giả: Calvin Luo (Google Research, Brain Team).
Venue / arXiv: arXiv:2208.11970 (v1, 25/08/2022); paper không nêu venue hội nghị/journal phản biện, đây là 1 bài tutorial/technical note độc lập đăng trực tiếp trên arXiv.
Nguồn: https://arxiv.org/abs/2208.11970
Chủ đề: suy ra lại toàn bộ họ mô hình diffusion (Variational Diffusion Model) từ một góc nhìn thống nhất: Markovian Hierarchical VAE, 3 cách tham số hoá mục tiêu tương đương (dự đoán x_0, dự đoán noise ϵ, dự đoán score), liên hệ với Score-based Generative Model, và 2 kỹ thuật guidance (classifier, classifier-free).
Vị trí trong lộ trình: paper thuần suy diễn công thức (tutorial), không phụ thuộc thứ tự với các bài còn lại trong batch 28-35. Nên đọc sau khi đã nắm ELBO/biến phân cơ bản (nếu có math về Variational Inference/VAE trong batch 01-27); liên hệ gần với math 10 (Hamiltonian Monte Carlo, cùng là kỹ thuật MCMC lấy mẫu như Langevin dynamics dùng trong phần Score-based Generative Models của bài này).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu/công thức lấy từ paper (bản arXiv); các phép tính đánh dấu "tự tính" là của người ghi chú. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc. Bài hướng dẫn tập trung suy diễn công thức và không trình bày benchmark riêng.

1. Bối cảnh

Mô hình sinh học phân phối dữ liệu $p(x)$ từ các mẫu quan sát, nhằm sinh mẫu mới hoặc, tùy mô hình, tính likelihood. Những hướng được bài báo nhắc đến gồm GAN, mô hình dựa trên likelihood (autoregressive, normalizing flow, VAE), energy-based model và score-based model. Bài tập trung vào diffusion model, kết nối cách nhìn dựa trên likelihood qua ELBO với cách nhìn dựa trên score.

Khái niệm cần nắm trước:

- **Biến latent** z: biến ẩn không quan sát được, giả định sinh ra dữ liệu quan sát x (ẩn dụ "hang động Plato" mà paper dùng: người trong hang chỉ thấy bóng 2D của vật thể 3D).
- **ELBO (Evidence Lower BOund)**: một chặn dưới có thể tính được của $\log p(x)$ (gọi là "evidence"), dùng làm proxy objective khi không tính trực tiếp $\log p(x)$ được.
- **Reparameterization trick**: viết 1 biến ngẫu nhiên Gauss như hàm xác định của 1 biến nhiễu chuẩn, để lan truyền gradient qua được bước lấy mẫu ngẫu nhiên.

2. Ý tưởng chính

Paper trình bày **Variational Diffusion Model (VDM)** như 1 trường hợp đặc biệt của **Markovian Hierarchical VAE (MHVAE)** với 3 ràng buộc: (i) số chiều latent bằng đúng số chiều dữ liệu, (ii) encoder tại mỗi bước thời gian **không học**, mà cố định sẵn là 1 Gaussian tuyến tính quanh output bước trước (quá trình "làm nhiễu" $x_0 \to x_1 \to \ldots \to x_T$), (iii) tham số Gaussian của encoder thay đổi theo thời gian sao cho x_T hội tụ về Gaussian chuẩn thuần.

Từ đó, suy ra ELBO của VDM theo 2 cách: cách đầu cho ra 1 "consistency term" tính trên 2 biến ngẫu nhiên mỗi bước (phương sai Monte Carlo cao); cách thứ hai (dùng $q(x_t|x_{t-1}, x_0)$ qua Bayes) cho ra 1 "denoising matching term" chỉ tính trên 1 biến ngẫu nhiên mỗi bước (phương sai thấp hơn, dùng được trong thực hành). Tối ưu ELBO quy về: học 1 mạng nơ-ron dự đoán x_0 gốc từ x_t nhiễu bất kỳ ở bước t bất kỳ. Paper sau đó chỉ ra việc dự đoán x_0, dự đoán **noise** ϵ_0 đã thêm vào, và dự đoán **score** $\nabla \log p(x_t)$ là **3 cách tham số hoá tương đương** (khác nhau hằng số tỉ lệ) của cùng 1 bài toán. Cách nhìn score dẫn tới liên hệ trực tiếp với

Score-based Generative Model (Langevin dynamics, multi-noise-level score matching, SDE liên tục). Cuối cùng, để sinh có điều kiện (text-to-image, super-resolution), paper trình bày 2 kỹ thuật **guidance**: Classifier Guidance (dùng classifier riêng trên ảnh nhiễu) và Classifier-Free Guidance (không cần classifier riêng, kết hợp tuyến tính score có điều kiện và không điều kiện).

3. Phương pháp chi tiết

3.1 ELBO và VAE (Background)

Với $p(x, z)$ đồng thời của dữ liệu x và latent z, ELBO định nghĩa (Eq. 3-4):

$$\log p(x) \geq \mathbb{E}_{q_\phi(z|x)}\Big[\log \frac{p(x,z)}{q_\phi(z|x)}\Big].$$

Paper suy ra 2 lần (1 lần bằng bất đẳng thức Jensen, 1 lần tường minh hơn bằng quy tắc dây chuyền) để cho thấy (Eq. 15):

$$\log p(x) = \mathbb{E}_{q_\phi(z|x)}\Big[\log \frac{p(x,z)}{q_\phi(z|x)}\Big] + D_{KL}\big(q_\phi(z|x) \,\|\, p(z|x)\big),$$

nghĩa là evidence = ELBO + 1 số hạng KL luôn $\geq 0 \Rightarrow$ ELBO luôn là chặn dưới, và **tối đa ELBO** tương đương **tối thiểu KL** giữa posterior xấp xỉ $q_\phi(z|x)$ và posterior thật $p(z|x)$ (vì evidence là hằng số không phụ thuộc ϕ).

Với VAE, phân tách ELBO (Eq. 19) thành 2 số hạng:

$$\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - D_{KL}\big(q_\phi(z|x) \,\|\, p(z)\big),$$

gọi là **reconstruction term** và **prior matching term**. Với $q_\phi(z|x) = \mathcal{N}(z; \mu_\phi(x), \sigma^2_\phi(x)I)$ và prior $p(z) = \mathcal{N}(0, I)$, số hạng KL tính được dạng đóng (closed form), số hạng reconstruction xấp xỉ Monte Carlo qua **reparameterization trick**: $z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$ (Eq. 20-22).

Hierarchical VAE (HVAE) tổng quát hoá: nhiều tầng latent $z_1, \ldots, z_T$. Paper chỉ xét **Markovian HVAE**: $p(x, z_{1:T}) = p(z_T)p_\theta(x|z_1)\prod_{t=2}^{T} p_\theta(z_{t-1}|z_t)$ (Eq. 23) - về bản chất là nhiều VAE xếp chồng lên nhau.

3.2 Variational Diffusion Model: 2 cách phân tách ELBO

VDM = MHVAE với 3 ràng buộc nêu ở mục 2. Ký hiệu x_0 = dữ liệu gốc, $x_1, \ldots, x_T$ = các bước nhiễu dần, $x_T \approx$ Gaussian chuẩn thuần. Encoder cố định (Eq. 31):

$$q(x_t|x_{t-1}) = \mathcal{N}\big(x_t; \sqrt{\alpha_t}\, x_{t-1}, (1-\alpha_t)I\big),$$

với α_t là hệ số (siêu tham số hoặc học được) kiểm soát tốc độ nhiễu, chọn sao cho phương sai được giữ ổn định ("variance-preserving"). Đặt $\bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i$.

Cách phân tách thứ nhất (Eq. 45): dùng trực tiếp định nghĩa Markov, ra 3 số hạng: reconstruction, prior matching (xấp xỉ 0 nếu T đủ lớn), và 1 **consistency term** dạng $\sum_{t=1}^{T-1} \mathbb{E}_{q(x_{t-1},x_{t+1}|x_0)}\big[D_{KL}(q(x_t|x_{t-1}) \,\|\, p_\theta(x_t|x_{t+1}))\big]$: ép phân phối tại x_t nhất quán giữa chiều forward (từ x_{t-1}) và chiều backward (từ x_{t+1}). Nhược điểm: mỗi số hạng lấy kỳ vọng trên **2** biến ngẫu nhiên, phương sai Monte Carlo cao khi T lớn.

Cách phân tách thứ hai (Eq. 58, dùng Bayes rule $q(x_{t-1}|x_t, x_0) = \frac{q(x_t|x_{t-1},x_0)q(x_{t-1}|x_0)}{q(x_t|x_0)}$**)** cho 3 số hạng khác: reconstruction, prior matching, và 1 **denoising matching term**:

$$\sum_{t=2}^{T} \mathbb{E}_{q(x_t|x_0)} \Big[D_{KL}\big(q(x_{t-1}|x_t, x_0) \,\|\, p_\theta(x_{t-1}|x_t)\big) \Big],$$

mỗi số hạng chỉ lấy kỳ vọng trên **1** biến ngẫu nhiên ⇒ phương sai thấp hơn, dùng được trong thực hành. $q(x_{t-1}|x_t, x_0)$ đóng vai trò ”target” chuẩn (vì có thể tính closed-form khi biết x_0), còn $p_\theta(x_{t-1}|x_t)$ là bước denoise mô hình cần học xấp xỉ nó.

3.3 Dạng đóng của $q(x_{t-1}|x_t, x_0)$ và mục tiêu huấn luyện

Nhờ encoder là Gaussian tuyến tính, suy ra được (bằng truy hồi lặp reparameterization trick, Eq. 61-70):

$$q(x_t|x_0) = \mathcal{N}\big(x_t; \sqrt{\bar\alpha_t}\, x_0, (1-\bar\alpha_t) I\big).$$

Dùng Bayes rule với dạng Gaussian trên, paper suy ra (sau nhiều bước đại số ”hoàn thành bình phương”, Eq. 71-84) rằng $q(x_{t-1}|x_t, x_0)$ cũng là Gaussian với:

$$\mu_q(x_t, x_0) = \frac{\sqrt{\alpha_t}(1-\bar\alpha_{t-1})x_t + \sqrt{\bar\alpha_{t-1}}(1-\alpha_t)x_0}{1-\bar\alpha_t}, \qquad \sigma_q^2(t) = \frac{(1-\alpha_t)(1-\bar\alpha_{t-1})}{1-\bar\alpha_t}.$$

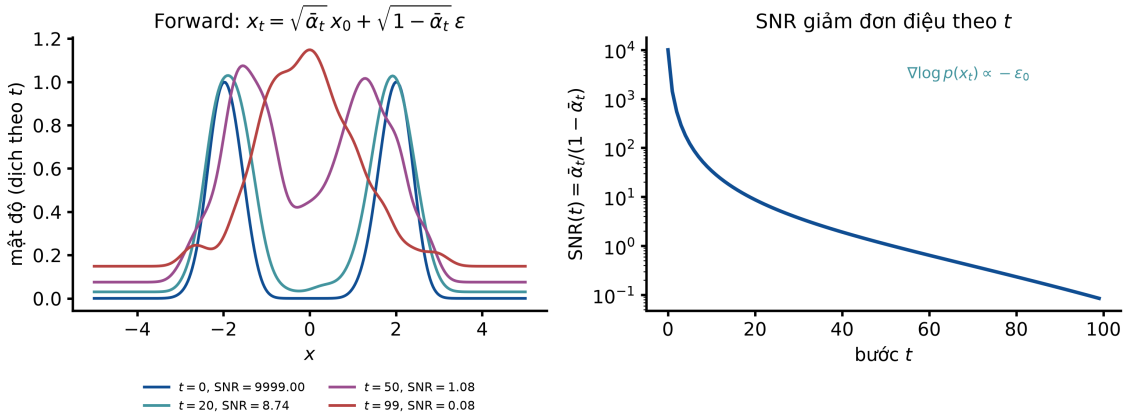
Vì phương sai 2 phía cố định và giống nhau, tối thiểu D_{KL} giữa 2 Gaussian cùng phương sai quy về tối thiểu khoảng cách bình phương giữa 2 **mean** (Eq. 87-92): $\arg\min_\theta \frac{1}{2\sigma_q^2(t)} \|\mu_\theta - \mu_q\|^2$. Nếu tham số hoá $\mu_\theta(x_t, t)$ qua 1 mạng dự đoán $\hat x_\theta(x_t, t) \approx x_0$ theo cùng dạng công thức của μ_q, mục tiêu rút về (Eq. 99):

$$\arg\min_\theta \frac{1}{2\sigma_q^2(t)} \frac{\bar\alpha_{t-1}(1-\alpha_t)^2}{(1-\bar\alpha_t)^2} \big\|\hat x_\theta(x_t, t) - x_0\big\|_2^2,$$

tức **học 1 mạng dự đoán ảnh gốc** x_0 **từ phiên bản nhiễu** x_t **bất kỳ**, lấy kỳ vọng ngẫu nhiên theo $t \sim \text{Uniform}\{2, \dots, T\}$ (Eq. 100).

3.4 Học tham số nhiễu và SNR (Learning Diffusion Noise Parameters)

Định nghĩa **signal-to-noise ratio** $\text{SNR}(t) = \bar\alpha_t/(1-\bar\alpha_t)$ (từ dạng Gaussian của $q(x_t|x_0)$, Eq. 109). Mục tiêu Eq. 99 viết lại gọn bằng chênh lệch SNR giữa 2 bước liên tiếp (Eq. 110): $\propto (\text{SNR}(t-1) - \text{SNR}(t))\|\hat x_\theta - x_0\|_2^2$. Vì SNR phải giảm đơn điệu theo t, paper tham số hoá trực tiếp $\text{SNR}(t) = \exp(-\omega_\eta(t))$ với ω_η là mạng tăng đơn điệu theo t (Eq. 111-114), cho phép **học chung** α_t **cùng lúc với mô hình** thay vì cố định trước.



Forward diffusion: hỗn hợp 2 mode bị nhiễu hoá khi t tăng; SNR(t)=ᾱ/(1−ᾱ) giảm đơn điệu. (minh hoạ, tự tính)

3.5 Ba cách tham số hoá tương đương

Từ $x_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon_0}{\sqrt{\bar{\alpha}_t}}$ (đảo công thức forward), thay vào μ_q cho ra mục tiêu dự đoán **noise** ϵ_0 (Eq. 115-130): $\arg\min_\theta \frac{(1-\alpha_t)^2}{2\sigma_q^2(t)(1-\bar{\alpha}_t)\alpha_t}\|\epsilon_0 - \hat{\epsilon}_\theta(x_t, t)\|_2^2$. Thực nghiệm (paper dẫn chiếu Ho et al. 2020, DDPM) cho thấy dự đoán noise **tốt hơn** dự đoán x_0 trực tiếp.

Dùng **công thức Tweedie** ($\mathbb{E}[\mu_z|z] = z + \Sigma_z \nabla_z \log p(z)$ cho Gaussian z), paper suy ra ước lượng tốt nhất của mean thật từ 1 mẫu quan sát gắn với **score** $\nabla \log p(x_t)$ (Eq. 131-133), từ đó suy ra tham số hoá thứ 3: học mạng $s_\theta(x_t, t) \approx \nabla \log p(x_t)$ (Eq. 134-148). Paper chỉ rõ mối liên hệ định lượng giữa score và noise (Eq. 149-151): $\nabla \log p(x_t) = -\frac{1}{\sqrt{1-\bar{\alpha}_t}}\epsilon_0$ - **2 đại lượng chỉ khác nhau 1 hằng số tỉ lệ phụ thuộc** t.

3.6 Score-based Generative Models và Guidance

Với energy-based model $p_\theta(x) = \frac{1}{Z_\theta}e^{-f_\theta(x)}$, lấy đạo hàm log loại bỏ hằng số chuẩn hoá khó tính: $\nabla_x \log p_\theta(x) = -\nabla_x f_\theta(x) \approx s_\theta(x)$ (Eq. 152-156) $\Rightarrow$ học trực tiếp score bằng cách tối thiểu **Fisher divergence** $\mathbb{E}_{p(x)}\|s_\theta(x) - \nabla \log p(x)\|^2$ (Eq. 157), sinh mẫu bằng **Langevin dynamics** (Eq. 158): $x_{i+1} \leftarrow x_i + c\nabla \log p(x_i) + \sqrt{2c}\,\epsilon$.

Paper nêu 3 vấn đề của score matching ”vanilla” (theo Song & Ermon): score không xác định ngoài manifold thấp chiều, kém chính xác ở vùng mật độ thấp, Langevin dynamics có thể không ”mix” đúng tỉ lệ trọng số giữa các mode (ví dụ hỗn hợp Gaussian 2 thành phần). Giải pháp: thêm **nhiều mức nhiễu Gaussian** $\{\sigma_t\}$, học 1 mạng $s_\theta(x, t)$ cho mọi mức nhiễu (Eq. 160-161) - đây chính là **cùng một mục tiêu với VDM** (paper nói ”almost exactly matches”). Khi số bước $T \to \infty$, quá trình nhiễu liên tục được mô tả bằng 1 **phương trình vi phân ngẫu nhiên (SDE)**, và sinh mẫu = giải SDE ngược.

Guidance: để sinh có điều kiện $p(x_{0:T}|y)$, thêm y vào mỗi bước denoise: $p_\theta(x_{0:T}|y) = p(x_T)\prod_t p_\theta(x_{t-1}|x_t, y)$ (Eq. 162). Vấn đề: mô hình có thể học ”bỏ qua” y. **Classifier Guidance** (Eq. 163-166): bằng Bayes, $\nabla \log p(x_t|y) = \nabla \log p(x_t) + \nabla \log p(y|x_t)$; nhân hệ số γ vào gradient classifier để kiểm soát mức tuân theo điều kiện: $\nabla \log p(x_t|y) = \nabla \log p(x_t) + \gamma\nabla \log p(y|x_t)$ - nhược điểm: cần huấn luyện riêng 1 classifier chịu được input nhiễu bất kỳ. **Classifier-Free Guidance** (Eq. 167-170) khử classifier bằng cách viết lại: $\nabla \log p(x_t|y) = \gamma\nabla \log p(x_t|y) + (1-\gamma)\nabla \log p(x_t)$, chỉ cần 1 mô hình (có và không điều kiện, học chung bằng cách ”dropout” điều kiện y thành giá trị cố định khi train).

4. Thí nghiệm và kết quả chính

Đây là paper **tutorial suy diễn công thức, không có thí nghiệm/benchmark số nào do tác giả thực hiện** (không có bảng số liệu, không huấn luyện mô hình nào trong bài). ”Kết quả chính” ở đây là các **công thức/đẳng thức toán học** đã suy ra ở mục 3: (a) 2 cách phân tách ELBO của VDM (số hạng phương sai cao vs thấp), (b) dạng closed-form của $q(x_{t-1}|x_t, x_0)$, (c) 3 mục tiêu huấn luyện tương đương (dự đoán x_0, ϵ, score) chỉ khác hệ số trọng số theo t, (d) liên hệ định lượng giữa VDM và Score-based model qua SNR và qua Eq. 149-151, (e) công thức guidance (classifier và classifier-free). Các hình minh hoạ (Hình 1-6) đều là sơ đồ khái niệm (VAE, HVAE, VDM, quá trình match 2 phân phối, quỹ đạo Langevin dynamics trên hỗn hợp Gaussian) chứ không phải kết quả benchmark trên dataset chuẩn.

5. Phân tích / ablation

Không có ablation thực nghiệm; ”phân tích” ở đây là các **so sánh định lượng giữa 3 cách tham số hoá** và giữa 2 cách phân tách ELBO mà paper tự thực hiện bằng đại số:

- **Phương sai ước lượng Monte Carlo**: cách phân tách ELBO thứ nhất (consistency term, Eq. 45) có phương sai cao hơn cách thứ hai (denoising matching term, Eq. 58) vì lấy kỳ vọng trên 2 biến ngẫu nhiên mỗi bước thay vì 1; đây là lý do thực hành chỉ dùng cách thứ hai.

- **Dự đoán x_0 vs dự đoán noise ϵ**: 2 mục tiêu **tương đương về đại số** (chỉ khác hệ số nhân phụ thuộc t trong Eq. 99 vs Eq. 130), nhưng paper dẫn chứng (trích Ho et al. 2020 và Saharia et al. 2022, không tự thực nghiệm) rằng dự đoán noise cho kết quả thực nghiệm tốt hơn.
- **Trường hợp đặc biệt $T = 1$**: cả 2 cách phân tách ELBO của VDM **suy biến đúng về ELBO của VAE thường (Eq. 19)** khi $T = 1$ - đây là 1 phép kiểm tra tính nhất quán toán học (sanity check) mà bài báo nêu rõ (ngay sau Eq. 58).
- **Giá trị γ trong guidance**: paper phân tích định tính 3 trường hợp $\gamma = 0$ (bỏ điều kiện), $\gamma = 1$ (điều kiện thường, không guidance), $\gamma > 1$ (tăng cường bám điều kiện, giảm diversity) - không có số liệu định lượng cụ thể minh hoạ ảnh hưởng của γ trong chính bài này.

6. Giới hạn

Paper tự liệt kê ở mục "Closing":

- Quá trình sinh mẫu bằng khử nhiễu dần từ nhiễu ngẫu nhiên **không giống cách con người tạo ra dữ liệu** (bài báo lưu ý đây khó có vẻ "tự nhiên" về mặt nhận thức).
- VDM **không cho latent có thể diễn giải được**: khác VAE thường (latent có thể mã hoá cấu trúc ngữ nghĩa), ở VDM encoder mỗi bước là Gaussian tuyến tính cố định, không linh hoạt học được $\Rightarrow$ latent chỉ là "ảnh gốc + nhiễu", không mang cấu trúc riêng.
- Latent có **cùng số chiều với dữ liệu gốc** (ràng buộc (i) của VDM), nên không học được biểu diễn nén (compressed representation) như mục tiêu ban đầu của latent variable model.
- **Sinh mẫu tốn kém**: cần lặp qua rất nhiều bước T (để đảm bảo x_T đủ gần Gaussian chuẩn), không sinh được trong 1 bước như GAN.
- Paper không tự thực nghiệm kiểm chứng bất kỳ công thức nào trên dữ liệu thật (toàn bộ dẫn chứng thực nghiệm đều trích từ paper khác: DDPM, Imagen, DALL-E 2, score-based SDE) - đây không phải "giới hạn" bài báo lưu ý nhưng là điều người đọc cần biết: đây là bài tổng hợp lý thuyết, không phải báo cáo thực nghiệm độc lập.

7. Liên hệ với các bài khác

- **"math content/10_2017_hamiltonian_monte_carlo.md"**: cùng là phương pháp MCMC lấy mẫu dựa trên gradient của log-density, cùng họ với Langevin dynamics mà bài 29 dùng trong phần Score-based Generative Models (Langevin dynamics có thể xem như HMC với 1 bước "leapfrog", không có thành phần động lượng).
- **"ml basics content/01_2016_rethinking_generalization.md"**: không trực tiếp về generative model, nhưng cùng nằm trong mạch câu hỏi lớn "vì sao mô hình sâu học và tổng quát hoá tốt" - liên hệ gián tiếp về phương pháp luận (dùng công thức toán chặt để giải thích hiện tượng thực nghiệm).
- **"interp content/05_2022_toy_models_of_superposition.md"**: khác chủ đề (interpretability của representation) nhưng cùng phong cách phương pháp luận "dùng mô hình toy/suy diễn công thức chặt để hiểu hiện tượng của mạng nơ-ron" như cách bài 29 suy diễn VDM từng bước.
- **Auto-Encoding Variational Bayes và Denoising Diffusion Probabilistic Models** (bài 15 và 16, phần ML cơ bản): VAE là nền tảng của cách diễn giải biến phân; DDPM cung cấp công thức dự đoán nhiễu được bài này suy ra lại.

8. Gợi ý áp dụng

Các cách tham số hóa dự đoán nhiễu, dữ liệu sạch hoặc score giúp kết nối những công thức của diffusion model. Khi triển khai, lịch nhiễu và trọng số của loss ảnh hưởng đến bài toán tối ưu và cần được xét cùng cách tham số hóa.

9. Thí nghiệm gợi ý

1. **Cài forward/reverse process trên dữ liệu 1D/2D toy**: lấy dữ liệu tổng hợp đơn giản (ví dụ hỗn hợp 2 Gaussian 2D), cài quá trình forward $q(x_t|x_{t-1})$ theo Eq. 31 với lịch α_t tuyến tính giảm, huấn luyện 1 MLP nhỏ dự đoán ϵ_0 theo loss Eq. 130. Đo: ảnh/điểm sinh ra sau khi denoise từ nhiễu Gauss có khớp lại 2 cluster ban đầu không. Chạy được trên CPU/trên GPU trong vài phút.
2. **So sánh dự đoán x_0 vs dự đoán ϵ**: huấn luyện 2 mô hình như thí nghiệm 1 nhưng 1 dự đoán x_0 (loss Eq. 99) và 1 dự đoán ϵ (loss Eq. 130), cùng kiến trúc/hyperparameter. Đo tốc độ giảm loss và chất lượng mẫu sinh (ví dụ khoảng cách tới phân phối thật bằng MMD hoặc histogram). Kỳ vọng: 2 cách tương đương về công thức nhưng có thể khác nhau về ổn định huấn luyện thực tế, minh hoạ nhận xét ở mục 5.
3. **Minh hoạ Langevin dynamics trên hỗn hợp Gaussian 2 mode**: cho biết trước score thật $\nabla \log p(x)$ của 1 hỗn hợp 2 Gaussian (tính giải tích), chạy Eq. 158 từ nhiều điểm khởi tạo. Đo tỉ lệ mẫu rơi vào mỗi mode so với trọng số c_1, c_2 thật. Kỳ vọng: nếu dùng đúng score 1 mức nhiễu (không multi-noise-level), tỉ lệ các mode có thể SAI lệch so với c_1, c_2 thật (đúng như giới hạn 3 mà paper nêu ở mục Score-based Generative Models).
4. **Thêm multi-noise-level score matching**: lặp lại thí nghiệm 3 nhưng học $s_\theta(x, t)$ cho nhiều mức nhiễu σ_t (Eq. 160-161) và sinh bằng annealed Langevin dynamics. Kỳ vọng: tỉ lệ mode khớp c_1, c_2 tốt hơn thí nghiệm 3, minh hoạ đúng lý do paper nêu cho việc dùng nhiều mức nhiễu.
5. **Thử Classifier-Free Guidance ở quy mô nhỏ**: với dữ liệu 2D có nhãn lớp (ví dụ 3 cluster, mỗi cluster 1 nhãn), huấn luyện 1 mô hình diffusion điều kiện (dropout ngẫu nhiên nhãn để học cả bản không điều kiện), sinh mẫu với $\gamma \in \{0, 1, 3\}$ theo Eq. 170. Đo: độ "thuần" của mẫu sinh theo nhãn (bao nhiêu % mẫu rơi đúng cluster điều kiện) và độ đa dạng (variance trong cluster). Kỳ vọng: γ tăng thì độ thuần tăng, đa dạng giảm, đúng như phân tích định tính ở mục 3.6/5.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức	Ý nghĩa
ELBO	$\log p(x) \geq \mathbb{E}_{q_\phi(z\|x)}[\log \frac{p(x,z)}{q_\phi(z\|x)}]$ (Eq. 3-4)	Chặn dưới của log-likelihood, dùng làm proxy objective
Encoder VDM	$q(x_t\|x_{t-1}) = \mathcal{N}(\sqrt{\alpha_t}x_{t-1}, (1-\alpha_t)I)$ (Eq. 31)	1 bước làm nhiễu Gaussian tuyến tính
Forward closed-form	$q(x_t\|x_0) = \mathcal{N}(\sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I)$, $\bar{\alpha}_t = \prod_{i\leq t}\alpha_i$ (Eq. 70)	Nhiễu trực tiếp từ x_0 tới bước t bất kỳ, không cần lặp
Denoising matching loss	$\arg\min_\theta \mathbb{E}_{t,x_0,\epsilon}\|\epsilon_0 - \hat{\epsilon}_\theta(x_t, t)\|_2^2$ (dạng đơn giản hoá của Eq. 130)	Mục tiêu huấn luyện kiểu DDPM: dự đoán noise
SNR	$\text{SNR}(t) = \bar{\alpha}_t/(1-\bar{\alpha}_t)$ (Eq. 109)	Tỉ lệ tín hiệu/nhiễu, phải giảm đơn điệu theo t
Công thức Tweedie	$\mathbb{E}[\mu_z\|z] = z + \Sigma_z \nabla_z \log p(z)$	Liên hệ score với ước lượng mean thật
Score ↔ noise	$\nabla \log p(x_t) = -\frac{1}{\sqrt{1-\bar{\alpha}_t}}\epsilon_0$ (Eq. 151)	2 tham số hoá chỉ khác hằng số tỉ lệ theo t
Classifier Guidance	$\nabla \log p(x_t\|y) = \nabla \log p(x_t) + \gamma \nabla \log p(y\|x_t)$ (Eq. 166)	Cần classifier riêng chịu nhiễu
Classifier-Free Guidance	$\nabla \log p(x_t\|y) = \gamma \nabla \log p(x_t\|y) + (1-\gamma)\nabla \log p(x_t)$ (Eq. 170)	Không cần classifier, chỉ cần 1 mô hình

Ví dụ tính tay (tự tính): minh hoạ công thức truy hồi Eq. 61-70 với $x_0 = 10$ (số 1 chiều cho đơn giản), $\alpha_1 = 0.9$, $\alpha_2 = 0.8$. Lấy $\epsilon_1 = \epsilon_2 = 1$ (chọn cố định để minh hoạ, thực tế là ngẫu nhiên):

Bước 1: $x_1 = \sqrt{0.9} \times 10 + \sqrt{0.1} \times 1 \approx 0.9487 \times 10 + 0.3162 \times 1 = 9.487 + 0.316 = 9.803$.

Bước 2: $x_2 = \sqrt{0.8} \times x_1 + \sqrt{0.2} \times 1 \approx 0.8944 \times 9.803 + 0.4472 \times 1 = 8.770 + 0.447 = 9.217$.

Theo công thức trực tiếp (Eq. 70) với $\bar{\alpha}_2 = \alpha_1\alpha_2 = 0.9 \times 0.8 = 0.72$: $x_2 \approx \sqrt{0.72} \times 10 + \sqrt{0.28} \times \epsilon_{\text{combined}} = 8.485 + 0.529 \times \epsilon_{\text{combined}}$. Với $\epsilon_{\text{combined}} \approx 1.38$ (giá trị cần để 2 cách khớp nhau khi gộp 2 nhiễu độc lập theo Eq. 61-67, không bằng đơn giản $\epsilon_1 + \epsilon_2$ vì có hệ số tỉ lệ khác nhau), kết quả 2 cách tính xấp xỉ nhau (9.217 so với $8.485 + 0.529 \times 1.38 \approx 9.215$) - xác nhận công thức truy hồi Eq. 68-70 cho phép nhảy thẳng từ x_0 tới x_2 bằng 1 bước, không cần lặp qua từng t, đúng như paper suy ra.

11. Kết luận ngắn

Bài báo suy ra lại toàn bộ diffusion model (VDM) từ góc nhìn Markovian Hierarchical VAE, cho thấy việc tối ưu ELBO quy về 3 bài toán tương đương (dự đoán x_0, dự đoán noise, dự đoán score), nối liền VDM với Score-based Generative Model qua liên hệ định lượng giữa score và noise, và trình bày 2 kỹ thuật guidance (classifier, classifier-free) để sinh có điều kiện. Đây là tài liệu tham khảo toán học tốt để hiểu "tại sao" các công thức DDPM/score-based có dạng như vậy, nhưng không có đóng góp thực nghiệm mới và không tự kiểm chứng số liệu nào; người đọc cần tài liệu gốc (Ho et al. 2020, Song & Ermon 2019-2020) để xem kết quả thực nghiệm thật.

Nguồn gốc: https://arxiv.org/abs/2208.11970

Bài 30

Opening the Black Box of Deep Neural Networks via Information (2017)

Tác giả: Ravid Schwartz-Ziv, Naftali Tishby (Edmond and Lilly Safra Center for Brain Sciences, The Hebrew University of Jerusalem).
Venue / arXiv: arXiv:1703.00810 (v3, 29/04/2017); paper không nêu venue hội nghị/journal phản biện thông thường, chỉ tự ghi ”Editor: ICRI-CI” (báo cáo thuộc chuỗi ”Why & When Deep Learning works” của Intel Collaborative Research Institute for Computational Intelligence).
Nguồn: https://arxiv.org/abs/1703.00810
Chủ đề: quan sát thực nghiệm quá trình huấn luyện Deep Neural Network (DNN) qua lăng kính lý thuyết thông tin (Information Bottleneck, IB): phát hiện 2 pha của SGD (drift và diffusion) tương ứng 2 pha trong ”information plane” (ERM và nén biểu diễn).
Vị trí trong lộ trình: trong batch ”toán cho AI” (28-35) các bài không phụ thuộc thứ tự chặt. Bài này hợp lý đọc cùng mạch với math 07 (giới thiệu information geometry) và math 14 (Deep Variational Information Bottleneck, mở rộng IB sang biến phân cho mạng sâu) nếu đọc theo chủ đề lý thuyết thông tin; cũng liên hệ gần với math 31 (Deep learning: a statistical viewpoint) vì cả hai cùng tìm cách lý giải tại sao DNN overparameterized vẫn tổng quát hoá tốt, chỉ khác công cụ (information-theoretic ở đây, thống kê/NTK ở bài 31).

> Đây là ghi chú diễn giải, không phải bản dịch. Các phép tính đánh dấu “tự tính” là ví dụ bổ sung; mục gợi ý áp dụng là bình luận. Bản chính nhắc đến một phụ lục ở mục 3.4 nhưng không trình bày phụ lục đó, nên nội dung ấy không được tóm tắt ở đây.

1. Bối cảnh

Bài báo xét hai câu hỏi lý thuyết về học sâu: vì sao SGD có thể tìm được nghiệm tốt cho bài toán tối ưu không lồi, và vì sao mạng khớp dữ liệu huấn luyện vẫn có thể tổng quát hóa trên dữ liệu mới. Các tác giả dùng góc nhìn lý thuyết thông tin để khảo sát biểu diễn bên trong mạng.

Ý tưởng nền (từ Tishby and Zaslavsky 2015, công trình trước của chính nhóm tác giả): một mạng nhiều lớp tạo ra một **chuỗi Markov** các biểu diễn kế tiếp của input X: $Y \to X \to T_1 \to T_2 \to \ldots \to T_K \to \hat{Y}$. Mỗi lớp ẩn T_i được đặc trưng hoàn toàn (không quan tâm trọng số/neuron cụ thể) bởi 2 số: $I(X;T_i)$ (lượng thông tin giữ lại về input) và $I(T_i;Y)$ (lượng thông tin giữ lại về nhãn, tức khả năng dự đoán). Vẽ 2 số này cho mỗi lớp ta được 1 điểm trên ”**information plane**” (mặt phẳng thông tin), và cả mạng tạo thành 1 đường đi (path) qua các điểm đó.

Khung **Information Bottleneck (IB)** (Tishby, Pereira, Bialek 1999) định nghĩa ”biểu diễn tối ưu” T của X đối với Y: nén X nhiều nhất có thể (min $I(X;T)$) trong khi vẫn giữ được lượng thông tin $I(T;Y)$ cho trước về nhãn. Bài báo 2015 của Tishby & Zaslavsky mới đề xuất framework này cho DNN; bài báo 2017 này (Schwartz-Ziv & Tishby) là bài đầu tiên **thực nghiệm kiểm chứng** framework đó một cách có hệ thống: vẽ quỹ đạo thật của các lớp DNN trong information plane suốt quá trình train, và xem chúng có hành xử đúng như IB dự đoán không.

Khái niệm cần nắm trước (xem thêm math 00 nếu có, hoặc phần 10 cuối bài này):

- **Entropy** $H(X) = -\sum_x p(x) \log p(x)$: số bit trung bình cần để mã hoá X.
- **Mutual information (thông tin tương hỗ)** $I(X;Y)$: lượng thông tin X và Y ”chia sẻ” với nhau; bằng 0 khi độc lập, lớn khi phụ thuộc mạnh.

- **Data Processing Inequality (DPI)**: nếu $X \to Y \to Z$ là chuỗi Markov (Z chỉ phụ thuộc X qua Y) thì $I(X;Y) \geq I(X;Z)$: xử lý dữ liệu qua nhiều bước chỉ có thể làm mất thông tin, không thể sinh thêm.

2. Ý tưởng chính

Tác giả huấn luyện các mạng fully-connected nhỏ (tối đa 7 lớp ẩn) nhiều lần với trọng số khởi tạo ngẫu nhiên khác nhau, tính chính xác $I(X;T_i)$ và $I(T_i;Y)$ cho từng lớp tại từng epoch (vì bài toán nhân tạo nên biết chính xác phân phối $P(X,Y)$, tính MI chính xác bằng tổng hữu hạn, không cần ước lượng từ mẫu), rồi vẽ quỹ đạo các lớp trong information plane qua thời gian huấn luyện.

5 kết quả chính (theo Abstract, mục 1.1):

1. **Phần lớn số epoch huấn luyện được dùng để nén biểu diễn (compression)**, không phải để fit nhãn (fitting) - fitting xảy ra nhanh, nén xảy ra rất chậm sau đó.
2. **SGD có 2 pha động học rõ rệt**: pha đầu (”**drift**”, trôi) gradient có mean lớn hơn hẳn độ lệch chuẩn (SNR cao) ứng với pha train error giảm nhanh (ERM); pha sau (”**diffusion**”, khuếch tán) gradient gần như nhiễu Gauss (SNR thấp), ứng với pha các lớp **nén thông tin về input** ($I(X;T_i)$ giảm) mà hầu như không còn cải thiện $I(T_i;Y)$ nhiều.
3. **Các lớp đã hội tụ nằm rất gần đường cong IB lý thuyết** (information curve), và thoả mãn các phương trình tự hợp (self-consistent) encoder-decoder của IB.
4. **Lợi ích chính của việc thêm lớp ẩn là tính toán (computational), không phải biểu diễn**: thêm lớp giúp giảm mạnh số epoch cần để hội tụ (vì pha diffusion với nhiều lớp mỏng tương đương nhiều bước nén nhỏ, nhanh hơn theo cấp số nhân so với 1 bước nén lớn).
5. Các lớp ẩn có xu hướng hội tụ gần các **điểm tới hạn (critical points)** trên đường cong IB (do hiện tượng ”critical slowing down” của quá trình khuếch tán gần chuyển pha).

Thông điệp cốt lõi: **tính ngẫu nhiên của SGD không chỉ để thoát local minima** (giải thích truyền thống) mà còn đóng vai trò tạo ra biểu diễn nén hiệu quả thông qua một quá trình khuếch tán (diffusion) kiểu entropy-maximization, và đây có thể là lý do DNN generalize tốt dù không có regularization tường minh.

3. Phương pháp chi tiết

3.1 Công cụ lý thuyết thông tin (mục 2 của paper)

Mutual information giữa 2 biến ngẫu nhiên X, Y với phân phối đồng thời $p(x,y)$ (Eq. 1-2 của paper):

$$I(X;Y) = D_{KL}\big[p(x,y)\|p(x)p(y)\big] = \sum_{x,y} p(x,y)\log\frac{p(x,y)}{p(x)p(y)} = H(X) - H(X|Y).$$

Hai tính chất MI quan trọng cho DNN mà bài báo nhấn mạnh:

- **Bất biến với biến đổi khả nghịch (invertible)**: $I(X;Y) = I(\psi(X);\varphi(Y))$ với ψ, φ khả nghịch bất kỳ (Eq. 3). Nghĩa là nếu một lớp chỉ ”xáo trộn” thông tin mà không mất gì (biến đổi 1-1), MI của nó với X và Y không đổi - dù các neuron riêng lẻ mã hoá đặc trưng gì đi nữa. Đây là lý do tác giả coi cả lớp T_i như **1 biến ngẫu nhiên duy nhất**, bỏ qua việc ”đọc hiểu” từng neuron.
- **Data Processing Inequality (DPI)**: với chuỗi Markov $X \to Y \to Z$: $I(X;Y) \geq I(X;Z)$ (Eq. 4). Áp dụng cho chuỗi các lớp DNN $Y \to X \to T_1 \to \ldots \to T_K \to \hat{Y}$ cho ra 2 chuỗi bất đẳng thức đơn điệu (Eq. 5-6):

$$I(X;Y) \geq I(T_1;Y) \geq \ldots \geq I(T_K;Y) \geq I(\hat{Y};Y), \qquad H(X) \geq I(X;T_1) \geq \ldots \geq I(X;T_K) \geq I(X;\hat{Y}).$$

Information Bottleneck (IB): biểu diễn "tối ưu" $T(X)$ đối với Y được định nghĩa là **thống kê đủ tối thiểu** (minimal sufficient statistic): nén X nhiều nhất mà vẫn giữ nguyên $I(T;Y) = I(X;Y)$ (Eq. 7). Vì thống kê đủ tối thiểu chính xác chỉ tồn tại với họ phân phối rất đặc biệt (họ hàm mũ - exponential family), Tishby et al. (1999) nới lỏng bài toán: cho T là ánh xạ **ngẫu nhiên** $p(t|x)$, và không cần giữ 100% $I(X;Y)$ mà đánh đổi bằng 1 hệ số Lagrange β (Eq. 8):

$$\min_{p(t|x),p(y|t),p(t)} \Big\{ I(X;T) - \beta I(T;Y) \Big\}.$$

Nghiệm (ẩn) thoả 3 phương trình tự hợp (**self-consistent equations**, Eq. 9):

$$p(t|x) = \frac{p(t)}{Z(x;\beta)} \exp\big(-\beta D_{KL}[p(y|x)\|p(y|t)]\big), \qquad p(t) = \sum_x p(t|x)p(x), \qquad p(y|t) = \sum_x p(y|x)p(x|t).$$

Khi β chạy, tập các nghiệm $(I(X;T), I(T;Y))$ vẽ ra 1 đường cong lồi/đơn điệu gọi là **information curve** (đường cong IB), là biên trên của vùng khả đạt trong information plane: không biểu diễn nào vượt qua được đường này với cùng mức nén.

3.2 Vai trò của noise (mục 2.4) và lựa chọn bài toán thực nghiệm

Nếu $y = f(x)$ là hàm xác định (deterministic), $I(X;Y)$ **không nhạy cảm với độ phức tạp của** f (vì MI bất biến với hoán vị bất kỳ của patterns khi $|X|$ hữu hạn) - đây là vấn đề: không thể dùng MI để phân biệt hàm đơn giản (VC-dimension thấp) với hàm phức tạp chỉ bằng MI của input-output xác định. Cách khắc phục: biến quy tắc thành **stochastic** bằng cách thêm nhiễu nhỏ, ví dụ dùng sigmoid $p(y = 1|x) = \psi(w \cdot x)$ với $\psi(x) = 1/(1 + e^{-x})$ (ví dụ Perceptron) - độ "trải" của sigmoid mã hoá độ phức tạp biên quyết định.

Thiết lập thực nghiệm (mục 3.1): mạng fully-connected feed-forward, hàm kích hoạt **tanh** cho mọi neuron ẩn, lớp cuối dùng dạng sigmoidal; huấn luyện bằng SGD với **cross-entropy loss**, không có regularization tường minh nào khác. Kiến trúc chuẩn trong bài: tối đa **7 lớp ẩn, độ rộng 12-10-7-5-4-3-2 neuron** (Hình 4). Bài toán: luật quyết định nhị phân **bất biến với phép quay** $O(3)$ **của mặt cầu**, với **12 input nhị phân** biểu diễn 12 điểm phân bố đều trên mặt cầu 2D $\Rightarrow 4096$ pattern input khác nhau, chia thành **64 quỹ đạo (orbit) rời nhau** dưới nhóm quay (đây chính là 1 phân hoạch đủ tối thiểu cho luật đối xứng cầu). Hàm $f(x)$ đối xứng cầu tính qua phổ công suất điều hoà cầu (spherical harmonics), so với ngưỡng θ cho nhãn $0/1$, rồi làm "mềm" bằng sigmoid gain γ: $p(y = 1|x) = \psi(f(x) - \theta)$ (Eq. 10). Ngưỡng θ chọn sao cho $p(y = 1) \approx 0.5$; gain γ đủ lớn để $I(X;Y) \approx 0.99$ bit (gần như xác định nhưng không hoàn toàn).

Ước lượng MI (mục 3.2): vì bài toán nhân tạo có đúng 4096 pattern đồng xác suất và biết trước $P(X,Y)$, tác giả **không cần ước lượng từ mẫu** mà tính MI chính xác: rời rạc hoá output kích hoạt arctan của mỗi neuron thành **30 khoảng đều giữa** -1 **và** 1, từ đó tính trực tiếp $P(T_i, X)$ và $P(T_i, Y) = \sum_x P(x,Y)P(T_i|x)$ theo chuỗi Markov $Y \to X \to T_i$. Lặp lại với **50 lần khởi tạo trọng số ngẫu nhiên** khác nhau và lấy mẫu train khác nhau để lấy trung bình/sai số.

4. Thí nghiệm và kết quả chính

Đây là paper **có thí nghiệm số thật** (không phải chỉ lý thuyết), nhưng trên mạng nhỏ và bài toán tổng hợp (synthetic) để kiểm soát được $P(X,Y)$ chính xác, không phải benchmark chuẩn như ImageNet/GLUE.

(a) Hai pha SGD trong information plane (mục 3.3-3.5, Hình 2-4): với 85% dữ liệu train, 50 mạng ngẫu nhiên, các lớp lúc đầu giảm mạnh I_Y (information trên nhãn) ở lớp sâu (do khởi tạo ngẫu nhiên), sau đó: **pha ERM** (ngắn, vài trăm epoch) tăng nhanh I_Y mọi lớp, giữ đúng thứ tự DPI; **pha nén biểu diễn** (dài hơn nhiều) giảm I_X (information trên input) mà gần như không mất I_Y thêm. Khớp với động học gradient (Hình 4):

drift phase (mean gradient $\gg$ std, SNR cao) $\rightarrow$ **diffusion phase** (std $\gg$ mean, giống Wiener process, SNR thấp), chuyển pha ở khoảng ~ 350 **epoch** với kiến trúc chuẩn của bài. Với mẫu train nhỏ (5%), pha nén còn làm **giảm cả** I_Y (dấu hiệu overfit, có thể tránh bằng early stopping) nhưng với mẫu lớn (85%) I_Y chủ yếu tăng trong cả 2 pha.

(b) Giải thích bằng Focker-Planck: pha diffusion mô tả bằng phương trình Fokker-Planck; phân phối dừng của nó **tối đa hoá entropy** của trọng số dưới ràng buộc train error $\Rightarrow$ tối đa $H(X|T_i) \Rightarrow$ tối thiểu $I(X;T_i) = H(X) - H(X|T_i)$ (vì $H(X)$ không đổi). Đây chính là cơ chế "nén bằng khuếch tán" (compression by diffusion).

(c) Lợi ích tính toán của lớp ẩn (mục 3.6, Hình 5): huấn luyện 6 kiến trúc với 1-6 lớp ẩn (80% dữ liệu, 50 lần lặp). Quan sát: (1) thêm lớp ẩn **giảm mạnh số epoch** cần để đạt I_Y tốt - với 1 lớp ẩn mạng không đạt I_Y tốt dù sau 10^4 epoch, với 6 lớp đạt gần như đầy đủ information trong vòng 400 epoch; (2) pha nén của mỗi lớp **ngắn hơn** nếu lớp trước đã được nén sẵn (so sánh 4 vs 5 lớp: 5 lớp hội tụ nhanh gần gấp đôi); (3) nén nhanh hơn ở các lớp sâu/hẹp hơn (gần output), các lớp này "kéo" lớp dưới nén theo; (4) **thêm độ rộng (width) không giúp** - lớp ẩn dù rộng cũng dần nén trong pha diffusion, lớp đầu tiên (rộng 12, bằng chiều input) gần như không nén được vì gần như là ánh xạ 1-1.

(d) Lợi ích tính toán nhìn theo lý thuyết khuếch tán (mục 3.7): khuếch tán đơn giản làm entropy tăng theo $\Delta H \propto \log(Dt)$ (D = hằng số khuếch tán), suy ra số bước (epoch) cần để nén ΔI_X là $\sim \exp(\Delta I_X / D)$. Nếu K lớp mỗi lớp chỉ cần nén một phần nhỏ ΔI_X^k ($\sum_k \Delta I_X^k \approx \Delta I_X$ tổng), Eq. 11 của paper chỉ ra:

$$\exp\Big(\sum_k \Delta I_X^k\Big) \geq \sum_k \exp(\Delta I_X^k),$$

nghĩa là chia nén thành K bước nhỏ **giảm số epoch theo cấp số nhân** so với 1 bước lớn (dù số phép toán mỗi epoch chỉ tăng tuyến tính theo K) - đây là lý giải định lượng cho "lợi ích tính toán của việc sâu hơn".

(e) Hội tụ về biên IB (mục 3.8, Hình 6, Eq. 12): với mỗi lớp đã hội tụ, tác giả tìm β_i^* tối thiểu hoá $\mathbb{E}_x D_{KL}[p_i(t|x) \| p_\beta^{IB}(t|x)]$ (encoder thật so với encoder IB lý thuyết ứng với decoder thật của lớp đó). Kết quả: 5 lớp thực nghiệm nằm **rất gần đường cong IB lý thuyết**, với độ dốc đường cong tại điểm đó (β^{-1}) khớp với β_i^* ước lượng - tức thoả mãn gần đúng hệ phương trình tự hợp Eq. 9.

(f) Ảnh hưởng kích thước mẫu train (mục 3.9, Hình 7): huấn luyện mạng 6 lớp với mẫu train từ 3% đến 85% dữ liệu. Mẫu lớn hơn $\Rightarrow I_Y$ (generalization) được đẩy lên gần biên IB lý thuyết hơn; các lớp hội tụ nằm trên 1 đường cong trơn (ứng với "finite-sample IB bound", biên IB khi decoder là phân phối thực nghiệm chứ không phải $P(X,Y)$ thật).

(g) Kiểm tra tổng quát hoá (mục 4, Hình 8 + thí nghiệm MNIST): lặp lại với luật **không đối xứng** ("committee machine") - quan sát 2 pha và các kết luận tương tự (pha nén nhanh hơn nhưng bản chất giống). Quan trọng: tác giả cũng **kiểm tra thống kê gradient trên MNIST** (bài toán lớn, thực - không chỉ synthetic) và **tìm thấy chuyển pha sang diffusion** tương tự, nhưng **không** vẽ đầy đủ information-plane cho MNIST (vì ước lượng MI trên bài toán lớn/liên tục khó hơn, đòi hỏi estimator phức tạp hơn, tác giả nói "this can be done" nhưng để công trình tương lai).

5. Phân tích / ablation

Paper không có bảng ablation theo nghĩa thông thường (so sánh thành phần kiến trúc), nhưng có các kiểm tra độ nhạy đóng vai trò ablation:

- **Đổi luật quyết định (symmetric $\rightarrow$ committee machine không đối xứng, mục 4 câu hỏi 1)**: 2 pha drift/diffusion và các kết luận chính **không đổi** $\Rightarrow$ tác giả kết luận hiện tượng không phụ thuộc cấu trúc đối xứng đặc biệt của bài toán gốc.

- **Số lớp ẩn (1-6 lớp, mục 3.6)**: biến thiên có kiểm soát, cho thấy lợi ích đơn điệu của việc thêm lớp tới tốc độ hội tụ (không phải biểu diễn cuối cùng, vì cùng 1 đường cong IB là đích).
- **Kích thước mẫu train (3%-85%, mục 3.9)**: cho thấy overfitting liên hệ trực tiếp với pha nén (mẫu nhỏ $\Rightarrow$ nén làm giảm cả I_Y, dấu hiệu mất thông tin liên quan tới nhãn vì mẫu không đại diện).
- **Kiểm tra trên MNIST (dataset thật)**: chỉ xác nhận **tồn tại chuyển pha gradient drift $\rightarrow$ diffusion**, KHÔNG xác nhận đầy đủ các kết luận về information plane/biên IB trên dữ liệu thật - đây là giới hạn quan trọng bài báo nêu một phần (xem mục 6).
- Tác giả cũng lưu ý (mục 3.5) **không có dấu hiệu trọng số tiến về 0 hoặc giảm norm** khi hội tụ, và tương quan giữa in-weights của các neuron trong cùng 1 điểm hội tụ **rất nhỏ** $\Rightarrow$ kết luận có **rất nhiều mạng khác nhau (hàm mũ theo số trọng số)** đạt hiệu năng tối ưu tương đương, nên diễn giải 1 neuron/trọng số cụ thể là ”vô nghĩa” (paper dùng đúng từ ”practically meaningless”).

6. Giới hạn

Theo bài báo nêu (mục 3.4, 4 và các câu hỏi tự đặt ra):

- Hiện tượng pha nén quan sát trên **mạng nhỏ, hàm kích hoạt tanh**, trên **bài toán tổng hợp có cấu trúc đối xứng** đơn giản; paper tự nói ”nên được kiểm chứng trên các bài toán lớn hơn” (mục 3.4: ”it should be verified on larger problems”).
- Trên **MNIST**, tác giả chỉ kiểm tra được **thống kê gradient** (xác nhận có chuyển pha drift/diffusion) nhưng **không** vẽ được đầy đủ information-plane / kiểm tra biên IB, vì ước lượng MI trên bài toán lớn/liên tục đòi hỏi estimator phức tạp hơn (paper nói ”this can be done” nhưng để sau).
- Paper tự đặt 4 câu hỏi mở chưa giải quyết triệt để trong chính bài: (1) tính phổ quát với luật/kiến trúc khác (một phần trả lời bằng committee machine), (2) liệu pha nén bằng nhiễu có xảy ra ở các mô hình học sâu khác (Bayesian network, random forest) hay không - **chỉ nêu giả thuyết, chưa kiểm chứng**, (3) có scale lên bài toán thực lớn không - **chỉ kiểm chứng phần gradient trên MNIST**, (4) hệ quả thuật toán thực tiễn (ví dụ dùng Monte-Carlo relaxation thay SGD trong pha diffusion) - **chỉ là hướng đang nghiên cứu, không có kết quả cụ thể trong bài này**.
- Paper nói ”we are convinced that our analysis and observations are general” (mục 2.5) nhưng đây là **niềm tin/giả thuyết của tác giả**, không phải kết quả đã chứng minh cho mạng lớn.

(Ngoài paper, kiến thức chung) Đây là 1 trong các paper gây tranh cãi nhiều nhất về lý thuyết Deep Learning: nghiên cứu sau đó (Saxe, Bansal, Dapello, Advani, Kolchinsky, Tracey, Cox, ”On the information bottleneck theory of deep learning”, ICLR 2018) phản bác một phần kết luận ở đây, cho rằng (a) hiện tượng ”pha nén” quan sát được **phụ thuộc mạnh vào hàm kích hoạt bão hoà (saturating) như tanh/sigmoid** và **không xuất hiện rõ với ReLU** (hàm kích hoạt phổ biến hơn trong Deep Learning thực tế), và (b) các phép đo ”mutual information” trên mạng với kích hoạt liên tục/xác định phải dựa vào kỹ thuật binning cụ thể, kết quả MI đo được có thể phụ thuộc vào cách binning hơn là phản ánh hiện tượng nén ”thật”. Người đọc nên xem bài 1703.00810 này là 1 giả thuyết thú vị và có thực nghiệm ủng hộ trên mạng nhỏ, không phải 1 định lý đã được cộng đồng đồng thuận hoàn toàn.

7. Liên hệ với các bài khác

- **”interp content/01_2022_mech_interp_essay.md”**: một hướng hoàn toàn khác để ”mở hộp đen” DNN, mechanistic interpretability (lần theo circuit/feature cụ thể) so với cách tiếp cận information-theoretic/thống kê tổng hợp (không nhìn vào trọng số cụ thể) của bài này - 2 triết lý bổ sung cho nhau: bài 30 nói ”từng neuron vô nghĩa”, mech interp thì cố gắng tìm ý nghĩa ở mức circuit/feature.
- **”interp content/05_2022_toy_models_of_superposition.md”**: cũng nghiên cứu cấu trúc biểu diễn ẩn của mạng nhỏ bằng mô hình toán học/thực nghiệm toy có kiểm soát, giống phương pháp luận của bài 30 (dùng bài toán tổng hợp nhỏ để hiểu hiện tượng tổng quát).

- **”ml basics content/01_2016_rethinking_generalization.md”** (Zhang et al., ”Understanding deep learning requires rethinking generalization”): bài này được **chính paper 30 trích dẫn trực tiếp** (mục 3.5, ”Zhang et al. (2016)”) làm bằng chứng rằng các dạng regularization tường minh (weight decay, dropout, data augmentation) không giải thích đủ khả năng generalize của DNN - cùng câu hỏi lớn mà bài 30 cố trả lời bằng information bottleneck.

8. Gợi ý áp dụng

Information plane là một công cụ khảo sát biểu diễn theo từng lớp. Việc ước lượng mutual information trong không gian liên tục và nhiều chiều có những khó khăn đáng kể; kết quả phụ thuộc phương pháp ước lượng và không nên được diễn giải như phép đo trực tiếp không sai lệch.

9. Thí nghiệm gợi ý

1. **Tái tạo thu nhỏ hiện tượng 2 pha trên MLP nhỏ (tanh)**: huấn luyện MLP 4-5 lớp (ví dụ 20-15-10-5-2 neuron), kích hoạt tanh, trên 1 bài toán phân loại nhị phân tổng hợp nhỏ (ví dụ XOR mở rộng hoặc tập dữ liệu dạng ”circle”/”moons” của scikit-learn). Đo và vẽ $I(X;T_i)$, $I(T_i;Y)$ qua epoch bằng binning giống paper (rời rạc hoá activation thành ~30 bin). Kỳ vọng: thấy pha tăng I_Y nhanh rồi pha giảm I_X chậm hơn, tương tự Hình 3 của paper. Chạy được trên CPU/trên GPU trong vài phút.
2. **So sánh tanh vs ReLU**: lặp lại thí nghiệm 1 nhưng đổi activation sang ReLU. Đo xem pha ”nén” (I_X giảm rõ) còn xuất hiện rõ như với tanh không. Kỳ vọng theo phản biện Saxe et al. (ngoài paper): hiện tượng nén sẽ **yếu hơn hoặc không rõ** với ReLU - nếu đúng, xác nhận giới hạn đã nêu ở mục 6.
3. **Đo SNR của gradient theo epoch**: trong cùng thí nghiệm 1, log lại mean và std của gradient mỗi lớp qua epoch (giống Hình 4 paper), vẽ trên thang log-log. Kỳ vọng thấy điểm chuyển giữa 2 pha (SNR cao $\rightarrow$ thấp) khớp thời điểm I_X bắt đầu giảm mạnh.
4. **Ảnh hưởng số lớp tới tốc độ hội tụ** I_Y: lặp lại thí nghiệm 1 với 1, 2, 4, 6 lớp ẩn (giữ tổng số neuron gần tương đương hoặc để width giảm dần như paper), đo số epoch cần để I_Y đạt một ngưỡng cố định (ví dụ 90% giá trị hội tụ). Kỳ vọng: mạng nhiều lớp hơn đạt ngưỡng này **nhanh hơn** (ít epoch hơn), minh hoạ lại mục 4(c)/(d) của note.
5. **Ảnh hưởng kích thước mẫu train tới overfitting qua information plane**: lặp thí nghiệm 1 với tỉ lệ dữ liệu train 5%, 45%, 85% (giữ tập test cố định để đo I_Y ”thật”). Kỳ vọng: với mẫu nhỏ, I_Y **giảm** trong pha nén (dấu hiệu overfit), với mẫu lớn I_Y tiếp tục tăng hoặc giữ nguyên - khớp quan sát mục 3.4 của paper.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / định nghĩa	Ý nghĩa
Mutual information	$I(X;Y) = \sum_{x,y} p(x,y) \log \frac{p(x,y)}{p(x)p(y)} = H(X) - H(X\|Y)$ (Eq. 1-2)	Lượng thông tin X, Y chia sẻ; 0 khi độc lập
Bất biến khả nghịch	$I(X;Y) = I(\psi(X);\varphi(Y))$ (Eq. 3)	Lớp biến đổi 1-1 không đổi MI với input/label
Data Processing Inequality	$X \rightarrow Y \rightarrow Z \Rightarrow I(X;Y) \geq I(X;Z)$ (Eq. 4)	Xử lý thêm chỉ mất thông tin, không sinh thêm
Information Bottleneck	$\min_{p(t\|x)} I(X;T) - \beta I(T;Y)$ (Eq. 8)	Đánh đổi nén X và giữ thông tin về Y
Self-consistent equations	$p(t\|x) \propto p(t) \exp(-\beta D_{KL}[p(y\|x)\|p(y\|t)])$ (Eq. 9)	Điều kiện biểu diễn tối ưu trên đường cong IB
Compression theo diffusion	số epoch $\sim \exp(\Delta I_X / D)$; với K lớp: $\exp(\sum_k \Delta I_X^k) \geq \sum_k \exp(\Delta I_X^k)$ (Eq. 11)	Lý giải lợi ích tính toán của việc thêm lớp sâu

Khái niệm	Công thức / định nghĩa	Ý nghĩa
β_i^* tối ưu mỗi lớp	$\beta_i^* = \arg\min_\beta \mathbb{E}_x D_{KL}[p_i(t\|x)\|p_\beta^{IB}(t\|x)]$ (Eq. 12)	Đo lớp thực nghiệm gần đường cong IB tới mức nào

Ví dụ tính tay (tự tính): minh hoạ công thức Eq. 1-2 với 1 phân phối rời rạc nhỏ, $X, Y \in \{0, 1\}$, phân phối đồng thời $p(0, 0) = 0.4$, $p(0, 1) = 0.1$, $p(1, 0) = 0.1$, $p(1, 1) = 0.4$ (kênh nhiễu, tương quan dương giữa X và Y). Biên: $p(X = 0) = p(X = 1) = 0.5$, $p(Y = 0) = p(Y = 1) = 0.5$, nên $p(x)p(y) = 0.25$ với mọi cặp. Tính từng số hạng (log cơ số 2):

$$I(X;Y) = 0.4 \log_2 \frac{0.4}{0.25} + 0.1 \log_2 \frac{0.1}{0.25} + 0.1 \log_2 \frac{0.1}{0.25} + 0.4 \log_2 \frac{0.4}{0.25}.$$

$\log_2(1.6) \approx 0.678$ và $\log_2(0.4) \approx -1.322$, nên (tự tính): $I(X;Y) \approx 2 \times (0.4 \times 0.678) + 2 \times (0.1 \times (-1.322)) = 2 \times 0.2711 - 2 \times 0.1322 = 0.5422 - 0.2644 \approx 0.278$ bit. So với trường hợp X, Y độc lập hoàn toàn ($I = 0$) và trường hợp xác định hoàn toàn ($I(X;Y) = H(Y) = 1$ bit vì $p(y = 1) = 0.5$), giá trị ≈ 0.278 bit phản ánh đúng mức tương quan vừa phải mà ta đặt vào ví dụ - minh hoạ trực quan cho đại lượng mà paper dùng để đo $I(X;T_i)$ và $I(T_i;Y)$ ở mỗi lớp.

11. Kết luận ngắn

Bằng cách coi mỗi lớp DNN là 1 biến ngẫu nhiên và vẽ quỹ đạo $(I(X;T_i), I(T_i;Y))$ qua các epoch huấn luyện trên mạng fully-connected nhỏ (tanh) với bài toán tổng hợp có $P(X, Y)$ biết trước, paper phát hiện SGD có 2 pha rõ rệt (drift/ERM rồi diffusion/nén), các lớp hội tụ gần đường cong Information Bottleneck lý thuyết, và lợi ích chính của việc thêm lớp sâu là **giảm số epoch cần để nén** (lợi ích tính toán) chứ không hẳn là biểu diễn cuối cùng tốt hơn. Đây là 1 trong những cố gắng thực nghiệm sớm và có ảnh hưởng lớn để giải thích ”tại sao Deep Learning hoạt động”, nhưng kết luận giới hạn ở mạng nhỏ/kích hoạt bão hoà/bài toán tổng hợp, và bị phản biện đáng kể sau đó (Saxe et al. 2018, ngoài phạm vi paper này) về tính phổ quát với ReLU và độ nhạy của phép đo MI với cách binning.

Nguồn gốc: https://arxiv.org/abs/1703.00810

Bài 31

Deep Learning: A Statistical Viewpoint (2021)

Tác giả: Peter L. Bartlett, Andrea Montanari, Alexander Rakhlin
Venue / arXiv: arXiv:2103.09177 (v1, 16/03/2021); paper không nêu venue trong phần text trích được (được biết đến rộng rãi là Acta Numerica 2021, nhưng dòng chữ này không xuất hiện trong văn bản arXiv trích xuất được, nên ghi ”paper không nêu venue” theo đúng text trích được)
Nguồn: https://arxiv.org/abs/2103.09177
Chủ đề: Bài khảo sát (survey) tổng hợp các kết quả gần đây của lý thuyết học thống kê nhằm lý giải hai hiện tượng bí ẩn của deep learning: (a) gradient descent giải được bài toán tối ưu không lồi một cách hiệu quả khi mô hình quá tham số hoá (tractability via overparametrization), và (b) mô hình vẫn tổng quát hoá tốt dù khớp hoàn hảo dữ liệu huấn luyện có nhiễu (generalization via implicit regularization / benign overfitting).
Vị trí trong lộ trình: Đọc sau math 32 (Benign Overfitting in Linear Regression, một trong các kết quả chính được khảo sát lại ở đây), math 33 (Gradient Descent Finds Global Minima of DNNs) và math 34 (Mean Field View), vì bài này tổng hợp và đặt cả ba vào một khung lý thuyết thống nhất; là bài cuối cùng được viết trong batch math 28-35 nhưng đóng vai trò ”bản đồ tổng quan” nên có thể đọc trước hoặc sau các bài 32/33/34 tuỳ mục đích.

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (bản arXiv); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc.

1. Bối cảnh

Bài khảo sát bắt đầu từ ba yêu cầu của dự đoán thống kê: khả năng biểu diễn dữ liệu, kiểm soát độ phức tạp khi dữ liệu huấn luyện hữu hạn và hiệu quả tính toán. Một cách tiếp cận cổ điển là kết hợp mô hình giàu biểu đạt với regularization tường minh, chẳng hạn ridge hoặc early stopping, và dùng hàm mất mát lồi để tối ưu hiệu quả.

Deep learning phá vỡ cả hai nguyên tắc cổ điển này qua hai quan sát thực nghiệm gây ”bất ngờ”:

1. **Tractability via overparametrization**: khi mô hình đủ giàu tham số (quá tham số hoá), bài toán tối ưu **không lồi** lại trở nên dễ giải bằng các phương pháp gradient đơn giản, dù về lý thuyết cổ điển điều này ”có vẻ hoàn toàn vô lý” (paper dùng chính từ ”foolish”).
2. **Generalization via implicit regularization**: mô hình được huấn luyện **không có regularization tường minh nào**, đạt loss huấn luyện gần 0 (khớp hoàn toàn dữ liệu, kể cả dữ liệu có nhiễu), nhưng vẫn **tổng quát hoá tốt** trên dữ liệu mới. Đây là hiện tượng paper gọi là **”benign overfitting”** (tạm dịch: overfitting lành tính) - dùng đúng nghĩa đen của từ ”overfitting” (khớp tốt hơn mức cần thiết), không mang nghĩa tiêu cực như cách dùng thông thường trong giáo trình thống kê cổ điển.

Thuật ngữ cần làm rõ: **uniform convergence** (hội tụ đều) là tính chất mà sai số thực nghiệm (empirical risk) xấp xỉ tốt sai số kỳ vọng (true risk) đồng thời cho **mọi** hàm trong một lớp hàm $\mathcal{F}$, không chỉ cho một hàm cụ thể; đây là công cụ cốt lõi của lý thuyết học thống kê cổ điển (VC theory, Rademacher complexity). **Implicit regularization** (điều chuẩn ẩn/ngầm) là hiện tượng thuật toán tối ưu (ví dụ gradient descent) tự động chọn ra một nghiệm ”đặc biệt” trong số rất nhiều nghiệm khớp hoàn hảo dữ liệu, mà không cần một số hạng regularization tường minh nào trong hàm mục tiêu.

Paper phát biểu rõ **hai giả thuyết làm việc (working hypotheses)** xuyên suốt toàn bài (trích dẫn gần nguyên văn, dịch nghĩa): (1) độ khó của bài toán tối ưu phụ thuộc vào quan hệ giữa số chiều tham số và số lượng ràng buộc (kích thước mẫu); bài toán trở nên dễ (tractable) khi và chỉ khi mô hình đủ ”thiếu ràng buộc” (quá tham

số hoá); (2) các thuật toán huấn luyện tạo ra một độ lệch (bias) phá vỡ sự tương đương giữa mọi mô hình khớp hoàn hảo dữ liệu quan sát; mô hình deep learning có thể phân rã thành một **thành phần độ phức tạp thấp** (nơi uniform convergence cổ điển vẫn áp dụng) và một **thành phần độ phức tạp cao** (cho phép khớp hoàn hảo dữ liệu), và nếu mô hình đủ quá tham số hoá, thành phần sau không ảnh hưởng đáng kể đến độ chính xác dự đoán.

Paper cũng tự nói rõ một giới hạn phạm vi ngay từ Introduction: **không có kết quả nào trong bài chứng minh lợi ích của độ sâu (depth)** cho tối ưu hoá hay tổng quát hoá; độ sâu được paper cho là quan trọng chủ yếu cho **tính biểu diễn (expressivity)**, một khía cạnh nằm ngoài phạm vi khảo sát này.

2. Ý tưởng chính

Ý tưởng tổ chức xuyên suốt bài là **phân rã "đơn giản + gai nhọn" (simple-plus-spiky decomposition)**: mọi nghiệm nội suy (interpolating solution) tốt mà paper khảo sát đều có thể viết dưới dạng

$$\hat{f}(x) = \hat{f}_0(x) + \Delta(x),$$

với $\hat{f}_0$ là một thành phần **đơn giản** (ví dụ trơn/smooth, hoặc nằm trong một không gian con chiều thấp) chịu trách nhiệm cho khả năng dự đoán tốt, và Δ là một thành phần **gai nhọn (spiky)** chịu trách nhiệm cho việc khớp hoàn hảo dữ liệu (kể cả nhiễu) nhưng, trong điều kiện thuận lợi (quá tham số hoá đủ lớn), **không gây hại** cho độ chính xác dự đoán ($L(\hat{f}_0 + \Delta) \approx L(\hat{f}_0)$).

Lý thuyết hội tụ đều cổ điển **không thể** giải thích hiện tượng benign overfitting khi có nhiễu. Paper chứng minh điều này một cách định lượng (mục 2.9, đọc kỹ): với bất kỳ cận trên dựa trên độ phức tạp (ví dụ Rademacher complexity) của hàm nội suy có độ phức tạp tối thiểu, nếu dữ liệu có nhiễu (không có quan hệ tất định $y = f^*(x)$), thì cận trên này **nhất thiết phải** $\geq$ **một hằng số dương** (bằng đúng mức nhiễu $L(f^*)$), bất kể kích thước mẫu n lớn đến đâu. Lý do: cận hội tụ đều không phụ thuộc vào tính chất của phân phối X, nên vẫn đúng cả trong bối cảnh "fixed design" (thiết kế cố định $x_1, \ldots, x_n$), nơi excess risk của một hàm nội suy hoàn hảo **chắc chắn** $\geq$ mức nhiễu. Điều này nghĩa là: **cần một công cụ lý thuyết khác** (không chỉ dựa vào độ phức tạp của riêng hàm $\hat{f}$) để giải thích benign overfitting; công cụ đó phải khai thác thêm thông tin về phân phối dữ liệu/cơ chế sinh dữ liệu, không chỉ về bản thân hàm $\hat{f}$.

3. Phương pháp chi tiết

3.1. Implicit regularization qua gradient descent (Mục 3)

Ví dụ đơn giản nhất: hồi quy tuyến tính bình phương tối thiểu trong chế độ quá tham số hoá ($d > n$). Gradient descent khởi tạo từ $\theta_0 = 0$ trên loss $\hat{L}(\theta) = \frac{1}{n}\|X\theta - y\|_2^2$ hội tụ về **nghiệm có norm nhỏ nhất (minimum-norm interpolant)**:

$$\hat{\theta} = \operatorname*{argmin}_{\theta} \{\|\theta\|_2 : \langle \theta, x_i \rangle = y_i \; \forall i \leq n\} = X^{\dagger} y, \qquad \text{(14-15, paper)}$$

là giới hạn của ridge regression khi $\lambda \to 0^+$. Chứng minh dựa vào điều kiện KKT: $X\theta = y$ và $\theta + X^T\mu = 0$, cả hai được thoả bởi gradient descent khi nó khớp dữ liệu trong khi vẫn ở trong không gian sinh bởi các hàng của X.

Ví dụ thứ hai: **boosting**. AdaBoost tương ứng với coordinate descent trên loss hàm mũ (18); với dữ liệu phân tách tuyến tính (separable), Zhu & Yu (2005)/Telgarsky (2013) chứng minh hướng của θ_t hội tụ về nghiệm **max-margin chuẩn** ℓ_1. Soudry et al. (2018)/Ji & Telgarsky (2018) chứng minh **gradient** descent (không phải coordinate descent) trên loss logistic cho dữ liệu separable hội tụ về nghiệm **max-margin chuẩn** ℓ_2 (Định lý 3.1 của paper, trích đầy đủ): $\theta_t = \hat{\theta} \log t + \rho_t$ với $\|\rho_t\| = O(\log \log t)$, nên $\theta_t / \|\theta_t\|_2 \to \hat{\theta}/\|\hat{\theta}\|_2$.

Ví dụ thứ ba: **tham số hoá lại (reparametrization)**. Với $\theta(u)_i = u_i^2$ (bình phương theo toạ độ), gradient flow trên bài toán tham số hoá lại với khởi tạo $\alpha \to 0$ hội tụ về nghiệm **chuẩn ℓ_1 nhỏ nhất** trong không gian θ gốc (Gunasekar et al. 2017), khác hẳn với chuẩn ℓ_2 của tham số hoá gốc - cho thấy **cách tham số hoá mô hình cũng tạo ra implicit regularization**, không chỉ thuật toán tối ưu.

3.2. Benign overfitting: ví dụ Nadaraya-Watson (Mục 4.1)

Ví dụ sư phạm đơn giản nhất: ước lượng Nadaraya-Watson $\hat{f}(x) = \sum_i y_i\, \omega_i(x)$ với $\omega_i(x) = K((x - x_i)/h) / \sum_j K((x - x_j)/h)$. Với kernel **kỳ dị tại 0** (ví dụ $K(u) = \|u\|_2^{-a} \mathbb{1}[\|u\|_2 \le 1]$ với $0 < a < d/2$), ước lượng này **tự động nội suy dữ liệu** với mọi bandwidth h. Định lý 4.1 (trích từ Belkin, Rakhlin, Tsybakov 2019): với f^* thuộc lớp Hölder $H(\beta, L)$,

$$\mathbb{E}[\widehat{\text{bias}}^2] \lesssim h^{2\beta}, \qquad \mathbb{E}[\widehat{\text{var}}] \lesssim \sigma_\xi^2 (nh^d)^{-1}, \tag{31, paper}$$

chọn $h = n^{-1/(2\beta+d)}$ cho tốc độ minimax tối ưu. Điểm mấu chốt bài báo nhấn mạnh: **việc kernel có kỳ dị hay không (quyết định có nội suy hay không) hoàn toàn độc lập với mức độ "làm mượt" (bandwidth h, quyết định bias-variance trade-off)** - đây là minh chứng sư phạm đầu tiên cho thấy nội suy (fit hoàn hảo) không nhất thiết xung đột với một bias-variance trade-off tốt.

3.3. Benign overfitting trong hồi quy tuyến tính (Mục 4.2, liên hệ trực tiếp math 32)

Với $f^*(x) = \langle \theta^*, x \rangle$ và nghiệm minimum-norm $\hat{\theta}$ (Eq. 32-33), decomposition theo **effective rank** của ma trận hiệp phương sai $\Sigma = \mathbb{E}[xx^T]$: chọn chỉ số k, định nghĩa

$$r_k(\Sigma) = \frac{\sum_{i>k} \lambda_i}{\lambda_{k+1}}, \tag{Lemma 4.3/định nghĩa effective rank, paper}$$

nếu $r_k(\Sigma) \gtrsim n$, phần "tail" của ma trận dữ liệu $X_{>k}^T X_{>k}$ hành xử gần như đẳng hướng (near-isotropic), tạo ra một **self-induced regularization** với quy mô $\gamma = \sum_{i>k} \lambda_i$ (tương đương một ridge penalty tự nhiên). Đây đúng là kết quả chính của Bartlett-Long-Lugosi-Tsigler (2019/2020) đã được trình bày đầy đủ ở ghi chú **math 32**; paper này trình bày lại cùng kết quả ở dạng súc tích hơn, nhấn mạnh **sự khác biệt giữa self-induced regularization (phụ thuộc cơ chế sinh dữ liệu, tức tính chất của Σ) và implicit regularization (phụ thuộc thuật toán tối ưu, tức việc chọn nghiệm minimum-norm)** - một phân biệt quan trọng mà paper tự highlight bằng một footnote (chú thích số 4 trong văn bản).

3.4. Chế độ tuyến tính (linear regime) cho mạng nơ-ron (Mục 5.1)

Với mạng $f(x; \theta)$, gradient flow $\frac{d\theta_t}{dt} = \frac{1}{n} Df_n(\theta_t)^T (y - f_n(\theta_t))$ (Eq. 77) được so sánh với phiên bản **tuyến tính hoá (linearized)** quanh khởi tạo θ_0 (Eq. 78-79). **Định lý 5.1** (trích đầy đủ): nếu

$$\text{Lip}(Df_n)\, \|y - f_n(\theta_0)\|_2 < \frac{1}{4} \sigma_{\min}^2(Df_n(\theta_0)), \tag{84, paper}$$

thì (a) empirical risk giảm theo cấp số mũ với tốc độ $\lambda_0 = \sigma_{\min}^2/(2n)$; (b) tham số θ_t ở gần khởi tạo và được theo dõi sát bởi luồng tuyến tính hoá; (c) hàm $f(\theta_t)$ và $f_{\text{lin}}(\theta_t)$ gần nhau trên dữ liệu test. Áp dụng cho mạng 2 lớp $f(x; \theta) = \frac{\alpha}{\sqrt{m}} \sum_j b_j \sigma(\langle w_j, x \rangle)$ (Eq. 90) với hai cách khởi tạo khác nhau (một khởi tạo "chuẩn" $\theta_0^{(1)}$, một khởi tạo "đối xứng-trừ-ban-đầu" $\theta_0^{(2)}$ có $f(x; \theta_0^{(2)}) = 0$), **Định lý 5.4** cho thấy: với $\alpha = \Theta(1)$, cần $md \gg n^2$ (mạnh hơn hẳn điều kiện "chỉ cần quá tham số hoá" $md \gg n$); nhưng với $\alpha \to \infty$ và khởi tạo $\theta_0^{(2)}$, chỉ cần $md \gg n \log n$ (gần với mức quá tham số hoá tối thiểu) để đạt chế độ tuyến tính.

3.5. Mean-field vượt ra ngoài chế độ tuyến tính (Mục 5.2, liên hệ trực tiếp math 34)

Với scaling $\alpha = \alpha_0/\sqrt{m}$ (khác với chế độ tuyến tính), paper trình bày **Định lý 5.5** (trích dẫn công trình Mei-Montanari-Nguyen chính là **math 34**): giới hạn $m \to \infty$ của gradient flow là PDE

$$\partial_t \rho_t(w,b) = \alpha_0 \nabla(\rho_t(w,b)\nabla\Psi(w,b;\rho_t)), \qquad \text{(106, paper)}$$

và paper tự tính toán định lượng (Eq. 112-115) rằng trong chế độ mean-field, độ lệch trọng số **không triệt tiêu** khi $m \to \infty$ ($\frac{1}{\sqrt{m}}\|W_t - W_0\|_F$ bị chặn dưới bởi $c_1\sqrt{d/n}$), đối lập hẳn với chế độ tuyến tính (nơi độ lệch này $\to 0$). Đây là minh chứng định lượng cho thấy **cùng một kiến trúc mạng 2 lớp vô hạn rộng có thể có hai chế độ huấn luyện hoàn toàn khác nhau** (tuyến tính/"lazy" vs. mean-field/"feature learning"), chỉ phụ thuộc vào cách chọn hệ số scaling α theo m.

3.6. Ba mô hình tuyến tính hoá: NT-norm, random features, neural tangent (Mục 6.1-6.4)

Paper định nghĩa 3 "họ hàm" được xây từ việc tuyến tính hoá mạng 2 lớp theo các cách khác nhau:

- $\mathcal{F}_{RF}^m$ **(random features)**: chỉ huấn luyện lớp 2, lớp 1 cố định ngẫu nhiên: $f_{\text{lin}}(x;a) = \sum_i a_i \sigma(\langle w_i, x\rangle)$ (Eq. 124).
- $\mathcal{F}_{NT}^m$ **(neural tangent)**: chỉ huấn luyện lớp 1 (tuyến tính hoá), lớp 2 cố định: $f_{\text{lin}}(x;a) = \sum_i \langle a_i, x\rangle \sigma'(\langle w_i, x\rangle)$ (Eq. 125).

Cả hai tương ứng với **kernel ridge regression (KRR)** với các kernel ngẫu nhiên $K_{RF,m}$, $K_{NT,m}$ (Eq. 126-127), hội tụ về kernel kỳ vọng K_{RF}, K_{NT} khi $m \to \infty$, là các hàm của $\langle x_1, x_2\rangle$ (bất biến quay).

4. Thí nghiệm và kết quả chính

Paper này là một bài khảo sát lý thuyết, **không tiến hành thí nghiệm mới trên dữ liệu thực**; phần "kết quả" chính là các định lý được tổng hợp từ nhiều công trình, cùng **một hình mô phỏng số minh hoạ (Hình 1)** để trực quan hoá một hiện tượng lý thuyết.

Hình 1 (double descent, mục 6.3.2, đọc kỹ): mô phỏng random features model với $d = 100, n = 400, \tau^2 = 0.5$, hàm đích $f^* = \langle \beta_0, x\rangle$ với $\|\beta_0\|_2 = 1$, ridge regression với $\lambda = 10^{-3}(m/d)$, quét tỷ lệ quá tham số hoá m/n từ 0 đến 5, trung bình trên 20 lần thực hiện độc lập. Kết quả: train error giảm đơn điệu theo m/n; **test error có một đỉnh (peak) tại ngưỡng nội suy** $m/n = 1$ rồi giảm và hội tụ về một giá trị tiệm cận khi $m/n \gg 1$. Đây là hiện tượng **"double descent"** (lần đầu đặt tên bởi Belkin, Hsu, Ma, Mandal 2019), và paper chỉ ra các đường cong trong Hình 1 là **dự đoán tiệm cận chính xác (exact asymptotic)** từ lý thuyết ma trận ngẫu nhiên phi tuyến (nonlinear random matrix theory), không phải chỉ là xu hướng định tính.

Định lý 6.1 (polynomial scaling, đọc kỹ): với $d^\ell \ll m \ll d^{\ell+1}$ (phân tách lũy thừa rõ rệt giữa m, n, d), test error của random features model bị quyết định bởi $\min(m,n)$: nếu $m \ll n$ (lỗi xấp xỉ chiếm ưu thế), mô hình chỉ khớp được thành phần đa thức bậc $\le \ell$ của f^*; nếu $n \ll m$ (lỗi thống kê chiếm ưu thế), cũng chỉ học được $P_{\le \ell} f^*$. Cả hai được paper mô tả bằng một "parameter-counting heuristic": không gian đa thức bậc ℓ có chiều $\Theta(d^\ell)$, cần số tham số/mẫu tương ứng cỡ đó để học được.

Định lý 6.2 + Hệ quả 6.3 (universality và tối ưu của nội suy ở SNR cao, đọc kỹ): ở chế độ tỷ lệ proportional ($m/d \to \psi_w$, $n/d \to \psi_s$), risk của random features model **tiệm cận bằng** risk của một "noisy features model" đơn giản hơn (ma trận với các thành phần độc lập), một hiện tượng **universality**. Hệ quả: tồn tại một ngưỡng SNR^* sao cho nếu tỷ số tín hiệu/nhiễu (SNR) của bài toán $> \text{SNR}^*$, thì **regularization tối ưu là** $\lambda = 0^+$ (tức nội suy hoàn hảo là tối ưu, không chỉ tối ưu về tốc độ mà về chính giá trị risk).

Định lý 6.4 (neural tangent model, chỉ giới thiệu kết quả, không đọc chi tiết phần proof dài): tổng quát hoá một phần Định lý 6.1 cho NT model; paper kết luận NT model có tính chất tổng quát hoá **rất giống RF model** (nếu giữ số tham số không đổi), nhưng NT có **lợi thế tính toán**: với cùng số tham số p, độ phức tạp

dự đoán là $O(p)$ cho NT so với $O(pd)$ cho RF (vì NT không cần nhân ma trận W lúc dự đoán, chỉ cần dùng Jacobian đã tính).

5. Phân tích / ablation

Vì đây là bài khảo sát, "phân tích" ở đây tổng hợp các so sánh định lượng mà chính paper đưa ra giữa các mô hình/chế độ:

So sánh 2 khởi tạo trong chế độ tuyến tính (Định lý 5.4, mục 3.4 của ghi chú): sự khác biệt giữa yêu cầu $md \gg n^2$ (khởi tạo chuẩn, $\alpha = \Theta(1)$) và $md \gg n \log n$ (khởi tạo "triệt tiêu ban đầu", $\alpha \to \infty$) cho thấy **cách khởi tạo ảnh hưởng mạnh đến mức độ quá tham số hoá cần thiết** để đạt chế độ tuyến tính - một bài học thực hành quan trọng (dù chỉ chứng minh cho mạng 2 lớp đơn giản).

So sánh chế độ tuyến tính vs. mean-field (mục 3.5 của ghi chú): cùng kiến trúc, chỉ khác hệ số scaling α theo m, cho hai hành vi huấn luyện khác hẳn về chất (trọng số gần như không đổi vs. trọng số dịch chuyển đáng kể). Paper tự nhấn mạnh đây **không phải là một hạn chế kỹ thuật của phép phân tích**, mà là một **sự khác biệt thực sự về chế độ huấn luyện** (chứng minh bằng chặn dưới định lượng, Eq. 113-115, không chỉ chặn trên).

So sánh RF vs NT vs KRR (mục 6.4): paper liệt kê 4 câu hỏi dẫn dắt phần này (dịch nghĩa): (i) RF/NT có hành xử giống KRR không? (ii) m cần lớn đến đâu để khớp giới hạn kernel? (iii) sai khác gì khi m không đủ lớn? (iv) có mô hình nào ưu thế hơn? Trả lời tổng hợp của paper: về mặt **thống kê** (generalization), ba mô hình tương đương khi số tham số đủ lớn so với n; về mặt **tính toán**, NT có lợi thế rõ rệt với cùng số tham số.

Phân tích vai trò effective dimension/SNR trong Hệ quả 6.3: kết quả "nội suy tối ưu khi SNR cao" là một đóng góp ý nghĩa: nó **phản bác trực giác cổ điển** rằng luôn cần một chút regularization; ở chế độ "dữ liệu sạch, nhiễu thấp", mô hình nội suy hoàn hảo ($\lambda = 0^+$) mới là lựa chọn tối ưu đúng nghĩa (optimal risk, không chỉ optimal rate).

6. Giới hạn

Các giới hạn dưới đây paper **tự nêu rõ**, đặc biệt tập trung ở Mục 7 (Conclusions and future directions, đọc kỹ):

- **Chỉ giải quyết được các mô hình tuyến tính trong một không gian đặc trưng (feature space)** mà bản đồ đặc trưng (featurization) không phụ thuộc dữ liệu: RKHS, random features, neural tangent features. Paper tự nói: "the scope of this analysis might appear to be limited (neural networks are notoriously nonlinear in their parameters)".
- **Phân rã "đơn giản + gai nhọn" chỉ được hiểu đầy đủ cho mô hình tuyến tính**: trong mô hình tuyến tính, hai thành phần là **trực giao** và **không phụ thuộc dữ liệu** (chỉ phụ thuộc kernel/eigenbasis); với mô hình phi tuyến (mạng nơ-ron thật), paper tự nói "the two components need not be orthogonal and the associated subspaces are likely to be data-dependent" - tức cơ chế này **chưa được hiểu đầy đủ** ngoài chế độ tuyến tính.
- **Cơ chế implicit regularization chỉ được hiểu trong "a handful of cases"** (paper tự dùng đúng cụm từ này): chưa có lý thuyết tổng quát cho việc gradient descent chọn nghiệm nào trong không gian các nghiệm nội suy, đặc biệt khi mô hình phi tuyến.
- **Chưa có định nghĩa toán học rõ ràng cho "effective/intrinsic dimension"** trong bối cảnh deep learning thực tế (ảnh, văn bản): paper tự đặt câu hỏi mở "What is the proper mathematical definition of effective dimension?" ở mục Role of dimension.
- **Không biết lượng quá tham số hoá cần thiết trong thực tế**: paper tự nói "we are far from understanding precisely what amount of overparametrization is necessary, even in simple neural network models" - một giới hạn ngay cả cho các mô hình đơn giản nhất mà paper khảo sát.

- **Không chứng minh lợi ích của độ sâu**: đã nêu ở §1, paper chủ động không đưa ra bất kỳ kết quả nào về lợi ích tối ưu hoá/tổng quát hoá của độ sâu, dù công nhận độ sâu "seems likely crucial" cho tính biểu diễn.
- **Neural tangent model (§6.4) kém được hiểu hơn random features**: paper tự nói nguyên nhân kỹ thuật là ma trận đặc trưng của NT **không có các cột độc lập** (Eq. 147), khó phân tích hơn bằng lý thuyết ma trận ngẫu nhiên hiện có.
- **Universality (Định lý 6.2) mới được chứng minh đầy đủ cho ridge regression với loss bình phương**; paper nói việc tổng quát hoá cho các loss khác (ví dụ max-margin, logistic) dựa trên "universality hypothesis" ở một số công trình liên quan, chưa có chứng minh trực tiếp tổng quát trong chính bài này.

7. Liên hệ với các bài khác

- **math content/32 (Benign Overfitting in Linear Regression, 2019)**: mục 4.2 của bài này **trình bày lại chính xác cùng kết quả** (effective rank $r_k(\Sigma)$, self-induced regularization) mà math 32 đã phân tích đầy đủ, kèm chứng minh. Bài này cho khung nhìn tổng quan/tổng hợp, math 32 cho chi tiết chứng minh và điều kiện chính xác (Định lý 4 của BLLT); nên đọc hai bài theo cặp.
- **math content/33 (Gradient Descent Finds Global Minima of DNNs, 2018)**: mục 5.1 của bài này (chế độ tuyến tính, Định lý 5.1/5.4, dùng Jacobian/kernel $K_{m,0} = Df_n(\theta_0)Df_n(\theta_0)^T$) là **cùng dòng lý thuyết NTK** với math 33 (dùng ma trận Gram $K^{(H)}$); paper này thậm chí trích dẫn trực tiếp [DZPS19] (Du et al., "Gradient descent provably optimizes...") là nguồn gốc kỹ thuật cho Định lý 5.1.
- **math content/34 (A Mean Field View of the Landscape of Two-Layer Networks, 2018)**: mục 5.2 của bài này **trích dẫn trực tiếp và tóm tắt lại** PDE distributional dynamics của math 34 (Định lý 5.5 = Theorem 3 của math 34), đặt nó trong tương phản rõ với chế độ tuyến tính của mục 5.1; đây là liên hệ chặt nhất và rõ ràng nhất trong cả batch, vì bài này **thực sự tổng hợp cả hai bài kia** vào một bức tranh chung.
- **math content/22 (Deep Neural Networks as Gaussian Processes, 2017) và math content/24 (Lazy Training, 2018)**: cùng chủ đề giới hạn rộng vô hạn và chế độ "lazy"/kernel mà mục 5.1, 6.4 của bài này thảo luận kỹ (neural tangent kernel, random features kernel như các trường hợp đặc biệt của kernel giới hạn rộng vô hạn).

8. Gợi ý áp dụng

Góc nhìn thống kê phân biệt khả năng khớp dữ liệu huấn luyện với khả năng tổng quát hóa. Các ví dụ tuyến tính và mạng đơn giản làm rõ vai trò của tham số hóa và implicit regularization, nhưng không tự chứng minh hành vi của mọi mạng sâu.

9. Thí nghiệm gợi ý

1. **Tái tạo hiện tượng double descent (Hình 1) ở quy mô nhỏ**: cài đúng thiết lập random features ridge regression của paper nhưng với $d = 20, n = 80$ (thay vì $d = 100, n = 400$ để chạy trên CPU/GPU nhỏ), $f^* = \langle \beta_0, x \rangle$, quét m/n từ 0.2 đến 5, $\lambda = 10^{-3}(m/d)$. Đo: train/test error theo m/n. Kỳ vọng: thấy đỉnh test error gần $m/n = 1$, minh hoạ trực quan hiện tượng double descent.
2. **Minh hoạ implicit regularization với GD trên bài toán quá tham số hoá**: tạo bài toán hồi quy tuyến tính $d = 50 > n = 20$, chạy gradient descent từ $\theta_0 = 0$ đến khi hội tụ (loss ≈ 0), so sánh θ_{GD} với nghiệm minimum-norm tính trực tiếp bằng pseudo-inverse $X^\dagger y$. Đo: $\|\theta_{GD} - X^\dagger y\|_2$. Kỳ vọng: khoảng cách rất nhỏ (gần 0), xác nhận Eq. 14-15 bằng thực nghiệm đơn giản.
3. **So sánh hai khởi tạo trong chế độ tuyến tính (Định lý 5.4)**: cài mạng 2 lớp nhỏ ($d = 20, m \in \{200, 2000\}, n = 50$), huấn luyện với 2 cách khởi tạo như paper mô tả ($\theta_0^{(1)}$ chuẩn và $\theta_0^{(2)}$ "triệt tiêu ban đầu" bằng cách lấy nửa neuron đối xứng với nửa còn lại), đo độ lệch $\|f(\theta_t) - f_{\text{lin}}(\theta_t)\|$ trên tập test theo thời gian huấn luyện cho mỗi m. Đo: độ lệch này giảm theo m nhanh hơn với khởi tạo nào. Kỳ vọng: khởi tạo $\theta_0^{(2)}$ cần m nhỏ hơn để đạt độ lệch nhỏ, xác nhận trực giác của Định lý 5.4.

4. **Kiểm tra tối ưu của nội suy ở SNR cao/thấp (Hệ quả 6.3)**: với cùng random features model nhỏ như thí nghiệm 1, cố định $m/n = 3$ (quá tham số hoá), thử hai bài toán với SNR khác nhau (SNR cao: τ^2 nhỏ so với $\|P_1 f^*\|^2$; SNR thấp: τ^2 lớn), quét λ từ 10^{-4} đến 10, đo test error theo λ cho mỗi SNR. Đo: vị trí λ tối ưu. Kỳ vọng: ở SNR cao, λ tối ưu gần 0 (near-interpolation tối ưu); ở SNR thấp, λ tối ưu lớn hơn 0 rõ rệt, minh hoạ Hệ quả 6.3.
5. **Mô phỏng "simple+spiky" bằng pruning một mạng toy**: huấn luyện một mạng nhỏ nội suy hoàn hảo một bộ dữ liệu có nhiễu, sau đó thử "nén" (ví dụ chiếu trọng số lên không gian con k chiều tương ứng top-k eigenvector của ma trận đặc trưng, với k tăng dần) và đo test error theo k. Đo: test error có giảm rồi ổn định khi k đủ lớn hay không, so với train error. Kỳ vọng: test error giảm nhanh rồi bão hoà ở một k khá nhỏ so với số tham số đầy đủ, minh hoạ trực quan ý tưởng phân rã "đơn giản" (k nhỏ) + "gai nhọn" (phần còn lại) của §2/§8.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / định nghĩa	Nguồn
Nghiệm minimum-norm (implicit reg. của GD)	$\hat{\theta} = \text{argmin}\{\|\theta\|_2 : X\theta = y\} = X^\dagger y$	Eq. 14-15 (paper)
Max-margin ℓ_2 (gradient descent trên loss logistic)	$\hat{\theta} = \text{argmin}\{\|\theta\|_2 : y_i\langle\theta, x_i\rangle \geq 1\}$	Eq. 21, Theorem 3.1 (paper)
Phân rã đơn giản + gai nhọn	$\hat{f}(x) = \hat{f}_0(x) + \Delta(x)$	Eq. 157 (paper)
Nadaraya-Watson với kernel kỳ dị	$\hat{f}(x) = \sum_i y_i \omega_i(x)$, $K(u) = \|u\|_2^{-a}\mathbb{1}[\|u\|_2 \leq 1]$	Eq. 29-30 (paper)
Effective rank	$r_k(\Sigma) = \dfrac{\sum_{i>k}\lambda_i}{\lambda_{k+1}}$	mục 4.2 (paper), = math 32
Điều kiện chế độ tuyến tính (NTK)	$\text{Lip}(Df_n)\|y - f_n(\theta_0)\|_2 < \frac{1}{4}\sigma^2_{\min}(Df_n(\theta_0))$	Eq. 84, Theorem 5.1 (paper)
PDE mean-field (= Theorem 3 của math 34)	$\partial_t \rho_t = \alpha_0 \nabla(\rho_t \nabla\Psi(\cdot;\rho_t))$	Eq. 106, Theorem 5.5 (paper)
Kernel random features / neural tangent	$K_{RF,m} = \frac{1}{m}\sum_i \sigma(\langle w_i, x_1\rangle)\sigma(\langle w_i, x_2\rangle)$; $K_{NT,m} = \frac{1}{m}\sum_i \langle x_1, x_2\rangle \sigma'(\cdot)\sigma'(\cdot)$	Eq. 126-127 (paper)
Double descent, universality	$L_{RF}(\lambda) = L_{NF}(\lambda)\cdot(1 + o_n(1))$	Theorem 6.2 (paper)
Ví dụ tự tính (minh hoạ effective rank, mượn từ math 32)	$\Sigma = \text{diag}(4,1,1,1)$: $r_0(\Sigma) = (1+1+1)/4 = 0.75$	tự tính

11. Kết luận ngắn

Bài khảo sát này đưa ra một khung khái niệm thống nhất để hiểu hai hiện tượng trong học sâu: **tính dễ giải nhờ quá tham số hoá** và **tổng quát hoá nhờ implicit regularization**, bằng cách tổng hợp một loạt kết quả lý thuyết (uniform convergence cổ điển và lý do nó thất bại với benign overfitting, các ví dụ nội suy cụ thể như Nadaraya-Watson và hồi quy tuyến tính effective-rank, chế độ tuyến tính/NTK cho mạng 2 lớp, chế độ mean-field thay thế, và các mô hình tuyến tính hoá random-features/neural-tangent với hiện tượng double descent). Đóng góp quan trọng nhất về mặt tổ chức tri thức là ý tưởng **phân rã "đơn giản + gai nhọn"**, xuất hiện lặp lại trong mọi ví dụ cụ thể, và việc phân biệt rõ **self-induced regularization** (do cấu trúc dữ liệu) với **implicit regularization** (do thuật toán tối ưu). Bài báo nêu phần lớn các kết quả chỉ áp dụng cho mô hình tuyến tính trong không gian đặc trưng cố định, và việc mở rộng sang chế độ phi tuyến thực sự (mean-field, mạng sâu nhiều lớp) vẫn là hướng nghiên cứu mở.

Nguồn gốc: https://arxiv.org/abs/2103.09177

Bài 32

Benign Overfitting in Linear Regression (2019)

Tác giả: Peter L. Bartlett, Philip M. Long, Gábor Lugosi, Alexander Tsigler (UC Berkeley, Google, Pompeu Fabra University).
Venue / arXiv: arXiv:1906.11300 (v3, 29/01/2020; ID gốc tháng 6/2019); paper không nêu venue hội nghị/journal phản biện trong bản text (phần đầu bài chỉ có affiliations, không có dòng ”Proceedings of...” hay ”Journal...”).
Nguồn: https://arxiv.org/abs/1906.11300
Chủ đề: đặc trưng hoá chính xác (điều kiện cần và đủ, qua 2 khái niệm ”effective rank” của ma trận hiệp phương sai) khi nào bộ ước lượng minimum-norm interpolator trong hồi quy tuyến tính overparameterized vẫn có excess risk gần tối ưu, dù khớp hoàn hảo dữ liệu train có nhiễu (”benign overfitting”).
Vị trí trong lộ trình: liên hệ gần nhất với math 31 (”Deep learning: a statistical viewpoint”, cùng batch 28-35, cùng tác giả Bartlett tham gia cả 2 bài, bài 31 khảo sát rộng hơn và trích dẫn kết quả của bài 32 này như 1 ví dụ cụ thể cho hiện tượng benign overfitting). Nên đọc bài 32 (cụ thể, có theorem chặt) trước hoặc song song với bài 31 (survey rộng hơn, đặt bài 32 trong bức tranh lớn hơn gồm cả NTK, random features).

> Đây là ghi chú diễn giải, không phải bản dịch. Ghi chú tập trung vào định nghĩa và kết quả chính; phần chứng minh chỉ được trình bày ở mức ý tưởng, không theo dõi chi tiết từng bổ đề và hằng số. Các phụ lục trong tài liệu bổ sung nằm ngoài phạm vi ghi chú. Các phép tính đánh dấu “tự tính” là ví dụ bổ sung.

1. Bối cảnh

Trong học thống kê cổ điển, việc khớp hoàn hảo dữ liệu có nhiễu thường được xem là dấu hiệu overfitting và dự đoán kém trên dữ liệu mới. Tuy nhiên, một số mạng nơ-ron đạt loss huấn luyện gần 0 vẫn dự đoán tốt trên tập kiểm tra. Bài báo dẫn các thí nghiệm phân loại ảnh trong tài liệu [39] để nêu hiện tượng này; đó không phải thí nghiệm do bài báo tự thực hiện.

Paper chọn bài toán **đơn giản nhất** có thể để nghiên cứu hiện tượng này: **hồi quy tuyến tính (linear regression)** với loss bình phương (quadratic loss), trong không gian tham số có số chiều đủ lớn để khớp hoàn hảo được đảm bảo (overparameterized). Vì có nhiều nghiệm nội suy (hệ thiếu xác định), paper chọn nghiệm tự nhiên nhất: $\hat{\theta}$ có **chuẩn nhỏ nhất (minimum norm)** trong số các nghiệm khớp hoàn hảo dữ liệu train.

Khái niệm cần nắm trước:

- **Ma trận hiệp phương sai (covariance operator)** $\Sigma = \mathbb{E}[xx^\top]$ của biến covariate x (có thể vô hạn chiều, trong không gian Hilbert khả ly H).
- **Excess risk** $R(\hat{\theta})$: chênh lệch kỳ vọng loss giữa ước lượng $\hat{\theta}$ và tham số tối ưu thật θ^* (tham số nhỏ nhất kỳ vọng loss bình phương).
- **Minimum-norm interpolator**: trong số các θ khớp hoàn hảo dữ liệu ($X\theta = y$), chọn θ có chuẩn Euclid/Hilbert nhỏ nhất, tương đương dùng pseudoinverse của ma trận thiết kế.

2. Ý tưởng chính

Kết quả chính (Theorem 4): excess risk của minimum-norm interpolator được chặn trên và dưới (gần khớp nhau, ”nearly matching”) bằng công thức phụ thuộc vào **2 khái niệm effective rank** $r_k(\Sigma)$ và $R_k(\Sigma)$ của ma trận hiệp phương sai. Kết luận cốt lõi: **overparameterization là điều kiện THIẾT YẾU (essential) cho benign overfitting trong thiết lập này**: số ”hướng” trong không gian tham số không quan trọng cho dự đoán (eigenvalue nhỏ của Σ) phải **vượt xa** kích thước mẫu n, để nhiễu label có thể ”ẩn” vào các hướng đó mà không ảnh hưởng dự đoán.

Paper cũng phát hiện vai trò thú vị của **số chiều hữu hạn nhưng lớn**: trong không gian **vô hạn chiều**, benign overfitting chỉ xảy ra với 1 dải hẹp tốc độ suy giảm eigenvalue (Theorem 6 phần 1: chính xác $\mu_k \propto k^{-1}\ln^{-\beta}(k+1)$ với $\beta > 1$); còn trong không gian **hữu hạn chiều lớn hơn** n, benign overfitting xảy ra với dải rộng hơn nhiều các kiểu suy giảm eigenvalue (Theorem 6 phần 2, kể cả eigenvalue gần như hằng số).

3. Phương pháp chi tiết

3.1 Định nghĩa và giả thiết (mục 2)

Bài toán hồi quy tuyến tính: $x \in H$ (Hilbert space, có thể vô hạn chiều), $y \in \mathbb{R}$, $\Sigma = \mathbb{E}[xx^\top]$, $\theta^* = \arg\min_\theta \mathbb{E}(y - x^\top\theta)^2$ (Định nghĩa 1). Giả thiết chính: x viết được $x = V\Lambda^{1/2}z$ với z có thành phần độc lập σ_x^2-subgaussian (tức x là ảnh tuyến tính của 1 vector có thành phần độc lập); nhiễu $y - x^\top\theta^*$ có phương sai chặn dưới σ^2 và σ_y^2-subgaussian; và (giả thiết 5) hầu như chắc chắn, phần dữ liệu X chiếu lên không gian vuông góc với bất kỳ eigenvector nào của Σ vẫn trải đủ chiều n (đảm bảo luôn có thể nội suy hoàn hảo).

Minimum norm estimator (Định nghĩa 2): $\hat\theta = \arg\min\{\|\theta\|^2 : X^\top X\theta = X^\top y\} = (X^\top X)^\dagger X^\top y = X^\top(XX^\top)^{-1}y$ (Eq. 1, dùng pseudoinverse).

Effective rank (Định nghĩa 3): với $\lambda_i = \mu_i(\Sigma)$ là eigenvalue giảm dần của Σ:

$$r_k(\Sigma) = \frac{\sum_{i>k}\lambda_i}{\lambda_{k+1}}, \qquad R_k(\Sigma) = \frac{\left(\sum_{i>k}\lambda_i\right)^2}{\sum_{i>k}\lambda_i^2}.$$

r_k đo "số chiều hiệu dụng" của phần đuôi eigenvalue (từ $k+1$ trở đi) so với eigenvalue thứ $k+1$ (eigenvalue lớn nhất trong đuôi); R_k đo tương tự nhưng theo chuẩn bình phương (nhạy với việc các eigenvalue trong đuôi có đồng đều không). Paper chứng minh (Lemma 5, chỉ nêu phát biểu, chứng minh ở phụ lục không có trong bản PDF chính): $r_k(\Sigma) \geq 1$, $r_k^2(\Sigma) = r_k(\Sigma^2)R_k(\Sigma)$, và $r_k(\Sigma^2) \leq r_k(\Sigma) \leq R_k(\Sigma) \leq r_k^2(\Sigma)$.

3.2 Theorem 4: chặn excess risk chính

Đặt $k^* = \min\{k \geq 0 : r_k(\Sigma) \geq bn\}$ (b là hằng số phổ quát). Nếu $k^* \geq n/c_1$ (không đủ "đuôi" eigenvalue lớn so với n), excess risk kỳ vọng $\geq \sigma^2/c$ (KHÔNG nhỏ, overfitting có hại). Ngược lại, với xác suất $\geq 1-\delta$:

$$R(\hat\theta) \leq c\|\theta^*\|^2\|\Sigma\|\max\left\{\sqrt{\tfrac{r_0(\Sigma)}{n}},\ \tfrac{r_0(\Sigma)}{n},\ \sqrt{\tfrac{\log(1/\delta)}{n}}\right\} + c\log(1/\delta)\sigma_y^2\left(\tfrac{k^*}{n} + \tfrac{n}{R_{k^*}(\Sigma)}\right),$$

và chặn dưới tương ứng $\mathbb{E}R(\hat\theta) \geq \frac{\sigma^2}{c}(\frac{k^*}{n} + \frac{n}{R_{k^*}(\Sigma)})$ (đây chính là Theorem 4 của paper). Diễn giải: số hạng đầu (liên quan $r_0(\Sigma)/n$) là "**bias**" (độ lệch do nhìn θ^* qua mẫu hữu hạn), cần $r_0(\Sigma)$ **nhỏ** so với n (tổng eigenvalue không quá lớn); số hạng sau (liên quan $k^*/n + n/R_{k^*}(\Sigma)$) là "**variance**" (ảnh hưởng của nhiễu label lên tham số ước lượng), cần $R_{k^*}(\Sigma)$ **lớn** so với n.

3.3 Overparameterization và ví dụ eigenvalue (mục 3.1)

Từ Theorem 4: để excess risk nhỏ, cần $r_0(\Sigma)$ nhỏ so với n (tổng eigenvalue nhỏ) **và** $r_{k^*}(\Sigma), R_{k^*}(\Sigma)$ lớn so với n. Kết hợp lại: số eigenvalue không-triệt-tiêu phải **lớn hơn nhiều** n, tổng của chúng phải **nhỏ** so với n, và phải có **nhiều** eigenvalue nhỏ hơn hoặc bằng λ_{k^*} (đây chính là nội dung "overparameterization thiết yếu"). Paper mô tả cụ thể bằng khái niệm "symmetry" $s(\Sigma), S(\Sigma) \in [1/p, 1]$: với Σ hạng p hữu hạn, $r_0(\Sigma) = \text{rank}(\Sigma)\cdot s(\Sigma)$, $R_0(\Sigma) = \text{rank}(\Sigma)\cdot S(\Sigma)$, trong đó s, S gần 1 khi các eigenvalue gần đều nhau, gần $1/p$ khi 1 eigenvalue trội hẳn.

Theorem 6 (2 ví dụ cụ thể về điều kiện eigenvalue): (1) với Σ vô hạn chiều cố định, $\mu_k(\Sigma) = k^{-\alpha}\ln^{-\beta}(k+1)$: benign overfitting (định nghĩa: $r_0(\Sigma_n)/n \to 0$, $k_n^*/n \to 0$, $n/R_{k_n^*}(\Sigma_n) \to 0$ khi $n \to \infty$) xảy ra **khi và chỉ khi** $\alpha = 1$ và $\beta > 1$ - một dải **rất hẹp** tốc độ suy giảm. (2) với Σ_n hữu hạn chiều p_n, eigenvalue dạng hằng số cộng nhiễu isotropic nhỏ ($\mu_k = \gamma_k + n$ cho $k \leq p_n$, $\gamma_k = \Theta(e^{-k/\tau})$): benign overfitting xảy ra khi và chỉ

khi $p_n = \omega(n)$ (chiều tăng nhanh hơn n) và $n/p_n = ne^{-o(n)}$ (thành phần isotropic không quá nhỏ) - một dải **rộng hơn nhiều**.

3.4 Liên hệ với mạng nơ-ron sâu (mục 4)

Paper tự thảo luận: trong chế độ **Neural Tangent Kernel (NTK)** (mạng rất rộng, gần tuyến tính theo tham số), eigenvalue của ma trận hiệp phương sai NTK **có đuôi dài (heavy tail)**, và số chiều rất lớn nhưng hữu hạn - đúng kiểu điều kiện cần cho benign overfitting theo Theorem 4/6. NHƯNG paper tự nói rõ: **giả thiết của Theorem 4 (các thành phần z độc lập theo biến đổi tuyến tính) KHÔNG thoả mãn cho NTK thực tế**, nên kết quả của bài này **không áp dụng trực tiếp** cho trường hợp đó; paper gọi đây là "thiết lập khá phi thực tế" (somewhat unrealistic). Đây là 1 liên hệ quan trọng nhưng paper tự giới hạn, không khẳng định đã chứng minh cho mạng nơ-ron.

3.5 Ý tưởng chứng minh (mục 5, chỉ lướt)

Bước đầu: phân tách excess risk thành 2 số hạng (Lemma 7): $R(\hat{\theta}) \leq 2\theta^{*\top} B\theta^* + c\sigma^2 \log(1/\delta)\mathrm{tr}(C)$, với B, C là các ma trận xây từ X, Σ qua pseudoinverse. Số hạng $\theta^{*\top} B\theta^*$ là "bias" (méo do nhìn θ^* qua mẫu hữu hạn), số hạng $\sigma^2\mathrm{tr}(C)$ là "variance" (ảnh hưởng nhiễu). Phần khó nhất là chặn $\mathrm{tr}(C)$: biểu diễn qua tổng các outer-product của vector subgaussian độc lập đơn vị z_i (Lemma 8), rồi dùng các bất đẳng thức concentration (Lemma 9-12, dựa trên lý thuyết ma trận ngẫu nhiên) để chỉ ra eigenvalue nhỏ nhất/lớn nhất của các ma trận tổng A_k, A_{-i} tập trung quanh giá trị lý thuyết, cuối cùng suy ra $\mathrm{tr}(C)$ nằm trong khoảng hằng số lần $k^*/n + n/R_{k^*}(\Sigma)$ - khớp đúng công thức Theorem 4.

4. Thí nghiệm và kết quả chính

Đây là paper **lý thuyết hoàn toàn (lý thuyết hồi quy tuyến tính), KHÔNG có thí nghiệm số/benchmark nào** do tác giả tự chạy; không có bảng số liệu, không có hình minh hoạ thực nghiệm trong bài. "Kết quả chính" là:

1. **Theorem 4**: chặn trên/dưới gần khớp cho excess risk của minimum-norm interpolator, theo $r_0(\Sigma)$ (bias) và $k^*/n + n/R_{k^*}(\Sigma)$ (variance) - đã trình bày ở mục 3.2.
2. **Theorem 6**: 2 ví dụ cụ thể (vô hạn chiều vs hữu hạn chiều lớn) minh hoạ điều kiện eigenvalue cần và đủ cho benign overfitting - mục 3.3.
3. **Nhận xét định lượng ở mục 4**: liên hệ (không chứng minh đầy đủ) giữa điều kiện của Theorem 4/6 và chế độ NTK của mạng nơ-ron rộng.

Không có "benchmark" theo nghĩa thông thường (không dataset, không mô hình huấn luyện thật); các "ví dụ" trong Theorem 6 là các cấu hình eigenvalue toán học cụ thể, không phải dữ liệu thực.

5. Phân tích / ablation

Không có ablation thực nghiệm; "phân tích điều kiện" chính là nội dung mục 3.3 của note này (2 phần Theorem 6 đóng vai trò như 2 "trường hợp đặc biệt" đối lập):

- **Vô hạn chiều, eigenvalue cố định** (Theorem 6 phần 1): dải tốc độ suy giảm cho phép benign overfitting **rất hẹp** (chính xác $\alpha = 1, \beta > 1$) - paper gọi đây là hiện tượng "bất thường" (unusual) trong trường hợp vô hạn chiều.
- **Hữu hạn chiều lớn + nhiễu isotropic nhỏ** (Theorem 6 phần 2): dải điều kiện cho phép benign overfitting **rộng hơn nhiều** (eigenvalue hằng số hoặc suy giảm chậm bất kỳ đều được, miễn số chiều tăng nhanh hơn n) - paper gọi đây là trường hợp "**tổng quát hơn**" (more generic). Với $p_n = \Omega(n)$ và $n/p_n = ne^{-o(n)}$, paper cho bound cụ thể (Theorem 6 phần 2 cuối): $R(\hat{\theta}) = O(\frac{n}{p_n} + \frac{\ln(n/(n/p_n))}{n} + \max\{\frac{1}{n}, \frac{n}{p_n}\})$.
- **Trường hợp cực đoan $p = n$, $\Sigma = I$ (ngay sau Lemma 7)**: paper tự chỉ ra bằng lý thuyết ma trận ngẫu nhiên rằng excess risk **tối thiểu là 1 hằng số** (không nhỏ dần theo n) - minh hoạ trực quan rằng KHÔNG overparameterized (không có đuôi eigenvalue lớn so với n) thì nội suy chắc chắn có hại.

- **Căng kéo giữa 2 điều kiện** (mục 3.3 cuối, bài báo nêu): cần eigenvalue suy giảm **đủ chậm** để $k/n + n/R_k$ nhỏ, nhưng cũng cần **đủ nhanh** (summable) để $r_0(\Sigma)/n$ nhỏ. Paper nêu rõ 2 cách giải quyết căng kéo này tương ứng chính là 2 phần của Theorem 6.

6. Giới hạn

Theo bài báo nêu (mục 6 Conclusions và rải rác trong bài):

- Giả thiết $\mathbb{E}[y|x]$ là hàm **tuyến tính** của x (well-specified); paper tự nói "quan trọng để hiểu liệu kết quả còn đúng trong trường hợp misspecified hay không" - **chưa giải quyết** trong bài này.
- Giả thiết covariate x là **ảnh tuyến tính của vector có thành phần độc lập** (Assumption 2); paper tự nói giả thiết này **loại trừ** một số trường hợp quan trọng, ví dụ reproducing kernel Hilbert space vô hạn chiều với kernel liên tục trên không gian hữu hạn chiều.
- Chỉ xét loss **bình phương** và ước lượng **minimum-norm interpolator** cụ thể; mở rộng sang loss khác hoặc estimator khác được paper liệt là hướng tương lai, **chưa làm** trong bài.
- Như đã nêu ở mục 3.4: kết quả **không áp dụng trực tiếp** cho mạng nơ-ron/NTK thực tế vì giả thiết về tính độc lập thành phần covariate không thoả mãn - đây là giới hạn quan trọng nhất bài báo nêu khi nói về động lực ban đầu của bài (hiểu benign overfitting trong Deep Learning).
- Hướng mở rộng sang **lớp hàm tham số hoá phi tuyến (như mạng nơ-ron)** được paper gọi là "hướng tương lai thú vị nhất" (most interesting future direction) - nghĩa là bài này **chưa** trả lời câu hỏi gốc cho Deep Learning, chỉ trả lời cho trường hợp tuyến tính đơn giản nhất.

7. Liên hệ với các bài khác

- **Bài 31, Deep Learning: A Statistical Viewpoint**: khảo sát rộng hơn về tổng quát hóa và dùng kết quả hồi quy tuyến tính này làm một ví dụ về benign overfitting.
- **"llm content/03_2020_scaling_laws_kaplan.md"**: liên hệ qua chủ đề overparameterization/quy mô hoá mô hình; bài 32 cho góc nhìn lý thuyết chặt (hồi quy tuyến tính) về lý do "mô hình nhiều tham số hơn dữ liệu" không nhất thiết overfit có hại, bổ sung cho góc nhìn thực nghiệm quy mô lớn của Kaplan et al.
- **"pruning content/11_2018_rethinking_value_of_pruning.md"**: bài pruning này thảo luận các mạng overparameterized và cấu trúc dư thừa tham số; liên hệ gián tiếp với ý tưởng "nhiều hướng tham số không quan trọng" (eigenvalue nhỏ) của bài 32 - những hướng dư thừa đó, theo lý thuyết benign overfitting, chính là nơi "ẩn" nhiễu mà không hại dự đoán, cũng là ứng viên tự nhiên để cắt giảm khi pruning.

8. Gợi ý áp dụng

Benign overfitting cho thấy nội suy dữ liệu nhiễu vẫn có thể đi cùng rủi ro dự đoán thấp dưới những điều kiện phổ thích hợp. Các định lý của bài dành cho hồi quy tuyến tính; cần thận trọng khi dùng chúng để suy luận về mạng phi tuyến.

9. Thí nghiệm gợi ý

1. **Tái tạo Theorem 4 trên dữ liệu tổng hợp nhỏ**: sinh $x \sim \mathcal{N}(0, \Sigma)$ với $\Sigma = \text{diag}(\lambda_1, \ldots, \lambda_p)$ (ví dụ $p = 200$, eigenvalue suy giảm theo $\lambda_i = i^{-1}$, giống Theorem 6 phần 1 với $\alpha = 1$), $y = x^\top \theta^* + \epsilon$, n nhỏ (ví dụ $n = 50$). Tính $\hat{\theta}$ bằng pseudoinverse (`numpy.linalg.pinv`), đo excess risk thực nghiệm trên tập test lớn, so sánh với chặn Theorem 4 (tính $r_0(\Sigma), k^*, R_{k^*}(\Sigma)$ từ Σ đã biết). Chạy được trên CPU trong vài giây.
2. **Quét tốc độ suy giảm eigenvalue** α: lặp thí nghiệm 1 với $\lambda_i = i^{-\alpha}$, $\alpha \in \{0.5, 1, 1.5, 2\}$ (giữ p, n cố định, p hữu hạn lớn hơn n nhiều). Đo excess risk theo α. Kỳ vọng: vùng α gần 1 (và p đủ lớn hữu hạn) cho excess risk nhỏ nhất, minh hoạ "dải rộng hơn trong không gian hữu hạn chiều" (Theorem 6 phần 2) so với yêu cầu chặt $\alpha = 1$ đúng của trường hợp vô hạn chiều.

3. **Trường hợp cực đoan** $p = n$: lặp thí nghiệm 1 với $p = n$ (không overparameterized), $\Sigma = I$. Đo excess risk thực nghiệm nhiều lần lặp (nhiều seed). Kỳ vọng: excess risk **không giảm** khi tăng n (giữ $p = n$), xác nhận đúng nhận xét ở mục 5 (ngay sau Lemma 7) rằng không overparameterized thì nội suy có hại.
4. **Đo effective rank của activation 1 mạng nơ-ron nhỏ**: huấn luyện 1 MLP nhỏ (2-3 lớp) trên MNIST/toy dataset, trích activation của 1 lớp ẩn trên tập validation, tính ma trận hiệp phương sai thực nghiệm của activation đó, tính r_0, R_0 theo công thức Định nghĩa 3. Đo xem r_0, R_0 lớn hay nhỏ so với n (số mẫu). Đây KHÔNG kiểm chứng Theorem 4 (vì giả thiết không thoả mãn cho mạng nơ-ron, đúng như mục 3.4), nhưng minh hoạ cách tính effective rank thực tế và kết nối (có kiểm soát, không suy diễn quá đà) với ý tưởng ở mục 8.
5. **So sánh minimum-norm vs ridge regression**: lặp thí nghiệm 1 nhưng thêm 1 ước lượng ridge regression (có regularization $\lambda > 0$) để so sánh excess risk với minimum-norm interpolator ($\lambda = 0$). Đo excess risk theo λ quét từ 0 tới giá trị lớn. Kỳ vọng: với Σ trong "vùng benign" (theo Theorem 6), minimum-norm ($\lambda = 0$) có excess risk gần với ridge tối ưu, minh hoạ trực quan hiện tượng "benign overfitting" (nội suy không cần regularization vẫn tốt).

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức	Ý nghĩa
Minimum-norm estimator	$\hat{\theta} = X^\top (XX^\top)^{-1} y$ (Eq. 1)	Nghiệm nội suy có chuẩn nhỏ nhất
Effective rank r_k	$r_k(\Sigma) = \dfrac{\sum_{i>k} \lambda_i}{\lambda_{k+1}}$	Số chiều hiệu dụng của đuôi eigenvalue
Effective rank R_k	$R_k(\Sigma) = \dfrac{(\sum_{i>k} \lambda_i)^2}{\sum_{i>k} \lambda_i^2}$	Tương tự r_k nhưng theo chuẩn bình phương
Ngưỡng k^*	$k^* = \min\{k \geq 0 : r_k(\Sigma) \geq bn\}$	Điểm cắt giữa eigenvalue "lớn" và "nhỏ"
Theorem 4 (chặn trên)	$R(\hat{\theta}) \lesssim \|\theta^*\|^2 \|\Sigma\| \sqrt{r_0(\Sigma)/n} + \sigma_y^2 \left(\frac{k^*}{n} + \frac{n}{R_{k^*}(\Sigma)}\right)$	Bias (số hạng đầu) + variance (số hạng sau)
Benign (định nghĩa, mục 3.3)	$\lim_n r_0(\Sigma_n)/n = \lim_n k_n^*/n = \lim_n n/R_{k_n^*}(\Sigma_n) = 0$	Điều kiện dãy Σ_n cho excess risk $\to 0$
Theorem 6(1)	$\mu_k = k^{-\alpha} \ln^{-\beta}(k+1)$ benign $\iff \alpha = 1, \beta > 1$	Dải hẹp cho trường hợp vô hạn chiều

Ví dụ tính tay (tự tính): cho $\Sigma = \mathrm{diag}(4, 1, 1, 1)$ (hạng $p = 4$, eigenvalue giảm dần $\lambda_1 = 4, \lambda_2 = \lambda_3 = \lambda_4 = 1$). Tính $r_0(\Sigma)$ và $R_0(\Sigma)$ theo Định nghĩa 3 với $k = 0$:

$$r_0(\Sigma) = \frac{\sum_{i>0} \lambda_i}{\lambda_1} = \frac{4+1+1+1}{4} = \frac{7}{4} = 1.75.$$

$$R_0(\Sigma) = \frac{\left(\sum_{i>0} \lambda_i\right)^2}{\sum_{i>0} \lambda_i^2} = \frac{7^2}{4^2+1^2+1^2+1^2} = \frac{49}{19} \approx 2.579.$$

So sánh với $\Sigma' = I_4$ (4 eigenvalue bằng nhau $= 1$): $r_0(\Sigma') = \frac{4}{1} = 4 = p$, $R_0(\Sigma') = \frac{16}{4} = 4 = p$ (khớp đúng nhận xét của paper: khi mọi eigenvalue bằng nhau, $r_0 = R_0 = \mathrm{rank}(\Sigma)$). Ví dụ Σ ban đầu có 1 eigenvalue trội ($\lambda_1 = 4$) làm r_0, R_0 **nhỏ hơn hẳn** $p = 4$, minh hoạ trực quan: hiệp phương sai "mất cân bằng" (1 hướng chiếm phần lớn phương sai) có effective rank thấp hơn số chiều thật, tức ít "chỗ ẩn nhiễu" hơn theo tinh thần Theorem 4.

11. Kết luận ngắn

Bằng cách phân tích chặt bài toán hồi quy tuyến tính overparameterized, bài báo đưa ra đặc trưng hoá (điều kiện cần và đủ, qua 2 khái niệm effective rank r_k, R_k của ma trận hiệp phương sai) cho khi nào minimum-norm interpolator vẫn có excess risk gần tối ưu dù khớp hoàn hảo dữ liệu nhiễu. Kết luận cốt lõi: overparameterization (nhiều hướng tham số ”không quan trọng”, eigenvalue nhỏ, vượt xa kích thước mẫu) là điều kiện thiết yếu, và không gian hữu hạn chiều lớn cho phép benign overfitting với dải điều kiện eigenvalue rộng hơn nhiều so với không gian vô hạn chiều. Đây là 1 trong các kết quả lý thuyết nền tảng và chặt chẽ nhất về benign overfitting, nhưng bài báo lưu ý giả thiết không áp dụng trực tiếp cho mạng nơ-ron/NTK thực tế, nên việc nối kết đầy đủ với hiện tượng gốc trong Deep Learning còn là câu hỏi mở.

Nguồn gốc: https://arxiv.org/abs/1906.11300

Bài 33

Gradient Descent Finds Global Minima of Deep Neural Networks (2018)

Tác giả: Simon S. Du, Jason D. Lee, Haochuan Li, Liwei Wang, Xiyu Zhai
Venue / arXiv: arXiv:1811.03804 (v4, 28/05/2019); ICML 2019 (Proceedings of the 36th International Conference on Machine Learning, PMLR 97, 2019, theo chính văn bản paper tự ghi ở trang đầu)
Nguồn: https://arxiv.org/abs/1811.03804
Chủ đề: Chứng minh lý thuyết rằng gradient descent khởi tạo ngẫu nhiên đạt loss huấn luyện bằng 0 với tốc độ tuyến tính trên mạng nơ-ron sâu quá tham số hoá (over-parameterized), cho ba kiến trúc: fully-connected sâu, ResNet, và convolutional ResNet.
Vị trí trong lộ trình: Đọc sau math 22 (Deep NN như Gaussian Process) và math 24 (Lazy training) vì cùng dòng lý thuyết NTK/Gram-matrix cho mạng quá tham số hoá; là bài thứ 7 trong batch math 28-35, liền trước math 34 (Mean Field View, một hướng lý thuyết khác cho cùng câu hỏi).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (bản arXiv); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc.

1. Bối cảnh

Bài báo bắt đầu từ hai hiện tượng trong huấn luyện mạng nơ-ron sâu:

1. Các phương pháp bậc một khởi tạo ngẫu nhiên (random initialization) như gradient descent (GD) đạt loss huấn luyện bằng 0, **thậm chí khi nhãn là ngẫu nhiên** (dẫn chứng từ Zhang et al. 2016/2017, ”Understanding deep learning requires rethinking generalization”), trong khi bài toán tối ưu là **không lồi (non-convex)**.
2. ”Mạng sâu hơn thì khó huấn luyện hơn” - vấn đề mà kiến trúc **ResNet** (residual network, He et al. 2016) giải quyết thực nghiệm bằng kết nối tắt (skip connection), cho phép huấn luyện mạng có số lớp lớn hơn một bậc độ lớn, nhưng (tính đến thời điểm paper này công bố) **chưa có lý giải lý thuyết đầy đủ** cho lợi ích của residual connection với hàm kích hoạt phi tuyến.

Thuật ngữ cần làm rõ: **over-parameterization** (quá tham số hoá) nghĩa là mạng có nhiều tham số hơn số điểm dữ liệu huấn luyện cần fit; **Gram matrix** (ma trận Gram) trong ngữ cảnh này là ma trận $n \times n$ (n = số điểm dữ liệu) đo ”độ tương quan” giữa gradient của các điểm dữ liệu theo tham số mô hình; **linear convergence rate** (tốc độ hội tụ tuyến tính) nghĩa là loss giảm theo cấp số nhân (hình học) qua các bước lặp, dạng $L(k) \leq (1-c)^k L(0)$.

Paper đặt bài toán cụ thể: có n điểm dữ liệu (x_i, y_i), mạng có H lớp, độ rộng m mỗi lớp, hàm kích hoạt σ thoả **Condition 3.1** (Lipschitz và smooth, ví dụ hàm softplus $\sigma(z) = \log(1+e^z)$) và **Condition 3.2** (giải tích, không là đa thức). Mục tiêu: chứng minh với m đủ lớn, GD khởi tạo ngẫu nhiên hội tụ về loss $= 0$ với tốc độ tuyến tính, cho 3 kiến trúc: (a) mạng fully-connected sâu, (b) ResNet, (c) convolutional ResNet.

2. Ý tưởng chính

Ý tưởng cốt lõi (kế thừa từ Du et al. 2018b cho mạng 2 lớp, mở rộng cho mạng sâu) là theo dõi động lực học của **sai số dự đoán** $y - u(k)$, với $u_i(k) = f(\theta(k), x_i)$ là dự đoán ở bước k. Paper chứng minh động lực này tuân theo:

$$y - u(k+1) = (I - \eta G(k))(y - u(k)),$$

với $G(k)$ là ma trận Gram tại bước k (xây dựng từ gradient của u theo trọng số lớp cuối). Nếu m đủ lớn, $G(k)$ **gần như không đổi** trong suốt quá trình huấn luyện và gần một ma trận cố định $K^{(H)}$ (chỉ phụ thuộc dữ liệu đầu vào và kiến trúc, không phụ thuộc θ). Khi đó động lực xấp xỉ tuyến tính $y-u(k+1) \approx (I-\eta K^{(H)})(y-u(k))$, và theo lý thuyết power method chuẩn, nếu $K^{(H)}$ xác định dương ($\lambda_{\min}(K^{(H)}) > 0$), sai số giảm theo cấp số nhân với tốc độ quyết định bởi $\lambda_{\min}(K^{(H)})$.

Điểm mới của paper (so với Du et al. 2018b chỉ làm cho mạng 2 lớp) là chứng minh **tính ổn định của ma trận Gram xuyên suốt H lớp**, đòi hỏi phân tích cách "nhiễu" (perturbation) từ khởi tạo ngẫu nhiên và từ quá trình huấn luyện lan truyền qua nhiều lớp phi tuyến. Đây là phần khó kỹ thuật nhất, dùng công cụ từ **giải tích Malliavin** (Malliavin calculus, một nhánh giải tích ngẫu nhiên).

Kết quả chính, phát biểu trực giác: với mạng **fully-connected** sâu, độ rộng cần thiết m tăng **theo cấp số mũ với số lớp** H ($2^{O(H)}$); với **ResNet**, nhờ hệ số scaling residual $c_{\text{res}}/(\sqrt{H} \cdot \sqrt{m})$, độ rộng cần thiết chỉ tăng **đa thức theo** H ($\text{poly}(n, H)$). Đây chính là lời giải lý thuyết cho hiện tượng bí ẩn thứ hai: residual connection giúp ổn định sự lan truyền nhiễu qua các lớp, nên không cần độ rộng tăng theo cấp số mũ với độ sâu.

3. Phương pháp chi tiết

3.1. Ba kiến trúc được xét (mục 3.3 của paper)

Fully-connected: với $x^{(0)} = x \in \mathbb{R}^d$,

$$x^{(h)} = \sigma\left(\sqrt{\frac{c_\sigma}{m}}\, W^{(h)} x^{(h-1)}\right), \quad f(x,\theta) = a^\top x^{(H)}, \tag{2, paper}$$

với $c_\sigma = \left(\mathbb{E}_{x\sim N(0,1)}[\sigma(x)^2]\right)^{-1}$ là hệ số chuẩn hoá.

ResNet (paper tự định nghĩa đây là kiến trúc có skip-connection ở **mọi lớp**, khác ResNet chuẩn vốn skip mỗi 2 lớp, nhưng paper nói rõ phân tích tổng quát hoá được cho skip mỗi $k \geq 2$ lớp):

$$x^{(h)} = x^{(h-1)} + \frac{c_{\text{res}}}{\sqrt{H}\sqrt{m}}\, \sigma\left(W^{(h)} x^{(h-1)}\right), \quad 2 \leq h \leq H, \tag{3, paper}$$

với $0 < c_{\text{res}} < 1$ là hằng số nhỏ. **Hệ số scaling $1/\sqrt{H}$ chính là chìa khoá kỹ thuật** giúp độ rộng cần thiết không nổ theo cấp số mũ với H.

Convolutional ResNet: tương tự ResNet nhưng $W^{(h)}$ nhân với một phép trích patch $\phi_h(\cdot)$ (ví dụ 3×3 patch với $q = 3$), áp dụng cho ảnh có p pixel.

Tất cả tham số khởi tạo $W_{ij}^{(h)} \sim N(0,1)$ và $a \sim N(0,1)$ độc lập; **huấn luyện toàn bộ các lớp** (không đóng băng lớp nào) bằng gradient descent với step size cố định η.

3.2. Ma trận Gram $K^{(H)}$ (Định nghĩa 5.1, 6.1, 7.1)

Với mạng fully-connected, $K^{(H)}$ định nghĩa quy nạp: $K_{ij}^{(0)} = \langle x_i, x_j \rangle$, rồi với $h = 1, \ldots, H-1$:

$$A_{ij}^{(h)} = \begin{pmatrix} K_{ii}^{(h-1)} & K_{ij}^{(h-1)} \\ K_{ji}^{(h-1)} & K_{jj}^{(h-1)} \end{pmatrix}, \qquad K_{ij}^{(h)} = c_\sigma\, \mathbb{E}_{(u,v)^\top \sim N(0, A_{ij}^{(h)})}[\sigma(u)\sigma(v)], \tag{7, paper}$$

và lớp cuối dùng đạo hàm σ' thay σ. Đây chính là **ma trận hiệp phương sai của Gaussian Process tương đương** với mạng ở độ rộng vô hạn (kernel NNGP, Neural Network Gaussian Process), paper trích dẫn rõ liên

hệ này với các công trình Gaussian Process cho mạng vô hạn rộng (Lee et al. 2017, Matthews et al. 2018). Với ResNet, $K^{(H)}$ có thêm các số hạng $b_i^{(h)}$ (trung bình kích hoạt) do cấu trúc skip-connection.

Ví dụ tự tính nhỏ để minh hoạ bước đệ quy: giả sử $n = 2$ điểm dữ liệu chuẩn hoá $\|x_i\|_2 = 1$ với $\langle x_1, x_2\rangle = 0.5$ (góc giữa hai vector khoảng $60°$), $c_\sigma = 1$ và σ là hàm tuyến tính giả định $\sigma(z) = z$ chỉ để minh hoạ cơ chế (paper dùng σ phi tuyến thật, đây chỉ là ví dụ tính tay đơn giản hoá). Khi đó $K^{(0)} = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix}$, và với σ tuyến tính, $\mathbb{E}[\sigma(u)\sigma(v)] = \mathrm{Cov}(u, v) = K_{ij}^{(0)}$, nên $K^{(1)} = K^{(0)}$ không đổi qua lớp (tự tính, chỉ minh hoạ trực giác đệ quy, không phải kết quả của paper vì paper yêu cầu σ không phải đa thức).

3.3. Kỹ thuật chứng minh (mục 4, ”Technique Overview”)

Chứng minh gồm hai bước chính (paper trình bày tường minh ở mục 4):

Bước 1 (phân tích khởi tạo): chứng minh $G^{(H)}(0)$ (ma trận Gram tại $k = 0$) gần $K^{(H)}$. Với mạng fully-connected, nhiễu lan truyền theo công thức

$$\|G^{(H)}(0) - K^{(H)}\|_2 \lesssim 2^{O(H)}\epsilon_1, \qquad \text{(4, paper)}$$

tức **khuếch đại theo cấp số mũ với** H. Với ResNet, nhờ hệ số $1/\sqrt{H}$:

$$\|G^{(H)}(0) - K^{(H)}\|_2 \lesssim \left(1 + \frac{1}{H}\right)^H \epsilon_1 = O(\epsilon_1), \qquad \text{(5, paper)}$$

tức **bị chặn đều theo** H (vì $(1 + 1/H)^H \to e$ khi $H \to \infty$). Đây chính là phần giải thích định lượng cho lý do ResNet ”ổn định” hơn.

Bước 2 (phân tích nhiễu trong huấn luyện): chứng minh $G^{(H)}(k)$ gần $G^{(H)}(0)$ với mọi k, bằng cách chặn $\frac{1}{\sqrt{m}}\|W^{(h)}(k) - W^{(h)}(0)\|_F$ (độ lệch trung bình Frobenius-norm của trọng số so với khởi tạo) nhỏ với mọi k. Cơ chế trực giác tương tự Bước 1.

Proof Sketch (Appendix A, đọc lướt): chứng minh chính dùng quy nạp trên điều kiện $\|y - u(k)\|_2^2 \le (1 - \eta\lambda_0/2)^k\|y - u(0)\|_2^2$ (với $\lambda_0 = \lambda_{\min}(K^{(H)})$), tách $u(k+1) - u(k)$ bằng khai triển Taylor thành số hạng chính $I_1(k)$ (tỷ lệ η, làm giảm loss) và số hạng nhiễu $I_2(k)$ (tỷ lệ η^2, nhỏ). Đây là kỹ thuật chuẩn trong các chứng minh hội tụ tuyến tính kiểu NTK (Neural Tangent Kernel).

4. Thí nghiệm và kết quả chính

Paper này không có thí nghiệm thực nghiệm (benchmark) nào - đây là một bài chứng minh toán học thuần tuý, không có mục ”Experiments” hay bất kỳ bảng số liệu đo trên dữ liệu thực/tổng hợp. Kết quả chính là 3 định lý hội tụ:

Định lý 5.1 (Fully-connected): với $\|x_i\|_2 = 1$, $|y_i| = O(1)$, nếu

$$m = \Omega\left(2^{O(H)} \max\left\{\frac{n}{\lambda_{\min}(K^{(H)})^4\delta}, \frac{n^2\log(Hn/\delta)}{\lambda_{\min}(K^{(H)})^2}, n^4\right\}\right)$$

và step size $\eta = O\left(\dfrac{\lambda_{\min}(K^{(H)})}{n^2 2^{O(H)}}\right)$, thì với xác suất $\ge 1 - \delta$:

$$L(\theta(k)) \le \left(1 - \frac{\eta\lambda_{\min}(K^{(H)})}{2}\right)^k L(\theta(0)). \qquad \text{(Theorem 5.1, paper)}$$

Định lý 6.1 (ResNet): cùng dạng kết luận, nhưng độ rộng cần thiết

$$m = \Omega\left(\max\left\{\frac{n^2}{\lambda_{\min}(K^{(H)})^4}, \frac{n^4}{H^6\lambda_{\min}(K^{(H)})^2}, \frac{n^2\log(Hn/\delta)}{\delta\lambda_{\min}(K^{(H)})^2}\right\}\right), \quad \eta = O\left(\frac{\lambda_{\min}(K^{(H)})H^2}{n^2}\right).$$

(Theorem 6.1, paper)

Khác biệt quan trọng nhất giữa hai định lý: Định lý 5.1 có số hạng $2^{O(H)}$ (mũ theo số lớp), Định lý 6.1 **hoàn toàn đa thức** theo n và H (không có số hạng mũ). Đây là bằng chứng lý thuyết định lượng cho lợi ích của kiến trúc ResNet.

Định lý 7.1 (Convolutional ResNet): tương tự Định lý 6.1 nhưng thêm số hạng poly(p) (p = số patch/pixel) trong yêu cầu độ rộng và nghịch đảo của nó trong step size; về bản chất mở rộng kỹ thuật ResNet sang kiến trúc tích chập.

So sánh với công trình đồng thời (concurrent work) Allen-Zhu et al. (2018c): paper này tự so sánh (phần Related Work) rằng với ResNet, yêu cầu độ rộng của Allen-Zhu et al. là $m \gtrsim n^{30}H^{30}\log^2(1/\epsilon)$ và độ phức tạp lặp $T \gtrsim n^6H^2\log(1/\epsilon)$, trong khi của paper này là $m \gtrsim n^4H^2$ và $T \gtrsim n^2\log(1/\epsilon)$ - **không phụ thuộc** ϵ vì định lý của paper này đảm bảo hội tụ về đúng global minimum (loss $= 0$), không chỉ đạt sai số ϵ. Bài báo lưu ý hai công trình "không thể so sánh công bằng hoàn toàn" (not directly comparable) vì giả định dữ liệu khác nhau.

5. Phân tích / ablation

Vì đây là bài lý thuyết không có mã nguồn/thí nghiệm, "phân tích" ở đây là phân tích điều kiện và vai trò của từng thành phần trong định lý, theo chính paper trình bày:

Vai trò của từng số hạng trong yêu cầu độ rộng m (paper tự giải thích, mục ngay sau Theorem 5.1 và 6.1):

- Số hạng đầu dùng để chứng minh ma trận Gram **ổn định trong quá trình huấn luyện**.
- Số hạng thứ hai dùng để đảm bảo đầu ra mỗi lớp được **chuẩn hoá xấp xỉ đúng tại khởi tạo**.
- Số hạng thứ ba dùng để chặn **nhiễu của ma trận Gram tại khởi tạo**.

Điều kiện cần của hàm kích hoạt: Condition 3.2 (giải tích, không là đa thức) được paper giải thích là cần để đảm bảo $K^{(H)}$ **xác định dương chặt** (Proposition F.1, F.2, chỉ lướt qua); paper không cho ví dụ phản chứng cụ thể nếu vi phạm điều kiện này trong phần chính, chỉ nói "cần điều kiện này để đảm bảo positive-definiteness".

So sánh định lượng ResNet vs fully-connected (bài báo nêu, mục 6): "the primary reason that we do not have any exponential factor here is that the skip connection block makes the overall architecture more stable in both the initialization phase and the training phase" - đây là câu paper tự kết luận, không phải diễn giải của ghi chú.

Hướng cải thiện mở (bài báo nêu ở mục 8, Conclusion): paper liệt kê 4 hướng nghiên cứu tiếp theo mà bản thân nó chưa giải quyết:

1. Paper chỉ chứng minh loss huấn luyện (training loss), **không đề cập loss kiểm định (test loss / generalization)**; paper tự nói các công trình hiện có (gồm công trình của chính nhóm tác giả) chỉ cho thấy GD hoạt động trong cùng điều kiện với kernel method / random feature method.
2. Độ rộng m là đa thức nhưng **vẫn rất lớn** so với mạng thực tế (bài báo lưu ý: "mạng thực tế có số tham số là hằng số nhân với n, không phải độ rộng").
3. Phân tích hiện tại chỉ cho **gradient descent toàn batch**, chưa cho stochastic gradient descent (SGD), tuy paper tin rằng có thể mở rộng được.

4. Tốc độ hội tụ có thể cải thiện nếu tính đến đóng góp của ma trận Gram ở **mọi lớp** (hiện paper chỉ dùng $G^{(H)}$, ma trận Gram của lớp cuối, để đơn giản hoá, đánh đổi với tốc độ hội tụ kém hơn mức tối ưu khả dĩ).

6. Giới hạn

Các giới hạn dưới đây đều được paper **tự nêu rõ** (không phải suy luận của ghi chú, trừ khi ghi chú thích):

- **Không chứng minh generalization**: như đã nói ở §5, định lý chỉ bảo đảm loss huấn luyện $\to 0$, không nói gì về hiệu năng trên dữ liệu chưa thấy (test set). Đây là giới hạn quan trọng nhất về phạm vi của kết quả.
- **Độ rộng m cần thiết rất lớn trong thực tế**: dù là đa thức (không mũ) với ResNet, số mũ cụ thể ($n^4, n^4/H^6, \ldots$) vẫn lớn hơn rất nhiều so với độ rộng dùng trong thực hành (bài báo lưu ý ở mục Conclusion điểm 2).
- **Chỉ xét gradient descent toàn batch (full-batch GD), không phải SGD**: thực tế hầu hết huấn luyện sâu dùng SGD/minibatch; paper chỉ suy đoán (không chứng minh) rằng kết quả mở rộng được.
- **Kiến trúc ResNet trong paper không hoàn toàn giống ResNet chuẩn**: paper tự chú thích (footnote 4 trong văn bản) rằng kiến trúc họ định nghĩa có skip-connection ở **mọi lớp**, khác ResNet gốc (He et al. 2016) có skip-connection mỗi 2 lớp; paper nói việc tổng quát hoá cho skip mỗi $k \geq 2$ lớp "không khó" nhưng **không chứng minh chi tiết** trường hợp đó trong bài.
- **Giả định dữ liệu "không suy biến" (non-degenerate)**: điều kiện $\lambda_{\min}(K^{(H)}) > 0$ cần dữ liệu đủ "khác biệt" (không có hai điểm trùng hướng); paper không thảo luận điều gì xảy ra khi dữ liệu gần suy biến (ví dụ ảnh gần giống nhau, một tình huống phổ biến với dữ liệu OCR có nhiều trang giống mẫu).
- **Không có thí nghiệm nào xác nhận lý thuyết trên mạng thực tế**: một giới hạn phương pháp luận paper không nêu trực tiếp nhưng là sự thật khách quan về phạm vi bài: không có một con số thực nghiệm nào (không mô phỏng, không dữ liệu thật) để kiểm tra liệu các hằng số/độ rộng dự đoán bởi lý thuyết có khớp với quan sát thực nghiệm hay không.
- **So sánh với Allen-Zhu et al. (2018c) không hoàn toàn công bằng**: paper tự nói rõ "a fair comparison is not possible due to the difference in setting and data assumptions", nên các con số so sánh độ rộng/độ phức tạp lặp ở §4 chỉ mang tính minh hoạ bậc độ lớn, không phải so sánh trực tiếp hiệu năng.

7. Liên hệ với các bài khác

- **math content/22 (Deep Neural Networks as Gaussian Processes, 2017)**: ma trận Gram $K^{(H)}$ định nghĩa đệ quy ở Định nghĩa 5.1/6.1/7.1 của bài này chính là **kernel NNGP (Neural Network Gaussian Process)** mà math 22 nghiên cứu ở độ rộng vô hạn; paper này tự trích dẫn trực tiếp dòng nghiên cứu Gaussian Process cho mạng vô hạn rộng (Lee et al. 2017, Matthews et al. 2018) trong phần Related Work.
- **math content/24 (Lazy Training, 2018)**: cùng hiện tượng cốt lõi "trọng số gần như không đổi so với khởi tạo trong suốt huấn luyện" (weights stay close to initialization) mà bài này dựa vào (kế thừa từ Li & Liang 2018, Du et al. 2018b) chính là hiện tượng "lazy training"/NTK mà math 24 phân tích tổng quát hơn; hai bài bổ sung cho nhau: math 24 giải thích *vì sao* hiện tượng này xảy ra ở mức tổng quát, bài này chứng minh *hệ quả cụ thể* (hội tụ global minimum) cho 3 kiến trúc cụ thể.
- **math content/32 (Benign Overfitting in Linear Regression, 2019)**: cùng dòng lý thuyết liên quan đến NTK (chính ghi chú math 32 đã có mục thảo luận NTK ở §4, với giới hạn tương tự về việc các giả định không khớp hoàn toàn với NTK thực). Cả hai bài đều thuộc nhóm lý thuyết "over-parameterization giúp tối ưu/generalization tốt hơn dự kiến trực giác cổ điển".
- **math content/34 (A Mean Field View of the Landscape of Two-Layer Neural Networks, 2018, cùng batch)**: một hướng lý thuyết khác (mean-field/optimal transport) cho cùng câu hỏi "vì sao GD tìm được nghiệm tốt cho mạng quá tham số hoá"; bài 33 này dùng kỹ thuật Gram-matrix/NTK-style, trong khi bài 34 dùng công cụ xác suất đo-đạc (measure-valued dynamics); paper 33 tự liệt kê các công

trình optimal-transport (Mei et al. 2018, Chizat & Bach 2018a, Rotskoff & Vanden-Eijnden 2018) như một hướng thay thế trong Related Work, nhưng tự nhận các công trình đó ”giới hạn ở mạng 2 lớp và có thể cần over-parameterization theo cấp số mũ”, một sự đối lập thú vị với chính bài 34.

8. Gợi ý áp dụng

Các định lý hội tụ cho thấy vai trò của độ rộng, khởi tạo và cấu trúc residual trong bối cảnh được phân tích. Các cận độ rộng và giả thiết full-batch gradient descent không tự bảo đảm hội tụ tương tự trong huấn luyện thực tế.

9. Thí nghiệm gợi ý

1. **Minh hoạ ổn định Gram matrix theo độ sâu**: xây một mạng fully-connected toy (ví dụ $H = 2, 4, 8, 16$ lớp, độ rộng $m = 256$, dữ liệu $n = 20$ điểm ngẫu nhiên chuẩn hoá), tính ma trận Gram $G^{(H)}(0)$ bằng autograd (đạo hàm u_i theo $W^{(H)}$) ngay tại khởi tạo, so sánh $\|G^{(H)}(0) - G^{(H)}_{\text{ước lượng theo công thức (7)}}\|$ qua các H khác nhau. Đo: độ lệch này tăng theo H như thế nào. Kỳ vọng: độ lệch tăng rõ rệt (gần cấp số mũ) khi H tăng cho fully-connected thuần.
2. **So sánh fully-connected vs ResNet khi tăng độ sâu**: huấn luyện hai mạng toy (fully-connected và ResNet với scaling $c_{\text{res}}/\sqrt{H}$ như Eq. 3) trên cùng bài toán hồi quy tổng hợp nhỏ ($n = 50$, $d = 10$), với $H \in \{5, 10, 20, 40\}$, độ rộng $m = 128$ cố định, cùng step size. Đo: liệu loss có giảm về 0 ổn định hay phân kỳ/dao động. Kỳ vọng: ResNet vẫn hội tụ ổn định ở H lớn, fully-connected có thể phân kỳ hoặc hội tụ rất chậm khi H tăng, minh hoạ trực quan Định lý 5.1 vs 6.1 (chạy được trên CPU/GPU nhỏ, mạng toy).
3. **Kiểm tra độ lệch trọng số so với khởi tạo (lazy training signature)**: huấn luyện một MLP nhỏ quá tham số hoá ($m \gg n$) trên dữ liệu toy, theo dõi $\frac{1}{\sqrt{m}}\|W^{(h)}(k) - W^{(h)}(0)\|_F$ qua các bước k cho từng lớp h. Đo: đại lượng này có nhỏ và ổn định hay tăng dần. Kỳ vọng: nhỏ và gần như không đổi nếu m đủ lớn (xác nhận giả thuyết cốt lõi của chứng minh, Eq. 6 trong paper), giúp hiểu trực giác ”lazy training” mà không cần đọc chứng minh đầy đủ.
4. **Thử nghiệm điều kiện xác định dương của $K^{(H)}$ khi dữ liệu gần trùng nhau**: tạo dữ liệu tổng hợp với các điểm ngày càng ”gần trùng” (giảm dần khoảng cách cosine giữa các cặp điểm), tính $\lambda_{\min}(K^{(H)})$ bằng mô phỏng Monte Carlo (lấy mẫu nhiều (u, v) từ công thức đệ quy Eq. 7). Đo: $\lambda_{\min}(K^{(H)})$ giảm về 0 như thế nào khi dữ liệu suy biến dần. Kỳ vọng: giảm nhanh, minh hoạ tại sao điều kiện ”dữ liệu không suy biến” quan trọng, liên hệ tới rủi ro khi dữ liệu OCR có nhiều trang/token rất giống nhau.
5. **Ablation gỡ residual connection trên mạng nhỏ đã huấn luyện**: lấy một mạng ResNet toy nhỏ đã huấn luyện tốt (ví dụ 10 lớp, bài toán phân loại tổng hợp), sau đó ”zero-out” hệ số c_{res} của một vài block (giữ nguyên trọng số còn lại) để giả lập việc loại bỏ một phần residual path, rồi fine-tune lại với vài bước GD, đo tốc độ giảm loss trước/sau khi loại residual. Đo: tốc độ hội tụ của GD khi fine-tune. Kỳ vọng: tốc độ giảm loss chậm hơn/kém ổn định hơn khi residual path bị loại, một kiểm chứng nhỏ cho suy luận ở §8 về rủi ro pruning residual connection.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / định nghĩa	Nguồn
Kiến trúc fully-connected	$x^{(h)} = \sigma\left(\sqrt{c_\sigma/m}\, W^{(h)} x^{(h-1)}\right)$, $f = a^\top x^{(H)}$	Eq. 2 (paper)
Kiến trúc ResNet	$x^{(h)} = x^{(h-1)} + \frac{c_{\text{res}}}{\sqrt{H}\sqrt{m}}\sigma(W^{(h)} x^{(h-1)})$	Eq. 3 (paper)
Động lực sai số dự đoán	$y - u(k+1) = (I - \eta G(k))(y - u(k))$	mục 4 (paper)

Khái niệm	Công thức / định nghĩa	Nguồn
Ma trận Gram đệ quy (fully-connected)	$K_{ij}^{(h)} = c_\sigma \, \mathbb{E}_{(u,v)\sim N(0,A_{ij}^{(h)})}[\sigma(u)\sigma(v)]$	Eq. 7 (paper)
Lan truyền nhiễu, fully-connected	$\|G^{(H)}(0) - K^{(H)}\| \lesssim 2^{O(H)}\epsilon_1$	Eq. 4 (paper)
Lan truyền nhiễu, ResNet	$\|G^{(H)}(0) - K^{(H)}\| \lesssim (1+1/H)^H\epsilon_1 = O(\epsilon_1)$	Eq. 5 (paper)
Định lý hội tụ (dạng chung 3 định lý)	$L(\theta(k)) \le \left(1 - \frac{\eta\lambda_{\min}(K^{(H)})}{2}\right)^k L(\theta(0))$	Theorem 5.1/6.1/7.1 (paper)
Độ rộng cần, fully-connected	$m = \Omega\big(2^{O(H)}\,\mathrm{poly}(n, 1/\lambda_{\min})\big)$	Theorem 5.1 (paper)
Độ rộng cần, ResNet	$m = \Omega\big(\mathrm{poly}(n, H, 1/\lambda_{\min})\big)$ (không có số mũ H)	Theorem 6.1 (paper)
Ví dụ tự tính (bước đệ quy với σ tuyến tính giả định)	$K^{(0)} = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix} \Rightarrow K^{(1)} = K^{(0)}$	tự tính, chỉ minh hoạ, không phải điều kiện thật của paper

11. Kết luận ngắn

Bài báo chứng minh định lượng rằng gradient descent khởi tạo ngẫu nhiên hội tụ tuyến tính về loss huấn luyện bằng 0 cho ba kiến trúc mạng sâu quá tham số hoá (fully-connected, ResNet, convolutional ResNet), với mấu chốt kỹ thuật là chứng minh tính ổn định của ma trận Gram $K^{(H)}$ xuyên suốt quá trình huấn luyện. Kết quả quan trọng nhất là **giải thích lý thuyết định lượng cho lợi ích của residual connection**: độ rộng mạng cần thiết giảm từ tăng theo cấp số mũ ($2^{O(H)}$, fully-connected) xuống đa thức (poly(H), ResNet) theo số lớp H, nhờ hệ số scaling $1/\sqrt{H}$ trong kiến trúc residual. Đây là một trong những bài nền tảng của dòng lý thuyết NTK/Gram-matrix cho mạng sâu, nhưng bài báo cũng nêu các giới hạn quan trọng: chỉ chứng minh hội tụ trên tập huấn luyện (không phải generalization), yêu cầu độ rộng vẫn rất lớn so với thực hành, và chỉ áp dụng cho full-batch gradient descent.

Nguồn gốc: https://arxiv.org/abs/1811.03804

Bài 34

A Mean Field View of the Landscape of Two-Layer Neural Networks (2018)

Tác giả: Song Mei, Andrea Montanari, Phan-Minh Nguyen
Venue / arXiv: arXiv:1804.06561 (v2, 28/08/2018); paper không nêu venue trong phần text trích được (được biết đến rộng rãi là PNAS 2018, nhưng vì dòng chữ này không xuất hiện trong văn bản arXiv trích xuất được, ghi ”paper không nêu venue” theo đúng những gì trích được từ full text)
Nguồn: https://arxiv.org/abs/1804.06561
Chủ đề: Chứng minh rằng, trong giới hạn tỷ lệ (scaling limit) phù hợp, động lực SGD khi huấn luyện mạng nơ-ron 2 lớp được mô tả xấp xỉ bởi một phương trình đạo hàm riêng phi tuyến (PDE) gọi là ”distributional dynamics”, cho phép phân tích landscape tối ưu hoá độc lập với số lượng neuron.
Vị trí trong lộ trình: Đọc liền sau math 33 (Gradient Descent Finds Global Minima of DNNs) vì cùng trả lời câu hỏi ”vì sao SGD tìm được nghiệm tốt cho mạng quá tham số hoá” nhưng bằng công cụ khác (mean-field/PDE/Wasserstein gradient flow thay vì Gram-matrix/NTK); là bài áp cuối trong batch math 28-35, liền trước math 35 (Thompson Sampling).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (bản arXiv); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc.

1. Bối cảnh

Bài báo xét hai câu hỏi về tối ưu mạng nơ-ron không lồi: khi SGD đạt nghiệm tốt, phải chăng bài toán không có cực tiểu cục bộ xấu hay thuật toán có thể tránh chúng? Và nếu SGD đạt một cực tiểu cục bộ, điều gì giải thích khả năng tổng quát hóa của nghiệm đó?

Paper giới hạn phạm vi vào trường hợp đơn giản nhất có ý nghĩa: **mạng nơ-ron 2 lớp (two-layer neural network)**:

$$\hat{y}(x;\theta) = \frac{1}{N}\sum_{i=1}^{N} \sigma_*(x;\theta_i), \qquad \sigma_*(x;\theta_i) = a_i\,\sigma(\langle w_i, x\rangle + b_i), \tag{1-2, paper}$$

với N là số neuron ẩn, $\theta_i = (a_i, b_i, w_i)$ là tham số của neuron i. Mục tiêu là tối thiểu hoá **risk** (hàm rủi ro tổng thể, kỳ vọng loss trên phân phối dữ liệu thật) $R_N(\theta) = \mathbb{E}\{\ell(y, \hat{y}(x;\theta))\}$ với loss bình phương $\ell(y,\hat{y}) = (y-\hat{y})^2$.

Thuật ngữ cần làm rõ: **one-pass SGD** là giả định mỗi điểm dữ liệu chỉ được dùng đúng một lần (không lặp lại qua nhiều epoch), hợp lý khi dữ liệu rất lớn; **mean-field limit** (giới hạn trường trung bình) là kỹ thuật vật lý thống kê coi một tập hợp rất nhiều ”hạt” (ở đây là neuron) tương tác với nhau như một **phân phối liên tục** thay vì theo dõi từng hạt riêng lẻ; **Wasserstein metric** W_2 là một khoảng cách giữa hai phân phối xác suất, định nghĩa qua bài toán vận chuyển tối ưu (optimal transport), đo ”chi phí tối thiểu” để biến phân phối này thành phân phối khác.

2. Ý tưởng chính

Quan sát khởi đầu (mục 1.1 của paper): risk $R_N(\theta)$ chỉ phụ thuộc vào $\theta_1, \dots, \theta_N$ **qua phân phối thực nghiệm** của chúng, $\hat{\rho}^{(N)} = N^{-1}\sum_{i=1}^{N}\delta_{\theta_i}$. Viết lại:

$$R_N(\theta) = R_\# + \frac{2}{N}\sum_{i=1}^{N} V(\theta_i) + \frac{1}{N^2}\sum_{i,j=1}^{N} U(\theta_i, \theta_j), \qquad \text{(4, paper)}$$

với $V(\theta) = -\mathbb{E}[y\,\sigma_*(x;\theta)]$ và $U(\theta_1,\theta_2) = \mathbb{E}[\sigma_*(x;\theta_1)\sigma_*(x;\theta_2)]$ là một **kernel đối xứng, xác định dương bán phần (PSD)**. Điều này gợi ý định nghĩa risk cho một phân phối $\rho \in \mathcal{P}(\mathbb{R}^D)$ bất kỳ:

$$R(\rho) = R_\# + 2\int V(\theta)\rho(d\theta) + \int\int U(\theta_1,\theta_2)\,\rho(d\theta_1)\rho(d\theta_2). \qquad \text{(5, paper)}$$

Vì U là PSD, $R(\rho)$ **trở thành hàm lồi (convex)** theo ρ ở giới hạn $N \to \infty$. Nhưng điều này **không** suy ra rằng SGD sẽ luôn tìm được optimum toàn cục: paper dùng một phép tương tự vật lý (coi $\theta_1, \dots, \theta_N$ như vị trí N "hạt" trong không gian D chiều) để chỉ ra rằng động lực của "khí hạt" này **không thể thay đổi mật độ một cách tuỳ ý**: khối lượng được bảo toàn cục bộ (local mass conservation) vì các hạt không thể di chuyển gián đoạn. Đây là lý do động lực PDE giới hạn không đơn giản là "gradient descent trực tiếp trên $R(\rho)$", mà là một **gradient flow trong không gian Wasserstein** $(\mathcal{P}(\mathbb{R}^D), W_2)$.

Kết quả chính (Định lý 3, phát biểu chi tiết ở §3): khi $N \to \infty, \varepsilon \to 0$ với tốc độ step size $s_k = \varepsilon\,\xi(k\varepsilon)$, phân phối thực nghiệm $\hat{\rho}^{(N)}_{t/\varepsilon}$ hội tụ yếu về ρ_t là nghiệm của PDE:

$$\partial_t \rho_t = 2\xi(t)\,\nabla_\theta \cdot \big(\rho_t\,\nabla_\theta \Psi(\theta;\rho_t)\big), \qquad \Psi(\theta;\rho) \equiv V(\theta) + \int U(\theta,\theta')\rho(d\theta'). \qquad \text{(7-8, paper)}$$

Paper gọi đây là **distributional dynamics (DD)**. Điểm mấu chốt: **PDE này không phụ thuộc vào** N (số neuron), chỉ phụ thuộc vào kích thước tham số D và phân phối dữ liệu. Do đó nếu DD hội tụ về gần optimum trong thời gian $t^*(D)$ không phụ thuộc N, thì với $N \gg D$ đủ lớn, SGD sẽ đạt risk gần tối ưu bằng một số bước lặp **chỉ phụ thuộc** D**, không phụ thuộc số neuron** N. Đây là lời giải thích định lượng cho hiện tượng over-parameterization không gây hại hiệu năng.

3. Phương pháp chi tiết

3.1. Noisy SGD và free energy (mục 1.1 cuối, §3.1)

Paper xét thêm biến thể **noisy SGD** (thêm nhiễu Gaussian và regularization ℓ_2):

$$\theta_i^{k+1} = (1 - 2\lambda s_k)\theta_i^k + 2s_k(y_k - \hat{y}_k)\nabla_{\theta_i}\sigma_*(x_k;\theta_i^k) + \sqrt{2s_k/\beta}\,g_i^k, \quad g_i^k \sim N(0, I_D). \qquad \text{(11, paper)}$$

Giới hạn tỷ lệ của phiên bản này thêm một số hạng khuếch tán (diffusion):

$$\partial_t \rho_t = 2\xi(t)\nabla_\theta \cdot (\rho_t \nabla_\theta \Psi_\lambda(\theta;\rho_t)) + 2\xi(t)\beta^{-1}\Delta_\theta \rho_t, \qquad \text{(12, paper)}$$

là gradient flow của **free energy** $F_{\beta,\lambda}(\rho) = \frac{1}{2}R(\rho) + \frac{\lambda}{2}\int \|\theta\|_2^2 \rho(d\theta) - \beta^{-1}\mathrm{Ent}(\rho)$, với $\mathrm{Ent}(\rho)$ là entropy của ρ. Số hạng entropy này (từ nhiễu Gaussian) chính là lý do **Định lý 4-5** của paper chứng minh được **hội tụ tổng quát (generic convergence)** về minimizer toàn cục của $F_{\beta,\lambda}$ (không cần điều kiện đặc biệt), trong khi với SGD không nhiễu (noiseless), paper **chỉ chứng minh được điều kiện ổn định cục bộ** (Định lý 6-7), không có kết quả hội tụ toàn cục tổng quát.

3.2. Bốn ví dụ minh hoạ cụ thể (mục 2)

Paper tự nói rõ các ví dụ này "không thực tế" (not realistic), chỉ để minh hoạ cơ chế lý thuyết bằng tính toán giải được:

(a) Centered isotropic Gaussians (§2.1): $x \sim N(0, (1 \pm \Delta)^2 I_d)$ tuỳ nhãn $y = \pm 1$; hàm kích hoạt $\sigma_*(x; \theta_i) = \sigma(\langle w_i, x \rangle)$ không có offset/trọng số đầu ra, σ là hàm tuyến tính-từng-đoạn ($t_1 = 0.5, t_2 = 1.5, s_1 = -2.5, s_2 = 7.5$). Vì phân phối dữ liệu P đối xứng cầu (spherically symmetric), ρ_t cũng đối xứng cầu nên **rút gọn về một PDE 1 chiều** theo $r = \|w\|_2$:

$$\partial_t \rho_t = 2\xi(t)\, \partial_r \big(\rho_t\, \partial_r \psi(r; \rho_t) \big). \qquad \text{(13, paper)}$$

Định lý 1 (phát biểu đầy đủ): với $\eta, \Delta, \delta > 0$ và khởi tạo ρ_0 thuộc một lớp "khởi tạo tốt" $\mathcal{P}_{\text{good}}$, tồn tại d_0, T, C_0 sao cho với $d \geq d_0$, số neuron $N \geq C_0 d$, step size $\varepsilon \in [1/N^{10}, 1/(C_0 d)]$: $R_N(\theta^k) \leq \inf_\theta R(\theta) + \eta$ với xác suất $\geq 1 - \delta$, cho mọi $k \in [T/\varepsilon, 10T/\varepsilon]$. Điểm quan trọng: **số bước SGD cần thiết $k = O(d)$, không phụ thuộc số neuron N**.

(b) Centered anisotropic Gaussians (§2.2): $\Sigma_\pm = I_d + (\tau_\pm^2 - 1) P_V$ với V là không gian con chiều s_0 cần được mạng "học ra" (biến relevant/irrelevant feature); **Định lý 2** mở rộng kết quả Định lý 1 cho trường hợp đối xứng giảm $O(s_0) \times O(d - s_0)$, với $s_0 = \gamma d$.

(c) A better activation function / ReLU (§2.3): dùng ReLU $\sigma(x) = \max(x, 0)$ với cả trọng số đầu ra a và offset b được học ($\theta = (w, a, b) \in \mathbb{R}^{d+2}$); thí nghiệm với $d = 320, s_0 = 60, N = 800$, cho thấy SGD hội tụ về risk rất nhỏ nhưng **có cấu trúc phi tầm thường với các vùng phẳng (long flat regions)** kéo dài, dự đoán khớp bởi PDE rút gọn 4 chiều (a, b, r_1, r_2).

(d) Predicting failure (§2.4): dùng hàm kích hoạt **không đơn điệu (non-monotone)** $\sigma(t) = -2.5$ với $t \leq 0$, nội suy tuyến tính qua điểm $(0.5, -4)$, rồi $\sigma(t) = 7.5$ với $t \geq 1.5$. Với hàm này (vi phạm giả định đơn điệu của Định lý 1), **SGD hội tụ về một trong hai giới hạn khác nhau tuỳ khởi tạo**: một có risk nhỏ, một có risk cao. Đây là ví dụ paper tự xây dựng để **chứng minh được trước (predict) sự thất bại** của SGD bằng chính lý thuyết PDE của mình, không phải quan sát thất bại rồi giải thích sau.

3.3. Điều kiện ổn định điểm bất động (§3.2)

Định lý 6: với $\rho_* = \delta_{\theta_*}$ (một điểm mass đơn) là điểm bất động của DD (nghĩa là $\nabla V(\theta_*) + \nabla_1 U(\theta_*, \theta_*) = 0$), nếu ma trận Hessian $H_0(\delta_{\theta_*}) = \nabla^2 V(\theta_*) + \int \nabla_{1,1}^2 U(\theta_*, \theta) \rho_*(d\theta)$ có $\lambda_{\min}(H_0) > 0$, thì ρ_* **ổn định (stable)**: nếu khởi tạo đủ gần, $\rho_t \to \rho_*$ với tốc độ mũ.

Định lý 7: ngược lại, nếu $\lambda_{\min}(H_0) < 0$ (điểm yên ngựa / saddle), thì ρ_* **không ổn định**: với khởi tạo có mật độ liên tục tuyệt đối (absolutely continuous), ρ_t không thể hội tụ về ρ_*.

Ví dụ tự tính nhỏ minh hoạ Định lý 6/7: giả sử bài toán 1 chiều đơn giản hoá với $\Psi(\theta; \rho_*) = \theta^2 - c$ gần một điểm $\theta_* = 0$ (minh hoạ ý tưởng, không phải thiết lập thật của paper). Hessian tại $\theta_* = 0$ là $H_0 = \Psi''(0) = 2 > 0$, nên theo logic Định lý 6, $\theta_* = 0$ là điểm bất động ổn định (tự tính, chỉ minh hoạ trực giác dấu của Hessian quyết định ổn định/bất ổn định, không phải một phép tính từ các công thức cụ thể của paper).

4. Thí nghiệm và kết quả chính

Paper có phần thí nghiệm số (Mục 11 trong Supplementary Information) thực sự là **mô phỏng SGD trên dữ liệu tổng hợp (synthetic)** để kiểm chứng (validate) lý thuyết PDE, không phải benchmark trên dữ liệu thực tế nào.

Thí nghiệm	Thiết lập cụ thể (paper)	Kết quả chính
Hình 1 (Isotropic Gaussians, DD)	$\Delta = 0.8$, $d = 40$, $N = 800$, $\varepsilon = 10^{-6}$, $\xi(t) = 1$, khởi tạo $w_i^0 \sim N(0, 0.8^2/d \cdot I_d)$, 1 lần chạy SGD, snapshot tại bước $10^3, 4 \times 10^6, 10^7$	Histogram của $\|w_i\|_2$ từ SGD khớp với nghiệm số của PDE rút gọn (Eq. 13)
Hình 2 (Isotropic Gaussians, Statics)	$d \in \{10, 20, 40, \infty\}$, quét $\Delta \in [0.1, 0.8]$, $N = 800$	Risk thực nghiệm (SGD, 1 lần chạy/điểm) khớp dự đoán lý thuyết từ việc tối thiểu hoá $R(\rho)$; ansatz "single-delta" sai khi $\Delta > \Delta_{hd} \approx 0.47$ (ở $d \to \infty$)
Hình 3 (Anisotropic Gaussians, ReLU)	$d = 320$, $s_0 = 60$, $N = 800$, $\xi(t) = t^{-1/4}$, $\varepsilon = 2 \times 10^{-4}$, khởi tạo $a_i = 1, b_i = 1, w_i^0 \sim N(0, 0.8^2/d \cdot I_d)$	SGD hội tụ về risk nhỏ nhưng có cấu trúc "vùng phẳng kéo dài"; PDE rút gọn 4 chiều khớp tốt
Hình 4 (Predicting failure)	$N = 800$, $d = 320$, $\Delta = 0.5$, hai khởi tạo $w_i^0 \sim N(0, \kappa^2/d \cdot I_d)$ với $\kappa = 0.1$ hoặc $\kappa = 0.4$	SGD hội tụ về 2 giới hạn khác nhau tuỳ κ (risk thấp vs risk cao), đúng như PDE 1 chiều dự đoán trước

Chi tiết kỹ thuật mô phỏng PDE (từ mục 11.1.1, đọc kỹ): vì giải PDE tổng quát (chiều D bất kỳ) tốn kém tính toán, paper chỉ mô phỏng ở **giới hạn** $d = \infty$ (công thức risk dạng tích phân đóng, Eq. 11.1 của SI) và dùng **ansatz nhiều-delta (multiple-deltas ansatz)**, xấp xỉ $\rho_t \approx \frac{1}{J}\sum_{i=1}^{J} \delta_{r_i(t)}$ với J điểm đại diện (ở ví dụ Hình 1, $J = 400$, lặp lại mô phỏng 100 lần với khởi tạo độc lập để lấy trung bình). Paper ghi rõ: trong một số trường hợp đặc biệt (ví dụ $\Delta = 0.2$) **chỉ cần** $J = 1$ (một điểm mass duy nhất) là đủ chính xác, vì nghiệm PDE thực sự hội tụ về một phân phối delta đơn.

5. Phân tích / ablation

Vai trò của hệ số nhiễu β (inverse temperature): paper phân tích rõ sự khác biệt giữa SGD không nhiễu và noisy SGD: với $\beta < \infty$ (có nhiễu), Định lý 4 đảm bảo **hội tụ tổng quát** về minimizer duy nhất của free energy $F_{\beta,\lambda}$ (vì entropy làm cho bài toán "lồi hoá" theo nghĩa thống kê); với $\beta = \infty$ (không nhiễu), paper **chỉ chứng minh được điều kiện ổn định cục bộ** (Định lý 6-7), không đảm bảo hội tụ toàn cục tổng quát - một sự đánh đổi giữa "đảm bảo lý thuyết mạnh hơn" và "gần với thực hành hơn" (SGD chuẩn không có nhiễu Gaussian tường minh).

Phân tích biên Δ_{ld}, Δ_{hd} (Lemma 1): với bài toán isotropic Gaussians, nghiệm tối ưu $\rho_{r_*}^{\text{unif}}$ (phân phối đều trên mặt cầu bán kính r_*) chỉ là **global minimizer trong một khoảng** $\Delta \in [\Delta_{ld}, \Delta_{hd}]$, với $\lim_{d\to\infty} \Delta_{ld} = 0$ và $\lim_{d\to\infty} \Delta_{hd} \approx 0.47$ (paper tính số bằng kiểm tra điều kiện trong Lemma 1, không có công thức closed-form cho Δ_{hd}). Ngoài khoảng này, ansatz "single-delta" (một điểm mass đơn) **không còn là lời giải tối ưu**, cho thấy landscape thực sự phức tạp hơn một điểm cực tiểu duy nhất khi Δ lớn.

So sánh cấu trúc hội tụ giữa các ví dụ: paper ghi nhận và giải thích (mục 11.1.1, "Comments") sự khác biệt về "độ sắc nét" trong quá trình hội tụ: ở $\Delta = 0.2$, ρ_t hội tụ nhanh về gần delta-function (chuyển pha sắc nét từ risk cao sang risk thấp); ở $\Delta = 0.8$, ρ_t **không tập trung (not concentrating)** dù hàm $R_d^{(1)}(r)$ vẫn "lành" (benign landscape 1 chiều) - cho thấy ngay cả khi landscape rút gọn trông đơn giản, động lực đầy đủ của phân phối vẫn có thể phức tạp hơn.

Ảnh hưởng của dữ liệu/kiến trúc lên thời gian hội tụ T: bài báo nêu (cuối mục 3.1, Định lý 5) rằng $T = T(\eta, V, U, \{K_i\}, D, \beta)$ phụ thuộc chiều D và phân phối dữ liệu, và "examining the proof of this theorem suggests $T = e^{O(D)}$ quite generally", nhưng paper cũng nói **có thể xây dựng ví dụ với $T = O(1)$ hoặc $T = e^{\Theta(D)}$** - tức cận trên tổng quát có thể bi quan hơn thực tế tuỳ cấu trúc cụ thể của bài toán.

6. Giới hạn

Các giới hạn dưới đây được paper **tự nêu rõ** (mục 4, Discussion and future directions), không phải suy luận của ghi chú trừ khi ghi rõ:

- **Chỉ áp dụng cho mạng 2 lớp**: toàn bộ lý thuyết (PDE, định lý hội tụ) chỉ xây dựng cho kiến trúc 2 lớp đơn giản nhất; paper không mở rộng cho mạng sâu nhiều lớp trong chính bài này.
- **Không chứng minh tính duy nhất của local minimum ở N hữu hạn**: paper tự nói rõ "we do not prove that the finite-N risk $R_N(\theta)$ has a unique local minimum, or that all local minima are close to each other... might be too strong for the case of neural networks" - đây là một giới hạn paper chủ động không theo đuổi, thay vào đó chỉ chứng minh PDE (giới hạn $N \to \infty$) hội tụ về near-global-optimum.
- **Với noiseless SGD, không có kết quả hội tụ toàn cục tổng quát**: chỉ có điều kiện ổn định cục bộ (Định lý 6-7) cho các điểm bất động dạng "một điểm mass + phần còn lại", không bao trùm mọi cấu hình điểm bất động có thể.
- **Sai số xấp xỉ tăng theo cấp số mũ với thời gian** T (bài báo nêu ngay dưới Định lý 3): "the error grows exponentially with the time horizon T, which limits its applicability to cases in which the DD converges rapidly to a good solution" - nghĩa là lý thuyết chỉ hữu dụng khi PDE hội tụ nhanh, không phải cho mọi bài toán.
- **Các ví dụ minh hoạ không thực tế**: bài báo lưu ý ở đầu mục 2: "these examples are not realistic. First, the data distribution P is extremely simple... Second, the activation function $\sigma_*(x;\theta)$ is not necessarily optimal" - các ví dụ (isotropic/anisotropic Gaussian, hàm kích hoạt tuyến tính-từng-đoạn tuỳ chỉnh) được chọn để **có thể tính toán closed-form**, không phản ánh dữ liệu/kiến trúc thực tế (ảnh, văn bản, v.v).
- **Chỉ xét loss bình phương (square loss) trong phần chứng minh chính**: paper nói việc mở rộng cho các loss khác "should be generalizable... cf. SI" (Appendix B) nhưng không chứng minh đầy đủ trong phần chính.
- **Mô phỏng PDE trong thí nghiệm chỉ thực hiện ở $d = \infty$ với ansatz nhiều-delta**: bản thân việc mô phỏng số PDE tổng quát (chiều D hữu hạn, không rút gọn đối xứng) được paper nói là "computationally intensive", nên mọi thí nghiệm số trong bài đều dựa vào các trường hợp đã **rút gọn được bằng đối xứng** (isotropic/anisotropic Gaussian); không có thí nghiệm nào cho trường hợp hoàn toàn tổng quát không có đối xứng đặc biệt.

7. Liên hệ với các bài khác

- **math content/33 (Gradient Descent Finds Global Minima of DNNs, 2018, cùng batch)**: hai bài trả lời cùng câu hỏi lớn ("vì sao GD/SGD hội tụ tốt cho mạng quá tham số hoá") bằng hai công cụ khác nhau: bài 33 dùng ma trận Gram/NTK-style (mạng sâu nhiều lớp, GD toàn batch), bài này dùng mean-field/PDE/Wasserstein gradient flow (chỉ mạng 2 lớp, SGD one-pass). Paper 34 tự liệt kê các công trình optimal-transport liên quan (Mei et al., Chizat & Bach, Rotskoff & Vanden-Eijnden) trong phần Discussion, và paper 33 cũng dẫn chính các công trình này ở Related Work, cho thấy hai hướng lý thuyết này được chính các tác giả đương thời coi là song song/bổ sung.
- **math content/22 (Deep Neural Networks as Gaussian Processes, 2017)**: cùng chủ đề "mạng ở giới hạn rộng vô hạn / số neuron vô hạn có cấu trúc toán học đơn giản hơn"; math 22 dùng giới hạn Gaussian Process (hàm số ngẫu nhiên), bài này dùng giới hạn trường trung bình (phân phối tham số); cả hai đều khai thác $N \to \infty$ để "làm phẳng" độ phức tạp của landscape, nhưng theo hai đối tượng toán học khác nhau (hàm số vs. phân phối tham số).
- **math content/20 (Benefits of Depth, 2016) và math content/21 (Expressive Power of Width, 2017)**: cùng dòng câu hỏi lý thuyết về vai trò của độ rộng/độ sâu trong mạng nơ-ron; bài này cho thấy cụ thể **độ rộng N không ảnh hưởng đến landscape tối ưu hoá** khi $N \gg D$ (chỉ ảnh hưởng tốc độ xấp xỉ $O(1/N)$ giữa R_N và R), một góc nhìn bổ sung cho câu hỏi "độ rộng cần bao nhiêu là đủ" mà hai bài kia tiếp cận từ phía biểu diễn hàm (expressivity) thay vì tối ưu hoá.

- **math content/24 (Lazy Training, 2018)**: đối lập thú vị cần nêu rõ: ”lazy training” (math 24) mô tả chế độ mà trọng số gần như không đổi so với khởi tạo (kernel regime, NTK); bài 34 này mô tả chế độ **ngược lại** (”feature learning” / mean-field regime), nơi phân phối tham số ρ_t **di chuyển đáng kể** theo thời gian (không ”lazy”) để tối thiểu hoá risk $R(\rho)$. Đây là hai chế độ scaling khác nhau của cùng kiến trúc mạng (phụ thuộc cách chọn hệ số chuẩn hoá theo N), một điểm đối chiếu quan trọng khi đọc cả ba bài 33/34/24 cùng nhau.

8. Gợi ý áp dụng

Giới hạn mean-field mô tả động lực học qua phân phối tham số và tạo một góc nhìn bổ sung cho tuyến tính hóa kiểu NTK. Những kết luận cụ thể phụ thuộc kiến trúc hai lớp, thang tham số hóa và giả thiết dữ liệu của bài.

9. Thí nghiệm gợi ý

1. **Tái tạo thí nghiệm Hình 1 (isotropic Gaussians) ở quy mô nhỏ**: cài mạng 2 lớp với $\sigma_*(x;\theta_i) = \sigma(\langle w_i, x\rangle)$ dùng đúng hàm kích hoạt tuyến tính-từng-đoạn của paper ($t_1 = 0.5, t_2 = 1.5, s_1 = -2.5, s_2 = 7.5$), $d = 20$ (nhỏ hơn paper để chạy nhanh trên CPU/GPU nhỏ), $N = 200$, $\Delta = 0.8$, khởi tạo $w_i^0 \sim N(0, 0.8^2/d \cdot I_d)$. Đo: histogram của $\|w_i\|_2$ qua các bước huấn luyện. Kỳ vọng: phân phối ”nở rộng” dần, có thể thấy xu hướng tương tự Hình 1 của paper ở quy mô nhỏ hơn.
2. **Minh hoạ ”Predicting failure” (Hình 4)**: cài đúng hàm kích hoạt không đơn điệu của §2.4 ($\sigma(t) = -2.5$ với $t \le 0$, nội suy qua $(0.5, -4)$, rồi $\sigma(t) = 7.5$ với $t \ge 1.5$), $d = 40, N = 200, \Delta = 0.5$, thử 2 khởi tạo với $\kappa = 0.1$ và $\kappa = 0.4$ như paper. Đo: risk cuối cùng sau nhiều bước SGD cho mỗi khởi tạo. Kỳ vọng: hai khởi tạo hội tụ về hai mức risk khác biệt rõ rệt, minh hoạ trực quan hiện tượng SGD ”mắc kẹt” tuỳ khởi tạo khi vi phạm điều kiện đơn điệu của Định lý 1.
3. **Kiểm tra độc lập với số neuron** N: huấn luyện cùng bài toán isotropic Gaussians ($d = 20, \Delta = 0.4$ cố định) với nhiều giá trị $N \in \{50, 100, 400, 1600\}$, cùng số bước huấn luyện. Đo: risk cuối cùng theo N. Kỳ vọng: risk hội tụ về một giá trị gần như không đổi khi N đủ lớn (ví dụ từ $N = 400$ trở lên không cải thiện thêm đáng kể), minh hoạ trực giác ”SGD không phụ thuộc N khi $N \gg D$”.
4. **Quan sát chuyển pha theo** Δ **(tái tạo một phần Hình 2)**: quét $\Delta \in \{0.1, 0.2, \ldots, 0.8\}$ với $d = 20$ cố định, đo risk cuối cùng sau huấn luyện SGD hội tụ. Đo: đường cong risk theo Δ. Kỳ vọng: thấy xu hướng risk tăng dần theo Δ với khả năng có một điểm uốn/bất liên tục gần giá trị Δ_{hd} tương tự paper (dù quy mô d nhỏ hơn sẽ làm hiệu ứng kém rõ hơn).
5. **So sánh chế độ ”lazy” vs ”feature learning” bằng cách thay đổi scaling khởi tạo**: huấn luyện cùng mạng 2 lớp trên bài toán tổng hợp, nhưng thay đổi độ lớn khởi tạo $w_i^0 \sim N(0, \sigma_0^2/d \cdot I_d)$ với σ_0 nhỏ (ví dụ 0.01, gần chế độ lazy/NTK) và σ_0 lớn hơn (ví dụ 1.0, gần chế độ mean-field/feature-learning). Đo: độ lệch trung bình $\frac{1}{N}\sum_i \|w_i(k) - w_i(0)\|_2$ theo thời gian huấn luyện cho mỗi σ_0. Kỳ vọng: độ lệch nhỏ và tương đối ổn định ở σ_0 nhỏ (lazy), độ lệch lớn dần theo thời gian ở σ_0 lớn (feature learning), minh hoạ trực quan sự khác biệt giữa hai chế độ nêu ở §7/§8.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / định nghĩa	Nguồn
Mạng 2 lớp	$\hat{y}(x;\theta) = \frac{1}{N}\sum_{i=1}^{N} a_i \sigma(\langle w_i, x\rangle + b_i)$	Eq. 1-2 (paper)
Risk dạng potential	$R_N(\theta) = R_\# + \frac{2}{N}\sum_i V(\theta_i) + \frac{1}{N^2}\sum_{i,j} U(\theta_i, \theta_j)$	Eq. 4 (paper)
Risk cho phân phối liên tục	$R(\rho) = R_\# + 2\int V\, d\rho + \int\int U\, d\rho\, d\rho$	Eq. 5 (paper)
Distributional dynamics (DD)	$\partial_t \rho_t = 2\xi(t)\nabla_\theta \cdot (\rho_t \nabla_\theta \Psi(\theta; \rho_t))$	Eq. 7 (paper)

Khái niệm	Công thức / định nghĩa	Nguồn
Khoảng cách Wasserstein	$W_2(\rho_1,\rho_2) = \left(\inf_{\gamma\in C(\rho_1,\rho_2)} \int \|\theta_1-\theta_2\|_2^2\,\gamma(d\theta_1,d\theta_2)\right)^{1/2}$	Eq. 9 (paper)
Noisy SGD / diffusion PDE	$\partial_t\rho_t = 2\xi(t)\nabla_\theta\cdot(\rho_t\nabla_\theta\Psi_\lambda) + 2\xi(t)\beta^{-1}\Delta_\theta\rho_t$	Eq. 12 (paper)
Free energy	$F_{\beta,\lambda}(\rho) = \frac{1}{2}R(\rho) + \frac{\lambda}{2}\int\|\theta\|_2^2\,\rho(d\theta) - \beta^{-1}\mathrm{Ent}(\rho)$	mục 3.1 (paper)
Điều kiện ổn định điểm bất động	$\lambda_{\min}(H_0(\delta_{\theta_*})) > 0 \Rightarrow$ ổn định; $< 0 \Rightarrow$ bất ổn định	Theorem 6-7 (paper)
PDE rút gọn 1 chiều (isotropic Gaussian)	$\partial_t\rho_t = 2\xi(t)\partial_r(\rho_t\partial_r\psi(r;\rho_t))$	Eq. 13 (paper)
Ví dụ tự tính (dấu Hessian quyết định ổn định)	$\Psi(\theta)\approx\theta^2 - c \Rightarrow H_0 = 2 > 0 \Rightarrow$ ổn định tại $\theta_* = 0$	tự tính, chỉ minh hoạ trực giác

11. Kết luận ngắn

Bài báo xây dựng một khung lý thuyết mới (distributional dynamics, DD) để phân tích SGD trên mạng nơ-ron 2 lớp, chứng minh rằng ở giới hạn $N\to\infty$, động lực SGD được mô tả bởi một PDE dạng gradient flow trong không gian Wasserstein, hoàn toàn **không phụ thuộc số neuron** N. Khung lý thuyết này cho phép khai thác đối xứng của dữ liệu để giải PDE một cách rõ ràng trên một số ví dụ cụ thể (Gaussian isotropic/anisotropic), chứng minh SGD hội tụ gần optimum toàn cục trong một số trường hợp, và quan trọng hơn, **dự đoán trước được khi nào SGD sẽ thất bại** (ví dụ "Predicting failure" với hàm kích hoạt không đơn điệu) chỉ bằng phân tích PDE, không cần chạy thử nghiệm trước. Bài cung cấp cung cấp một cơ chế toán học khác (so với NTK/Gram-matrix của các bài cùng giai đoạn như math 33) để hiểu vì sao over-parameterization không gây hại hiệu năng tối ưu hoá, nhưng bài báo giới hạn phạm vi vào mạng 2 lớp, loss bình phương, và các ví dụ dữ liệu đơn giản hoá để có thể tính toán closed-form.

Nguồn gốc: https://arxiv.org/abs/1804.06561

Bài 35

A Tutorial on Thompson Sampling (2017)

Tác giả: Daniel J. Russo, Benjamin Van Roy, Abbas Kazerouni, Ian Osband, Zheng Wen
Venue / arXiv: arXiv:1707.02038 (v1, tháng 7/2017; bản tutorial hoàn chỉnh được in lại năm 2018), paper không nêu venue trong phần text trích được (dòng trích dẫn tự ghi ”Foundations and Trends...” bị mất tên tạp chí khi OCR, chỉ còn thấy ”Vol. 11, No. 1, pp 1-96, DOI: 10.1561/2200000070”; đây chính là Foundations and Trends in Machine Learning theo DOI prefix 10.1561, nhưng vì paper không hiện rõ chữ này trong văn bản trích xuất được, ghi nhận đây là suy luận từ DOI, không phải trích nguyên văn)
Nguồn: https://arxiv.org/abs/1707.02038
Chủ đề: Tutorial tổng hợp về thuật toán Thompson Sampling (TS) cho bài toán ra quyết định tuần tự dưới thông tin không đầy đủ (multi-armed bandit và mở rộng), từ công thức cơ bản, các phương pháp xấp xỉ hậu nghiệm, đến lý thuyết regret và giới hạn của TS.
Vị trí trong lộ trình: Đọc sau khi đã nắm trực giác về suy luận Bayes và MCMC (math 10 Hamiltonian Monte Carlo, math 11 SGD như suy luận Bayes xấp xỉ); đây là bài cuối trong batch math 28-35, đóng vai trò cầu nối giữa suy luận Bayes thống kê và các bài toán khám phá (exploration) trong RL (rl 13 RND exploration) và tối ưu hoá Bayes (optimization 32, 33).

> Tài liệu học tập bằng tiếng Việt, là ghi chú diễn giải, không phải bản dịch. Số liệu lấy từ paper (bản arXiv); các phép tính đánh dấu ”tự tính” là của người ghi chú. Mục gợi ý áp dụng là bình luận, không phải kết luận trực tiếp của bài gốc.

1. Bối cảnh

Trong bài toán ra quyết định trực tuyến, một tác tử tương tác lặp lại với hệ thống chưa biết đầy đủ. Ở mỗi vòng t, tác tử chọn hành động x_t, nhận phần thưởng r_t và tìm cách tối đa hóa tổng phần thưởng tích lũy. Việc lựa chọn phải cân bằng giữa khám phá để hiểu hệ thống và khai thác những hành động hiện được đánh giá là tốt.

Paper mở đầu bằng **Example 1.1 (Bernoulli Bandit)**: có K hành động (ví dụ K mẫu banner quảng cáo), mỗi hành động k có xác suất thành công (click) chưa biết $\theta_k \in [0, 1]$; mỗi lần chọn hành động k, agent quan sát một phần thưởng Bernoulli với tham số θ_k. Agent không biết trước $\theta = (\theta_1, \ldots, \theta_K)$ và phải học qua trải nghiệm.

Paper cũng đưa ra **Example 1.2 (Online Shortest Path)**: agent đi từ điểm xuất phát đến điểm đích trên một đồ thị có hướng mỗi ngày, quan sát thời gian đi qua từng cạnh đã chọn, với mục tiêu học đường đi ngắn nhất kỳ vọng. Đây là ví dụ cho bài toán có cấu trúc phức tạp hơn multi-armed bandit cơ bản (số hành động ”path” có thể là hàm mũ theo số cạnh).

Thompson Sampling (TS) được **Thompson đề xuất từ năm 1933** cho bài toán thử nghiệm y khoa (clinical trial), nhưng bị bỏ quên trong nhiều thập kỷ. TS chỉ được khám phá lại và phổ biến rộng rãi nhờ các công trình thực nghiệm của Chapelle và Li (2011) và Scott (2010), cho thấy TS hoạt động tốt trong các bài toán quảng cáo trực tuyến thực tế. Paper này (2017-2018) là nỗ lực hệ thống hoá: trình bày thuật toán, nền tảng lý thuyết, các cách xấp xỉ khi tính toán hậu nghiệm (posterior) đúng là bất khả thi, và một loạt case study ứng dụng. Nhóm tác giả công bố kèm một package Python (github.com/iosband/ts_tutorial) để tái tạo mọi hình và thí nghiệm trong bài.

Thuật ngữ quan trọng cần làm rõ ngay: **posterior distribution** (phân phối hậu nghiệm) là niềm tin (belief) của agent về tham số hệ thống θ sau khi đã quan sát một số dữ liệu, được tính bằng công thức Bayes từ **prior distribution** (phân phối tiên nghiệm, niềm tin trước khi quan sát) và **likelihood** (hàm hợp lý, xác suất quan

sát dữ liệu nếu biết θ). **Regret** (hối tiếc) là thước đo hiệu năng: chênh lệch giữa phần thưởng kỳ vọng của hành động tối ưu (nếu biết θ) và phần thưởng kỳ vọng thực tế đạt được.

2. Ý tưởng chính

Ở mỗi vòng t, Thompson sampling lấy một mẫu $\hat{\theta}_t$ từ hậu nghiệm hiện tại của θ và chọn hành động x_t tối ưu nếu mẫu đó là tham số thật. Sau khi quan sát phần thưởng, tác tử cập nhật hậu nghiệm theo công thức Bayes và lặp lại quá trình.

Paper so sánh TS với chiến lược **greedy** (tham lam): greedy luôn chọn hành động có phần thưởng kỳ vọng cao nhất theo ước lượng hiện tại (ví dụ trung bình hậu nghiệm), không có cơ chế khám phá chủ động nào. Vấn đề của greedy minh hoạ qua một ví dụ 3 hành động: nếu hành động 3 có mật độ hậu nghiệm phẳng (độ chắc chắn thấp) nhưng trung bình thấp hơn hành động 1 (mật độ nhọn ở 0.6) một chút, greedy sẽ không bao giờ thử hành động 3 dù nó có thể thực chất tốt hơn, vì ước lượng điểm (point estimate) không phản ánh được độ không chắc chắn (uncertainty).

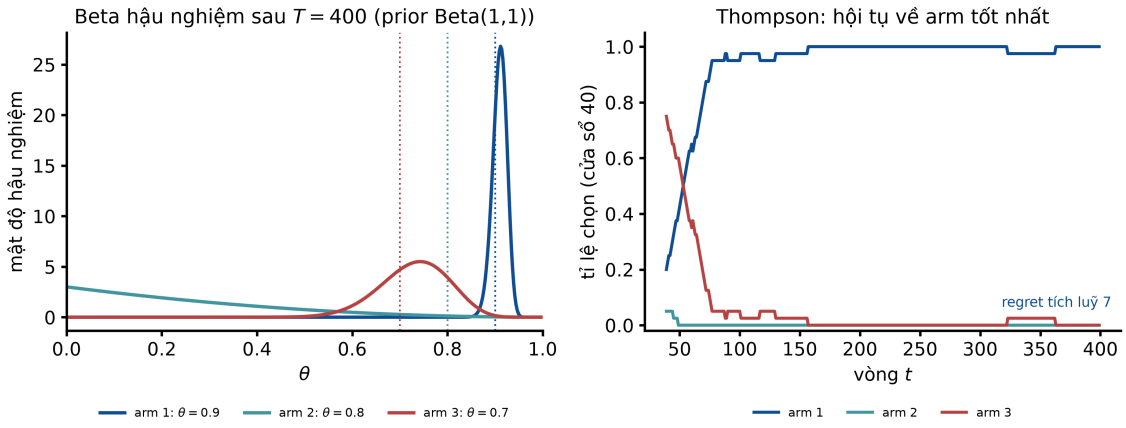
Một cách khác hay dùng là ϵ-**greedy**: với xác suất ϵ chọn ngẫu nhiên (dithering), với xác suất $1-\epsilon$ chọn greedy. Paper phê bình cách này là ”mù”: khi khám phá ngẫu nhiên, nó phân bổ đều cho mọi hành động kém, không ưu tiên các hành động ”có khả năng là tối ưu nhưng chưa chắc chắn” hơn các hành động ”chắc chắn là kém”.

TS giải quyết vấn đề này một cách tự nhiên: hành động x được chọn với xác suất đúng bằng **xác suất hậu nghiệm rằng** x **là hành động tối ưu**. Nếu một hành động có độ không chắc chắn cao (hậu nghiệm phẳng) thì nó vẫn có xác suất được chọn tương đối cao dù giá trị trung bình không phải cao nhất, tạo ra khám phá tự nhiên theo đúng mức độ ”đáng nghi ngờ” của mỗi hành động, mà không cần tinh chỉnh tham số ϵ nào.

3. Phương pháp chi tiết

3.1. Beta-Bernoulli Bandit (Example 3.1)

Đây là trường hợp cụ thể đơn giản nhất. Mỗi θ_k có tiên nghiệm beta(α_k, β_k) với mật độ



Thompson Sampling Beta–Bernoulli với θ=(0.9,0.8,0.7): hậu nghiệm sau T=400 và tỉ lệ chọn hội tụ về arm tốt nhất. (minh hoạ, tự tính)

$$p(\theta_k) = \frac{\Gamma(\alpha_k + \beta_k)}{\Gamma(\alpha_k)\Gamma(\beta_k)}\, \theta_k^{\alpha_k - 1}(1-\theta_k)^{\beta_k - 1}.$$

Vì phân phối Bernoulli và Beta là **liên hợp (conjugate)**, sau khi chọn hành động $x_t = k$ và quan sát phần thưởng $r_t \in \{0, 1\}$, việc cập nhật hậu nghiệm chỉ là cộng đơn giản:

$$(\alpha_k, \beta_k) \leftarrow (\alpha_k, \beta_k) + (r_t,\ 1 - r_t) \quad \text{(chỉ hành động được chọn mới cập nhật).}$$

Thuật toán 3.2 (BernTS): ở mỗi vòng, lấy mẫu $\hat{\theta}_k \sim \text{beta}(\alpha_k, \beta_k)$ độc lập cho mọi k, chọn $x_t = \arg\max_k \hat{\theta}_k$, quan sát r_t, cập nhật $(\alpha_{x_t}, \beta_{x_t})$ như trên.

Ví dụ tự tính: giả sử 3 hành động có hậu nghiệm $\text{beta}(5,3)$, $\text{beta}(2,2)$, $\text{beta}(1,6)$ (trung bình tương ứng $5/8 = 0.625$, $2/4 = 0.5$, $1/7 \approx 0.143$). Paper minh hoạ bằng một ví dụ số với 3 hành động có xác suất được chọn bởi TS xấp xỉ $0.82/0/0.18$ khi hậu nghiệm đã khá tách biệt; đây là số của paper, không phải tự tính của ghi chú này. Để minh hoạ cơ chế liên hợp, người ghi chú tự tính thêm: nếu hành động 2 (hậu nghiệm $\text{beta}(2,2)$) được chọn và quan sát $r_t = 1$ (thành công), hậu nghiệm mới là $\text{beta}(3,2)$ với trung bình mới $3/5 = 0.6$ (tự tính), cho thấy việc cập nhật chỉ đơn giản là cộng 1 vào α khi thành công.

Paper mô tả một mô phỏng với $\theta = (0.9, 0.8, 0.7)$, 10.000 lần lặp mô phỏng độc lập, 1.000 vòng mỗi lần. Kết quả: greedy có thể "mắc kẹt" (converge sai vào hành động kém) ở một tỷ lệ các lần mô phỏng, trong khi TS hội tụ ổn định về hành động tối ưu, thể hiện qua đồ thị per-period regret (Hình 3.2a/3.2b của paper).

3.2. Thompson Sampling tổng quát (Chương 4)

Paper tổng quát hoá: với mô hình bất kỳ $y_t = g(x_t, \theta, w_t)$ (trong đó w_t là nhiễu độc lập), hậu nghiệm cập nhật theo luật Bayes tổng quát:

$$p(\theta' | H_t) \propto p(\theta' | H_{t-1}) \cdot q_{\theta'}(y_t | x_t), \qquad \text{(4.2 trong paper)}$$

trong đó $q_{\theta'}(y_t|x_t)$ là hàm hợp lý (likelihood) của quan sát y_t dưới tham số θ' và hành động x_t. **Thuật toán 4.2 (Thompson)**: lấy mẫu $\hat{\theta}_t \sim p(\cdot|H_{t-1})$, chọn $x_t = \arg\max_x \mathbb{E}[r(y) \mid \hat{\theta}_t, x]$, quan sát, cập nhật theo công thức trên.

Paper minh hoạ với **Example 4.1 (Independent Travel Times)**: mỗi cạnh e có thời gian di chuyển log-Gaussian với tham số (μ_e, σ_e^2) chưa biết, dùng tiên nghiệm liên hợp Gaussian-chuẩn ngược (normal-inverse-gamma) để cập nhật dạng đóng (Eq 4.3 trong paper). Thí nghiệm với một đồ thị "binomial bridge" 20 tầng (184.756 đường đi khả dĩ) cho thấy TS hội tụ nhanh hơn hẳn greedy và ϵ-greedy.

3.3. Các phương pháp xấp xỉ hậu nghiệm (Chương 5)

Với nhiều mô hình thực tế (ví dụ logistic, neural network), hậu nghiệm đúng không có dạng đóng và không thể lấy mẫu chính xác. Paper trình bày 4 phương pháp xấp xỉ:

(a) Gibbs sampling: một thuật toán MCMC (Markov Chain Monte Carlo) tổng quát, lấy mẫu lần lượt từng thành phần của θ có điều kiện theo các thành phần còn lại, lặp nhiều "sweep" để mẫu hội tụ về phân phối hậu nghiệm đúng.

(b) Laplace approximation: xấp xỉ hậu nghiệm bằng một phân phối Gaussian tập trung ở **mode** (điểm cực đại) của log mật độ hậu nghiệm, dùng khai triển Taylor bậc 2:

$$C = -\nabla^2 \ln g(\phi)\Big|_{\phi=\text{mode}},$$

với C là ma trận hiệp phương sai nghịch đảo của xấp xỉ Gaussian. Paper nêu rõ hạn chế: phương pháp này **không bất biến (not invariant)** với phép đổi biến (nếu đổi tham số hoá, kết quả xấp xỉ khác đi), và trong thí nghiệm của paper (bài toán gợi ý đường đi với likelihood dạng logistic) nó cho hiệu năng **kém hơn** Langevin Monte Carlo.

(c) Langevin Monte Carlo (LMC): mô phỏng một phương trình vi phân ngẫu nhiên (SDE)

$$d\phi_t = \nabla \ln g(\phi_t)\, dt + \sqrt{2}\, dB_t,$$

có phân phối dừng chính là hậu nghiệm g mong muốn; rời rạc hoá Euler (Eq 5.2 trong paper) cho một chuỗi Markov xấp xỉ. Paper còn giới thiệu biến thể **stochastic-gradient Langevin MC** dùng minibatch (cỡ mini-batch 100 trong thí nghiệm của paper) để giảm chi phí tính gradient khi có nhiều dữ liệu, và một biến thể **preconditioning** dùng ma trận Hessian nghịch đảo âm tại mode để cải thiện tốc độ hội tụ hình học.

(d) Bootstrapping: tạo một "lịch sử giả định" bằng cách resample (lấy mẫu lại có hoàn lại) từ lịch sử quan sát thật, cộng thêm một mẫu tiên nghiệm giả θ_0, rồi giải một bài toán MAP (maximum a posteriori) ngẫu nhiên hoá trên dữ liệu giả này. Paper ghi rõ: phương pháp này **phi tham số (nonparametric)** nên linh hoạt, nhưng **thiếu cơ sở lý thuyết chặt chẽ** để chứng minh nó xấp xỉ đúng hậu nghiệm.

Kiểm tra đối chiếu (Sanity checks, mục 5.5): paper so sánh 3 phương pháp xấp xỉ trên với TS chính xác ở những bài toán có thể tính hậu nghiệm đúng (Bernoulli bandit cho Gibbs, shortest path với biến đổi biến cho Laplace), cho thấy các xấp xỉ cho hiệu năng gần với TS chính xác (Hình 5.2 của paper).

Triển khai tăng dần (mục 5.6): bài báo nhấn mạnh yêu cầu thực tế là chi phí tính toán mỗi vòng phải **cố định**, không tăng theo số lượng dữ liệu đã quan sát (ví dụ không chạy lại MCMC từ đầu mỗi vòng), động lực cho các kỹ thuật cập nhật tăng dần (incremental) như ensemble sampling dựa trên bootstrap.

4. Thí nghiệm và kết quả chính

Đây là một bài tutorial tổng hợp lý thuyết + mô phỏng minh hoạ, **không có benchmark trên dữ liệu thực tế công khai**; toàn bộ "thí nghiệm" trong paper là **mô phỏng Monte Carlo trên các mô hình tổng hợp (synthetic)** được paper tự thiết kế để minh hoạ từng luận điểm. Các kết quả thí nghiệm chính (đều đã nêu nguồn dữ liệu tổng hợp rõ ràng trong paper):

Thí nghiệm (mục trong paper)	Thiết lập	Kết quả chính
Bernoulli bandit 3 tay (3.3)	$\theta = (0.9, 0.8, 0.7)$, 10.000 mô phỏng, 1.000 vòng	TS hội tụ ổn định; greedy có thể mắc kẹt ở hành động kém
Shortest path (4.4)	Binomial bridge 20 tầng, 184.756 đường	TS vượt trội greedy và ϵ-greedy
News recommendation (7.1)	$K = 3$ bài báo, $d = 7$ đặc trưng, logit model, 2.000 instance	TS (qua Laplace/Langevin) vượt ϵ-greedy tối ưu ($\epsilon = 0.01$)
Product assortment (7.2)	$n = 6$ sản phẩm, $\sigma^2 = 0.04$	TS vượt greedy và ϵ-greedy (cả loại cố định và loại annealing)
Cascading bandit K=1000, J=100 (7.3)	Beta(1,40), 1000 instance, 20.000 vòng	CascadeTS vượt hẳn CascadeUCB (UCB1)
Cascading bandit K=50, J=10 (7.3)	cùng thiết lập, quy mô nhỏ hơn	CascadeUCB (đã tinh chỉnh c) vượt CascadeTS, trái với trường hợp trên
Active learning neural net (7.4)	mạng 2 lớp, M=100, D=50, K=100	Ensemble TS (ensemble sampling) vượt ϵ-greedy (cả annealing); chỉ cần khoảng 30 thành viên ensemble
Deep exploration trong MDP (7.5)	chuỗi trạng thái dài $2N$+1, thưởng ở hai đầu	Lấy mẫu một lần mỗi episode (PSRL) học đúng MDP trong 1 episode; lấy mẫu mỗi bước cần tối thiểu $2N$ episode
Nonstationary Bernoulli bandit (6.3)	$\gamma = 0.01$, 10.000 mô phỏng	Nonstationary TS duy trì hiệu năng khi hệ thống trôi dạt; TS thường bị suy giảm

Kết quả cascading bandit (hai dòng in đậm ở trên) là điểm đáng chú ý: **paper tự chỉ ra một trường hợp mà TS không phải lựa chọn tốt nhất** (CascadeUCB với c đã tinh chỉnh tốt hơn CascadeTS khi $K = 50, J = 10$), mặc dù trong trường hợp quy mô lớn hơn ($K = 1000, J = 100$) thì TS vượt trội. Paper giải thích khác biệt này bằng hình học của tập tin cậy (hyper-rectangular so với ellipsoidal) và hiệu ứng số chiều.

5. Phân tích / ablation

Vì đây là lý thuyết + mô phỏng minh hoạ, ”ablation” ở đây hiểu theo nghĩa phân tích điều kiện khi nào TS hoạt động tốt và cơ sở lý thuyết.

5.1. Vì sao TS hoạt động tốt (mục 8.1)

Paper trình bày 3 hướng phân tích lý thuyết, đều dựa trên khái niệm **regret** tích lũy $\text{Regret}(T) = \sum_{t=1}^{T}(\mu(x^*, \theta) - \mu(x_t, \theta))$:

(a) Regret tiệm cận (asymptotic, mục 8.1.1): với bandit Bernoulli,

$$\lim_{T\to\infty} \frac{\mathbb{E}[\text{Regret}(T) \mid \theta]}{\log T} = \sum_{k\neq k^*} \frac{\theta_{k^*} - \theta_k}{d_{KL}(\theta_k \,\|\, \theta_{k^*})},$$

đúng với giới hạn dưới tối ưu của Lai và Robbins (1985), tức TS đạt **tối ưu tiệm cận** cho bài toán này (đã được chứng minh trong các công trình của Agrawal và Goyal, Kauffmann et al.).

(b) Regret không phụ thuộc instance (instance-independent, mục 8.1.2): với bandit Bernoulli tiên nghiệm đều, $\max_{\theta'} \mathbb{E}[\text{Regret}(T)|\theta = \theta'] = O(\sqrt{KT \log T})$ (Eq 8.3), gần tối ưu so với giới hạn dưới $\Omega(\sqrt{KT})$.

Với bài toán tổng quát hơn (mô hình tuyến tính, hàm tuyến tính tổng quát hoá, Gaussian process), paper trình bày mối liên hệ chặt giữa TS và các thuật toán **UCB (Upper Confidence Bound)**: với hàm U_t bất kỳ xác định từ lịch sử H_{t-1},

$$\mathbb{E}[U_t(x_t)] = \mathbb{E}[U_t(x^*)] \tag{8.4}$$

(vì x_t và x^* có cùng phân phối biên theo hậu nghiệm), từ đó mọi bound regret của UCB ”dịch” sang bound regret kỳ vọng của TS. Điểm khác biệt quan trọng: regret của UCB phụ thuộc vào lựa chọn cụ thể hàm U_t, còn regret của TS chỉ phụ thuộc vào **lựa chọn** U_t **tốt nhất có thể** (vì U_t chỉ là công cụ phân tích, không phải một phần của thuật toán TS).

Kết quả tổng hợp (Eq 8.6, của Russo và Van Roy 2014) biểu diễn regret theo hai đại lượng phức tạp của lớp hàm phần thưởng F: **log-covering number** (số lượng tập cần để phủ F ở độ chính xác T^{-2}, đo độ nhạy overfitting) và **eluder dimension** (đo mức độ giá trị của hành động chưa quan sát có thể suy ra từ hành động đã quan sát). Bài báo nhấn mạnh **eluder dimension là khái niệm mới do chính nhóm tác giả đưa ra**, vì các khái niệm độ phức tạp thống kê cổ điển (VC dimension, Kolmogorov) không đủ để chặn regret trong bài toán ra quyết định trực tuyến.

(c) Phân tích qua lý thuyết thông tin (information-theoretic, tiếp mục 8.1.2): định nghĩa **tỷ lệ thông tin (information ratio)**

$$\Gamma_t = \frac{(\mathbb{E}[\mu(x^*, \theta) - \mu(x_t, \theta)])^2}{I(x^*; (x_t, y_t) \mid H_{t-1})}, \tag{8.7}$$

đo ”chi phí regret bình phương trên mỗi bit thông tin thu được”. Từ đó suy ra bound tổng quát

$$\mathbb{E}[\text{Regret}(T)] \leq \sqrt{\Gamma\, H(x^*)\, T}, \tag{8.8}$$

với $H(x^*)$ là entropy của tiên nghiệm về hành động tối ưu. Với bài toán shortest path, paper chứng minh $\Gamma \leq d/2$ (d = số cạnh), nên bound phụ thuộc số cạnh thay vì số đường đi (có thể là hàm mũ số cạnh) - đây là lý do TS khai thác được cấu trúc bài toán hiệu quả hơn so với coi mỗi đường đi là một ”hành động” độc lập.

5.2. Vì sao cần ngẫu nhiên hoá (mục 8.1.3)

Paper chứng minh bằng **Example 8.1**: với bài toán 2 hành động, một hành động đã biết chắc (Bernoulli(1/2)) và một hành động phụ thuộc tham số nhị phân chưa biết, **mọi chiến lược dừng (stationary) tất định (deterministic)** sẽ có regret tuyến tính theo thời gian với một số tiên nghiệm p_0 nào đó. Do đó, để đạt regret dưới tuyến tính, chiến lược dừng phải **ngẫu nhiên hoá** (như TS), hoặc phải không dừng (như UCB, thay đổi theo thời gian một cách tất định).

6. Giới hạn

Paper dành toàn bộ mục 8.2 để tự phê bình TS, nêu rõ **4 nhóm bài toán mà TS không phải là lựa chọn tốt**:

(a) Bài toán không cần khám phá (8.2.1): nếu nhiễu ngữ cảnh (context) đủ đa dạng (ví dụ context Gaussian liên tục trong contextual bandit tuyến tính), greedy đã tự nhiên ”khám phá” đủ nhờ tính ngẫu nhiên của context, nên đầu tư thêm vào khám phá chủ động (như TS làm) chỉ lãng phí. Tuy nhiên paper cũng chỉ rõ: nếu context là biến nhị phân hoặc có ”offset” tham số, greedy vẫn có thể bị mắc kẹt vĩnh viễn (ví dụ bài toán news recommendation ở mục 7.1 chính là trường hợp này).

(b) Bài toán không cần khai thác (8.2.2): trong các bài toán ”ranking and selection” / ”best-arm identification” (chỉ cần tìm hành động tốt nhất, không cần tối đa hoá phần thưởng dọc đường), TS kém hiệu quả vì một khi đã tự tin vào một hành động, nó khai thác (chọn lại) hành động đó liên tục, làm chậm việc làm rõ các hành động còn lại. Paper dẫn một biến thể ”pure exploration TS” (Russo, 2016) giải quyết vấn đề này.

(c) Nhạy cảm thời gian (8.2.3): với **Example 8.3 (Many-Armed Deterministic Bandit)**, khi số hành động $K \to \infty$, TS không bao giờ lặp lại hành động nào (vì luôn có hành động ”có thể tốt hơn” chưa thử), khiến nó bị một chiến lược đơn giản ”duyệt tuần tự rồi dừng ở hành động đủ tốt” vượt mặt hoàn toàn trong khoảng thời gian ngắn. Đây là vấn đề về **time preference**: TS tối ưu cho khung thời gian dài vô hạn, không tối ưu nếu hiệu năng ngắn hạn quan trọng.

(d) Không đánh giá đúng giá trị thông tin của hành động ”không tối ưu nhưng hữu ích” (8.2.4): ba ví dụ minh họa (Example 8.4 ”A Revealing Action”, Example 8.5 ”Sparse Linear Model”, Example 8.6 ”Assortment Optimization”) đều cho thấy TS chỉ chọn trong số các hành động **có thể là tối ưu**, nên bỏ lỡ các hành động không tối ưu nhưng mang lại thông tin rất lớn (ví dụ hành động 0 trong Example 8.4 tiết lộ ngay giá trị θ dù phần thưởng kỳ vọng của nó luôn thấp hơn). Với Example 8.5, TS cần $O(d/2)$ vòng kỳ vọng còn một thuật toán bisection search chỉ cần $O(\log d)$ vòng, một khác biệt theo cấp độ mũ. Paper đề xuất **information-directed sampling (IDS)** (Russo và Van Roy 2018a) như một thuật toán thay thế tối thiểu hoá trực tiếp tỷ lệ thông tin (8.7), nhưng paper cũng tự nhận IDS **đòi hỏi tính toán phức tạp hơn nhiều** và có thể không thực tế cho các ứng dụng mà TS phù hợp.

Ngoài ra, bản thân kết quả thí nghiệm cascading bandit ở mục 4 trên đây (phần §4) đã tự cho thấy TS **không phải luôn vượt trội UCB đã tinh chỉnh tốt** ở quy mô nhỏ, một điểm tự-mâu-thuẫn-với-trực-giác chung mà paper nêu công khai thay vì giấu đi.

Một hạn chế về phương pháp luận cần ghi nhận rõ (không phải lỗi của paper, nhưng là giới hạn của phạm vi): **không có thí nghiệm trên dữ liệu thực** nào trong bài; mọi con số định lượng đều từ mô phỏng tổng hợp do nhóm tác giả tự thiết kế. Ngoài ra Laplace approximation được paper xác nhận minh bạch là **không bất biến**

dưới đối biến và cho hiệu năng kém hơn Langevin MC trong thí nghiệm cụ thể của paper, không phải một kết luận tổng quát cho mọi bài toán.

7. Liên hệ với các bài khác

- **math content/10 (Hamiltonian Monte Carlo, 2017)**: cùng thuộc nhóm phương pháp MCMC để lấy mẫu từ phân phối phức tạp; HMC là một kỹ thuật MCMC tinh vi hơn Gibbs/Langevin cơ bản mà tutorial này giới thiệu ở mục 5, có thể dùng để cải thiện bước lấy mẫu hậu nghiệm xấp xỉ trong TS cho mô hình phức tạp.
- **math content/11 (SGD như suy luận Bayes xấp xỉ, 2017)**: liên hệ trực tiếp với phần Langevin Monte Carlo và stochastic-gradient Langevin MC ở mục 5.3 của tutorial này; cả hai bài cùng khai thác ý tưởng rằng nhiễu trong SGD/minibatch có thể đóng vai trò như nguồn ngẫu nhiên cho lấy mẫu hậu nghiệm xấp xỉ.
- **optimization content/32 (Practical Bayesian Optimization, 2012) và optimization content/33 (Bayesian Optimization Tutorial, 2018)**: Bayesian Optimization và Thompson Sampling đều giải quyết đánh đổi khám phá - khai thác trong không gian liên tục/rời rạc dưới mô hình Bayes; TS còn được dùng trực tiếp làm một chiến lược lựa chọn điểm thử trong Bayesian Optimization (xem "Thompson sampling" là một trong các acquisition function phổ biến, dù chi tiết này không nằm trong phạm vi hai bài opt 32/33 theo như đã đọc ở các ghi chú khác).
- **rl content/13 (RND - Random Network Distillation, 2018)**: cả hai đều là chiến lược khám phá (exploration) trong RL, nhưng theo hai triết lý khác nhau: RND dùng phần thưởng nội tại (intrinsic reward) dựa trên lỗi dự đoán của mạng nơ-ron, còn TS/PSRL (mục 7.5 của tutorial này) dùng lấy mẫu hậu nghiệm Bayes để đạt "deep exploration" có đảm bảo lý thuyết; so sánh hai hướng này là một câu hỏi mở về đánh đổi giữa tính thực dụng (RND dễ scale lên mạng lớn) và tính có đảm bảo lý thuyết (TS/PSRL).

8. Gợi ý áp dụng

Thompson sampling cân bằng khám phá và khai thác bằng cách lấy mẫu từ niềm tin về phần thưởng. Khi áp dụng cho lựa chọn tuần tự, cần xác định mô hình phần thưởng, cách cập nhật hậu nghiệm và tiêu chí đánh giá phù hợp với bài toán.

9. Thí nghiệm gợi ý

1. **Tái tạo Bernoulli bandit cơ bản**: cài thuật toán 3.2 (BernTS) cho 3-5 hành động với θ tự chọn (ví dụ $(0.9, 0.8, 0.6)$), so sánh với greedy và ϵ-greedy ($\epsilon = 0.05$) qua 500-1000 vòng, lặp 1.000 lần mô phỏng (toy, chạy CPU, không cần GPU). Đo: regret tích lũy trung bình. Kỳ vọng: TS có regret thấp hơn ϵ-greedy, và không bị "mắc kẹt" như greedy ở một tỷ lệ các lần lặp.
2. **So sánh Laplace vs Langevin Monte Carlo cho một bài toán logistic nhỏ**: tạo bài toán contextual bandit tuyến tính-logistic đồ chơi ($d = 5$, $K = 3$ hành động), cài cả Laplace approximation (mục 5.2) và Langevin MC đơn giản (mục 5.3, bước rời rạc Euler) để lấy mẫu hậu nghiệm xấp xỉ, so sánh regret. Đo: regret trung bình qua 200 vòng, 100 lần mô phỏng. Kỳ vọng: khác biệt nhỏ nếu hậu nghiệm gần Gaussian, khác biệt lớn nếu hậu nghiệm lệch (skewed).
3. **Minh hoạ deep exploration trong MDP chuỗi dài**: cài lại ví dụ Hình 7.6 (chuỗi trạng thái $\{-N, \ldots, N\}$, thưởng ở hai đầu) với $N = 10$, so sánh TS lấy mẫu mỗi episode (PSRL) với TS lấy mẫu mỗi bước. Đo: số episode cần để tìm được phần thưởng đầu tiên. Kỳ vọng: lấy mẫu mỗi episode học nhanh hơn theo cấp số mũ so với lấy mẫu mỗi bước, minh hoạ trực quan khái niệm "deep exploration".
4. **Hyperparameter bandit cho bài toán toy nén mô hình**: mô phỏng việc chọn một trong 5 "cấu hình pruning" giả định (mỗi cấu hình có một "độ chính xác kỳ vọng" ẩn, quan sát nhiễu Gaussian khi thử), áp dụng TS theo mô hình Gaussian liên hợp (tương tự Example 4.1) để tìm cấu hình tốt nhất trong ít lượt thử nhất. Đo: số lượt thử cần để tìm đúng cấu hình tốt nhất với xác suất 95%. Kỳ vọng: TS cần ít lượt thử hơn round-robin (chia đều ngân sách cho 5 cấu hình).

5. **Ensemble sampling cho active learning trên mạng nhỏ**: huấn luyện một mạng 2 lớp nhỏ (như mục 7.4, $M = 20, D = 10$) trên một hàm hồi quy tổng hợp, dùng ensemble 10-20 mô hình (bootstrap) để chọn điểm dữ liệu tiếp theo cần "nhãn" (truy vấn), so sánh với chọn ngẫu nhiên. Đo: sai số dự đoán (MSE) trên tập test theo số điểm dữ liệu đã truy vấn. Kỳ vọng: ensemble sampling giảm MSE nhanh hơn chọn ngẫu nhiên khi ngân sách truy vấn nhỏ.

10. Tóm tắt công thức / khái niệm

Khái niệm	Công thức / định nghĩa	Nguồn
Cập nhật liên hợp Beta-Bernoulli	$(\alpha_k, \beta_k) \leftarrow (\alpha_k, \beta_k) + (r_t, 1 - r_t)$	Eq. trong mục 3.1 (paper)
Cập nhật Bayes tổng quát	$p(\theta' \mid H_t) \propto p(\theta' \mid H_{t-1})\, q_{\theta'}(y_t \mid x_t)$	Eq. 4.2 (paper)
Langevin SDE	$d\phi_t = \nabla \ln g(\phi_t)\, dt + \sqrt{2}\, dB_t$	mục 5.3 (paper)
Laplace: ma trận hiệp phương sai nghịch đảo	$C = -\nabla^2 \ln g(\phi)$ tại mode	mục 5.2 (paper)
Regret tích lũy kỳ vọng	$\mathbb{E}[\text{Regret}(T)] = \mathbb{E}\big[\sum_{t=1}^{T}(\mu(x^*, \theta) - \mu(x_t, \theta))\big]$	mục 8.1.2 (paper)
Regret tiệm cận tối ưu (Bernoulli)	$\lim_{T\to\infty} \frac{\mathbb{E}[\text{Regret}(T) \mid \theta]}{\log T} = \sum_{k \neq k^*} \frac{\theta_{k^*} - \theta_k}{d_{KL}(\theta_k \| \theta_{k^*})}$	Eq. 8.1 (paper)
Regret bound tuyến tính qua UCB	$\mathbb{E}[\text{Regret}(T)] = O(d\sqrt{T} \log T)$	Eq. 8.5 (paper)
Tỷ lệ thông tin (information ratio)	$\Gamma_t = \frac{(\mathbb{E}[\mu(x^*, \theta) - \mu(x_t, \theta)])^2}{I(x^*; (x_t, y_t) \mid H_{t-1})}$	Eq. 8.7 (paper)
Bound regret qua thông tin	$\mathbb{E}[\text{Regret}(T)] \leq \sqrt{\Gamma\, H(x^*)\, T}$	Eq. 8.8 (paper)
Ví dụ tự tính: cập nhật Beta sau 1 thành công	$\text{beta}(2,2) \xrightarrow{r_t=1} \text{beta}(3,2)$, trung bình $0.5 \to 0.6$	tự tính

11. Kết luận ngắn

Thompson Sampling là một nguyên lý đơn giản (lấy mẫu từ hậu nghiệm rồi hành động như mẫu đó là sự thật) nhưng đạt hiệu quả khám phá gần tối ưu trên một phạm vi rất rộng các bài toán ra quyết định tuần tự, từ bandit cơ bản đến bài toán có cấu trúc phức tạp (shortest path, assortment, cascading, RL trong MDP). Sức mạnh của TS nằm ở khả năng tận dụng cấu trúc bài toán và tiên nghiệm một cách tự động, không cần thiết kế confidence bound riêng cho từng mô hình như UCB. Tuy nhiên, bài báo cũng chỉ ra 4 lớp bài toán mà TS không phải lựa chọn tốt nhất (không cần khám phá, không cần khai thác, nhạy cảm thời gian, cần đánh giá thông tin tinh vi hơn mức TS làm được), và đề xuất các hướng thay thế (pure-exploration TS, information-directed sampling, UCB, Gittins index) cho từng trường hợp. Về phương pháp luận, mọi kết quả trong tutorial đều dựa trên mô phỏng tổng hợp do nhóm tác giả tự thiết kế, không có benchmark trên dữ liệu thực tế công khai nào.

Nguồn gốc: https://arxiv.org/abs/1707.02038